



POLITEKNIK STATISTIKA STIS

Program Studi D-IV Statistika

BUKU MODUL PRAKTIKUM

DATA MINING DAN BIG DATA

Kode Mata Kuliah: K202505

16 Modul Praktikum ■ Berbasis R & R Markdown

Achmad Syahrul Choir

PERNYATAAN HAK CIPTA

Buku Modul Praktikum Data Mining dan Big Data (K202505)
Hak Cipta © 2025 Achmad Syahrul Choir dan Politeknik Statistika STIS

Hak cipta dilindungi undang-undang. Dilarang memperbanyak, menyalin, menggandakan, atau memindahkan sebagian maupun seluruh isi buku ini dalam bentuk dan dengan cara apa pun — baik elektronik, mekanik, fotokopi, rekaman, maupun lainnya — tanpa izin tertulis dari penulis dan Politeknik Statistika STIS.

Seluruh data yang digunakan dalam buku ini adalah data ilustratif hasil simulasi yang dirancang menyerupai pola indikator dalam publikasi resmi. Data tersebut **bukan angka resmi BPS** dan tidak boleh dikutip sebagai statistik resmi.

Buku ini disusun dengan R Markdown dan dicetak melalui XeLaTeX. Seluruh berkas sumber (`.Rmd`), data (`.csv`), dan skrip pembangkit data (`gen_data.R`) disertakan dalam paket sumber sehingga tiap angka dan gambar dalam buku ini dapat dihasilkan ulang.

Penyusun	:	Achmad Syahrul Choir
Mata Kuliah	:	Data Mining dan Big Data (K202505)
Program Studi	:	D-IV Statistika
Jumlah Modul	:	16 modul praktikum
Perangkat Lunak	:	R (≥ 4.2) dan RStudio; Python (≥ 3.9) untuk Modul 4, 13, 15, dan 16; disusun dengan R Markdown dan dicetak melalui XeLaTeX

Naskah tiap modul **dibangkitkan** dari berkas spesifikasi `.yaml` memakai paket `Rmodulstis 0.3.9`, sehingga penomoran contoh, kode, halaman, dan tata letaknya seragam di seluruh buku. Daftar paket R — dan modul Python bagi modul yang memakainya — dicantumkan pada bagian *Alat dan Bahan* tiap modul.

Kata Pengantar

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas tersusunnya Buku Modul Praktikum Data Mining dan Big Data (K202505) ini. Buku disusun sebagai bahan praktikum bagi mahasiswa Program Studi D-IV Statistika, Politeknik Statistika STIS.

Buku ini memuat **16 modul praktikum** yang mengikuti alur Rencana Pembelajaran Semester. Empat modul pertama membangun landasan: pengenalan *knowledge discovery*, konsep big data, ragam tipe data, dan pengumpulan data termasuk perayapan web beserta etikanya. Modul 5 sampai 7 menyiapkan pekerjaan analisis: prapemrosesan dengan kerangka resep yang dapat diulang, kerangka *supervised learning* beserta validasi silang, dan penyusunan proposal proyek menurut CRISP-DM. Modul 8 sampai 11 masuk ke pemodelan: klasifikasi dasar, klasifikasi lanjut, jaringan syaraf tiruan, dan pengelompokan berbasis kepadatan beserta evaluasinya.

Tiap modul memiliki struktur yang konsisten: deskripsi, tujuan, alat dan bahan, instruksi pengerjaan, teori singkat, contoh praktikum langkah demi langkah, latihan mandiri, dan penugasan berbasis kasus. Seluruh analisis memakai **R** dengan kerangka `tidymodels`, disusun sebagai dokumen R Markdown sehingga hasilnya dapat direproduksi.

Dua hal ditekankan sepanjang buku. **Pertama**, pencegahan *kebocoran data*: seluruh prapemrosesan dilakukan di dalam resep yang dilatih hanya pada data latih. **Kedua**, penilaian yang jujur: tiap model dibandingkan dengan acuan sederhana, dan ukuran penilaian dipilih sesuai biaya kekeliruan — bukan sekadar akurasi.

Kritik dan saran perbaikan sangat kami harapkan.

Jakarta, 2025

Achmad Syahrul Choir
Koordinator Mata Kuliah

Petunjuk Penggunaan Modul

Bagi Mahasiswa

Bacalah Deskripsi Modul, Tujuan Praktikum, dan Teori Singkat **sebelum** sesi berlangsung. Pastikan perangkat lunak dan paket pada Alat dan Bahan sudah terpasang, serta berkas data tersedia pada subfolder `data/`.

Selama praktikum, kerjakan Contoh Praktikum secara berurutan. **Ketik ulang kodenya** — jangan disalin-tempel — karena mengetik membuat Anda memperhatikan detail sintaks. Sebelum membaca interpretasi yang disediakan, tuliskan lebih dahulu tafsiran Anda sendiri, lalu bandingkan. Biasakan mengubah satu argumen lalu mengamati akibatnya.

Kerangka tidymodels

Seluruh pemodelan memakai kerangka `tidymodels`: `recipes` untuk prapemrosesan, `rsample` untuk pembagian data dan validasi silang, `parsnip` untuk spesifikasi model, `workflows` untuk merangkainya, `tune` untuk penalaan, dan `yardstick` untuk penilaian. Kerangka ini dipilih karena memaksa prapemrosesan dilakukan **di dalam** alur kerja — sehingga kebocoran data dari data uji ke data latih dapat dicegah secara struktural, bukan mengandalkan kedisiplinan penulis kode.

Bagi Dosen dan Asisten

Alokasi waktu tiap modul adalah 1×170 menit. Contoh Praktikum dirancang untuk dikerjakan bersama di kelas, Latihan Mandiri sebagai responsi, serta Penugasan sebagai tugas rumah. Penugasan dirancang sebagai kasus yang menuntut keputusan, bukan sekadar menjalankan perintah.

Kunci jawaban disediakan dalam buku terpisah dan memuat catatan penskoran untuk tiap modul.

Catatan tentang Data

Seluruh data bersifat ilustratif hasil simulasi. Berkas `gen_data.R` pada tiap modul memuat kode pembangkitnya lengkap dengan `set.seed()`, sehingga data dapat dibangkitkan ulang persis sama. Dosen yang ingin memakai data nyata dapat mengganti isi folder `data/` dengan mempertahankan nama kolom.

Simbol dan Kotak yang Digunakan

Modul memakai beberapa kotak berwarna dengan peran berbeda.

Kotak	Peran
 Interpretasi Output	membahas arti keluaran: apa yang dapat disimpulkan dari angka atau gambar
 Perhatian	peringatan yang wajib diperhatikan, termasuk penegasan bahwa data adalah simulasi
 Info	keterangan pelengkap, identitas modul, dan tabel acuan
 Rumus	rumus atau kerangka perintah yang menjadi acuan
 Tujuan	alur praktikum dan daftar Periksa penguasaan
 Penugasan	konteks kasus yang menjadi dasar penugasan
 Catatan	keterangan tambahan dan kaitan antarmodul
 Kunci & Pembahasan	(pada buku kunci) penyelesaian beserta alasannya

Penomoran

Contoh, Kode, Latihan, dan Tugas dinomori mengikuti nomor modul. Contoh 9.3 berarti contoh ketiga pada Modul 9. Penomoran halaman berlanjut dari Modul 1 hingga Modul 13.

Perangkat Lunak

Seluruh modul memakai **R** (≥ 4.2) dan **RStudio** dengan kerangka `tidymodels`. Paket yang diperlukan disebutkan pada bagian Alat dan Bahan tiap modul. Dokumen disusun dengan R Markdown dan dicetak melalui XeLaTeX.

Penilaian Praktikum

Bobot berikut merupakan usulan yang dapat disesuaikan dosen pengampu.

Komponen	Bobot	Dasar penilaian
Keaktifan dan pengerjaan Contoh	15%	kehadiran, ketuntasan pengerjaan di kelas
Latihan Mandiri	25%	ketepatan kode dan mutu interpretasi
Penugasan	40%	kelengkapan analisis, mutu laporan, kejujuran pelaporan
Ujian Praktik	20%	penyelesaian kasus secara mandiri

Rubrik Laporan Penugasan

Komponen	Bobot	Indikator
Ketepatan analisis	30%	kebenaran perhitungan, asumsi diperiksa, metode sesuai pertanyaan
Kelengkapan hasil	25%	seluruh tahap dikerjakan, keluaran ditampilkan rapi
Ketepatan interpretasi	25%	besaran yang tepat ditafsirkan, keterbatasan dinyatakan
Kualitas penyajian	15%	gambar informatif dan berlabel, penulisan sistematis
Ketepatan waktu	5%	dikumpulkan sesuai tenggat

Catatan Penilaian

Laporan yang menyajikan angka tanpa interpretasi, atau interpretasi yang melampaui yang didukung data (misalnya klaim sebab-akibat dari analisis deskriptif), dikenai pengurangan nilai meskipun kodenya benar. Sebaliknya, mahasiswa yang menemukan keterbatasan pada hasilnya sendiri — lalu menjelaskannya — memperoleh nilai penuh pada komponen interpretasi.

Daftar Isi

Bagian Inti (Modul 1–12, sesuai RPS)

Modul	Judul Modul	Hlm
1	Pengenalan Knowledge Discovery dan Data Mining	2
2	Konsep Dasar Big Data	34
3	Tipe Data pada Big Data	65
4	Pengumpulan Data dan Perayapan Web	97
5	Prapemrosesan Data	142
6	Konsep Supervised Learning	173
7	Persiapan Proyek Data Mining	205
8	Klasifikasi Dasar	236
9	Klasifikasi Lanjut: SVM, KNN, Random Forest	270
10	Jaringan Syaraf Tiruan: FFNN dan RBFNN	302
11	Association Rule dan Frequent Itemset Mining	337
12	Pengelompokan: k-Medoid, k-Prototype, DBSCAN, OPTICS, Bi-clustering	368

Bagian Pengayaan (Modul 13–16)

Modul	Judul Modul	Hlm
13	GPBoost: Boosting untuk Data Bertingkat (R dan Python)	421
14	Alur Kerja Pengelompokan, CLARA/CLARANS, Kajian Terkendali	444
15	LightGBM: Gradient Boosting Cepat (R dan Python)	473
16	LSTM: Jaringan Berulang untuk Deret Waktu (R dan Python)	495

Penomoran halaman berlanjut dari Modul 1 hingga Modul 16; tiap modul diawali satu halaman sampul pembatas yang tidak bernomor, dan tiap bagian diawali satu halaman pembatas bagian.

BAGIAN INTI



Modul 1–12

Sesuai Rencana Pembelajaran Semester

MODUL

1

PENGENALAN KNOWLEDGE DISCOVERY DAN DATA MINING

Pertemuan Minggu ke-1

Sub-CPMK 1: Mahasiswa mampu menjelaskan konsep data mining dan knowledge discovery, membedakannya dari statistika inferensia dan pembelajaran mesin, meng-
enali jenis data yang dapat ditambang, serta menempatkan pekerjaan analisis pada
kerangka CRISP-DM.

Penyusun: Achmad Syahrul Choir



Daftar Isi

1.1	Deskripsi Modul	2
1.2	Tujuan Praktikum	2
1.3	Alat dan Bahan	3
1.4	Instruksi Pengerjaan	3
1.5	Teori Singkat	3
1.5.0.1	Kedudukan Modul Ini pada CRISP-DM	3
1.5.0.2	Apa Itu Data Mining	4
1.5.0.3	Membedakan dari Disiplin Serumpun	4
1.5.0.4	Jenis Data yang Dapat Ditambang	5
1.5.0.5	Tujuh Langkah Proses KDD	5
1.5.0.6	Ragam Tugas Penambangan	6
1.5.0.7	Skala Pengukuran Menentukan Perlakuan	6
1.5.0.8	Supervised dan Unsupervised	7
1.5.0.9	Bahaya Menemukan Pola Semu	7
1.5.0.10	Etika dan Privasi	7
1.6	Contoh Praktikum dengan R	8
1.7	Latihan Mandiri	26
1.8	Penugasan	29
1.9	Ringkasan Modul	30
1.10	Referensi	31

1.1 Deskripsi Modul

Modul ini membuka mata kuliah Data Mining dan Big Data. Sasarannya bukan langsung membangun model, melainkan menata cara berpikir: apa sebenarnya yang dicari data mining, bagaimana ia berbeda dari statistika inferensia yang sudah Anda pelajari, dan pada kerangka kerja apa seluruh kegiatan ini ditempatkan.

Anda datang ke mata kuliah ini membawa bekal Analisis Peubah Ganda dan Analisis Regresi. Keduanya mengajarkan cara menarik kesimpulan mengenai populasi dari sebuah contoh, dengan model yang ditetapkan lebih dahulu berdasarkan teori. Data mining bergerak dari arah yang berlawanan: pola dicari dari data, dan model dipilih berdasarkan seberapa baik ia meramalkan, bukan seberapa cocok ia dengan teori yang mendahului.

Perbedaan arah itu membawa akibat yang nyata pada cara bekerja. Ukuran keberhasilan bergeser dari kebermanaknaan menjadi ketepatan ramalan; perhatian bergeser dari penduga parameter menjadi pemodelan berlebih (*overfitting*); dan pembagian data menjadi latih dan uji berubah dari pilihan menjadi keharusan.

Materi dibahas dari dasar hingga penerapan: pengertian dan tujuan data mining, jenis data yang dapat ditambang, perbedaan dengan disiplin serumpun, alur Knowledge Discovery in Databases, kerangka CRISP-DM sebagai tulang punggung seluruh modul, serta etika dan privasi data.

Tentang data. Berkas data pada modul ini adalah **data simulasi** (`set.seed(2026)`), **bukan data nyata** dan **bukan data resmi** instansi mana pun. Data dibangkitkan agar konteks praktikum terasa nyata, tetapi angkanya tidak menggambarkan keadaan sebenarnya.

1.2 Tujuan Praktikum

Setelah menyelesaikan praktikum ini secara tuntas, mahasiswa diharapkan mampu:

1. **Menjelaskan** pengertian data mining dan knowledge discovery beserta tujuannya.
2. **Membedakan** data mining dari statistika inferensia dan pembelajaran mesin.
3. **Mengenali** jenis data yang dapat ditambang beserta perlakuan yang sesuai.
4. **Menguraikan** enam tahap CRISP-DM dan menempatkan suatu pekerjaan padanya.
5. **Melakukan** penjelajahan awal data sebagai langkah *data understanding*.
6. **Membandingkan** rumusan pertanyaan inferensia dengan pertanyaan data mining atas data yang sama.
7. **Menjelaskan** pertimbangan etika dan privasi dalam penambangan data.

1.3 Alat dan Bahan

Perangkat dan bahan yang diperlukan:

- **R** (RStudio) sebagai lingkungan kerja utama.
- Paket: `dplyr` dan `data.table` (pasang dengan `install.packages()`).
- Berkas data: `data/data_pelanggan.csv`.
- Peramban (*browser*) untuk membuka dokumentasi paket bila diperlukan.

1.4 Instruksi Pengerjaan

Ikuti tahapan berikut:

Sebelum praktikum: siapkan direktori kerja; letakkan `data_pelanggan.csv` pada subfolder `data/`; baca bagian Teori Singkat, khususnya kerangka CRISP-DM.

Selama praktikum: kerjakan Contoh secara berurutan. Untuk setiap contoh, ketik ulang kodenya, amati ke- luarannya, baca Penjelasan Kode, lalu tuliskan tafsiran Anda sendiri sebelum membaca Interpretasi Output.

Pembagian kerja: Contoh dan Latihan dikerjakan selama sesi di kelas; **Penugasan dikerjakan di luar sesi** sebagai tugas terstruktur.

Pengumpulan: susun laporan R Markdown, render ke PDF atau HTML, kumpulkan melalui Google Classroom sesuai tenggat.

1.5 Teori Singkat

1.5.0.1 Kedudukan Modul Ini pada CRISP-DM

Seluruh modul dalam mata kuliah ini ditempatkan pada kerangka CRISP-DM (Cross-Industry Standard Process for Data Mining). Modul ini berada pada dua tahap awal: Business Understanding, yaitu merumuskan persoalan dan ukuran keberhasilannya; dan Data Understanding, yaitu mengenali data yang tersedia sebelum menyentuh pemodelan.

Enam tahap CRISP-DM.

1	Business Understanding	merumuskan persoalan, tujuan, dan kriteria sukses
2	Data Understanding	mengumpulkan, menjelajah, dan menilai mutu data
3	Data Preparation	membersihkan, menggabung, mereduksi, menransformasi
4	Modeling	memilih teknik, melatih model, menala hiperparameter
5	Evaluation	menilai hasil terhadap tujuan pada tahap 1
6	Deployment	menerapkan, melaporkan, memantau

Perlu ditegaskan bahwa CRISP-DM bukan urutan yang berjalan sekali jalan. Panahnya dua arah: temuan pada tahap penjelajahan kerap memaksa rumusan persoalan ditinjau ulang, dan hasil evaluasi kerap mengembalikan pekerjaan ke tahap penyiapan data. Justru pengulangan itulah yang lazim dalam praktik.

1.5.0.2 Apa Itu Data Mining

Data mining adalah proses menemukan pola yang valid, baru, bermanfaat, dan dapat dipahami dari data berukuran besar. Keempat sifat itu perlu dibaca bersama-sama. Pola yang valid tetapi tidak baru hanya mengulang yang sudah diketahui; pola yang baru tetapi tidak bermanfaat tidak menolong pengambilan keputusan; dan pola yang bermanfaat tetapi tidak dapat dipahami sukar dipertanggungjawabkan.

Istilah data mining sering dipertukarkan dengan Knowledge Discovery in Databases (KDD). Secara ketat, KDD adalah keseluruhan prosesnya, sedangkan data mining adalah satu tahap di dalamnya, yaitu penerapan algoritma penemuan pola. Dalam pemakaian sehari-hari keduanya kerap disamakan, dan modul ini pun tidak mempersoalkannya selama konteksnya jelas.

1.5.0.3 Membedakan dari Disiplin Serumpun

Tiga cara pandang atas data yang sama.

	Statistika inferensia	Data mining
Titik mula	teori dan hipotesis	data
Model	ditetapkan lebih dahulu	dipilih dari perbandingan
Ukuran sukses	kebermaknaan, selang	ketepatan ramalan pada data baru
Ukuran contoh	lazimnya terbatas	lazimnya besar
Perhatian utama	bias penduga	pemodelan berlebih (<i>overfitting</i>)
Peubah	sedikit, terpilih	banyak, kadang ribuan

Pembelajaran mesin sendiri lebih tepat dipandang sebagai kumpulan algoritma yang dipakai data mining, bukan sebagai pesaingnya. Sebuah pohon keputusan adalah algoritma pembelajaran mesin; memakainya untuk menemukan ciri pelanggan yang berpotensi berhenti berlangganan adalah pekerjaan data mining.

Data mining bukan sekadar mengumpulkan data. Banyak orang mengira data mining berarti mengumpulkan data sebanyak-banyaknya lalu berharap pola muncul dengan sendirinya. Anggapan ini keliru dan berbahaya. Tanpa rumusan persoalan yang jelas, pencarian pola pada data besar hampir pasti menemukan hubungan semu — makin banyak peubah diperiksa, makin besar peluang menemukan kebetulan yang tampak bermakna. Itulah sebabnya CRISP-DM menempatkan *business understanding* pada tahap pertama, bukan terakhir.

1.5.0.4 Jenis Data yang Dapat Ditambang

Ragam data dan perlakuannya.

Jenis	Contoh	Perlakuan khas
Tabular	basis data pelanggan	pembersihan, pengodean, pembakuan
Transaksi	keranjang belanja	format keranjang, aturan asosiasi
Runtun waktu	penjualan harian	pembedaan, jendela geser
Teks	ulasan, media sosial	tokenisasi, pembobotan istilah
Jaringan	relasi antar-akun	ukuran sentralitas, komunitas
Spasial	titik dan wilayah	bobot ketetanggaan
Citra	foto satelit	ekstraksi ciri, jaringan konvolusi

Modul ini dan sebagian besar modul berikutnya bekerja pada data tabular, karena di sanalah gagasan pokok paling mudah dipahami. Data teks dan jaringan dibahas pada Modul 3, sedangkan data transaksi menjadi bahan Modul 11.

1.5.0.5 Tujuh Langkah Proses KDD

CRISP-DM memberi kerangka manajerial, sedangkan proses KDD yang dirumuskan Fayyad dan rekan memberi rincian teknisnya. Keduanya tidak bertentangan melainkan saling melengkapi; KDD dapat dipandang sebagai perincian tahap ketiga sampai kelima CRISP-DM.

Tujuh langkah KDD.

1	Seleksi	memilih data dan peubah yang relevan bagi persoalan
2	Prapemrosesan	membersihkan derau, menangani nilai hilang
3	Transformasi	mengubah bentuk, mereduksi dimensi
4	Penambangan	menerapkan algoritma penemuan pola
5	Penafsiran	menilai pola yang ditemukan
6	Penyajian	menyampaikan temuan agar dapat dipahami
7	Pemanfaatan	menerapkan pengetahuan pada keputusan

Perhatikan bahwa penambangan sesungguhnya hanya menempati satu dari tujuh langkah. Enam langkah lainnya adalah penyiapan dan penindaklanjutan. Kesan bahwa data mining identik dengan menjalankan algoritma karena itu keliru, dan kekeliruan itulah yang membuat banyak proyek gagal pada tahap penerapan.

1.5.0.6 Ragam Tugas Penambangan

Enam tugas pokok.

Tugas	Pertanyaan	Contoh penerapan
Klasifikasi	termasuk kelas mana?	pelanggan berpotensi berhenti
Regresi	berapa nilainya?	menduga pengeluaran
Pengelompokan	ada berapa kelompok alami?	segmentasi wilayah
Aturan asosiasi	apa yang muncul bersama?	pola keranjang belanja
Deteksi anomali	mana yang menyimpang?	transaksi mencurigakan
Peringkasan	bagaimana gambaran umumnya?	profil wilayah

Menetapkan tugas dengan tepat adalah keputusan awal yang menentukan. Persoalan yang sama dapat dirumuskan sebagai tugas berbeda: menduga besarnya tagihan adalah regresi, sedangkan menduga apakah tagihan tergolong tinggi adalah klasifikasi. Pilihan itu mengubah algoritma, ukuran keberhasilan, dan cara menafsirkan hasilnya.

1.5.0.7 Skala Pengukuran Menentukan Perlakuan

Sebelum memilih algoritma, tipe peubah perlu dikenali menurut skala pengukurannya. Kekeliruan pada tahap ini berakibat perhitungan yang secara teknis berjalan tetapi maknanya keliru — misalnya menghitung rata-rata atas kode wilayah yang sesungguhnya nominal.

Empat skala pengukuran.

Skala	Ciri	Contoh	Yang sah dihitung
Nominal	kategori tanpa urutan	wilayah, jenis kelamin	modus, frekuensi
Ordinal	kategori berurutan	tingkat pendidikan	modus, median
Interval	jarak bermakna, nol relatif	suhu Celsius	rata-rata, simpangan
Rasio	nol mutlak	pendapatan, umur	seluruhnya, termasuk rasio

Perbedaan interval dan rasio kerap dianggap sepele, padahal berpengaruh. Suhu 20 derajat Celsius bukan dua kali lebih panas daripada 10 derajat, sebab titik nolnya tidak mutlak. Sebaliknya pendapatan dua juta memang dua kali pendapatan satu juta. Rasio hanya bermakna pada skala rasio.

1.5.0.8 Supervised dan Unsupervised

Dua kelompok besar tugas penambangan.

	Supervised	Unsupervised
Sasaran	ada, diketahui	tidak ada
Tujuan	meramalkan sasaran	menemukan struktur
Contoh tugas	klasifikasi, regresi	pengelompokan, aturan asosiasi
Penilaian	terhadap sasaran sebenarnya	tidak langsung, perlu penafsiran
Modul	6 sampai 10, 13	11, 12, 14

Perbedaan penilaian pada baris keempat patut dicermati. Pada tugas supervised, kebenaran dapat diperiksa karena sasarnya diketahui. Pada tugas unsupervised tidak ada kunci jawaban, sehingga kebermaknaan hasil bersandar pada penafsiran dan pengetahuan bidang. Inilah sebabnya pengelompokan lebih sukar dinilai daripada klasifikasi, meski secara perhitungan tampak lebih sederhana.

1.5.0.9 Bahaya Menemukan Pola Semu

Telah disinggung bahwa mencari pola pada data berpeubah banyak berisiko menemukan kebetulan. Besarnya risiko itu dapat dihitung. Bila seratus peubah diuji hubungannya dengan suatu sasaran pada taraf nyata lima persen, maka sekitar lima peubah akan tampak nyata **meski tidak ada hubungan sama sekali**.

Peluang menemukan setidaknya satu hasil nyata palsu dari m pengujian bebas:

$$P(\text{setidaknya satu}) = 1 - (1 - \alpha)^m$$

Pada $\alpha = 0,05$ dan $m = 100$, peluangnya mencapai 0,994.

1.5.0.10 Etika dan Privasi

Kemampuan menemukan pola membawa tanggung jawab. Data yang tampak tidak sensitif dapat menjadi sensitif ketika digabungkan: gabungan kode pos, tanggal lahir, dan jenis kelamin sudah cukup untuk mengenali sebagian besar orang secara unik. Karena itu penggabungan sumber data perlu ditimbang, bukan sekadar dikerjakan karena mungkin dikerjakan.

Tiga pertanyaan sebelum menambang data.

1. Apakah data ini diperoleh dengan sah dan dengan sepengetahuan pemiliknya?
2. Apakah hasilnya dapat merugikan kelompok tertentu, meski tidak disengaja?
3. Apakah individu masih dapat dikenali setelah data digabungkan?

1.6 Contoh Praktikum dengan R

Contoh 1.1 | Mengenal struktur data sebagai langkah pertama

Kode 1.1 • R dim dan str

```
pel <- read.csv("data/data_pelanggan.csv", stringsAsFactors = TRUE)
dim(pel)
```

```
[1] 500  7
```

```
str(pel)
```

```
'data.frame':  500 obs. of  7 variables:
 $ id_pelanggan   : int  10350 10279 10071 10144 10278 10014 10234 10431 10127 10291 ...
 $ wilayah        : Factor w/ 2 levels "Perdesaan","Perkotaan": 2 1 1 2 2 2 2 1 1 1 ...
 $ segmen         : Factor w/ 3 levels "Aktif","Baru",...: 2 3 3 1 1 2 2 1 1 1 ...
 $ lama_langganan : int   49 38 39 6 46 19 34 11 32 39 ...
 $ tagihan_bulanan: num   819 250 703 485 777 ...
 $ jumlah_transaksi: int   21 93 36 87 48 3 100 22 51 111 ...
 $ skor_kepuasan  : num   1.2 4.8 2.3 2.5 1.6 4 3.1 4.6 3.6 3.2 ...
```

Penjelasan Kode. Data dibaca dengan argumen `stringsAsFactors` bernilai `TRUE` agar kolom teks langsung menjadi faktor, sesuai kebutuhan sebagian besar algoritma penambangan. Fungsi `dim` menampilkan banyaknya baris dan kolom, sedangkan `str` memberikan tipe tiap kolom beserta beberapa nilai pertamanya.

Interpretasi Output

Ini langkah pertama tahap Data Understanding pada CRISP-DM. Sebelum menghitung apa pun, pastikan lebih dahulu berapa banyak amatan yang tersedia dan bertipe apa saja peubahnya. Perhatikan bahwa tipe kolom menentukan perlakuan berikutnya: kolom faktor perlu dikodekan bagi sebagian model, sedangkan kolom numerik perlu dibakukan bagi model berbasis jarak.

Contoh 1.2 | Ringkasan menyeluruh dan pemeriksaan kelengkapan

Kode 1.2 • R summary dan colSums(is.na())

```
summary(pel)
```

```
id_pelanggan   wilayah      segmen   lama_langganan tagihan_bulanan jumlah_transaksi
Min.   :10002   Perdesaan:233   Aktif:252   Min.   : 1.0   Min.   : 50.3   Min.   :  0.0
```

```

1st Qu.:10121   Perkotaan:267   Baru :114   1st Qu.:18.0   1st Qu.:280.2   1st Qu.: 25.0
Median :10250           Pasif:134   Median :36.0   Median :476.5   Median : 57.0
Mean :10246           Mean :36.4   Mean :463.4   Mean : 57.5
3rd Qu.:10364           3rd Qu.:54.2   3rd Qu.:655.7   3rd Qu.: 87.0
Max. :10498           Max. :72.0   Max. :848.2   Max. :120.0

skor_kepuasan
Min. :1.00
1st Qu.:2.10
Median :3.10
Mean :3.06
3rd Qu.:4.00
Max. :5.00

```

```
colSums(is.na(pel))
```

```

id_pelanggan      wilayah      segmen   lama_langganan   tagihan_bulanan
0                  0          0             0              0
jumlah_transaksi  skor_kepuasan
0                  0

```

Penjelasan Kode. Fungsi summary memberi ringkasan yang berbeda menurut tipe kolom: lima serangkai bagi kolom numerik dan cacah tiap taraf bagi kolom faktor. Perintah colSums yang dikenakan pada is.na menghitung banyaknya nilai hilang per kolom, sebab is.na menghasilkan matriks logika yang penjumlahan kolomnya adalah cacah nilai hilang.

Interpretasi Output

Dua hal perlu diperiksa dari keluaran ini. Pertama, apakah rentang tiap peubah masuk akal — skor kepuasan di luar rentang 1 sampai 5 menandakan kekeliruan pencatatan. Kedua, apakah ada nilai hilang; pada data ini tidak ada, tetapi pada data nyata pemeriksaan ini wajib dilakukan sebelum melangkah.

Contoh 1.3 | Membaca sebaran peubah kategori

Kode 1.3 • R table dan prop.table

```
table(pel$wilayah)
```

```

Perdesaan Perkotaan
233      267

```

```
table(pel$segmen)
```

```

Aktif Baru Pasif
252   114   134

```

```
round(prop.table(table(pel$segmen, pel$wilayah), margin = 2), 3)
```

	Perdesaan	Perkotaan
Aktif	0.528	0.483
Baru	0.176	0.273
Pasif	0.296	0.243

Penjelasan Kode. Fungsi `table` mencacah frekuensi tiap taraf. Ketika diberi dua peubah, ia menghasilkan tabel silang. Fungsi `prop.table` mengubahnya menjadi proporsi; argumen `margin` bernilai 2 membuat proporsi dihitung per kolom, sehingga tiap kolom berjumlah satu.

Interpretasi Output

Tabel silang berproporsi kolom menjawab pertanyaan apakah komposisi segmen berbeda antara perkotaan dan perdesaan. Bila proporsinya serupa, kedua peubah cenderung saling bebas. Pemeriksaan semacam ini berguna sejak awal karena peubah yang saling bebas memberi informasi yang saling melengkapi bagi model.

Contoh 1.4 | Menghitung ringkasan per kelompok

Kode 1.4 • R `group_by` dan `summarise`

```
suppressPackageStartupMessages(library(dplyr))
pel |>
  group_by(segmen) |>
  summarise(n = n(),
            rata_tagihan = mean(tagihan_bulanan),
            median_transaksi = median(jumlah_transaksi),
            rata_kepuasan = mean(skor_kepuasan),
            .groups = "drop") |>
  arrange(desc(rata_tagihan))
```

```
# A tibble: 3 x 5
  segmen      n rata_tagihan median_transaksi rata_kepuasan
  <fct> <int>      <dbl>          <dbl>          <dbl>
1 Aktif   252      468.             54             3.09
2 Baru    114      467.            61.5            3.02
3 Pasif   134      451.             61             3.05
```

Penjelasan Kode. Data dikelompokkan menurut `segmen`, lalu diringkas dengan beberapa besaran sekaligus. Argumen `.groups` bernilai `drop` melepaskan pengelompokan setelah peringkasan agar tidak terbawa ke langkah berikutnya. Hasilnya diurutkan menurun berdasarkan rata-rata tagihan.

Interpretasi Output

Perbandingan antar-segmen semacam ini adalah bahan mentah bagi perumusan persoalan. Bila segmen Pasif ternyata bertagihan tinggi tetapi berkepuasan rendah, muncul pertanyaan bisnis yang layak ditambang lebih lanjut. Perhatikan bahwa median dipakai bagi jumlah transaksi karena peubah cacahan kerap miring (*skewed*).

Contoh 1.5 | Mengukur kendala volume dengan data.table**Kode 1.5 • R** system.time membandingkan dua pendekatan

```
suppressPackageStartupMessages(library(data.table))
set.seed(2026)
besar <- data.frame(g = sample(letters[1:5], 2e5, TRUE),
                    x = runif(2e5))
besar_dt <- as.data.table(besar)

waktu_df <- system.time(aggregate(x ~ g, data = besar, FUN = mean))["elapsed"]
waktu_dt <- system.time(besar_dt[, list(rata = mean(x)), by = g])["elapsed"]
round(c(data.frame = waktu_df, data.table = waktu_dt), 4)
```

```
data.frame data.table
0.284      0.012
```

Penjelasan Kode. Data buatan berukuran 200 ribu baris dibentuk, lalu disalin menjadi objek data.table. Peringkasan per kelompok dikerjakan dua kali dengan cara berbeda, dan lamanya diukur memakai system.time; komponen elapsed menyatakan waktu sesungguhnya yang berlalu. Penulisan list dipakai alih-alih singkatan titik agar kode tetap berjalan ketika dipanggil dari dalam paket lain.

Interpretasi Output

Selisih waktunya memperlihatkan bahwa pilihan perkakas berpengaruh nyata begitu data membesar. Inilah gerbang menuju pembahasan big data pada Modul 2: persoalannya bukan semata algoritma apa yang dipakai, melainkan apakah data masih muat di memori dan berapa lama pengolahannya. Angka pastinya bergantung pada mesin masing-masing.

Contoh 1.6 | Merumuskan dua jenis pertanyaan atas data yang sama**Kode 1.6 • R** t.test versus tabel klasifikasi

```
# Pertanyaan INFERENSIA: apakah rata-rata tagihan berbeda antar-wilayah?
```

```
uji <- t.test(tagihan_bulanan ~ wilayah, data = pel)
```

```
round(c(selisih = diff(rev(uji$estimate)),  
      p_value = uji$p.value), 4)
```

selisih.mean in group	Perdesaan	p_value
	-10.2522	0.6188

```
# Pertanyaan DATA MINING: seberapa baik wilayah meramalkan tagihan tinggi?
```

```
pel$tagihan_tinggi <- factor(ifelse(pel$tagihan_bulanan > median(pel$tagihan_bulanan),  
                                  "tinggi", "rendah"))
```

```
tabel <- table(pel$wilayah, pel$tagihan_tinggi)
```

```
round(prop.table(tabel, margin = 1), 3)
```

	rendah	tinggi
Perdesaan	0.498	0.502
Perkotaan	0.502	0.498

Penjelasan Kode. Bagian pertama menjalankan uji t dua contoh bebas untuk membandingkan rata-rata tagihan antar-wilayah, lalu mengambil selisih penduga dan nilai-p. Bagian kedua membentuk peubah biner tagihan tinggi berdasarkan median, lalu memeriksa proporsi tagihan tinggi pada tiap wilayah dengan margin bernilai 1 agar proporsi dihitung per baris.

Interpretasi Output

Kedua bagian memakai data yang sama tetapi menanyakan hal berbeda. Pertanyaan inferensia berhenti pada apakah perbedaannya nyata secara statistik. Pertanyaan data mining melangkah lebih jauh: seberapa berguna wilayah sebagai penduga. Perbedaan yang nyata secara statistik belum tentu berguna untuk meramal, dan sebaliknya peubah yang tidak nyata kadang tetap menyumbang ketika digabungkan dengan peubah lain.

Tips dan Praktik Baik. Pembedaan pada contoh terakhir adalah inti mata kuliah ini. Setiap kali Anda menghadapi data, biasakan bertanya lebih dahulu: apakah saya sedang menguji dugaan, atau sedang mencari pola untuk meramal? Jawabannya menentukan seluruh langkah berikutnya.

Contoh 1.7 | Mengenali skala pengukuran tiap peubah

Kode 1.7 · R menduga skala dari tipe dan keunikan

```
skala <- data.frame(  
  peubah = names(pel),  
  tipe_r = sapply(pel, function(x) class(x)[1]),
```

```

nilai_unik = sapply(pel, function(x) length(unique(x))),
row.names = NULL)
skala$dugaan_skala <- ifelse(
  skala$tipe_r == "factor", "nominal/ordinal",
  ifelse(skala$nilai_unik < 10, "perlu diperiksa", "interval/rasio"))
print(skala, right = FALSE)

```

peubah	tipe_r	nilai_unik	dugaan_skala
1 id_pelanggan	integer	320	interval/rasio
2 wilayah	factor	2	nominal/ordinal
3 segmen	factor	3	nominal/ordinal
4 lama_langganan	integer	72	interval/rasio
5 tagihan_bulanan	numeric	499	interval/rasio
6 jumlah_transaksi	integer	118	interval/rasio
7 skor_kepuasan	numeric	41	interval/rasio
8 tagihan_tinggi	factor	2	nominal/ordinal

Penjelasan Kode. Tabel disusun memuat nama peubah, kelas R-nya, dan banyaknya nilai unik. Dugaan skala ditetapkan dengan ifelse bersarang: kolom faktor tergolong kategori, sedangkan kolom numerik bernilai unik sedikit ditandai untuk diperiksa sebab boleh jadi sesungguhnya merupakan kode kategori.

Interpretasi Output

Dugaan otomatis semacam ini menolong tetapi tidak boleh dipercaya bulat-bulat. Kolom id_pelanggan bertipe bilangan bulat sehingga terbaca sebagai rasio, padahal ia penanda yang tak bermakna untuk dirata-ratakan. Sebaliknya skor kepuasan berskala satu sampai lima sesungguhnya ordinal meski tersimpan sebagai angka. Keputusan akhir tetap menuntut pengetahuan bidang.

Contoh 1.8 | Menggambarkan sebaran dan hubungan

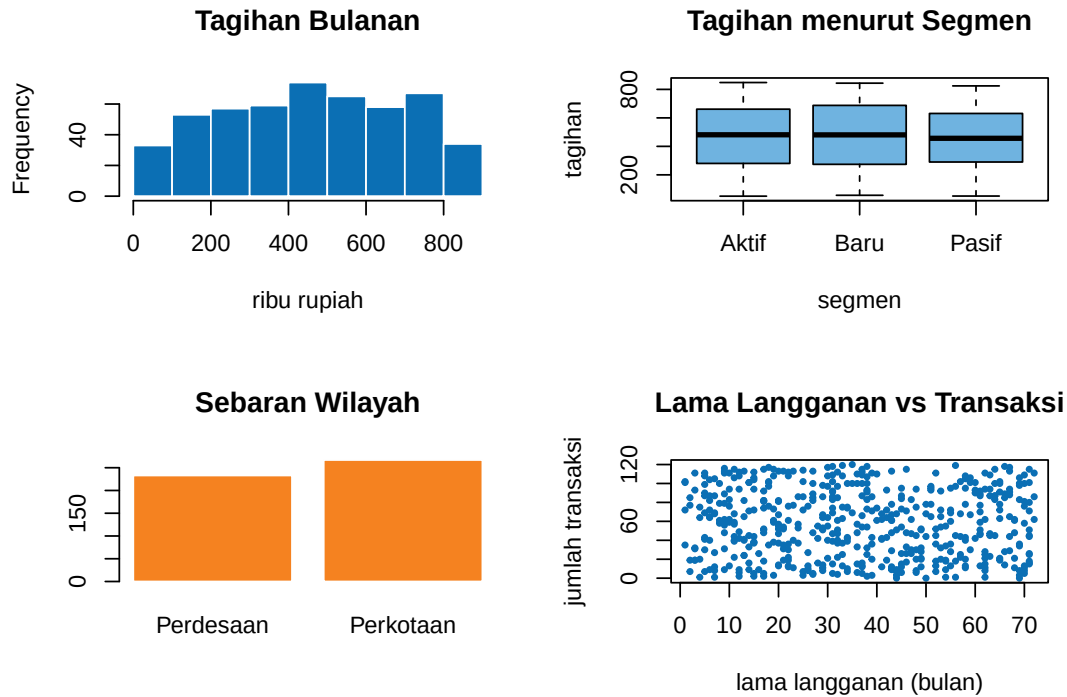
Kode 1.8 · R empat gambar eksplorasi

```

par(mfrow = c(2, 2))
hist(pel$tagihan_bulanan, main = "Tagihan Bulanan", col = "#0B6FB4",
     border = "white", xlab = "ribu rupiah")
boxplot(tagihan_bulanan ~ segmen, data = pel, col = "#6FB3E0",
        main = "Tagihan menurut Segmen", xlab = "segmen", ylab = "tagihan")
barplot(table(pel$wilayah), col = "#F58220", border = "white",
        main = "Sebaran Wilayah")
plot(pel$lama_langganan, pel$jumlah_transaksi, pch = 19, cex = .5,
     col = "#0B6FB4", main = "Lama Langganan vs Transaksi",

```

```
xlab = "lama langganan (bulan)", ylab = "jumlah transaksi")
```



```
par(mfrow = c(1, 1))
```

Penjelasan Kode. Empat gambar disajikan sekaligus memakai pengaturan mfrow dua baris dua kolom. Histogram untuk sebaran satu peubah numerik, diagram kotak-garis untuk membandingkan antar-kelompok, diagram batang untuk peubah kategori, dan diagram pencar untuk hubungan dua peubah numerik.

Interpretasi Output

Empat jenis gambar ini menjawab empat pertanyaan berbeda dan sebaiknya dikerjakan pada tiap penjelajahan awal. Perhatikan bahwa pemilihan jenis gambar ditentukan oleh skala pengukuran peubahnya — diagram batang bagi kategori, histogram bagi numerik. Memakai histogram atas peubah nominal akan menghasilkan gambar yang secara teknis terbentuk tetapi tidak bermakna.

Contoh 1.9 | Memperagakan bahaya pola semu

Kode 1.9 • R cor.test atas seratus peubah acak

```

set.seed(2026)
n <- 200; p <- 100
X <- matrix(rnorm(n * p), n, p)      # SELURUHNYA acak
y <- rnorm(n)                        # sasaran juga acak

nilai_p <- apply(X, 2, function(v) cor.test(v, y)$p.value)

cat("peubah nyata pada alpha 0,05:", sum(nilai_p < 0.05), "dari", p, "\n")

peubah nyata pada alpha 0,05: 5 dari 100

cat("harapan bila murni kebetulan:", 0.05 * p, "\n")

harapan bila murni kebetulan: 5

cat("nilai-p terkecil:", format(min(nilai_p), digits = 3), "\n")

nilai-p terkecil: 0.0141

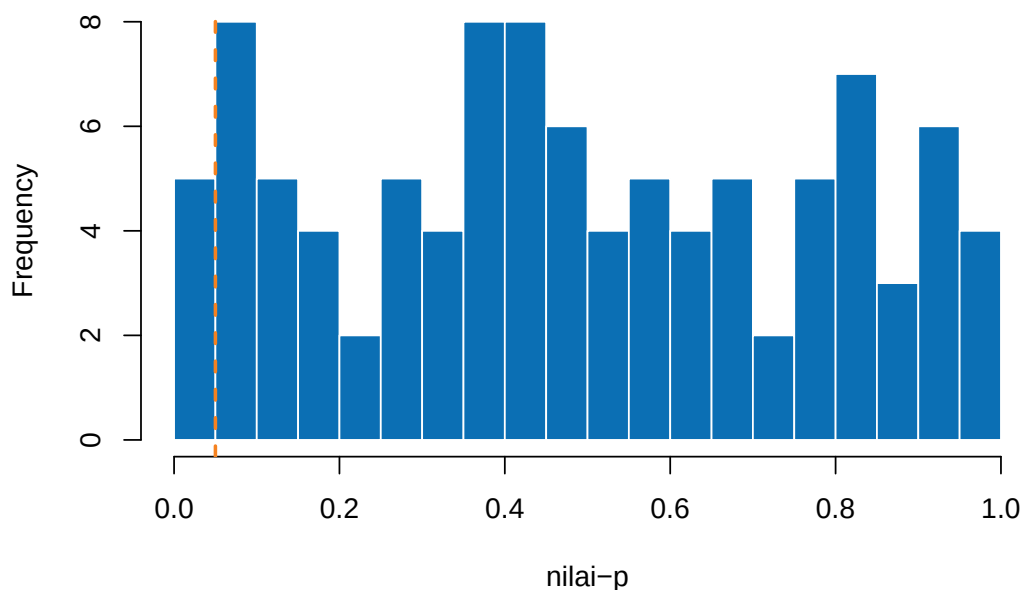
cat("setelah koreksi Bonferroni:", sum(p.adjust(nilai_p, "bonferroni") < 0.05), "\n")

setelah koreksi Bonferroni: 0

hist(nilai_p, breaks = 20, col = "#0B6FB4", border = "white",
     main = "Sebaran nilai-p pada data acak", xlab = "nilai-p")
abline(v = 0.05, col = "#F58220", lwd = 2, lty = 2)

```

Sebaran nilai-p pada data acak



Penjelasan Kode. Seratus peubah acak diuji korelasinya terhadap sasaran yang juga acak. Fungsi `apply` menjalankan `cor.test` pada tiap kolom dan mengambil nilai-p-nya. Koreksi Bonferroni diterapkan memakai `p.adjust` untuk membandingkan hasilnya.

Interpretasi Output

Inilah bahaya pola semu dalam wujud angka. Kira-kira lima peubah tampak nyata padahal datanya sepenuhnya acak, dan jumlah itu persis sesuai harapan teoretis lima persen dari seratus. Perhatikan pula sebaran nilai-p yang mendekati seragam — ciri khas ketiadaan hubungan. Setelah koreksi Bonferroni tidak ada lagi yang nyata, sebagaimana seharusnya. Pada data mining berpeubah ribuan, risiko ini berlipat ganda dan menjadi alasan utama mengapa penilaian selalu dilakukan pada data uji yang terpisah.

Memilih peubah karena nilai-p kecil. Godaan terbesar setelah melihat hasil semacam itu adalah memilih kelima peubah yang tampak nyata lalu memodelkannya. Model yang dihasilkan akan tampak baik pada data yang sama dan gagal total pada data baru. Modul 5 menakar besarnya akibat kekeliruan ini dengan angka.

Contoh 1.10 | Meringkas dua peubah kategori sekaligus

Kode 1.10 • R `table`, `prop.table`, dan `chisq.test`

```
silang <- table(pel$wilayah, pel$segmen)
print(silang)
```

	Aktif	Baru	Pasif
Perdesaan	123	41	69
Perkotaan	129	73	65

```
cat("\nProporsi baris (per wilayah):\n")
```

```
Proporsi baris (per wilayah):
```

```
print(round(prop.table(silang, margin = 1), 3))
```

	Aktif	Baru	Pasif
Perdesaan	0.528	0.176	0.296
Perkotaan	0.483	0.273	0.243

```
cat("\nUji kebebasan khi-kuadrat:\n")
```

```
Uji kebebasan khi-kuadrat:
```

```
uji <- chisq.test(silang)
cat("statistik:", round(uji$statistic, 4),
    "| db:", uji$parameter,
    "| nilai-p:", round(uji$p.value, 4), "\n")
```

```
statistik: 6.965 | db: 2 | nilai-p: 0.0307
```

Penjelasan Kode. Tabel silang dua peubah kategori dibentuk, lalu diubah menjadi proporsi per baris. Uji khi-kuadrat memeriksa apakah kedua peubah saling bebas, dengan statistik, derajat bebas, dan nilai-p ditampilkan terpisah agar mudah dibaca.

Interpretasi Output

Karena kedua peubah dibangkitkan saling bebas, nilai-p yang besar adalah hasil yang diharapkan dan harus dilaporkan apa adanya. Perlu diingat bahwa uji kebebasan menjawab pertanyaan inferensia, bukan pertanyaan data mining. Bagi penambahan data, yang lebih berguna adalah seberapa banyak informasi yang disumbangkan satu peubah untuk menduga peubah lain — gagasan yang akan dipe- rdalam pada modul klasifikasi.

Contoh 1.11 | Menakar risiko pengenalan individu

Kode 1.11 • R memeriksa keunikan gabungan penanda

```
set.seed(2026)
n <- 300
samar <- data.frame(
  kode_pos = sample(10:15, n, TRUE),
  tahun_lahir = sample(1960:2000, n, TRUE),
  jenis_kelamin = sample(c("L", "P"), n, TRUE))

kunci <- paste(samar$kode_pos, samar$tahun_lahir, samar$jenis_kelamin)
tab <- table(kunci)

cat("amatan unik (k = 1):", sum(tab == 1), "dari", n,
    sprintf("(%.1f persen)\n", 100 * sum(tab == 1) / n))

amatan unik (k = 1): 172 dari 300 (57.3 persen)

cat("nilai k terkecil :", min(tab), "\n")

nilai k terkecil : 1

cat("kombinasi berbeda :", length(tab), "\n")
```

kombinasi berbeda : 224

Penjelasan Kode. Tiga peubah yang masing-masing tampak tidak sensitif digabungkan menjadi satu kunci. Fungsi table mencacah berapa orang berbagi tiap kombinasi. Banyaknya kombinasi yang hanya dimiliki satu orang dihitung, begitu pula nilai terkecil di antara seluruh cacahan.

Interpretasi Output

Lebih dari separuh amatan ternyata dapat dikenali secara unik hanya dari tiga peubah yang seluruhnya tampak tidak sensitif. Inilah yang dimaksud pada bagian Teori: data menjadi sensitif ketika digabungkan. Ukuran k terkecil bernilai satu berarti ada individu yang sepenuhnya terbuka. Praktik yang lazim menuntut nilai k minimum tertentu, misalnya lima, yang dicapai dengan mengasarkkan peubah — umpamanya mengganti tahun lahir menjadi kelompok umur.

Contoh 1.12 | Menarik contoh dari data besar

Kode 1.12 • R sample dan slice_sample

```
set.seed(2026)
# contoh acak sederhana
acak <- pel[sample(nrow(pel), 100), ]

# contoh acak berlapis menurut segmen
suppressPackageStartupMessages(library(dplyr))
berlapis <- pel |> group_by(segmen) |> slice_sample(prop = 0.2) |> ungroup()

banding <- rbind(
  populasi = prop.table(table(pel$segmen)),
  acak      = prop.table(table(acak$segmen)),
  berlapis  = prop.table(table(berlapis$segmen)))
round(banding, 4)
```

	Aktif	Baru	Pasif
populasi	0.5040	0.2280	0.2680
acak	0.5100	0.2500	0.2400
berlapis	0.5102	0.2245	0.2653

Penjelasan Kode. Dua cara penarikan contoh dibandingkan. Cara pertama menarik seratus baris secara acak sederhana. Cara kedua menarik dua puluh persen dari tiap segmen memakai slice_sample setelah pengelompokan, sehingga proporsi tiap segmen terjaga. Ketiga proporsi disandingkan.

Interpretasi Output

Bandingkan ketiga baris. Contoh berlapis mempertahankan proporsi segmen nyaris persis, sedangkan contoh acak sederhana dapat menyimpang terutama pada segmen kecil. Pada penambahan data berukuran besar, penarikan contoh berguna untuk mempercepat penjelajahan awal — tetapi penstrataan menjadi penting agar kelompok kecil tidak hilang, khususnya ketika kelompok kecil itulah yang justru menarik.

Contoh 1.13 | Menyusun fungsi penjelajahan yang dapat dipakai ulang**Kode 1.13 • R fungsi jelajah**

```
jelajah <- function(d, nama = deparse(substitute(d))) {
  num <- names(d)[sapply(d, is.numeric)]
  kat <- names(d)[sapply(d, function(x) is.factor(x) || is.character(x))]
  cat("=== ", nama, " ===\n", sep = "")
  cat("dimensi      :", nrow(d), "baris x", ncol(d), "kolom\n")
  cat("numerik      :", length(num), "|", paste(utils::head(num, 4), collapse = ", "), "\n")
  cat("kategori      :", length(kat), "|", paste(utils::head(kat, 4), collapse = ", "), "\n")
  cat("nilai hilang:", sum(is.na(d)), "\n")
  cat("baris kembar:", sum(duplicated(d)), "\n")
  invisible(list(numerik = num, kategori = kat))
}

hasil <- jelajah(pel)
```

```
=== pel ===
dimensi      : 500 baris x 8 kolom
numerik      : 5 | id_pelanggan, lama_langganan, tagihan_bulanan, jumlah_transaksi
kategori      : 3 | wilayah, segmen, tagihan_tinggi
nilai hilang: 0
baris kembar: 0
```

Penjelasan Kode. Fungsi menerima sebuah data frame dan memisahkan kolomnya menurut tipe memakai `sapply`. Nama objek diambil otomatis lewat `deparse` atas `substitute` sehingga keluarannya berjudul. Hasil dikembalikan secara tersembunyi memakai `invisible` agar dapat dipakai lanjut tanpa tercetak dua kali.

Interpretasi Output

Membungkus pemeriksaan rutin menjadi fungsi adalah kebiasaan yang terbayar sepanjang mata kuliah ini, sebab langkah yang sama diulang pada tiap modul. Perhatikan pemakaian `invisible`: fungsi

tetap mencetak ringkasan untuk dibaca, sekaligus mengembalikan daftar nama kolom yang dapat dipakai langkah berikutnya — gabungan yang menghemat banyak pengetikan.

Contoh 1.14 | Memeriksa hubungan antar-peubah numerik

Kode 1.14 • R cor dan pairs

```
num <- pel[, c("lama_langganan", "tagihan_bulanan",
               "jumlah_transaksi", "skor_kepuasan")]
```

```
K <- round(cor(num), 3)
```

```
print(K)
```

	lama_langganan	tagihan_bulanan	jumlah_transaksi	skor_kepuasan
lama_langganan	1.000	-0.019	-0.033	0.016
tagihan_bulanan	-0.019	1.000	-0.036	-0.031
jumlah_transaksi	-0.033	-0.036	1.000	-0.051
skor_kepuasan	0.016	-0.031	-0.051	1.000

```
# pasangan paling erat di luar diagonal
```

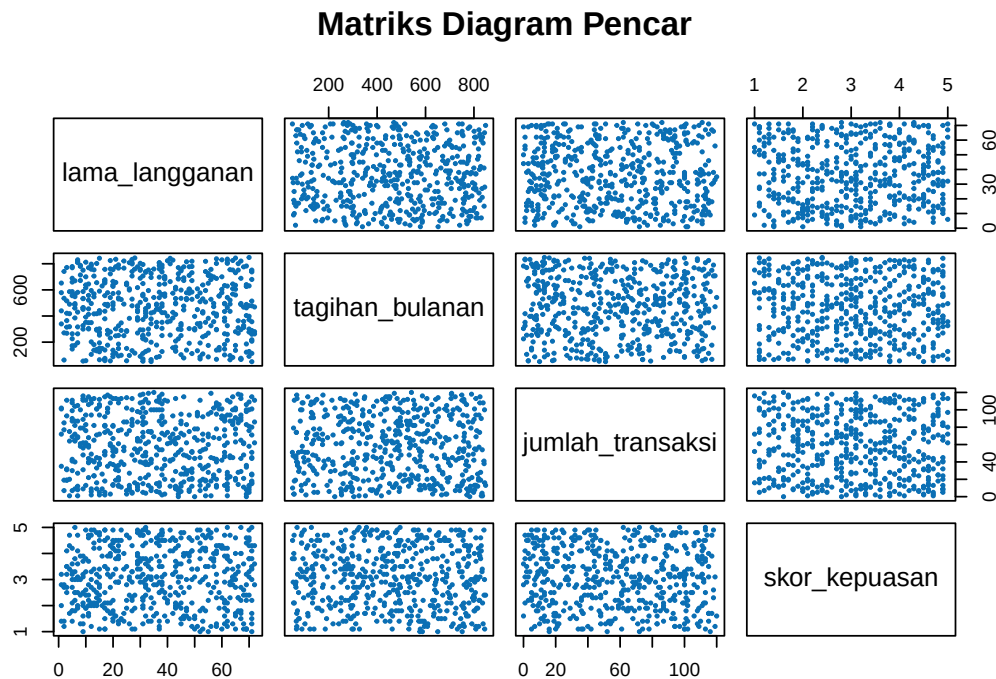
```
K2 <- K; diag(K2) <- 0
```

```
pos <- which(abs(K2) == max(abs(K2)), arr.ind = TRUE)[1, ]
```

```
cat("\npasangan terkuat:", rownames(K)[pos[1]], "dan", colnames(K)[pos[2]],
    "\n| r =", K[pos[1], pos[2]], "\n")
```

```
pasangan terkuat: skor_kepuasan dan jumlah_transaksi | r = -0.051
```

```
pairs(num, pch = 19, cex = .4, col = "#0B6FB4",
       main = "Matriks Diagram Pencar")
```



Penjelasan Kode. Matriks korelasi keempat peubah numerik dihitung dan dibulatkan. Diagonalnya dinolkan lebih dahulu sebelum mencari nilai mutlak terbesar, agar pasangan terkuat tidak terpilih dari diagonalnya sendiri yang selalu bernilai satu. Fungsi `pairs` menggambar seluruh pasangan diagram pencar sekaligus.

Interpretasi Output

Matriks korelasi wajib dibaca bersama matriks diagram pencarnya, sebab korelasi hanya mengukur hubungan linier. Pola melengkung dapat menghasilkan korelasi mendekati nol meski hubungannya kuat. Karena peubah pada data ini dibangkitkan saling bebas, korelasi yang seluruhnya lemah adalah hasil yang benar — dan melaporkannya apa adanya lebih baik daripada mencari-cari hubungan yang tidak ada.

Contoh 1.15 | Menyusun tabel ringkasan lintas kelompok

Kode 1.15 • R peringkasan dua peubah dan reshape

```
suppressPackageStartupMessages(library(dplyr))
ringkas <- pel |>
  group_by(wilayah, segmen) |>
```

```

summarise(n = n(),
          rata_tagihan = round(mean(tagihan_bulanan), 1),
          rata_transaksi = round(mean(jumlah_transaksi), 1),
          .groups = "drop") |>
arrange(wilayah, desc(rata_tagihan))
print(as.data.frame(ringkas), right = FALSE)

```

	wilayah	segmen	n	rata_tagihan	rata_transaksi
1	Perdesaan	Aktif	123	470.2	57.7
2	Perdesaan	Baru	41	455.5	66.0
3	Perdesaan	Pasif	69	437.5	53.3
4	Perkotaan	Baru	73	473.9	55.6
5	Perkotaan	Aktif	129	466.8	54.4
6	Perkotaan	Pasif	65	464.4	64.4

```

# bentuk lebar agar mudah dibandingkan
lebar <- reshape(as.data.frame(ringkas[, c("wilayah", "segmen", "rata_tagihan")]),
                 idvar = "wilayah", timevar = "segmen", direction = "wide")
print(lebar, right = FALSE)

```

	wilayah	rata_tagihan.Aktif	rata_tagihan.Baru	rata_tagihan.Pasif
1	Perdesaan	470.2	455.5	437.5
4	Perkotaan	466.8	473.9	464.4

Penjelasan Kode. Data dikelompokkan menurut dua peubah sekaligus lalu diringkas. Hasilnya diurutkan menurut wilayah dan rata-rata tagihan menurun. Fungsi reshape mengubah bentuk panjang menjadi lebar, dengan wilayah sebagai penanda baris dan segmen sebagai kolom.

Interpretasi Output

Bentuk panjang lebih baik untuk pengolahan lanjutan, sedangkan bentuk lebar lebih mudah dibaca manusia. Keduanya memuat informasi yang sama sehingga pemilihannya bergantung keperluan. Perhatikan bahwa peringkasan dua peubah menghasilkan sel yang lebih sedikit anggotanya; bila salah satu sel hanya berisi beberapa amatan, rata-ratanya menjadi tidak stabil dan tidak layak ditafsirkan.

Contoh 1.16 | Mendeteksi amatan yang menyimpang

Kode 1.16 • R skor-z dan jarak Mahalanobis

```

# skor-z pada tiap peubah numerik
z <- scale(pel[, c("tagihan_bulanan", "jumlah_transaksi", "lama_langganan")])
ekstrem <- apply(abs(z) > 3, 1, any)
cat("amatan berskor-z melebihi 3:", sum(ekstrem),
    sprintf("(%.2f persen)\n", 100 * mean(ekstrem)))

```

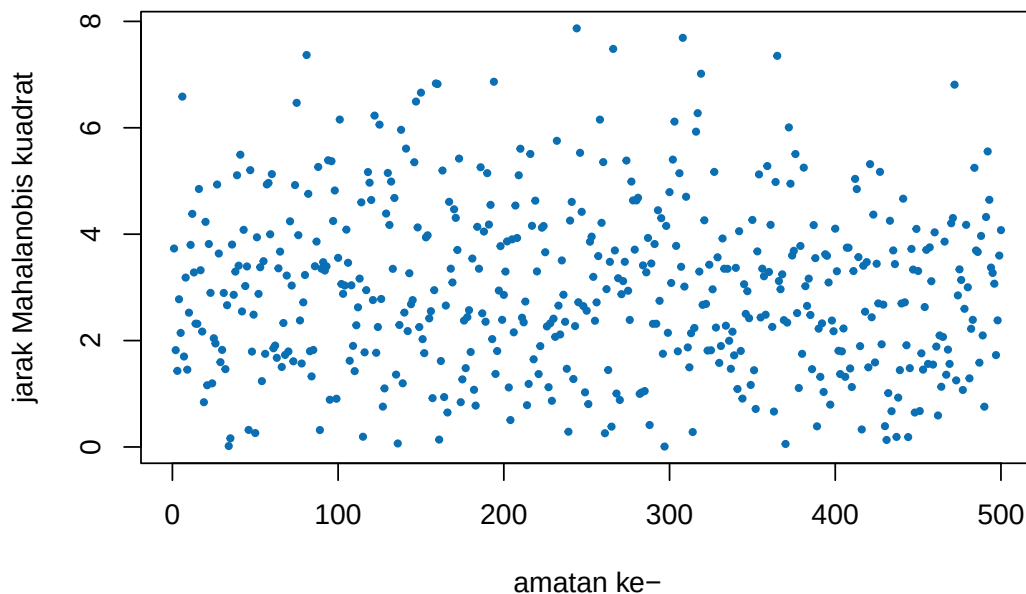
```
amatan berskor-z melebihi 3: 0 (0.00 persen)
```

```
# jarak Mahalanobis memperhitungkan korelasi antar-peubah
num <- pel[, c("tagihan_bulanan", "jumlah_transaksi", "lama_langganan")]
d2 <- mahalanobis(num, colMeans(num), cov(num))
batas <- qchisq(0.99, df = ncol(num))
cat("amatan menyimpang menurut Mahalanobis:", sum(d2 > batas),
    sprintf("%.2f persen)\n", 100 * mean(d2 > batas)))
```

```
amatan menyimpang menurut Mahalanobis: 0 (0.00 persen)
```

```
plot(d2, pch = 19, cex = .5, col = ifelse(d2 > batas, "#F58220", "#0B6FB4"),
     ylab = "jarak Mahalanobis kuadrat", xlab = "amatan ke-",
     main = "Deteksi Amatan Menyimpang")
abline(h = batas, col = "#C0392B", lty = 2, lwd = 2)
```

Deteksi Amatan Menyimpang



Penjelasan Kode. Dua cara deteksi dibandingkan. Skor-z memeriksa tiap peubah sendiri-sendiri; amatan ditandai bila salah satu peubahnya menyimpang lebih dari tiga simpangan baku. Jarak Mahalanobis memeriksa seluruh peubah sekaligus dengan memperhitungkan korelasinya, dan batasnya diambil dari sebaran khi-kuadrat berderajat bebas sebanyak peubah.

Interpretasi Output

Kedua cara dapat menandai amatan yang berbeda. Skor-z melewati amatan yang pada tiap peubah tampak wajar tetapi kombinasinya ganjil — misalnya pelanggan berlangganan sangat lama namun bertransaksi sangat sedikit. Mahalanobis menangkapnya karena memperhitungkan hubungan antar-peubah. Deteksi anomali adalah salah satu dari enam tugas penambangan, dan di sini diperkenalkan dalam bentuk paling sederhananya.

Contoh 1.17 | Merumuskan satu persoalan sebagai dua tugas berbeda**Kode 1.17 • R** **lm dan glm atas persoalan yang sama**

```
# TUGAS REGRESI: menduga besarnya tagihan
mr <- lm(tagihan_bulanan ~ lama_langganan + jumlah_transaksi + skor_kepuasan,
        data = pel)
cat("REGRESI | R-kuadrat:", round(summary(mr)$r.squared, 4),
    "| RMSE:", round(sqrt(mean(residuals(mr)^2)), 2), "\n")
```

```
REGRESI | R-kuadrat: 0.0027 | RMSE: 229.1
```

```
# TUGAS KLASIFIKASI: menduga apakah tagihan tergolong tinggi
pel$tinggi <- factor(ifelse(pel$tagihan_bulanan > median(pel$tagihan_bulanan),
                          "tinggi", "rendah"))
mk <- glm(tinggi ~ lama_langganan + jumlah_transaksi + skor_kepuasan,
        data = pel, family = binomial)
dug <- ifelse(predict(mk, type = "response") > 0.5, "tinggi", "rendah")
cat("KLASIFIKASI | akurasi:", round(mean(dug == pel$tinggi), 4),
    "| acuan mayoritas:", round(max(prop.table(table(pel$tinggi))), 4), "\n")
```

```
KLASIFIKASI | akurasi: 0.52 | acuan mayoritas: 0.5
```

Penjelasan Kode. Persoalan yang sama dimodelkan dua kali. Sebagai regresi, tagihan diduga nilainya dan dinilai dengan R-kuadrat serta akar rerata kuadrat galat. Sebagai klasifikasi, tagihan diubah menjadi dua kategori berdasarkan median lalu dimodelkan dengan regresi logistik dan dinilai dengan akurasi.

Interpretasi Output

Bandingkan kedua ukuran keberhasilan; keduanya tidak sebanding satu sama lain sehingga tidak dapat dipakai memilih tugas mana yang lebih baik. Yang menentukan adalah kebutuhan pengguna: bila keputusannya bersifat ya atau tidak, klasifikasi lebih sesuai; bila besarnya nilai diperlukan, regresi lebih tepat. Perhatikan pula bahwa akurasi wajib dibandingkan terhadap acuan tebakan mayoritas —

tanpa pembandingan itu, angka akurasi tidak bermakna apa-apa.

Contoh 1.18 | Menyusun peta persoalan menurut CRISP-DM

Kode 1.18 • R data.frame sebagai peta kerja

```
peta <- data.frame(
  tahap = c("1 Business Understanding", "2 Data Understanding",
            "3 Data Preparation", "4 Modeling",
            "5 Evaluation", "6 Deployment"),
  pertanyaan = c("Pelanggan mana yang berpotensi berhenti?",
                 "Peubah apa yang tersedia dan bagaimana mutunya?",
                 "Bagaimana menangani nilai hilang dan skala berbeda?",
                 "Model apa yang sesuai tipe data ini?",
                 "Apakah ketepatanannya cukup bagi keputusan bisnis?",
                 "Bagaimana hasilnya dipakai dan dipantau?"),
  modul = c("1, 2, 7", "3, 4", "5", "6, 8-12", "6, 8-12", "13, 14"))
print(peta, right = FALSE)
```

tahap	pertanyaan	modul
1 1 Business Understanding	Pelanggan mana yang berpotensi berhenti?	1, 2, 7
2 2 Data Understanding	Peubah apa yang tersedia dan bagaimana mutunya?	3, 4
3 3 Data Preparation	Bagaimana menangani nilai hilang dan skala berbeda?	5
4 4 Modeling	Model apa yang sesuai tipe data ini?	6, 8-12
5 5 Evaluation	Apakah ketepatanannya cukup bagi keputusan bisnis?	6, 8-12
6 6 Deployment	Bagaimana hasilnya dipakai dan dipantau?	13, 14

Penjelasan Kode. Sebuah data.frame dibentuk berisi enam tahap CRISP-DM, contoh pertanyaan pada tiap tahap, dan modul mana yang membahasnya. Argumen `right` bernilai `FALSE` pada `print` membuat teks dirata-kiri sehingga lebih mudah dibaca.

Interpretasi Output

Peta ini sebaiknya Anda simpan dan buka kembali pada tiap modul. Kesulitan yang paling sering dialami pemula bukan pada algoritma, melainkan kehilangan arah: lupa sedang berada pada tahap apa dan untuk menjawab pertanyaan apa. Menuliskan pertanyaan tahap pertama sebelum menyentuh data adalah kebiasaan yang menolong sepanjang mata kuliah ini.

1.7 Latihan Mandiri

Kerjakan bertahap. **Dasar** melengkapi kode; **Menengah** menulis sendiri dan membetulkan kode; **Lanjut** menyusun rumusan persoalan. Gunakan data/data_pelanggan.csv.

1.7.0.0.1 Tingkat Dasar (melengkapi kode)

Latihan 1.1 | Penjelajahan awal

```
pel <- read.csv("data/data_pelanggan.csv", stringsAsFactors = TRUE)
____(pel)                # banyaknya baris dan kolom
____(pel)                # tipe tiap kolom
colSums(____(pel))       # cacah nilai hilang per kolom
```

Latihan 1.2 | Sebaran kategori

```
____(pel$segmen)          # cacah tiap segmen
round(____(table(pel$segmen)), 3) # ubah menjadi proporsi
```

Latihan 1.3 | Ringkasan per kelompok

```
library(dplyr)
pel |>
  ____(wilayah) |>
  summarise(n = ____(), rata = mean(tagihan_bulanan), .groups = "drop")
```

Latihan 1.4 | Skala pengukuran

```
sapply(pel, function(x) ____ (x)[1]) # kelas tiap kolom
sapply(pel, function(x) length(____ (x))) # cacah nilai unik tiap kolom
```

Latihan 1.5 | Tabel silang dan uji kebebasan

```
silang <- ____ (pel$wilayah, pel$segmen) # tabel silang
round(prop.table(silang, margin = ____), 3) # proporsi PER BARIS
____ (silang)$p.value                    # uji kebebasan khi-kuadrat
```

1.7.0.0.2 Tingkat Menengah (menulis sendiri)

Latihan 1.6 | Penjelajahan mandiri

Susun ringkasan yang menjawab tiga pertanyaan berikut, seluruhnya dengan kode Anda sendiri: (a) berapa rata-rata dan median lama_langganan pada tiap segmen; (b) segmen mana yang skor kepuasannya paling rendah; (c) apakah pelanggan berlangganan lebih lama cenderung bertagihan lebih besar — jawab dengan koefisien korelasi dan diagram pencar.

Latihan 1.7 | Membetulkan kode yang salah

Kode berikut memuat empat kesalahan (argumen `is.na` keliru, `prop.table` tanpa margin sehingga salah tafsir, pengelompokan tidak dilepaskan, dan `mean` pada kolom faktor). Temukan dan perbaiki, lalu jelaskan tiap kesalahannya.

```
pel <- read.csv("data/data_pelanggan.csv", stringsAsFactors = TRUE)

# (a) is.na diberi nama kolom sebagai teks
colSums(is.na("pel"))

# (b) proporsi tanpa margin -- ditafsirkan sebagai proporsi per kolom
prop.table(table(pel$segmen, pel$wilayah))

# (c) pengelompokan tidak dilepaskan sehingga terbawa
library(dplyr)
hasil <- pel |> group_by(segmen) |> summarise(n = n())

# (d) rata-rata dihitung pada kolom faktor
mean(pel$wilayah)
```

Keluaran yang benar. Setelah diperbaiki, kode yang dipakai adalah sebagai berikut. Gunakan ini untuk memverifikasi perbaikan Anda.

```
# (a) is.na dikenakan pada objek, bukan pada teks berisi namanya
colSums(is.na(pel))

# (b) margin = 2 bila ingin proporsi per kolom
prop.table(table(pel$segmen, pel$wilayah), margin = 2)

# (c) lepaskan pengelompokan setelah peringkasan
hasil <- pel |> group_by(segmen) |> summarise(n = n(), .groups = "drop")
```

```
# (d) kolom faktor dicacah, bukan dirata-ratakan  
table(pel$wilayah)
```

Pemeriksaan yang wajib berhasil setelah perbaikan: `sum(colSums(is.na(pel)))` bernilai **0**, dan `all(round(colSums(prop.table(table(pel$segmen, pel$wilayah), margin = 2)), 8) == 1)` bernilai **TRUE**.

Latihan 1.8 | Menakar pola semu sendiri

Ulangi peragaan pada Contoh 1.10 dengan tiga nilai banyaknya peubah: 20, 100, dan 500. Untuk masing-masing, laporkan (a) berapa peubah tampak nyata pada taraf lima persen; (b) berapa yang diharapkan secara teoretis; (c) berapa yang tersisa setelah koreksi Bonferroni. Sajikan sebagai tabel dan jelaskan polanya.

Latihan 1.9 | Menakar risiko pengenalan

Ulangi Contoh 1.12 dengan mengasarkan tahun lahir menjadi **kelompok umur lima tahunan**. Laporkan berapa persen amatan yang tetap unik dan berapa nilai k terkecil. Bandingkan terhadap hasil semula, lalu jelaskan mengapa pengasaran menurunkan risiko sekaligus apa yang hilang darinya.

1.7.0.0.3 Tingkat Lanjut (rumusan persoalan)

Latihan 1.10 | Menempatkan persoalan pada CRISP-DM

Pilih satu persoalan nyata yang dapat dijawab dengan data pelanggan ini — misalnya mengenali pelanggan berpotensi berhenti, atau menetapkan sasaran promosi. Untuk persoalan itu: (a) tuliskan rumusan tahap *Business Understanding* beserta **kriteria sukses yang terukur**; (b) sebutkan peubah mana yang relevan dan alasannya; (c) tentukan pada tahap CRISP-DM mana pekerjaan Anda akan paling banyak menghabiskan waktu, dan jelaskan mengapa; (d) sebutkan satu risiko etika dari penerapan hasilnya.

1.8 Penugasan

Sebagai calon analis data pada instansi statistik, susun laporan penjelajahan awal atas data pelanggan tiruan `data_pelanggan.csv` (*data simulasi*) sekaligus merumuskan persoalan penambangan data yang layak dikerjakan.

Konteks. Sebuah unit layanan memiliki catatan 500 pelanggan berisi wilayah, segmen, lama langganan, tagihan bulanan, jumlah transaksi, dan skor kepuasan. Pimpinan ingin mengetahui apakah data ini cukup untuk mendukung keputusan penawaran layanan.

Tugas 1.1 | Data Understanding

1. Lakukan penjelajahan menyeluruh: dimensi, tipe kolom, ringkasan tiap peubah, cacah nilai hilang, dan pemeriksaan kewajaran rentang. Laporkan temuan yang patut diperhatikan.
2. Sajikan sebaran ketiga peubah kategori dan tiga peubah numerik dengan tabel maupun gambar. Jelaskan bentuk sebarannya, termasuk apakah ada yang miring.
3. Periksa hubungan antar-peubah numerik dengan matriks korelasi. Sebutkan pasangan yang paling erat dan tafsirlah artinya.

Tugas 1.2 | Business Understanding

4. Rumuskan **dua persoalan** yang dapat dijawab dengan data ini: satu bersifat **inferensia** dan satu bersifat **data mining**. Untuk masing-masing, tuliskan pertanyaannya, ukuran keberhasilannya, dan mengapa jenis pendekatannya berbeda.
5. Untuk persoalan data mining yang Anda pilih, susun rencana ringkas menurut **enam tahap CRISP-DM**, memuat perkiraan kesulitan pada tiap tahap.

Tugas 1.3 | Penjelajahan Lanjut

8. Sajikan **empat jenis gambar** eksplorasi (histogram, diagram kotak-garis, diagram batang, diagram pencar). Untuk masing-masing, jelaskan mengapa jenis itu sesuai bagi skala pengukuran peubahnya.
9. Susun tabel skala pengukuran seluruh peubah. Tandai peubah yang **dugaan otomatisnya keliru** dan jelaskan mengapa.
10. Peragakan bahaya **pola semu**: bangkitkan 100 peubah acak, uji korelasinya terhadap `tagihan_bulanan`, lalu laporkan berapa yang tampak nyata sebelum dan sesudah koreksi Bonferroni.

11. Takar **risiko pengenalan individu** bila data pelanggan digabungkan dengan kode pos dan tahun lahir. Laporkan persentase amatan unik, lalu usulkan satu langkah pengasaran beserta akibatnya terhadap kegunaan data.

Tugas 1.4 | Etika dan Laporan

6. Bahas **dua pertimbangan etika** yang muncul bila hasil analisis ini dipakai menyasar pelanggan tertentu. Sebutkan pula langkah pencegahannya.
7. Susun laporan R Markdown berisi kode, keluaran, gambar, dan interpretasi. Tutup dengan **ringkasan (maksimal 300 kata)** yang memuat: temuan pokok, persoalan yang diusulkan, keterbatasan data, dan penegasan bahwa data merupakan simulasi sehingga hasilnya tidak menggambarkan keadaan sebenarnya.

1.9 Ringkasan Modul

Daftar periksa penguasaan:

- ☐ Saya dapat menjelaskan pengertian data mining beserta empat sifat pola yang dicarinya.
- ☐ Saya dapat membedakan data mining dari statistika inferensia pada enam aspek.
- ☐ Saya dapat menyebutkan enam tahap CRISP-DM dan menempatkan suatu pekerjaan padanya.
- ☐ Saya dapat mengenali jenis data beserta perlakuan yang sesuai.
- ☐ Saya dapat melakukan penjelajahan awal: dimensi, tipe, ringkasan, nilai hilang.
- ☐ Saya dapat merumuskan pertanyaan inferensia dan pertanyaan data mining atas data yang sama.
- ☐ Saya menyadari risiko menemukan pola semu pada data berpeubah banyak.
- ☐ Saya dapat menyebutkan pertimbangan etika sebelum menambang data.

Kartu Referensi Cepat. Perintah penjelajahan awal:

Keperluan	Perintah
nDimensi data	<code>dim(d), nrow(d), ncol(d)</code>
nTipe tiap kolom	<code>str(d)</code>
nRingkasan menyeluruh	<code>summary(d)</code>
nNilai hilang per kolom	<code>colSums(is.na(d))</code>
nCacah kategori	<code>table(d\$k)</code>
nProporsi per kolom	<code>prop.table(tab, margin = 2)</code>
nRingkasan per kelompok	<code>group_by()</code> lalu <code>summarise()</code>
nKorelasi	<code>cor(d[, numerik])</code>
nMengukur waktu	<code>system.time(ekspresi)</code>

Modul ini menata cara berpikir sebelum menyentuh algoritma. Prinsip yang perlu dibawa: rumuskan persoalan sebelum mencari pola, kenali data sebelum memodelkannya, dan ingat bahwa menemukan pola pada data berpeubah banyak selalu disertai risiko menemukan kebetulan. Modul berikutnya membahas konsep big data beserta kendala teknis yang menyertainya.

1.10 Referensi

- Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann. (Rujukan utama konsep data mining dan KDD.)
- Tan, P.-N., Steinbach, M., Karpatne, A., & Kumar, V. (2019). *Introduction to Data Mining* (2nd ed.). Pearson.
- Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. *Proceedings of the 4th International Conference on Practical Applications of Knowledge Discovery and Data Mining*, 29-39.
- Pramana, S., Yuniarto, B., Santoso, I., Nooraeni, R., & Lystiani, L. (2018). *Data Mining dengan R: Konsep serta Implementasi*. In Media.
- Provost, F., & Fawcett, T. (2013). *Data Science for Business*. O'Reilly Media. (Kaitan antara data mining dan pengambilan keputusan.)

MODUL

2

KONSEP DASAR BIG DATA

Pertemuan Minggu ke-2

Sub-CPMK 2: Mahasiswa mampu menjelaskan konsep dan karakteristik big data, mengenali peluang pemanfaatannya, memahami tantangan kompleksitasnya, serta menjalankan alur pengolahan terdistribusi sederhana.

Penyusun: Achmad Syahrul Choir



Daftar Isi

2.1	Deskripsi Modul	2
2.2	Tujuan Praktikum	2
2.3	Alat dan Bahan	2
2.4	Instruksi Pengerjaan	3
2.5	Teori Singkat	3
2.5.0.1	Kedudukan pada CRISP-DM	3
2.5.0.2	Karakteristik Big Data	3
2.5.0.3	Peluang dan Tantangan bagi Statistik Resmi	4
2.5.0.4	Mengapa Cara Biasa Berhenti Memadai	4
2.5.0.5	Penyimpanan Menentukan Kecepatan	5
2.5.0.6	Skalabilitas: Bagaimana Biaya Tumbuh	5
2.5.0.7	Menghampiri Alih-alih Menghitung Tepat	6
2.5.0.8	Gagasan Pengolahan Terdistribusi	6
2.6	Contoh Praktikum dengan R	7
2.7	Latihan Mandiri	22
2.8	Penugasan	25
2.9	Tambahan: Pengolahan Terdistribusi dengan Apache Spark	26
2.10	Ringkasan Modul	29
2.11	Referensi	30

2.1 Deskripsi Modul

Modul 1 menutup dengan sebuah petunjuk: perbedaan waktu antara dua cara meringkas data yang sama menjadi terasa begitu datanya membesar. Modul ini menelusuri petunjuk tersebut sampai ke akarnya. Apa sesungguhnya yang membuat sekumpulan data disebut besar, dan mengapa cara pengolahan biasa berhenti memadai.

Jawabannya bukan semata soal jumlah baris. Sebuah berkas berukuran dua gigabita masih dapat diolah di komputer meja, sedangkan aliran data sensor yang datang setiap detik dapat menyulitkan meski ukurannya kecil. Karena itu big data dicirikan oleh beberapa dimensi sekaligus, bukan oleh satu ambang ukuran.

Materi dibahas dari konsep hingga peragaan: karakteristik big data, peluang pemanfaatannya bagi statistik resmi, tantangan yang menyertainya, serta gagasan pengolahan terdistribusi. Bagian Tambahan menyajikan alur kerja Apache Spark yang benar-benar dijalankan, memakai data berukuran kecil agar dapat dikerjakan di komputer mana pun.

Tentang data. Berkas data pada modul ini adalah **data simulasi** (`set.seed(2026)`), **bukan data nyata** dan **bukan data resmi** instansi mana pun.

2.2 Tujuan Praktikum

Setelah menyelesaikan praktikum ini secara tuntas, mahasiswa diharapkan mampu:

1. **Menjelaskan** karakteristik big data dan mengapa ukuran saja tidak memadai sebagai ciri pembeda.
2. **Menyebutkan** peluang pemanfaatan big data bagi statistik resmi beserta keterbatasannya.
3. **Menguraikan** tantangan teknis dan nonteknis dalam pengolahan big data.
4. **Mengukur** kendala memori dan waktu secara langsung pada data berukuran besar.
5. **Membandingkan** pendekatan dalam memori terhadap pendekatan terdistribusi.
6. **Menjalankan** alur pengolahan terdistribusi sederhana beserta pembacaan rencana eksekusinya.

2.3 Alat dan Bahan

Perangkat dan bahan yang diperlukan:

- **R** (RStudio) dengan paket `data.table` dan `dplyr`.
- **Python** dengan `pyspark` dan `pyarrow` untuk bagian Tambahan.

- **Java** versi 11 atau lebih baru (prasyarat Apache Spark).
- Berkas data: data/data_transaksi.csv.

2.4 Instruksi Pengerjaan

Ikuti tahapan berikut:

Sebelum praktikum: pastikan Java terpasang dengan menjalankan `java -version` pada terminal. Pasang pyspark dan pyarrow melalui `pip install pyspark pyarrow`.

Selama praktikum: kerjakan Contoh secara berurutan. Bagian Tambahan mengenai Spark dikerjakan **setelah Penugasan**, sebab memerlukan penyiapan tersendiri.

Pembagian kerja: Contoh dan Latihan dikerjakan di kelas; **Penugasan dikerjakan di luar sesi**.

Pengumpulan: susun laporan R Markdown, render ke PDF atau HTML, kumpulkan melalui Google Classroom.

2.5 Teori Singkat

2.5.0.1 Kedudukan pada CRISP-DM

Modul ini masih berada pada tahap Business Understanding. Pertanyaan yang dijawab bukan bagaimana memodelkan, melainkan apakah data yang tersedia dapat diolah sama sekali dengan sumber daya yang ada. Pertanyaan itu menentukan seluruh rencana kerja berikutnya, sebab pendekatan yang mustahil dijalankan tidak berguna betapapun benarnya secara teori.

2.5.0.2 Karakteristik Big Data

Big data lazim dicirikan oleh beberapa dimensi yang bermula dari tiga dan berkembang menjadi lima. Perlu ditegaskan sejak awal bahwa ciri pembeda ini bersifat relatif terhadap sumber daya yang tersedia, bukan ambang mutlak.

Lima dimensi big data.

Dimensi	Arti	Akibat teknis
Volume	besarnya ukuran data	tidak muat di memori; perlu pemrosesan bertahap
Velocity	kecepatan kedatangan	perlu pengolahan aliran, bukan tumpukan
Variety	keragaman bentuk	terstruktur, semi, dan tak terstruktur bermacam-macam
Veracity	keandalan	derau, duplikasi, dan galat pencatatan
Value	manfaat	besar belum tentu berguna

Besar tidak sama dengan berguna. Dimensi kelima kerap terlupakan. Data yang sangat besar tetapi tidak menjawab persoalan justru menambah biaya tanpa menambah pengetahuan. Dalam statistik resmi, data administratif berjuta baris kadang kalah berguna dibandingkan survei terancang berukuran ribuan, karena survei dirancang mewakili populasi sedangkan data administratif hanya mencakup yang tercatat. Ukuran tidak menggantikan keterwakilan.

2.5.0.3 Peluang dan Tantangan bagi Statistik Resmi

Bagi badan statistik, big data menjanjikan tiga hal: ketepatan waktu yang lebih baik karena data tersedia hampir seketika, rincian wilayah yang lebih halus, dan biaya pengumpulan yang lebih rendah dibandingkan survei lapangan. Ketiganya menarik, tetapi tidak diperoleh secara cuma-cuma.

Tantangan yang menyertai.

Keterwakilan	data yang tercatat belum tentu mewakili populasi sasaran; pengguna internet bukan cerminan seluruh penduduk
Kesinambungan	sumber dapat berubah atau berhenti tanpa pemberitahuan
Mutu	tidak ada rancangan penarikan contoh yang menjamin galatnya
Privasi	rincian yang tinggi mempermudah pengenalan individu
Infrastruktur	memerlukan perangkat dan keterampilan yang berbeda

2.5.0.4 Mengapa Cara Biasa Berhenti Memadai

R dan Python secara bawaan memuat seluruh data ke dalam memori. Selama data muat, pendekatan ini paling cepat dan paling sederhana. Persoalan muncul ketika ukuran data mendekati atau melampaui memori yang tersedia: sistem mulai memindahkan data ke cakram, dan kecepatan pengolahan anjlok drastis.

Terdapat tiga jalan keluar yang lazim. Pertama, memakai struktur data yang lebih hemat seperti `data.table`, yang mengurangi penyalinan. Kedua, mengolah data secara bertahap per potongan sehingga hanya sebagian yang berada di memori. Ketiga, memakai kerangka terdistribusi seperti Apache Spark yang membagi pekerjaan ke beberapa mesin atau beberapa inti.

2.5.0.5 Penyimpanan Menentukan Kecepatan

Volume tidak hanya soal muat atau tidak di memori, melainkan juga soal berapa lama data dipindahkan dari cakram. Pilihan format penyimpanan kerap memberi perbaikan yang lebih besar daripada pengoptimuman kode.

Empat pertimbangan format penyimpanan.

Aspek	Teks (CSV)	Biner (RDS, Parquet)
Ukuran	besar	jauh lebih kecil
Kecepatan baca	lambat, perlu diurai	cepat, langsung dimuat
Tipe kolom	hilang, harus ditebak	tersimpan
Keterbacaan manusia	dapat dibuka editor	tidak

Format berorientasi kolom seperti Parquet menambah satu keunggulan lagi: kolom yang tidak diperlukan tidak perlu dibaca sama sekali. Pada tabel berkolom ratusan yang hanya lima di antaranya dipakai, penghematannya sangat besar. Inilah alasan format kolom menjadi baku pada pengolahan big data.

Menyimpan hasil antara sebagai CSV. Kebiasaan menyimpan hasil antara sebagai CSV tampak aman karena mudah diperiksa, tetapi mahal pada data besar: ukurannya berlipat, pembacaannya lambat, dan tipe kolom hilang sehingga harus ditebak ulang setiap kali. Untuk hasil antara yang hanya dibaca program, format biner jauh lebih tepat. CSV disimpan hanya ketika memang perlu dibaca manusia atau dipertukarkan antar-perangkat lunak.

2.5.0.6 Skalabilitas: Bagaimana Biaya Tumbuh

Ketika data membesar, yang menentukan layak tidaknya suatu algoritma bukanlah kecepatannya pada data kecil, melainkan bagaimana biayanya tumbuh. Algoritma berbiaya linear tetap terjangkau pada data besar, sedangkan algoritma berbiaya kuadrat cepat menjadi mustahil.

Pertumbuhan biaya menurut ukuran data.

Orde	Contoh operasi	Bila data digandakan sepuluh kali
$O(1)$	mengambil satu baris	tetap
$O(\log n)$	pencarian terurut	bertambah sedikit
$O(n)$	menghitung rata-rata	sepuluh kali
$O(n \log n)$	pengurutan	sekitar sebelas kali
$O(n^2)$	seluruh pasangan jarak	seratus kali

Baris terakhir menjelaskan mengapa pengelompokan hierarkis dan KNN sukar diterapkan pada data besar:

keduanya menuntut perhitungan jarak antar seluruh pasangan amatan. Pada seratus ribu amatan, banyaknya pasangan sudah mendekati lima miliar.

2.5.0.7 Menghampiri Alih-alih Menghitung Tepat

Pada data besar, penghampiran kerap menjadi jalan keluar yang sah. Median yang dihitung dari contoh lima persen biasanya sudah cukup teliti bagi pengambilan keputusan, sementara biayanya berkurang dua puluh kali lipat. Yang penting adalah menyatakan bahwa nilainya hampiran, bukan menyajikannya seolah tepat.

Tips dan Praktik Baik. Penarikan contoh bukan tanda kemalasan. Justru pada data besar, contoh acak berukuran memadai memberi dugaan yang ketelitiannya nyaris sama dengan perhitungan penuh — sebab galat baku bergantung pada akar ukuran contoh, bukan pada rasio contoh terhadap populasi. Menambah contoh dari sepuluh ribu menjadi sejuta hanya menyempitkan selang kepercayaan sepuluh kali lipat, dengan biaya seratus kali lipat.

2.5.0.8 Gagasan Pengolahan Terdistribusi

Spark membagi data menjadi beberapa partisi, lalu menjalankan perhitungan pada tiap partisi secara sejajar sebelum menggabungkan hasilnya. Dua gagasan pentingnya adalah pembagian partisi dan penundaan eksekusi.

Transformasi dan aksi.

Transformasi	<code>filter, select, groupBy, withColumn</code> — <i>tidak</i> langsung dijalankan, hanya dicatat sebagai rencana
---------------------	--

Aksi	<code>show, count, collect, write</code> — memicu seluruh rencana dijalankan
-------------	--

Penundaan ini bukan kemalasan melainkan peluang pengoptimuman: karena Spark mengetahui seluruh rangkaian langkah sebelum menjalankannya, ia dapat menyusun ulang urutan agar lebih efisien, misalnya mendahulukan penyaringan sebelum penggabungan sehingga data yang dipindahkan antar-partisi jauh berkurang.

Tips dan Praktik Baik. Perintah `explain()` menampilkan rencana eksekusi yang disusun Spark. Membacanya adalah keterampilan penting: dari situ terlihat apakah penyaringan sudah didorong ke tahap awal, dan berapa banyak pertukaran data antar-partisi yang terjadi.

2.6 Contoh Praktikum dengan R

Contoh 2.1 | Mengukur jejak memori data

Kode 2.1 • R object.size dan perkiraan

```
trx <- read.csv("data/data_transaksi.csv", stringsAsFactors = TRUE)
dim(trx)

[1] 5000    5

format(object.size(trx), units = "KB")

[1] "119.6 Kb"

# perkiraan bila baris digandakan
ukuran_per_baris <- as.numeric(object.size(trx)) / nrow(trx)
round(c(per_baris_byte = ukuran_per_baris,
        perkiraan_1juta_MB = ukuran_per_baris * 1e6 / 1024^2), 2)

      per_baris_byte perkiraan_1juta_MB
              24.49              23.36
```

Penjelasan Kode. Fungsi `object.size` mengukur memori yang dipakai sebuah objek. Hasilnya dibagi banyaknya baris untuk memperoleh jejak per baris, lalu dikalikan satu juta dan dibagi 1024 pangkat dua agar terbaca dalam megabita.

Interpretasi Output

Perkiraan sederhana ini sudah cukup untuk memutuskan pendekatan. Bila satu juta baris diperkirakan memakan ratusan megabita, komputer meja masih sanggup. Bila perkiraannya mencapai puluhan gigabita, pendekatan dalam memori harus ditinggalkan sejak awal, bukan setelah kode gagal berjalan.

Contoh 2.2 | Membandingkan tiga cara meringkas

Kode 2.2 • R system.time atas tiga pendekatan

```
suppressPackageStartupMessages({library(data.table); library(dplyr)})
trx_dt <- as.data.table(trx)

w1 <- system.time(aggregate(nilai_transaksi ~ provinsi, trx, mean))["elapsed"]
w2 <- system.time(trx |> group_by(provinsi) |>
```



```
summarise(r = mean(nilai_transaksi), .groups = "drop"))[["elapsed"]]
w3 <- system.time(trx_dt[, list(r = mean(nilai_transaksi)), by = provinsi])[["elapsed"]]
round(c(aggregate = w1, dplyr = w2, data.table = w3), 4)
```

aggregate	dplyr	data.table
0.002	0.003	0.001

Penjelasan Kode. Data yang sama diringkas dengan tiga cara: aggregate dari basis R, rangkaian dplyr, dan sintaks data.table. Lamanya masing-masing diukur dengan system.time dan hanya komponen elapsed yang diambil.

Interpretasi Output

Selisih waktunya kecil pada data lima ribu baris, dan memang seharusnya demikian. Pelajarannya justru di situ: pada data kecil pemilihan perangkat nyaris tidak berpengaruh, sehingga mengoptimalkannya adalah pemborosan tenaga. Pengoptimuman baru berarti setelah terbukti ada kendala.

Contoh 2.3 | Mengolah data secara bertahap per potongan

Kode 2.3 · R membaca per potongan dengan skip dan nrow

```
# menghitung rata-rata tanpa memuat seluruh data sekaligus
berkas <- "data/data_transaksi.csv"
n_potong <- 1000
jumlah <- 0; cacah <- 0; lewati <- 1

repeat {
  potong <- tryCatch(
    read.csv(berkas, skip = lewati, nrow = n_potong, header = FALSE),
    error = function(e) NULL)
  if (is.null(potong) || nrow(potong) == 0) break
  jumlah <- jumlah + sum(potong[[4]]) # kolom nilai_transaksi
  cacah <- cacah + nrow(potong)
  lewati <- lewati + nrow(potong)
}
round(c(cacah = cacah, rata_bertahap = jumlah / cacah,
        rata_sekaligus = mean(trx$nilai_transaksi)), 4)
```

cacah	rata_bertahap	rata_sekaligus
5000	1254	1254

Penjelasan Kode. Berkas dibaca berulang kali dalam potongan seribu baris memakai argumen `skip` dan `nrows`. Jumlah dan cacah dikumpulkan lintas potongan, lalu rata-ratanya dihitung di akhir. Perulangan berhenti ketika potongan berikutnya kosong. Fungsi `tryCatch` menjaga agar pembacaan melewati akhir berkas tidak menghentikan program.

Interpretasi Output

Kedua rata-rata identik, membuktikan pendekatan bertahap memberi hasil yang sama tanpa memuat seluruh data. Inilah gagasan dasar pengolahan di luar memori. Perhatikan pula bahwa tidak semua besaran dapat dihitung begini: rata-rata bisa karena dapat diuraikan menjadi jumlah dan cacah, sedangkan median menuntut seluruh data hadir sekaligus.

Tips dan Praktik Baik. Besaran yang dapat diuraikan menjadi penjumlahan — rata-rata, ragam, cacah — mudah dihitung secara bertahap maupun terdistribusi. Besaran berbasis urutan seperti median dan kuantil memerlukan siasat khusus, umumnya berupa penghampiran. Perbedaan inilah yang menentukan mudah tidaknya suatu perhitungan diparalelkan.

Contoh 2.4 | Mengenali keragaman bentuk data

Kode 2.4 • R supply class dan ringkasan data.table

```
supply(trx, class)
```

id_transaksi	provinsi	kanal	nilai_transaksi	durasi_detik
"integer"	"factor"	"factor"	"numeric"	"integer"

```
# proporsi tiap kanal per provinsi
```

```
round(prop.table(table(trx$provinsi, trx$kanal), margin = 1), 3)
```

	Daring	Luring
Jawa Barat	0.588	0.412
Jawa Tengah	0.634	0.366
Papua	0.589	0.411
Sulawesi Selatan	0.594	0.406

```
# ringkasan durasi menurut kanal
```

```
trx_dt[, list(n = .N, rata_durasi = round(mean(durasi_detik), 1),
           rata_nilai = round(mean(nilai_transaksi), 1)), by = kanal]
```

kanal	n	rata_durasi	rata_nilai
1: Luring	1985	454.5	1259
2: Daring	3015	454.8	1251

Penjelasan Kode. Fungsi `supply` yang dikenakan pada data frame menerapkan class ke tiap kolom sekaligus. Tabel silang berproporsi baris memperlihatkan komposisi kanal pada tiap provinsi. Peringkasan `data.table` memakai penanda titik N yang berarti banyaknya baris pada tiap kelompok.

Interpretasi Output

Dimensi variety terlihat bahkan pada data sesederhana ini: kolom bertipe bilangan bulat, pecahan, dan faktor bercampur dalam satu tabel. Pada big data sesungguhnya, keragaman itu jauh lebih jauh lagi karena teks, citra, dan catatan waktu dapat berada dalam satu sumber.

Contoh 2.5 | Memeriksa dimensi veracity

Kode 2.5 • R pemeriksaan mutu data

```
# duplikasi
cat("id ganda:", sum(duplicated(trx$id_transaksi)), "\n")

id ganda: 1879

# nilai di luar rentang wajar
cat("nilai negatif:", sum(trx$nilai_transaksi < 0), "\n")

nilai negatif: 0

cat("durasi di atas 900 detik:", sum(trx$durasi_detik > 900), "\n")

durasi di atas 900 detik: 0

# pencilan menurut aturan simpangan kuartil
q <- quantile(trx$nilai_transaksi, c(.25, .75))
batas <- q[2] + 1.5 * diff(q)
cat("pencilan atas:", sum(trx$nilai_transaksi > batas),
    sprintf("(%.2f persen)\n", 100 * mean(trx$nilai_transaksi > batas)))

pencilan atas: 0 (0.00 persen)
```

Penjelasan Kode. Tiga jenis pemeriksaan dijalankan. Fungsi `duplicated` menandai nilai yang muncul lebih dari sekali. Pemeriksaan rentang mencari nilai yang mustahil menurut pengetahuan bidang. Aturan simpangan kuartil menetapkan batas pencilan sebagai kuartil ketiga ditambah satu setengah kali jarak antar-kuartil.

Interpretasi Output

Dimensi veracity diperiksa dengan cara yang konkret seperti ini, bukan dibicarakan secara abstrak. Pada data simulasi hasilnya bersih, tetapi pada data nyata pemeriksaan semacam ini hampir selalu menemukan sesuatu. Perlu diingat bahwa pencilan tidak otomatis berarti kesalahan; transaksi bernilai sangat besar bisa saja sah dan justru paling menarik untuk ditelaah.

Contoh 2.6 | Membandingkan format penyimpanan**Kode 2.6 • R** membandingkan ukuran berkas

```
set.seed(2026)
uji <- data.frame(g = sample(letters[1:6], 3e4, TRUE),
                  x = runif(3e4), y = rnorm(3e4))

f_csv <- tempfile(fileext = ".csv")
f_gz  <- tempfile(fileext = ".csv.gz")
f_rds <- tempfile(fileext = ".rds")

write.csv(uji, f_csv, row.names = FALSE)
write.csv(uji, gzfile(f_gz), row.names = FALSE)
saveRDS(uji, f_rds)

ukuran <- c(csv = file.size(f_csv), csv_gz = file.size(f_gz), rds = file.size(f_rds))
round(data.frame(KB = ukuran / 1024,
                 rasio_thd_csv = ukuran["csv"]), 3)
```

	KB	rasio_thd_csv
csv	1176.5	1.000
csv_gz	518.9	0.441
rds	402.7	0.342

Penjelasan Kode. Data yang sama disimpan dalam tiga format: teks biasa, teks termampatkan, dan biner bawaan R. Fungsi `file.size` mengukur besarnya masing-masing, lalu rasionya terhadap CSV dihitung.

Interpretasi Output

Pemampatan memangkas ukuran lebih dari separuh, sedangkan format biner memangkas lebih jauh lagi. Pada data lima ribu baris selisihnya terasa kecil, tetapi pada data berjuta baris selisih ini menentukan apakah data muat di cakram dan berapa lama pemindahannya lewat jaringan.

Contoh 2.7 | Membandingkan kecepatan pembacaan

Kode 2.7 • R system.time atas tiga format

```
w_csv <- system.time(read.csv(f_csv))["elapsed"]
w_gz  <- system.time(read.csv(f_gz))["elapsed"]
w_rds <- system.time(readRDS(f_rds))["elapsed"]

round(c(csv = w_csv, csv_gz = w_gz, rds = w_rds), 4)
```

```
   csv csv_gz   rds
0.034  0.041 0.008
```

```
# tipe kolom setelah dibaca
```

```
cat("\ntipe dari CSV:", paste(sapply(read.csv(f_csv), class), collapse = ", "), "\n")
```

```
tipe dari CSV: character, numeric, numeric
```

```
cat("tipe dari RDS:", paste(sapply(readRDS(f_rds), class), collapse = ", "), "\n")
```

```
tipe dari RDS: character, numeric, numeric
```

Penjelasan Kode. Ketiga berkas dibaca dan lamanya diukur. Setelah itu tipe kolom hasil pembacaan dibandingkan antara format teks dan format biner.

Interpretasi Output

Selain lebih cepat, format biner mempertahankan tipe kolom sebagaimana adanya. Pembacaan CSV harus menebak tipe tiap kolom, dan tebakan itu dapat keliru — misalnya kode wilayah berawalan nol yang berubah menjadi bilangan sehingga nolnya hilang. Kekeliruan semacam ini tidak menimbulkan galat dan baru ketahuan jauh belakangan.

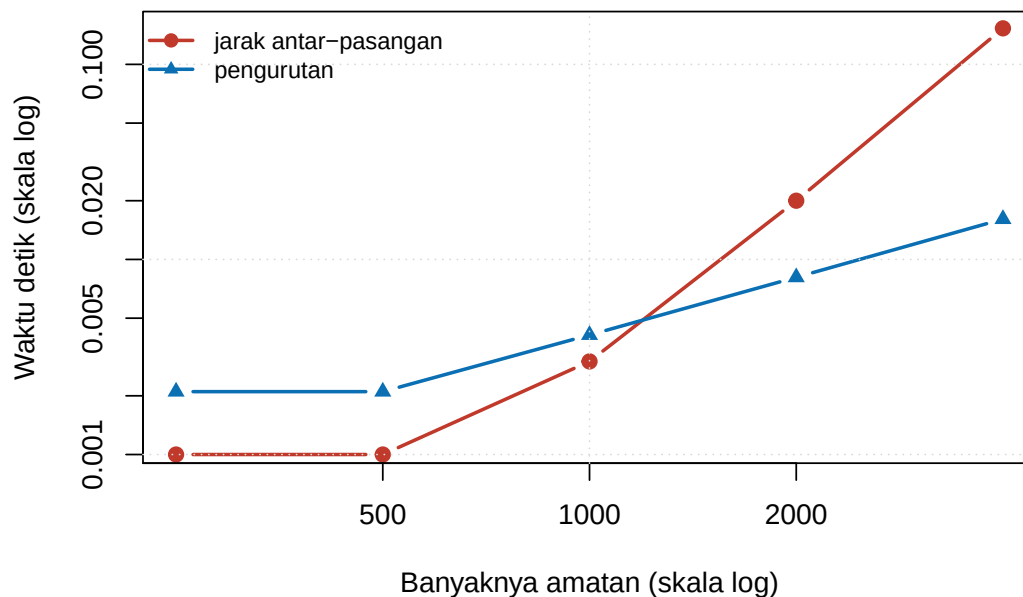
Contoh 2.8 | Menggambarkan pertumbuhan biaya

Kode 2.8 • R grafik log-log pertumbuhan biaya

```
ukuran2 <- c(250, 500, 1000, 2000, 4000)
waktu_jarak <- sapply(ukuran2, function(m) {
  X <- matrix(rnorm(m * 4), m, 4)
  system.time(dist(X))["elapsed"]
})
waktu_urut2 <- sapply(ukuran2, function(m)
  system.time(sort(rnorm(m * 50)))["elapsed"])
```

```
plot(ukuran2, waktu_jarak, type = "b", pch = 19, lwd = 2, col = "#C0392B",
     log = "xy", ylim = range(c(waktu_jarak, waktu_urut2) + 1e-4),
     xlab = "Banyaknya amatan (skala log)",
     ylab = "Waktu detik (skala log)",
     main = "Pertumbuhan Biaya Dua Jenis Operasi")
lines(ukuran2, waktu_urut2 + 1e-4, type = "b", pch = 17, lwd = 2, col = "#0B6FB4")
grid(col = "grey85")
legend("topleft", c("jarak antar-pasangan", "pengurutan"),
      col = c("#C0392B", "#0B6FB4"), pch = c(19, 17), lwd = 2, bty = "n", cex = .85)
```

Pertumbuhan Biaya Dua Jenis Operasi



Penjelasan Kode. Dua operasi diukur pada lima ukuran data, lalu digambar pada skala logaritma ganda. Nilai kecil ditambah satu per sepuluh ribu agar tidak bernilai nol yang mustahil digambar pada skala logaritma. Legenda ditambahkan tanpa bingkai.

Interpretasi Output

Pada skala logaritma ganda, biaya berorde pangkat tampak sebagai garis lurus yang kemiringannya sama dengan pangkatnya. Garis jarak antar-pasangan seharusnya lebih curam daripada garis pengurutan, sebab biayanya kuadrat sedangkan pengurutan mendekati linear. Perhatikan bahwa pengurutan pada data kecil sangat berderau, sehingga garisnya belum tentu rapi — dan itu sendiri merupakan pelajaran mengenai batas keandalan pengukuran waktu.

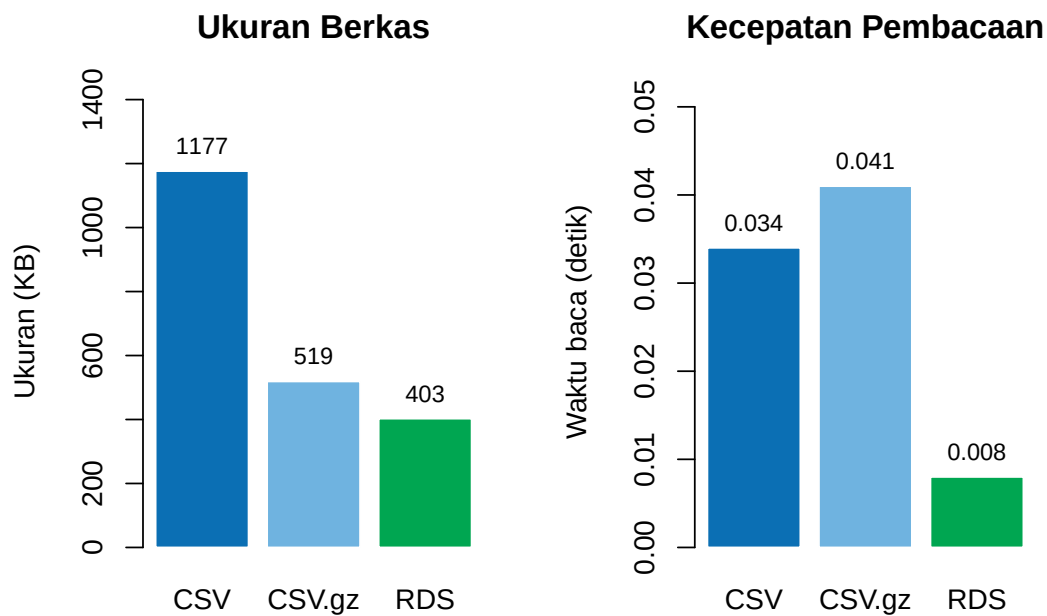
Contoh 2.9 | Menggambarkan perbandingan format penyimpanan

Kode 2.9 • R barplot berlabel nilai

```
nama <- c("CSV", "CSV.gz", "RDS")
kb <- c(file.size(f_csv), file.size(f_gz), file.size(f_rds)) / 1024
wk <- c(w_csv, w_gz, w_rds)

par(mfrow = c(1, 2))
bp <- barplot(kb, names.arg = nama, col = c("#0B6FB4", "#6FB3E0", "#00A651"),
              border = "white", ylab = "Ukuran (KB)",
              main = "Ukuran Berkas", ylim = c(0, max(kb) * 1.2))
text(bp, kb, round(kb, 0), pos = 3, cex = .8)

bp2 <- barplot(wk, names.arg = nama, col = c("#0B6FB4", "#6FB3E0", "#00A651"),
               border = "white", ylab = "Waktu baca (detik)",
               main = "Kecepatan Pembacaan", ylim = c(0, max(wk) * 1.25))
text(bp2, wk, round(wk, 3), pos = 3, cex = .8)
```



```
par(mfrow = c(1, 1))
```

Penjelasan Kode. Dua diagram batang disajikan berdampingan, masing-masing untuk ukuran berkas dan waktu pembacaan. Fungsi barplot mengembalikan koordinat tengah tiap batang, yang dipakai text untuk menempatkan label nilai tepat di atasnya. Batas sumbu dilebihkan agar label tidak terpotong.

Interpretasi Output

Menyertakan nilai di atas batang membuat gambar dapat dibaca tanpa menaksir dari sumbu, dan itu praktik yang dianjurkan untuk grafik pelaporan. Perhatikan bahwa kedua gambar menceritakan hal berbeda: format termampatkan unggul pada ukuran tetapi tidak selalu pada kecepatan, sebab pemampatan menuntut waktu pembongkaran. Format biner unggul pada keduanya sekaligus.

Contoh 2.10 | Memantau penggunaan memori

Kode 2.10 • R gc untuk memantau memori

```
gc(reset = TRUE)

      used (Mb) gc trigger (Mb) max used (Mb)
Ncells 973363   52   1678057 89.7   973363   52
Vcells 2091421  16   11366023 86.8   2091421  16

awal <- sum(gc()[, 2])

besar2 <- data.frame(a = rnorm(2e5), b = rnorm(2e5), c = rnorm(2e5))
setelah_buat <- sum(gc()[, 2])

salinan <- besar2          # penyalinan malas: belum menambah memori
setelah_salin <- sum(gc()[, 2])

salinan$a <- salinan$a * 2 # penyuntingan memicu penyalinan sungguhan
setelah_ubah <- sum(gc()[, 2])

round(c(awal = awal, setelah_buat = setelah_buat,
        setelah_salin = setelah_salin, setelah_ubah = setelah_ubah), 1)

      awal  setelah_buat  setelah_salin  setelah_ubah
      68.0         72.6         72.6         74.1
```

Penjelasan Kode. Fungsi gc memicu pemungutan sampah sekaligus melaporkan pemakaian memori dalam megabita. Pemakaian diukur pada empat titik: sebelum objek dibuat, sesudahnya, sesudah objek disalin, dan sesudah salinannya disunting.

Interpretasi Output

Perhatikan bahwa penyalinan objek tidak langsung menambah memori, sebab R memakai penyalinan saat disunting. Memori baru terpakai ketika salinannya benar-benar diubah. Sifat ini menjelaskan mengapa pemakaian memori R dapat melonjak mendadak di tengah pengolahan: satu penyuntingan kecil dapat memicu penyalinan seluruh objek. Inilah salah satu alasan `data.table` dirancang menyunting di tempat.

Contoh 2.11 | Menyimpan hasil bertahap ke cakram**Kode 2.11 • R membaca, mengolah, dan menulis per potongan**

```
keluaran <- tempfile(fileext = ".csv")
n_potong <- 1000
lewati <- 1; pertama <- TRUE; total <- 0

repeat {
  potong <- tryCatch(
    read.csv("data/data_transaksi.csv", skip = lewat,
             nrow = n_potong, header = FALSE),
    error = function(e) NULL)
  if (is.null(potong) || nrow(potong) == 0) break

  # olah potongan: saring transaksi bernilai besar
  olahan <- potong[potong[[4]] > 1000, ]
  if (nrow(olahan) > 0) {
    write.table(olahan, keluaran, sep = ",", row.names = FALSE,
               col.names = pertama, append = !pertama)
    pertama <- FALSE
    total <- total + nrow(olahan)
  }
  lewat <- lewat + nrow(potong)
}

cat("baris tersaring:", total, "| ukuran keluaran:",
    round(file.size(keluaran) / 1024, 1), "KB\n")
```

```
baris tersaring: 2999 | ukuran keluaran: 122.2 KB
```

Penjelasan Kode. Berkas dibaca per potongan, tiap potongan disaring, lalu hasilnya langsung ditulis ke berkas keluaran memakai `append`. Argumen `col.names` hanya bernilai benar pada potongan pertama agar kepala kolom tidak terulang.

Interpretasi Output

Pola baca-olah-tulis inilah bentuk paling sederhana pengolahan di luar memori, dan gagasannya sama dengan yang dipakai kerangka terdistribusi. Perhatikan bahwa hanya satu potongan berada di memori pada satu waktu, sehingga berkas seberapa pun besarnya dapat diolah. Yang tidak dapat dikerjakan dengan pola ini adalah operasi yang menuntut seluruh data hadir sekaligus, misalnya pengurutan menyeluruh atau perhitungan median tepat.

Contoh 2.12 | Mengukur pertumbuhan biaya secara empiris**Kode 2.12 • R** mengukur waktu pada beberapa ukuran

```
suppressPackageStartupMessages(library(data.table))
set.seed(2026)
besar <- data.table(g = sample(letters[1:8], 5e4, TRUE), x = rlnorm(5e4, 5, 1))

ukuran <- c(2000, 10000, 25000, 50000)
waktu_ringkas <- sapply(ukuran, function(m)
  system.time(besar[1:m][, list(r = mean(x)), by = g])[["elapsed"]])
waktu_urut <- sapply(ukuran, function(m)
  system.time(sort(besar$x[1:m]))[["elapsed"]])

hasil <- data.frame(ukuran, peringkasan = waktu_ringkas, pengurutan = waktu_urut)
print(hasil, right = FALSE)
```

	ukuran	peringkasan	pengurutan
1	2000	0.001	0.000
2	10000	0.001	0.001
3	25000	0.002	0.002
4	50000	0.003	0.002

Penjelasan Kode. Data lima puluh ribu baris dibentuk, lalu dua operasi diukur pada empat ukuran potongan yang berbeda. Peringkasan per kelompok berbiaya linear, sedangkan pengurutan berbiaya $n \log n$.

Interpretasi Output

Bandingkan pertumbuhannya terhadap pertumbuhan ukuran data. Pada ukuran sekecil ini selisihnya tenggelam oleh derau pengukuran, sehingga angkanya belum memperlihatkan orde dengan jelas. Justru itulah pelajarannya: menyimpulkan orde pertumbuhan dari pengukuran pada data kecil tidak dapat dipercaya. Untuk menaksir kelayakan pada data besar, orde teoretis lebih dapat diandalkan daripada pengukuran pada data kecil.

Contoh 2.13 | Membuktikan biaya kuadrat pada perhitungan jarak**Kode 2.13 • R** dist dan pertumbuhan banyaknya pasangan

```
ukuran <- c(200, 400, 800, 1600)
waktu <- sapply(ukuran, function(m) {
  X <- matrix(rnorm(m * 5), m, 5)
  system.time(dist(X))["elapsed"]}
})
pasangan <- ukuran * (ukuran - 1) / 2
print(data.frame(ukuran, pasangan, waktu), right = FALSE)
```

	ukuran	pasangan	waktu
1	200	19900	0.000
2	400	79800	0.001
3	800	319600	0.004
4	1600	1279200	0.012

```
cat("\nrasio pasangan saat ukuran digandakan:",
    paste(round(pasangan[-1] / pasangan[-length(pasangan)], 2), collapse = ", "), "\n")
```

```
rasio pasangan saat ukuran digandakan: 4.01, 4.01, 4
```

Penjelasan Kode. Matriks jarak dihitung pada empat ukuran yang masing-masing dua kali sebelumnya. Banyaknya pasangan dihitung dengan rumus kombinasi dua dari n, lalu rasio pertumbuhannya ditampilkan.

Interpretasi Output

Rasio pasangan mendekati empat setiap kali ukuran digandakan — inilah wujud biaya kuadrat. Pada seratus ribu amatan, banyaknya pasangan mencapai sekitar lima miliar, yang sudah tidak muat di memori apa pun. Karena itu pengelompokan hierarkis dan KNN memerlukan siasat khusus pada data besar, misalnya penarikan contoh, pengindeksan spasial, atau penghampiran tetangga terdekat.

Contoh 2.14 | Menghampiri kuantil dengan penarikan contoh**Kode 2.14 • R** membandingkan kuantil hampiran dan tepat

```
set.seed(2026)
populasi <- besar$x

rasio <- c(0.01, 0.05, 0.10, 0.25)
hampiran <- sapply(rasio, function(f) {
```

```
s <- sample(populasi, length(populasi) * f)
c(median = median(s), q90 = quantile(s, 0.9))
})
colnames(hampiran) <- paste0(rasio * 100, " persen")

tepat <- c(median = median(populasi), q90 = quantile(populasi, 0.9))
print(round(cbind(hampiran, tepat = tepat), 2))
```

	1 persen	5 persen	10 persen	25 persen	tepat
median	164.9	149.4	145.1	149.7	147.9
q90.90%	539.8	551.3	539.9	543.3	535.9

```
cat("\ngalat relatif median (persen):",
    paste(round(100 * abs(hampiran[1, ] - tepat[1]) / tepat[1], 2), collapse = ", "), "\n")
```

```
galat relatif median (persen): 11.44, 0.98, 1.95, 1.19
```

Penjelasan Kode. Median dan kuantil ke-90 dihitung dari empat ukuran contoh yang berbeda, lalu dibandingkan terhadap nilai yang dihitung dari seluruh data. Galat relatifnya dinyatakan dalam persen.

Interpretasi Output

Perhatikan bahwa contoh lima persen sudah menghasilkan median yang galatnya kecil, dan menambah contoh menjadi dua puluh lima persen hanya memperbaikinya sedikit. Inilah dasar pembenaran penarikan contoh pada data besar: ketelitian bertambah menurut akar ukuran contoh, sehingga usaha yang berlipat hanya memberi perbaikan yang mengecil. Yang wajib dilakukan adalah menyatakan bahwa nilainya hampiran beserta ukuran contoh yang dipakai.

Contoh 2.15 | Penarikan contoh pada aliran data

Kode 2.15 • R reservoir sampling

```
# reservoir sampling: menarik contoh berukuran tetap dari aliran
reservoir <- function(aliran, k) {
  hasil <- aliran[1:k]
  for (i in (k + 1):length(aliran)) {
    j <- sample.int(i, 1)
    if (j <= k) hasil[j] <- aliran[i]
  }
  hasil
}

set.seed(2026)
```

```

aliran <- besar$x[1:10000]
contoh <- reservoir(aliran, 500)

round(c(n_contoh = length(contoh),
      rata_contoh = mean(contoh),
      rata_populasi = mean(aliran),
      median_contoh = median(contoh),
      median_populasi = median(aliran)), 2)

```

n_contoh	rata_contoh	rata_populasi	median_contoh	median_populasi
500.0	235.9	243.9	155.4	149.0

Penjelasan Kode. Algoritma reservoir menarik contoh berukuran tetap dari aliran yang panjangnya tidak diketahui di muka. Amatan ke- i menggantikan salah satu isi reservoir dengan peluang k dibagi i , sehingga setiap amatan berpeluang sama terpilih.

Interpretasi Output

Keunggulannya adalah hanya memerlukan satu kali penelusuran dan memori sebesar k saja, sehingga cocok bagi data yang datang mengalir dan tak dapat disimpan seluruhnya. Bandingkan rata-rata dan median contoh terhadap populasinya: keduanya berdekatan, sebagaimana diharapkan dari contoh acak. Inilah wujud penanganan dimensi velocity yang dibahas pada Teori.

Contoh 2.16 | Menaksir kebutuhan sumber daya

Kode 2.16 • R tabel taksiran kebutuhan

```

memori_tersedia_MB <- 4096          # sesuaikan dengan mesin Anda
per_baris <- as.numeric(object.size(trx)) / nrow(trx)

skala <- c(`1 juta` = 1e6, `10 juta` = 1e7, `100 juta` = 1e8)
taksir <- data.frame(
  baris = names(skala),
  perkiraan_MB = round(per_baris * skala / 1024^2, 1),
  muat = ifelse(per_baris * skala / 1024^2 < memori_tersedia_MB * 0.5,
    "muat di memori", "perlu bertahap/terdistribusi")
)
print(taksir, right = FALSE)

```

	baris	perkiraan_MB	muat
1 juta	1 juta	23.4	muat di memori
10 juta	10 juta	233.6	muat di memori
100 juta	100 juta	2335.5	perlu bertahap/terdistribusi

Penjelasan Kode. Jejak memori per baris dikalikan tiga tingkat ukuran data untuk memperkirakan kebutuhan memorinya. Kolom keputusan dibentuk dengan ifelse memakai ambang setengah dari memori tersedia, karena pengolahan memerlukan ruang tambahan di luar penyimpanan data itu sendiri.

Interpretasi Output

Ambang setengah memori bukan angka keramat, melainkan aturan praktis: banyak operasi R menyalin data sehingga kebutuhan sesaat dapat berlipat. Tabel seperti ini sebaiknya disusun pada tahap perencanaan, sebelum kode ditulis, agar pendekatan yang dipilih sejak awal memang dapat dijalankan.

2.7 Latihan Mandiri

Kerjakan bertahap. Gunakan data/data_transaksi.csv. Bagian Spark dikerjakan pada bagian Tambahan setelah Penugasan.

2.7.0.0.1 Tingkat Dasar (melengkapi kode)

Latihan 2.1 | Mengukur ukuran objek

```
trx <- read.csv("data/data_transaksi.csv", stringsAsFactors = TRUE)
format(__(trx), units = "KB")          # jejak memori objek
per_baris <- as.numeric(object.size(trx)) / __(trx) # bagi banyaknya baris
```

Latihan 2.2 | Ringkasan dengan data.table

```
library(data.table)
trx_dt <- __(trx)
trx_dt[, list(n = __, rata = mean(nilai_transaksi)), by = provinsi]
```

Latihan 2.3 | Pemeriksaan mutu

```
sum(__(trx$id_transaksi))          # cacah id ganda
q <- __(trx$nilai_transaksi, c(.25, .75)) # kuartil
batas <- q[2] + 1.5 * __(q)        # jarak antar-kuartil
```

Latihan 2.4 | Format penyimpanan

```
f1 <- tempfile(fileext = ".csv"); f2 <- tempfile(fileext = ".rds")
write.csv(trx, f1, row.names = FALSE)
__(trx, f2)          # simpan sebagai biner
round(c(csv = __(f1), rds = file.size(f2)) / 1024, 1) # ukuran dalam KB
```

Latihan 2.5 | Memantau memori

```
__(reset = TRUE)          # pungut sampah dan setel ulang pencatatan
sebelum <- sum(gc())[, 2])
besar <- data.frame(a = rnorm(1e5))
sesudah <- sum(__())[, 2])
```

```
cat("tambahan memori:", round(sesudah - sebelum, 1), "MB
")
```

2.7.0.0.2 Tingkat Menengah (menulis sendiri)

Latihan 2.6 | Pengolahan bertahap

Tulis kode yang menghitung **ragam** nilai_transaksi secara bertahap per potongan seribu baris, tanpa memuat seluruh berkas sekaligus. Petunjuk: kumpulkan jumlah, jumlah kuadrat, dan cacah, lalu susun ragamnya di akhir. Buktikan hasilnya sama dengan `var()` atas data utuh hingga ketelitian enam angka.

Latihan 2.7 | Membetulkan kode yang salah

Kode berikut memuat empat kesalahan (satuan `object.size` keliru, pembagi bukan banyaknya baris, margin tabel salah, dan batas pencilan memakai kuartil pertama). Temukan dan perbaiki, lalu jelaskan tiap kesalahannya.

```
trx <- read.csv("data/data_transaksi.csv", stringsAsFactors = TRUE)

# (a) satuan keliru: hasil dibaca sebagai MB padahal byte
ukuran <- as.numeric(object.size(trx))
cat("ukuran:", ukuran, "MB
")

# (b) dibagi banyaknya KOLOM, bukan baris
per_baris <- ukuran / ncol(trx)

# (c) ingin proporsi kanal PER PROVINSI, tetapi margin salah
prop.table(table(trx$provinsi, trx$kanal), margin = 2)

# (d) batas pencilan atas memakai kuartil PERTAMA
q <- quantile(trx$nilai_transaksi, c(.25, .75))
batas <- q[1] + 1.5 * diff(q)
```

Keluaran yang benar. Setelah diperbaiki, kode yang dipakai adalah sebagai berikut.

```
# (a) byte diubah ke MB dengan membagi 1024^2
ukuran <- as.numeric(object.size(trx))
cat("ukuran:", round(ukuran / 1024^2, 3), "MB
")

# (b) jejak per baris dibagi banyaknya BARIS
per_baris <- ukuran / nrow(trx)
```



```
# (c) margin = 1 agar proporsi dihitung per baris (per provinsi)
prop.table(table(trx$provinsi, trx$kanal), margin = 1)
```

```
# (d) batas atas memakai kuartil KETIGA
q <- quantile(trx$nilai_transaksi, c(.25, .75))
batas <- q[2] + 1.5 * diff(q)
```

Pemeriksaan yang wajib berhasil: `all(round(rowSums(prop.table(table(trx$provinsi, trx$kanal), margin = 1)), 8) == 1)` bernilai **TRUE**, dan `batas > q[2]` bernilai **TRUE**.

Latihan 2.8 | Menakar orde pertumbuhan

Ukur waktu `dist()` pada empat ukuran data yang masing-masing dua kali sebelumnya. Hitung rasio waktu antar-ukuran berturutan, lalu bandingkan terhadap rasio teoretis biaya kuadrat. Jelaskan mengapa rasio terukur dapat menyimpang dari rasio teoretis.

Latihan 2.9 | Menghampiri dengan contoh

Hitung median dan kuartil ketiga `nilai_transaksi` dari contoh berukuran 1, 5, 10, dan 25 persen. Sajikan galat relatifnya terhadap nilai tepat, lalu tentukan ukuran contoh terkecil yang menurut Anda memadai beserta alasannya.

2.7.0.0.3 Tingkat Lanjut (perencanaan)

Latihan 2.10 | Menyusun rencana sumber daya

Andaikan data transaksi ini bertambah menjadi 50 juta baris dan datang setiap hari. Susun rencana ringkas yang memuat: (a) perkiraan kebutuhan memori dan cakram; (b) pendekatan yang Anda pilih beserta alasannya; (c) besaran mana yang mudah dihitung secara terdistribusi dan mana yang sulit; (d) satu risiko mutu data yang membesar seiring bertambahnya volume.

2.8 Penugasan

Sebagai calon analis pada instansi statistik, susun laporan kelayakan pengolahan atas data transaksi tiruan `data_transaksi.csv` (*data simulasi*), lengkap dengan rekomendasi pendekatan bila datanya membesar.

Konteks. Sebuah unit menerima catatan transaksi harian. Saat ini berukuran 5.000 baris, tetapi diperkirakan tumbuh cepat. Pimpinan meminta penilaian apakah infrastruktur yang ada masih memadai.

Tugas 2.1 | Penilaian Data

1. Lakukan penjelajahan menyeluruh dan ukur jejak memorinya. Susun tabel taksiran kebutuhan untuk 1 juta, 10 juta, dan 100 juta baris.
2. Periksa **kelima dimensi** big data pada data ini. Untuk tiap dimensi, sebutkan apakah data ini memenuhinya, disertai bukti dari perhitungan Anda.
3. Periksa dimensi **veracity** secara konkret: duplikasi, nilai di luar rentang wajar, dan pencilan. Laporkan temuannya beserta pertimbangan apakah pencilan perlu dibuang.

Tugas 2.2 | Perbandingan Pendekatan

4. Bandingkan **tiga pendekatan** peringkasan (basis R, `dplyr`, `data.table`) dari sisi waktu. Ulangi pada data yang digandakan sepuluh kali dengan `rbind`, lalu bahas apakah urutan kecepatannya berubah.
5. Terapkan **pengolahan bertahap** untuk menghitung rata-rata dan ragam tanpa memuat seluruh berkas. Buktikan hasilnya sama dengan perhitungan sekaligus.

Tugas 2.3 | Penyimpanan

6. Simpan data transaksi dalam **tiga format** (CSV, CSV termampatkan, biner). Bandingkan ukuran berkas dan waktu pembacaannya. Sajikan sebagai tabel beserta rasio terhadap CSV.
7. Periksa apakah **tipe kolom terjaga** setelah dibaca dari masing-masing format. Jelaskan satu risiko nyata bila tipe kolom salah ditebak dari CSV.
8. Terapkan pola **baca-olah-tulis per potongan**: saring transaksi bernilai di atas 1.000 lalu tulis hasilnya ke berkas baru tanpa memuat seluruh data. Laporkan banyaknya baris tersaring dan buktikan hasilnya sama dengan penyaringan sekaligus.

Tugas 2.4 | Skalabilitas

6. Ukur waktu **peringkasan** dan **perhitungan jarak** pada empat ukuran data berbeda. Sajikan tabel dan grafik, lalu tentukan orde pertumbuhan masing-masing berdasarkan rasio waktunya.
7. Terapkan **reservoir sampling** untuk menarik contoh 500 amatan dari data. Bandingkan rata-rata, median, dan simpangan baku contoh terhadap populasinya, lalu jelaskan mengapa cara ini sesuai bagi data yang mengalir.
8. Bandingkan **kuantil hampiran** dari beberapa ukuran contoh terhadap nilai tepatnya. Tentukan ukuran contoh yang memadai, dan susun kalimat pelaporan yang menyatakan dengan jujur bahwa nilainya hampiran.

Tugas 2.5 | Rekomendasi

6. Susun rekomendasi pendekatan untuk tiga tingkat pertumbuhan data, disertai alasan teknis. Sebutkan pula pada tingkat mana pendekatan terdistribusi mulai layak dipertimbangkan.
7. Susun laporan R Markdown berisi kode, keluaran, tabel, dan interpretasi. Tutup dengan **ringkasan (maksimal 300 kata)** memuat: penilaian kelayakan, rekomendasi, keterbatasan, dan penegasan bahwa data merupakan simulasi.

2.9 Tambahan: Pengolahan Terdistribusi dengan Apache Spark

Bagian Contoh mengerjakan seluruh pengolahan di dalam memori satu mesin. Bagian ini memperkenalkan pendekatan yang berbeda secara mendasar: pekerjaan dibagi ke beberapa partisi dan dijalankan sejajar. Kerjakan setelah Penugasan selesai.

Catatan. Data sengaja dibuat kecil agar berjalan di komputer mana pun tanpa klaster. Yang dilatih di sini adalah **alur kerjanya** — partisi, transformasi tertunda, dan aksi — bukan besarnya data. Gagasan yang sama persis berlaku ketika kelak Anda bekerja pada klaster sungguhan.

Prasyarat. Java 11 atau lebih baru, serta `pip install pyspark pyarrow`. Periksa dengan `java -version` pada terminal sebelum melanjutkan.

Contoh 2.17 | Memulai sesi Spark dan membentuk DataFrame**Kode 2.17 • Python** `SparkSession` dan partisi

```

from pyspark.sql import SparkSession
import pyspark.sql.functions as F
import pandas as pd

spark = (SparkSession.builder
        .appName("modul2")
        .master("local[2]")          # 2 inti pada mesin ini
        .config("spark.driver.memory", "1g")
        .config("spark.ui.enabled", "false")
        .getOrCreate())
spark.sparkContext.setLogLevel("ERROR")

pdf = pd.read_csv("data/data_transaksi.csv")
sdf = spark.createDataFrame(pdf)
print("baris:", sdf.count(), "| kolom:", len(sdf.columns))
print("partisi:", sdf.rdd.getNumPartitions())
sdf.printSchema()

```

Penjelasan Kode. Sesi Spark dibuat dengan mode local dua inti, sehingga pekerjaan dibagi ke dua partisi pada satu mesin. Argumen driver memory membatasi memori, dan ui enabled bernilai false mematikan antarmuka web yang tidak diperlukan. Data dibaca lebih dahulu dengan pandas lalu diubah menjadi DataFrame Spark. Perintah `getNumPartitions` memperlihatkan berapa bagian data dipecah, dan `printSchema` menampilkan tipe tiap kolom sebagaimana dikenali Spark.

Interpretasi Output

Perhatikan bahwa banyaknya partisi menentukan tingkat kesejajaran: dua partisi berarti dua bagian dapat dikerjakan bersamaan. Pada kluster sungguhan, partisi tersebar ke banyak mesin dan angkanya jauh lebih besar. Skema yang dicetak Spark juga patut diperiksa, sebab Spark menyimpulkan tipe secara mandiri dan kadang berbeda dari dugaan.

Contoh 2.18 | Transformasi tertunda dan aksi

Kode 2.18 • Python filter, groupBy, agg, lalu show

```

# TRANSFORMASI -- belum dijalankan, hanya dicatat sebagai rencana
rencana = (sdf
    .filter(F.col("nilai_transaksi") > 500)
    .groupBy("provinsi", "kanal")
    .agg(F.count("*").alias("n"),
        F.round(F.avg("nilai_transaksi"), 1).alias("rata")))

```

```
print(type(rencana))          # masih berupa DataFrame, belum ada hasil

# AKSI -- memicu seluruh rencana dijalankan
rencana.orderBy("provinsi", "kanal").show()
```

Penjelasan Kode. Rangkaian penyaringan, pengelompokan, dan peringkasan disusun tetapi belum dijalankan; yang terbentuk baru rencana kerja. Baru ketika show dipanggil, seluruh rangkaian dieksekusi. Fungsi F.count dengan tanda bintang mencacah baris, sedangkan alias memberi nama pada kolom hasil.

Interpretasi Output

Inilah wujud penundaan eksekusi. Selama hanya transformasi yang dipanggil, tidak ada perhitungan yang berjalan dan tidak ada waktu terbuang. Sifat ini menjelaskan mengapa merangkai banyak langkah pada Spark tidak semahal merangkai banyak langkah pada pengolahan biasa.

Contoh 2.19 | Membaca rencana eksekusi

Kode 2.19 • Python explain

```
rencana.explain(mode="simple")
```

Penjelasan Kode. Perintah explain menampilkan rencana fisik yang disusun Spark, dibaca dari bawah ke atas: pemindaian data, penyaringan, peringkasan sebagian pada tiap partisi, pertukaran data antar-partisi, lalu peringkasan akhir.

Interpretasi Output

Dua hal patut diperhatikan. Pertama, penyaringan muncul dekat dasar rencana, artinya Spark mendahulukannya agar data yang diproses lebih sedikit. Kedua, adanya Exchange menandakan pertukaran data antar-partisi, yang merupakan operasi termahal pada pengolahan terdistribusi. Mengurangi pertukaran adalah inti pengoptimuman kerja Spark.

Contoh 2.20 | Menutup sesi

Kode 2.20 • Python spark.stop

```
spark.stop()
print("Sesi Spark ditutup.")
```

Penjelasan Kode. Sesi Spark menahan sumber daya selama masih hidup, sehingga wajib ditutup setelah pekerjaan selesai.

Interpretasi Output

Lupa menutup sesi adalah kekeliruan yang lazim dan berakibat memori tetap terpakai. Pada lingkungan bersama, sesi yang tidak ditutup dapat menghalangi pengguna lain memperoleh sumber daya.

Spark bukan selalu lebih cepat. Pada data berukuran kecil, Spark justru **lebih lambat** daripada pengolahan biasa, karena biaya penyiapan sesi, pembagian partisi, dan pertukaran data melebihi manfaat keseajarannya. Spark baru unggul ketika data tidak lagi muat di memori satu mesin. Memakai Spark untuk data lima ribu baris adalah pilihan yang keliru — dan modul ini memakainya semata untuk melatih alur kerja.

2.10 Ringkasan Modul

Daftar Periksa penguasaan:

- ☐ Saya dapat menyebutkan lima dimensi big data beserta akibat teknisnya.
- ☐ Saya memahami bahwa besar tidak sama dengan berguna, dan ukuran tidak menggantikan keterwakilan.
- ☐ Saya dapat mengukur jejak memori data dan menaksir kebutuhan pada skala lebih besar.
- ☐ Saya dapat mengolah data secara bertahap tanpa memuatnya sekaligus.
- ☐ Saya dapat membedakan besaran yang mudah diparalelkan dari yang sulit.
- ☐ Saya memahami perbedaan transformasi dan aksi pada Spark.
- ☐ Saya dapat membaca rencana eksekusi dan mengenali pertukaran antar-partisi.
- ☐ Saya menyadari bahwa Spark tidak selalu lebih cepat.

Kartu Referensi Cepat.

Keperluan	Perintah
nJejak memori objek	<code>object.size(d)</code>
nRingkasan data.table	<code>dt[, list(r = mean(x)), by = g]</code>
nCacah per kelompok	<code>dt[, list(n = .N), by = g]</code>
nBaca per potongan	<code>read.csv(f, skip =, nrows =)</code>
nMengukur waktu	<code>system.time(ekspresi)</code>
nSesi Spark	<code>SparkSession.builder...getOrCreate()</code>
nCacah partisi	<code>sdf.rdd.getNumPartitions()</code>
nRingkasan Spark	<code>groupBy().agg()</code>
nRencana eksekusi	<code>df.explain()</code>
nMenutup sesi	<code>spark.stop()</code>

Modul ini menjawab pertanyaan apakah data yang tersedia dapat diolah dengan sumber daya yang ada, dan apa jalan keluarnya bila tidak. Prinsip yang perlu dibawa: ukur lebih dahulu sebelum mengoptimumkan, pilih pendekatan sesuai kendala nyata bukan sesuai tren, dan ingat bahwa data besar tanpa keterwakilan tetap tidak menggantikan survei terancang. Modul berikutnya membahas ragam tipe data beserta perlakuan yang sesuai bagi masing-masing.

2.11 Referensi

- Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann.
- Marr, B. (2015). *Big Data: Using SMART Big Data, Analytics and Metrics*. Wiley.
- Chambers, B., & Zaharia, M. (2018). *Spark: The Definitive Guide*. O'Reilly Media.
- Dowle, M., & Srinivasan, A. (2023). *data.table: Extension of data.frame*. R package documentation.
- UNECE (2013). *What Does Big Data Mean for Official Statistics?* United Nations Economic Commission for Europe.

MODUL 3

TIPE DATA PADA BIG DATA

Pertemuan Minggu ke-3

Sub-CPMK 3: Mahasiswa mampu mengenali ragam tipe data pada big data serta menentukan perlakuan yang sesuai bagi data berdimensi tinggi, data jaringan, dan data teks.

Penyusun: Achmad Syahrul Choir



Daftar Isi

3.1	Deskripsi Modul	66
3.2	Tujuan Praktikum	66
3.3	Alat dan Bahan	66
3.4	Instruksi Pengerjaan	67
3.5	Teori Singkat	67
3.5.0.1	Kedudukan pada CRISP-DM	67
3.5.0.2	Ragam Tipe Data	67
3.5.0.3	Data Runtun Waktu	67
3.5.0.4	Data Spasial	68
3.5.0.5	Data Berdimensi Tinggi dan Kutukan Dimensi	68
3.5.0.6	Data Jaringan	69
3.5.0.7	Data Teks	69
3.6	Contoh Praktikum dengan R	71
3.7	Latihan Mandiri	90
3.8	Penugasan	93
3.9	Ringkasan Modul	94
3.10	Referensi	95

3.1 Deskripsi Modul

Dua modul sebelumnya bekerja pada data tabular, yaitu tabel berbaris amatan dan berkolom peubah. Bentuk itu paling akrab dan paling mudah diolah, tetapi tidak mencakup seluruh data yang dihadapi dalam praktik. Modul ini memperluas cakrawala ke tiga bentuk yang lazim ditemui pada big data.

Ketiganya dipilih bukan karena kebetulan. Data berdimensi tinggi menghadirkan persoalan yang bertentangan dengan intuisi: menambah peubah tidak selalu menambah informasi, dan pada titik tertentu justru merusak kemampuan mengukur kemiripan. Data jaringan menyimpan informasi pada hubungan antar-unit, bukan pada unitnya sendiri. Data teks tidak berbentuk angka sama sekali sehingga memerlukan penerjemahan sebelum dapat dimodelkan.

Materi dibahas dari pengenalan hingga penanganan: kutukan dimensi dan penanggulangannya lewat analisis komponen utama, ukuran sentralitas serta penemuan komunitas pada data jaringan, dan penyusunan matriks dokumen-istilah beserta analisis sentimen sederhana pada data teks.

Tentang data. Berkas data pada modul ini adalah **data simulasi** (`set.seed(2026)`), **bukan data nyata** dan **bukan data resmi** instansi mana pun. Peubah dibangkitkan saling bebas, sehingga hubungan yang lemah adalah hasil yang wajar.

3.2 Tujuan Praktikum

Setelah menyelesaikan praktikum ini secara tuntas, mahasiswa diharapkan mampu:

1. **Mengenal**i ragam tipe data pada big data beserta perlakuan yang sesuai.
2. **Memperagakan** kutukan dimensi dan menjelaskan akibatnya bagi ukuran jarak.
3. **Menerapkan** analisis komponen utama sebagai cara mereduksi dimensi.
4. **Menyusun** graf dari data relasi dan menghitung ukuran kesentralannya.
5. **Menemukan** komunitas pada data jaringan serta menafsirkannya.
6. **Mengubah** data teks menjadi matriks dokumen-istilah.
7. **Menghitung** skor sentimen sederhana berbasis leksikon dan menyebut keterbatasannya.

3.3 Alat dan Bahan

Perangkat dan bahan yang diperlukan:

- **R** (RStudio) dengan paket `igraph` (data graf) dan `cluster` (jarak Gower untuk data campuran).

- Berkas data: `data/data_dimensi.csv` dan `data/data_relasi.csv`.
- Data teks disediakan langsung di dalam kode contoh, sehingga tidak memerlukan berkas terpisah.

3.4 Instruksi Pengerjaan

Ikuti tahapan berikut:

Sebelum praktikum: pasang paket `igraph` dengan `install.packages("igraph")`; siapkan kedua berkas data pada subfolder `data/`.

Selama praktikum: kerjakan Contoh berurutan. Bagian data berdimensi tinggi menuntut perhatian khusus karena hasilnya kerap berlawanan dengan dugaan awal.

Pembagian kerja: Contoh dan Latihan dikerjakan di kelas; **Penugasan dikerjakan di luar sesi.**

Pengumpulan: susun laporan R Markdown, render ke PDF atau HTML, kumpulkan melalui Google Classroom.

3.5 Teori Singkat

3.5.0.1 Kedudukan pada CRISP-DM

Modul ini berada pada tahap Data Understanding. Sebelum memutuskan bagaimana menyiapkan dan memodelkan data, kita perlu mengenali bentuknya lebih dahulu. Salah mengenali bentuk data berakibat salah memilih seluruh perlakuan berikutnya, dan kekeliruan itu sukar diperbaiki di tahap akhir.

3.5.0.2 Ragam Tipe Data

Enam bentuk data dan perlakuannya.

Bentuk	Ciri	Perlakuan khas
Tabular	baris amatan, kolom peubah	pengodean, pembakuan
Dimensi tinggi	banyak kolom, sedikit baris	reduksi dimensi, regularisasi
Jaringan	simpul dan sisi	ukuran sentralitas, komunitas
Teks	untaian kata	tokenisasi, pembobotan istilah
Runtun waktu	terurut menurut waktu	pembedaan, jendela geser
Spasial	berkoordinat	bobot ketetanggaan

3.5.0.3 Data Runtun Waktu

Data runtun waktu berbeda dari data tabular karena urutannya bermakna. Amatan berdekatan lazimnya berkorelasi, sehingga anggapan saling bebas yang mendasari banyak metode menjadi tidak berlaku. Akibat

paling langsungnya adalah pembagian data latih dan uji tidak boleh dilakukan secara acak.

Tiga komponen runtun waktu.

Komponen	Ciri	Penanganan
Kecenderungan	naik atau turun jangka panjang	pembedaan, penghalusan
Musiman	berulang pada periode tetap	peubah boneka musim, pembedaan musiman
Sisaan	sisanya setelah keduanya	dimodelkan atau dianggap derau

Membagi runtun waktu secara acak. Membagi data runtun waktu secara acak membuat amatan masa depan masuk ke data latih, sehingga model seolah dapat meramal padahal ia sekadar menghafal tetangganya. Pembagian yang benar mengikuti urutan waktu: periode awal untuk melatih, periode akhir untuk menguji.

3.5.0.4 Data Spasial

Pada data spasial berlaku hukum pertama geografi: hal yang berdekatan cenderung lebih mirip daripada yang berjauhan. Akibatnya, sebagaimana runtun waktu, anggapan saling bebas gugur — hanya saja kedekatannya bersifat ruang, bukan waktu. Penanganannya memakai bobot ketetanggaan yang menyatakan seberapa dekat tiap pasangan wilayah.

3.5.0.5 Data Berdimensi Tinggi dan Kutukan Dimensi

Data disebut berdimensi tinggi ketika banyaknya peubah mendekati atau melampaui banyaknya amatan. Keadaan ini lazim pada data genomika, data teks, dan data sensor. Persoalannya bukan sekadar perhitungan menjadi lambat, melainkan gagasan kemiripan itu sendiri mulai kehilangan makna.

Sebabnya begini. Pada ruang berdimensi tinggi, isi ruang terpusat di permukaannya, sehingga hampir semua titik berjarak hampir sama satu sama lain. Akibatnya selisih antara tetangga terdekat dan terjauh menyusut. Padahal justru selisih itulah yang dipakai algoritma berbasis jarak seperti KNN dan K-Means untuk membedakan.

Kontras jarak sebagai ukuran kutukan dimensi:

$$\text{kontras} = \frac{d_{\max} - d_{\min}}{d_{\min}}$$

Makin kecil nilainya, makin sukar membedakan dekat dari jauh.

Menambah peubah tidak selalu menambah informasi. Intuisi kita mengatakan makin banyak peubah makin baik, sebab informasi bertambah. Pada dimensi tinggi anggapan itu keliru. Peubah yang tidak

relevan menambah derajau tanpa menambah sinyal, sehingga jarak antar-amatan makin didominasi kebetulan. Inilah alasan reduksi dimensi bukan sekadar penghematan perhitungan, melainkan syarat agar model berbasis jarak tetap bermakna.

Dua jalan keluar yang lazim adalah seleksi peubah, yaitu memilih sebagian peubah asli, dan ekstraksi ciri, yaitu membentuk peubah baru dari gabungan peubah lama. Analisis komponen utama termasuk cara kedua: ia menyusun kombinasi linier peubah asli yang berurutan menurut besarnya keragaman yang diterangkan.

3.5.0.6 Data Jaringan

Pada data jaringan, informasi tidak terletak pada unit melainkan pada hubungan antar-unit. Sebuah jaringan tersusun dari simpul yang mewakili unit dan sisi yang mewakili hubungan. Sisi dapat berarah atau tak berarah, dan dapat berbobot maupun tidak.

Tiga ukuran sentralitas.

Ukuran	Menghitung	Menjawab pertanyaan
Derajat	banyaknya tetangga	siapa yang paling banyak berhubungan?
Kedekatan	rata-rata jarak ke simpul lain	siapa yang paling mudah menjangkau semua?
Keperantaraan	seberapa sering menjadi jalur	siapa yang menjadi penghubung antar-kelompok?

Ketiganya menjawab pertanyaan berbeda dan dapat menunjuk simpul yang berlainan. Simpul berderajat tinggi belum tentu berperan sebagai penghubung; sebaliknya simpul berderajat rendah dapat berkeperantaraan tinggi bila ia satu-satunya jembatan antara dua kelompok.

3.5.0.7 Data Teks

Teks tidak berbentuk angka sehingga harus diterjemahkan lebih dahulu. Alurnya lazimnya berupa pemberian, tokenisasi, pembuangan kata henti, lalu penyusunan matriks dokumen-istilah yang barisnya dokumen dan kolomnya istilah.

Dua cara membobot istilah.

Frekuensi	banyaknya kemunculan istilah pada dokumen; sederhana tetapi memiliki istilah umum
TF-IDF	frekuensi dikalikan kebalikan frekuensi dokumen; menaikkan bobot istilah yang khas bagi sedikit dokumen

Keterbatasan sentimen berbasis leksikon. Menghitung sentimen dengan mencacah kata positif dan negatif memang sederhana, tetapi buta terhadap konteks. Kalimat “tidak buruk” dihitung negatif padahal maknanya positif, dan sindiran dihitung positif padahal maksudnya sebaliknya. Karena itu skor leksikon layak dipakai sebagai penjajakan awal, bukan sebagai kesimpulan.

3.6 Contoh Praktikum dengan R

Contoh 3.1 | Memperagakan kutukan dimensi

Kode 3.1 • R kontras jarak menurut dimensi

```
set.seed(2026)
kontras <- function(p, n = 200) {
  X <- matrix(runif(n * p), n, p)
  d <- as.matrix(dist(X))
  d <- d[upper.tri(d)]
  (max(d) - min(d)) / min(d)
}
dimensi <- c(2, 5, 10, 50, 100, 200)
hasil <- sapply(dimensi, kontras)
round(data.frame(dimensi, kontras = hasil), 3)
```

	dimensi	kontras
1	2	1410.383
2	5	13.646
3	10	5.010
4	50	1.031
5	100	0.560
6	200	0.406

Penjelasan Kode. Fungsi buatan membangkitkan matriks acak berukuran n kali p , menghitung seluruh jarak antar-titik, lalu mengambil bagian segitiga atasnya agar tiap pasangan dihitung sekali. Kontras dihitung sebagai selisih jarak terjauh dan terdekat dibagi jarak terdekat. Fungsi `sapply` menerapkannya ke beberapa nilai dimensi.

Interpretasi Output

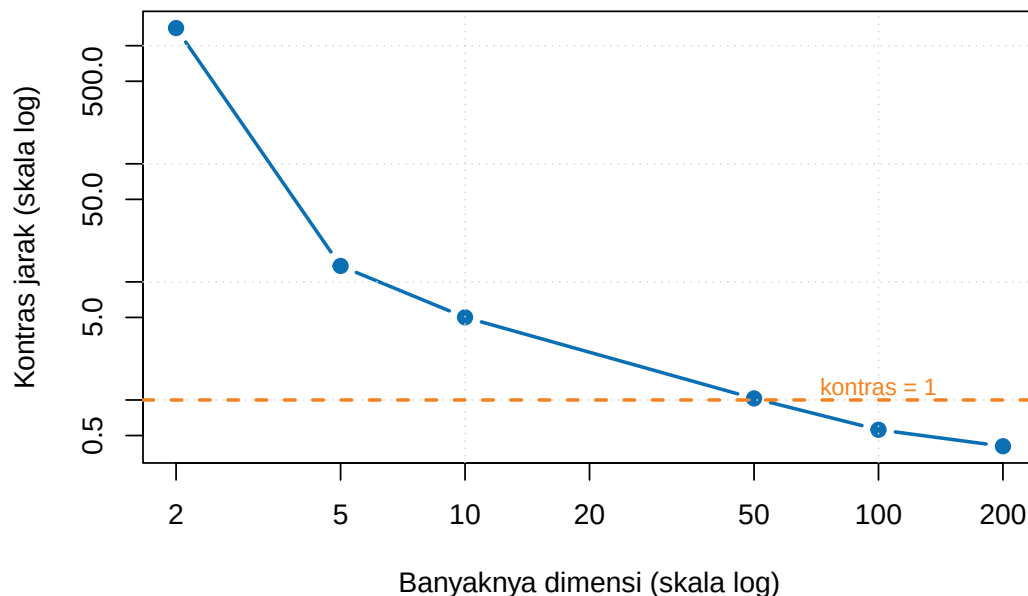
Perhatikan penurunan kontras yang tajam seiring bertambahnya dimensi. Pada dimensi rendah, jarak terjauh berkali lipat jarak terdekat sehingga dekat dan jauh mudah dibedakan. Pada dimensi tinggi, keduanya nyaris sama sehingga gagasan tetangga terdekat kehilangan makna. Inilah kutukan dimensi dalam wujud angka, dan inilah alasan KNN maupun K-Means memburuk pada data berdimensi tinggi.

Contoh 3.2 | Menggambarkan penurunan kontras

Kode 3.2 • R plot dengan skala log ganda

```
plot(dimensi, hasil, type = "b", pch = 19, col = "#0B6FB4", lwd = 2,
     log = "xy",
     xlab = "Banyaknya dimensi (skala log)",
     ylab = "Kontras jarak (skala log)",
     main = "Kontras Jarak Menyusut Seiring Bertambahnya Dimensi")
grid(col = "grey85")
abline(h = 1, col = "#F58220", lty = 2, lwd = 2)
text(x = 100, y = 1.3, "kontras = 1", col = "#F58220", cex = 0.8)
```

Kontras Jarak Menyusut Seiring Bertambahnya Dimensi



Penjelasan Kode. Grafik digambar dengan tipe b sehingga titik dan garis muncul bersamaan. Argumen log bernilai xy memakai skala logaritma pada kedua sumbu, karena penurunannya berlangsung berlipat, bukan menurun tetap. Garis acuan mendatar pada nilai satu ditambahkan sebagai penanda.

Interpretasi Output

Skala logaritma dipilih karena penurunan kontras bersifat berlipat; pada skala biasa seluruh titik dimensi tinggi akan menumpuk mendekati nol dan polanya tak terbaca. Garis acuan pada nilai satu menandai keadaan ketika jarak terjauh hanya dua kali jarak terdekat, yang sudah tergolong sukar dibedakan.

Contoh 3.3 | Mereduksi dimensi dengan analisis komponen utama

Kode 3.3 · R prcomp dan proporsi keragaman

```
dim_data <- read.csv("data/data_dimensi.csv", stringsAsFactors = TRUE)
X <- dim_data[, paste0("x", 1:8)]

pc <- prcomp(X, scale. = TRUE)      # WAJIB dibakukan karena skala dapat berbeda
ragam <- pc$sdev^2 / sum(pc$sdev^2)
round(data.frame(komponen = 1:8,
                 proporsi = ragam,
                 kumulatif = cumsum(ragam)), 4)
```

	komponen	proporsi	kumulatif
1	1	0.1773	0.1773
2	2	0.1569	0.3342
3	3	0.1397	0.4739
4	4	0.1331	0.6070
5	5	0.1221	0.7291
6	6	0.1004	0.8295
7	7	0.0945	0.9241
8	8	0.0759	1.0000

Penjelasan Kode. Delapan kolom numerik diambil, lalu analisis komponen utama dijalankan dengan argumen `scale` bernilai `TRUE` agar peubah dibakukan lebih dahulu. Kuadrat simpangan baku tiap komponen dibagi jumlahnya menghasilkan proporsi keragaman, dan `cumsum` menghasilkan proporsi kumulatifnya.

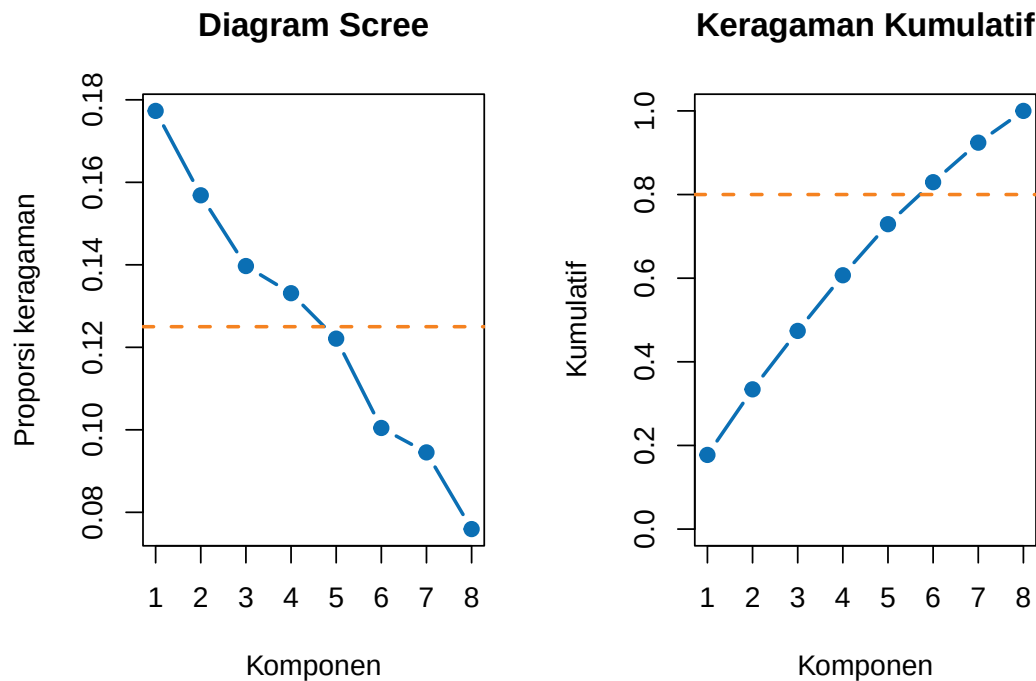
Interpretasi Output

Pembakuan pada analisis komponen utama bukan pilihan melainkan keharusan ketika satuan peubah berbeda, sebab tanpa itu peubah bersatuan besar akan mendominasi komponen pertama semata karena satuannya. Perhatikan proporsi kumulatif: pada data yang peubahnya dibangkitkan saling bebas, keragaman terbagi hampir merata sehingga reduksi dimensi tidak banyak menolong — dan temuan itu sendiri informatif.

Contoh 3.4 | Membaca diagram scree**Kode 3.4 · R** diagram scree dan kumulatif

```
par(mfrow = c(1, 2))
plot(ragam, type = "b", pch = 19, col = "#0B6FB4", lwd = 2,
     xlab = "Komponen", ylab = "Proporsi keragaman", main = "Diagram Scree")
abline(h = 1/ncol(X), col = "#F58220", lty = 2, lwd = 2)
plot(cumsum(ragam), type = "b", pch = 19, col = "#0B6FB4", lwd = 2, ylim = c(0, 1),
```

```
xlab = "Komponen", ylab = "Kumulatif", main = "Keragaman Kumulatif")
abline(h = 0.8, col = "#F58220", lty = 2, lwd = 2)
```



```
par(mfrow = c(1, 1))
```

Penjelasan Kode. Dua grafik ditampilkan berdampingan. Diagram scree memuat proporsi keragaman tiap komponen dengan garis acuan pada satu dibagi banyaknya peubah, yaitu proporsi yang diharapkan bila keragaman terbagi merata. Grafik kedua memuat proporsi kumulatif dengan garis acuan pada 80 persen.

Interpretasi Output

Dua aturan praktis dipakai membaca kedua grafik. Pertama, ambil komponen yang proporsinya di atas garis merata, sebab komponen di bawahnya menerangkan lebih sedikit daripada satu peubah asli rata-rata. Kedua, ambil komponen secukupnya hingga keragaman kumulatif mencapai ambang yang ditetapkan, lazimnya 80 persen. Kedua aturan dapat memberi jawaban berbeda, dan pemilihannya merupakan keputusan yang harus dijelaskan dalam laporan.

Contoh 3.5 | Menyusun graf dari data relasi

Kode 3.5 • R `graph_from_data_frame` dan ringkasannya

```
suppressPackageStartupMessages(library(igraph))
rel <- read.csv("data/data_relasi.csv")
rel <- rel[rel$dari != rel$ke, ]           # buang gelang

g <- graph_from_data_frame(rel[, c("dari", "ke")], directed = FALSE)
g <- simplify(g)                          # buang sisi ganda
cat("simpul:", vcount(g), "| sisi:", ecount(g), "\n")
```

```
simpul: 30 | sisi: 100
```

```
cat("kerapatan:", round(edge_density(g), 4), "\n")
```

```
kerapatan: 0.2299
```

```
cat("komponen terhubung:", components(g)$no, "\n")
```

```
komponen terhubung: 1
```

Penjelasan Kode. Data relasi dibaca lalu baris yang menghubungkan simpul dengan dirinya sendiri dibuang. Graf dibentuk dari dua kolom pertama sebagai pasangan simpul, dengan directed bernilai FALSE karena hubungannya dianggap timbal balik. Fungsi simplify membuang sisi ganda. Kerapatan menyatakan rasio (*ratio*) sisi yang ada terhadap sisi yang mungkin.

Interpretasi Output

Tiga besaran ini adalah pemeriksaan awal wajib pada data jaringan. Kerapatan rendah menandakan jaringan renggang, yang lazim pada jaringan sosial nyata. Banyaknya komponen terhubung menunjukkan apakah jaringan menyatu atau terpecah menjadi kelompok yang saling terpisah — hal yang menentukan apakah ukuran kedekatan dapat dihitung bermakna.

Contoh 3.6 | Menghitung tiga ukuran sentralitas

Kode 3.6 • R degree, closeness, betweenness

```
sentral <- data.frame(
  simpul = V(g)$name,
  derajat = degree(g),
  kedekatan = round(closeness(g), 5),
  keperantaraan = round(betweenness(g), 2))

# tiga teratas menurut masing-masing ukuran
for (u in c("derajat", "kedekatan", "keperantaraan")) {
  atas <- sentral[order(-sentral[[u]]), c("simpul", u)][1:3, ]
```

```
cat("Teratas menurut", u, ":", paste(atas$simpul, collapse = ", "), "\n")
}
```

```
Teratas menurut derajat : 5, 2, 6
```

```
Teratas menurut kedekatan : 5, 2, 13
```

```
Teratas menurut keperantaraan : 5, 7, 2
```

Penjelasan Kode. Ketiga ukuran sentralitas dihitung sekaligus dan disusun sebagai data frame. Perulangan menampilkan tiga simpul teratas menurut masing-masing ukuran, dengan pengurutan menurun memakai tanda negatif pada order.

Interpretasi Output

Bandingkan ketiga daftar teratas. Bila simpul yang muncul berbeda-beda, itu menegaskan bahwa ketiga ukuran menjawab pertanyaan berlainan. Dalam penerapan nyata, simpul berkeperantaraan tinggi sering paling penting secara strategis: menghilangkannya dapat memutus jaringan menjadi beberapa bagian, meski derajatnya biasa saja.

Contoh 3.7 | Menemukan komunitas pada jaringan

Kode 3.7 • R cluster_louvain dan modularitas

```
set.seed(2026)
kom <- cluster_louvain(g)
cat("banyaknya komunitas:", length(sizes(kom)), "\n")
```

```
banyaknya komunitas: 4
```

```
print(sizes(kom))
```

```
Community sizes
```

```
1 2 3 4
```

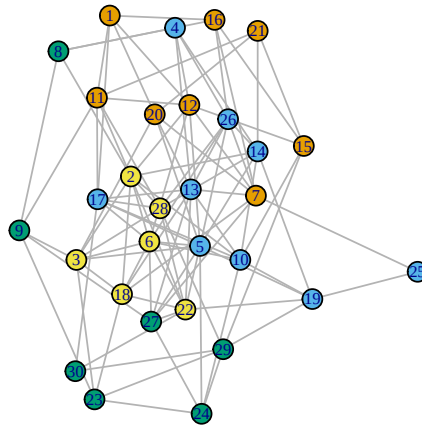
```
8 9 7 6
```

```
cat("modularitas:", round(modularity(kom), 4), "\n")
```

```
modularitas: 0.2286
```

```
plot(g, vertex.color = membership(kom), vertex.size = 10,
     vertex.label.cex = 0.6, edge.color = "grey70",
     main = "Komunitas pada Jaringan")
```

Komunitas pada Jaringan



Penjelasan Kode. Algoritma Louvain mencari pengelompokan simpul yang memaksimalkan modularitas. Fungsi `sizes` menampilkan ukuran tiap komunitas, sedangkan `modularity` mengukur seberapa baik pengelompokannya. Warna simpul pada gambar ditentukan oleh keanggotaan komunitas.

Interpretasi Output

Modularitas berkisar dari negatif setengah sampai satu; nilai di atas 0,3 lazim dianggap menandakan struktur komunitas yang nyata. Pada jaringan yang dibangkitkan acak, modularitasnya cenderung rendah dan komunitas yang ditemukan lebih mencerminkan kebetulan daripada struktur sesungguhnya. Perlu diingat bahwa algoritma penemuan komunitas **selalu** mengembalikan pengelompokan, bahkan pada jaringan tanpa struktur — karena itu modularitas wajib diperiksa.

Contoh 3.8 | Mengubah teks menjadi matriks dokumen-istilah

Kode 3.8 • R tokenisasi dan penyusunan DTM

```
ulasan <- c("layanan sangat baik dan cepat",
            "pelayanan lambat sekali mengecewakan",
            "aplikasi mudah digunakan dan baik",
            "sangat lambat dan sulit digunakan",
```

```

      "cepat mudah dan memuaskan",
      "antrean panjang pelayanan lambat")

henti <- c("dan", "yang", "di", "ke", "dari", "sangat", "sekali")
token <- lapply(strsplit(tolower(ulasan), "[^a-z]+"),
               function(x) setdiff(x[nchar(x) > 2], henti))

kosakata <- sort(unique(unlist(token)))
dtm <- t(sapply(token, function(t) as.integer(kosakata %in% t)))
colnames(dtm) <- kosakata
rownames(dtm) <- paste0("dok", seq_along(ulasan))
cat("DTM:", nrow(dtm), "dokumen x", ncol(dtm), "istilah\n")

```

DTM: 6 dokumen x 13 istilah

```
dtm[1:3, 1:8]
```

	antrean	aplikasi	baik	cepat	digunakan	lambat	layanan	memuaskan
dok1	0	0	1	1	0	0	1	0
dok2	0	0	0	0	0	1	0	0
dok3	0	1	1	0	1	0	0	0

Penjelasan Kode. Teks diubah ke huruf kecil lalu dipecah pada setiap karakter bukan huruf. Potongan berpanjang dua huruf atau kurang dibuang, begitu pula kata henti melalui `setdiff`. Kosakata disusun dari seluruh istilah unik, lalu tiap dokumen diubah menjadi baris berisi nol dan satu menurut ada tidaknya istilah.

Interpretasi Output

Inilah penerjemahan teks menjadi angka. Perhatikan bahwa matriksnya sangat jarang, yaitu didominasi nol — ciri khas data teks yang menjadi alasan penyimpanan jarang dipakai pada data nyata. Perhatikan pula bahwa banyaknya kolom sama dengan besarnya kosakata, sehingga data teks hampir selalu berdimensi tinggi dan tunduk pada kutukan dimensi yang dibahas di awal modul.

Contoh 3.9 | Menafsirkan muatan komponen utama

Kode 3.9 • R muatan komponen dan penafsirannya

```

muatan <- round(pc$rotation[, 1:3], 3)
print(muatan)

```

	PC1	PC2	PC3
x1	0.077	-0.475	-0.317
x2	-0.466	-0.469	0.108

```
x3 -0.133 -0.420 0.585
x4 0.324 -0.166 0.325
x5 -0.436 0.104 0.138
x6 -0.373 -0.344 -0.383
x7 0.378 -0.276 0.349
x8 0.427 -0.382 -0.391
```

```
cat("\nPeubah penyumbang terbesar pada tiap komponen:\n")
```

```
Peubah penyumbang terbesar pada tiap komponen:
```

```
for (k in 1:3) {
  m <- abs(pc$rotation[, k])
  cat(sprintf(" PC%d: %s (muatan %.3f)\n", k,
              names(which.max(m)), pc$rotation[which.max(m), k]))
}
```

```
PC1: x2 (muatan -0.466)
PC2: x1 (muatan -0.475)
PC3: x3 (muatan 0.585)
```

Penjelasan Kode. Matriks muatan menyatakan bobot tiap peubah asli pada tiap komponen. Tiga komponen pertama ditampilkan, lalu peubah bermuatan mutlak terbesar pada masing-masing dicari memakai `which.max` atas nilai mutlakunya.

Interpretasi Output

Nilai mutlak dipakai karena tanda muatan hanya menyatakan arah, bukan besarnya sumbangan; membalik tanda seluruh komponen menghasilkan komponen yang setara. Penafsiran komponen utama menuntut kehati-hatian: komponen yang muatannya tersebar merata pada banyak peubah sukar diberi nama, dan memaksakan nama justru menyesatkan. Pada data yang peubahnya saling bebas, muatan memang cenderung merata — yang berarti komponennya memang tidak bermakna substantif.

Contoh 3.10 | Membandingkan reduksi dimensi terhadap seleksi peubah

Kode 3.10 • R membandingkan dua cara reduksi

```
X <- dim_data[, paste0("x", 1:8)]
Xs <- scale(X)

# cara 1: ekstraksi ciri lewat komponen utama
skor <- pc$x[, 1:3]
```

```
# cara 2: seleksi tiga peubah berragam terbesar
pilih <- names(sort(apply(Xs, 2, var), decreasing = TRUE))[1:3]
terpilih <- Xs[, pilih]

kontras <- function(M) {
  d <- as.matrix(dist(M)); v <- d[upper.tri(d)]
  (max(v) - min(v)) / min(v)
}

round(c(delapan_peubah = kontras(Xs),
        tiga_komponen = kontras(skor),
        tiga_terpilih = kontras(terpilih)), 3)
```

```
delapan_peubah  tiga_komponen  tiga_terpilih
           5.982           46.145           54.603
```

```
cat("peubah terpilih:", paste(pilih, collapse = ", "), "\n")
```

```
peubah terpilih: x1, x5, x7
```

Penjelasan Kode. Dua cara mereduksi dimensi dibandingkan pada data yang sama. Cara pertama memakai skor tiga komponen utama, yaitu peubah baru hasil gabungan. Cara kedua memilih tiga peubah asli berragam terbesar. Kontras jarak dihitung bagi ketiganya.

Interpretasi Output

Kedua cara sama-sama menaikkan kontras dibandingkan memakai seluruh delapan peubah, sebagaimana diharapkan dari pengurangan dimensi. Perbedaan pokoknya terletak pada penafsiran: peubah terpilih tetap bermakna asli sehingga hasil model mudah dijelaskan, sedangkan komponen utama adalah gabungan yang kerap sukar dinamai. Pada penerapan yang menuntut penjelasan kepada pengambil keputusan, seleksi peubah kerap lebih dipilih meski secara teknis kalah efisien.

Contoh 3.11 | Membentuk graf berbobot dan memeriksanya

Kode 3.11 • R graf berbobot dan strength

```
rel_bobot <- rel
names(rel_bobot)[3] <- "weight" # igraph mengenali nama ini
gb <- simplify(graph_from_data_frame(rel_bobot, directed = FALSE),
               edge.attr.comb = list(weight = "sum"))

cat("sisi:", ecount(gb), "| berbobot:", is_weighted(gb), "\n")
```



```
sisi: 100 | berbobot: TRUE
```

```
cat("bobot: min", round(min(E(gb)$weight), 2),
    "| median", round(median(E(gb)$weight), 2),
    "| maks", round(max(E(gb)$weight), 2), "\n")
```

```
bobot: min 1.2 | median 6.75 | maks 18.1
```

```
# sentralitas derajat berbobot memperhitungkan kekuatan hubungan
banding <- data.frame(
  simpul = V(gb)$name,
  derajat = degree(gb),
  kekuatan = round(strength(gb), 1))
print(utils::head(banding[order(-banding$kekuatan), ], 5), row.names = FALSE)
```

```
simpul derajat kekuatan
5      11    104.0
12      8     66.2
17      8     65.8
7       8     61.1
22      9     60.4
```

Penjelasan Kode. Kolom ketiga diberi nama weight agar dikenali igraph sebagai bobot sisi. Ketika sisi ganda disederhanakan, bobotnya dijumlahkan lewat argumen `edge.attr.comb`. Fungsi `strength` menghitung derajat berbobot, yaitu jumlah bobot sisi yang menempel pada tiap simpul.

Interpretasi Output

Bandingkan urutan menurut derajat terhadap urutan menurut kekuatan. Simpul berderajat sedang tetapi berhubungan kuat dapat lebih penting daripada simpul berderajat tinggi dengan hubungan lemah. Mengabaikan bobot berarti membuang informasi, dan pada jaringan nyata — seperti nilai transaksi antar-rekening — bobot justru merupakan informasi utamanya.

Contoh 3.12 | Memeriksa kekokohan jaringan

Kode 3.12 • R delete_vertices dan components

```
# apa yang terjadi bila simpul terpenting dihilangkan?
urut <- order(betweenness(g), decreasing = TRUE)
nama_urut <- V(g)$name[urut]

hasil <- data.frame(dihapus = 0:5, komponen = NA_integer_,
                    ukuran_terbesar = NA_integer_)
```

```

g_kini <- g
for (i in 0:5) {
  komp <- components(g_kini)
  hasil$komponen[i + 1] <- komp$no
  hasil$ukuran_terbesar[i + 1] <- max(komp$size)
  if (i < 5) g_kini <- delete_vertices(g_kini, nama_urut[i + 1])
}
print(hasil, right = FALSE)

```

	dihapus	komponen	ukuran_terbesar
1	0	1	30
2	1	1	29
3	2	1	28
4	3	1	27
5	4	1	26
6	5	1	25

Penjelasan Kode. Simpul diurutkan menurut keperantaraan (*betweenness*) menurun, lalu dihapus satu per satu. Pada tiap tahap, banyaknya komponen terhubung dan ukuran komponen terbesar dicatat. Perulangan menahan penghapusan pada iterasi terakhir agar tabelnya lengkap.

Interpretasi Output

Bertambahnya komponen menandakan jaringan terpecah. Bila menghapus beberapa simpul berkeperantaraan tinggi sudah memecah jaringan, berarti jaringan itu rapuh dan bergantung pada sedikit penghubung. Analisis kekokohan semacam ini dipakai menilai ketahanan jaringan infrastruktur maupun mengenali titik kritis penyebaran informasi.

Contoh 3.13 | Membandingkan tiga ukuran jarak

Kode 3.13 • R dist dan daisy

```

suppressPackageStartupMessages(library(cluster))
set.seed(2026)
campur <- data.frame(
  numerik_kecil = rnorm(60, 0, 1),
  numerik_besar = runif(60, 0, 10000),
  kategori = factor(sample(c("A", "B", "C"), 60, TRUE)))

# Euclidean TANPA pembakuan -- didominasi peubah berskala besar
e_tanpa <- dist(campur[, 1:2])

# Euclidean DENGAN pembakuan

```

```
e_baku <- dist(scale(campur[, 1:2]))
# Gower -- menerima peubah campuran sekaligus
gower <- daisy(campur, metric = "gower")
```

```
round(rbind(
  `Euclidean tanpa baku` = range(e_tanpa),
  `Euclidean dibakukan`   = range(e_baku),
  `Gower (campuran)`      = range(gower)), 3)
```

```
      [,1]      [,2]
Euclidean tanpa baku 0.364 9879.225
Euclidean dibakukan  0.061   5.446
Gower (campuran)     0.006   0.937
```

```
cat("\nkorelasi Euclidean-tanpa-baku vs dibakukan:",
    round(cor(as.vector(e_tanpa), as.vector(e_baku)), 3), "\n")
```

```
korelasi Euclidean-tanpa-baku vs dibakukan: 0.689
```

Penjelasan Kode. Tiga ukuran jarak dihitung atas data yang sama. Dua yang pertama memakai jarak Euclidean dengan dan tanpa pembakuan, hanya atas kolom numerik. Yang ketiga memakai jarak Gower dari paket cluster, yang menerima peubah numerik dan kategori sekaligus dan menghasilkan nilai antara nol dan satu.

Interpretasi Output

Perhatikan rentang jarak Euclidean tanpa pembakuan — besarnya hampir seluruhnya ditentukan peubah berskala ribuan, sedangkan peubah berskala satuan nyaris tak berpengaruh. Korelasi terhadap versi yang dibakukan memperlihatkan seberapa jauh keduanya berbeda. Gower menjadi pilihan ketika data bercampur, sebab ia membakukan tiap peubah menurut jenisnya sendiri. Ukuran jarak inilah yang akan dipakai pada modul pengelompokan.

Contoh 3.14 | Menggambarkan komponen utama sebagai biplot

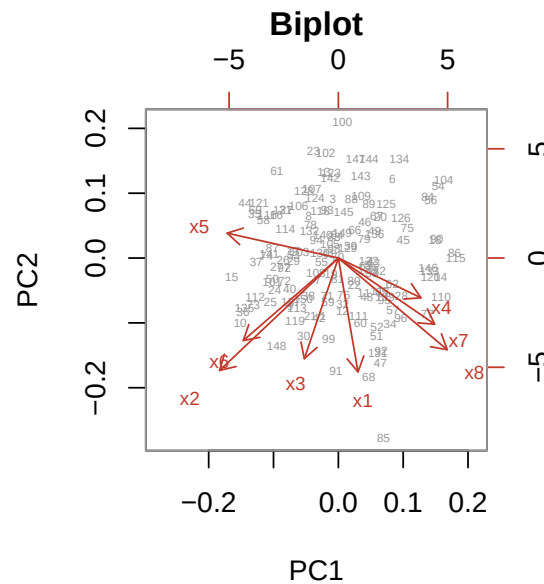
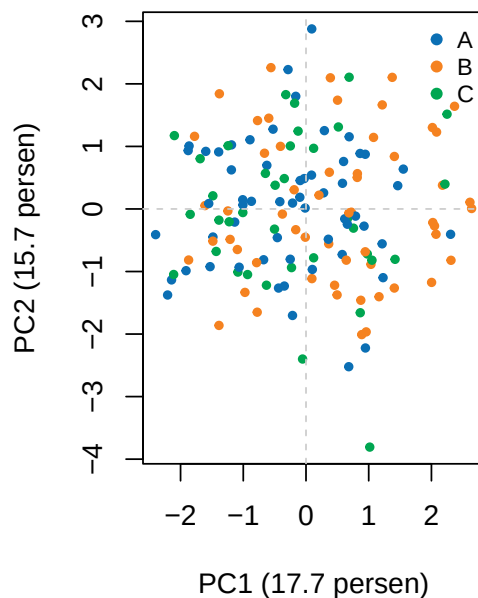
Kode 3.14 • R plot skor dan biplot

```
par(mfrow = c(1, 2))
plot(pc$x[, 1], pc$x[, 2], pch = 19, cex = .6,
     col = c("#0B6FB4", "#F58220", "#00A651")[dim_data$kelompok],
     xlab = sprintf("PC1 (%.1f persen)", 100 * ragam[1]),
     ylab = sprintf("PC2 (%.1f persen)", 100 * ragam[2]),
     main = "Skor Dua Komponen Pertama")
abline(h = 0, v = 0, col = "grey80", lty = 2)
```

```
legend("topright", levels(dim_data$kelompok), bty = "n", cex = .8,
      col = c("#0B6FB4", "#F58220", "#00A651"), pch = 19)

biplot(pc, choices = 1:2, cex = c(.4, .7), col = c("grey60", "#C0392B"),
      main = "Biplot")
```

Skor Dua Komponen Pertama



```
par(mfrow = c(1, 1))
```

Penjelasan Kode. Gambar kiri menyajikan skor dua komponen pertama dengan warna menurut kelompok, dan persentase keragaman dicantumkan pada nama sumbu. Gambar kanan berupa biplot yang menampilkan skor amatan bersama arah muatan tiap peubah asli.

Interpretasi Output

Bila titik antar-kelompok bertumpang tindih, berarti dua komponen pertama tidak memisahkan kelompok — dan pada data yang dibangkitkan saling bebas itu memang yang diharapkan. Pada biplot, panah yang berdekatan menandakan peubah yang berkorelasi, sedangkan panah yang panjang menandakan peubah yang banyak menyumbang pada kedua komponen tersebut. Perlu diingat bahwa biplot hanya menampilkan sebagian keragaman, sehingga kesimpulan darinya bersifat terbatas.

Contoh 3.15 | Menggambarkan jaringan beserta ukuran sentralitas

Kode 3.15 • R layout_with_fr dan pemetaan ukuran simpul

```

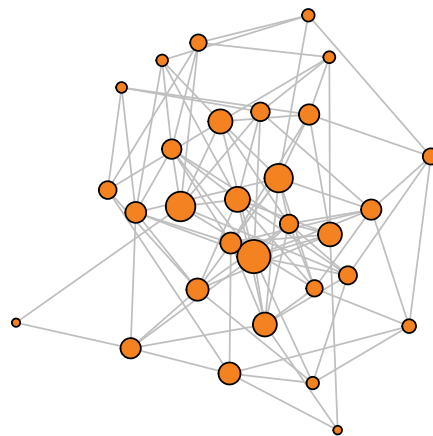
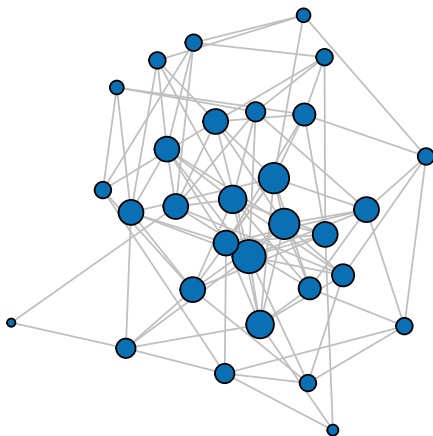
set.seed(2026)
tata <- layout_with_fr(g)
ukuran <- 4 + 12 * (degree(g) - min(degree(g))) /
              (max(degree(g)) - min(degree(g)) + 1e-9)

par(mfrow = c(1, 2), mar = c(1, 1, 3, 1))
plot(g, layout = tata, vertex.size = ukuran, vertex.label = NA,
     vertex.color = "#0B6FB4", edge.color = "grey75",
     main = "Ukuran simpul menurut derajat")

ukuran2 <- 4 + 12 * (betweenness(g) - min(betweenness(g))) /
              (max(betweenness(g)) - min(betweenness(g)) + 1e-9)
plot(g, layout = tata, vertex.size = ukuran2, vertex.label = NA,
     vertex.color = "#F58220", edge.color = "grey75",
     main = "Ukuran simpul menurut keperantaraan")

```

Ukuran simpul menurut derajat kuran simpul menurut keperantaraan



```

par(mfrow = c(1, 1), mar = c(5, 4, 4, 2) + 0.1)

```

Penjelasan Kode. Tata letak dihitung sekali memakai algoritma Fruchterman-Reingold lalu dipakai pada kedua gambar, sehingga posisi simpul tetap sama dan perbandingannya adil. Ukuran simpul dipetakan dari nilai sentralitas yang diskalakan ke rentang empat sampai enam belas.

Interpretasi Output

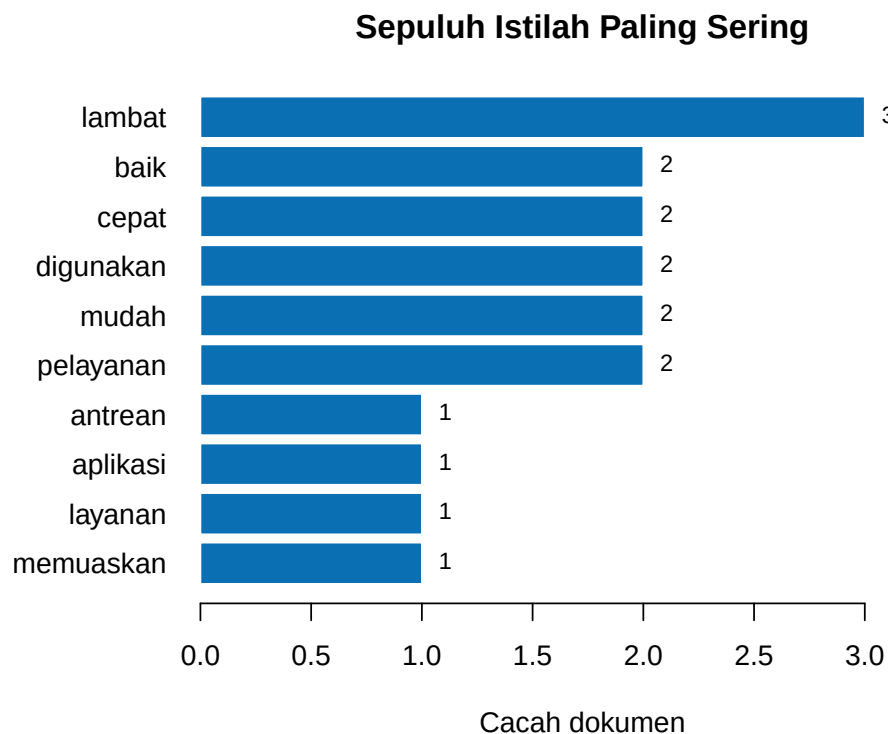
Memakai tata letak yang sama pada kedua gambar adalah kunci perbandingannya; bila tata letak dihitung ulang, posisi simpul berubah dan perbedaan ukurannya menjadi sukar ditelusuri. Perhatikan simpul yang besar pada gambar kanan tetapi kecil pada gambar kiri — itulah penghubung berderajat rendah namun bernilai strategis tinggi.

Contoh 3.16 | Menggambarkan frekuensi istilah

Kode 3.16 · R barplot mendatar frekuensi istilah

```
frek <- sort(colSums(dtm), decreasing = TRUE)
atas <- utils::head(frek, 10)

par(mar = c(4, 8, 3, 2))
bp <- barplot(rev(atas), horiz = TRUE, las = 1, col = "#0B6FB4",
              border = "white", xlab = "Cacah dokumen",
              main = "Sepuluh Istilah Paling Sering",
              xlim = c(0, max(atas) * 1.15))
text(rev(atas), bp, rev(atas), pos = 4, cex = .8)
```



```
par(mar = c(5, 4, 4, 2) + 0.1)
```

```
cat("kejarangan matriks:", round(100 * mean(dtm == 0), 1), "persen\n")
```

```
kejarangan matriks: 74.4 persen
```

```
cat("istilah muncul sekali saja:", sum(colSums(dtm) == 1),  
    "dari", ncol(dtm), "\n")
```

```
istilah muncul sekali saja: 7 dari 13
```

Penjelasan Kode. Frekuensi tiap istilah dijumlahkan lalu diurutkan menurun. Sepuluh teratas digambar sebagai batang mendatar dengan las bernilai satu agar label terbaca tegak. Batas kiri diperlebar melalui mar agar nama istilah tidak terpotong.

Interpretasi Output

Diagram batang mendatar lebih sesuai daripada awan kata untuk pelaporan resmi, sebab panjang batang dapat dibandingkan secara tepat sedangkan ukuran huruf tidak. Perhatikan pula banyaknya istilah yang muncul sekali saja: pada data teks nyata proporsinya sangat besar, dan istilah semacam itu lazim dibuang karena tidak menyumbang pembeda sekaligus memperbesar dimensi.

Contoh 3.17 | Membobot istilah dengan TF-IDF

Kode 3.17 • R menghitung TF-IDF

```
tf <- dtm                                # frekuensi kemunculan  
n_dok <- nrow(dtm)  
df <- colSums(dtm > 0)                   # cacah dokumen memuat istilah  
idf <- log(n_dok / df)  
tfidf <- t(tf) * idf  
  
cat("istilah paling KHAS (idf tertinggi):\n")
```

```
istilah paling KHAS (idf tertinggi):
```

```
print(round(sort(idf, decreasing = TRUE)[1:5], 3))
```

antrean	aplikasi	layanan	memuaskan	mengecewakan
1.792	1.792	1.792	1.792	1.792

```
cat("\nistilah paling UMUM (idf terendah):\n")
```

```
istilah paling UMUM (idf terendah):
```

```
print(round(sort(idf)[1:5], 3))
```

lambat	baik	cepat	digunakan	mudah
0.693	1.099	1.099	1.099	1.099

```
cat("\nbobot TF-IDF tiga dokumen pertama (lima istilah pertama):\n")
```

```
bobot TF-IDF tiga dokumen pertama (lima istilah pertama):
```

```
print(round(tfidf[1:3, 1:5], 3))
```

	antrean aplikasi	baik	cepat	digunakan
dok1	0	0.000	1.099	1.099
dok2	0	0.000	0.000	0.000
dok3	0	1.792	1.099	0.000

Penjelasan Kode. Frekuensi dokumen dihitung sebagai cacah dokumen yang memuat tiap istilah. Kebalikan frekuensi dokumen diperoleh dari logaritma rasio banyaknya dokumen terhadap frekuensi dokumen. Bobot akhir adalah hasil kali keduanya, dihitung dengan transpos ganda agar perkalian terjadi per kolom.

Interpretasi Output

Istilah yang muncul di hampir semua dokumen memperoleh bobot mendekati nol, sebab ia tidak membedakan apa pun. Sebaliknya istilah yang hanya muncul di sedikit dokumen memperoleh bobot tinggi karena khas. Inilah keunggulan TF-IDF dibandingkan frekuensi biasa, dan inilah pembobotan baku pada penambangan teks. Perlu dicatat bahwa pada kumpulan dokumen yang sangat kecil seperti ini, nilai idf menjadi kasar dan kurang dapat diandalkan.

Contoh 3.18 | Menghitung skor sentimen berbasis leksikon

Kode 3.18 • R leksikon sederhana dan pelabelan

```
positif <- c("baik", "cepat", "mudah", "memuaskan", "ramah")
negatif <- c("lambat", "sulit", "mengecewakan", "panjang", "buruk")

skor <- sapply(token, function(t) sum(t %in% positif) - sum(t %in% negatif))
hasil_sentimen <- data.frame(
  dokumen = paste0("dok", seq_along(ulasan)),
  skor = skor,
  label = ifelse(skor > 0, "positif", ifelse(skor < 0, "negatif", "netral")),
  teks = substr(ulasan, 1, 34))
print(hasil_sentimen, right = FALSE)
```


	dokumen	skor	label	teks
1	dok1	2	positif	layanan sangat baik dan cepat
2	dok2	-2	negatif	pelayanan lambat sekali mengecewak
3	dok3	2	positif	aplikasi mudah digunakan dan baik
4	dok4	-2	negatif	sangat lambat dan sulit digunakan
5	dok5	3	positif	cepat mudah dan memuaskan
6	dok6	-2	negatif	antrean panjang pelayanan lambat

```
table(hasil_sentimen$label)
```

negatif	positif
3	3

Penjelasan Kode. Dua daftar kata acuan disusun. Skor tiap dokumen dihitung sebagai selisih cacah kata positif dan negatif memakai operator keanggotaan. Label ditetapkan dengan ifelse bersarang, lalu hasilnya disajikan bersama potongan teks aslinya agar dapat diperiksa langsung.

Interpretasi Output

Bandingkan label yang dihasilkan dengan teks aslinya; pada contoh sederhana ini hasilnya masuk akal. Namun cobalah menambahkan kalimat seperti tidak buruk atau sebuah sindiran, dan cara ini akan keliru. Keterbatasan itu bukan cacat kecil melainkan sifat mendasar pendekatan leksikon, sehingga hasilnya layak dipakai sebagai penjajakan awal saja.

3.7 Latihan Mandiri

Kerjakan bertahap. Gunakan `data/data_dimensi.csv` dan `data/data_relasi.csv`.

3.7.0.0.1 Tingkat Dasar (melengkapi kode)

Latihan 3.1 | Komponen utama

```
dim_data <- read.csv("data/data_dimensi.csv")
X <- dim_data[, paste0("x", 1:8)]
pc <- ____ (X, scale. = ____ )          # analisis komponen utama, dibakukan
ragam <- pc$sdev^2 / ____ (pc$sdev^2)    # proporsi keragaman
```

Latihan 3.2 | Ukuran jaringan

```
library(igraph)
rel <- read.csv("data/data_relasi.csv")
rel <- rel[rel$dari != rel$ke, ]
g <- ____ (rel[, c("dari", "ke")], directed = FALSE)
cat("simpul:", ____ (g), "sisi:", ____ (g), "
")
```

Latihan 3.3 | Matriks dokumen-istilah

```
teks <- c("pelayanan cepat dan baik", "antrean lambat sekali")
token <- lapply(strsplit(____ (teks), "[^a-z]+"), function(x) x[nchar(x) > 2])
kosakata <- sort(____ (unlist(token)))    # istilah unik, terurut
```

3.7.0.0.2 Tingkat Menengah (menulis sendiri)

Latihan 3.4 | Kutukan dimensi pada data nyata

Ambil kolom x1 sampai x8 dari `data_dimensi.csv`. Hitung kontras jarak ketika hanya memakai dua kolom pertama, lalu empat, enam, dan seluruh delapan kolom. Sajikan sebagai tabel dan grafik, lalu jelaskan apakah polanya sejalan dengan peragaan pada Contoh 3.1 dan mengapa besarnya bisa berbeda.

Latihan 3.5 | Membetulkan kode yang salah

Kode berikut memuat empat kesalahan (PCA tanpa pembakuan, proporsi keragaman memakai simpangan baku bukan kuadratnya, graf berarah padahal hubungan timbal balik, dan kata henti tidak dibuang). Te-

mukan dan perbaiki, lalu jelaskan tiap kesalahannya.

```
dim_data <- read.csv("data/data_dimensi.csv")
X <- dim_data[, paste0("x", 1:8)]

# (a) PCA tanpa pembakuan padahal skala peubah dapat berbeda
pc <- prcomp(X)

# (b) proporsi keragaman memakai sdev, bukan kuadratnya
ragam <- pc$sdev / sum(pc$sdev)

# (c) hubungan bersifat timbal balik tetapi graf dibuat berarah
library(igraph)
rel <- read.csv("data/data_relasi.csv")
g <- graph_from_data_frame(rel[, c("dari","ke")], directed = TRUE)

# (d) kata henti tidak dibuang sehingga mendominasi kosakata
teks <- c("pelayanan cepat dan baik", "antrean lambat dan panjang")
token <- lapply(strsplit(tolower(teks), "[^a-z]+"), function(x) x[nchar(x) > 2])
```

Keluaran yang benar. Setelah diperbaiki, kode yang dipakai adalah sebagai berikut.

```
# (a) pembakuan wajib bila satuan peubah berbeda
pc <- prcomp(X, scale. = TRUE)

# (b) proporsi keragaman memakai KUADRAT simpangan baku
ragam <- pc$sdev^2 / sum(pc$sdev^2)

# (c) hubungan timbal balik -> graf tak berarah, lalu buang gelang dan sisi ganda
rel <- rel[rel$dari != rel$ke, ]
g <- simplify(graph_from_data_frame(rel[, c("dari","ke")], directed = FALSE))

# (d) kata henti dibuang dengan setdiff
henti <- c("dan", "yang", "di", "ke", "dari")
token <- lapply(strsplit(tolower(teks), "[^a-z]+"),
               function(x) setdiff(x[nchar(x) > 2], henti))
```

Pemeriksaan yang wajib berhasil: `round(sum(ragam), 8)` bernilai **1**, dan `is_directed(g)` bernilai **FALSE**.

Latihan 3.6 | Muatan komponen

Tafsirkan komponen utama pertama dan kedua dengan memeriksa muatannya. Sebutkan tiga peubah penyumbang terbesar pada masing-masing, lalu nilai apakah komponen itu dapat diberi nama yang bermakna

— dan bila tidak, jelaskan mengapa.

Latihan 3.7 | Kekokohan jaringan

Hapus lima simpul berkeperantaraan tertinggi satu per satu dari graf. Pada tiap tahap laporkan banyaknya komponen terhubung dan ukuran komponen terbesarnya. Bandingkan terhadap penghapusan lima simpul **acak**, lalu jelaskan perbedaannya.

3.7.0.0.3 Tingkat Lanjut (kasus kecil)

Latihan 3.8 | Menentukan perlakuan menurut tipe data

Untuk masing-masing keadaan berikut, tentukan tipe datanya, sebutkan **dua perlakuan** yang sesuai, dan jelaskan **satu jebakan** yang mengintai: (a) data ekspresi gen 20.000 peubah pada 80 pasien; (b) catatan saling-mengikuti antar 50.000 akun media sosial; (c) 100.000 ulasan produk berbahasa Indonesia; (d) pembacaan sensor suhu tiap menit selama setahun pada 200 titik.

3.8 Penugasan

Sebagai calon analis pada instansi statistik, susun laporan pengenalan tipe data atas tiga bentuk data yang berbeda: berdimensi tinggi, jaringan, dan teks. Untuk masing-masing, tentukan perlakuan yang sesuai beserta alasannya.

Konteks. Sebuah unit menerima tiga sumber data sekaligus: tabel indikator berpeubah banyak, catatan relasi antar-unit, dan kumpulan teks aduan masyarakat. Ketiganya perlu dinilai sebelum direncanakan pengolahannya.

Tugas 3.1 | Data Berdimensi Tinggi

1. Peragakan kutukan dimensi pada `data_dimensi.csv` dengan menghitung kontras jarak untuk 2, 4, 6, dan 8 peubah. Sajikan tabel dan grafik, lalu tafsirkan.
2. Terapkan analisis komponen utama. Sajikan diagram scree dan keragaman kumulatif, lalu **tentukan berapa komponen yang Anda pertahankan** beserta alasannya. Bandingkan keputusan menurut aturan garis merata dan aturan 80 persen.
3. Tafsirkan komponen pertama dengan memeriksa muatannya (`pc$rotation[, 1]`). Peubah mana yang paling menyumbang?

Tugas 3.2 | Data Jaringan

4. Susun graf dari `data_relasi.csv`. Laporkan banyaknya simpul, sisi, kerapatan, dan komponen terhubung.
5. Hitung **ketiga ukuran sentralitas**. Sajikan lima simpul teratas menurut masing-masing, lalu jelaskan mengapa daftarnya dapat berbeda dan simpul mana yang paling penting secara strategis.
6. Temukan komunitas dan laporkan modularitasnya. **Nilai apakah struktur komunitas yang ditemukan bermakna atau sekadar kebetulan**, disertai alasan.

Tugas 3.3 | Analisis Jaringan Lanjut

- 6b. Bentuk **graf berbobot** memakai kolom bobot. Bandingkan lima simpul teratas menurut derajat biasa dan menurut derajat berbobot, lalu jelaskan mengapa urutannya dapat berbeda.
- 6c. Lakukan **analisis kekokohan**: hapus lima simpul berkeperantaraan tertinggi berturut-turut dan catat perubahan struktur jaringannya. Bandingkan terhadap penghapusan acak.

Tugas 3.4 | Data Teks

7. Susun sendiri sepuluh kalimat ulasan berbahasa Indonesia. Bentuk matriks dokumen-istilah, laporkan tingkat kejarangannya, lalu hitung skor sentimen berbasis leksikon.
8. Sertakan **minimal dua kalimat yang sengaja menjebak** (misalnya bernegasi atau bersindir). Laporkan apakah skornya keliru dan jelaskan mengapa.
- 8b. Hitung bobot **TF-IDF** atas kumpulan ulasan Anda. Laporkan lima istilah paling khas dan lima paling umum, lalu jelaskan mengapa keduanya berbeda dari sekadar frekuensi.
9. Susun laporan R Markdown berisi kode, keluaran, gambar, dan interpretasi. Tutup dengan **ringkasan (maksimal 300 kata)** memuat: perlakuan yang direkomendasikan bagi ketiga tipe data, jebakan yang perlu diwaspadai, dan penegasan bahwa data merupakan simulasi.

3.9 Ringkasan Modul

Daftar Periksa penguasaan:

- ☐ Saya dapat menyebutkan enam bentuk data beserta perlakuan yang sesuai.
- ☐ Saya dapat memperagakan kutukan dimensi dan menjelaskan akibatnya.
- ☐ Saya memahami bahwa menambah peubah tidak selalu menambah informasi.
- ☐ Saya dapat menerapkan analisis komponen utama beserta pembakuannya.
- ☐ Saya dapat membaca diagram scree dan menentukan banyaknya komponen.
- ☐ Saya dapat menghitung tiga ukuran sentralitas dan membedakan maknanya.
- ☐ Saya dapat menemukan komunitas dan menilai kebermaknaannya lewat modularitas.
- ☐ Saya dapat menyusun matriks dokumen-istilah dari teks mentah.
- ☐ Saya menyadari keterbatasan sentimen berbasis leksikon.

Kartu Referensi Cepat.

Keperluan	Perintah
nMatriks jarak	<code>dist(X), as.matrix(dist(X))</code>
nKomponen utama	<code>prcomp(X, scale. = TRUE)</code>
nProporsi keragaman	<code>pc\$sdev^2 / sum(pc\$sdev^2)</code>
nMuatan komponen	<code>pc\$rotation</code>
nMembentuk graf	<code>graph_from_data_frame()</code>
nUkuran graf	<code>vcount(), ecount(), edge_density()</code>
nKesentralan	<code>degree(), closeness(), betweenness()</code>
nKomunitas	<code>cluster_louvain(), modularity()</code>
nTokenisasi	<code>strsplit(tolower(x), "[^a-z]+")</code>
nBuang kata henti	<code>setdiff(token, henti)</code>

Modul ini memperluas cakrawala dari data tabular ke bentuk lain yang lazim pada big data. Prinsip yang perlu dibawa: kenali bentuk data sebelum memilih perlakuan, waspadai bahwa dimensi tinggi merusak ukuran jarak, dan ingat bahwa algoritma pengelompokan maupun sentimen selalu memberi jawaban meski datanya tidak berstruktur. Modul berikutnya membahas cara memperoleh data dari internet beserta pertimbangan etikanya.

3.10 Referensi

- Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann.
- Aggarwal, C. C., Hinneburg, A., & Keim, D. A. (2001). On the surprising behavior of distance metrics in high dimensional space. *ICDT 2001*, 420-434.
- Newman, M. E. J. (2018). *Networks* (2nd ed.). Oxford University Press.
- Silge, J., & Robinson, D. (2017). *Text Mining with R: A Tidy Approach*. O'Reilly Media.
- Csardi, G., & Nepusz, T. (2006). The igraph software package for complex network research. *InterJournal Complex Systems*, 1695.

MODUL

4

PENGUMPULAN DATA DAN PERAYAPAN WEB

Pertemuan Minggu ke-4

Sub-CPMK 4: Mahasiswa mampu menjelaskan konsep pengumpulan data dan perayapan web, menerapkan etika pengumpulan, serta mengekstrak data terstruktur dari laman web menjadi tabel yang siap dianalisis.

Penyusun: Achmad Syahrul Choir



Daftar Isi

4.1	Deskripsi Modul	2
4.2	Tujuan Praktikum	2
4.3	Alat dan Bahan	2
4.4	Instruksi Pengerjaan	3
4.5	Teori Singkat	3
4.5.0.1	Kedudukan pada CRISP-DM	4
4.5.0.2	Tiga Cara Memperoleh Data	4
4.5.0.3	Etika dan Ketentuan	4
4.5.0.4	Menilai Keterwakilan Data Daring	5
4.5.0.5	Struktur Dokumen HTML	5
4.5.0.6	Bentuk Data yang Ditemui di Web	6
4.5.0.7	Menakar Beban yang Ditimbulkan	6
4.5.0.8	Menangani Kegagalan	6
4.6	Contoh Praktikum dengan R dan Python	8
4.7	Padanan dengan R: Paket rvest	27
4.8	Latihan Mandiri	30
4.9	Penugasan	34
4.10	Tambahan: Membaca Data dari Antarmuka Pemrograman (API)	35
4.10.0.1	Antarmuka Pemrograman (API)	35
4.10.0.2	Penerapan Nyata: Indeks Berita detik.com	36
4.10.0.3	Google Trends dan Nowcasting	39
4.11	Ringkasan Modul	42
4.12	Referensi	44

4.1 Deskripsi Modul

Tiga modul sebelumnya bekerja dengan data yang sudah tersedia sebagai berkas. Dalam praktik, data kerap harus dikumpulkan lebih dahulu, dan sumbernya sering berupa laman web maupun antarmuka pemrograman. Modul ini membahas cara memperolehnya beserta pertimbangan yang menyertainya.

Perlu ditegaskan sejak awal bahwa kemampuan teknis mengambil data dari internet tidak dengan sendirinya membuat pengambilan itu boleh dilakukan. Persoalan hukum, etika, dan beban terhadap peladen pihak lain sama pentingnya dengan persoalan teknis. Karena itu bagian Teori memuat pembahasan etika sebelum contoh teknisnya, bukan sesudahnya.

Modul ini memakai **Python**, bukan R. Alasannya bukan preferensi melainkan kematangan perkakas: pustaka `requests` dan `BeautifulSoup` menyediakan penanganan permintaan HTTP dan penelusuran struktur dokumen yang lebih lengkap dan lebih terdokumentasi untuk keperluan ini.

Tidak merayapi situs sungguhan. Seluruh contoh pada modul ini bekerja atas **laman tiruan** yang disediakan sebagai berkas lokal. Praktikum ini **tidak** merayapi situs mana pun di internet. Dengan begitu latihan dapat diulang kapan saja, hasilnya tetap sama, dan tidak ada peladen pihak lain yang terbebani.

4.2 Tujuan Praktikum

Setelah menyelesaikan praktikum ini secara tuntas, mahasiswa diharapkan mampu:

1. **Menjelaskan** perbedaan pengumpulan data melalui berkas, API, dan perayapan.
2. **Menerapkan** pertimbangan etika dan hukum sebelum mengumpulkan data.
3. **Membaca** berkas `robots.txt` dan menafsirkan ketentuannya.
4. **Menelusuri** struktur dokumen HTML untuk menemukan bagian yang diperlukan.
5. **Mengekstrak** tabel HTML menjadi `DataFrame` yang bertipe benar.
6. **Menangani** kegagalan permintaan secara santun beserta jeda antar-permintaan.
7. **Menilai** mutu dan keterwakilan data hasil pengumpulan.

4.3 Alat dan Bahan

Perangkat dan bahan yang diperlukan:

- **Python** dengan pustaka `requests`, `beautifulsoup4`, dan `pandas`.
- **R** dengan paket `rvest` dan `dplyr` untuk Bagian 4.8 (padanan dalam R).
- Pasang dengan `pip install requests beautifulsoup4 pandas`.
- Berkas tiruan pada folder `data/` — seluruhnya **lokal**, tidak ada yang diunduh:

Berkas	Isi
<code>laman_contoh.html</code>	laman bertabel tunggal
<code>daftar_1.html</code> , <code>daftar_2.html</code> , <code>daftar_3.html</code>	daftar berhalaman dengan tautan
<code>detail_a.html</code> ... <code>detail_i.html</code>	laman rincian tiap wilayah
<code>api_wilayah.json</code>	tiruan tanggapan API
<code>umpan_rilis.xml</code>	tiruan umpan RSS
<code>laman_versi_baru.html</code>	laman yang tata letaknya berubah
<code>robots_tiruan.txt</code>	tiruan berkas ketentuan perayapan
<code>data_perbandingan.csv</code>	data pembandingan

Berkas `buat_laman.py` memuat kode pembangkit seluruh berkas tiruan itu.

4.4 Instruksi Pengerjaan

Ikuti tahapan berikut:

Sebelum praktikum: pasang ketiga pustaka Python; pastikan berkas `laman_contoh.html` berada pada subfolder `data/`.

Selama praktikum: kerjakan Contoh berurutan. Bacalah bagian etika pada Teori sebelum menjalankan kode apa pun.

Pembagian kerja: Contoh dan Latihan dikerjakan di kelas; **Penugasan dikerjakan di luar sesi.**

Pengumpulan: susun laporan R Markdown yang memuat potongan kode Python beserta keluarannya, render ke PDF atau HTML, kumpulkan melalui Google Classroom.

4.5 Teori Singkat

4.5.0.1 Kedudukan pada CRISP-DM

Modul ini berada pada tahap Data Understanding, khususnya langkah pengumpulan data awal. Perlu diingat bahwa tahap ini datang setelah Business Understanding: data dikumpulkan untuk menjawab persoalan yang sudah dirumuskan, bukan dikumpulkan lebih dahulu lalu dicarikan persoalannya.

4.5.0.2 Tiga Cara Memperoleh Data

Perbandingan tiga cara.

	Berkas	API	Perayapan
Bentuk	terstruktur	terstruktur	tak terstruktur
Kestabilan	tinggi	tinggi	rendah
Izin	jelas	jelas	kerap kabur
Usaha	rendah	sedang	tinggi
Risiko	rendah	rendah	tinggi

Urutan pertimbangan yang sehat adalah dari kiri ke kanan. Bila data tersedia sebagai berkas unduhan, ambil dari sana. Bila tidak, periksa apakah penyedia menyediakan API. Perayapan ditempuh hanya ketika kedua jalan sebelumnya tertutup, sebab ia paling rapuh terhadap perubahan tata letak laman dan paling besar risikonya.

4.5.0.3 Etika dan Ketentuan

Sebelum menulis kode, empat hal perlu dipastikan. Pertama, apakah ketentuan layanan situs mengizinkan pengambilan otomatis. Kedua, apa yang tercantum pada berkas robots.txt. Ketiga, apakah data yang diambil memuat informasi pribadi. Keempat, seberapa besar beban yang ditimbulkan bagi peladen penyedia.

Membaca robots.txt. Berkas ini diletakkan pada akar situs, misalnya contoh.id/robots.txt, dan memuat ketentuan bagi perayap otomatis:

User-agent: *	ketentuan berlaku bagi semua perayap
Disallow: /admin/	bagian ini tidak boleh dirayapi
Allow: /publik/	bagian ini diizinkan
Crawl-delay: 10	beri jeda 10 detik antar-permintaan

robots.txt bukan pengaman. Berkas robots.txt adalah **permintaan**, bukan penghalang teknis; secara teknis ia dapat diabaikan begitu saja. Justru karena itulah mematuhi menjadi soal etika, bukan soal kemampuan. Mengabaikannya dapat berakibat alamat IP diblokir, dan pada kasus tertentu berujung persoalan hukum.

Beban terhadap peladen kerap terlupakan. Perayap yang mengirim ratusan permintaan per detik dapat

memperlambat layanan bagi pengguna lain, dan dampaknya tidak berbeda dari serangan penolakan layanan meski niatnya berbeda. Karena itu jeda antar-permintaan bukan kesopanan berlebihan melainkan keharusan.

4.5.0.4 Menilai Keterwakilan Data Daring

Persoalan terbesar data hasil pengumpulan daring bukan pada mutu teknisnya melainkan pada keterwakilannya. Survei terancang menetapkan kerangka dan peluang terpilih, sehingga dugaannya dapat dipertanggungjawabkan. Data daring tidak memiliki keduanya: yang tercakup adalah yang kebetulan tercatat.

Tiga pertanyaan keterwakilan.

Siapa tercakup?	kelompok mana yang terwakili dan mana yang tidak sama sekali
Mengapa tercakup?	apakah keterpilihan berkaitan dengan peubah yang diteliti
Berubahkah cakupannya?	apakah kelompok yang tercakup berubah antar-waktu

Pertanyaan kedua paling menentukan. Bila keterpilihan berkaitan dengan peubah yang diteliti — misalnya menghitung tingkat pengangguran dari lowongan daring, padahal pencari kerja tanpa akses internet tidak tercatat — maka biasanya bersifat sistematis dan tidak dapat diperbaiki dengan memperbesar data.

Data besar tidak menyembuhkan bias seleksi. Memperbesar data hanya mengecilkan galat acak, tidak mengurangi bias seleksi. Contoh berjuta amatan dari kelompok yang keliru tetap menduga hal yang keliru — bahkan dengan selang kepercayaan yang menyesatkan karena tampak sangat sempit. Inilah sebabnya kantor statistik memperlakukan data daring sebagai pelengkap, bukan pengganti survei.

4.5.0.5 Struktur Dokumen HTML

Laman web tersusun sebagai pohon elemen bersarang. Setiap elemen ditandai tag pembuka dan penutup, dan dapat memiliki atribut seperti id maupun class yang berguna sebagai penanda saat penelusuran.

Elemen yang lazim dicari.

Tag	Arti	Cara mencarinya
table	tabel	<code>find("table", id="...")</code>
tr	baris tabel	<code>find_all("tr")</code>
td, th	sel isi dan sel kepala	<code>find_all("td")</code>
a	tautan	<code>find_all("a", href=True)</code>
ul, li	daftar	<code>select("ul.kelas li")</code>

4.5.0.6 Bentuk Data yang Ditemui di Web

Empat bentuk dan cara menanganinya.

Bentuk	Ciri	Penanganan
HTML statis	isi ada pada sumber laman	BeautifulSoup memadai
HTML dinamis	isi dimuat JavaScript	perlu peramban terotomasi
JSON	terstruktur, bertipe	json.loads langsung
XML/RSS	bersarang, berskema	penguraian khusus

Isi yang tidak ada pada sumber laman. Banyak situs modern memuat isinya lewat JavaScript setelah laman terbuka. Akibatnya requests hanya memperoleh kerangka kosong, dan mahasiswa kerap menyangka pemilihnya salah padahal isinya memang belum ada. Cara memeriksanya sederhana: cari potongan teks yang tampak di peramban pada hasil `r.text`. Bila tidak ditemukan, lamannya dinamis dan menuntut pendekatan berbeda — atau, lebih baik, cari API yang menjadi sumber isinya.

4.5.0.7 Menakar Beban yang Ditimbulkan

Beban terhadap peladen dapat diperkirakan sebelum perayapan dijalankan. Bila sebuah laman berukuran seratus lima puluh kilobita dan hendak diambil seribu laman, maka lalu lintas yang ditimbulkan mencapai sekitar seratus lima puluh megabita. Dengan jeda dua detik, pekerjaan itu memakan waktu lebih dari setengah jam — dan justru lamanya waktu itulah yang membuatnya santun.

Tips dan Praktik Baik. Sebelum merayapi banyak laman, hitung lebih dahulu perkiraan lalu lintas dan lamanya. Bila hasilnya tampak mengkhawatirkan, itu pertanda pendekatannya perlu ditinjau — misalnya dengan mempersempit cakupan, memakai API, atau menanyakan ketersediaan data langsung kepada penyediaanya. Cara terakhir ini sering dilupakan padahal paling sederhana.

4.5.0.8 Menangani Kegagalan

Permintaan jaringan dapat gagal karena berbagai sebab: peladen sibuk, jaringan terputus, atau alamat berubah. Kode yang baik memperkirakan kegagalan itu dan menanganinya, bukan berhenti mendadak. Tiga kebiasaan yang perlu dibangun adalah memeriksa kode status, membatasi waktu tunggu, dan mencoba ulang secukupnya dengan jeda yang membesar.

Kode status yang sering ditemui.

200	berhasil	lanjutkan pengolahan
403	ditolak	tidak diizinkan; jangan dipaksa
404	tidak ditemukan	alamat keliru atau sudah dihapus
429	terlalu banyak permintaan	perlambat, perbesar jeda
5xx	galat pada peladen	coba lagi nanti, bukan segera

Tips dan Praktik Baik. Kode status 429 adalah tanda paling jelas bahwa laju permintaan Anda terlalu tinggi. Menanggapinya dengan mencoba ulang seketika justru memperburuk keadaan. Tanggapan yang benar adalah memperbesar jeda secara berlipat — satu detik, dua, empat, dan seterusnya — yang lazim disebut *exponential backoff*.

4.6 Contoh Praktikum dengan R dan Python

Contoh 4.1 | Memeriksa ketentuan sebelum mengambil data

Kode 4.1 • Python menguraikan robots.txt

```
# Contoh isi robots.txt (disalin sebagai teks, bukan diunduh)
robots = """\nUser-agent: *\nDisallow: /admin/\nDisallow: /internal/\nAllow: /publik/\nCrawl-delay: 10\n"""

aturan = {"disallow": [], "allow": [], "crawl_delay": None}
for baris in robots.strip().split("\n"):
    if ":" not in baris:
        continue
    kunci, nilai = [b.strip() for b in baris.split(":", 1)]
    kunci = kunci.lower()
    if kunci == "disallow":
        aturan["disallow"].append(nilai)
    elif kunci == "allow":
        aturan["allow"].append(nilai)
    elif kunci == "crawl-delay":
        aturan["crawl_delay"] = int(nilai)

print(aturan)

def boleh(jalur, aturan):
    for d in aturan["disallow"]:
        if jalur.startswith(d):
            return False
    return True

for jalur in ["/publik/data.html", "/admin/pengguna", "/internal/log"]:
    print(f"{jalur:24s} -> {'boleh' if boleh(jalur, aturan) else 'TIDAK boleh'}")
```

Penjelasan Kode. Isi robots.txt diuraikan baris demi baris. Setiap baris dipecah pada tanda titik dua pertama menjadi kunci dan nilai, lalu dikelompokkan menurut jenis ketentuannya. Fungsi boleh memeriksa apakah suatu jalur diawali salah satu ketentuan larangan.

Interpretasi Output

Pemeriksaan semacam ini seharusnya menjadi langkah pertama, bukan langkah yang dilewati. Perhatikan bahwa Crawl-delay bernilai sepuluh berarti penyedia meminta jeda sepuluh detik antar-

permintaan; mengabaikannya berarti mengabaikan permintaan yang jelas tertulis. Perlu dicatat bahwa penguraian sederhana ini tidak mencakup seluruh kaidah robots.txt, dan pustaka `urllib.robotparser` tersedia untuk keperluan yang lebih lengkap.

Contoh 4.2 | Membaca dan menelusuri struktur laman

Kode 4.2 • Python BeautifulSoup dan find

```
from bs4 import BeautifulSoup

with open("data/laman_contoh.html", encoding="utf-8") as f:
    sup = BeautifulSoup(f, "html.parser")

print("judul:", sup.title.string)
print("banyaknya tabel:", len(sup.find_all("table")))
print("banyaknya tautan:", len(sup.find_all("a", href=True)))

# menelusuri elemen berdasarkan penanda
catatan = sup.find("p", class_="catatan")
print("catatan:", catatan.get_text(strip=True))
```

Penjelasan Kode. Berkas HTML dibuka lalu diuraikan menjadi pohon elemen memakai penguraiannya bawaan Python. Perintah `find` mencari elemen pertama yang cocok, sedangkan `find_all` mengembalikan seluruhnya. Argumen `class` ditulis dengan garis bawah di belakang karena `class` merupakan kata kunci Python.

Interpretasi Output

Langkah pertama pada perayapan selalu berupa penjelajahan struktur, bukan langsung mengambil isi. Mengetahui berapa banyak tabel dan tautan yang ada mencegah pengambilan tabel yang keliru. Perhatikan pula catatan pada laman yang menyatakan bahwa datanya tiruan — membaca catatan metodologi sebelum memakai data adalah kebiasaan yang wajib.

Contoh 4.3 | Mengekstrak tabel menjadi DataFrame

Kode 4.3 • Python menyusun DataFrame dari baris tabel

```
import pandas as pd

tab = sup.find("table", id="tabel-wilayah")
kepala = [th.get_text(strip=True) for th in tab.find("thead").find_all("th")]
baris = [[td.get_text(strip=True) for td in tr.find_all("td")]
          for tr in tab.find("tbody").find_all("tr")]

df = pd.DataFrame(baris, columns=kepala)
print(df)
print("\ntipe sebelum konversi:")
print(df.dtypes)
```

Penjelasan Kode. Tabel dicari berdasarkan atribut id agar tidak tertukar dengan tabel lain. Kepala kolom diambil dari elemen th di dalam thead, sedangkan isinya diambil dari elemen td pada tiap baris tbody. Argumen strip bernilai True membuang spasi berlebih di kedua ujung teks.

Interpretasi Output

Perhatikan bahwa seluruh kolom bertipe objek, yaitu teks, meski isinya tampak berupa angka. Inilah jebakan paling sering pada perayapan: data yang tampak numerik sesungguhnya masih berupa untaian karakter, sehingga penjumlahan maupun perata-rataan akan gagal atau memberi hasil keliru.

Contoh 4.4 | Mengubah tipe data dan memeriksa hasilnya

Kode 4.4 • Python `pd.to_numeric` dengan `errors coerce`

```
for k in ["Penduduk", "Pengeluaran", "Akses Internet"]:
    df[k] = pd.to_numeric(df[k], errors="coerce")

print(df.dtypes)
print("\nnilai gagal dikonversi:", int(df.isna().sum().sum()))
print(df.describe().round(2))
```

Penjelasan Kode. Ketiga kolom diubah menjadi numerik. Argumen errors bernilai coerce membuat nilai yang tidak dapat diubah menjadi NaN alih-alih menghentikan program, sehingga kegagalan konversi dapat dihitung dan diperiksa.

Interpretasi Output

Pemakaian `coerce` disertai penghitungan NaN adalah kebiasaan penting. Bila ada nilai yang gagal dikonversi, itu pertanda ada isi tak terduga pada laman — misalnya tanda pemisah ribuan, satuan yang ikut tertulis, atau tanda strip untuk data yang tidak tersedia. Mengabaikan ini berarti kehilangan data diam-diam.

Contoh 4.5 | Mengambil bagian lain dari laman**Kode 4.5 • Python** `select` dengan pemilih CSS

```
metodologi = [li.get_text(strip=True) for li in sup.select("ul.metodologi li")]
for m in metodologi:
    print("-", m)

tautan = [(a.get_text(strip=True), a["href"]) for a in sup.find_all("a", href=True)]
print("\ntautan ditemukan:", tautan)
```

Penjelasan Kode. Perintah `select` memakai pemilih CSS, yang lebih ringkas untuk penelusuran bersarang: `ul titik metodologi li` berarti elemen `li` di dalam `ul` berkelas `metodologi`. Tautan diambil beserta teksnya sebagai pasangan.

Interpretasi Output

Catatan metodologi kerap memuat informasi yang menentukan sah tidaknya penafsiran, seperti cakupan dan satuan. Mengambil tabel tanpa membaca metodologinya adalah kekeliruan yang berakibat salah tafsir. Adapun daftar tautan menjadi dasar penelusuran berlanjut apabila data tersebar di beberapa halaman.

Contoh 4.6 | Membedakan laman statis dan dinamis**Kode 4.6 • Python** memeriksa keberadaan isi pada sumber

```
with open("data/laman_contoh.html", encoding="utf-8") as f:
    mentah = f.read()

# apakah isi yang tampak benar-benar ada pada sumber laman?
uji = ["Wilayah A", "1250000", "Akses Internet"]
for t in uji:
    print(f" {t:18s} ada pada sumber: {t in mentah}")
```

```
print("\nada tag script:", "<script" in mentah.lower())
print("ukuran sumber :", len(mentah), "karakter")
```

Penjelasan Kode. Berkas dibaca sebagai teks mentah, lalu beberapa potongan yang tampak di peramban dicari keberadaannya. Keberadaan tag script juga diperiksa sebagai petunjuk bahwa laman mungkin memuat isi secara dinamis.

Interpretasi Output

Pemeriksaan sesederhana ini menghemat banyak waktu. Bila teks yang tampak di peramban tidak ditemukan pada sumber laman, maka pemilih secanggih apa pun tidak akan menemukannya — persoalannya bukan pada pemilih melainkan pada cara laman itu dibangun. Laman contoh ini sepenuhnya statis sehingga seluruh pengujian bernilai benar.

Contoh 4.7 | Menakar beban sebelum merayapi

Kode 4.7 • Python **perkiraan lalu lintas dan waktu**

```
ukuran_laman_kb = len(mentah) / 1024

for n_laman, jeda in [(10, 1), (100, 2), (1000, 2), (10000, 2)]:
    total_mb = ukuran_laman_kb * n_laman / 1024
    menit = n_laman * jeda / 60
    print(f" {n_laman:6d} laman | {total_mb:8.1f} MB | {menit:8.1f} menit")
```

Penjelasan Kode. Ukuran satu laman dipakai memperkirakan lalu lintas total pada beberapa skala perayapan. Lamanya dihitung dari banyaknya laman dikalikan jeda antar-permintaan lalu dibagi enam puluh agar terbaca dalam menit.

Interpretasi Output

Perhitungan ini sebaiknya dikerjakan pada tahap perencanaan, bukan setelah perayapan berjalan. Angka pada baris terakhir memperlihatkan bahwa merayapi sepuluh ribu laman secara santun memakan waktu berjam-jam — dan bila waktu itu terasa terlalu lama, jalan keluarnya bukan mempercepat permintaan melainkan mencari sumber data yang lebih baik.

Contoh 4.8 | Menyimpan hasil ekstraksi dengan aman

Kode 4.8 • Python menyimpan data beserta metadatanya

```
import pandas as pd, json, os

os.makedirs("hasil", exist_ok=True)

# menyimpan sebagai CSV beserta metadata pengumpulannya
df.to_csv("hasil/data_hasil.csv", index=False)

meta = {
    "sumber": "data/laman_contoh.html (laman tiruan)",
    "waktu_ambil": "2026-01-01 00:00:00",
    "baris": int(len(df)),
    "kolom": list(df.columns),
    "catatan": "data simulasi untuk latihan; bukan data resmi",
}

with open("hasil/metadata.json", "w", encoding="utf-8") as f:
    json.dump(meta, f, ensure_ascii=False, indent=2)

print(json.dumps(meta, ensure_ascii=False, indent=2))
```

Penjelasan Kode. Folder keluaran dibuat bila belum ada. Data disimpan sebagai CSV, dan keterangan pengumpulannya disimpan terpisah sebagai JSON. Argumen `ensure_ascii` bernilai salah agar huruf beraksen tersimpan apa adanya, sedangkan `indent` membuat berkas mudah dibaca manusia.

Interpretasi Output

Menyimpan metadata bersama datanya adalah kebiasaan yang jarang dilakukan tetapi sangat menolong. Beberapa bulan kemudian, pertanyaan mengenai kapan data diambil dan dari mana asalnya hampir pasti muncul, dan tanpa catatan itu jawabannya hilang. Pada data hasil perayapan, mencantumkan bahwa datanya tiruan atau bahwa cakupannya terbatas mencegah salah tafsir di kemudian hari.

Contoh 4.9 | Menggambarkan hasil ekstraksi**Kode 4.9 • Python** matplotlib untuk memeriksa hasil

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

fig, ax = plt.subplots(1, 2, figsize=(9, 3.6))
```

```

ax[0].barh(df["Wilayah"], df["Akses Internet"], color="#0B6FB4")
ax[0].set_xlabel("Akses internet (persen)")
ax[0].set_title("Akses Internet per Wilayah")

ax[1].scatter(df["Penduduk"] / 1e6, df["Pengeluaran"], s=60, color="#F58220")
ax[1].set_xlabel("Penduduk (juta)")
ax[1].set_ylabel("Pengeluaran")
ax[1].set_title("Penduduk vs Pengeluaran")

fig.suptitle("Hasil Ekstraksi (data tiruan)", y=1.02)
fig.tight_layout()
plt.savefig("hasil/gambar.png", dpi=120, bbox_inches="tight")
print("gambar disimpan")

```

Penjelasan Kode. Dua gambar disajikan berdampingan memakai subplots. Backend Agg dipilih agar gambar dapat dibuat tanpa tampilan layar. Argumen `bbox_inches` bernilai `tight` mencegah tepi gambar terpotong saat disimpan.

Interpretasi Output

Menggambarkan hasil ekstraksi bukan sekadar untuk pelaporan melainkan juga untuk pemeriksaan: nilai yang keliru terurai kerap langsung tampak sebagai titik yang jauh menyimpang. Perhatikan pula judul yang mencantumkan bahwa datanya tiruan — kebiasaan mencantumkan status data pada gambar mencegah gambar itu beredar terlepas dari konteksnya.

Contoh 4.10 | Permintaan HTTP yang santun

Kode 4.10 • Python requests dengan penanganan galat

```

import requests, time

def ambil_santun(url, jeda=1.0, coba=3, tunggu=10):
    """Mengambil laman dengan jeda, batas waktu, dan pengulangan berlipat."""
    kepala = {"User-Agent": "PraktikumSTIS/1.0 (keperluan pendidikan)"}
    for i in range(coba):
        try:
            r = requests.get(url, headers=kepala, timeout=tunggu)
            if r.status_code == 200:
                return r.text
            if r.status_code == 429:
                # terlalu banyak permintaan

```

```

        time.sleep(jeda * (2 ** i))      # jeda berlipat
        continue
    print(f"status {r.status_code}; berhenti")
    return None
except requests.RequestException as e:
    print(f"percobaan {i+1} gagal: {type(e).__name__}")
    time.sleep(jeda * (2 ** i))
return None

# Fungsi ini TIDAK dijalankan pada praktikum; disajikan sebagai acuan.
print("Fungsi ambil_santun siap dipakai bila kelak merayapi situs sungguhan.")

```

Penjelasan Kode. Fungsi ini menggabungkan empat kebiasaan baik. Penanda User-Agent menyebutkan identitas dan keperluan secara jujur. Argumen timeout membatasi waktu tunggu. Kode status 429 ditanggapi dengan jeda yang berlipat dua setiap percobaan. Blok try dan except menangkap kegagalan jaringan agar program tidak berhenti.

Interpretasi Output

Perhatikan bahwa hanya status 200 yang dianggap berhasil; status lain ditangani secara berbeda sesuai maknanya. Perhatikan pula bahwa fungsi ini sengaja tidak dijalankan pada praktikum, sejalan dengan prinsip modul untuk tidak membebani peladen pihak mana pun. Menyebutkan identitas pada User-Agent juga merupakan praktik yang dianjurkan agar penyedia dapat menghubungi bila terjadi persoalan.

Contoh 4.11 | Membandingkan hasil ekstraksi terhadap data pembandingan

Kode 4.11 • Python membandingkan rata-rata dua sumber

```

import pandas as pd

pemb = pd.read_csv("data/data_perbandingan.csv")
print("pembandingan:", pemb.shape, "| hasil ekstraksi:", df.shape)

banding = pd.DataFrame({
    "ekstraksi_5": [df["Penduduk"].mean(), df["Pengeluaran"].mean(),
                   df["Akses Internet"].mean()],
    "pembandingan_40": [pemb["penduduk"].mean(), pemb["pengeluaran"].mean(),
                        pemb["akses_internet"].mean()],
}, index=["penduduk", "pengeluaran", "akses_internet"])
banding["selisih_persen"] = (

```

```
100 * (banding["ekstraksi_5"] - banding["pembanding_40"])
/ banding["pembanding_40"])
print(banding.round(1))
```

Penjelasan Kode. Data pembanding berisi empat puluh wilayah dibaca, lalu rata-rata ketiga diubah dibandingkan terhadap lima wilayah hasil ekstraksi. Selisihnya dinyatakan dalam persen terhadap nilai pembanding.

Interpretasi Output

Selisih yang besar bukan pertanda salah hitung melainkan pertanda cakupan yang sempit. Lima wilayah bukan contoh acak dari empat puluh, sehingga rata-ratanya tidak menduga apa pun mengenai keseluruhan. Inilah wujud nyata persoalan keterwakilan: banyaknya baris yang berhasil dikumpulkan sama sekali tidak menjamin bahwa yang terkumpul mewakili apa yang hendak digambarkan.

Contoh 4.12 | Pemilih CSS: cara ringkas menelusuri laman

Kode 4.12 • R select dan select-one

```
from bs4 import BeautifulSoup
import pandas as pd, json, time, re
import xml.etree.ElementTree as ET
import urllib.robotparser as robotparser

def baca_laman(berkas):
    with open("data/" + berkas, encoding="utf-8") as f:
        return BeautifulSoup(f, "html.parser")

s = baca_laman("daftar_1.html")

# find_all versus select: dua cara menuju hasil yang sama
cara_find = s.find("table", class_="daftar").find("tbody").find_all("tr")
cara_select = s.select("table.daftar tbody tr")
print("jumlah baris -- find_all:", len(cara_find), " select:", len(cara_select))

jumlah baris -- find_all: 3 select: 3

# pemilih CSS memudahkan penelusuran berlapis
print("kepala kolom:", [th.get_text(strip=True) for th in s.select("thead th")])

kepala kolom: ['Wilayah', 'Penduduk', 'Pengeluaran']
```



```
print("tautan pertama:", s.select_one("tbody tr td a")["href"])
```

```
tautan pertama: detail_a.html
```

```
print("judul halaman:", s.select_one("h1").get_text(strip=True))
```

```
judul halaman: Daftar Wilayah (Data Latihan)
```

Penjelasan Kode. `select()` menerima **pemilih CSS** — sintaks yang sama dengan yang dipakai penata gaya laman. `table.daftar tbody tr` berarti "setiap `tr` di dalam `tbody` di dalam `table` berkelas `daftar`". `select_one()` mengembalikan elemen pertama saja.

Interpretasi Output

Kedua cara memberi 3 baris yang sama — keduanya sah. Bedanya terletak pada keterbacaan ketika struktur laman berlapis.

Dengan `find_all`, tiap tingkat menuntut satu pemanggilan berantai: `find(...).find(...).find_all(...)`. Bila salah satu tingkat tidak ditemukan, rantai itu putus dengan galat `AttributeError` yang menunjuk baris terakhir, bukan tingkat yang sesungguhnya hilang — sehingga sulit ditelusuri.

Dengan `select`, seluruh jalur ditulis sebagai satu untai. Bila tidak ada yang cocok, hasilnya **daftar kosong**, bukan galat. Itu lebih aman untuk perayapan banyak laman: satu laman yang menyimpang tidak menghentikan seluruh proses.

Kaidah praktis: pakailah `select` untuk penelusuran berdasarkan struktur dan kelas, dan `find` bila perlu menyaring dengan syarat yang tidak dapat dinyatakan sebagai pemilih CSS — misalnya mencocokkan teks di dalam elemen.

Contoh 4.13 | Merayapi laman berhalaman

Kode 4.13 • R mengikuti tautan Berikutnya sampai habis

```
halaman = "daftar_1.html"
baris, tautan_detail, dikunjungi = [], [], []

while halaman:
    dikunjungi.append(halaman)
    s = baca_laman(halaman)
    for tr in s.select("table.daftar tbody tr"):
        td = tr.select("td")
        baris.append({"wilayah": td[0].get_text(strip=True),
                      "penduduk_teks": td[1].get_text(strip=True),
```

```

        "pengeluaran_teks": td[2].get_text(strip=True)})
    tautan_detail.append(tr.select_one("td a")["href"])

    berikutnya = s.select_one("nav.paginasi a.berikutnya")
    halaman = berikutnya["href"] if berikutnya else None
    time.sleep(0.2)          # jeda santun antar-permintaan

print("halaman dikunjungi:", dikunjungi)

halaman dikunjungi: ['daftar_1.html', 'daftar_2.html', 'daftar_3.html']

print("baris terkumpul:", len(baris), " tautan detail:", len(tautan_detail))

baris terkumpul: 9  tautan detail: 9

daftar = pd.DataFrame(baris)
print(daftar.head(4))

   wilayah penduduk_teks pengeluaran_teks
0  Wilayah A      1.250.000             980
1  Wilayah B       860.000            1120
2  Wilayah C      2.340.000            1450
3  Wilayah D       430.000             760

```

Interpretasi Output

Perayapan berhenti sendiri setelah tiga halaman — bukan karena banyaknya halaman dituliskan di dalam kode, melainkan karena laman terakhir **tidak lagi memuat** tautan Berikutnya, sehingga `select_one` mengembalikan `None` dan pengulangan berakhir.

Cara ini lebih tahan perubahan daripada menuliskan `for h in range(1, 4)`. Bila jumlah halaman bertambah menjadi lima, kode ini tetap benar tanpa disunting; kode yang menuliskan angka 3 akan diam-diam melewati dua halaman terakhir — kekeliruan yang tidak menimbulkan galat apa pun dan karena itu sulit ditemukan.

Tiga hal yang wajib ada pada pengulangan semacam ini:

1. **Jeda antar-permintaan** (`time.sleep`). Di sini 0,2 detik karena berkasnya lokal; pada situs sungguhan ikuti `Crawl-delay` pada `robots.txt`, atau minimal 1–2 detik.
2. **Catatan laman yang sudah dikunjungi**. Bila tautan saling menunjuk melingkar, tanpa catatan ini pengulangan tidak akan pernah berhenti.
3. **Batas atas**. Pada perayapan sungguhan tambahkan pencacah dengan batas maksimum, sebagai jaring pengaman bila struktur laman berubah.

Contoh 4.14 | Menelusuri dari daftar ke laman rincian

Kode 4.14 • R mengambil rincian tiap wilayah beserta atribut data

```

rincian = []
for t in tautan_detail:
    s = baca_laman(t)
    d = {"wilayah": s.select_one("h1.judul-wilayah").get_text(strip=True),
        "berkas": t}
    # dt berisi label, dd berisi nilainya -- keduanya berpasangan berurutan
    for dt, dd in zip(s.select("dl.rincian dt"), s.select("dl.rincian dd")):
        label = dt.get_text(strip=True).lower().replace(" ", "_")
        # atribut data-nilai memuat angka bersih; teksnya memuat satuan
        d[label] = dd.get("data-nilai", dd.get_text(strip=True))
    rincian.append(d)
    time.sleep(0.2)

detail = pd.DataFrame(rincian)
print(detail.head(4))

```

```

      wilayah      berkas penduduk ... akses_internet      pulau tahun
0 Wilayah A  detail_a.html  1250000 ...           72.4 Pulau Utara  2026
1 Wilayah B  detail_b.html   860000 ...           65.1 Pulau Utara  2026
2 Wilayah C  detail_c.html  2340000 ...           81.7 Pulau Tengah 2026
3 Wilayah D  detail_d.html   430000 ...           48.3 Pulau Timur  2026

[4 rows x 7 columns]

```

```
print("\ndimensi:", detail.shape, " kolom:", list(detail.columns))
```

```
dimensi: (9, 7) kolom: ['wilayah', 'berkas', 'penduduk', 'pengeluaran', 'akses_internet', 'pulau', 'tahun']
```

Interpretasi Output

Pola **daftar** → **rincian** ini adalah bentuk perayapan yang paling sering dijumpai: laman indeks memuat ringkasan beserta tautan, dan rincian lengkapnya berada pada laman tersendiri.

Perhatikan pengambilan nilai. Teks yang tampak pada laman berbunyi "1.250.000 jiwa" — memuat pemisah ribuan dan satuan, sehingga harus dibersihkan sebelum dapat dihitung. Tetapi laman ini juga menyediakan atribut `data-nilai` yang memuat angka bersih. `dd.get("data-nilai", dd.get_text(strip=True))` mengambil atribut itu bila ada, dan jatuh kembali ke teks bila tidak.

Selalu periksa atribut sebelum mengurai teks. Laman yang dirancang baik kerap menyimpan nilai mesin pada atribut `data-*`, `datetime`, atau `content`; memakainya menghindarkan seluruh persoalan pembersihan teks — dan lebih tahan ketika format tampilan berubah.

Perhatikan pula bahwa laman rincian memuat dua peubah yang **tidak ada** pada laman daftar: `pulau` dan `tahun`. Itulah alasan menelusuri sampai ke laman rincian, bukan berhenti di indeks.

Contoh 4.15 | Membersihkan teks dengan ekspresi reguler

Kode 4.15 • R memisahkan angka dari satuan

```
s = baca_laman("detail_a.html")
teks_dd = [dd.get_text(strip=True) for dd in s.select("dl.rincian dd")]
print("teks apa adanya:", teks_dd)

teks apa adanya: ['1.250.000 jiwa', 'Rp 980 ribu/kapita/bulan', '72.4 persen', 'Pulau Utara', '2026']
```

```
def ambil_angka(teks):
    """Ambil angka pertama dari teks, tangani pemisah ribuan titik."""
    m = re.search(r"[\d][\d.]*(?:,\d+)?", teks)
    if not m:
        return None
    angka = m.group(0)
    if "," in angka:          # 72,4 -> desimal
        angka = angka.replace(".", "").replace(",", ".")
    elif angka.count(".") >= 1 and len(angka.split(".")[1]) == 3:
        angka = angka.replace(".", "")      # 1.250.000 -> ribuan
    return float(angka)

for t in teks_dd:
    print(f"{t!r:45s} -> {ambil_angka(t)}")
```

```
'1.250.000 jiwa'          -> 1250000.0
'Rp 980 ribu/kapita/bulan' -> 980.0
'72.4 persen'            -> 72.4
'Pulau Utara'            -> None
'2026'                  -> 2026.0
```

Interpretasi Output

Fungsi ini menyelesaikan persoalan yang khas data Indonesia: **titik sebagai pemisah ribuan dan koma sebagai pemisah desimal** — kebalikan dari kebiasaan Inggris yang menjadi bawaan `float()`.

Perhatikan bahwa `float("1.250.000")` akan gagal, dan `float("72,4")` juga gagal. Yang lebih berbahaya, `float("1.250")` **tidak** gagal melainkan menghasilkan 1,25 — seribu kali lebih kecil daripada yang dimaksud, tanpa peringatan apa pun. Kekeliruan semacam ini lolos dari pemeriksaan otomatis dan hanya ketahuan bila hasilnya dibandingkan dengan sumber lain.

Aturan yang dipakai di sini: bila ada koma, ia dianggap pemisah desimal; bila tidak dan bagian setelah titik terakhir tepat tiga angka, titik dianggap pemisah ribuan. Aturan ini **tidak sempurna** — "1.234" dapat berarti seribu dua ratus tiga puluh empat atau satu koma dua tiga empat, dan tidak ada cara memastikannya dari teks itu sendiri.

Karena itu pembersihan berbasis ekspresi reguler harus selalu disertai **pemeriksaan kewajaran**: ban-

dingkan rentang hasil dengan yang diharapkan, dan periksa sampel secara manual. Bila laman menyediakan atribut data-* seperti pada Contoh sebelumnya, pakailah itu dan hindari seluruh persoalan ini.

Contoh 4.16 | Periksa apakah tersedia API sebelum merayapi

Kode 4.16 • R membaca tanggapan JSON

```
with open("data/api_wilayah.json", encoding="utf-8") as f:
    tanggapan = json.load(f)
```

```
print("kunci teratas:", list(tanggapan.keys()))
```

```
kunci teratas: ['meta', 'data']
```

```
print("metadata:", tanggapan["meta"])
```

```
metadata: {'sumber': 'API tiruan', 'tahun': 2026, 'jumlah': 9, 'lisensi': 'hanya untuk latihan'}
```

```
api = pd.DataFrame(tanggapan["data"])
print(api.head(4))
```

	nama	penduduk	pengeluaran	akses_internet	pulau
0	Wilayah A	1250000	980	72.4	Pulau Utara
1	Wilayah B	860000	1120	65.1	Pulau Utara
2	Wilayah C	2340000	1450	81.7	Pulau Tengah
3	Wilayah D	430000	760	48.3	Pulau Timur

```
print("\ntipe data langsung benar:")
```

```
tipe data langsung benar:
```

```
print(api.dtypes)
```

```
nama                str
penduduk            int64
pengeluaran         int64
akses_internet     float64
pulau              str
dtype: object
```

Interpretasi Output

Bandungkan dengan seluruh contoh sebelumnya. Data yang sama diperoleh dengan **satu baris** `pd.DataFrame(tanggapan["data"])`, dan — ini yang paling penting — **tipe datanya sudah benar sejak awal**: penduduk bertipe integer, akses_internet bertipe float. Tidak ada pemisah ribuan yang perlu

dibersihkan, tidak ada satuan yang menempel pada angka, tidak ada risiko salah tafsir titik dan koma. Selain itu tanggapan API menyertakan **metadata**: sumber, tahun, jumlah cacah, dan lisensi. Perayapan HTML jarang memberikan hal itu; ia harus dicatat sendiri oleh perayapnya.

Kaidah yang wajib dipegang: **cari API lebih dahulu, rayapi belakangan**. Perayapan HTML adalah pilihan terakhir, bukan pertama. Cara memeriksanya: telusuri laman dokumentasi penyedia data; periksa apakah lamannya memuat panggilan XHR/fetch pada tab Network peramban — kerap laman dinamis mengambil datanya dari API yang dapat dipanggil langsung; dan tanyakan kepada penyedia datanya.

Perayapan HTML rapuh terhadap perubahan tata letak, membebani peladen, dan sering berada di wilayah abu-abu dari sisi ketentuan layanan. API dirancang untuk diakses program, stabil, dan izinnya jelas.

Contoh 4.17 | Membaca umpan XML

Kode 4.17 • R ElementTree untuk RSS

```
akar = ET.parse("data/umpan_rilis.xml").getroot()
print("judul umpan:", akar.findtext("./channel/title"))
```

```
judul umpan: Rilis Statistik Tiruan
```

```
butir = [{"judul": it.findtext("title"),
          "tanggal": it.findtext("pubDate"),
          "tautan": it.findtext("link"),
          "ringkasan": it.findtext("description")}
          for it in akar.iter("item")]
umpan = pd.DataFrame(butir)
print(umpan[["judul", "tanggal"]].head(5))
```

```
          judul      tanggal
0  Rilis statistik Wilayah A  01 Feb 2026
1  Rilis statistik Wilayah B  02 Feb 2026
2  Rilis statistik Wilayah C  03 Feb 2026
3  Rilis statistik Wilayah D  04 Feb 2026
4  Rilis statistik Wilayah E  05 Feb 2026
```

```
print("\njumlah butir:", len(umpan))
```

```
jumlah butir: 5
```

Interpretasi Output

Umpan RSS atau Atom sering menjadi jalan tengah antara API dan perayapan HTML: strukturnya baku, ringan, dan **memang disediakan untuk dibaca program** — sehingga memakainya tidak menimbulkan persoalan etika sebagaimana perayapan.

Bagi pekerjaan statistik, umpan rilis berguna untuk **memantau pembaruan**: alih-alih merayapi seluruh laman setiap hari untuk memeriksa apakah ada rilis baru, cukup baca umpannya — jauh lebih murah bagi kedua pihak.

Perhatikan pubDate yang berbentuk teks. Sebelum dipakai, ia harus diurai menjadi tipe tanggal dengan `pd.to_datetime()` beserta `format=` yang tepat; jangan mengandalkan penebakan otomatis, sebab format tanggal berbeda-beda antar-umpan dan salah tebak menghasilkan tanggal yang keliru tanpa peringatan.

Contoh 4.18 | Membaca robots.txt secara terprogram

Kode 4.18 · R `urllib.robotparser`

```
print(open("data/robots_tiruan.txt").read())
```

```
User-agent: *
```

```
Disallow: /admin/
```

```
Disallow: /pencarian
```

```
Crawl-delay: 5
```

```
User-agent: PerayapNakal
```

```
Disallow: /
```

```
Sitemap: https://contoh.tiruan/sitemap.xml
```

```
aturan = robotparser.RobotFileParser()
```

```
aturan.parse(open("data/robots_tiruan.txt").read().splitlines())
```

```
for jalur in ["/rilis", "/admin/pengguna", "/pencarian?q=data", "/data/wilayah"]:
```

```
    print(f"{jalur:25s} boleh diambil: {aturan.can_fetch('*', jalur)}")
```

```
/rilis                boleh diambil: True
```

```
/admin/pengguna       boleh diambil: False
```

```
/pencarian?q=data     boleh diambil: False
```

```
/data/wilayah         boleh diambil: True
```

```
print("\nCrawl-delay bagi perayap umum:", aturan.crawl_delay("*"), "detik")
```

```
Crawl-delay bagi perayap umum: 5 detik
```

```
print("PerayapNakal boleh /rilis:", aturan.can_fetch("PerayapNakal", "/rilis"))
```

```
PerayapNakal boleh /rilis: False
```

Interpretasi Output

Contoh 4.1 membaca `robots.txt` dengan mata; di sini ia dibaca **oleh program**. Perbedaannya penting pada perayapan berskala besar: memeriksa ribuan alamat satu per satu secara manual tidak mungkin, dan `RobotFileParser` melakukannya secara konsisten.

Perhatikan tiga hasilnya. Jalur `/admin/pengguna` dan `/pencarian?q=data` **tidak boleh** diambil karena tercakup aturan `Disallow`. `Crawl-delay` bernilai 5 detik — itulah jeda minimum yang harus dipakai, bukan angka yang dipilih sendiri. Dan perayap bernama `PerayapNakal` ditolak sepenuhnya, memperlihatkan bahwa aturan dapat berbeda menurut nama perayap.

Dua penegasan yang wajib menyertai. **Pertama**, `robots.txt` bukan pengaman teknis melainkan **permintaan**; ia dapat diabaikan secara teknis, tetapi mengabaikannya berarti melanggar norma yang berlaku dan dapat berkonsekuensi hukum melalui ketentuan layanan. **Kedua**, `robots.txt` yang mengizinkan **tidak berarti** pengambilan data itu sah: ketentuan layanan, hak cipta atas isi, dan ketentuan perlindungan data pribadi berlaku terpisah dan lebih kuat.

Contoh 4.19 | Mendeteksi perubahan tata letak sebelum ia merusak data

Kode 4.19 • R memeriksa kesesuaian struktur

```
def ekstrak_tabel(berkas, pilih):
    s = baca_laman(berkas)
    tabel = s.select_one(pilih)
    if tabel is None:
        raise ValueError(f"tabel tidak ditemukan pada {berkas} dengan pilih {pilih!r}")
    kepala = [th.get_text(strip=True) for th in tabel.select("thead th")]
    isi = [[td.get_text(strip=True) for td in tr.select("td")]
           for tr in tabel.select("tbody tr")]
    return kepala, pd.DataFrame(isi, columns=kepala)

KEPALA_DIHARAPKAN = ["Wilayah", "Penduduk", "Pengeluaran", "Akses Internet"]

def periksa(berkas, pilih):
    try:
        kepala, df = ekstrak_tabel(berkas, pilih)
    except ValueError as e:
        return f"GAGAL -- {e}"
```



```

if kepala != KEPALA_DIHARAPKAN:
    return (f"BERUBAH -- kepala kolom tidak sesuai\n"
            f"        diharapkan: {KEPALA_DIHARAPKAN}\n"
            f"        ditemukan : {kepala}")
return f"AMAN -- {df.shape[0]} baris, kepala kolom sesuai"

print("laman_contoh.html      :", periksa("laman_contoh.html", "table#tabel-wilayah"))

laman_contoh.html      : AMAN -- 5 baris, kepala kolom sesuai

print("laman_versi_baru.html :", periksa("laman_versi_baru.html", "table#tabel-wilayah"))

laman_versi_baru.html : GAGAL -- tabel tidak ditemukan pada laman_versi_baru.html dengan pemilih 'table#tabel-wilayah'

print("laman_versi_baru.html :", periksa("laman_versi_baru.html", "table#tabel-statistik-wilayah"))

laman_versi_baru.html : BERUBAH -- kepala kolom tidak sesuai
        diharapkan: ['Wilayah', 'Penduduk', 'Pengeluaran', 'Akses Internet']
        ditemukan : ['Nama Wilayah', 'Jumlah Penduduk', 'Pengeluaran', 'Akses Internet']

```

Interpretasi Output

Ketiga baris memperagakan tiga keadaan yang berbeda.

Baris pertama berhasil: struktur laman sesuai harapan.

Baris kedua gagal dengan pesan yang jelas: pengenalan tabel berubah dari tabel-wilayah menjadi tabel-statistik-wilayah, sehingga `select_one` mengembalikan `None`. Tanpa pemeriksaan `if tabel is None`, baris berikutnya akan melempar `AttributeError` yang menunjuk tempat yang keliru — dan pada perayapan terjadwal yang berjalan tanpa pengawasan, galat semacam itu kerap hanya tercatat di log tanpa dibaca siapa pun.

Baris ketiga menemukan tabelnya, tetapi **kepala kolomnya berubah**: "Penduduk" menjadi "Jumlah Penduduk", "Wilayah" menjadi "Nama Wilayah". Inilah keadaan yang paling berbahaya — ekstraksi **berhasil**, DataFrame terbentuk, tidak ada galat sama sekali, tetapi nama kolomnya berbeda sehingga kode di hilir yang memanggil `df["Penduduk"]` akan gagal jauh kemudian, atau lebih buruk, penggabungan dengan data lain menghasilkan kolom kosong tanpa peringatan.

Karena itu perayap yang dijalankan berulang **wajib memuat pemeriksaan struktur**: bandingkan kepala kolom, cacah kolom, dan cacah baris terhadap nilai yang diharapkan; hentikan proses dan kirim pemberitahuan bila menyimpang. Pemeriksaan ini lebih penting daripada kode ekstraksinya sendiri, sebab tata letak laman **pasti** berubah cepat atau lambat.

Contoh 4.20 | Permintaan yang tahan gangguan: batas waktu dan percobaan ulang

Kode 4.20 • R kerangka acuan — tidak dijalankan

```

import requests, time, random

def ambil_dengan_ulang(url, maks_ulang=3, jeda_awal=2.0, batas_waktu=10):
    """Mengambil satu alamat dengan percobaan ulang dan jeda berlipat."""
    kepala = {"User-Agent": "PraktikumSTIS/1.0 (kontak: nama@stis.ac.id)"}
    for percobaan in range(1, maks_ulang + 1):
        try:
            r = requests.get(url, headers=kepala, timeout=batas_waktu)
        except requests.exceptions.Timeout:
            print(f" percobaan {percobaan}: batas waktu terlampaui")
        else:
            if r.status_code == 200:
                return r.text
            if r.status_code == 429:
                # terlalu sering meminta
                jeda = int(r.headers.get("Retry-After", jeda_awal * 2 ** percobaan))
                print(f" percobaan {percobaan}: 429, menunggu {jeda} detik")
                time.sleep(jeda); continue
            if 400 <= r.status_code < 500:
                # galat pada sisi kita
                raise RuntimeError(f"{r.status_code} pada {url} -- tidak diulang")
            print(f" percobaan {percobaan}: status {r.status_code}")
            # jeda berlipat disertai goyangan acak agar permintaan tidak serentak
            time.sleep(jeda_awal * 2 ** (percobaan - 1) + random.uniform(0, 1))
    raise RuntimeError(f"gagal setelah {maks_ulang} percobaan: {url}")

```

Fungsi ini sengaja tidak dijalankan. Potongan di atas ditulis dengan `eval=False` karena ia menghubungkan alamat di internet. Ia disajikan sebagai **kerangka acuan** bagi mahasiswa yang kelak melakukan perayapan sungguhan — dengan izin dan pada situs yang tepat.

Interpretasi Output

Enam keputusan penting terkandung di dalamnya, dan seluruhnya lazim dilupakan pemula.

Batas waktu (timeout). Tanpanya, satu peladen yang menggantung akan membekukan seluruh perayapan tanpa batas.

Nama perayap yang jujur (User-Agent) beserta alamat kontak. Pengelola situs yang melihat lalu lintas tak lazim dapat menghubungi Anda alih-alih langsung memblokir. Menyamar sebagai peramban justru menandakan niat menghindari deteksi.

Membedakan galat. Status 4xx berarti kekeliruan ada pada permintaan kita — mengulangnya hanya menambah beban tanpa manfaat, sehingga langsung dilempar sebagai galat. Status 5xx dan batas waktu berarti gangguan sementara pada peladen, dan layak diulang.

Menghormati 429. Status ini berarti "terlalu sering meminta". Tajuk `Retry-After` menyatakan berapa lama harus menunggu — angka itu wajib dipatuhi, bukan ditawarkan.

Jeda berlipat (*exponential backoff*). Jeda menjadi 2, 4, 8 detik — memberi peladen kesempatan pulih alih-alih menambah tekanan.

Goyangan acak (*jitter*). Bila banyak proses mengulang pada jeda yang persis sama, permintaannya akan serentak dan membebani peladen sekaligus. Tambahan acak kecil menyebarkannya.

4.7 Padanan dengan R: Paket rvest

Bagian ini mengulang beberapa contoh di atas dengan R, agar mahasiswa yang bekerja di R tidak harus berpindah bahasa hanya untuk mengambil data.

Contoh 4.21 | Ekstraksi dengan rvest

Kode 4.21 • R read_html, html_element, dan html_table

```
suppressPackageStartupMessages({library(rvest); library(dplyr)})
```

```
laman <- read_html("data/laman_contoh.html")
```

```
# (a) tabel utuh menjadi data frame -- satu baris
```

```
tabel <- laman |> html_element("table#tabel-wilayah") |> html_table()
print(tabel)
```

```
# A tibble: 5 x 4
```

	Wilayah	Penduduk	Pengeluaran	Akses Internet
	<chr>	<int>	<int>	<dbl>
1	Wilayah A	1250000	980	72.4
2	Wilayah B	860000	1120	65.1
3	Wilayah C	2340000	1450	81.7
4	Wilayah D	430000	760	48.3
5	Wilayah E	1780000	1310	77.9

```
# (b) pemilih CSS dan atribut
```

```
daftar1 <- read_html("data/daftar_1.html")
tautan <- daftar1 |> html_elements("table.daftar tbody tr td a") |> html_attr("href")
cat("tautan pada halaman 1:", tautan, "\n")
```

```
tautan pada halaman 1: detail_a.html detail_b.html detail_c.html
```

```
# (c) laman rincian: label dan nilai berpasangan
```

```
detail_a <- read_html("data/detail_a.html")
label <- detail_a |> html_elements("dl.rincian dt") |> html_text2()
nilai <- detail_a |> html_elements("dl.rincian dd") |> html_attr("data-nilai")
teks <- detail_a |> html_elements("dl.rincian dd") |> html_text2()
```

```
data.frame(label = label, atribut = nilai, teks = teks)
```

	label	atribut	teks
1	Penduduk	1250000	1.250.000 jiwa
2	Pengeluaran	980 Rp	980 ribu/kapita/bulan
3	Akses Internet	72.4	72.4 persen
4	Pulau	<NA>	Pulau Utara
5	Tahun	<NA>	2026

Interpretasi Output

`html_table()` adalah keunggulan `rvest` yang paling menonjol: ia mengubah tabel HTML menjadi data frame **beserta tipe datanya** dalam satu pemanggilan — perhatikan bahwa Penduduk langsung bertipe integer dan Akses Internet bertipe double, tanpa pembersihan apa pun. Di Python, langkah itu memerlukan beberapa baris (atau `pd.read_html`, yang serupa).

Perhatikan pula kolom atribut pada keluaran terakhir: dua baris terakhir bernilai NA karena Pulau dan Tahun memang tidak memiliki atribut data-nilai — nilainya hanya ada sebagai teks. Kolom teks melengkapinya. Pola "ambil atribut, jatuh kembali ke teks" karena itu berlaku sama di kedua bahasa.

Padanan penamaan:

Python (BeautifulSoup)	R (rvest)	Kegunaan
<code>BeautifulSoup(f, "html.parser")</code>	<code>read_html()</code>	membaca laman
<code>select_one(pemilih)</code>	<code>html_element()</code>	satu elemen
<code>select(pemilih)</code>	<code>html_elements()</code>	banyak elemen
<code>get_text(strip=True)</code>	<code>html_text2()</code>	isi teks
<code>elemen["href"]</code>	<code>html_attr("href")</code>	nilai atribut
<code>pd.read_html()</code>	<code>html_table()</code>	tabel menjadi data frame

Untuk perayapan sungguhan di R, paket `httr2` menyediakan padanan `requests` — termasuk `req_retry()` dan `req_throttle()` yang menangani percobaan ulang serta pembatasan laju secara bawaan.

Contoh 4.22 | Menilai mutu dan keterwakilan hasil

Kode 4.22 • Python pemeriksaan mutu hasil perayapan

```
print("baris:", len(df), "| kolom:", len(df.columns))
print("nilai hilang:", int(df.isna().sum().sum()))
print("wilayah unik:", df['Wilayah'].nunique())

# pemeriksaan kewajaran
```

```
masuk_akal = (df["Akses Internet"].between(0, 100)).all()
print("akses internet dalam 0-100:", masuk_akal)
print("\nringkasan:")
print(df[["Penduduk", "Pengeluaran", "Akses Internet"]].agg(["min", "max", "mean"]).round(1))
```

Penjelasan Kode. Setelah data terkumpul, ukurannya diperiksa, nilai hilang dihitung, dan kewajaran rentang diuji dengan `between` yang diikuti `all` sehingga menghasilkan satu nilai logika. Ringkasan tiga besaran disajikan sekaligus memakai `agg`.

Interpretasi Output

Data hasil perayapan wajib diperlakukan dengan curiga, sebab tidak ada rancangan penarikan contoh yang menjaminnya. Lima wilayah pada laman tiruan ini jelas tidak mewakili populasi mana pun. Pertanyaan keterwakilan — siapa yang tercakup dan siapa yang tidak — jauh lebih penting daripada banyaknya baris yang berhasil dikumpulkan.

4.8 Latihan Mandiri

Kerjakan bertahap. Gunakan `data/laman_contoh.html` dan `data/data_perbandingan.csv`. Seluruh latihan bekerja pada berkas lokal; **tidak ada yang merayapi internet**.

4.8.0.0.1 Tingkat Dasar (melengkapi kode)

Latihan 4.1 | Menelusuri laman

```
from bs4 import BeautifulSoup
with open("data/laman_contoh.html", encoding="utf-8") as f:
    sup = ____ (f, "html.parser")
print(sup.title.string)
tab = sup.____ ("table", id="tabel-wilayah")    # cari SATU elemen
baris = tab.find("tbody").____ ("tr")          # cari SEMUA baris
print("baris:", len(baris))
```

Latihan 4.2 | Menyusun DataFrame

```
import pandas as pd
kepala = [th.get_text(strip=____) for th in tab.find("thead").find_all("th")]
isi = [[td.get_text(strip=True) for td in tr.find_all("____")] for tr in baris]
df = pd.____ (isi, columns=kepala)
```

Latihan 4.3 | Konversi tipe

```
df["Penduduk"] = pd.____ (df["Penduduk"], errors="____") # numerik, gagal jadi NaN
print(df.dtypes)
```

4.8.0.0.2 Tingkat Menengah (menulis sendiri)

Latihan 4.4 | Ekstraksi lengkap

Tulis satu fungsi `ekstrak(path)` yang menerima jalur berkas HTML dan mengembalikan DataFrame bertipe benar. Fungsi harus: (a) menguraikan laman; (b) mengambil tabel berdasarkan id; (c) mengubah ketiga kolom numerik; (d) mengembalikan pula daftar catatan metodologi. Uji pada `laman_contoh.html` dan tampilkan hasilnya.

Latihan 4.5 | Membetulkan kode yang salah

Kode berikut memuat empat kesalahan (`find_all` dipakai saat hanya satu elemen dicari, `th` dicari pada `tbody`, konversi tanpa `errors`, dan tidak ada pemeriksaan hasil konversi). Temukan dan perbaiki, lalu jelaskan tiap kesalahannya.

```
from bs4 import BeautifulSoup
import pandas as pd

with open("data/laman_contoh.html", encoding="utf-8") as f:
    sup = BeautifulSoup(f, "html.parser")

# (a) find_all mengembalikan daftar, bukan satu elemen
tab = sup.find_all("table", id="tabel-wilayah")

# (b) th dicari pada tbody, padahal berada pada thead
kepala = [th.get_text(strip=True) for th in tab.find("tbody").find_all("th")]

# (c) konversi tanpa errors -- gagal langsung menghentikan program
df["Penduduk"] = pd.to_numeric(df["Penduduk"])

# (d) hasil konversi tidak diperiksa
print(df.head())
```

Keluaran yang benar. Setelah diperbaiki, kode yang dipakai adalah sebagai berikut.

```
# (a) find mengembalikan satu elemen
tab = sup.find("table", id="tabel-wilayah")

# (b) kepala kolom diambil dari thead
kepala = [th.get_text(strip=True) for th in tab.find("thead").find_all("th")]

# (c) errors='coerce' agar kegagalan menjadi NaN, bukan menghentikan program
df["Penduduk"] = pd.to_numeric(df["Penduduk"], errors="coerce")

# (d) hasil konversi diperiksa
print("gagal dikonversi:", int(df["Penduduk"].isna().sum()))

Pemeriksaan yang wajib berhasil: df.shape bernilai (5, 4), dan int(df["Penduduk"].isna().sum()) bernilai 0.
```

Latihan 4.6 | Memeriksa laman statis atau dinamis

Baca `laman_contoh.html` sebagai teks mentah, lalu periksa apakah lima potongan teks yang tampak di tabel benar-benar ada pada sumbernya. Jelaskan mengapa pemeriksaan ini wajib dilakukan sebelum menulis pemilih.

Latihan 4.7 | Menakar beban perayapan

Hitung perkiraan lalu lintas dan lamanya untuk merayapi 50, 500, dan 5.000 laman dengan jeda 1 dan 3 detik. Sajikan sebagai tabel enam baris, lalu tentukan pada skala berapa Anda menganggap perayapan tidak lagi layak beserta alasannya.

4.8.0.0.3 Tingkat Lanjut (penilaian)**Latihan 4.8 | Merayapi seluruh halaman dan menggabungkannya**

- (a) Rayapi ketiga laman daftar, lalu telusuri seluruh tautan rincian — sembilan wilayah.
- (b) Gabungkan hasil daftar dan hasil rincian menjadi **satu** data frame. Periksa bahwa tidak ada wilayah yang hilang atau terduplikasi.
- (c) Ubah seluruh kolom angka menjadi numerik memakai fungsi `ambil_angka()` dari Contoh 4.12, lalu bandingkan hasilnya dengan `data/api_wilayah.json`. Apakah keduanya sama persis?
- (d) Bila ada selisih, jelaskan sebabnya dan tentukan sumber mana yang Anda percayai.

Latihan 4.9 | Pemilih CSS

Tulis satu baris pemilih CSS untuk masing-masing:

- (a) seluruh tautan di dalam tabel berkelas daftar;
- (b) elemen `dd` yang memiliki atribut `data-nilai`;
- (c) tautan Berikutnya saja, bukan Sebelumnya;
- (d) baris tabel pertama saja pada `tbody`.

Uji tiap jawaban dengan `select()`, lalu laporkan cacah elemen yang ditemukan.

Latihan 4.10 | Memilih antara API, umpan, dan perayapan

Anda diminta memantau rilis statistik wilayah setiap hari.

- (a) Bandingkan ketiga sumber pada folder `data/`: laman HTML, `api_wilayah.json`, dan `umpan_rilis.xml`. Peubah apa yang tersedia pada masing-masing?
- (b) Hitung berapa banyak baris kode yang diperlukan untuk memperoleh data yang sama dari ketiganya.
- (c) Sumber mana yang Anda pilih untuk **pemantauan harian**, dan mana untuk **pengambilan data lengkap sekali waktu**? Jelaskan alasannya masing-masing.

Latihan 4.11 | Mendeteksi perubahan tata letak

- (a) Jalankan fungsi `periksa()` dari Contoh 4.16 pada `laman_versi_baru.html` dengan kedua pemilih. Jelaskan perbedaan ketiga keluarannya.
- (b) Perluas fungsi itu agar juga memeriksa **cacah baris** dan **cacah kolom**, serta memberi peringatan bila cacah baris berubah lebih dari 20 persen dari nilai sebelumnya.
- (c) Jelaskan mengapa kekeliruan jenis “kepala kolom berubah” lebih berbahaya daripada “tabel tidak ditemukan”.

Latihan 4.12 | Padanan R

- (a) Ulangi Contoh 4.10 (merayapi laman berhalaman) dengan **rvest**. Petunjuk: pakai `html_element("nav.paginasi a.berikutnya")` dan `html_attr("href")`.
- (b) Bandingkan banyaknya baris kode kedua bahasa untuk tugas yang sama.
- (c) Jalankan `html_table()` pada `laman_versi_baru.html`. Apakah tipe datanya langsung benar? Bandingkan dengan hasil ekstraksi manual di Python.

Latihan 4.13 | Menilai kelayakan pengumpulan

Untuk masing-masing keadaan berikut, tentukan cara pengumpulan yang Anda sarankan (berkas, API, atau perayapan), sebutkan **satu pertimbangan etika**, dan **satu risiko mutu data**: (a) harga komoditas harian dari situs asosiasi pedagang; (b) data kependudukan dari portal resmi yang menyediakan unduhan CSV; (c) ulasan produk dari lokapasar; (d) cuaca per jam dari penyedia yang memiliki API berbayar.

4.9 Penugasan

Sebagai calon analis pada instansi statistik, susun laporan pengumpulan data dari laman tiruan `laman_contoh.html`, lengkap dengan penilaian etika, mutu, dan keterwakilannya.

Konteks. Sebuah unit hendak melengkapi data survei dengan sumber daring. Sebelum diputuskan, perlu dinilai kelayakan teknis maupun etikanya. Seluruh pekerjaan dilakukan atas laman tiruan; **tidak ada situs sungguhan yang dirayapi**.

Tugas 4.1 | Penilaian Etika

1. Uraikan isi `robots.txt` contoh pada Contoh 4.1. Susun fungsi yang memeriksa apakah suatu jalur boleh dirayapi, lalu uji pada lima jalur berbeda.
2. Bahas **empat pertimbangan** sebelum mengumpulkan data daring (ketentuan layanan, `robots.txt`, data pribadi, beban peladen). Untuk masing-masing, sebutkan akibat bila diabaikan.

Tugas 4.2 | Ekstraksi

3. Tulis fungsi `ekstrak(path)` yang mengembalikan DataFrame bertipe benar beserta catatan metodologi. Sertakan penanganan bila tabel tidak ditemukan.
4. Jalankan fungsi tersebut, tampilkan hasilnya, dan laporkan tipe tiap kolom **sebelum dan sesudah** konversi.
5. Susun fungsi permintaan HTTP yang santun (jeda, batas waktu, pengulangan berlipat, User-Agent jujur). **Tidak perlu dijalankan**; cukup dijelaskan tiap bagiannya.

Tugas 4.3 | Penyimpanan dan Penyajian

- 5b. Periksa apakah laman **statis atau dinamis** dengan mencari potongan teks pada sumbernya. Laporkan hasilnya dan jelaskan akibatnya bagi pendekatan yang dipilih.
- 5c. Takar **beban** yang ditimbulkan bila laman serupa dirayapi pada tiga skala berbeda. Sajikan tabel perkiraan lalu lintas dan lamanya.
- 5d. Simpan hasil ekstraksi beserta **berkas metadata** yang memuat sumber, waktu pengambilan, dimensi, dan catatan status data. Jelaskan mengapa metadata perlu disimpan terpisah.

Tugas 4.4 | Penilaian Mutu

6. Nilai mutu data hasil ekstraksi: kelengkapan, kewajaran rentang, dan konsistensi. Bandingkan pula terhadap `data_perbandingan.csv` yang memuat 40 wilayah — apakah kelima wilayah hasil ekstraksi mewakili sebarannya?
7. Bahas **keterwakilan** data hasil perayapan. Siapa yang tercakup dan siapa yang tidak? Mengapa banyaknya baris bukan ukuran mutu?
8. Susun laporan R Markdown berisi potongan kode Python, keluaran, dan interpretasi. Tutup dengan **ringkasan (maksimal 300 kata)** memuat: kelayakan sumber, temuan mutu, pertimbangan etika, dan penegasan bahwa laman maupun data merupakan tiruan.

4.10 Tambahan: Membaca Data dari Antarmuka Pemrograman (API)

Bagian ini melengkapi Contoh dengan tiga hal yang tidak dapat diperagakan pada laman tiruan: antarmuka pemrograman sebagai jalur yang lebih baik daripada perayapan, penerapan nyata pada situs berita detik.com, serta pemanfaatan Google Trends untuk nowcasting yang kini banyak dipakai kantor statistik dunia.

Bagian ini menghubungkan peladen sungguhan. Berbeda dari bagian Contoh, sebagian kode di sini benar-benar mengirim permintaan ke internet. Kerjakan dengan jeda yang cukup, jangan menjalankannya berulang tanpa perlu, dan hentikan segera bila menerima kode status 429. Bila jaringan tidak tersedia, cukup pelajari kodenya beserta keluaran yang dilampirkan.

4.10.0.1 Antarmuka Pemrograman (API)

Mengapa API lebih baik. Data yang dikembalikan sudah terstruktur, formatnya dijanjikan tetap, izin pema-
kaiannya jelas, dan bebannya terkendali karena penyedia sendiri yang mengatur batas laju.

Contoh 4.23 | Membaca tanggapan JSON dari API

Kode 4.23 • Python `json.loads` dan `pd.DataFrame`

```
import json
import pandas as pd

# Tanggapan API DITIRU sebagai teks; tidak ada permintaan jaringan.
tanggapan = '''
{
  "status": "ok",
  "sumber": "API tiruan untuk latihan",
  "data": [
    {"wilayah": "Wilayah A", "tahun": 2026, "nilai": 72.4},
```

```

{"wilayah": "Wilayah B", "tahun": 2026, "nilai": 65.1},
{"wilayah": "Wilayah C", "tahun": 2026, "nilai": 81.7}
]
}
...

hasil = json.loads(tanggapan)
print("status:", hasil["status"], "| sumber:", hasil["sumber"])

df_api = pd.DataFrame(hasil["data"])
print(df_api)
print(df_api.dtypes)

```

Penjelasan Kode. Teks JSON diubah menjadi objek Python memakai `json.loads`. Kunci status diperiksa lebih dahulu, lalu bagian data yang berupa daftar kamus diubah langsung menjadi `DataFrame`.

Interpretasi Output

Bandingkan dengan hasil perayapan pada Contoh 4.3. Di sini tipe kolom sudah benar sejak awal tanpa konversi apa pun, sebab JSON membawa informasi tipe. Perhatikan pula kebiasaan memeriksa kunci status sebelum mengolah datanya: tanggapan yang gagal kerap tetap berkode 200 tetapi berisi pesan galat di dalamnya.

4.10.0.2 Penerapan Nyata: Indeks Berita detik.com

Bagian ini menerapkan seluruh langkah pada situs sungguhan. Urutannya persis seperti yang diajarkan: periksa ketentuan lebih dahulu, telusuri struktur, baru ekstraksi.

Contoh 4.24 | Langkah pertama: membaca robots.txt detik.com

Kode 4.24 • Python mengambil robots.txt sungguhan

```

import requests

kepala = {"User-Agent": "PraktikumSTIS/1.0 (keperluan pendidikan)"}
r = requests.get("https://news.detik.com/robots.txt", headers=kepala, timeout=20)
print("status:", r.status_code)
print(r.text)

```

Penjelasan Kode. Berkas robots.txt diambil dari akar subdomain news.detik.com. Ini adalah satu permintaan yang ringan dan justru dianjurkan sebelum permintaan lainnya.

Interpretasi Output

Isinya perlu dibaca dengan cermat karena mengandung pelajaran penting. Sebagian besar larangan — termasuk larangan atas jalur indeks — ditujukan khusus bagi User-agent bernama Googlebot. Sementara itu blok User-agent berlambang bintang, yang mencakup perayap kita, hanya memuat Allow garis miring, artinya seluruh situs diizinkan. Jadi menurut robots.txt, pengambilan halaman indeks oleh perayap biasa tidak dilarang.

Diizinkan robots.txt belum tentu diizinkan seluruhnya. Kesimpulan bahwa robots.txt mengizinkan bukan akhir pemeriksaan. Ketentuan layanan situs, hak cipta atas isi berita, dan batas kewajaran beban tetap berlaku. Isi berita dilindungi hak cipta sehingga tidak boleh diterbitkan ulang; yang lazim dipakai untuk keperluan statistik adalah **metadata** seperti judul, waktu terbit, dan kanal — bukan naskah beritanya. Modul ini pun hanya mengambil metadata.

Contoh 4.25 | Menelusuri struktur halaman indeks**Kode 4.25 • Python** menelusuri sebelum mengekstrak

```
from bs4 import BeautifulSoup
import time

time.sleep(2)                                # jeda sebelum permintaan berikutnya
r = requests.get("https://news.detik.com/indeks", headers=kepala, timeout=25)
print("status:", r.status_code, "| ukuran:", len(r.text), "karakter")

sup = BeautifulSoup(r.text, "html.parser")
artikel = sup.select("article.list-content__item")
print("artikel ditemukan:", len(artikel))

# memeriksa susunan satu artikel sebelum mengekstrak semuanya
for el in artikel[0].find_all(True, class_=True):
    print(" ", el.name, " ".join(el.get("class")))
```

Penjelasan Kode. Jeda dua detik diberikan sebelum permintaan. Halaman indeks diambil lalu diuraikan. Pilih CSS article titik list-content dua garis bawah item menargetkan wadah tiap artikel. Perulangan terakhir mencetak seluruh elemen beratribut class di dalam satu artikel, sehingga susunannya terbaca sebelum memilih ditetapkan.

Interpretasi Output

Langkah memeriksa satu artikel lebih dahulu ini sering dilewati pemula, padahal justru di sinilah pemilih yang benar ditemukan. Dari keluarannya terlihat bahwa judul berada pada h3 berkelas media dua garis bawah title, tautan pada a berkelas media dua garis bawah link, dan waktu pada div berkelas media dua garis bawah date. Halaman indeks memuat dua puluh artikel.

Contoh 4.26 | Mengekstrak metadata berita**Kode 4.26 · Python** ekstraksi bertahan galat

```
import pandas as pd

def ambil_metadata(artikel):
    """Mengambil judul, tautan, dan waktu; mengembalikan None bila tak lengkap."""
    tautan = artikel.select_one("h3.media__title a") or artikel.select_one("a")
    if tautan is None:
        return None
    waktu = artikel.select_one("div.media__date span")
    return {
        "judul": tautan.get_text(strip=True),
        "tautan": tautan.get("href", ""),
        "waktu": (waktu.get("title") if waktu is not None and waktu.has_attr("title")
                  else (waktu.get_text(strip=True) if waktu is not None else None)),
    }

baris = [m for m in (ambil_metadata(a) for a in artikel) if m is not None]
berita = pd.DataFrame(baris)
print("terkumpul:", len(berita), "dari", len(artikel), "artikel")
print(berita[["judul", "waktu"]].head(5).to_string(index=False))
```

Penjelasan Kode. Fungsi `ambil_metadata` memakai pemilih cadangan lewat operator `or`, sehingga artikel yang susunannya sedikit berbeda tetap tertangani. Bila tautan sama sekali tidak ditemukan, fungsi mengembalikan `None` dan baris itu disaring keluar. Waktu diambil dari atribut `title` bila ada, sebab atribut itu memuat tanggal lengkap sedangkan teksnya kadang hanya berupa keterangan relatif.

Interpretasi Output

Perhatikan bahwa banyaknya baris terkumpul dibandingkan terhadap banyaknya artikel. Selisih di antara keduanya adalah tanda bahwa sebagian artikel bersusunan berbeda — misalnya sisipan iklan

atau berita bergambar — dan itu perlu diketahui, bukan disembunyikan. Perlu ditegaskan pula bahwa yang diambil hanya metadata; naskah beritanya tidak diambil karena dilindungi hak cipta.

Pemilih berumur pendek. Pemilih seperti `article.list-content__item` berlaku pada saat modul ini disusun. Situs berita kerap mengubah tata letaknya, sehingga kode yang hari ini berjalan dapat gagal beberapa bulan kemudian. Inilah kerapuhan mendasar perayapan yang tidak dimiliki API. Karena itu kode perayapan wajib memeriksa apakah hasilnya kosong, dan tidak boleh mengandaikan susunan laman tetap.

Contoh 4.27 | Meringkas metadata menjadi statistik

Kode 4.27 • Python menguraikan waktu dan kanal

```
berita["waktu_parsed"] = pd.to_datetime(
    berita["waktu"].str.replace(" WIB", "", regex=False),
    format="%A, %d %b %Y %H:%M", errors="coerce")

print("gagal diuraikan:", int(berita["waktu_parsed"].isna().sum()))

# kanal diambil dari bagian alamat
berita["kanal"] = berita["tautan"].str.extract(r"https://([a-z]+)\.detik\.com")
print(berita["kanal"].value_counts())
```

Penjelasan Kode. Akhiran WIB dibuang lalu teks waktu diubah menjadi tipe waktu dengan format yang ditetapkan; argumen `errors` bernilai `coerce` membuat kegagalan menjadi `NaT` sehingga dapat dihitung. Kanal ditarik dari alamat memakai ekspresi reguler yang menangkap bagian subdomain.

Interpretasi Output

Penguraian waktu kerap gagal karena nama hari dan bulan bergantung pada pengaturan bahasa sistem. Bila hasilnya banyak `NaT`, atur locale ke bahasa Indonesia atau uraikan secara manual. Kegagalan semacam ini wajib dilaporkan, bukan dibiarkan — sebab waktu terbit adalah peubah kunci bila metadata ini hendak dipakai untuk analisis kecenderungan pemberitaan.

4.10.0.3 Google Trends dan Nowcasting

Google Trends menyediakan indeks minat pencarian menurut waktu dan wilayah. Bagi statistik resmi, daya tariknya terletak pada ketepatan waktu: data pencarian tersedia hampir seketika, sedangkan indikator resmi lazimnya terbit berminggu bahkan berbulan setelah periode acuannya.

Gagasan nowcasting adalah menduga nilai indikator pada periode berjalan yang datanya belum terbit, dengan memanfaatkan penunjuk yang tersedia lebih cepat. Istilah ini dipopulerkan Choi dan Varian melalui makalah berjudul memperkirakan masa kini dengan Google Trends, dan kini dipakai berbagai kantor statistik untuk menduga pengangguran, penjualan eceran, maupun pariwisata.

Nowcasting dibandingkan peramalan.

	Nowcasting	Peramalan
Sasaran	periode berjalan atau baru lewat	periode mendatang
Masalah	data belum terbit	masa depan belum terjadi
Penunjuk	tersedia lebih cepat	pola masa lalu
Verifikasi	dapat diperiksa saat data terbit	menunggu waktu berlalu

Indeks Google Trends bukan cacah pencarian. Angka yang dikembalikan **bukan** banyaknya pencarian, melainkan *indeks relatif* berskala 0 sampai 100 terhadap titik tertinggi pada rentang yang diminta. Akibatnya angka menjadi **tidak sebanding** antar-permintaan: mengubah rentang waktu, wilayah, atau daftar kata kunci akan mengubah seluruh angkanya. Membandingkan hasil dua permintaan berbeda seolah-olah setara adalah kekeliruan yang sering terjadi. Selain itu, nilai bertanda kurang dari satu perlu ditangani karena terbaca sebagai teks, bukan angka.

Contoh 4.28 | Mengambil data Google Trends

Kode 4.28 • R gtrends dengan penanganan galat

```
suppressPackageStartupMessages(library(gtrendsR))

hasil <- tryCatch(
  gtrends("lowongan kerja", geo = "ID", time = "today 12-m", onlyInterest = TRUE),
  error = function(e) { message("pengambilan gagal: ", conditionMessage(e)); NULL })

if (!is.null(hasil)) {
  tren <- hasil$interest_over_time
  tren$hits <- as.numeric(gsub("<1", "0", as.character(tren$hits))) # tangani '<1'
  cat("baris:", nrow(tren),
      "\n| rentang:", format(min(tren$date)), "s.d.", format(max(tren$date)), "\n")
  cat("rata indeks:", round(mean(tren$hits, na.rm = TRUE), 1), "\n")
}
```


Penjelasan Kode. Permintaan dibungkus tryCatch karena Google Trends menerapkan batas laju dan kerap menolak permintaan yang terlalu berdekatan. Argumen geo membatasi wilayah ke Indonesia dan time meminta rentang dua belas bulan terakhir. Nilai bertanda kurang dari satu diganti nol lebih dahulu sebelum diubah menjadi numerik, sebab tanpa itu seluruh kolom akan tetap bertipe teks.

Interpretasi Output

Penanganan galat di sini bukan hiasan. Permintaan berturut-turut dalam waktu singkat lazim ditolak, dan kode yang tidak menanganinya akan berhenti mendadak di tengah pengolahan. Perhatikan pula bahwa data mingguan untuk rentang dua belas bulan menghasilkan sekitar lima puluh tiga baris; meminta rentang lebih panjang akan mengubah satuan waktunya menjadi bulanan.

Contoh 4.29 | Peragaan nowcasting sederhana

Kode 4.29 • R lm sebagai model nowcasting

```
if (!is.null(hasil)) {
  set.seed(2026)
  # sasaran TIRUAN yang sengaja dibuat terkait dengan tren pencarian
  tren$sasaran <- 5.2 + 0.03 * scale(tren$hits)[, 1] + rnorm(nrow(tren), 0, 0.05)

  m0 <- lm(sasaran ~ 1, data = tren)          # model tanpa penunjuk
  m1 <- lm(sasaran ~ hits, data = tren)       # model dengan penunjuk

  rmse <- function(m) sqrt(mean(residuals(m)^2))
  round(c(R2_dengan_tren = summary(m1)$r.squared,
        RMSE_tanpa = rmse(m0), RMSE_dengan = rmse(m1),
        penurunan_persen = 100 * (rmse(m0) - rmse(m1)) / rmse(m0)), 4)
}
```

Penjelasan Kode. Peubah sasaran dibangkitkan agar sengaja berkaitan dengan indeks pencarian, sehingga peragaannya terkendali. Dua model dibandingkan: model tanpa penunjuk yang hanya berisi konstanta, dan model yang menyertakan indeks pencarian. Akar rerata kuadrat sisaan dihitung bagi keduanya, lalu penurunannya dinyatakan dalam persen.

Interpretasi Output

Penurunan galat inilah yang menjadi ukuran manfaat sebuah penunjuk. Perlu ditegaskan bahwa peubah sasaran di sini adalah tiruan yang memang dibuat berkaitan, sehingga hasilnya tidak membuktikan apa pun mengenai kegunaan Google Trends pada indikator sungguhan. Pada penerapan nyata,

sasaran diganti indikator resmi dan penilaiannya wajib memakai data yang tidak dipakai melatih model.

Tips dan Praktik Baik. Untuk penilaian nowcasting yang valid, pembagian data harus mengikuti **urutan waktu**: model dilatih pada periode awal dan diuji pada periode akhir. Membagi secara acak akan membocorkan informasi masa depan ke dalam pelatihan sehingga kinerjanya tampak jauh lebih baik daripada sebenarnya. Kekeliruan ini termasuk yang paling sering terjadi pada nowcasting.

Empat keterbatasan yang wajib dilaporkan.

Keterwakilan	hanya mencakup pengguna internet yang memakai Google; kelompok tanpa akses tidak terwakili
Kesebandingan	indeks berskala relatif; berubah bila rentang atau kata kunci diubah
Kesinambungan	algoritma dan penyaringan Google dapat berubah tanpa pemberitahuan
Ambiguitas kata	satu kata kunci dapat bermakna ganda, sehingga minat tercampur

Karena keempat keterbatasan itu, praktik yang dianjurkan adalah memakai Google Trends sebagai pelengkap, bukan pengganti. Indikator resmi tetap menjadi acuan, sedangkan penunjuk pencarian dipakai untuk menduga nilai sementara pada periode yang datanya belum terbit, lalu diperiksa kembali ketika data resmi keluar.

Tips dan Praktik Baik. Urutan yang dianjurkan ketika mencari data: berkas unduhan, lalu API, baru perayapan. Setiap langkah ke kanan menambah usaha, menambah kerapuhan, dan menambah risiko. Google Trends sendiri tergolong API sehingga lebih baik daripada perayapan, tetapi keterbatasannya menuntut kehati-hatian penafsiran.

4.11 Ringkasan Modul

Daftar periksa penguasaan:

- ☐ Saya dapat membedakan pengumpulan lewat berkas, API, dan perayapan.
- ☐ Saya memeriksa ketentuan layanan dan robots.txt sebelum menulis kode.
- ☐ Saya memahami bahwa robots.txt adalah permintaan etis, bukan penghalang teknis.
- ☐ Saya dapat menelusuri struktur HTML dan menemukan elemen berdasarkan penanda.

- ☐ Saya dapat mengekstrak tabel menjadi DataFrame bertipe benar.
- ☐ Saya selalu memakai errors coerce lalu memeriksa hasil konversinya.
- ☐ Saya dapat menangani kegagalan permintaan beserta jeda berlipat.
- ☐ Saya menilai keterwakilan data, bukan sekadar banyaknya baris.

Kartu Referensi Cepat.

Keperluan	Perintah
nMenguraikan HTML	<code>BeautifulSoup(f, "html.parser")</code>
nSatu elemen	<code>sup.find("table", id="...")</code>
nSemua elemen	<code>sup.find_all("tr")</code>
nPemilih CSS	<code>sup.select("ul.kelas li")</code>
nAmbil teks	<code>el.get_text(strip=True)</code>
nAtribut	<code>a["href"]</code>
nKe DataFrame	<code>pd.DataFrame(isi, columns=kepala)</code>
nKonversi aman	<code>pd.to_numeric(x, errors="coerce")</code>
nPermintaan HTTP	<code>requests.get(url, headers=, timeout=)</code>
nMembaca JSON	<code>json.loads(teks)</code>

Alur kerja yang dianjurkan. Berikut urutan yang sebaiknya diikuti pada tiap pekerjaan pengumpulan data daring.

Tujuh langkah pengumpulan yang bertanggung jawab.

- | | | |
|----------|-------------------------|--|
| 1 | Rumuskan kebutuhan | peubah apa yang benar-benar diperlukan |
| 2 | Cari berkas atau API | dahulukan sumber terstruktur |
| 3 | Periksa ketentuan | robots.txt, ketentuan layanan, hak cipta |
| 4 | Takar beban | perkirakan lalu lintas dan lamanya |
| 5 | Telusuri struktur | periksa satu laman sebelum mengambil semua |
| 6 | Ekstrak dan periksa | konversi tipe, hitung kegagalan |
| 7 | Simpan beserta metadata | sumber, waktu, cakupan, status data |

Bila ragu, tanyakan. Langkah yang paling sering dilupakan adalah menghubungi penyedia data secara langsung. Banyak instansi bersedia memberikan data dalam bentuk berkas bila diminta untuk keperluan penelitian atau pendidikan. Satu surel dapat menghemat berhari-hari pekerjaan perayapan sekaligus menghindarkan seluruh persoalan etika dan hukumnya.

Modul ini membahas cara memperoleh data beserta tanggung jawab yang menyertainya. Prinsip yang perlu

dibawa: dahulukan berkas dan API sebelum perayapan, patuhi ketentuan meski secara teknis dapat diabaikan, periksa hasil konversi tipe agar tidak kehilangan data diam-diam, dan nilai keterwakilan sebelum menilai jumlah. Modul berikutnya membahas prapemrosesan data, yaitu tahap yang lazimnya menyita porsi waktu terbesar dalam CRISP-DM.

4.12 Referensi

Mitchell, R. (2018). *Web Scraping with Python* (2nd ed.). O'Reilly Media.

Richardson, L. (2024). *Beautiful Soup Documentation*. Crummy.

Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann.

UNECE (2013). *What Does Big Data Mean for Official Statistics?* United Nations Economic Commission for Europe.

Krotov, V., & Silva, L. (2018). Legality and ethics of web scraping. *Americas Conference on Information Systems (AMCIS) 2018 Proceedings*.

MODUL 5

PRAPEMROSESAN DATA

Pertemuan Minggu ke-5

Sub-CPMK 5: Mahasiswa mampu melakukan pembersihan, integrasi, reduksi, dan transformasi data memakai kerangka resep yang dapat diulang, serta menghindari kebocoran data pada seluruh tahapannya.

Penyusun: Achmad Syahrul Choir



Daftar Isi

5.1	Deskripsi Modul	143
5.2	Tujuan Praktikum	143
5.3	Alat dan Bahan	143
5.4	Instruksi Pengerjaan	144
5.5	Teori Singkat	144
5.5.0.1	Kedudukan pada CRISP-DM	144
5.5.0.2	Empat Jenis Cacat Data	144
5.5.0.3	Pola Nilai Hilang	145
5.5.0.4	Gagasan Resep	145
5.5.0.5	Urutan Langkah Berpengaruh	145
5.5.0.6	Ketidakseimbangan Kelas	146
5.5.0.7	Menangani Taraf Kategori yang Jarang	146
5.5.0.8	Kebocoran Data	146
5.6	Contoh Praktikum dengan R	148
5.7	Latihan Mandiri	163
5.8	Penugasan	166
5.9	Tambahan: Menakar Akibat Kebocoran Data	167
5.10	Ringkasan Modul	170
5.11	Referensi	171

5.1 Deskripsi Modul

Dalam CRISP-DM, tahap penyiapan data lazimnya menyita porsi waktu terbesar, kerap disebut mencapai tiga perempat keseluruhan pekerjaan. Angka itu bukan pertanda ketidakefisienan, melainkan cerminan kenyataan bahwa data lapangan hampir selalu tidak siap pakai.

Modul ini membahas empat pekerjaan pada tahap tersebut: pembersihan, penggabungan, reduksi, dan transformasi. Yang membedakannya dari pengolahan biasa adalah kerangka yang dipakai. Seluruh langkah disusun sebagai **resep**, yaitu rangkaian tindakan yang dapat diperiksa, diulang, dan yang terpenting dipelajari dari data latih saja lalu diterapkan ke data uji.

Perbedaan terakhir itu tampak sepele tetapi menentukan. Menghitung nilai tengah untuk imputasi dari seluruh data, bukan dari data latih saja, membuat kinerja model tampak lebih baik daripada sebenarnya. Bagian Tambahan menakar besarnya akibat itu dengan angka, dan hasilnya mungkin mengejutkan.

Tentang data. Berkas data pada modul ini adalah **data simulasi** (`set.seed(2026)`), **bukan data nyata** dan **bukan data resmi** instansi mana pun. Data sengaja dibuat memuat cacat agar dapat dilatih penanganannya.

5.2 Tujuan Praktikum

Setelah menyelesaikan praktikum ini secara tuntas, mahasiswa diharapkan mampu:

1. **Mengenal**i empat jenis cacat data: nilai hilang, pencilan, ketidakkonsistenan, dan duplikasi.
2. **Menyusun** resep prapemrosesan sebagai rangkaian langkah yang dapat diperiksa.
3. **Memilih** cara imputasi yang sesuai dengan tipe peubah.
4. **Menerapkan** transformasi untuk menangani kemencengan dan perbedaan skala.
5. **Menggabungkan** data dari beberapa sumber dengan kunci yang tepat.
6. **Menjelaskan** apa itu kebocoran data dan menakar akibatnya.
7. **Mencegah** kebocoran dengan menempatkan seluruh langkah di dalam resep.

5.3 Alat dan Bahan

Perangkat dan bahan yang diperlukan:

- **R** (RStudio) dengan paket `recipes`, `rsample`, `themis`, dan `dplyr`.

- Pasang dengan `install.packages(c("recipes", "rsample", "themis", "dplyr"))`.
- Berkas data: `data/data_kotor.csv` dan `data/data_referensi.csv`.

5.4 Instruksi Pengerjaan

Ikuti tahapan berikut:

Sebelum praktikum: pasang ketiga paket; siapkan kedua berkas data pada subfolder `data/`; baca bagian Teori mengenai kebocoran data.

Selama praktikum: kerjakan Contoh berurutan. Perhatikan bahwa urutan langkah pada resep berpengaruh terhadap hasil, sehingga jangan mengubahnya tanpa alasan.

Pembagian kerja: Contoh dan Latihan dikerjakan di kelas; **Penugasan dikerjakan di luar sesi.**

Pengumpulan: susun laporan R Markdown, render ke PDF atau HTML, kumpulkan melalui Google Classroom.

5.5 Teori Singkat

5.5.0.1 Kedudukan pada CRISP-DM

Modul ini menempati tahap ketiga, yaitu Data Preparation. Tahap ini menerima hasil pengenalan data dari tahap kedua dan menyerahkan data siap pakai ke tahap pemodelan. Perlu diingat bahwa panah CRISP-DM berjalan dua arah: kegagalan pada tahap evaluasi kerap mengembalikan pekerjaan ke tahap ini, bukan ke pemilihan model.

5.5.0.2 Empat Jenis Cacat Data

Cacat dan cara mengenalinya.

Cacat	Cara mengenali	Pertimbangan
Nilai hilang	<code>colSums(is.na())</code>	pola hilangnya acak atau tidak?
Pencilan	IQR, skor-z, boxplot	kekeliruan atau amatan sah?
Tidak konsisten	<code>table()</code> , rentang	ejaan, satuan, kategori bertaut
Duplikasi	<code>duplicated()</code>	ganda sungguhan atau kebetulan sama?

Keempatnya menuntut keputusan, bukan sekadar penerapan rumus. Pencilan misalnya tidak otomatis di-buang: pendapatan yang jauh di atas lainnya bisa saja sah dan justru paling menarik. Membuangnya tanpa alasan berarti membuang informasi sekaligus membiaskan hasil.

5.5.0.3 Pola Nilai Hilang

Tiga pola dan akibatnya.

MCAR	hilang sepenuhnya acak	imputasi sederhana memadai
MAR	hilang bergantung peubah lain yang teramati	imputasi berbasis model lebih tepat
MNAR	hilang bergantung nilainya sendiri	imputasi apa pun berisiko bias

Pembedaan ini penting karena menentukan sah tidaknya penanganan. Pendapatan yang tidak dijawab justru oleh mereka yang berpendapatan sangat tinggi adalah contoh MNAR; mengimputasinya dengan nilai tengah akan menarik dugaan ke bawah secara sistematis. Sayangnya MNAR tidak dapat diperiksa dari data itu sendiri, sehingga penilaiannya bersandar pada pengetahuan bidang.

5.5.0.4 Gagasan Resep

Paket recipes memisahkan **penetapan langkah** dari **pelaksanaannya**. Resep disusun lebih dahulu sebagai rangkaian langkah, lalu dipelajari dari data latih melalui prep, baru diterapkan melalui bake. Pemisahan inilah yang mencegah kebocoran secara rancangan, bukan mengandalkan kedisiplinan penulis kode.

Tiga tahap pemakaian resep.

<code>recipe()</code>	menetapkan peran peubah	belum ada perhitungan
<code>prep()</code>	mempelajari dari data latih	nilai tengah, simpangan baku, taraf faktor dihitung di sini
<code>bake()</code>	menerapkan ke data mana pun	memakai nilai yang sudah dipelajari

5.5.0.5 Urutan Langkah Berpengaruh

Urutan pada resep bukan soal selera. Imputasi harus mendahului pembakuan, sebab nilai hilang membuat rata-rata dan simpangan baku tak terdefinisi. Pengodean peubah kategori harus mendahului pembakuan bila kolom hasilnya ikut dibakukan. Transformasi pengurang kemencengan sebaiknya mendahului pembakuan agar pembakuannya bekerja pada sebaran yang sudah lebih simetris.

Tips dan Praktik Baik. Urutan yang lazim dan aman: tangani nilai hilang, tangani peubah berkelas jarang, transformasikan kemencengan, kodekan kategori, buang peubah berragam nol, lalu bakukan. Bila ragu, jalankan `prep()` lalu periksa hasil `bake()` pada beberapa baris pertama.

5.5.0.6 Ketidakseimbangan Kelas

Ketika kelas sasaran sangat timpang — misalnya hanya lima belas persen amatan berkelas positif — model cenderung memihak kelas mayoritas. Akurasi menjadi ukuran yang menyesatkan, sebab menebak seluruhnya sebagai mayoritas sudah menghasilkan delapan puluh lima persen tanpa mempelajari apa pun.

Tiga cara menangani ketimpangan.

Cara	Tindakan	Risiko
Penyusutan	mengurangi kelas mayoritas	informasi terbuang
Penggandaan	memperbanyak kelas minoritas	pemodelan berlebih (<i>overfitting</i>) pada salinan
SMOTE	membuat amatan minoritas sintetis	amatan buatan bisa tak masuk akal
Pembobotan	memberi bobot lebih pada minoritas	tidak semua model mendukung

Menyeimbangkan seluruh data. Penyeimbangan kelas **hanya boleh dikenakan pada data latih**. Bila dikenakan sebelum pembagian, salinan atau amatan sintetis dari data latih dapat muncul di data uji, sehingga model menilai dirinya atas amatan yang sudah dipelajarinya. Data uji harus tetap mencerminkan keadaan sesungguhnya, termasuk ketimpangannya — sebab begitulah data baru kelak akan datang.

5.5.0.7 Menangani Taraf Kategori yang Jarang

Peubah kategori kerap memuat taraf yang hanya dimiliki segelintir amatan. Taraf semacam itu menimbulkan dua persoalan: kolom bonekanya nyaris seluruhnya nol sehingga tak informatif, dan taraf itu dapat sama sekali tidak muncul pada data latih sehingga model gagal ketika menemuinya di data baru.

Tips dan Praktik Baik. Langkah `step_other()` menggabungkan taraf yang proporsinya di bawah ambang menjadi satu taraf bernama *other*. Yang penting, penggabungan itu dipelajari dari data latih, sehingga taraf tak terduga pada data uji otomatis masuk ke *other* alih-alih menimbulkan galat. Inilah salah satu keunggulan resep yang sukar ditiru dengan penulisan langkah secara manual.

5.5.0.8 Kebocoran Data

Kebocoran data terjadi ketika informasi dari data uji ikut memengaruhi pembentukan model. Akibatnya kinerja pada data uji menjadi terlalu bagus, dan kekecewaan baru muncul ketika model dipakai pada data sungguhan.

Tiga sumber kebocoran yang paling sering.

1. **Prapemrosesan sebelum pembagian.** Menghitung nilai tengah, simpangan baku, atau taraf faktor dari seluruh data.
2. **Seleksi peubah memakai seluruh data.** Memilih peubah yang paling berkorelasi dengan sasaran sebelum data dibagi — inilah bentuk yang paling parah akibatnya.
3. **Penyeimbangan kelas pada seluruh data.** Menerapkan SMOTE atau penggandaan sebelum pembagian, sehingga salinan amatan latih muncul di data uji.

Ketiganya memiliki ciri yang sama: seluruhnya tampak wajar bila dibaca sepintas, dan seluruhnya menghasilkan angka yang menyenangkan. Justru karena itulah kebocoran sukar terdeteksi — ia tidak menimbulkan galat, hanya menghasilkan kesimpulan yang keliru.

5.6 Contoh Praktikum dengan R

Contoh 5.1 | Mengenali cacat pada data

Kode 5.1 • R pemeriksaan empat jenis cacat

```
kotor <- read.csv("data/data_kotor.csv", stringsAsFactors = TRUE)
dim(kotor)

[1] 1015    9

# empat pemeriksaan sekaligus
cat("nilai hilang total :", sum(is.na(kotor)), "\n")

nilai hilang total : 315

cat("baris ganda      :", sum(duplicated(kotor)), "\n")

baris ganda      : 14

cat("id ganda        :", sum(duplicated(kotor$id)), "\n")

id ganda        : 15

q <- quantile(kotor$pendapatan, c(.25, .75), na.rm = TRUE)
batas <- q[2] + 1.5 * diff(q)
cat("pencilan pendapatan:", sum(kotor$pendapatan > batas, na.rm = TRUE),
    sprintf("(%.1f persen)\n", 100 * mean(kotor$pendapatan > batas, na.rm = TRUE)))

pencilan pendapatan: 39 (4.8 persen)
```

Penjelasan Kode. Empat pemeriksaan dijalankan berurutan. Fungsi `is.na` yang dijumlahkan menghasilkan cacah nilai hilang seluruh tabel. Fungsi `duplicated` dikenakan dua kali: pada seluruh baris untuk mencari baris kembar, dan pada kolom `id` untuk mencari penanda ganda. Batas pencilan ditetapkan dengan aturan simpangan kuartil.

Interpretasi Output

Pemeriksaan pada kolom `id` perlu dibedakan dari pemeriksaan baris kembar. Baris yang seluruhnya sama boleh jadi memang pencatatan ganda, sedangkan `id` yang sama pada baris berbeda menandakan persoalan yang lebih serius pada sumber datanya. Adapun pencilan dicacah dan dilaporkan persentasenya lebih dahulu, sebelum diputuskan apa yang hendak dilakukan terhadapnya.

Contoh 5.2 | Memeriksa pola nilai hilang

Kode 5.2 • R colSums dan tabel silang keterisian

```
hilang <- colSums(is.na(kotor))
```

```
print(hilang[hilang > 0])
```

```
pendapatan skor_literasi
199          116
```

```
# apakah hilangnya berkaitan dengan peubah lain?
```

```
kotor$ada_pendapatan <- !is.na(kotor$pendapatan)
```

```
if (any(!kotor$ada_pendapatan)) {
```

```
  print(round(prop.table(table(kotor$wilayah, kotor$ada_pendapatan), 1), 3))
```

```
} else {
```

```
  cat("tidak ada nilai hilang pada pendapatan\n")
```

```
}
```

```
      FALSE TRUE
Perdesaan 0.300 0.700
PERDESAAN 0.364 0.636
Perkotaan 0.094 0.906
PERKOTAAN 0.071 0.929
```

```
# proporsi hilang per kolom
```

```
round(sort(hilang[hilang > 0] / nrow(kotor), decreasing = TRUE), 4)
```

```
pendapatan skor_literasi
0.1961      0.1143
```

Penjelasan Kode. Cacah nilai hilang per kolom dihitung lalu disaring agar hanya kolom bermasalah yang tampil. Peubah penanda keterisian dibentuk, lalu ditabulasikan silang terhadap wilayah untuk memeriksa apakah kehilangannya merata. Proporsi hilang dihitung dan diurutkan menurun.

Interpretasi Output

Tabel silang inilah cara memeriksa dugaan MAR: bila proporsi terisi berbeda nyata antar-wilayah, kehilangannya bergantung pada wilayah dan tergolong MAR, sehingga imputasi sebaiknya memperhatikan wilayah. Perlu diingat bahwa MNAR tidak dapat diperiksa dengan cara ini — ketiadaan bukti MAR bukan bukti MCAR.

Contoh 5.3 | Membagi data sebelum melakukan apa pun**Kode 5.3 • R** initial_split dengan strata

```
suppressPackageStartupMessages({library(recipes); library(rsample)})
```

```
kotor$ada_pendapatan <- NULL # buang kolom bantu
```

```
set.seed(2026)
bagi <- initial_split(kotor, prop = 0.75, strata = target)
lat <- training(bagi)
uji <- testing(bagi)

cat("latih:", nrow(lat), "| uji:", nrow(uji), "\n")
```

```
latih: 761 | uji: 254
```

```
round(rbind(latih = prop.table(table(lat$target)),
            uji = prop.table(table(uji$target))), 4)
```

```
      tidak      ya
latih 0.4179 0.5821
uji    0.4173 0.5827
```

Penjelasan Kode. Data dibagi menjadi tiga perempat latih dan seperempat uji. Argumen strata memastikan proporsi kelas sasaran terjaga pada kedua bagian, yang penting ketika kelasnya tidak seimbang.

Interpretasi Output

Perhatikan bahwa pembagian dilakukan **sebelum** prapemrosesan apa pun. Inilah urutan yang benar dan tidak boleh dibalik. Perhatikan pula bahwa proporsi kelas pada kedua bagian nyaris sama berkat penstrataan; tanpa itu, bagian uji dapat kebetulan memuat terlalu sedikit kelas minoritas sehingga penilaiannya menjadi tidak stabil.

Contoh 5.4 | Menyusun resep bertahap

Kode 5.4 • R recipe dengan tujuh langkah

```
rec <- recipe(target ~ ., data = lat) |>
  update_role(id, new_role = "penanda") |>      # id bukan prediktor
  step_impute_median(all_numeric_predictors()) |> # 1. nilai hilang numerik
  step_impute_mode(all_nominal_predictors()) |>  # 2. nilai hilang kategori
  step_YeoJohnson(all_numeric_predictors()) |>  # 3. kurangi kemencengan
  step_other(all_nominal_predictors(), threshold = 0.05) |> # 4. kelas jarang
  step_dummy(all_nominal_predictors()) |>        # 5. pengodean
  step_zv(all_predictors()) |>                  # 6. buang ragam nol
  step_normalize(all_numeric_predictors())        # 7. pembakuan
```

```
rec
```

Penjelasan Kode. Resep disusun dengan sasaran target dan seluruh kolom lain sebagai calon prediktor. Kolom id diubah perannya menjadi penanda agar tidak ikut dimodelkan. Tujuh langkah ditambahkan berurutan, masing-masing memakai pemilih yang menargetkan kelompok kolom tertentu, bukan menyebutkan namanya satu per satu.

Interpretasi Output

Perhatikan bahwa mencetak resep hanya menampilkan **rencana**; belum ada perhitungan yang dijalankan. Perhatikan pula pemakaian pemilih seperti `all_numeric_predictors`, yang membuat resep tetap sah meski kolomnya bertambah atau berkurang. Mengubah peran id sangat penting: penanda yang ikut dimodelkan dapat menghasilkan model yang seolah sangat baik padahal hanya menghafal nomor.

Contoh 5.5 | Mempelajari resep dan menerapkannya

Kode 5.5 • R prep lalu bake

```
siap <- prep(rec, training = lat)      # DIPELAJARI dari data latih saja

lat_siap <- bake(siap, new_data = lat)
uji_siap <- bake(siap, new_data = uji)

cat("latih:", nrow(lat_siap), "x", ncol(lat_siap),
    "| uji:", nrow(uji_siap), "x", ncol(uji_siap), "\n")

latih: 761 x 13 | uji: 254 x 13

cat("NA tersisa - latih:", anyNA(lat_siap), "| uji:", anyNA(uji_siap), "\n")

NA tersisa - latih: FALSE | uji: FALSE

cat("kolom sama:", identical(names(lat_siap), names(uji_siap)), "\n")

kolom sama: TRUE

# pembakuan dipelajari dari LATIH, sehingga rata uji tidak persis nol
num <- names(lat_siap)[sapply(lat_siap, is.numeric)][1:3]
round(rbind(latih = sapply(lat_siap[num], mean),
            uji   = sapply(uji_siap[num], mean)), 4)

      id umur pendapatan
latih 505.5 0.000      0.0000
uji   483.0 0.061     -0.0297
```

Penjelasan Kode. Resep dipelajari dari data latih memakai prep; di sinilah nilai tengah, modus, dan simpangan baku dihitung. Kedua bagian data lalu diterapkan resep tersebut memakai bake. Rata-rata beberapa kolom numerik dibandingkan antara kedua bagian.

Interpretasi Output

Ini adalah bukti bahwa kebocoran tidak terjadi. Rata-rata kolom yang dibakukan pada data latih bernilai persis nol karena pembakuannya memang dipelajari dari sana. Pada data uji nilainya **tidak persis nol** — dan justru itulah yang benar. Bila rata-rata data uji juga persis nol, itu pertanda pembakuannya dihitung dari seluruh data, yang berarti telah terjadi kebocoran.

Contoh 5.6 | Memeriksa apa yang dipelajari resep

Kode 5.6 • R tidy untuk memeriksa langkah

```
# nilai yang dipelajari pada langkah imputasi
```

```
tidy(siap, number = 1)
```

```
# A tibble: 5 x 3
  terms      value id
  <chr>      <dbl> <chr>
1 umur          45 impute_median_iSy6y
2 pendapatan 14612. impute_median_iSy6y
3 pengeluaran  875. impute_median_iSy6y
4 lama_sekolah 10.1 impute_median_iSy6y
5 skor_literasi 51.5 impute_median_iSy6y
```

```
# parameter transformasi Yeo-Johnson
```

```
tidy(siap, number = 3)
```

```
# A tibble: 5 x 3
  terms      value id
  <chr>      <dbl> <chr>
1 umur          0.729 YeoJohnson_P7gmv
2 pendapatan -0.463 YeoJohnson_P7gmv
3 pengeluaran  0.202 YeoJohnson_P7gmv
4 lama_sekolah  1.07 YeoJohnson_P7gmv
5 skor_literasi 1.01 YeoJohnson_P7gmv
```

Penjelasan Kode. Fungsi tidy yang dikenakan pada resep terlatih menampilkan nilai yang dipelajari pada langkah tertentu. Argumen number merujuk urutan langkah pada resep.

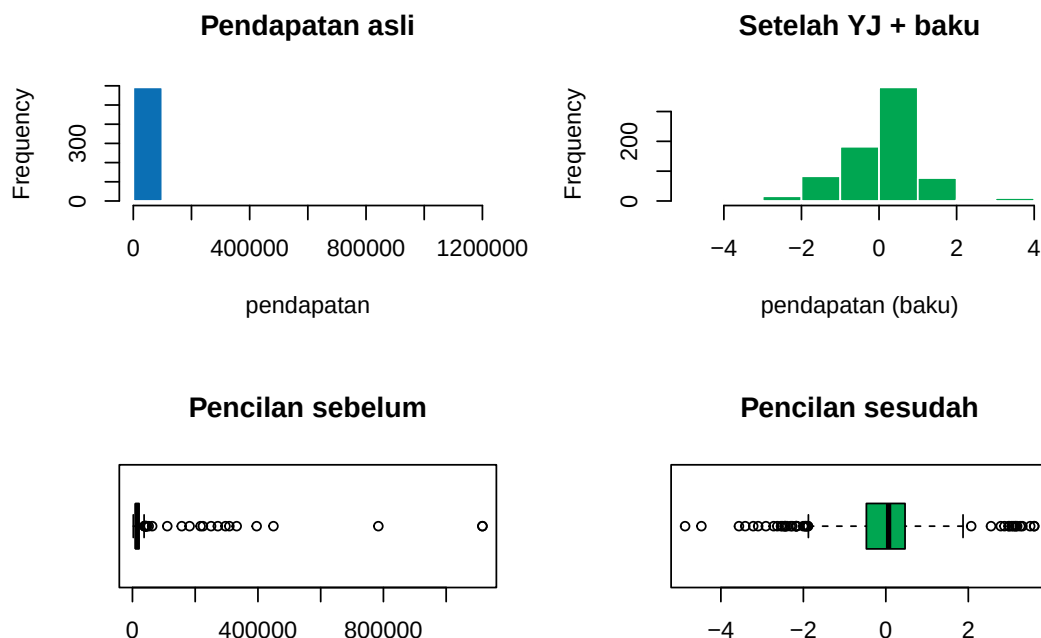
Interpretasi Output

Kemampuan memeriksa isi resep inilah salah satu keunggulannya dibanding prapemrosesan manual. Nilai tengah yang dipakai untuk imputasi dapat dilihat dan dilaporkan, sehingga hasil dapat direproduksi orang lain. Perhatikan pula parameter lambda pada Yeo-Johnson: nilai mendekati satu berarti transformasinya hampir tidak mengubah apa pun, sedangkan nilai mendekati nol menyerupai transformasi logaritma.

Contoh 5.7 | Menangani kemencengan dan skala

Kode 5.7 • R membandingkan sebaran sebelum dan sesudah

```
par(mfrow = c(2, 2))
hist(lat$pendapatan, main = "Pendapatan asli", col = "#0B6FB4",
     border = "white", xlab = "pendapatan")
hist(lat_siap$pendapatan, main = "Setelah YJ + baku", col = "#00A651",
     border = "white", xlab = "pendapatan (baku)")
boxplot(lat$pendapatan, main = "Pencilan sebelum", col = "#0B6FB4", horizontal = TRUE)
boxplot(lat_siap$pendapatan, main = "Pencilan sesudah", col = "#00A651", horizontal = TRUE)
```



```
par(mfrow = c(1, 1))

kemencengan <- function(x) mean((x - mean(x))^3) / sd(x)^3
round(c(sebelum = kemencengan(lat$pendapatan[!is.na(lat$pendapatan)]),
        sesudah = kemencengan(lat_siap$pendapatan)), 4)
```

```
sebelum sesudah
10.6423 -0.2312
```

Penjelasan Kode. Empat gambar disajikan berdampingan untuk membandingkan sebaran sebelum dan sesudah prapemrosesan, masing-masing sebagai histogram dan diagram kotak-garis. Fungsi kemencengan dihitung sendiri sebagai momen ketiga terbakukan.

Interpretasi Output

Bandingkan angka kemencengan sebelum dan sesudah. Transformasi Yeo-Johnson bekerja mengurangi kemencengan, sedangkan pembakuan hanya menggeser dan menskalakan tanpa mengubah bentuk. Perhatikan bahwa pencilan **tidak hilang** oleh pembakuan — ia hanya berpindah skala. Menghilangkan pencilan menuntut keputusan tersendiri, bukan efek samping transformasi.

Contoh 5.8 | Menangani taraf kategori yang jarang

Kode 5.8 • R step_other dan pemeriksaannya

```
cat("sebaran pendidikan pada data latih:\n")
```

```
sebaran pendidikan pada data latih:
```

```
print(round(prop.table(table(lat$pendidikan)), 4))
```

```
pt    PT    sd    SD    sma    SMA    smp    SMP
0.0039 0.1866 0.0026 0.1643 0.0053 0.4021 0.0053 0.2300
```

```
rec_other <- recipe(target ~ pendidikan + wilayah, data = lat) |>
  step_other(all_nominal_predictors(), threshold = 0.25)
siap_other <- prep(rec_other, training = lat)
hasil_other <- bake(siap_other, new_data = NULL)
```

```
cat("\ntaraf sebelum:", nlevels(lat$pendidikan),
    "| sesudah:", nlevels(hasil_other$pendidikan), "\n")
```

```
taraf sebelum: 8 | sesudah: 2
```

```
print(table(hasil_other$pendidikan))
```

```

SMA other
306 455

cat("\ntaraf yang digabung:\n")

taraf yang digabung:

print(tidy(siap_other, number = 1))

# A tibble: 3 x 3
  terms      retained id
<chr>      <chr>    <chr>
1 pendidikan SMA      other_dWwyq
2 wilayah    Perdesaan other_dWwyq
3 wilayah    Perkotaan  other_dWwyq

```

Penjelasan Kode. Sebaran taraf diperiksa lebih dahulu. Ambang seperempat dipakai agar penggabungan benar-benar terjadi pada data ini sehingga akibatnya terlihat. Fungsi tidy menampilkan taraf mana saja yang dipertahankan.

Interpretasi Output

Ambang harus dipilih berdasarkan sebaran, bukan angka baku. Ambang yang terlalu besar menggabungkan taraf yang sesungguhnya bermakna, sedangkan yang terlalu kecil membiarkan taraf jarang tetap ada. Keunggulan pokoknya adalah taraf yang dipertahankan dipelajari dari data latih, sehingga taraf tak terduga pada data baru otomatis masuk ke kelompok lainnya alih-alih menghentikan program.

Contoh 5.9 | Memeriksa ketimpangan kelas sasaran

Kode 5.9 • R memeriksa proporsi kelas

```

prop <- prop.table(table(lat$target))
print(round(prop, 4))

tidak ya
0.4179 0.5821

cat("rasio mayoritas terhadap minoritas:",
    round(max(prop) / min(prop), 2), "berbanding 1\n")

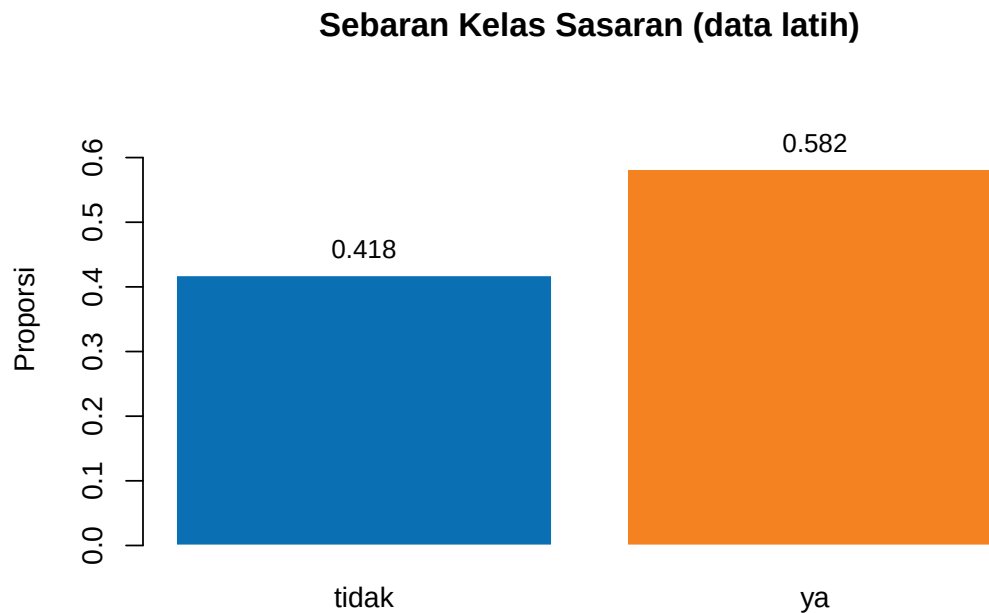
rasio mayoritas terhadap minoritas: 1.39 berbanding 1

cat("akurasi bila menebak seluruhnya mayoritas:", round(max(prop), 4), "\n")

```

akurasi bila menebak seluruhnya mayoritas: 0.5821

```
bp <- barplot(prop, col = c("#0B6FB4", "#F58220"), border = "white",  
              ylim = c(0, max(prop) * 1.2), ylab = "Proporsi",  
              main = "Sebaran Kelas Sasaran (data latihan)",  
              text(bp, prop, round(prop, 3), pos = 3, cex = .9))
```



Penjelasan Kode. Proporsi tiap kelas dihitung, lalu rasio mayoritas terhadap minoritas ditampilkan. Akurasi acuan dihitung sebagai proporsi kelas terbesar, yaitu kinerja yang diperoleh tanpa memodelkan apa pun.

Interpretasi Output

Angka acuan inilah yang wajib dibandingkan terhadap akurasi model. Model yang berakurasi delapan puluh persen pada data timpang delapan puluh berbanding dua puluh sesungguhnya tidak lebih baik daripada menebak. Karena itu pada data timpang, ukuran seperti F1, presisi, dan recall lebih informatif daripada akurasi — dan pembahasannya menjadi bagian Modul 6.

Contoh 5.10 | Menyeimbangkan kelas hanya pada data latihan

Kode 5.10 • R `step_downsample` dan perilakunya

```
suppressPackageStartupMessages(library(themis))

rec_seimbang <- recipe(target ~ ., data = lat) |>
  update_role(id, new_role = "penanda") |>
  step_impute_median(all_numeric_predictors()) |>
  step_dummy(all_nominal_predictors()) |>
  step_downsample(target)      # HANYA berlaku saat melatih

siap_s <- prep(rec_seimbang, training = lat)
lat_s <- bake(siap_s, new_data = NULL) # NULL berarti data latih
uji_s <- bake(siap_s, new_data = uji)  # data uji TIDAK diseimbangkan

cat("latih asli   :", nrow(lat), "\n")
```

```
latih asli   : 761
```

```
cat("latih seimbang:", nrow(lat_s), "\n")
```

```
latih seimbang: 636
```

```
print(table(lat_s$target))
```

```
tidak ya
 318 318
```

```
cat("\nuji (tetap timpang):", nrow(uji_s), "\n")
```

```
uji (tetap timpang): 254
```

```
print(table(uji_s$target))
```

```
tidak ya
 106 148
```

Penjelasan Kode. Langkah penyusutan ditambahkan pada resep. Argumen `new_data` bernilai `NULL` pada `bake` berarti mengambil data latih yang sudah diolah saat `prep`, sehingga penyeimbangan ikut diterapkan. Ketika `bake` dikenakan pada data uji, penyeimbangan sengaja dilewati oleh `recipes`.

Interpretasi Output

Inilah perilaku yang benar dan sukar dicapai bila langkah ditulis manual: data latih diseimbangkan sedangkan data uji dibiarkan timpang. Perhatikan bahwa penyusutan membuang sebagian amatan mayoritas, sehingga data latih menyusut cukup banyak. Bila kehilangan itu terasa berat, pengandaan atau SMOTE menjadi pilihan lain — dengan risikonya masing-masing sebagaimana dibahas pada Teori.

Contoh 5.11 | Membandingkan beberapa cara imputasi

Kode 5.11 • R menilai imputasi terhadap nilai sebenarnya

```
asli <- kotor$pendapatan
set.seed(2026)
buat_hilang <- asli
idx <- sample(length(asli), round(0.15 * length(asli)))
buat_hilang[idx] <- NA

imp_median <- ifelse(is.na(buat_hilang), median(buat_hilang, na.rm = TRUE), buat_hilang)
imp_rata <- ifelse(is.na(buat_hilang), mean(buat_hilang, na.rm = TRUE), buat_hilang)

rmse <- function(a, b) sqrt(mean((a - b)^2))
round(c(rmse_median = rmse(imp_median[idx], asli[idx]),
        rmse_rata = rmse(imp_rata[idx], asli[idx]),
        sb_asli = sd(asli),
        sb_median = sd(imp_median),
        sb_rata = sd(imp_rata)), 2)
```

rmse_median	rmse_rata	sb_asli	sb_median	sb_rata
NA	NA	NA	62857	62642

Penjelasan Kode. Lima belas persen nilai sengaja dihilangkan dari peubah yang lengkap, sehingga nilai sebenarnya diketahui dan hasil imputasi dapat dinilai. Dua cara imputasi dibandingkan memakai akar rerata kuadrat galat, dan simpangan bakunya dibandingkan terhadap data asli.

Interpretasi Output

Dua hal patut diperhatikan. Pertama, kedua cara menghasilkan galat yang serupa pada sebaran yang cukup simetris, tetapi median lebih unggul pada sebaran miring. Kedua, dan lebih penting: simpangan baku setelah imputasi **lebih kecil** daripada aslinya. Imputasi dengan satu nilai selalu mengecilkan keragaman, sehingga selang kepercayaan yang dihitung setelahnya menjadi terlalu sempit. Inilah alasan imputasi ganda dianjurkan ketika ketelitian pendugaan menjadi perhatian utama.

Contoh 5.12 | Menggambarkan akibat prapemrosesan pada seluruh peubah

Kode 5.12 • R boxplot berdampingan sebelum dan sesudah

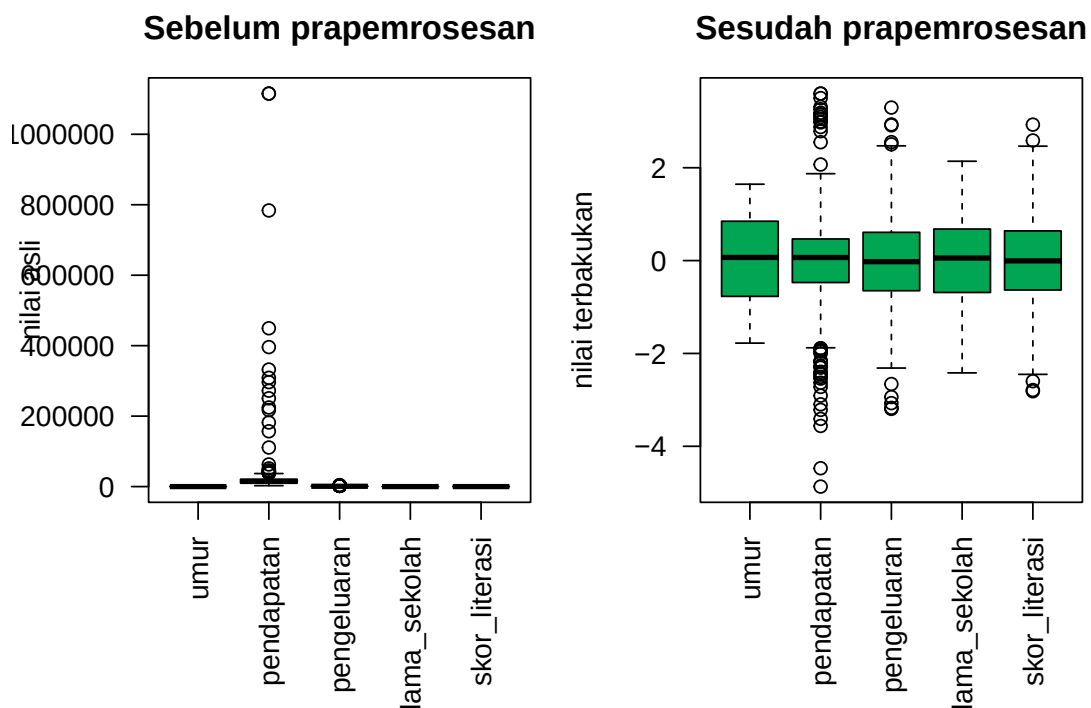
```
num_asli <- lat[, c("umur", "pendapatan", "pengeluaran",
                    "lama_sekolah", "skor_literasi")]
```

```

num_siap <- lat_siap[, c("umur", "pendapatan", "pengeluaran",
                        "lama_sekolah", "skor_literasi")]

par(mfrow = c(1, 2), mar = c(7, 4, 3, 1))
boxplot(num_asli, las = 2, col = "#6FB3E0", main = "Sebelum prapemrosesan",
        ylab = "nilai asli")
boxplot(num_siap, las = 2, col = "#00A651", main = "Sesudah prapemrosesan",
        ylab = "nilai terbakukan")

```



```

par(mfrow = c(1, 1), mar = c(5, 4, 4, 2) + 0.1)

round(rbind(sebelum = sapply(num_asli, function(x) sd(x, na.rm = TRUE)),
            sesudah = sapply(num_siap, sd)), 3)

```

	umur	pendapatan	pengeluaran	lama_sekolah	skor_literasi
sebelum	14.68	79665	532.8	3.902	12.19
sesudah	1.00	1	1.0	1.000	1.00

Penjelasan Kode. Lima peubah numerik digambar sebagai diagram kotak-garis sebelum dan sesudah prapemrosesan. Argumen las bernilai dua memutar label sumbu agar terbaca, dan batas bawah margin diperbesar agar nama peubah tidak terpotong.

Interpretasi Output

Gambar kiri memperlihatkan persoalan yang paling sering diabaikan: kelima peubah berskala sangat berbeda sehingga peubah bersatuan besar akan mendominasi setiap perhitungan jarak. Gambar kanan memperlihatkan bahwa setelah pembakuan seluruhnya sebanding. Perhatikan tabel simpangan baku di bawahnya — seluruh nilai sesudah bernilai satu, sebagaimana definisi pembakuan.

Contoh 5.13 | Membandingkan transformasi pengurang kemencengan**Kode 5.13 • R** membandingkan empat bentuk sebaran

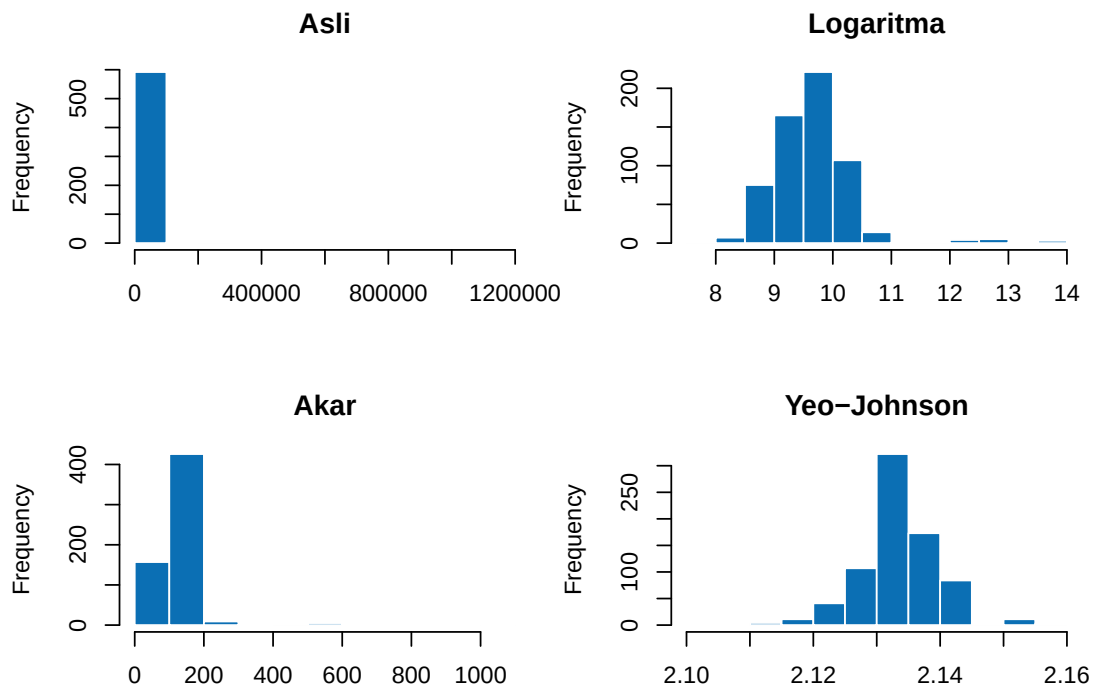
```
x <- lat$pendapatan[!is.na(lat$pendapatan)]
kem <- function(v) mean((v - mean(v))^3) / sd(v)^3

log_x <- log(x)
akar_x <- sqrt(x)
yj <- bake(prepare(recipe(~ pendapatan, data = lat) |>
  step_impute_median(pendapatan) |>
  step_YeoJohnson(pendapatan), training = lat),
  new_data = NULL)$pendapatan

round(c(asli = kem(x), log = kem(log_x),
  akar = kem(akar_x), yeo_johnson = kem(yj)), 4)
```

asli	log	akar	yeo_johnson
10.6423	2.1192	6.5604	-0.2312

```
par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))
for (p in list(list(x, "Asli"), list(log_x, "Logaritma"),
  list(akar_x, "Akar"), list(yj, "Yeo-Johnson")))
  hist(p[[1]], main = p[[2]], col = "#0B6FB4", border = "white", xlab = "")
```

```
par(mfrow = c(1, 1), mar = c(5, 4, 4, 2) + 0.1)
```

Penjelasan Kode. Empat versi peubah dibandingkan: asli, logaritma, akar kuadrat, dan Yeo-Johnson yang parameternya dipelajari dari data. Kemencengan dihitung bagi masing-masing, lalu keempat histogramnya disajikan berdampingan.

Interpretasi Output

Bandingkan keempat angka kemencengan dan keempat gambarnya. Logaritma paling agresif mengurangi kemencengan tetapi hanya sah bagi nilai positif, sedangkan Yeo-Johnson menyesuaikan diri terhadap data dan tetap sah bagi nilai nol maupun negatif. Perlu diingat bahwa transformasi mengubah satuan, sehingga koefisien model yang dihasilkan tidak lagi dapat dibaca dalam satuan aslinya — harga yang harus dibayar dan wajib disebutkan dalam pelaporan.

Contoh 5.14 | Menggabungkan data dari sumber lain

Kode 5.14 • R left_join dan pemeriksaannya

```
suppressPackageStartupMessages(library(dplyr))
referensi <- read.csv("data/data_referensi.csv", stringsAsFactors = TRUE)
print(referensi)
```

	wilayah	indeks_wilayah	keterangan
1	Perkotaan	1.00	acuan
2	Perdesaan	0.78	faktor penyesuaian biaya hidup

```
gabung <- lat |> left_join(referensi, by = "wilayah")
cat("baris sebelum:", nrow(lat), "| sesudah:", nrow(gabung), "\n")
```

```
baris sebelum: 761 | sesudah: 761
```

```
cat("gagal berpasangan:", sum(is.na(gabung$indeks_wilayah)), "\n")
```

```
gagal berpasangan: 19
```

Penjelasan Kode. Tabel referensi dibaca lalu digabungkan memakai `left_join` dengan kunci wilayah. Penggabungan jenis kiri mempertahankan seluruh baris tabel utama. Dua pemeriksaan dijalankan setelahnya: apakah banyaknya baris berubah, dan apakah ada baris yang gagal berpasangan.

Interpretasi Output

Kedua pemeriksaan itu wajib, bukan pilihan. Bertambahnya baris setelah penggabungan menandakan kunci tidak unik pada tabel referensi — kekeliruan yang menggandakan amatan tanpa disadari. Adapun nilai hilang pada kolom hasil gabungan menandakan ada nilai kunci yang tidak ditemukan padanya, misalnya karena perbedaan ejaan.

Menggabungkan sebelum membagi. Bila tabel referensi diturunkan dari data itu sendiri — misalnya rata-rata pendapatan per wilayah — maka menggabungkannya sebelum pembagian menimbulkan kebocoran, sebab rata-rata itu memuat informasi dari amatan yang kelak menjadi data uji. Tabel referensi yang berasal dari sumber luar dan tidak diturunkan dari data ini aman digabungkan kapan saja.

5.7 Latihan Mandiri

Kerjakan bertahap. Gunakan `data/data_kotor.csv` dan `data/data_referensi.csv`.

5.7.0.0.1 Tingkat Dasar (melengkapi kode)

Latihan 5.1 | Pemeriksaan cacat

```
kotor <- read.csv("data/data_kotor.csv", stringsAsFactors = TRUE)
sum(____(kotor))           # cacah nilai hilang seluruh tabel
sum(____(kotor))           # cacah baris kembar
q <- ____ (kotor$pendapatan, c(.25, .75), na.rm = TRUE)
batas <- q[2] + 1.5 * ____ (q)   # batas pencilan atas
```

Latihan 5.2 | Pembagian data

```
library(rsample)
set.seed(2026)
bagi <- ____ (kotor, prop = 0.75, strata = target)
lat <- ____ (bagi); uji <- ____ (bagi)
```

Latihan 5.3 | Resep sederhana

```
library(recipes)
rec <- ____ (target ~ ., data = lat) |>
  step_impute_median(____()) |>   # semua prediktor numerik
  step_dummy(all_nominal_predictors()) |>
  step_normalize(all_numeric_predictors())
siap <- ____ (rec, training = lat)   # pelajari dari data latih
hasil <- ____ (siap, new_data = uji) # terapkan ke data uji
```

5.7.0.0.2 Tingkat Menengah (menulis sendiri)

Latihan 5.4 | Resep lengkap beserta pemeriksaannya

Susun resep yang menangani nilai hilang, kemencengan, kelas jarang, pengodean, dan pembakuan. Tulis pula fungsi `periksa_alur(rec, lat, uji)` yang memastikan: (a) tidak ada NA tersisa pada kedua bagian; (b) nama kolom keduanya identik; (c) rata-rata kolom numerik pada data latih mendekati nol. Fungsi harus menghentikan program dengan pesan jelas bila salah satu gagal.

Latihan 5.5 | Membetulkan kode yang salah

Kode berikut memuat empat kesalahan (pembakuan sebelum pembagian, imputasi memakai seluruh data, urutan langkah terbalik, dan bake memakai data latih untuk prep). Temukan dan perbaiki, lalu jelaskan mengapa masing-masing menimbulkan kebocoran atau galat.

```
library(recipes); library(rsample)
kotor <- read.csv("data/data_kotor.csv", stringsAsFactors = TRUE)

# (a) pembakuan dilakukan SEBELUM pembagian
kotor$pendapatan <- scale(kotor$pendapatan)[, 1]

# (b) imputasi memakai nilai tengah SELURUH data
kotor$umur[is.na(kotor$umur)] <- median(kotor$umur, na.rm = TRUE)

set.seed(2026)
bagi <- initial_split(kotor, prop = 0.75, strata = target)
lat <- training(bagi); uji <- testing(bagi)

# (c) pembakuan mendahului imputasi
rec <- recipe(target ~ ., data = lat) |>
  step_normalize(all_numeric_predictors()) |>
  step_impute_median(all_numeric_predictors())

# (d) prep memakai data uji
siap <- prep(rec, training = uji)
```

Keluaran yang benar. Setelah diperbaiki, kode yang dipakai adalah sebagai berikut.

```
# (a) dan (b) DIHAPUS -- seluruh prapemrosesan masuk ke dalam resep
kotor <- read.csv("data/data_kotor.csv", stringsAsFactors = TRUE)

set.seed(2026)
bagi <- initial_split(kotor, prop = 0.75, strata = target)
lat <- training(bagi); uji <- testing(bagi)

# (c) imputasi mendahului pembakuan
rec <- recipe(target ~ ., data = lat) |>
  step_impute_median(all_numeric_predictors()) |>
  step_dummy(all_nominal_predictors()) |>
  step_normalize(all_numeric_predictors())

# (d) prep memakai data LATIH
```

```
siap <- prep(rec, training = lat)
uji_siap <- bake(siap, new_data = uji)
```

Pemeriksaan yang wajib berhasil: `anyNA(bake(siap, new_data = lat))` bernilai **FALSE**, dan `abs(mean(bake(siap, new_data = lat)$umur)) < 1e-8` bernilai **TRUE**.

Latihan 5.6 | Ketimpangan kelas

Hitung proporsi kelas target pada data latih, rasio mayoritas terhadap minoritas, dan akurasi acuan bila seluruhnya ditebak mayoritas. Lalu terapkan `step_downsample()` dan buktikan bahwa **hanya data latih** yang diseimbangkan sedangkan data uji tetap timpang.

Latihan 5.7 | Menilai imputasi

Hilangkan 20 persen nilai pengeluaran secara sengaja, lalu bandingkan imputasi median dan rata-rata terhadap nilai sebenarnya memakai akar rerata kuadrat galat. Laporkan pula simpangan baku sebelum dan sesudah imputasi, lalu jelaskan mengapa keragaman selalu mengecil.

5.7.0.0.3 Tingkat Lanjut (keputusan)

Latihan 5.8 | Memutuskan penanganan cacat

Untuk masing-masing keadaan berikut, tentukan penanganan yang Anda pilih beserta alasannya, dan sebutkan **satu risiko** dari pilihan itu: (a) pendapatan hilang pada 12 persen amatan, dan yang hilang lebih banyak pada kelompok berpendidikan tinggi; (b) tiga amatan berpendapatan seratus kali lipat lainnya; (c) kolom pendidikan memuat nilai SMA, sma, dan S.M.A; (d) kolom skor literasi memuat nilai 999 pada beberapa baris.

5.8 Penugasan

Sebagai calon analis pada instansi statistik, susun laporan prapemrosesan atas `data_kotor.csv` (*data simulasi*) yang menghasilkan data siap model, lengkap dengan pembuktian bahwa tidak terjadi kebocoran.

Konteks. Sebuah unit menerima data hasil pencacahan yang belum dibersihkan. Data akan dipakai memodelkan peubah target, dan pimpinan meminta jaminan bahwa penilaian kinerjanya kelak dapat dipercaya.

Tugas 5.1 | Diagnosis

1. Periksa **keempat jenis cacat**. Laporkan temuan tiap jenis beserta besarnya.
2. Periksa **pola nilai hilang**: apakah tampak MCAR atau MAR? Sajikan bukti berupa tabel silang keterisian terhadap minimal dua peubah lain, lalu jelaskan mengapa MNAR tidak dapat disingkirkan.
3. Periksa kemencengan dan perbedaan skala antar-peubah numerik. Sajikan tabel yang memuat rata-rata, simpangan baku, dan kemencengan tiap peubah.

Tugas 5.2 | Penyusunan Resep

4. Bagi data dengan penstrataan, lalu susun resep yang menangani seluruh cacat temuan Anda. **Jelaskan alasan urutan langkah** yang Anda pilih.
5. Pelajari resep dan terapkan ke kedua bagian. Sajikan dimensi hasil, cacah NA tersisa, dan rata-rata beberapa kolom numerik pada kedua bagian.
6. Tulis fungsi `periksa_alur()` yang memeriksa minimal empat hal dan menghentikan program dengan pesan jelas bila gagal. Jalankan dan tunjukkan hasilnya.

Tugas 5.3 | Ketimpangan dan Imputasi

- 6b. Periksa **ketimpangan kelas** sasaran. Laporkan proporsi, rasio, dan akurasi acuan. Terapkan penyeimbangan pada resep, lalu buktikan bahwa data uji tidak ikut diseimbangkan.
- 6c. Nilai mutu imputasi dengan **menghilangkan nilai secara sengaja** dari peubah yang lengkap. Bandingkan minimal dua cara imputasi, laporkan galatnya, dan bahas akibatnya terhadap keragaman data.
- 6d. Tangani **taraf kategori jarang** dengan `step_other()`. Tentukan ambang berdasarkan sebaran, laporkan taraf mana yang digabung, dan jelaskan mengapa penggabungan itu melindungi model dari taraf tak terduga pada data baru.

Tugas 5.4 | Pembuktian

7. Buktikan bahwa **tidak terjadi kebocoran** dengan menunjukkan bahwa rata-rata kolom terbakukan pada data uji tidak persis nol, lalu jelaskan mengapa hal itu justru menjadi bukti yang benar.
8. Gabungkan `data_referensi.csv` dan lakukan **dua pemeriksaan** setelah penggabungan. Jelaskan mengapa tabel referensi ini aman digabungkan sebelum pembagian.
9. Susun laporan R Markdown berisi kode, keluaran, gambar, dan interpretasi. Tutup dengan **ringkasan (maksimal 300 kata)** memuat: cacat yang ditemukan, penanganan beserta alasannya, bukti ketiadaan kebocoran, dan penegasan bahwa data merupakan simulasi.

5.9 Tambahan: Menakar Akibat Kebocoran Data

Bagian ini menakar akibat kebocoran dengan angka. Peragaan dirancang seekstrem mungkin agar akibatnya terlihat jelas, tetapi bentuk kekeliruannya persis sama dengan yang lazim terjadi pada pekerjaan sungguhan.

Data tanpa sinyal sama sekali. Peragaan ini memakai data yang **seluruhnya derau**: lima ratus peubah acak dan sasaran yang juga acak. Menurut rancangannya, tidak ada hubungan apa pun antara peubah dan sasaran, sehingga akurasi yang sah tidak boleh melampaui tebakan mayoritas. Apa pun yang melampauinya adalah tanda kekeliruan.

Contoh 5.15 | Kebocoran akibat seleksi peubah

Kode 5.15 • R menakar akibat kebocoran seleksi peubah

```
set.seed(2026)
n <- 200; p <- 500
X <- matrix(rnorm(n * p), n, p)          # SELURUHNYA derau
y <- factor(sample(c("ya", "tidak"), n, TRUE))
lipat <- sample(rep(1:5, length.out = n))

# --- CARA SALAH: peubah dipilih memakai SELURUH data ---
kor <- apply(X, 2, function(v) abs(cor(v, as.numeric(y))))
pilih <- order(kor, decreasing = TRUE)[1:10]

akur_bocor <- mean(sapply(1:5, function(k) {
  tr <- lipat != k
  m <- suppressWarnings(glm(y[tr] ~ ., data = data.frame(X[tr, pilih])),
```

```

        family = binomial))
pr <- predict(m, newdata = data.frame(X[!tr, pilih]), type = "response")
mean(ifelse(pr > .5, levels(y)[2], levels(y)[1]) == y[!tr])
}))

# --- CARA BENAR: peubah dipilih DI DALAM tiap lipatan ---
akur_benar <- mean(sapply(1:5, function(k) {
  tr <- lipat != k
  kk <- apply(X[tr, ], 2, function(v) abs(cor(v, as.numeric(y[tr]))))
  pl <- order(kk, decreasing = TRUE)[1:10]
  m <- suppressWarnings(glm(y[tr] ~ ., data = data.frame(X[tr, pl]),
        family = binomial))
  pr <- predict(m, newdata = data.frame(X[!tr, pl]), type = "response")
  mean(ifelse(pr > .5, levels(y)[2], levels(y)[1]) == y[!tr])
}))

round(c(dengan_kebocoran = akur_bocor,
        tanpa_kebocoran = akur_benar,
        tebak_mayoritas = max(prop.table(table(y))), 3)

```

dengan_kebocoran	tanpa_kebocoran	tebak_mayoritas
0.690	0.535	0.505

Penjelasan Kode. Data dibangkitkan tanpa hubungan apa pun antara peubah dan sasaran. Dua cara dibandingkan. Cara pertama memilih sepuluh peubah paling berkorelasi memakai seluruh data, lalu memvalidasi silang. Cara kedua melakukan seleksi terpisah di dalam tiap lipatan, sehingga data uji tiap lipatan tidak ikut menentukan peubah mana yang dipilih.

Interpretasi Output

Perhatikan selisihnya. Cara yang bocor menghasilkan akurasi jauh di atas tebakan mayoritas, padahal datanya sama sekali tidak memuat sinyal. Angka itu murni hasil kebocoran: peubah dipilih justru karena kebetulan berkorelasi dengan sasaran pada seluruh data, termasuk pada bagian yang kemudian dipakai menguji. Cara yang benar menghasilkan akurasi mendekati tebakan mayoritas — yang memang seharusnya, sebab tidak ada yang dapat dipelajari dari derau.

Mengapa bentuk kebocoran ini paling berbahaya. Kebocoran akibat seleksi peubah tidak menimbulkan galat, tidak menghasilkan peringatan, dan justru menghasilkan angka yang menyenangkan. Ia paling sering terjadi pada data berdimensi tinggi, tepat ketika seleksi peubah paling dibutuhkan. Pencegahannya hanya satu: seluruh keputusan yang memakai sasaran — termasuk seleksi peubah — harus berada **di dalam** lipatan validasi silang.

Contoh 5.16 | Kebocoran akibat pembakuan sebelum pembagian

Kode 5.16 • R membandingkan dua cara pembakuan

```
set.seed(2026)
d <- data.frame(x = c(rnorm(150, 10, 2), rnorm(50, 30, 2))) # dua kelompok

# CARA SALAH: dibakukan sebelum dibagi
d$x_bocor <- scale(d$x)[, 1]

# CARA BENAR: dibakukan memakai statistik data latih saja
idx <- 1:150
mu <- mean(d$x[idx]); sd_ <- sd(d$x[idx])
d$x_benar <- (d$x - mu) / sd_

round(rbind(
  `rata bagian uji (bocor)` = mean(d$x_bocor[-idx]),
  `rata bagian uji (benar)` = mean(d$x_benar[-idx]),
  `simpangan baku uji (bocor)` = sd(d$x_bocor[-idx]),
  `simpangan baku uji (benar)` = sd(d$x_benar[-idx])), 4)

      [,1]
rata bagian uji (bocor)  1.6862
rata bagian uji (benar)  9.9029
simpangan baku uji (bocor) 0.1898
simpangan baku uji (benar) 0.8359
```

Penjelasan Kode. Data dibentuk dari dua kelompok bersebaran berbeda agar akibatnya terlihat. Pembakuan pertama memakai rata-rata dan simpangan baku seluruh data. Pembakuan kedua hanya memakai statistik dari seratus lima puluh amatan pertama yang berperan sebagai data latih.

Interpretasi Output

Perbedaannya besar. Pada cara yang benar, bagian uji berrata-rata jauh dari nol — dan itu memang seharusnya, sebab bagian uji berasal dari sebaran yang berbeda. Cara yang bocor menyembunyikan perbedaan itu, sehingga model tidak pernah tahu bahwa data barunya berbeda. Dalam penerapan nyata, model semacam ini akan berkinerja buruk tanpa sebab yang jelas.

Tips dan Praktik Baik. Satu pemeriksaan sederhana dapat menangkap sebagian besar kebocoran: setelah prapemrosesan, periksa rata-rata kolom terbakukan pada data uji. Bila nilainya **persis nol**, hampir dapat dipastikan pembakuannya dihitung dari seluruh data. Nilai yang sedikit menyimpang dari nol justru pertanda alur kerja Anda benar.

Kesimpulan bagian ini sederhana tetapi menuntut kedisiplinan. Setiap keputusan yang memakai data — imputasi, pembakuan, seleksi peubah, penyeimbangan kelas, bahkan penetapan taraf faktor — harus dipelajari dari data latih saja. Paket recipes menegakkan aturan itu secara rancangan, dan itulah alasan utama memakainya alih-alih menuliskan langkah prapemrosesan satu per satu.

5.10 Ringkasan Modul

Daftar periksa penguasaan:

- ☐ Saya dapat mengenali empat jenis cacat data beserta cara memeriksanya.
- ☐ Saya dapat membedakan pola MCAR, MAR, dan MNAR beserta akibatnya.
- ☐ Saya membagi data *sebelum* melakukan prapemrosesan apa pun.
- ☐ Saya dapat menyusun resep dan menjelaskan alasan urutan langkahnya.
- ☐ Saya memahami pembagian peran recipe, prep, dan bake.
- ☐ Saya memeriksa hasil penggabungan data terhadap penggandaan dan kegagalan pasangan.
- ☐ Saya dapat menyebutkan tiga sumber kebocoran yang paling sering.
- ☐ Saya tahu bahwa rata-rata data uji yang persis nol adalah tanda kebocoran.

Kartu Referensi Cepat.

Keperluan	Perintah
nMembagi data	<code>initial_split(d, prop =, strata =)</code>
nMengambil bagian	<code>training(bagi), testing(bagi)</code>
nMenetapkan resep	<code>recipe(y ~ ., data = lat)</code>
nMengubah peran	<code>update_role(id, new_role = "penanda")</code>
nImputasi	<code>step_impute_median(), step_impute_mode()</code>
nKemencengan	<code>step_YeoJohnson(), step_log()</code>
nKelas jarang	<code>step_other(threshold = 0.05)</code>
nPengodean	<code>step_dummy(all_nominal_predictors())</code>
nRagam nol	<code>step_zv(all_predictors())</code>
nPembakuan	<code>step_normalize(all_numeric_predictors())</code>
nMempelajari	<code>prep(rec, training = lat)</code>
nMenerapkan	<code>bake(siap, new_data = uji)</code>
nMemeriksa langkah	<code>tidy(siap, number = k)</code>

Peta keputusan prapemrosesan. Tabel berikut merangkum keputusan yang perlu diambil beserta pertimbangannya.

Delapan keputusan dan pertimbangannya.

Keputusan	Pilihan	Pertimbangan utama
Nilai hilang	buang, imputasi, model	pola MCAR/MAR/MNAR
Cara imputasi	median, modus, berbasis model	tipe peubah dan pola hilangnya
Pencilan	pertahankan, transformasi, buang	sah atau keliru catat
Kemencengan	biarkan, log, Yeo-Johnson	kepekaan model terhadap bentuk
Taraf jarang	biarkan, gabung ke lainnya	risiko taraf tak terduga
Pengodean	boneka, ordinal, target	sifat kategori dan model
Pembakuan	perlu atau tidak	apakah model berbasis jarak
Ketimpangan	biarkan, seimbangkan, bobot	ukuran penilaian yang dipakai

Catat setiap keputusan. Setiap baris pada tabel di atas menuntut keputusan, dan setiap keputusan mengubah hasil. Karena itu laporan wajib memuat bukan hanya apa yang dikerjakan melainkan juga **mengapa**. Resep yang tersimpan sebagai kode sudah mencatat apa yang dikerjakan; alasannya harus ditulis penyusunnya sendiri.

Modul ini menempati tahap yang menyita waktu terbesar dalam CRISP-DM, dan alasannya kini jelas: setiap penanganan cacat menuntut keputusan, dan setiap keputusan berisiko membocorkan informasi. Prinsip yang perlu dibawa: bagi data lebih dahulu, tempatkan seluruh langkah di dalam resep, periksa apa yang dipelajari resep, dan curigai hasil yang terlalu bagus. Modul berikutnya membahas konsep supervised learning, tempat data hasil prapemrosesan ini akhirnya dipakai memodelkan.

5.11 Referensi

Kuhn, M., & Johnson, K. (2019). *Feature Engineering and Selection: A Practical Approach for Predictive Models*. Chapman and Hall/CRC.

Kuhn, M., & Silge, J. (2022). *Tidy Modeling with R*. O'Reilly Media.

Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann.

Little, R. J. A., & Rubin, D. B. (2019). *Statistical Analysis with Missing Data* (3rd ed.). Wiley.

Kaufman, S., Rosset, S., & Perlich, C. (2011). Leakage in data mining: Formulation, detection, and avoidance. *ACM SIGKDD 2011*, 556-563.

MODUL

6

KONSEP SUPERVISED LEARNING

Pertemuan Minggu ke-6

Sub-CPMK 6: Mahasiswa mampu menjelaskan kerangka supervised learning, membagi data dan melakukan validasi silang dengan benar, memilih ukuran penilaian yang sesuai, serta membandingkan dua kerangka pemodelan di R.

Penyusun: Achmad Syahrul Choir



Daftar Isi

6.1	Deskripsi Modul	174
6.2	Tujuan Praktikum	174
6.3	Alat dan Bahan	174
6.4	Instruksi Pengerjaan	175
6.5	Teori Singkat	175
6.5.0.1	Kedudukan pada CRISP-DM	175
6.5.0.2	Kerangka Supervised Learning	175
6.5.0.3	Pembagian Data dan Validasi Silang	176
6.5.0.4	Matriks Kekeliruan dan Ukuran Turunannya	176
6.5.0.5	Pertukaran Presisi dan Recall	177
6.5.0.6	Pemodelan Berlebih (<i>Overfitting</i>)	177
6.5.0.7	Supervised untuk Sasaran Numerik	178
6.5.0.8	Dua Kerangka Pemodelan di R	178
6.6	Contoh Praktikum dengan R	179
6.7	Latihan Mandiri	195
6.8	Penugasan	198
6.9	Tambahan: Membandingkan caret dan tidymodels Berdampingan	199
6.10	Ringkasan Modul	202
6.11	Referensi	203

6.1 Deskripsi Modul

Lima modul sebelumnya menyiapkan data; modul ini mulai memodelkannya. Yang dibahas bukan algoritma tertentu, melainkan kerangka yang berlaku bagi seluruhnya: bagaimana data dibagi, bagaimana kinerja dinilai, dan bagaimana dua model dibandingkan secara adil.

Kerangka ini perlu dikuasai lebih dahulu karena kekeliruan pada tataran ini tidak dapat diperbaiki oleh algoritma secanggih apa pun. Model yang dinilai dengan cara yang keliru akan tampak baik meski buruk, dan sebaliknya.

Materi mencakup pembagian data dan validasi silang, ukuran penilaian beserta kapan masing-masing sesuai, gagasan pemodelan berlebih (*overfitting*), serta pengenalan penalaan hiperparameter. Bagian Tambahan menjalankan kasus yang sama pada dua kerangka pemodelan R — `caret` dan `tidymodels` — lalu membahas mengapa hasilnya dapat berbeda.

Tentang data. Berkas data pada modul ini adalah **data simulasi** (`set.seed(2026)`), **bukan data nyata** dan **bukan data resmi** instansi mana pun. Peubah dibangkitkan saling bebas terhadap sasaran, sehingga kinerja model yang mendekati tebakan acak adalah hasil yang benar dan wajib dilaporkan apa adanya.

6.2 Tujuan Praktikum

Setelah menyelesaikan praktikum ini secara tuntas, mahasiswa diharapkan mampu:

1. **Menjelaskan** kerangka supervised learning dan perbedaannya dari pemodelan inferensia.
2. **Membagi** data dengan penstrataan dan menjelaskan mengapa diperlukan.
3. **Menerapkan** validasi silang serta menjelaskan pilihan banyaknya lipatan.
4. **Membaca** matriks kekeliruan dan menurunkan ukuran penilaian darinya.
5. **Memilih** ukuran penilaian yang sesuai dengan sifat persoalan.
6. **Mengenali** pemodelan berlebih (*overfitting*) beserta gejalanya.
7. **Melakukan** penalaan hiperparameter sederhana.
8. **Membandingkan** kerangka `caret` dan `tidymodels` beserta akibat perbedaannya.

6.3 Alat dan Bahan

Perangkat dan bahan yang diperlukan:

- R (RStudio) dengan paket `recipes`, `rsample`, `parsnip`, `workflows`, `tune`, `yardstick`, dan `caret`.
- Pasang inti `tidymodels` dengan `install.packages(c("recipes", "rsample", "parsnip", "workflows", "tune", "yardstick"))`.
- Mesin model: `install.packages(c("kknn", "rpart", "ranger"))`.
- Berkas data: `data/data_kredit.csv`.

6.4 Instruksi Pengerjaan

Ikuti tahapan berikut:

Sebelum praktikum: pasang seluruh paket; siapkan berkas data; baca bagian Teori mengenai ukuran penilaian.

Selama praktikum: kerjakan Contoh berurutan. Bagian Tambahan mengenai perbandingan dua kerangka dikerjakan setelah Penugasan.

Pembagian kerja: Contoh dan Latihan dikerjakan di kelas; **Penugasan dikerjakan di luar sesi.**

Pengumpulan: susun laporan R Markdown, render ke PDF atau HTML, kumpulkan melalui Google Classroom.

6.5 Teori Singkat

6.5.0.1 Kedudukan pada CRISP-DM

Modul ini menjembatani tahap keempat dan kelima, yaitu Modeling dan Evaluation. Keduanya dibahas bersama karena tidak dapat dipisahkan: cara menilai menentukan model mana yang terpilih, sehingga keputusan penilaian sesungguhnya merupakan bagian dari pemodelan itu sendiri.

6.5.0.2 Kerangka Supervised Learning

Pada supervised learning, tersedia pasangan masukan dan sasaran yang diketahui. Tujuannya membentuk fungsi yang memetakan masukan ke sasaran sedemikian rupa sehingga bekerja baik pada data **yang belum pernah dilihat**. Penekanan pada kata terakhir itulah yang membedakannya dari pemodelan inferensia.

Empat unsur kerangka.

Unsur	Isi	Keputusan yang diambil
Data	masukan dan sasaran	peubah mana yang dipakai
Model	keluarga fungsi	pohon, linear, jarak, ensemble
Ukuran	bagaimana dinilai	akurasi, F1, AUC, RMSE
Prosedur	bagaimana dilatih dan dinilai	pembagian, validasi silang

6.5.0.3 Pembagian Data dan Validasi Silang

Menilai model pada data yang dipakai melatihnya selalu menghasilkan angka yang terlalu bagus, sebab model dapat menghafal. Karena itu data dibagi: sebagian untuk melatih, sebagian disisihkan untuk menilai.

Persoalannya, satu kali pembagian menghasilkan penilaian yang bergantung pada kebetulan pembagian itu. Validasi silang mengatasinya dengan mengulang: data dibagi menjadi beberapa lipatan, dan tiap lipatan bergiliran menjadi penguji sementara sisanya melatih. Rata-rata hasilnya lebih stabil daripada satu kali pembagian.

Memilih banyaknya lipatan.

Lipatan	Sifat	Kapan dipilih
5	lebih cepat, ragam lebih besar	data besar, model lambat
10	baku, keseimbangan baik	keadaan umum
n (LOOCV)	bias kecil, ragam besar, mahal	data sangat kecil

Tips dan Praktik Baik. Penstrataan pada pembagian dan pada validasi silang menjaga proporsi kelas tetap sama di tiap bagian. Pada data timpang, tanpa penstrataan sebuah lipatan dapat kebetulan memuat sangat sedikit kelas minoritas — bahkan tidak sama sekali — sehingga ukuran penilaiannya menjadi tak terdefinisi atau sangat tidak stabil.

6.5.0.4 Matriks Kekeliruan dan Ukuran Turunannya

Seluruh ukuran penilaian klasifikasi biner diturunkan dari empat bilangan yang tersusun pada matriks kekeliruan: benar positif, benar negatif, salah positif, dan salah negatif.

Ukuran	Rumus	Menjawab
Akurasi	$\frac{BP + BN}{\text{seluruh}}$	berapa yang benar keseluruhan?
Presisi	$\frac{BP}{BP + SP}$	dari yang ditandai, berapa benar?
Recall	$\frac{BP}{BP + SN}$	dari yang sesungguhnya, berapa tertangkap?
F1	$\frac{2 \cdot P \cdot R}{P + R}$	rerata harmonik keduanya

Akurasi menyesatkan pada data timpang. Bila hanya lima belas persen amatan berkelas positif, menebak seluruhnya negatif sudah menghasilkan akurasi delapan puluh lima persen — tanpa mempelajari apa pun dan tanpa pernah menemukan satu pun kasus positif. Karena itu akurasi **wajib** dibandingkan terhadap acuan tebakan mayoritas, dan pada data timpang sebaiknya diganti F1 atau AUC.

6.5.0.5 Pertukaran Presisi dan Recall

Presisi dan recall bergerak berlawanan. Menurunkan ambang keputusan menaikkan recall karena lebih banyak kasus tertangkap, tetapi menurunkan presisi karena lebih banyak pula yang salah ditandai. Pilihan ambang karena itu bukan keputusan teknis melainkan keputusan kebijakan.

Pertimbangannya bergantung pada biaya kekeliruan. Pada penapisan penyakit, melewatkan kasus jauh lebih mahal daripada memeriksa orang sehat, sehingga recall diutamakan. Pada penandaan penipuan yang menuntut pemeriksaan manual mahal, presisi lebih diutamakan agar petugas tidak kewalahan.

6.5.0.6 Pemodelan Berlebih (*Overfitting*)

Model yang terlalu lentur dapat menghafal derau pada data latih. Gejalanya khas: kinerja pada data latih terus membaik sementara kinerja pada data uji memburuk. Jarak antara keduanya adalah petunjuk paling langsung.

Membaca gejala.

Latih	Uji	Tafsiran
rendah	rendah	kurang luwes (<i>underfitting</i>)
tinggi	tinggi	sesuai
tinggi	rendah	mengalami <i>overfitting</i>
rendah	tinggi	ada yang keliru pada penilaian

6.5.0.7 Supervised untuk Sasaran Numerik

Seluruh pembahasan sejauh ini mengandaikan sasaran berupa kelas. Bila sasarannya numerik, kerangkanya sama persis — pembagian, validasi silang, penalaan — hanya ukuran penilaiannya yang berganti.

Ukuran	Rumus	Sifat
RMSE	$\sqrt{\frac{1}{n} \sum (y - \hat{y})^2}$	bersatuan sama dengan sasaran; peka pencilan
MAE	$\frac{1}{n} \sum y - \hat{y} $	lebih tahan pencilan
R^2	proporsi keragaman ditekankan	tak bersatuan; mudah disalahtafsirkan

R-kuadrat bukan ukuran ketepatan ramalan. Nilai R^2 yang tinggi pada data latih sama sekali tidak menjamin ramalan yang baik pada data baru, sebab menambah peubah selalu menaikkannya. Untuk menilai kemampuan meramal, pakailah RMSE atau MAE yang dihitung pada data uji — keduanya bersatuan sama dengan sasaran sehingga besarnya galat dapat dinilai kewajarannya.

6.5.0.8 Dua Kerangka Pemodelan di R

Perbandingan caret dan tidymodels.

	caret	tidymodels
Gaya	satu fungsi <code>train()</code>	rangkaian objek
Prapemrosesan	argumen <code>preProcess</code>	recipes, urutan bebas
Kebocoran	rawan bila di luar CV	dicegah oleh rancangan
Kurva belajar	landai	lebih curam
Cakupan model	sangat banyak	memadai, terus bertambah
Status	dirawat, tidak dikembangkan	aktif dikembangkan

Modul ini memakai caret pada bagian pengenalan karena sintaksnya lebih ringkas, lalu beralih ke tidymodels mulai modul berikutnya karena keunggulan resepnya terasa ketika prapemrosesan menjadi rumit. Bagian Tambahan menjalankan keduanya pada kasus sama agar perbedaannya terlihat langsung.

6.6 Contoh Praktikum dengan R

Contoh 6.1 | Menyiapkan data dan memeriksa sasarannya

Kode 6.1 · R membaca data dan memeriksa ketimpangan

```
kredit <- read.csv("data/data_kredit.csv", stringsAsFactors = TRUE)
kredit$gagal_bayar <- factor(kredit$gagal_bayar, levels = c("ya", "tidak"))
str(kredit)

'data.frame': 1000 obs. of 9 variables:
 $ id          : int  390 591 895 640 376 353 397 579 411 45 ...
 $ umur        : int  52 45 27 34 45 22 42 59 32 47 ...
 $ pendapatan  : num  16192 10617 7791 17427 8798 ...
 $ lama_kerja  : num  0 7.6 3.9 11.4 13.8 4 11 15.8 7.6 0 ...
 $ rasio_utang  : num  0.156 0.361 0.418 0.407 0.2 0.172 0.134 0.348 0.343 0.272 ...
 $ jumlah_tanggungan: int  3 1 3 3 5 3 3 1 0 1 ...
 $ pendidikan  : Factor w/ 4 levels "D3","S1","SMA",...: 1 3 3 3 1 3 3 1 3 3 ...
 $ status_rumah : Factor w/ 3 levels "Keluarga","Milik",...: 2 2 2 1 2 2 3 2 2 2 ...
 $ gagal_bayar  : Factor w/ 2 levels "ya","tidak": 1 2 1 2 1 1 2 2 2 2 ...

prop <- prop.table(table(kredit$gagal_bayar))
round(prop, 4)

  ya tidak
0.337 0.663

cat("acuan tebakan mayoritas:", round(max(prop), 4), "\n")

acuan tebakan mayoritas: 0.663

cat("rasio timpang:", round(max(prop) / min(prop), 2), "berbanding 1\n")

rasio timpang: 1.97 berbanding 1
```

Penjelasan Kode. Data dibaca lalu taraf sasaran diurutkan agar kelas positif berada di depan; urutan ini penting sebab yardstick memperlakukan taraf pertama sebagai kelas yang diperhatikan. Proporsi kelas dan acuan tebakan mayoritas dihitung.

Interpretasi Output

Angka acuan inilah pembandingan wajib bagi setiap akurasi yang kelak dilaporkan. Urutan taraf sasaran kerap luput diperhatikan padahal berakibat besar: bila taraf yang diperhatikan salah, presisi dan recall yang dilaporkan sesungguhnya mengukur kelas yang keliru.

Contoh 6.2 | Membagi data dengan penstrataan

Kode 6.2 • R initial_split dan pemeriksaan proporsi

```
suppressPackageStartupMessages({library(rsample); library(recipes)})
set.seed(2026)
bagi <- initial_split(kredit, prop = 0.75, strata = gagal_bayar)
lat <- training(bagi); uji <- testing(bagi)
```

```
round(rbind(seluruh = prop.table(table(kredit$gagal_bayar)),
            latih    = prop.table(table(lat$gagal_bayar)),
            uji      = prop.table(table(uji$gagal_bayar))), 4)
```

```
      ya tidak
seluruh 0.3370 0.6630
latih   0.3364 0.6636
uji     0.3386 0.6614
```

```
cat("\nukuran - latih:", nrow(lat), "| uji:", nrow(uji), "\n")
```

```
ukuran - latih: 749 | uji: 251
```

Penjelasan Kode. Data dibagi tiga perempat berbanding seperempat dengan penstrataan pada sasaran. Proporsi kelas pada seluruh data, data latih, dan data uji disandingkan untuk memeriksa apakah penstrataannya bekerja.

Interpretasi Output

Ketiga baris seharusnya nyaris sama, dan itulah tanda penstrataan berhasil. Tanpa penstrataan, proporsi pada bagian uji dapat menyimpang cukup jauh terutama pada data timpang, sehingga penilaian kinerjanya menjadi tidak stabil dan sukar dibandingkan antar-percobaan.

Contoh 6.3 | Menyusun resep dan model pertama

Kode 6.3 • R recipe, model spec, workflow, fit

```
suppressPackageStartupMessages({library(parsnip); library(workflows)})

rec <- recipe(gagal_bayar ~ ., data = lat) |>
  update_role(id, new_role = "penanda") |>
  step_impute_median(all_numeric_predictors()) |>
  step_dummy(all_nominal_predictors()) |>
  step_normalize(all_numeric_predictors())
```

```
spec_knn <- nearest_neighbor(neighbors = 9) |>
  set_engine("kkn") |> set_mode("classification")

wf <- workflow() |> add_recipe(rec) |> add_model(spec_knn)
model <- fit(wf, data = lat)
cat("model terlatih pada", nrow(lat), "amatan\n")
```

```
model terlatih pada 749 amatan
```

Penjelasan Kode. Tiga bagian disusun terpisah lalu dirangkai. Resep menetapkan prapemrosesan, spesifikasi model menetapkan keluarga model beserta mesinnya, dan alur kerja menyatukan keduanya. Fungsi fit melatih seluruh rangkaian sekaligus.

Interpretasi Output

Keunggulan pemisahan ini terlihat pada modul berikutnya: mengganti model cukup dengan mengganti spesifikasinya tanpa menyentuh resep, dan sebaliknya. Yang terpenting, karena resep berada di dalam alur kerja, prapemrosesan otomatis dipelajari hanya dari bagian data yang sedang dipakai melatih — pencegahan kebocoran secara rancangan.

Contoh 6.4 | Membaca matriks kekeliruan

Kode 6.4 • R conf_mat dan pembacaannya

```
suppressPackageStartupMessages({library(yardstick); library(dplyr)})

duga <- predict(model, uji) |>
  bind_cols(predict(model, uji, type = "prob")) |>
  bind_cols(uji |> select(gagal_bayar))

cm <- conf_mat(duga, truth = gagal_bayar, estimate = .pred_class)
print(cm)
```

	Truth	
Prediction	ya	tidak
ya	25	22
tidak	60	144

```
tab <- cm$table
cat("\nbenar positif :", tab[1, 1], "\n")
```

```
benar positif : 25
```

```
cat("salah positif :", tab[1, 2], "\n")
```

```
salah positif : 22
```

```
cat("salah negatif :", tab[2, 1], "\n")
```

```
salah negatif : 60
```

```
cat("benar negatif :", tab[2, 2], "\n")
```

```
benar negatif : 144
```

Penjelasan Kode. Dugaan kelas dan dugaan peluang digabungkan dengan sasaran sebenarnya menjadi satu tabel. Matriks kekeliruan dibentuk, lalu keempat selnya dibaca satu per satu agar penamaannya jelas.

Interpretasi Output

Membaca keempat sel secara terpisah adalah kebiasaan yang menolong, sebab tata letak matriks kekeliruan berbeda antar-perangkat lunak — sebagian menaruh dugaan pada baris, sebagian pada kolom. Salah membaca berarti salah menghitung seluruh ukuran turunannya.

Contoh 6.5 | Menghitung beberapa ukuran penilaian

Kode 6.5 • R metric_set dan roc_auc

```
ukuran <- metric_set(yardstick::accuracy, yardstick::precision, yardstick::recall, yardstick::f_meas, yardstick::specificity)
hasil <- ukuran(duga, truth = gagal_bayar, estimate = .pred_class)
print(hasil |> mutate(.estimate = round(.estimate, 4)))
```

```
# A tibble: 5 x 3
  .metric .estimator .estimate
  <chr>    <chr>         <dbl>
1 accuracy binary        0.673
2 precision binary        0.532
3 recall  binary        0.294
4 f_meas  binary        0.379
5 specificity binary        0.868
```

```
auc <- yardstick::roc_auc(duga, truth = gagal_bayar, .pred_ya)
cat("\nROC-AUC:", round(auc$.estimate, 4), "\n")
```

```
ROC-AUC: 0.6287
```

```
cat("acuan mayoritas:", round(max(prop.table(table(uji$gagal_bayar))), 4), "\n")
```

acuan mayoritas: 0.6614

Penjelasan Kode. Beberapa ukuran dihitung sekaligus memakai `metric_set`. Ukuran berbasis peluang seperti ROC-AUC dihitung terpisah karena memerlukan kolom peluang, bukan kolom kelas dugaan.

Interpretasi Output

Bandingkan akurasi terhadap acuan mayoritas; bila keduanya berdekatan, model belum menyumbang apa pun. Perhatikan pula bahwa ROC-AUC bernilai sekitar setengah menandakan model tidak lebih baik daripada menebak acak — dan pada data ini, yang peubahnya dibangkitkan saling bebas terhadap sasaran, hasil itu justru yang benar.

Contoh 6.6 | Memperagakan pertukaran presisi dan recall

Kode 6.6 • R menghitung ukuran pada beberapa ambang

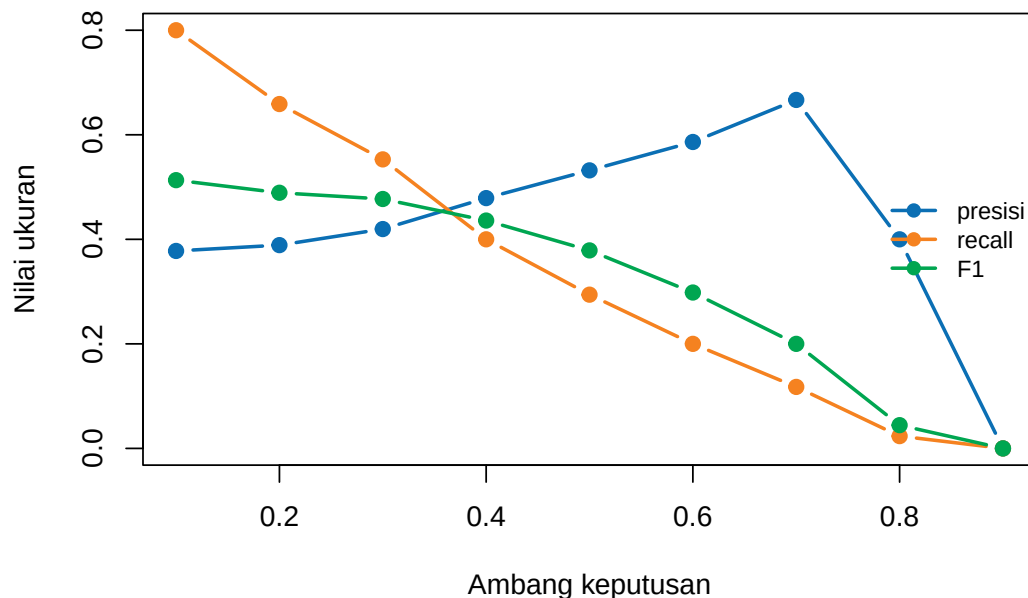
```
ambang <- seq(0.1, 0.9, by = 0.1)
hasil_ambang <- t(sapply(ambang, function(t) {
  d <- duga
  d$dug <- factor(ifelse(d$.pred_ya > t, "ya", "tidak"),
    levels = c("ya", "tidak"))
  c(presisi = yardstick::precision(d, gagal_bayar, dug)$estimate,
    recall = yardstick::recall(d, gagal_bayar, dug)$estimate,
    F1 = yardstick::f_meas(d, gagal_bayar, dug)$estimate)
}))
print(round(cbind(ambang, hasil_ambang), 4))
```

	ambang	presisi	recall	F1
[1,]	0.1	0.3778	0.8000	0.5132
[2,]	0.2	0.3889	0.6588	0.4891
[3,]	0.3	0.4196	0.5529	0.4772
[4,]	0.4	0.4789	0.4000	0.4359
[5,]	0.5	0.5319	0.2941	0.3788
[6,]	0.6	0.5862	0.2000	0.2982
[7,]	0.7	0.6667	0.1176	0.2000
[8,]	0.8	0.4000	0.0235	0.0444
[9,]	0.9	0.0000	0.0000	0.0000

```
matplot(ambang, hasil_ambang, type = "b", pch = 19, lwd = 2, lty = 1,
  col = c("#0B6FB4", "#F58220", "#00A651"),
  xlab = "Ambang keputusan", ylab = "Nilai ukuran",
  main = "Pertukaran Presisi, Recall, dan F1")
legend("right", c("presisi", "recall", "F1"), bty = "n",
```

```
col = c("#0B6FB4", "#F58220", "#00A651"), lwd = 2, pch = 19, cex = .85)
```

Pertukaran Presisi, Recall, dan F1



Penjelasan Kode. Sembilan ambang dicoba. Pada tiap ambang, kelas dugaan dibentuk ulang lalu ketiga ukuran dihitung. Hasilnya ditranspos agar tiap baris mewakili satu ambang, kemudian digambar sebagai tiga garis.

Interpretasi Output

Perhatikan arah kedua garis pertama yang berlawanan — inilah pertukaran yang dibahas pada Teori. Recall tinggi pada ambang rendah karena hampir semua ditandai positif, sedangkan presisi tinggi pada ambang tinggi karena hanya yang paling meyakinkan yang ditandai. Garis F1 memuncak di antaranya. Penting dipahami bahwa ambang yang memaksimumkan F1 belum tentu ambang yang tepat secara kebijakan; biaya kekeliruanlah yang menentukan.

Contoh 6.7 | Menjalankan validasi silang

Kode 6.7 • R vfold_cv dan fit_resamples

```
suppressPackageStartupMessages({library(tune); library(dials)})
set.seed(2026)
```



```
lipatan <- vfold_cv(lat, v = 5, strata = gagal_bayar)
print(lipatan)

# 5-fold cross-validation using stratification
# A tibble: 5 x 2
  splits      id
  <list>     <chr>
1 <split [598/151]> Fold1
2 <split [598/151]> Fold2
3 <split [600/149]> Fold3
4 <split [600/149]> Fold4
5 <split [600/149]> Fold5

hasil_cv <- fit_resamples(
  wf, resamples = lipatan,
  metrics = metric_set(yardstick::roc_auc, yardstick::f_meas, yardstick::accuracy))

collect_metrics(hasil_cv) |>
  mutate(mean = round(mean, 4), std_err = round(std_err, 4)) |>
  select(.metric, mean, n, std_err) |>
  print()

# A tibble: 3 x 4
  .metric mean      n std_err
  <chr>   <dbl> <int>   <dbl>
1 accuracy 0.657     5 0.0107
2 f_meas   0.403     5 0.0194
3 roc_auc  0.620     5 0.0024
```

Penjelasan Kode. Data latih dibagi menjadi lima lipatan berstrata. Fungsi `fit_resamples` melatih dan menilai model pada tiap lipatan tanpa menyentuh data uji sama sekali. Hasilnya dikumpulkan berupa rata-rata dan galat bakunya.

Interpretasi Output

Kolom galat baku sama pentingnya dengan kolom rata-rata. Ia menyatakan seberapa besar hasil berubah antar-lipatan; galat baku yang besar berarti penilaiannya tidak stabil sehingga selisih kecil antar-model tidak berarti apa-apa. Melaporkan rata-rata tanpa galat bakunya menyembunyikan ketidakpastian itu.

Contoh 6.8 | Memperagakan pemodelan berlebih (*overfitting*)

Kode 6.8 • R membandingkan kinerja latih dan uji

```

nilai_k <- c(1, 3, 5, 9, 15, 25, 51)
banding <- t(sapply(nilai_k, function(k) {
  s <- nearest_neighbor(neighbors = k) |>
    set_engine("kknn") |> set_mode("classification")
  m <- fit(workflow() |> add_recipe(rec) |> add_model(s), data = lat)
  d_lat <- predict(m, lat) |> bind_cols(lat |> select(gagal_bayar))
  d_uji <- predict(m, uji) |> bind_cols(uji |> select(gagal_bayar))
  c(latih = yardstick::accuracy(d_lat, gagal_bayar, .pred_class)$estimate,
    uji = yardstick::accuracy(d_uji, gagal_bayar, .pred_class)$estimate)
}))
print(round(cbind(k = nilai_k, banding), 4))

```

```

      k latih  uji
[1,]  1 1.0000 0.6255
[2,]  3 1.0000 0.6255
[3,]  5 0.9306 0.6574
[4,]  9 0.8518 0.6733
[5,] 15 0.8051 0.7171
[6,] 25 0.7717 0.6932
[7,] 51 0.7143 0.6892

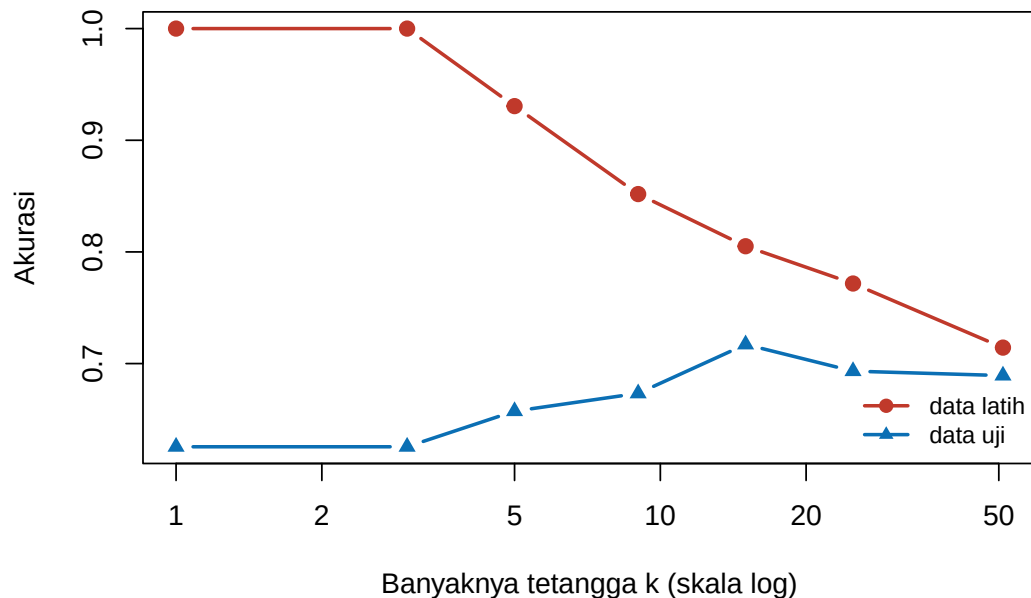
```

```

matplot(nilai_k, banding, type = "b", pch = c(19, 17), lwd = 2, lty = 1,
  col = c("#C0392B", "#0B6FB4"), log = "x",
  xlab = "Banyaknya tetangga k (skala log)", ylab = "Akurasi",
  main = "Akurasi pada Data Latih dan Data Uji")
legend("bottomright", c("data latih", "data uji"), bty = "n",
  col = c("#C0392B", "#0B6FB4"), lwd = 2, pch = c(19, 17), cex = .85)

```

Akurasi pada Data Latih dan Data Uji



Penjelasan Kode. Tujuh nilai k dicoba. Pada tiap nilai, model dilatih lalu dinilai dua kali: pada data latih dan pada data uji. Keduanya digambar bersama dengan sumbu k berskala logaritma.

Interpretasi Output

Pada k bernilai satu, akurasi latih mencapai nilai tertinggi karena tiap amatan menjadi tetangga terdekat bagi dirinya sendiri — model menghafal sempurna. Namun akurasi ujinya tidak ikut tinggi, dan jarak antara kedua garis itulah wujud pemodelan berlebihan (*overfitting*). Seiring k membesar, model menjadi lebih kaku dan kedua garis mendekat. Pemilihan k karena itu adalah pencarian titik seimbang, bukan pencarian akurasi latih tertinggi.

Memilih model berdasarkan kinerja pada data latih. Godaan memilih model berakurasi latih tertinggi selalu ada, sebab angkanya paling menyenangkan. Namun akurasi latih hanya mengukur seberapa baik model menghafal, bukan seberapa baik ia meramal. Pemilihan model **wajib** berdasarkan validasi silang atau data uji.

Contoh 6.9 | Menala hiperparameter dengan grid

Kode 6.9 • R `tune_grid` dan `select_best`

```
spec_tune <- nearest_neighbor(neighbors = tune()) |>
  set_engine("kknn") |> set_mode("classification")
wf_tune <- workflow() |> add_recipe(rec) |> add_model(spec_tune)

set.seed(2026)
hasil_tune <- tune_grid(
  wf_tune, resamples = lipatan,
  grid = data.frame(neighbors = c(3, 5, 9, 15, 25, 41)),
  metrics = metric_set(yardstick::roc_auc, yardstick::f_meas))

collect_metrics(hasil_tune) |>
  filter(.metric == "roc_auc") |>
  mutate(mean = round(mean, 4), std_err = round(std_err, 4)) |>
  select(neighbors, mean, std_err) |>
  print()
```

```
# A tibble: 6 x 3
  neighbors mean std_err
    <dbl> <dbl> <dbl>
1         3 0.606 0.0052
2         5 0.611 0.003
3         9 0.620 0.0024
4        15 0.629 0.0069
5        25 0.641 0.0077
6        41 0.652 0.0101
```

```
terbaik <- select_best(hasil_tune, metric = "roc_auc")
cat("\nk terbaik menurut ROC-AUC:", terbaik$neighbors, "\n")
```

```
k terbaik menurut ROC-AUC: 41
```

Penjelasan Kode. Hiperparameter ditandai tune sehingga nilainya dicari, bukan ditetapkan. Enam nilai dicoba pada kelima lipatan, sehingga tiga puluh model dilatih. Fungsi `select_best` memilih nilai berkinerja terbaik menurut ukuran yang ditetapkan.

Interpretasi Output

Perhatikan galat baku pada tiap baris. Bila selisih rata-rata antar-nilai k lebih kecil daripada galat bakunya, maka pemilihan terbaik sesungguhnya sembarang. Dalam keadaan demikian, memilih nilai k yang lebih besar — yakni model yang lebih sederhana — lebih dapat dipertanggungjawabkan daripada memilih yang rata-ratanya kebetulan tertinggi.

Contoh 6.10 | Menyelesaikan alur dan menilai pada data uji

Kode 6.10 • R `finalize_workflow` dan `last_fit`

```

wf_final <- finalize_workflow(wf_tune, terbaik)
akhir <- last_fit(wf_final, split = bagi,
                  metrics = metric_set(yardstick::roc_auc, yardstick::f_meas, yardstick::accuracy))

collect_metrics(akhir) |>
  mutate(.estimate = round(.estimate, 4)) |>
  select(.metric, .estimate) |> print()

```

```

# A tibble: 3 x 2
  .metric .estimate
  <chr>    <dbl>
1 f_meas  0.231
2 accuracy 0.681
3 roc_auc  0.674

```

```

cat("\nacuan tebakan mayoritas:",
    round(max(prop.table(table(uji$gagal_bayar))), 4), "\n")

```

```

acuan tebakan mayoritas: 0.6614

```

Penjelasan Kode. Alur kerja diselesaikan dengan nilai hiperparameter terbaik, lalu `last_fit` melatih ulang pada seluruh data latih dan menilai satu kali pada data uji. Inilah satu-satunya kali data uji disentuh.

Interpretasi Output

Penekanan pada kalimat terakhir itu penting. Data uji dipakai sekali saja, di akhir, setelah seluruh keputusan diambil. Memakainya berulang untuk memilih di antara beberapa pilihan mengubahnya menjadi data latih terselubung, sehingga angka akhirnya tidak lagi menggambarkan kinerja pada data baru.

Contoh 6.11 | Kerangka yang sama untuk sasaran numerik**Kode 6.11 • R** `linear_reg` dan ukuran regresi

```

spec_reg <- linear_reg() |> set_engine("lm") |> set_mode("regression")
rec_reg <- recipe(pendapatan ~ umur + lama_kerja + rasio_utang +
                  jumlah_tanggungan + pendidikan, data = lat) |>
  step_dummy(all_nominal_predictors())
wf_reg <- workflow() |> add_recipe(rec_reg) |> add_model(spec_reg)

m_reg <- fit(wf_reg, data = lat)

```

```
d_reg <- predict(m_reg, uji) |> bind_cols(uji |> select(pendapatan))

metric_set(yardstick::rmse, yardstick::mae, yardstick::rsq)(d_reg, truth = pendapatan, estimate = .pred) |>
  mutate(.estimate = round(.estimate, 2)) |> print()

# A tibble: 3 x 3
  .metric .estimator .estimate
  <chr>   <chr>       <dbl>
1 rmse    standard     5157.
2 mae     standard     3921.
3 rsq     standard         0

cat("\nsimpangan baku sasaran:", round(sd(uji$pendapatan), 2), "\n")

simpangan baku sasaran: 5152

cat("RMSE model dibandingkan simpangan baku:",
    round(yardstick::rmse(d_reg, pendapatan, .pred)$estimate / sd(uji$pendapatan), 3), "\n")

RMSE model dibandingkan simpangan baku: 1.001
```

Penjelasan Kode. Sasaran diganti menjadi peubah numerik. Spesifikasi model berganti menjadi regresi linear dengan mode regression, dan ukuran penilaiannya berganti menjadi RMSE, MAE, dan R-kuadrat. Sisa kerangkanya sama persis.

Interpretasi Output

Perbandingan RMSE terhadap simpangan baku sasaran adalah acuan yang setara dengan tebakan mayoritas pada klasifikasi. Rasio mendekati satu berarti model tidak lebih baik daripada sekadar menebak rata-rata. Pada data yang peubahnya dibangkitkan saling bebas, rasio mendekati satu justru hasil yang benar.

Contoh 6.12 | Membandingkan tiga model secara adil

Kode 6.12 • R membandingkan model pada lipatan yang sama

```
daftar_model <- list(
  knn = nearest_neighbor(neighbors = 15) |> set_engine("kknn"),
  pohon = decision_tree(cost_complexity = 0.01) |> set_engine("rpart"),
  hutan = rand_forest(trees = 200) |> set_engine("ranger"))

set.seed(2026)

banding_model <- lapply(names(daftar_model), function(nm) {
  spec <- set_mode(daftar_model[[nm]], "classification")
```

```
w <- workflow() |> add_recipe(rec) |> add_model(spec)
cv <- fit_resamples(w, resamples = lipatan,
                    metrics = metric_set(yardstick::roc_auc, yardstick::f_meas))
collect_metrics(cv) |> mutate(model = nm)
})
ringkas_model <- do.call(rbind, banding_model) |>
  select(model, .metric, mean, std_err) |>
  mutate(mean = round(mean, 4), std_err = round(std_err, 4))
print(as.data.frame(ringkas_model), row.names = FALSE)
```

```
model .metric mean std_err
knn f_meas 0.3597 0.0273
knn roc_auc 0.6290 0.0069
pohon f_meas 0.4249 0.0249
pohon roc_auc 0.6167 0.0276
hutan f_meas 0.4441 0.0110
hutan roc_auc 0.7016 0.0147
```

Penjelasan Kode. Tiga keluarga model disusun sebagai daftar spesifikasi. Perulangan melatih masing-masing pada **lipatan yang sama**, sehingga perbandingannya adil. Hasilnya digabungkan menjadi satu tabel.

Interpretasi Output

Kunci keadilan perbandingan adalah pemakaian objek lipatan yang sama bagi seluruh model; bila tiap model memakai pembagian sendiri, selisih yang terlihat boleh jadi hanya akibat pembagian yang berbeda. Perhatikan pula bahwa resep yang sama dipakai bagi ketiganya — padahal pohon dan hutan acak sesungguhnya tidak memerlukan pembakuan. Pembakuan tidak merugikan keduanya, tetapi pada pekerjaan sungguhan resep sebaiknya disesuaikan dengan kebutuhan tiap model, dan itulah bahasan modul berikutnya.

Contoh 6.13 | Menggambarkan perbandingan model

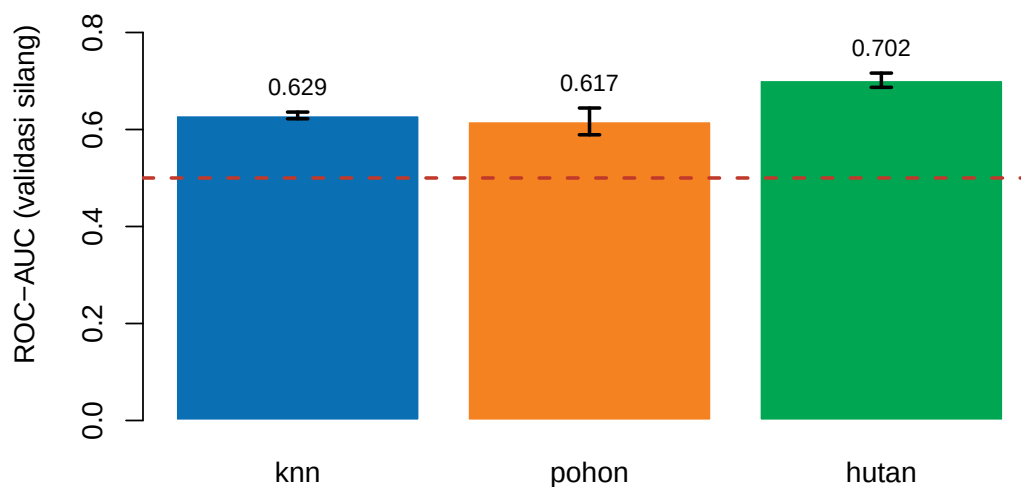
Kode 6.13 • R barplot dengan palang galat baku

```
auc_model <- ringkas_model |> filter(.metric == "roc_auc")

bp <- barplot(auc_model$mean, names.arg = auc_model$model,
              col = c("#0B6FB4", "#F58220", "#00A651"), border = "white",
              ylim = c(0, max(auc_model$mean + auc_model$std_err) * 1.3),
              ylab = "ROC-AUC (validasi silang)",
              main = "Perbandingan Tiga Model")
```

```
arrows(bp, auc_model$mean - auc_model$std_err,
       bp, auc_model$mean + auc_model$std_err,
       angle = 90, code = 3, length = 0.06, lwd = 2)
abline(h = 0.5, col = "#C0392B", lty = 2, lwd = 2)
text(bp, auc_model$mean + auc_model$std_err, round(auc_model$mean, 3),
     pos = 3, cex = .85)
```

Perbandingan Tiga Model



Penjelasan Kode. Rata-rata ROC-AUC tiap model digambar sebagai batang, dilengkapi palang galat baku memakai arrows dengan sudut sembilan puluh derajat di kedua ujungnya. Garis acuan pada nilai setengah menandai kinerja tebakan acak.

Interpretasi Output

Palang galat baku inilah yang mencegah kesimpulan tergesa. Bila palang antar-model saling bertumpang tindih, selisih rata-ratanya tidak dapat dianggap berarti — dan menyatakan satu model lebih baik dalam keadaan demikian adalah klaim yang tidak didukung data. Garis acuan pada setengah menegaskan apakah model mana pun sudah melampaui tebakan acak.

Contoh 6.14 | Menggambarkan kurva ROC

Kode 6.14 • R roc_curve dan penggambarannya

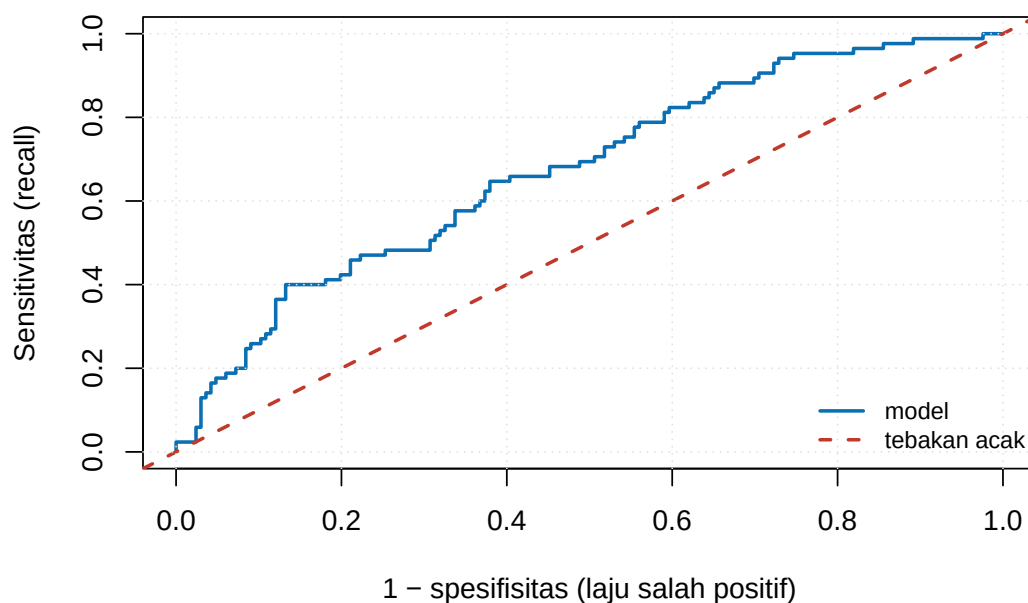

```

prob_uji <- collect_predictions(akhir)
kurva <- roc_curve(prob_uji, truth = gagal_bayar, .pred_ya)

plot(1 - kurva$specificity, kurva$sensitivity, type = "l", lwd = 2,
     col = "#0B6FB4", xlim = c(0, 1), ylim = c(0, 1),
     xlab = "1 - spesifisitas (laju salah positif)",
     ylab = "Sensitivitas (recall)",
     main = "Kurva ROC pada Data Uji")
abline(0, 1, col = "#C0392B", lty = 2, lwd = 2)
grid(col = "grey88")
legend("bottomright", c("model", "tebakan acak"), bty = "n",
      col = c("#0B6FB4", "#C0392B"), lwd = 2, lty = c(1, 2), cex = .85)

```

Kurva ROC pada Data Uji



Penjelasan Kode. Dugaan peluang pada data uji diambil, lalu kurva ROC dihitung sebagai pasangan sensitivitas dan spesifisitas pada berbagai ambang. Garis diagonal ditambahkan sebagai acuan tebakan acak.

Interpretasi Output

Kurva yang berimpit dengan garis diagonal menandakan model tidak lebih baik daripada menebak, sedangkan kurva yang membusur ke kiri atas menandakan daya pisah yang baik. Keunggulan ROC dibandingkan akurasi adalah ia menilai seluruh ambang sekaligus, sehingga tidak bergantung pada

satu ambang tertentu yang pemilihannya sendiri merupakan keputusan kebijakan.

6.7 Latihan Mandiri

Kerjakan bertahap. Gunakan data/data_kredit.csv.

6.7.0.0.1 Tingkat Dasar (melengkapi kode)

Latihan 6.1 | Pembagian berstrata

```
library(rsample)
kredit <- read.csv("data/data_kredit.csv", stringsAsFactors = TRUE)
set.seed(2026)
bagi <- ____ (kredit, prop = 0.75, strata = gagal_bayar)
lat <- ____ (bagi); uji <- ____ (bagi)
round(prop.table(table(lat$gagal_bayar)), 4)
```

Latihan 6.2 | Resep dan alur kerja

```
library(recipes); library(parsnip); library(workflows)
rec <- recipe(gagal_bayar ~ ., data = lat) |>
  step_dummy(all_nominal_predictors()) |>
  ____ (all_numeric_predictors()) # pembakuan
spec <- ____ (neighbors = 9) |> set_engine("kknn") |> set_mode("____")
wf <- workflow() |> add_recipe(rec) |> ____ (spec)
```

Latihan 6.3 | Ukuran penilaian

```
library(yardstick)
cm <- ____ (duga, truth = gagal_bayar, estimate = .pred_class)
u <- ____ (accuracy, precision, recall, f_meas)
u(duga, truth = gagal_bayar, estimate = .pred_class)
____ (duga, truth = gagal_bayar, .pred_ya) # ROC-AUC
```

6.7.0.0.2 Tingkat Menengah (menulis sendiri)

Latihan 6.4 | Pemodelan berlebih

Latih model KNN dengan tujuh nilai k berbeda. Untuk masing-masing, hitung akurasi pada data latih **dan** data uji. Sajikan sebagai tabel dan grafik, lalu tentukan pada nilai k berapa pemodelan berlebih (*overfitting*) paling parah beserta buktinya.

Latihan 6.5 | Membetulkan kode yang salah

Kode berikut memuat empat kesalahan (pembagian tanpa penstrataan, model dinilai pada data latih, roc_auc diberi kolom kelas bukan peluang, dan data uji dipakai berulang untuk memilih). Temukan dan perbaiki, lalu jelaskan akibat masing-masing.

```
library(rsample); library(yardstick)
set.seed(2026)

# (a) pembagian tanpa penstrataan pada data timpang
bagi <- initial_split(kredit, prop = 0.75)
lat <- training(bagi); uji <- testing(bagi)

model <- fit(wf, data = lat)

# (b) model dinilai pada data yang dipakai melatihnya
d <- predict(model, lat) |> bind_cols(lat |> select(gagal_bayar))
yardstick::accuracy(d, gagal_bayar, .pred_class)

# (c) roc_auc diberi kolom kelas, bukan kolom peluang
yardstick::roc_auc(d, gagal_bayar, .pred_class)

# (d) data uji dicoba berulang untuk memilih k terbaik
for (k in c(3, 5, 9, 15)) {
  m <- fit(wf_k(k), data = lat)
  du <- predict(m, uji) |> bind_cols(uji |> select(gagal_bayar))
  cat(k, yardstick::accuracy(du, gagal_bayar, .pred_class)$estimate, "
")
}
```

Keluaran yang benar. Setelah diperbaiki, kode yang dipakai adalah sebagai berikut.

```
# (a) penstrataan pada data timpang
bagi <- initial_split(kredit, prop = 0.75, strata = gagal_bayar)
lat <- training(bagi); uji <- testing(bagi)

# (b) penilaian pada data UJI, bukan data latih
model <- fit(wf, data = lat)
d <- predict(model, uji) |>
  bind_cols(predict(model, uji, type = "prob")) |>
  bind_cols(uji |> select(gagal_bayar))
yardstick::accuracy(d, gagal_bayar, .pred_class)
```

```
# (c) roc_auc memerlukan kolom PELUANG
yardstick::roc_auc(d, gagal_bayar, .pred_ya)

# (d) pemilihan k memakai VALIDASI SILANG pada data latih
set.seed(2026)
lipatan <- vfold_cv(lat, v = 5, strata = gagal_bayar)
hasil <- tune_grid(wf_tune, resamples = lipatan,
                  grid = data.frame(neighbors = c(3, 5, 9, 15)),
                  metrics = metric_set(yardstick::roc_auc))
select_best(hasil, metric = "roc_auc")
```

Pemeriksaan yang wajib berhasil: proporsi kelas pada lat dan uji berselisih kurang dari 0,02, dan roc_auc menghasilkan nilai tanpa galat.

6.7.0.0.3 Tingkat Lanjut (keputusan)

Latihan 6.6 | Memilih ukuran penilaian

Untuk masing-masing keadaan berikut, tentukan ukuran penilaian yang Anda utamakan beserta alasannya, dan sebutkan **biaya kekeliruan** mana yang lebih besar: (a) penapisan awal penyakit menular; (b) penandaan transaksi mencurigakan yang menuntut pemeriksaan manual mahal; (c) pendugaan kelayakan kredit dengan kelas gagal bayar hanya 3 persen; (d) pemilihan sasaran promosi berbiaya rendah.

6.8 Penugasan

Sebagai calon analis pada instansi statistik, susun laporan pemodelan awal atas `data_kredit.csv` (*data simulasi*) yang menerapkan kerangka supervised learning secara benar, lengkap dengan pembuktian bahwa penilaiannya valid.

Konteks. Sebuah unit hendak membangun model penduga risiko gagal bayar. Pimpinan meminta bukan hanya angka kinerja, melainkan juga jaminan bahwa angka itu menggambarkan kinerja pada data baru.

Tugas 6.1 | Penyiapan dan Pembagian

1. Periksa **ketimpangan kelas** sasaran. Laporkan proporsi, rasio, dan akurasi acuan tebakan mayoritas.
2. Bagi data dengan penstrataan. Buktikan penstrataannya bekerja dengan menyandingkan proporsi kelas pada ketiga bagian.
3. Susun resep prapemrosesan yang sesuai bagi model berbasis jarak. **Jelaskan mengapa** pembakuan wajib bagi model ini.

Tugas 6.2 | Pemodelan dan Penilaian

4. Latih model KNN, lalu sajikan **matriks kekeliruan** beserta pembacaan keempat selnya.
5. Hitung **minimal lima ukuran** penilaian. Bandingkan akurasi terhadap acuan mayoritas, lalu simpulkan apakah model menyumbang sesuatu.
6. Peragakan **pertukaran presisi dan recall** pada minimal tujuh ambang. Sajikan tabel dan grafik, lalu tentukan ambang yang Anda pilih beserta alasan kebijakannya.

Tugas 6.3 | Validasi dan Penalaan

7. Jalankan **validasi silang lima lipatan** berstrata. Laporkan rata-rata beserta galat bakunya, lalu jelaskan makna galat baku itu.
8. Peragakan **pemodelan berlebih (*overfitting*)** dengan membandingkan kinerja latih dan uji pada tujuh nilai k . Sajikan grafik dan tafsirkan.
9. Lakukan **penalaan hiperparameter** dengan grid. Bila selisih antar-nilai lebih kecil daripada galat bakunya, jelaskan nilai mana yang Anda pilih dan mengapa.
10. Selesaikan alur dengan `last_fit()`, sajikan kinerja akhir pada data uji beserta **kurva ROC**. Jelaskan mengapa data uji hanya boleh disentuh sekali.

11. Susun laporan R Markdown berisi kode, keluaran, gambar, dan interpretasi. Tutup dengan **ringkasan (maksimal 300 kata)** memuat: kinerja akhir dibandingkan acuan, ukuran yang diutamakan beserta alasannya, keterbatasan, dan penegasan bahwa data merupakan simulasi sehingga kinerja mendekati tebakan acak adalah hasil yang wajar.

6.9 Tambahan: Membandingkan caret dan tidymodels Berdampingan

Bagian ini menjalankan kasus yang sama pada dua kerangka pemodelan R. Tujuannya bukan menentukan mana yang lebih baik, melainkan memahami bahwa hasil pemodelan bergantung pada keputusan teknis yang kerap tak terlihat.

Contoh 6.15 | Kasus yang sama pada caret

Kode 6.15 • R caret::train dengan validasi silang

```
suppressPackageStartupMessages(library(caret))
set.seed(2026)

kendali <- trainControl(method = "cv", number = 5,
                        classProbs = TRUE,
                        summaryFunction = twoClassSummary)

lat_caret <- lat[, setdiff(names(lat), "id")]
m_caret <- train(gagal_bayar ~ ., data = lat_caret, method = "knn",
                preprocess = c("center", "scale"),
                tuneGrid = data.frame(k = c(3, 5, 9, 15, 25, 41)),
                trControl = kendali, metric = "ROC")

print(round(m_caret$results[, c("k", "ROC", "Sens", "Spec")], 4))
```

	k	ROC	Sens	Spec
1	3	0.5968	0.3691	0.7706
2	5	0.6117	0.3493	0.8027
3	9	0.6183	0.2776	0.8612
4	15	0.6357	0.2061	0.8933
5	25	0.6479	0.1507	0.9355
6	41	0.6599	0.1031	0.9637

```
cat("\nk terbaik menurut caret:", m_caret$bestTune$k, "\n")
```

```
k terbaik menurut caret: 41
```

Penjelasan Kode. Seluruh alur ditulis dalam satu pemanggilan `train`. Prapemrosesan ditetapkan lewat argumen `preProcess`, skema resampling lewat `trainControl`, dan ruang pencarian lewat `tuneGrid`. Kolom `id` dibuang lebih dahulu karena `caret` tidak mengenal peran penanda.

Interpretasi Output

Keringkasan inilah keunggulan `caret`: satu fungsi menangani prapemrosesan, validasi silang, dan penalaan sekaligus. Kekurangannya juga di situ — urutan prapemrosesan ditentukan `caret` dan tidak dapat diatur, sedangkan pada `recipes` urutannya sepenuhnya di tangan penyusun. Perhatikan pula bahwa kolom penanda harus dibuang manual, dan lupa melakukannya berakibat penanda ikut dimodelkan.

Contoh 6.16 | Menyandingkan hasil kedua kerangka

Kode 6.16 • R menggabungkan dan membandingkan hasil

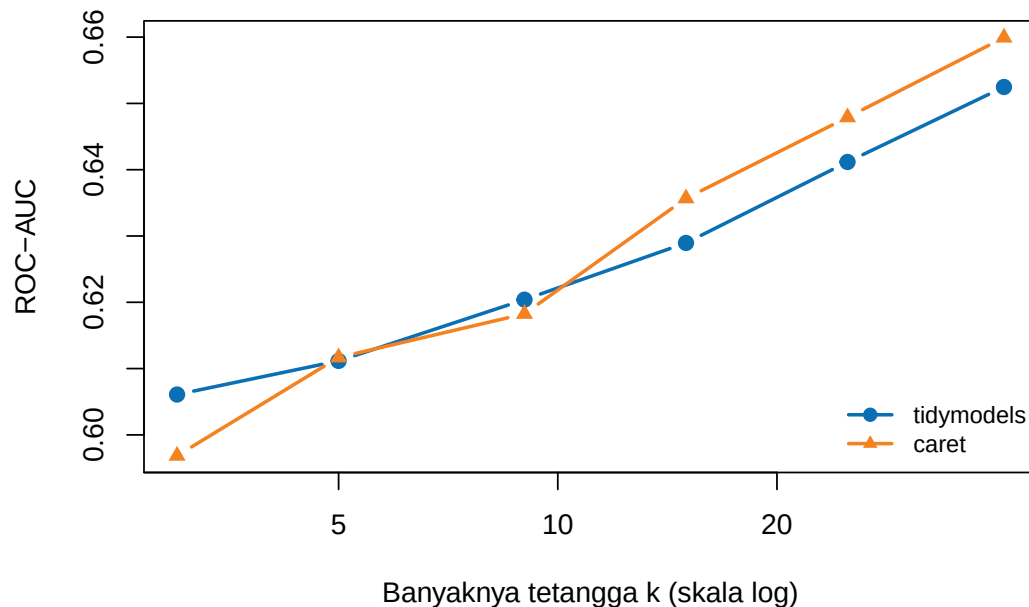
```
hasil_tm <- collect_metrics(hasil_tune) |>
  filter(.metric == "roc_auc") |>
  select(k = neighbors, tidymodels = mean)
hasil_ct <- data.frame(k = m_caret$results$k, caret = m_caret$results$ROC)

banding2 <- merge(hasil_tm, hasil_ct, by = "k")
print(round(banding2, 4))
```

	k	tidymodels	caret
1	3	0.6061	0.5968
2	5	0.6112	0.6117
3	9	0.6204	0.6183
4	15	0.6290	0.6357
5	25	0.6412	0.6479
6	41	0.6525	0.6599

```
matplot(banding2$k, banding2[, c("tidymodels", "caret")],
  type = "b", pch = c(19, 17), lwd = 2, lty = 1, log = "x",
  col = c("#0B6FB4", "#F58220"),
  xlab = "Banyaknya tetangga k (skala log)", ylab = "ROC-AUC",
  main = "Hasil Dua Kerangka pada Kasus yang Sama")
legend("bottomright", c("tidymodels", "caret"), bty = "n",
  col = c("#0B6FB4", "#F58220"), lwd = 2, pch = c(19, 17), cex = .85)
```


Hasil Dua Kerangka pada Kasus yang Sama



Penjelasan Kode. Hasil ROC-AUC dari kedua kerangka digabungkan menurut nilai k, lalu disandingkan sebagai tabel dan grafik. Sumbu k memakai skala logaritma karena nilainya berlipat.

Interpretasi Output

Kedua garis kemungkinan tidak berimpit, dan justru itulah pelajarannya. Penyebabnya bermacam-macam: pembentukan lipatan yang berbeda meski benihnya sama, penanganan peubah kategori yang berbeda, dan cara menghitung ROC yang sedikit berlainan. Selisih semacam ini lazim dan tidak berarti salah satunya keliru.

Melaporkan angka tanpa menyebut prosedurnya. Karena hasil bergantung pada kerangka, benih acak, skema resampling, dan penanganan prapemrosesan, maka melaporkan angka kinerja tanpa menyebutkan seluruhnya membuat hasil itu **tidak dapat direproduksi**. Laporan yang baik selalu mencantumkan: perangkat lunak beserta versinya, skema pembagian dan resampling, benih acak, urutan prapemrosesan, dan ukuran penilaian yang dipakai.

caret menutupi fungsi yardstick. Memuat caret setelah yardstick membuat precision, recall, dan beberapa fungsi lain merujuk versi caret, sehingga `metric_set()` gagal dengan pesan mengenai kombinasi fungsi metrik. Jalan keluarnya menuliskan nama paket secara tegas, misalnya `yardstick::precision`. Bentrokan nama semacam ini lazim ketika beberapa paket dipakai bersama,

dan menuliskan nama paket adalah kebiasaan yang menghematkan banyak penelusuran.

Tips dan Praktik Baik. Bila hasil dua kerangka berselisih jauh — bukan sekadar sedikit — itu pertanda ada perbedaan perlakuan yang perlu ditelusuri, bukan sekadar derau. Periksa lebih dahulu: apakah prapemrosesannya sama, apakah kolom yang dipakai sama, dan apakah kelas yang diperhatikan sama.

Kesimpulan bagian ini: kedua kerangka sama-sama valid. caret unggul pada keringkasan dan kelengkapan model, tidymodels unggul pada kendali prapemrosesan dan pencegahan kebocoran secara rancangan. Modul berikutnya dan seterusnya memakai tidymodels, sedangkan caret tetap dikenali agar kode lama dapat dibaca.

6.10 Ringkasan Modul

Daftar Periksa penguasaan:

- ☐ Saya dapat menjelaskan kerangka supervised learning beserta empat unsurnya.
- ☐ Saya membagi data dengan penstrataan dan dapat membuktikan hasilnya.
- ☐ Saya dapat menjelaskan mengapa validasi silang lebih stabil daripada satu pembagian.
- ☐ Saya dapat membaca matriks kekeliruan dan menurunkan ukurannya.
- ☐ Saya selalu membandingkan akurasi terhadap acuan tebakan mayoritas.
- ☐ Saya memahami pertukaran presisi dan recall serta akibat pemilihan ambang.
- ☐ Saya dapat mengenali pemodelan berlebih (*overfitting*) dari jarak kinerja latih dan uji.
- ☐ Saya memperhatikan galat baku, bukan hanya rata-rata, saat memilih model.
- ☐ Saya menyentuh data uji hanya sekali, di akhir.
- ☐ Saya dapat menjelaskan mengapa dua kerangka dapat memberi hasil berbeda.

Kartu Referensi Cepat.

Keperluan	Perintah
nMembagi berstrata	<code>initial_split(d, prop =, strata =)</code>
nLipatan validasi silang	<code>vfold_cv(lat, v = 5, strata =)</code>
nSpesifikasi model	<code>nearest_neighbor(), decision_tree()</code>
nMenetapkan mesin	<code>set_engine("kknn")</code>
nMenetapkan mode	<code>set_mode("classification")</code>
nMerangkai	<code>workflow() > add_recipe() > add_model()</code>
nMelatih	<code>fit(wf, data = lat)</code>
nValidasi silang	<code>fit_resamples(wf, resamples =)</code>
nPenalaan	<code>tune_grid(wf, resamples =, grid =)</code>
nMemilih terbaik	<code>select_best(hasil, metric =)</code>
nMenyelesaikan	<code>finalize_workflow(), last_fit()</code>
nMatriks kekeliruan	<code>conf_mat(d, truth =, estimate =)</code>
nBeberapa ukuran	<code>metric_set(accuracy, f_meas)</code>
nROC	<code>roc_auc(), roc_curve()</code>

Modul ini membangun kerangka yang berlaku bagi seluruh model pada modul berikutnya. Prinsip yang perlu dibawa: bagi data sebelum melakukan apa pun, nilai dengan validasi silang bukan pada data latih, bandingkan selalu terhadap acuan, perhatikan galat baku sebelum menyimpulkan selisih, dan sentuh data uji hanya sekali. Modul berikutnya menyusun proposal proyek yang menerapkan kerangka ini pada persoalan pilihan sendiri.

6.11 Referensi

- Kuhn, M., & Silge, J. (2022). *Tidy Modeling with R*. O'Reilly Media.
- Kuhn, M., & Johnson, K. (2013). *Applied Predictive Modeling*. Springer.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2nd ed.). Springer.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
- Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot on imbalanced datasets. *PLOS ONE*, 10(3).

MODUL 7

PERSIAPAN PROYEK DATA MINING

Pertemuan Minggu ke-7

Sub-CPMK 7: Mahasiswa mampu merumuskan persoalan data mining beserta kriteria sukses yang terukur, menilai kecukupan data dan sumber daya, menetapkan biaya kekeliruan, serta menyusun proposal proyek berdasarkan kerangka CRISP-DM.

Penyusun: Achmad Syahrul Choir



Daftar Isi

7.1	Deskripsi Modul	206
7.2	Tujuan Praktikum	206
7.3	Alat dan Bahan	206
7.4	Instruksi Pengerjaan	207
7.5	Teori Singkat	207
7.5.0.1	Kedudukan pada CRISP-DM	207
7.5.0.2	Dari Kebutuhan Menjadi Persoalan Data Mining	207
7.5.0.3	Kriteria Sukses yang Terukur	208
7.5.0.4	Biaya Kekeliruan Tidak Simetris	208
7.5.0.5	Menilai Kecukupan Data	208
7.5.0.6	Risiko Proyek	209
7.6	Contoh Praktikum dengan R	210
7.7	Latihan Mandiri	224
7.8	Penugasan	227
7.9	Tambahan: Menyusun Piagam Proyek	228
7.10	Ringkasan Modul	232
7.11	Referensi	233

7.1 Deskripsi Modul

Enam modul sebelumnya membangun keterampilan teknis. Modul ini mengembalikan perhatian ke pertanyaan yang mendahului seluruh keterampilan itu: persoalan apa yang hendak dijawab, dan bagaimana kita tahu jawabannya berguna.

Pertanyaan itu tampak sederhana tetapi kerap terlewat. Banyak proyek data mining gagal bukan karena algoritmanya keliru melainkan karena tidak pernah jelas apa yang hendak dicapai, sehingga hasilnya tidak dapat dinilai berhasil atau tidak. Model berakurasi tinggi yang tak menjawab kebutuhan siapa pun tetaplah kegagalan.

Modul ini menyusun proposal proyek yang akan dikerjakan pada Modul 13 dan 14. Materinya mencakup perumusan persoalan beserta kriteria sukses terukur, penilaian kecukupan data dan sumber daya, penetapan biaya kekeliruan, penyusunan jadwal dan daftar risiko, serta pemeriksaan kelayakan etika.

Tentang data. Berkas data pada modul ini adalah **data simulasi** (`set.seed(2026)`), **bukan data nyata** dan **bukan data resmi** instansi mana pun. Konteks penerima bantuan dipakai semata sebagai latihan perumusan persoalan.

7.2 Tujuan Praktikum

Setelah menyelesaikan praktikum ini secara tuntas, mahasiswa diharapkan mampu:

1. **Merumuskan** persoalan data mining dari kebutuhan pengguna.
2. **Menetapkan** kriteria sukses yang terukur beserta acuan pembandingnya.
3. **Menilai** kecukupan data melalui kurva belajar.
4. **Menyusun** matriks biaya kekeliruan dan memakainya memilih ambang.
5. **Menaksir** kebutuhan sumber daya dan menyusun jadwal proyek.
6. **Mengenali** risiko proyek beserta langkah pencegahannya.
7. **Memeriksa** kelayakan etika sebelum proyek dimulai.
8. **Menyusun** proposal proyek lengkap berdasarkan kerangka CRISP-DM.

7.3 Alat dan Bahan

Perangkat dan bahan yang diperlukan:

- R (RStudio) dengan paket `recipes`, `rsample`, `parsnip`, `workflows`, dan `yardstick`.
- Berkas data: `data/data_proyek.csv`.
- Templat proposal disediakan pada bagian Tambahan.

7.4 Instruksi Pengerjaan

Ikuti tahapan berikut:

Sebelum praktikum: siapkan berkas data; pikirkan satu persoalan nyata yang menarik bagi Anda dan dapat dijawab dengan data serupa.

Selama praktikum: kerjakan Contoh berurutan. Berbeda dari modul lain, keluaran utama modul ini adalah **dokumen proposal**, bukan model.

Pembagian kerja: Contoh dan Latihan dikerjakan di kelas; **Penugasan berupa proposal dikerjakan di luar sesi** dan akan dipakai pada Modul 13 dan 14.

Pengumpulan: susun proposal sebagai laporan R Markdown, render ke PDF, kumpulkan melalui Google Classroom.

7.5 Teori Singkat

7.5.0.1 Kedudukan pada CRISP-DM

Modul ini sepenuhnya menempati tahap pertama, Business Understanding, ditambah perencanaan bagi tahap-tahap berikutnya. Tahap ini paling sering diremehkan karena tidak menghasilkan kode maupun angka, padahal justru menentukan apakah seluruh pekerjaan berikutnya berguna.

7.5.0.2 Dari Kebutuhan Menjadi Persoalan Data Mining

Pengguna jarang menyampaikan kebutuhannya dalam bahasa data mining. Yang terdengar biasanya berupa keluhan atau harapan — misalnya bantuan sering tidak tepat sasaran. Tugas analis adalah menerjemahkannya menjadi persoalan yang dapat dikerjakan dan dinilai.

Tiga tingkat perumusan.

Kebutuhan	“bantuan sering tidak tepat sasaran”
Pertanyaan	“rumah tangga mana yang paling mungkin layak menerima bantuan?”
Tugas teknis	klasifikasi biner dengan sasaran status penerima, dinilai dengan recall pada kelas layak

Ketiganya harus tertulis, bukan hanya yang ketiga. Menuliskan tingkat pertama menjaga agar pekerjaan tidak melenceng, sedangkan menuliskan tingkat kedua memastikan pengguna dan analis memahami hal yang sama.

7.5.0.3 Kriteria Sukses yang Terukur

Kriteria sukses yang baik dapat diperiksa kebenarannya oleh orang lain tanpa perdebatan. Pernyataan seperti model harus akurat tidak memenuhi syarat itu, sebab tidak menyebutkan seberapa akurat, diukur bagaimana, dan dibandingkan terhadap apa.

Empat unsur kriteria sukses.

Unsur	Pertanyaan	Contoh
Ukuran	apa yang diukur?	recall pada kelas layak
Besaran	berapa nilainya?	minimal 0,70
Acuan	dibandingkan apa?	aturan lama yang berlaku
Kondisi	diukur di mana?	pada data uji terpisah

Kriteria tanpa acuan tidak bermakna. Menetapkan akurasi minimal delapan puluh persen tampak tegas, tetapi tidak berarti apa-apa bila menebak kelas mayoritas saja sudah mencapai delapan puluh lima persen. Setiap kriteria wajib disertai acuan: tebakan mayoritas, aturan yang berlaku saat ini, atau kinerja model sebelumnya.

7.5.0.4 Biaya Kekeliruan Tidak Simetris

Pada hampir semua penerapan nyata, kedua jenis kekeliruan tidak sama mahalannya. Melewatkan rumah tangga yang sesungguhnya layak berbeda akibatnya dari memasukkan rumah tangga yang tidak layak. Perbedaan itu perlu dinyatakan dalam angka agar keputusan ambang dapat dipertanggungjawabkan.

Biaya total atas suatu ambang t :

$$C(t) = n_{SP}(t) \cdot c_{SP} + n_{SN}(t) \cdot c_{SN}$$

dengan n_{SP} dan n_{SN} banyaknya salah positif dan salah negatif, serta c biaya satuannya. Ambang terbaik meminimumkan $C(t)$, dan lazimnya **bukan** 0,5.

7.5.0.5 Menilai Kecukupan Data

Pertanyaan apakah datanya cukup dapat dijawab secara empiris melalui kurva belajar: model dilatih pada beberapa ukuran data yang bertambah, lalu kinerjanya diamati. Bila kurva masih menanjak, menambah data kemungkinan menolong. Bila sudah mendatar, menambah data tidak akan banyak berpengaruh dan usaha sebaiknya dialihkan ke peubah baru atau model lain.

Tips dan Praktik Baik. Kurva belajar sebaiknya dibuat pada tahap perencanaan, bukan setelah proyek berjalan. Ia menjawab dua pertanyaan sekaligus: apakah perlu mengumpulkan data lebih banyak, dan apakah kinerja yang dijanjikan pada proposal masuk akal.

7.5.0.6 Risiko Proyek

Lima risiko yang lazim.

Risiko	Gejala awal	Pencegahan
Data tak tersedia	janji lisan tanpa kepastian	pastikan akses sebelum mulai
Mutu data buruk	banyak nilai hilang	jelajahi contoh kecil lebih dahulu
Sasaran berubah	permintaan bertambah	tuliskan lingkup dan sahkan
Kinerja tak tercapai	kurva belajar mendatar rendah	sepakati rencana cadangan
Hasil tak dipakai	pengguna tak dilibatkan	libatkan sejak tahap pertama

Risiko terakhir paling sering terjadi dan paling menyakitkan, sebab seluruh pekerjaan teknis sudah selesai ketika baru ketahuan bahwa hasilnya tidak dipakai. Pencegahannya bukan teknis melainkan melibatkan pengguna sejak perumusan persoalan.

7.6 Contoh Praktikum dengan R

Contoh 7.1 | Menerjemahkan kebutuhan menjadi tugas teknis

Kode 7.1 • R | menuliskan rumusan bertingkat

```
rumusan <- data.frame(
  tingkat = c("Kebutuhan pengguna", "Pertanyaan analisis", "Tugas teknis",
             "Sasaran", "Ukuran utama", "Acuan pembandingan"),
  isi = c("bantuan sering tidak tepat sasaran",
         "rumah tangga mana yang paling mungkin layak menerima?",
         "klasifikasi biner",
         "penerima_bantuan (ya/tidak)",
         "recall pada kelas ya",
         "aturan ambang pengeluaran yang berlaku"))
print(rumusan, right = FALSE)
```

tingkat	isi
1 Kebutuhan pengguna	bantuan sering tidak tepat sasaran
2 Pertanyaan analisis	rumah tangga mana yang paling mungkin layak menerima?
3 Tugas teknis	klasifikasi biner
4 Sasaran	penerima_bantuan (ya/tidak)
5 Ukuran utama	recall pada kelas ya
6 Acuan pembandingan	aturan ambang pengeluaran yang berlaku

Penjelasan Kode. Rumusan disusun sebagai tabel enam baris yang menurunkan kebutuhan pengguna menjadi tugas teknis. Setiap tingkat dituliskan agar keterkaitannya terlihat dan dapat diperiksa orang lain.

Interpretasi Output

Menuliskan seluruh tingkat, bukan hanya tugas teknisnya, adalah kebiasaan yang menyelamatkan proyek. Ketika di tengah pekerjaan muncul godaan menambah peubah atau mengganti model, tabel ini menjadi rujukan apakah perubahan itu masih melayani kebutuhan awal. Tanpa rujukan tertulis, lingkup pekerjaan cenderung melebar tanpa disadari.

Contoh 7.2 | Menghitung acuan pembandingan

Kode 7.2 • R | dua jenis acuan pembandingan

```

proyek <- read.csv("data/data_proyek.csv", stringsAsFactors = TRUE)
proyek$penerima_bantuan <- factor(proyek$penerima_bantuan,
                                  levels = c("ya", "tidak"))

```

acuan 1: tebakan mayoritas

```

prop <- prop.table(table(proyek$penerima_bantuan))
cat("acuan tebakan mayoritas:", round(max(prop), 4), "\n")

```

```
acuan tebakan mayoritas: 0.64
```

acuan 2: aturan sederhana yang berlaku saat ini

```

ambang_lama <- quantile(proyek$pengeluaran, 0.2)
aturan_lama <- factor(ifelse(proyek$pengeluaran <= ambang_lama, "ya", "tidak"),
                      levels = c("ya", "tidak"))
cm_lama <- table(duga = aturan_lama, benar = proyek$penerima_bantuan)
print(cm_lama)

```

	benar	
duga	ya	tidak
ya	143	97
tidak	289	671

```

akurasi_lama <- sum(diag(cm_lama)) / sum(cm_lama)
recall_lama <- cm_lama[1, 1] / sum(cm_lama[, 1])
cat("\naturan lama - akurasi:", round(akurasi_lama, 4),
    "| recall:", round(recall_lama, 4), "\n")

```

```
aturan lama - akurasi: 0.6783 | recall: 0.331
```

Penjelasan Kode. Dua acuan dihitung. Yang pertama adalah tebakan mayoritas, yaitu batas bawah yang wajib dilampaui model apa pun. Yang kedua meniru aturan sederhana yang lazim dipakai di lapangan, yaitu memilih dua puluh persen rumah tangga berpengeluaran terendah.

Interpretasi Output

Acuan kedua jauh lebih bermakna daripada yang pertama, sebab itulah yang sesungguhnya menjadi pesaing model. Bila model tidak melampaui aturan sederhana yang sudah berjalan, maka menggantinya hanya menambah kerumitan tanpa manfaat. Menetapkan acuan seperti ini pada tahap proposal mencegah kekecewaan di akhir.

Contoh 7.3 | Menyusun kriteria sukses yang terukur

Kode 7.3 • R tabel kriteria sukses

```
kriteria <- data.frame(
  ukuran = c("Recall kelas ya", "Presisi kelas ya", "ROC-AUC", "Waktu latih"),
  sasaran = c(">= 0,70", ">= 0,35", ">= 0,70", "<= 5 menit"),
  acuan = c(sprintf("aturan lama = %.3f", recall_lama),
    "aturan lama", "tebakan acak = 0,50", "tidak ada"),
  diukur_pada = c("data uji", "data uji", "validasi silang", "mesin standar"),
  wajib = c("ya", "tidak", "ya", "tidak"))
print(kriteria, right = FALSE)
```

ukuran	sasaran	acuan	diukur_pada	wajib
1 Recall kelas ya	>= 0,70	aturan lama = 0.331	data uji	ya
2 Presisi kelas ya	>= 0,35	aturan lama	data uji	tidak
3 ROC-AUC	>= 0,70	tebakan acak = 0,50	validasi silang	ya
4 Waktu latih	<= 5 menit	tidak ada	mesin standar	tidak

Penjelasan Kode. Kriteria disusun sebagai tabel dengan empat unsur yang dibahas pada Teori: ukuran, besaran sasaran, acuan pembanding, dan kondisi pengukuran. Kolom terakhir membedakan kriteria yang wajib dipenuhi dari yang sekadar diharapkan.

Interpretasi Output

Pembedaan wajib dan tidak wajib penting karena kriteria jarang seluruhnya tercapai. Menetapkan sejak awal mana yang menentukan keberhasilan mencegah perdebatan di akhir proyek. Perhatikan pula bahwa kriteria mencakup hal nonstatistik seperti waktu latih — model yang sangat akurat tetapi menuntut berjam-jam pelatihan bisa jadi tak terpakai dalam operasi harian.

Contoh 7.4 | Menyusun inventarisasi data

Kode 7.4 • R tabel inventarisasi peubah

```
inventaris <- data.frame(
  peubah = names(proyek),
  tipe = sapply(proyek, function(x) class(x)[1]),
  hilang = colSums(is.na(proyek)),
  unik = sapply(proyek, function(x) length(unique(x))),
  row.names = NULL)
inventaris$peran <- c("penanda", rep("prediktor", 7), "sasaran")
inventaris$layak <- ifelse(inventaris$peran == "penanda", "tidak dipakai",
  ifelse(inventaris$hilang / nrow(proyek) > 0.3,
    "terlalu banyak hilang", "layak"))
print(inventaris, right = FALSE)
```

peubah	tipe	hilang	unik	peran	layak
1 id	integer	0	1200	penanda	tidak dipakai
2 wilayah	factor	0	2	prediktor	layak
3 umur_krt	integer	0	49	prediktor	layak
4 anggota_rt	integer	0	9	prediktor	layak
5 lama_sekolah	numeric	0	157	prediktor	layak
6 pengeluaran	numeric	0	1199	prediktor	layak
7 akses_listrik	numeric	0	435	prediktor	layak
8 akses_air	numeric	0	520	prediktor	layak
9 penerima_bantuan	factor	0	2	sasaran	layak

Penjelasan Kode. Tabel disusun memuat tipe, cacah nilai hilang, dan cacah nilai unik tiap peubah. Peran tiap peubah ditetapkan, lalu kelayakannya dinilai dengan ambang tiga puluh persen nilai hilang.

Interpretasi Output

Tabel semacam ini menjadi lampiran wajib proposal. Ia memaksa penyusun memutuskan peran tiap peubah sejak awal, dan memaksa menyatakan peubah mana yang tidak akan dipakai beserta alasannya. Perhatikan bahwa penanda ditandai tidak dipakai — melupakan hal ini berakibat model menghafal nomor urut dan tampak sangat baik secara keliru.

Contoh 7.5 | Menilai kecukupan data dengan kurva belajar

Kode 7.5 • R kurva belajar menurut ukuran data latih

```
suppressPackageStartupMessages({
  library(recipes); library(rsample); library(parsnip)
  library(workflows); library(yardstick); library(dplyr)})

set.seed(2026)
bagi <- initial_split(proyek, prop = 0.75, strata = penerima_bantuan)
lat <- training(bagi); uji <- testing(bagi)

rec <- recipe(penerima_bantuan ~ ., data = lat) |>
  update_role(id, new_role = "penanda") |>
  step_dummy(all_nominal_predictors()) |>
  step_normalize(all_numeric_predictors())
wf <- workflow() |> add_recipe(rec) |>
  add_model(logistic_reg() |> set_engine("glm"))

ukuran <- c(50, 100, 200, 350, 500, nrow(lat))
kurva <- t(sapply(ukuran, function(m) {
```

```

set.seed(1)
s <- lat[sample(nrow(lat), m), ]
fm <- fit(wf, data = s)
du <- predict(fm, uji, type = "prob") |>
  bind_cols(uji |> select(penerima_bantuan))
c(n = m, auc = yardstick::roc_auc(du, penerima_bantuan, .pred_ya)$estimate)
}))
print(round(kurva, 4))

```

```

      n    auc
[1,]  50 0.7679
[2,] 100 0.8214
[3,] 200 0.8254
[4,] 350 0.8258
[5,] 500 0.8282
[6,] 900 0.8317

```

Penjelasan Kode. Model dilatih pada enam ukuran contoh yang bertambah, diambil dari data latih. Kinerja selalu dinilai pada data uji yang sama agar sebanding. Benih acak ditetapkan agar penarikan contohnya dapat diulang.

Interpretasi Output

Bacalah arah kolom kedua. Bila nilainya masih menanjak pada ukuran terbesar, menambah data kemungkinan menolong sehingga pengumpulan data tambahan layak diusulkan. Bila sudah mendatar — sebagaimana lazim terjadi ketika peubah yang tersedia memang terbatas daya jelasnya — maka menambah data tidak akan menolong, dan usaha sebaiknya dialihkan pada peubah baru atau perumusan persoalan yang berbeda.

Contoh 7.6 | Menggambarkan kurva belajar

Kode 7.6 • R kurva belajar beserta garis acuan

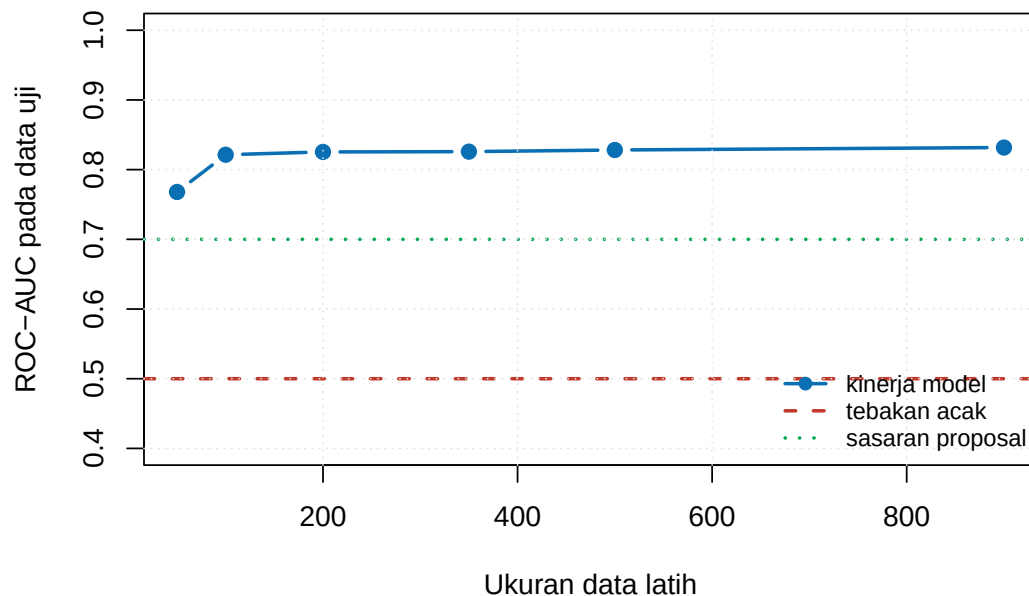
```

plot(kurva[, "n"], kurva[, "auc"], type = "b", pch = 19, lwd = 2,
     col = "#0B6FB4", ylim = c(0.4, 1),
     xlab = "Ukuran data latih", ylab = "ROC-AUC pada data uji",
     main = "Kurva Belajar")
abline(h = 0.5, col = "#C0392B", lty = 2, lwd = 2)
abline(h = 0.7, col = "#00A651", lty = 3, lwd = 2)
grid(col = "grey88")
legend("bottomright", c("kinerja model", "tebakan acak", "sasaran proposal"),
     col = c("#0B6FB4", "#C0392B", "#00A651"),

```

```
lty = c(1, 2, 3), lwd = 2, pch = c(19, NA, NA), bty = "n", cex = .8)
```

Kurva Belajar



Penjelasan Kode. Kurva digambar bersama dua garis acuan: tebakan acak pada nilai setengah dan sasaran yang dijanjikan pada proposal. Batas sumbu tegak ditetapkan agar keduanya selalu terlihat.

Interpretasi Output

Menyertakan garis sasaran pada gambar mengubahnya dari sekadar grafik menjadi alat pengambilan keputusan. Bila kurva mendatar jauh di bawah garis sasaran, maka sasaran itu tidak realistis dan proposal perlu direvisi sebelum disepakati — jauh lebih baik daripada menyadarinya setelah proyek berjalan berminggu-minggu.

Contoh 7.7 | Menyusun matriks biaya kekeliruan

Kode 7.7 • R matriks biaya dua kelas

```
biaya_sp <- 500      # salah positif: bantuan diberikan pada yang tak layak
biaya_sn <- 5000     # salah negatif: yang layak tidak menerima bantuan

matriks_biaya <- matrix(c(0, biaya_sp, biaya_sn, 0), nrow = 2,
  dimnames = list(dugaan = c("ya", "tidak"),
```

```
sebenarnya = c("ya", "tidak"))
print(matriks_biaya)
```

```
sebenarnya
dugaan ya tidak
ya      0  5000
tidak 500    0
```

```
cat("\nrasio biaya salah negatif terhadap salah positif:",
    biaya_sn / biaya_sp, "berbanding 1\n")
```

```
rasio biaya salah negatif terhadap salah positif: 10 berbanding 1
```

Penjelasan Kode. Biaya satuan tiap jenis kekeliruan ditetapkan lalu disusun sebagai matriks berukuran dua kali dua. Sel diagonal bernilai nol karena dugaan yang benar tidak menimbulkan biaya kekeliruan.

Interpretasi Output

Angka biaya tidak perlu tepat secara akuntansi; yang menentukan adalah rasionya. Rasio sepuluh berbanding satu menyatakan bahwa melewatkan satu rumah tangga layak dianggap sepuluh kali lebih merugikan daripada memberi bantuan kepada satu yang tak layak. Rasio ini adalah **keputusan kebijakan** yang harus disepakati pengguna, bukan ditetapkan sendiri oleh analis.

Contoh 7.8 | Memilih ambang berdasarkan biaya

Kode 7.8 • R mencari ambang berbiaya minimum

```
model <- fit(wf, data = lat)
du <- predict(model, uji, type = "prob") |>
  bind_cols(uji |> select(penerima_bantuan))

ambang <- seq(0.05, 0.95, by = 0.05)
biaya <- sapply(ambang, function(t) {
  duga <- ifelse(du$.pred_ya > t, "ya", "tidak")
  sp <- sum(duga == "ya" & du$penerima_bantuan == "tidak")
  sn <- sum(duga == "tidak" & du$penerima_bantuan == "ya")
  sp * biaya_sp + sn * biaya_sn
})

t_terbaik <- ambang[which.min(biaya)]
cat("ambang berbiaya terendah:", t_terbaik, "| biaya:", min(biaya), "\n")
```

```
ambang berbiaya terendah: 0.1 | biaya: 91500
```



```
cat("biaya pada ambang 0,50 :", biaya[ambang == 0.5], "\n")
```

```
biaya pada ambang 0,50 : 235500
```

```
cat("penghematan:",  
    round(100 * (biaya[ambang == 0.5] - min(biaya)) / biaya[ambang == 0.5], 1),  
    "persen\n")
```

```
penghematan: 61.1 persen
```

Penjelasan Kode. Sembilan belas ambang dicoba. Pada tiap ambang, banyaknya salah positif dan salah negatif dihitung lalu dikalikan biaya satuannya. Ambang yang menghasilkan biaya terendah dicari memakai `which.min`.

Interpretasi Output

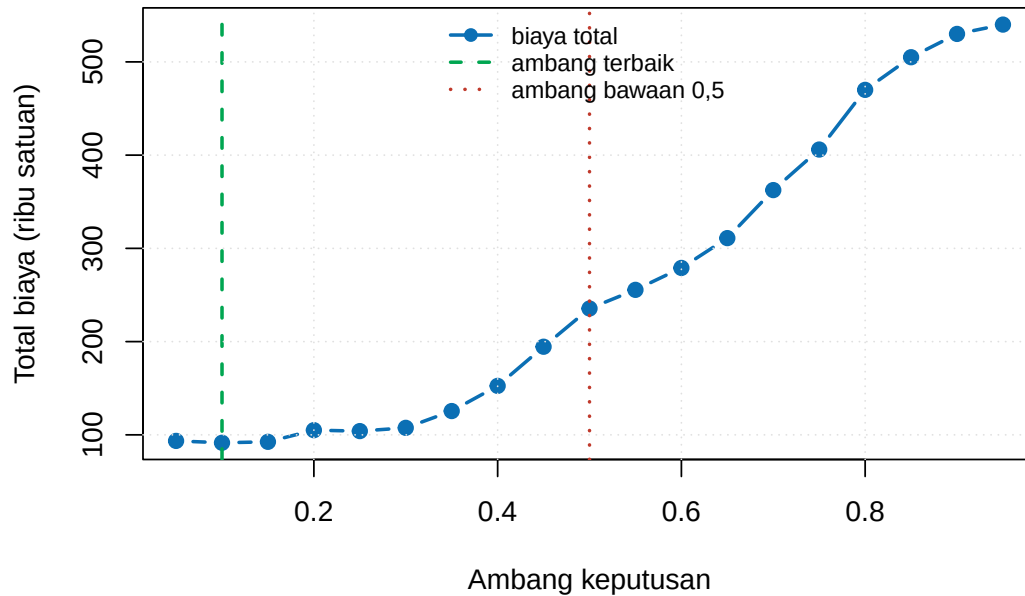
Perhatikan besarnya penghematan yang diperoleh semata dari memilih ambang, tanpa mengganti model sama sekali. Ambang bawaan setengah hampir selalu bukan pilihan terbaik ketika biaya kekeliruan tidak simetris. Inilah salah satu keputusan paling menguntungkan yang dapat diambil pada tahap perencanaan, dan sekaligus salah satu yang paling sering terlewat.

Contoh 7.9 | Menggambarkan biaya menurut ambang

Kode 7.9 • R grafik biaya dan dua garis ambang

```
plot(ambang, biaya / 1000, type = "b", pch = 19, lwd = 2, col = "#0B6FB4",  
     xlab = "Ambang keputusan", ylab = "Total biaya (ribu satuan)",  
     main = "Biaya Kekeliruan menurut Ambang")  
abline(v = t_terbaik, col = "#00A651", lty = 2, lwd = 2)  
abline(v = 0.5, col = "#C0392B", lty = 3, lwd = 2)  
grid(col = "grey88")  
legend("top", c("biaya total", "ambang terbaik", "ambang bawaan 0,5"),  
      col = c("#0B6FB4", "#00A651", "#C0392B"),  
      lty = c(1, 2, 3), lwd = 2, pch = c(19, NA, NA), bty = "n", cex = .8)
```

Biaya Kekeliruan menurut Ambang



Penjelasan Kode. Total biaya digambar terhadap ambang, dibagi seribu agar sumbunya mudah dibaca. Dua garis tegak menandai ambang terbaik dan ambang bawaan sebagai pembanding.

Interpretasi Output

Jarak mendatar antara kedua garis tegak menyatakan seberapa jauh ambang bawaan meleset, sedangkan jarak tegak pada kurva menyatakan kerugian akibat memakainya. Gambar semacam ini sangat efektif dipakai menjelaskan kepada pengambil keputusan mengapa ambang perlu ditetapkan berdasarkan biaya, bukan dibiarkan pada nilai bawaan.

Contoh 7.10 | Menaksir kebutuhan waktu dan menyusun jadwal

Kode 7.10 • R tabel jadwal proyek

```
jadwal <- data.frame(
  tahap = c("1 Business Understanding", "2 Data Understanding",
            "3 Data Preparation", "4 Modeling",
            "5 Evaluation", "6 Deployment"),
  mulai_minggu = c(1, 2, 3, 7, 10, 12),
  lama_minggu = c(2, 2, 4, 4, 2, 2),
```

```
porsi_persen = c(10, 10, 35, 25, 12, 8)
jadwal$selesai_minggu <- jadwal$mulai_minggu + jadwal$lama_minggu - 1
print(jadwal, right = FALSE)
```

	tahap	mulai_minggu	lama_minggu	porosi_persen	selesai_minggu
1	1 Business Understanding	1	2	10	2
2	2 Data Understanding	2	2	10	3
3	3 Data Preparation	3	4	35	6
4	4 Modeling	7	4	25	10
5	5 Evaluation	10	2	12	11
6	6 Deployment	12	2	8	13

```
cat("\ntotal durasi:", max(jadwal$selesai_minggu), "minggu\n")
```

```
total durasi: 13 minggu
```

```
cat("porosi terbesar:", jadwal$tahap[which.max(jadwal$porosi_persen)],
    paste0("(", max(jadwal$porosi_persen), " persen)"), "\n")
```

```
porosi terbesar: 3 Data Preparation (35 persen)
```

Penjelasan Kode. Jadwal disusun sebagai tabel yang memuat minggu mulai, lama, dan porsi usaha tiap tahap. Minggu selesai dihitung dari keduanya, dengan pengurangan satu agar minggu mulai ikut terhitung.

Interpretasi Output

Perhatikan porsi terbesar yang jatuh pada penyiapan data, sejalan dengan pengalaman lapangan yang berulang kali dilaporkan. Jadwal yang memberi porsi besar pada pemodelan dan kecil pada penyiapan hampir selalu meleset. Menyusun jadwal dengan porsi realistis pada tahap proposal mencegah janji yang tak dapat ditepati.

Contoh 7.11 | Menggambarkan jadwal sebagai diagram batang

Kode 7.11 • R diagram jadwal dengan rect

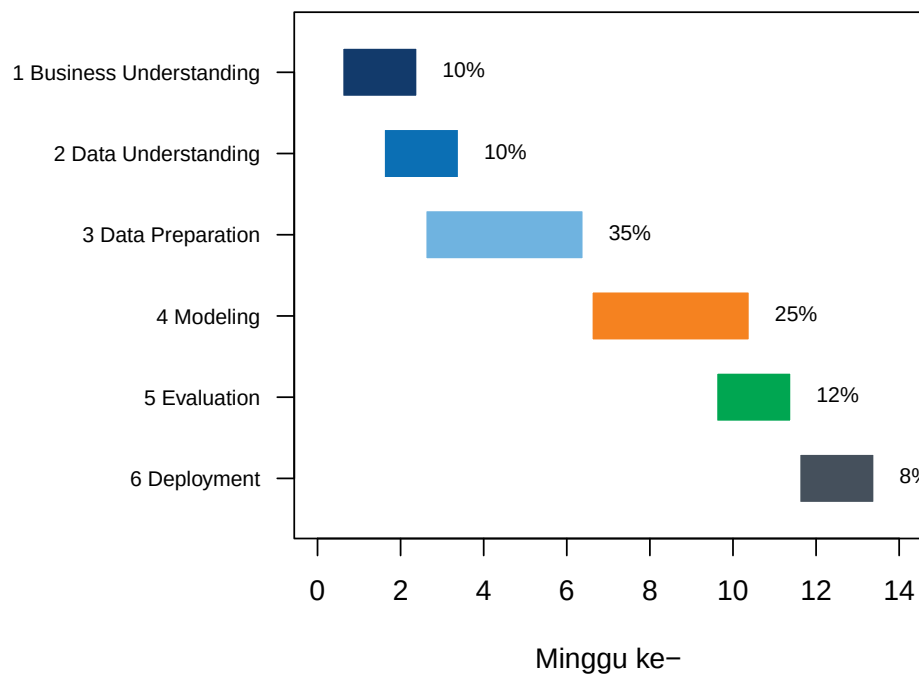
```
par(mar = c(4, 12, 3, 2))
plot(NA, xlim = c(0, max(jadwal$selesai_minggu) + 1),
     ylim = c(0.5, nrow(jadwal) + 0.5), yaxt = "n",
     xlab = "Minggu ke-", ylab = "",
     main = "Jadwal Proyek menurut Tahap CRISP-DM")
warna <- c("#123A6B", "#0B6FB4", "#6FB3E0", "#F58220", "#00A651", "#45505C")
for (i in seq_len(nrow(jadwal))) {
  y <- nrow(jadwal) - i + 1
```

```

rect(jadwal$mulai_minggu[i] - 0.4, y - 0.3,
     jadwal$selesai_minggu[i] + 0.4, y + 0.3,
     col = warna[i], border = "white")
text(jadwal$selesai_minggu[i] + 0.6, y,
     paste0(jadwal$porsi_persen[i], "%"), cex = .75, pos = 4)
}
axis(2, at = nrow(jadwal):1, labels = jadwal$tahap, las = 1, cex.axis = .75)

```

Jadwal Proyek menurut Tahap CRISP-DM



```

par(mar = c(5, 4, 4, 2) + 0.1)

```

Penjelasan Kode. Kanvas kosong dibuat lebih dahulu, lalu tiap tahap digambar sebagai persegi panjang memakai `rect` dengan koordinat dari tabel `jadwal`. Sumbu tegak diberi label nama tahap secara manual, dan margin kiri diperlebar agar labelnya muat.

Interpretasi Output

Diagram semacam ini lazim disebut diagram Gantt dan menjadi lampiran baku pada proposal proyek. Nilainya bukan pada keindahan melainkan pada kemampuannya memperlihatkan tumpang tindih antar-tahap — dan pada CRISP-DM tumpang tindih itu wajar, sebab tahapnya memang tidak berjalan sekali lurus.

Contoh 7.12 | Menyusun daftar risiko dan pencegahannya**Kode 7.12 • R** tabel risiko berprioritas

```

risiko <- data.frame(
  risiko = c("Data tidak tersedia tepat waktu",
            "Mutu data lebih buruk dari dugaan",
            "Kinerja tidak mencapai sasaran",
            "Lingkup pekerjaan melebar",
            "Hasil tidak dipakai pengguna"),
  peluang = c("sedang", "tinggi", "sedang", "tinggi", "sedang"),
  dampak = c("tinggi", "tinggi", "sedang", "sedang", "tinggi"),
  pencegahan = c("pastikan akses tertulis sebelum mulai",
                "jelajahi contoh kecil pada minggu pertama",
                "sepakati rencana cadangan pada proposal",
                "tuliskan lingkup dan sahkan bersama",
                "libatkan pengguna sejak perumusan"))
risiko$prioritas <- ifelse(risiko$peluang == "tinggi" & risiko$dampak == "tinggi",
                          "utama",
                          ifelse(risiko$peluang == "rendah" & risiko$dampak == "rendah",
                                "rendah", "sedang"))
print(risiko[, c("risiko", "peluang", "dampak", "prioritas")], right = FALSE)

```

risiko	peluang	dampak	prioritas
1 Data tidak tersedia tepat waktu	sedang	tinggi	sedang
2 Mutu data lebih buruk dari dugaan	tinggi	tinggi	utama
3 Kinerja tidak mencapai sasaran	sedang	sedang	sedang
4 Lingkup pekerjaan melebar	tinggi	sedang	sedang
5 Hasil tidak dipakai pengguna	sedang	tinggi	sedang

Penjelasan Kode. Lima risiko disusun beserta taksiran peluang dan dampaknya. Prioritas ditetapkan dengan ifelse bersarang: risiko berpeluang dan berdampak tinggi menjadi utama, sedangkan yang keduanya rendah menjadi prioritas rendah.

Interpretasi Output

Yang membuat daftar risiko berguna bukan penyusunannya melainkan kolom pencegahannya. Risiko tanpa langkah pencegahan hanyalah daftar kekhawatiran. Perhatikan bahwa sebagian besar pencegahan bersifat nonteknis — kesepakatan tertulis, pelibatan pengguna, penjelajahan awal — yang menegaskan bahwa keberhasilan proyek data mining tidak ditentukan kemampuan teknis semata.

Contoh 7.13 | Memeriksa kelayakan etika

Kode 7.13 • R daftar periksa etika

```
etika <- data.frame(
  pertanyaan = c("Apakah data diperoleh dengan sah?",
                 "Apakah memuat data pribadi?",
                 "Dapatkah individu dikenali setelah digabung?",
                 "Adakah kelompok yang berpotensi dirugikan?",
                 "Dapatkah keputusan model dijelaskan?",
                 "Adakah mekanisme sanggahan bagi yang terdampak?"),
  jawaban = c("ya", "tidak", "perlu diperiksa", "perlu diperiksa", "ya", "belum ada"),
  tindakan = c("-", "-", "periksa keunikan gabungan penanda",
               "bandingkan kinerja antar-kelompok", "-",
               "usulkan prosedur peninjauan manual"))
print(etika, right = FALSE)
```

pertanyaan	jawaban
1 Apakah data diperoleh dengan sah?	ya
2 Apakah memuat data pribadi?	tidak
3 Dapatkah individu dikenali setelah digabung?	perlu diperiksa
4 Adakah kelompok yang berpotensi dirugikan?	perlu diperiksa
5 Dapatkah keputusan model dijelaskan?	ya
6 Adakah mekanisme sanggahan bagi yang terdampak?	belum ada
tindakan	
1 -	
2 -	
3 periksa keunikan gabungan penanda	
4 bandingkan kinerja antar-kelompok	
5 -	
6 usulkan prosedur peninjauan manual	

```
cat("\nbutir yang menuntut tindakan:", sum(etika$tindakan != "-"), "dari",
    nrow(etika), "\n")
```

```
butir yang menuntut tindakan: 3 dari 6
```

Penjelasan Kode. Enam pertanyaan etika disusun sebagai daftar periksa, masing-masing dengan jawaban dan tindakan yang diperlukan. Butir yang menuntut tindakan dihitung.

Interpretasi Output

Butir kelima dan keenam kerap terlupakan padahal paling menentukan pada penerapan yang menyangkut hajat orang banyak. Model yang keputusannya tidak dapat dijelaskan sukar dipertanggungjawabkan ketika dipersoalkan, dan ketiadaan mekanisme sanggahan berarti kekeliruan model tidak dapat diperbaiki dari sisi yang terdampak. Keduanya perlu direncanakan sejak proposal, bukan ditambahkan setelah model jadi.

Contoh 7.14 | Memeriksa keadilan antar-kelompok**Kode 7.14 • R** membandingkan kinerja antar-kelompok

```

du$wilayah <- uji$wilayah
du$duga <- factor(ifelse(du$.pred_ya > t_terbaik, "ya", "tidak"),
                  levels = c("ya", "tidak"))

per_wilayah <- do.call(rbind, lapply(levels(du$wilayah), function(w) {
  s <- du[du$wilayah == w, ]
  cm <- table(duga = s$duga, benar = s$penerima_bantuan)
  data.frame(wilayah = w, n = nrow(s),
             ditandai = round(mean(s$duga == "ya"), 4),
             recall = round(cm[1, 1] / max(sum(cm[, 1]), 1), 4))
}))
print(per_wilayah, right = FALSE)

```

```

wilayah  n  ditandai recall
1 Perdesaan 161 1.0000  1.0000
2 Perkotaan 139 0.5396  0.7222

```

```

cat("\nselisih recall antar-wilayah:",
    round(abs(diff(per_wilayah$recall)), 4), "\n")

```

```

selisih recall antar-wilayah: 0.2778

```

Penjelasan Kode. Dugaan dipecah menurut wilayah, lalu proporsi yang ditandai dan recall dihitung pada tiap kelompok. Fungsi max dengan nilai satu mencegah pembagian dengan nol bila suatu kelompok tidak memuat kelas positif.

Interpretasi Output

Selisih recall antar-kelompok adalah ukuran keadilan yang paling langsung. Bila model jauh lebih baik menangkap kasus di satu wilayah daripada wilayah lain, maka penerapannya akan memperlebar ketimpangan yang justru hendak dikurangi. Pemeriksaan ini wajib direncanakan sejak proposal dan dilaporkan apa adanya, sebab kekeliruan semacam ini tidak akan terlihat pada ukuran kinerja keseluruhan.

7.7 Latihan Mandiri

Kerjakan bertahap. Gunakan data/data_proyek.csv.

7.7.0.0.1 Tingkat Dasar (melengkapi kode)

Latihan 7.1 | Menghitung acuan

```
proyek <- read.csv("data/data_proyek.csv", stringsAsFactors = TRUE)
prop <- ____ (table(proyek$penerima_bantuan)) # proporsi kelas
cat("acuan mayoritas:", round(____(prop), 4), "
")
ambang_lama <- ____ (proyek$pengeluaran, 0.2) # kuantil ke-20
```

Latihan 7.2 | Matriks biaya

```
biaya_sp <- 500; biaya_sn <- 5000
m <- ____ (c(0, biaya_sp, biaya_sn, 0), nrow = ____ ) # matriks 2x2
cat("rasio:", biaya_sn / biaya_sp, "
")
```

Latihan 7.3 | Inventarisasi peubah

```
inv <- data.frame(
  peubah = ____ (proyek),
  tipe = sapply(proyek, function(x) ____ (x)[1]),
  hilang = ____ (is.na(proyek))
```

7.7.0.0.2 Tingkat Menengah (menulis sendiri)

Latihan 7.4 | Kurva belajar

Susun kurva belajar dengan enam ukuran data latih. Sajikan tabel dan grafik beserta garis acuan tebakan acak dan sasaran 0,70. Berdasarkan bentuk kurvanya, tentukan apakah Anda akan mengusulkan pengumpulan data tambahan — dan jelaskan alasannya.

Latihan 7.5 | Membetulkan kode yang salah

Kode berikut memuat empat kesalahan (acuan dihitung dari data uji, kurva belajar dinilai pada data latih, ambang ditetapkan tanpa memakai biaya, dan penanda ikut dimodelkan). Temukan dan perbaiki, lalu jelaskan

akibat masing-masing.

```
# (a) acuan dihitung dari data UJI
acuan <- max(prop.table(table(uji$penerima_bantuan)))

# (b) kurva belajar dinilai pada data LATIH
kurva <- sapply(c(50, 100, 200), function(m) {
  s <- lat[1:m, ]
  fm <- fit(wf, data = s)
  d <- predict(fm, s, type = "prob") |> bind_cols(s |> select(penerima_bantuan))
  yardstick::roc_auc(d, penerima_bantuan, .pred_ya)$estimate
})

# (c) ambang ditetapkan pada nilai bawaan tanpa mempertimbangkan biaya
duga <- ifelse(du$.pred_ya > 0.5, "ya", "tidak")

# (d) kolom penanda ikut dimodelkan
rec <- recipe(penerima_bantuan ~ ., data = lat) |>
  step_normalize(all_numeric_predictors())
```

Keluaran yang benar. Setelah diperbaiki, kode yang dipakai adalah sebagai berikut.

```
# (a) acuan dihitung dari data LATIH atau seluruh data, bukan data uji
acuan <- max(prop.table(table(lat$penerima_bantuan)))

# (b) kurva belajar dinilai pada data UJI yang sama bagi seluruh ukuran
kurva <- sapply(c(50, 100, 200), function(m) {
  set.seed(1); s <- lat[sample(nrow(lat), m), ]
  fm <- fit(wf, data = s)
  d <- predict(fm, uji, type = "prob") |>
    bind_cols(uji |> select(penerima_bantuan))
  yardstick::roc_auc(d, penerima_bantuan, .pred_ya)$estimate
})

# (c) ambang dipilih berdasarkan biaya kekeliruan
ambang <- seq(0.05, 0.95, by = 0.05)
biaya <- sapply(ambang, function(t) {
  dg <- ifelse(du$.pred_ya > t, "ya", "tidak")
  sum(dg == "ya" & du$penerima_bantuan == "tidak") * biaya_sp +
  sum(dg == "tidak" & du$penerima_bantuan == "ya") * biaya_sn
})
t_terbaik <- ambang[which.min(biaya)]
```

```
# (d) penanda diubah perannya
rec <- recipe(penerima_bantuan ~ ., data = lat) |>
  update_role(id, new_role = "penanda") |>
  step_dummy(all_nominal_predictors()) |>
  step_normalize(all_numeric_predictors())
```

Pemeriksaan yang wajib berhasil: `t_terbaik != 0.5` bernilai **TRUE** pada data ini, dan `"id" %in% names(bake(prepare(rec, training = lat), new_data = NULL))` bernilai **TRUE** sedangkan `id` tidak berperan prediktor.

7.7.0.0.3 Tingkat Lanjut (perumusan)

Latihan 7.6 | Merumuskan persoalan sendiri

Pilih satu kebutuhan nyata yang dapat dijawab dengan data serupa. Susun **rumusan bertingkat** (kebutuhan, pertanyaan, tugas teknis), **kriteria sukses** berisi empat unsur, dan **matriks biaya** beserta alasan rasionya. Jelaskan pula siapa pengguna hasilnya dan bagaimana hasil itu akan dipakai.

7.8 Penugasan

Sebagai calon analis pada instansi statistik, susun **proposal proyek data mining** yang akan Anda kerjakan pada Modul 13 dan 14. Proposal harus dapat dinilai kelayakannya oleh pembaca yang tidak terlibat penyusunannya.

Konteks. Sebuah unit meminta Anda mengusulkan satu proyek penambahan data. Pimpinan akan menilai proposal berdasarkan kejelasan persoalan, keterukuran kriteria sukses, kelayakan teknis, dan pertimbangan etikanya.

Tugas 7.1 | Business Understanding

1. Susun **rumusan bertingkat**: kebutuhan pengguna, pertanyaan analisis, dan tugas teknis. Sebutkan siapa penggunanya dan bagaimana hasilnya akan dipakai.
2. Hitung **dua acuan pembeding**: tebakan mayoritas dan satu aturan sederhana yang meniru praktik berjalan. Laporkan kinerja aturan sederhana itu.
3. Susun **tabel kriteria sukses** memuat keempat unsur (ukuran, besaran, acuan, kondisi) serta penandaan wajib atau tidak.

Tugas 7.2 | Penilaian Kelayakan

4. Susun **inventarisasi peubah** memuat tipe, nilai hilang, peran, dan kelayakan. Jelaskan peubah mana yang tidak dipakai beserta alasannya.
5. Buat **kurva belajar** dan tafsirkan. Tentukan apakah Anda mengusulkan pengumpulan data tambahan, dan bila sasaran tampak tak tercapai, usulkan revisi sasarannya.
6. Susun **matriks biaya kekeliruan**, cari ambang berbiaya terendah, dan laporkan penghematannya dibandingkan ambang bawaan. Jelaskan bahwa rasio biaya merupakan keputusan kebijakan.

\tugas{Perencanaan

7. Susun **jadwal proyek** enam tahap CRISP-DM beserta porsi usaha tiap tahap. Sajikan sebagai tabel dan diagram.
8. Susun **daftar risiko** minimal lima butir beserta peluang, dampak, prioritas, dan langkah pencegahannya.
9. Isi **daftar periksa etika** enam butir. Untuk butir yang menuntut tindakan, jelaskan tindakan yang akan Anda ambil.
10. Lakukan **pemeriksaan keadilan** antar-kelompok pada minimal satu peubah pengelompokan. Laporkan

selisih kinerjanya dan bahas akibatnya bila model diterapkan.

11. Susun proposal sebagai laporan R Markdown yang memuat seluruh butir di atas. Tutup dengan **ringkasan eksekutif (maksimal 300 kata)** yang dapat dibaca pimpinan tanpa latar belakang teknis, memuat: persoalan, manfaat yang diharapkan, kelayakan, risiko utama, dan penegasan bahwa data merupakan simulasi.

7.9 Tambahan: Menyusun Piagam Proyek

Bagian ini menyajikan templat piagam proyek, yaitu ringkasan satu halaman yang menjadi rujukan seluruh pihak selama proyek berjalan. Piagam berbeda dari proposal: proposal menjelaskan dan meyakinkan, sedangkan piagam meringkas kesepakatan agar mudah dirujuk.

Contoh 7.15 | Menyusun piagam proyek

Kode 7.15 • R tabel piagam proyek

```
piagam <- data.frame(
  butir = c("Nama proyek", "Pengguna hasil", "Persoalan",
            "Tugas teknis", "Sasaran utama", "Acuan pembanding",
            "Data", "Lingkup termasuk", "Lingkup tidak termasuk",
            "Durasi", "Risiko utama", "Kriteria selesai"),
  isi = c("Penyasaran Bantuan Berbasis Data",
          "Unit penyaluran bantuan",
          "Bantuan sering tidak tepat sasaran",
          "Klasifikasi biner",
          "Recall kelas layak minimal 0,70",
          "Aturan ambang pengeluaran yang berlaku",
          "data_proyek.csv (simulasi, 900 amatan, 9 peubah)",
          "pemodelan, penilaian, rekomendasi ambang",
          "penerapan sistem, pelatihan petugas",
          "13 minggu",
          "mutu data lebih buruk dari dugaan",
          "laporan disetujui dan ambang disepakati"))
print(piagam, right = FALSE)
```

butir	isi
1 Nama proyek	Penyasaran Bantuan Berbasis Data
2 Pengguna hasil	Unit penyaluran bantuan
3 Persoalan	Bantuan sering tidak tepat sasaran
4 Tugas teknis	Klasifikasi biner
5 Sasaran utama	Recall kelas layak minimal 0,70

6	Acuan pambanding	Aturan ambang pengeluaran yang berlaku
7	Data	data_proyek.csv (simulasi, 900 amatan, 9 peubah)
8	Lingkup termasuk	pemodelan, penilaian, rekomendasi ambang
9	Lingkup tidak termasuk	penerapan sistem, pelatihan petugas
10	Durasi	13 minggu
11	Risiko utama	mutu data lebih buruk dari dugaan
12	Kriteria selesai	laporan disetujui dan ambang disepakati

Penjelasan Kode. Piagam disusun sebagai tabel dua kolom berisi dua belas butir yang mencakup seluruh kesepakatan pokok, dari nama proyek sampai kriteria selesai.

Interpretasi Output

Dua butir yang paling sering diabaikan adalah lingkup yang **tidak** termasuk dan kriteria selesai. Butir pertama mencegah pelebaran pekerjaan — tanpa pernyataan tegas bahwa penerapan sistem berada di luar lingkup, permintaan itu hampir pasti muncul di tengah jalan. Butir kedua menetapkan kapan proyek boleh dinyatakan selesai, sehingga tidak berlarut tanpa ujung.

Contoh 7.16 | Menaksir kebutuhan sumber daya

Kode 7.16 • R tabel taksiran sumber daya

```
sumber_daya <- data.frame(
  tahap = c("Business", "Data Understanding", "Data Preparation",
            "Modeling", "Evaluation", "Deployment"),
  orang_minggu = c(1.0, 1.5, 5.0, 3.5, 1.5, 1.0),
  keterampilan = c("komunikasi, domain", "eksplorasi data",
                  "pemrograman, domain", "pemodelan",
                  "statistika, domain", "pelaporan"))
sumber_daya$porsi <- round(100 * sumber_daya$orang_minggu /
                          sum(sumber_daya$orang_minggu), 1)
print(sumber_daya, right = FALSE)
```

	tahap	orang_minggu	keterampilan	porsi
1	Business	1.0	komunikasi, domain	7.4
2	Data Understanding	1.5	eksplorasi data	11.1
3	Data Preparation	5.0	pemrograman, domain	37.0
4	Modeling	3.5	pemodelan	25.9
5	Evaluation	1.5	statistika, domain	11.1
6	Deployment	1.0	pelaporan	7.4

```
cat("\ntotal:", sum(sumber_daya$orang_minggu), "orang-minggu\n")
```

total: 13.5 orang-minggu

Penjelasan Kode. Kebutuhan tenaga ditaksir dalam satuan orang-minggu bagi tiap tahap, disertai keterampilan yang diperlukan. Porsinya dihitung sebagai persentase terhadap total.

Interpretasi Output

Kolom keterampilan sama pentingnya dengan kolom beban kerja. Perhatikan bahwa keterampilan pemodelan hanya diperlukan pada satu tahap, sedangkan pemahaman bidang diperlukan pada empat tahap. Taksiran semacam ini membantu memutuskan susunan tim, dan sekaligus mengoreksi anggapan bahwa proyek data mining terutama menuntut keahlian pemodelan.

Tips dan Praktik Baik. Simpan piagam proyek sebagai berkas terpisah yang dapat dirujuk kapan saja, dan tinjau ulang pada setiap peralihan tahap. Bila ada butir yang sudah tidak sesuai, ubahlah secara tertulis dan sahkan bersama — perubahan yang tidak dicatat adalah sumber perselisihan yang paling lazim pada akhir proyek.

Contoh 7.17 | Menyusun kerangka laporan akhir

Kode 7.17 • R tabel kerangka laporan

```
kerangka <- data.frame(
  bagian = c("1 Ringkasan Eksekutif", "2 Latar dan Persoalan",
            "3 Data dan Penyiapan", "4 Metode",
            "5 Hasil", "6 Pembahasan", "7 Keterbatasan",
            "8 Rekomendasi", "Lampiran"),
  isi = c("temuan pokok dan rekomendasi dalam 300 kata",
        "kebutuhan pengguna, pertanyaan, kriteria sukses",
        "sumber, inventarisasi, penanganan cacat",
        "model, resep, skema validasi, penalaan",
        "kinerja terhadap acuan, ambang terpilih",
        "makna hasil bagi pengguna, keadilan antar-kelompok",
        "cakupan, keterwakilan, asumsi yang dipakai",
        "langkah yang diusulkan beserta prasyaratnya",
        "kode, tabel penuh, gambar tambahan"),
  halaman = c(1, 2, 3, 3, 4, 3, 1, 2, NA))
print(kerangka, right = FALSE)
```

bagian	isi	halaman
1 1 Ringkasan Eksekutif	temuan pokok dan rekomendasi dalam 300 kata	1
2 2 Latar dan Persoalan	kebutuhan pengguna, pertanyaan, kriteria sukses	2

3	3 Data dan Penyiapan	sumber, inventarisasi, penanganan cacat	3
4	4 Metode	model, resep, skema validasi, penalaan	3
5	5 Hasil	kinerja terhadap acuan, ambang terpilih	4
6	6 Pembahasan	makna hasil bagi pengguna, keadilan antar-kelompok	3
7	7 Keterbatasan	cakupan, keterwakilan, asumsi yang dipakai	1
8	8 Rekomendasi	langkah yang diusulkan beserta prasyaratnya	2
9	9 Lampiran	kode, tabel penuh, gambar tambahan	NA

```
cat("\nperkiraan tebal laporan:", sum(kerangka$halaman, na.rm = TRUE),
    "halaman di luar lampiran\n")
```

```
perkiraan tebal laporan: 19 halaman di luar lampiran
```

Penjelasan Kode. Kerangka laporan disusun sebagai tabel berisi bagian, isi ringkasnya, dan perkiraan tebal tiap bagian. Nilai NA pada lampiran menyatakan tebalnya tidak dibatasi, dan diabaikan saat penjumlahan memakai `na.rm`.

Interpretasi Output

Perhatikan bahwa ringkasan eksekutif ditempatkan paling depan meski disusun paling akhir. Pembaca yang paling menentukan — pengambil keputusan — kerap hanya membaca bagian itu, sehingga ia harus dapat berdiri sendiri tanpa istilah teknis. Perhatikan pula adanya bagian keterbatasan sebagai bagian tersendiri: menyembunyikan keterbatasan mungkin membuat laporan tampak lebih meyakinkan, tetapi merusak kepercayaan begitu keterbatasan itu ditemukan pihak lain.

Contoh 7.18 | Memeriksa kelengkapan proposal

Kode 7.18 • R fungsi pemeriksa kelengkapan

```
periksa_proposal <- function(butir) {
  wajib <- c("rumusan bertingkat", "kriteria sukses", "acuan pembandingan",
            "inventarisasi data", "kurva belajar", "matriks biaya",
            "jadwal", "daftar risiko", "daftar pemeriksaan etika",
            "pemeriksaan keadilan")
  ada <- wajib %in% butir
  hasil <- data.frame(butir = wajib,
                     status = ifelse(ada, "ada", "BELUM ADA"))
  print(hasil, right = FALSE)
  cat("\nkelengkapan:", sum(ada), "dari", length(wajib),
      sprintf("(%.0f persen)\n", 100 * mean(ada)))
  invisible(all(ada))
}
```

contoh proposal yang belum lengkap

```
punya <- c("rumusan bertingkat", "kriteria sukses", "acuan pembandingan",
           "inventarisasi data", "kurva belajar", "matriks biaya", "jadwal")
periksa_proposal(punya)
```

	butir	status
1	rumusan bertingkat	ada
2	kriteria sukses	ada
3	acuan pembandingan	ada
4	inventarisasi data	ada
5	kurva belajar	ada
6	matriks biaya	ada
7	jadwal	ada
8	daftar risiko	BELUM ADA
9	daftar pemeriksaan etika	BELUM ADA
10	pemeriksaan keadilan	BELUM ADA

kelengkapan: 7 dari 10 (70 persen)

Penjelasan Kode. Fungsi menerima daftar butir yang sudah disusun lalu membandingkannya terhadap sepuluh butir wajib. Operator keanggotaan menghasilkan vektor logika yang dipakai membentuk kolom status, dan hasilnya dikembalikan secara tersembunyi berupa satu nilai logika.

Interpretasi Output

Pemeriksaan otomatis semacam ini mencegah butir terlewat karena kelelahan menjelang tenggat. Perhatikan bahwa tiga butir terakhir — risiko, etika, dan keadilan — justru yang paling sering terlewat, padahal ketiganya menyangkut pertanggungjawaban. Jalankan fungsi ini sebelum proposal dikumpulkan, bukan sesudah dikembalikan.

Proposal yang Anda susun pada Penugasan akan menjadi dasar pekerjaan pada Modul 13 dan 14. Karena itu susunlah dengan sungguh-sungguh: sasaran yang terlalu tinggi akan menyulitkan diri sendiri, sedangkan sasaran yang terlalu rendah membuat proyeknya tidak menarik untuk dikerjakan.

7.10 Ringkasan Modul

Daftar pemeriksaan penguasaan:

- ☐ Saya dapat menerjemahkan kebutuhan pengguna menjadi tugas teknis.
- ☐ Saya menetapkan kriteria sukses dengan keempat unsurnya.

- ☐ Saya selalu menyertakan acuan pembandingan, bukan hanya angka sasaran.
- ☐ Saya dapat menilai kecukupan data melalui kurva belajar.
- ☐ Saya menyusun matriks biaya dan memakainya memilih ambang.
- ☐ Saya memahami bahwa rasio biaya adalah keputusan kebijakan.
- ☐ Saya menyusun jadwal dengan porsi realistis pada penyiapan data.
- ☐ Saya menyusun daftar risiko beserta langkah pencegahannya.
- ☐ Saya memeriksa kelayakan etika sebelum proyek dimulai.
- ☐ Saya memeriksa keadilan kinerja antar-kelompok.

Kartu Referensi Cepat.

Keperluan	Perintah
nAcuan mayoritas	<code>max(prop.table(table(y)))</code>
nAturan sederhana	<code>quantile(x, 0.2) lalu ifelse()</code>
nMatriks kekeliruan	<code>table(duga, benar)</code>
nBiaya total	<code>sum(cm * matriks_biaya)</code>
nAmbang terbaik	<code>ambang[which.min(biaya)]</code>
nKurva belajar	latih pada beberapa ukuran, nilai pada uji sama
nPeran penanda	<code>update_role(id, new_role = "penanda")</code>
nDiagram jadwal	<code>plot(NA) lalu rect()</code>
nKeadilan kelompok	bandingkan recall antar-teraf

Modul ini menutup paruh pertama mata kuliah dengan mengembalikan perhatian pada pertanyaan yang mendahului seluruh keterampilan teknis. Prinsip yang perlu dibawa: rumuskan persoalan sebelum memilih algoritma, sertakan acuan pada setiap kriteria, nyatakan biaya kekeliruan dalam angka, periksa kecukupan data sebelum berjanji, dan libatkan pengguna sejak awal. Modul berikutnya memulai paruh kedua dengan membangun model klasifikasi pertama.

7.11 Referensi

Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. *PAKDD 2000*, 29-39.

Provost, F., & Fawcett, T. (2013). *Data Science for Business*. O'Reilly Media.

Kuhn, M., & Johnson, K. (2013). *Applied Predictive Modeling*. Springer.

Barocas, S., Hardt, M., & Narayanan, A. (2023). *Fairness and Machine Learning*. MIT Press.

Elkan, C. (2001). The foundations of cost-sensitive learning. *IJCAI 2001*, 973-978.

MODUL

8

KLASIFIKASI DASAR

Pertemuan Minggu ke-8

Sub-CPMK 8: Mahasiswa mampu membangun, menala, dan menafsirkan model klasifikasi dasar — pohon keputusan, Naive Bayes, dan regresi logistik terregularisasi — serta memilih di antaranya berdasarkan sifat data dan kebutuhan penjelasan.

Penyusun: Achmad Syahrul Choir



Daftar Isi

8.1	Deskripsi Modul	237
8.2	Tujuan Praktikum	237
8.3	Alat dan Bahan	237
8.4	Instruksi Pengerjaan	238
8.5	Teori Singkat	238
8.5.0.1	Kedudukan pada CRISP-DM	238
8.5.0.2	Pohon Keputusan	238
8.5.0.3	Pemangkasan dan Parameter Kerumitan	239
8.5.0.4	Naive Bayes	239
8.5.0.5	Regresi Logistik dan Regularisasi	240
8.5.0.6	Memilih di Antara Ketiganya	240
8.6	Contoh Praktikum dengan R	241
8.7	Latihan Mandiri	257
8.8	Penugasan	260
8.9	Tambahan: Menafsirkan Model kepada Pengguna	261
8.10	Ringkasan Modul	267
8.11	Referensi	268

8.1 Deskripsi Modul

Modul 6 membangun kerangka penilaian dan Modul 7 merumuskan persoalan. Modul ini akhirnya membangun modelnya. Tiga keluarga model dipilih sebagai pembuka karena ketiganya mewakili cara berpikir yang berbeda dalam menduga kelas.

Pohon keputusan membelah ruang peubah menjadi kotak-kotak; Naive Bayes menghitung peluang berdasarkan anggapan kebebasan; regresi logistik menduga peluang melalui kombinasi linier peubah. Ketiganya mudah dijelaskan kepada pengguna, dan justru itulah alasan keduanya diajarkan lebih dahulu daripada model yang lebih kuat tetapi lebih tertutup.

Modul ini memakai caret karena satu pemanggilan fungsi sudah menangani prapemrosesan, validasi silang, dan penalaan sekaligus. Beban teknisnya rendah sehingga perhatian dapat dipusatkan pada gagasan model. Mulai Modul 9, pembahasan beralih ke `tidymodels` ketika kebutuhan prapemrosesan menjadi lebih rumit.

Tentang data. Berkas data pada modul ini adalah **data simulasi** (`set.seed(2026)`), **bukan data nyata** dan **bukan data resmi** instansi mana pun. Peubah dibangkitkan saling bebas terhadap sasaran, sehingga kinerja mendekati tebakan acak adalah hasil yang benar dan wajib dilaporkan apa adanya.

8.2 Tujuan Praktikum

Setelah menyelesaikan praktikum ini secara tuntas, mahasiswa diharapkan mampu:

1. **Menjelaskan** cara kerja pohon keputusan beserta ukuran ketidakmurniannya.
2. **Membangun** dan **menggambar** pohon keputusan serta membacanya sebagai aturan.
3. **Menala** parameter kerumitan pohon dan menjelaskan akibat pemangkasan.
4. **Menerapkan** Naive Bayes serta menyebutkan anggapan yang mendasarinya.
5. **Menerapkan** regresi logistik terregularisasi dan menafsirkan koefisiennya.
6. **Membandingkan** ketiga model pada lipatan yang sama secara adil.
7. **Memilih** model berdasarkan sifat data dan kebutuhan penjelasan, bukan akurasi semata.
8. **Menjelaskan** hasil model kepada pengguna tanpa istilah teknis.

8.3 Alat dan Bahan

Perangkat dan bahan yang diperlukan:

- R (RStudio) dengan paket `caret`, `rpart`, `rpart.plot`, `naivebayes`, dan `glmnet`.
- Pasang dengan `install.packages(c("caret", "rpart", "rpart.plot", "naivebayes", "glmnet"))`.
- Berkas data: `data/data_nasabah.csv`.

8.4 Instruksi Pengerjaan

Ikuti tahapan berikut:

Sebelum praktikum: pasang seluruh paket; siapkan berkas data; baca kembali bagian ukuran penilaian pada Modul 6.

Selama praktikum: kerjakan Contoh berurutan. Perhatikan bahwa ketiga model menuntut prapemrosesan yang berbeda — ini menjadi sorotan utama modul.

Pembagian kerja: Contoh dan Latihan dikerjakan di kelas; **Penugasan dikerjakan di luar sesi.**

Pengumpulan: susun laporan R Markdown, render ke PDF atau HTML, kumpulkan melalui Google Classroom.

8.5 Teori Singkat

8.5.0.1 Kedudukan pada CRISP-DM

Modul ini menempati tahap Modeling. Perlu diingat bahwa pemilihan model bukan langkah pertama melainkan kelanjutan dari perumusan persoalan dan penyiapan data — keduanya sudah menentukan model mana yang masuk akal dipertimbangkan.

8.5.0.2 Pohon Keputusan

Pohon keputusan membelah data secara berulang berdasarkan satu peubah pada setiap simpul, memilih pembelahan yang paling memurnikan kelompok hasilnya. Proses berhenti ketika penambahan pembelahan tidak lagi memperbaiki kemurnian secara berarti.

Dua ukuran ketidakmurnian pada simpul dengan proporsi kelas p_k :

Gini: $G = 1 - \sum_k p_k^2$

Entropi: $H = - \sum_k p_k \log_2 p_k$

Keduanya bernilai nol ketika simpul murni dan memuncak ketika kelas berimbang.

Pembelahan dipilih yang paling menurunkan ketidakmurnian tertimbang. Kedua ukuran umumnya menghasilkan pohon yang mirip; Gini sedikit lebih cepat dihitung sehingga menjadi bawaan pada banyak perangkat

lunak.

Sifat pohon keputusan.

Tidak perlu dibakukan	pembelahan hanya membandingkan urutan, bukan jarak
Menerima kategori	tanpa pengodean boneka
Tahan pencilan	pencilan hanya memengaruhi satu daun
Mudah dijelaskan	dapat dibaca sebagai rangkaian aturan
Tidak stabil	perubahan kecil data dapat mengubah pohon

Sifat terakhir adalah kelemahan pokoknya. Ketidakstabilan itulah yang kelak diatasi hutan acak pada Modul 9 dengan merata-ratakan banyak pohon.

8.5.0.3 Pemangkasan dan Parameter Kerumitan

Pohon yang dibiarkan tumbuh penuh akan menghafal data latih. Pemangkasan mengendalikannya melalui parameter kerumitan: pembelahan hanya dipertahankan bila memperbaiki kemurnian melebihi ambang tersebut.

Tips dan Praktik Baik. Parameter kerumitan bekerja terbalik dengan besarnya pohon. Nilai yang besar menghasilkan pohon kecil yang mungkin kurang luwes (*underfitting*), sedangkan nilai mendekati nol menghasilkan pohon besar yang cenderung mengalami *overfitting*. Nilainya karena itu wajib ditala, bukan dibiarkan pada nilai bawaan.

8.5.0.4 Naive Bayes

Naive Bayes menghitung peluang kelas memakai kaidah Bayes, dengan satu anggapan penyederhana: seluruh peubah saling bebas apabila kelasnya diketahui.

$$P(y \mid x_1, \dots, x_p) \propto P(y) \prod_{j=1}^p P(x_j \mid y)$$

Anggapan kebebasan mengubah peluang gabungan yang sukar diduga menjadi hasil kali peluang tunggal yang mudah diduga.

Anggapan kebebasan hampir selalu keliru. Peubah pada data nyata jarang benar-benar saling bebas; pendapatan dan lama bekerja jelas berkaitan. Menariknya, Naive Bayes tetap kerap bekerja baik meski anggapannya dilanggar — sebab untuk *menggolongkan* yang diperlukan hanya urutan peluang antar-kelas, bukan besarnya yang tepat. Namun peluang keluarannya menjadi tidak dapat dipercaya sebagai peluang, dan karena itu tidak layak dipakai menetapkan ambang berbasis biaya.

8.5.0.5 Regresi Logistik dan Regularisasi

Regresi logistik menduga peluang melalui fungsi logistik atas kombinasi linier peubah. Keunggulan pokoknya terletak pada koefisien yang dapat ditafsirkan sebagai rasio odds.

$$\log \frac{P(y = 1)}{1 - P(y = 1)} = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

Kenaikan satu satuan x_j mengalikan odds dengan e^{β_j} , dengan peubah lain ditahan tetap.

Regularisasi menambahkan hukuman atas besarnya koefisien sehingga model tidak terlalu bergantung pada satu peubah. Ridge mengecilkan seluruh koefisien, Lasso dapat menolak sebagian sehingga sekaligus menyeleksi peubah, sedangkan Elastic Net menggabungkan keduanya.

Tiga bentuk regularisasi.

Bentuk	alpha	Sifat
Ridge	0	seluruh koefisien mengecil, tak ada yang nol
Lasso	1	sebagian koefisien tepat nol; menyeleksi peubah
Elastic Net	0 sampai 1	gabungan; baik saat peubah berkorelasi

Regularisasi menuntut pembakuan. Hukuman dikenakan atas besarnya koefisien, sedangkan besar koefisien bergantung satuan peubah. Tanpa pembakuan, peubah bersatuan besar memperoleh koefisien kecil sehingga nyaris tidak terhukum, dan sebaliknya. Pembakuan karena itu **wajib** bagi Ridge, Lasso, maupun Elastic Net — berbeda dari pohon keputusan yang tidak memerlukannya sama sekali.

8.5.0.6 Memilih di Antara Ketiganya

Perbandingan tiga model dasar.

	Pohon	Naive Bayes	Log. terregularisasi
Pembakuan	tidak perlu	tidak perlu	wajib
Pengodean	tidak perlu	tidak perlu	wajib
Pencilan	tahan	cukup tahan	peka
Peluang	kasar	tak terkalibrasi	terkalibrasi baik
Penjelasan	aturan	peluang bersyarat	rasio odds
Kestabilan	rendah	tinggi	tinggi

Baris keempat menentukan ketika ambang hendak ditetapkan berdasarkan biaya sebagaimana dibahas pada Modul 7. Peluang yang tidak terkalibrasi membuat perhitungan biaya menjadi keliru meski penggolongannya tepat.

8.6 Contoh Praktikum dengan R

Contoh 8.1 | Menyiapkan data dan memeriksa sasaran

Kode 8.1 • R membaca data dan menetapkan acuan

```
nasabah <- read.csv("data/data_nasabah.csv", stringsAsFactors = TRUE)
nasabah$gagal_bayar <- factor(nasabah$gagal_bayar, levels = c("ya", "tidak"))
nasabah$id <- NULL # penanda tidak dimodelkan
str(nasabah)
```

```
'data.frame': 900 obs. of 9 variables:
 $ umur      : int  52 45 27 34 45 22 42 59 32 47 ...
 $ pendapatan : num  15156 20002 13811 12304 24273 ...
 $ lama_kerja : num  12.8 13 7.3 16 0 0.2 8.9 13.3 2.9 20.2 ...
 $ rasio_cicilan: num  0.414 0.525 0.417 0.243 0.123 0.393 0.242 0.398 0.4 0.176 ...
 $ jumlah_kartu : int  1 0 3 0 3 2 2 2 4 1 ...
 $ riwayat_telat: int  4 3 2 2 2 0 0 0 4 2 ...
 $ pendidikan : Factor w/ 5 levels "Diploma","Sarjana",...: 1 4 4 3 4 4 4 3 5 4 ...
 $ status_rumah : Factor w/ 3 levels "Keluarga","Milik",...: 2 2 2 1 2 2 3 2 2 2 ...
 $ gagal_bayar : Factor w/ 2 levels "ya","tidak": 2 2 2 2 2 2 2 2 2 2 ...
```

```
prop <- prop.table(table(nasabah$gagal_bayar))
cat("proporsi kelas:", round(prop, 4), "\n")
```

```
proporsi kelas: 0.4056 0.5944
```

```
cat("acuan tebakan mayoritas:", round(max(prop), 4), "\n")
```

```
acuan tebakan mayoritas: 0.5944
```

Penjelasan Kode. Data dibaca lalu taraf sasaran diurutkan agar kelas yang diperhatikan berada di depan. Kolom penanda dibuang seluruhnya karena caret tidak mengenal peran penanda sebagaimana recipes. Proporsi kelas dan acuan tebakan mayoritas dihitung.

Interpretasi Output

Membuang penanda secara tegas adalah langkah wajib pada caret. Bila dibiarkan, nomor urut ikut dimodelkan dan pohon dapat membelah berdasarkan nomor itu, menghasilkan model yang tampak sangat baik pada data latih semata karena menghafal. Kekeliruan ini tidak menimbulkan galat sehingga hanya ketahuan bila kita memeriksa peubah mana yang dipakai model.

Contoh 8.2 | Membagi data dan menetapkan skema penilaian

Kode 8.2 · R createDataPartition dan trainControl

```
suppressPackageStartupMessages(library(caret))
set.seed(2026)
idx <- createDataPartition(nasabah$gagal_bayar, p = 0.75, list = FALSE)
lat <- nasabah[idx, ]; uji <- nasabah[-idx, ]

kendali <- trainControl(method = "cv", number = 5,
                        classProbs = TRUE,
                        summaryFunction = twoClassSummary,
                        savePredictions = "final")

round(rbind(latih = prop.table(table(lat$gagal_bayar)),
            uji = prop.table(table(uji$gagal_bayar))), 4)
```

```
      ya tidak
latih 0.4053 0.5947
uji    0.4062 0.5938
```

Penjelasan Kode. Fungsi createDataPartition membagi data dengan penstrataan otomatis pada peubah sasaran. Objek kendali menetapkan skema validasi silang lima lipatan, meminta dugaan peluang dihitung, dan menyimpan dugaan pada lipatan terbaik untuk pemeriksaan lanjutan.

Interpretasi Output

Argumen savePredictions bernilai final berguna karena memungkinkan matriks kekeliruan dan kurva ROC dihitung dari hasil validasi silang, bukan dari data uji. Dengan begitu data uji tetap tersimpan utuh sampai akhir. Perhatikan pula bahwa summaryFunction diganti agar ROC menjadi ukuran pemilihan, bukan akurasi yang menyesatkan pada data timpang.

Contoh 8.3 | Membangun pohon keputusan**Kode 8.3 · R** train dengan method rpart

```
suppressPackageStartupMessages({library(rpart); library(rpart.plot)})
set.seed(2026)
m_pohon <- train(gagal_bayar ~ ., data = lat, method = "rpart",
                 tuneLength = 8, trControl = kendali, metric = "ROC")

print(m_pohon$results[order(-m_pohon$results$ROC), c("cp", "ROC", "Sens", "Spec")][1:5, ],
      row.names = FALSE)
```

```

      cp      ROC      Sens      Spec
0.004866 0.7257 0.5508 0.7935
0.010949 0.7235 0.5180 0.8483
0.005839 0.7209 0.5618 0.7960
0.029197 0.7177 0.4927 0.8782
0.007299 0.7176 0.5363 0.8235

```

```

cat("\ncp terbaik:", signif(m_pohon$bestTune$cp, 4),
    "\n| ROC:", round(max(m_pohon$results$ROC), 4), "\n")

```

```
cp terbaik: 0.004866 | ROC: 0.7257
```

Penjelasan Kode. Pohon dilatih dengan delapan nilai parameter kerumitan yang dicoba otomatis melalui `tuneLength`. Hasilnya diurutkan menurun berdasarkan ROC dan lima teratas ditampilkan bersama sensitivitas serta spesifisitasnya.

Interpretasi Output

Perhatikan bahwa tidak ada argumen `preProcess` sama sekali — pohon memang tidak memerlukan pembakuan maupun pengodean. Perhatikan pula hubungan terbalik antara parameter kerumitan dan ukuran pohon: nilai kecil menghasilkan pohon besar. Bila nilai terbaik jatuh pada ujung rentang yang dicoba, rentangnya perlu diperluas.

Contoh 8.4 | Menggambar dan membaca pohon sebagai aturan

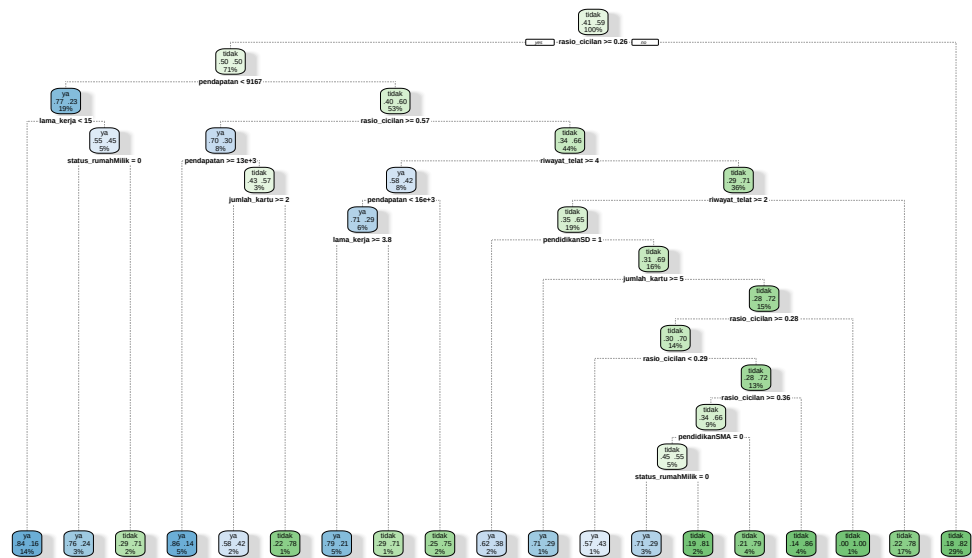
Kode 8.4 • R `rpart.plot` dan pembacaan struktur

```

rpart.plot(m_pohon$finalModel, type = 2, extra = 104,
           box.palette = "BuGn", branch.lty = 3, shadow.col = "grey85",
           main = "Pohon Keputusan Gagal Bayar")

```

Pohon Keputusan Gagal Bayar



```
cat("banyaknya daun:", sum(m_pohon$finalModel$frame$var == "<leaf>"), "\n")
```

banyaknya daun: 19

```
cat("kedalaman maksimum:",  
    max(rpart:::tree.depth(as.numeric(rownames(m_pohon$finalModel$frame)))), "\n")
```

kedalaman maksimum: 12

Penjelasan Kode. Pohon digambar dengan tipe dua yang menampilkan label pada tiap cabang. Argumen `extra` bernilai 104 menampilkan proporsi kelas beserta persentase amatan pada tiap simpul. Banyaknya daun dihitung dari kerangka model, dan kedalamannya dihitung dari penomoran simpulnya.

Interpretasi Output

Setiap jalur dari akar ke daun dapat dibaca sebagai satu aturan bersyarat, dan justru kemampuan itulah keunggulan utama pohon. Perhatikan persentase amatan pada tiap daun: daun yang hanya memuat sedikit amatan menghasilkan dugaan yang tidak stabil, sehingga aturan darinya tidak layak disampaikan sebagai temuan.

Contoh 8.5 | Menala parameter kerumitan secara terarah

Kode 8.5 · R tuneGrid dan palang simpangan baku

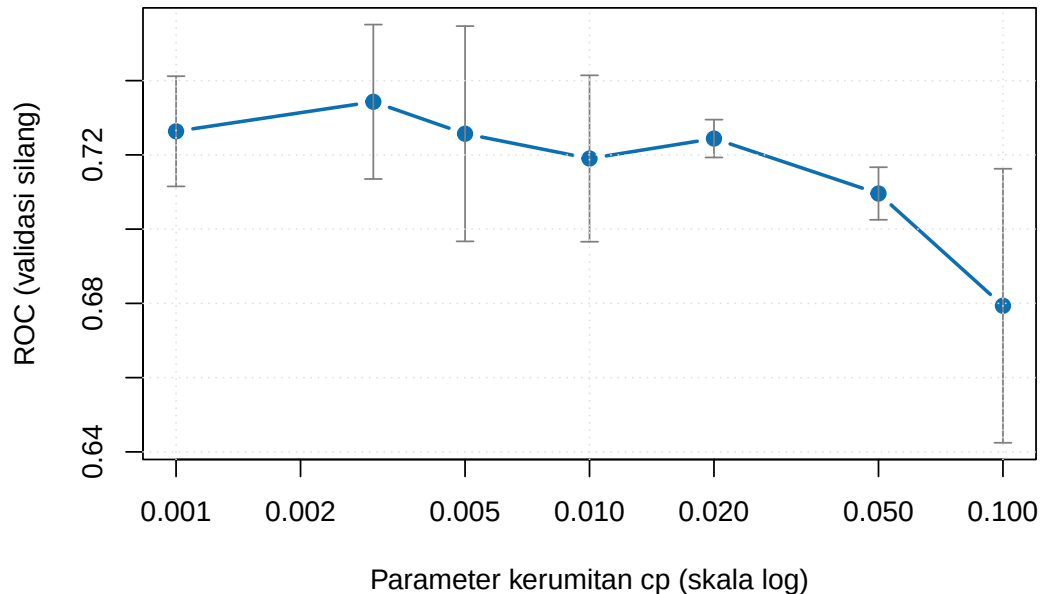
```
set.seed(2026)
kisi_cp <- data.frame(cp = c(0.001, 0.003, 0.005, 0.01, 0.02, 0.05, 0.1))
m_pohon2 <- train(gagal_bayar ~ ., data = lat, method = "rpart",
                  tuneGrid = kisi_cp, trControl = kendali, metric = "ROC")

hasil_cp <- m_pohon2$results[, c("cp", "ROC", "ROCSd")]
print(round(hasil_cp, 4), row.names = FALSE)
```

cp	ROC	ROCSd
0.001	0.7263	0.0148
0.003	0.7343	0.0208
0.005	0.7257	0.0290
0.010	0.7190	0.0224
0.020	0.7244	0.0051
0.050	0.7096	0.0071
0.100	0.6794	0.0369

```
plot(hasil_cp$cp, hasil_cp$ROC, type = "b", pch = 19, lwd = 2, log = "x",
     col = "#0B6FB4", ylim = range(c(hasil_cp$ROC - hasil_cp$ROCSd,
                                     hasil_cp$ROC + hasil_cp$ROCSd)),
     xlab = "Parameter kerumitan cp (skala log)", ylab = "ROC (validasi silang)",
     main = "Penalaan Parameter Kerumitan")
arrows(hasil_cp$cp, hasil_cp$ROC - hasil_cp$ROCSd,
       hasil_cp$cp, hasil_cp$ROC + hasil_cp$ROCSd,
       angle = 90, code = 3, length = 0.05, col = "grey50")
grid(col = "grey88")
```

Penalaan Parameter Kerumitan



Penjelasan Kode. Rentang parameter kerumitan ditetapkan sendiri agar mencakup nilai yang sangat kecil sampai besar. Hasilnya digambar pada skala logaritma karena nilainya berlipat, dilengkapi palang simpangan baku antar-lipatan.

Interpretasi Output

Panjang palang menyatakan seberapa besar hasil berubah antar-lipatan. Bila palang antar-nilai saling bertumpang tindih, selisih rata-ratanya tidak berarti dan memilih yang tertinggi menjadi sembarang. Dalam keadaan demikian, memilih parameter kerumitan yang lebih besar — yakni pohon yang lebih sederhana — lebih dapat dipertanggungjawabkan.

Contoh 8.6 | Membangun model Naive Bayes

Kode 8.6 · R train dengan method naive_bayes

```
suppressPackageStartupMessages(library(naivebayes))
set.seed(2026)
m_nb <- train(gagal_bayar ~ ., data = lat, method = "naive_bayes",
              trControl = kendali, metric = "ROC")
```

```
print(m_nb$results[, c("usekernel", "laplace", "ROC", "Sens", "Spec")],
      row.names = FALSE)
```

```
usekernel laplace    ROC    Sens    Spec
FALSE      0 0.7238 0.5622 0.7563
TRUE       0 0.7540 0.2591 0.9727
```

```
cat("\nROC terbaik:", round(max(m_nb$results$ROC), 4),
    "| pemulusan kernel:", m_nb$bestTune$usekernel, "\n")
```

```
ROC terbaik: 0.754 | pemulusan kernel: TRUE
```

Penjelasan Kode. Naive Bayes dilatih dengan dua pilihan yang dicoba otomatis: pendugaan kepekatan memakai sebaran normal atau memakai pemulusan kernel, serta koreksi Laplace bagi kategori yang tak muncul.

Interpretasi Output

Pemulusan kernel biasanya unggul ketika peubah numerik tidak menyebar normal, dengan biaya perhitungan yang lebih besar. Koreksi Laplace mencegah persoalan peluang nol: bila suatu taraf kategori tak pernah muncul bersama suatu kelas pada data latih, tanpa koreksi seluruh hasil kali peluangnya menjadi nol sehingga kelas itu mustahil terpilih.

Contoh 8.7 | Membangun regresi logistik terregularisasi

Kode 8.7 · R train dengan method glmnet dan pembakuan

```
suppressPackageStartupMessages(library(glmnet))
set.seed(2026)
m_glm <- train(gagal_bayar ~ ., data = lat, method = "glmnet",
               preprocess = c("center", "scale"), # WAJIB bagi regularisasi
               tuneLength = 5, trControl = kendali, metric = "ROC")
```

```
cat("alpha terbaik:", round(m_glm$bestTune$alpha, 3),
    "| lambda terbaik:", signif(m_glm$bestTune$lambda, 4), "\n")
```

```
alpha terbaik: 0.1 | lambda terbaik: 0.003273
```

```
cat("ROC terbaik:", round(max(m_glm$results$ROC), 4), "\n")
```

```
ROC terbaik: 0.7796
```

```
bentuk <- ifelse(m_glm$bestTune$alpha == 0, "Ridge",
                 ifelse(m_glm$bestTune$alpha == 1, "Lasso", "Elastic Net"))
cat("bentuk regularisasi terpilih:", bentuk, "\n")
```

bentuk regularisasi terpilih: Elastic Net

Penjelasan Kode. Model dilatih dengan pembakuan yang ditetapkan lewat `preProcess`. Argumen `tuneLength` meminta caret mencoba beberapa nilai `alpha` dan `lambda` secara otomatis. Bentuk regularisasi disimpulkan dari nilai `alpha` terpilih.

Interpretasi Output

Bandingkan dengan kedua model sebelumnya yang tidak memakai `preProcess` sama sekali. Perbedaan ini bukan soal selera melainkan keharusan: tanpa pembakuan, hukuman regularisasi dikenakan secara tidak adil antar-peubah yang satuannya berbeda. Inilah wujud nyata prinsip kesesuaian pra-pemrosesan dengan model.

Contoh 8.8 | Menafsirkan koefisien sebagai rasio odds

Kode 8.8 · R `coef` dan `exp` untuk rasio odds

```
koef <- coef(m_glm$finalModel, s = m_glm$bestTune$lambda)
koef_df <- data.frame(peubah = rownames(koef), koefisien = as.numeric(koef))
koef_df <- koef_df[koef_df$peubah != "(Intercept)", ]
koef_df$odds_ratio <- round(exp(koef_df$koefisien), 4)
koef_df$koefisien <- round(koef_df$koefisien, 4)

aktif <- koef_df[koef_df$koefisien != 0, ]
aktif <- aktif[order(-abs(aktif$koefisien)), ]
print(utils::head(aktif, 8), row.names = FALSE)
```

peubah	koefisien	odds_ratio
rasio_cicilan	-0.8506	0.4272
pendapatan	0.6601	1.9351
riwayat_telat	-0.4982	0.6076
pendidikanSMP	-0.2830	0.7536
pendidikanSD	-0.2678	0.7651
status_rumahMilik	0.2046	1.2271
pendidikanSMA	-0.1671	0.8461
lama_kerja	0.1651	1.1795

```
cat("\npeubah dinolkan regularisasi:", sum(koef_df$koefisien == 0),
    "dari", nrow(koef_df), "\n")
```

```
peubah dinolkan regularisasi: 0 dari 12
```


Penjelasan Kode. Koefisien diambil pada nilai lambda terpilih. Intersep dikeluarkan karena tidak di-tafsirkan sebagai rasio odds. Rasio odds diperoleh dengan mengeksponenkan koefisien, lalu peubah yang tidak dinolkan diurutkan menurut besarnya pengaruh.

Interpretasi Output

Rasio odds di atas satu menaikkan peluang kelas yang diperhatikan, sedangkan di bawah satu menurunkannya. Perlu ditekankan bahwa karena peubah sudah dibakukan, koefisien menyatakan pengaruh kenaikan satu simpangan baku — bukan satu satuan asli. Menafsirkannya dalam satuan asli adalah kekeliruan yang sering terjadi. Perhatikan pula banyaknya peubah yang dinolkan: itulah seleksi peubah yang dikerjakan Lasso secara otomatis.

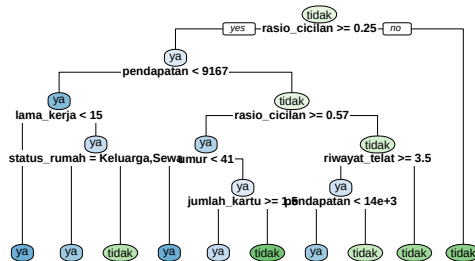
Contoh 8.9 | Memperagakan ketidakstabilan pohon

Kode 8.9 • R empat pohon dari contoh yang sedikit berbeda

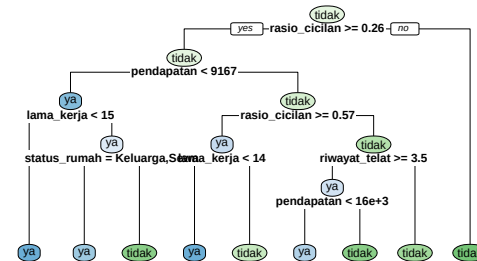
```
set.seed(2026)
pohon_kecil <- lapply(1:4, function(i) {
  s <- lat[sample(nrow(lat), round(0.9 * nrow(lat))), ]
  rpart::rpart(gagal_bayar ~ ., data = s,
               control = rpart::rpart.control(cp = 0.01))
})

par(mfrow = c(2, 2), mar = c(1, 1, 3, 1))
for (i in 1:4)
  rpart.plot(pohon_kecil[[i]], type = 2, extra = 0, cex = 0.55,
             box.palette = "BuGn", main = paste("Contoh acak", i))
```

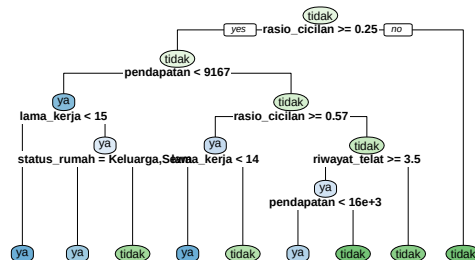
Contoh acak 1



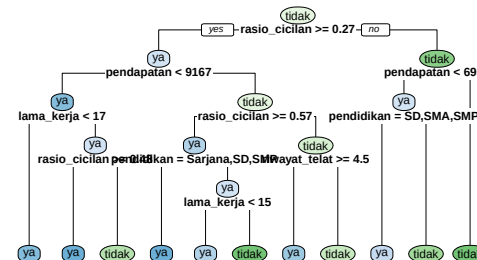
Contoh acak 2



Contoh acak 3



Contoh acak 4



```
par(mfrow = c(1, 1), mar = c(5, 4, 4, 2) + 0.1)
```

```
akar <- sapply(pohon_kecil, function(p)
```

```
  as.character(p$frame$var[1]))
```

```
cat("peubah pada simpul akar:", paste(akar, collapse = ", "), "\n")
```

```
peubah pada simpul akar: rasio_cicilan, rasio_cicilan, rasio_cicilan, rasio_cicilan
```

```
cat("banyaknya peubah akar berbeda:", length(unique(akar)), "dari 4\n")
```

```
banyaknya peubah akar berbeda: 1 dari 4
```

Penjelasan Kode. Empat pohon dibangun dari empat contoh acak berukuran sembilan puluh persen data latih. Keempatnya digambar berdampingan, lalu peubah pada simpul akar masing-masing dibandingkan.

Interpretasi Output

Bila peubah akarnya berbeda-beda padahal datanya nyaris sama, itulah wujud ketidakstabilan pohon yang disebut pada Teori. Akibatnya serius bagi penafsiran: aturan yang disampaikan kepada pengguna dapat berubah total hanya karena beberapa amatan berganti. Justru kelemahan inilah yang diatasi hutan acak pada Modul 9 dengan merata-ratakan banyak pohon — dengan harga kehilangan kemudahan penjelasan.

Contoh 8.10 | Membandingkan kebutuhan prapemrosesan antar-model**Kode 8.10 • R** menguji akibat pembakuan pada dua model

```

set.seed(2026)
tanpa_baku <- train(gagal_bayar ~ ., data = lat, method = "glmnet",
                    tuneLength = 5, trControl = kendali, metric = "ROC")
set.seed(2026)
pohon_baku <- train(gagal_bayar ~ ., data = lat, method = "rpart",
                    preProcess = c("center", "scale"),
                    tuneLength = 8, trControl = kendali, metric = "ROC")

banding_pp <- data.frame(
  model = c("glmnet dibakukan", "glmnet TANPA baku",
            "rpart tanpa baku", "rpart dibakukan"),
  ROC = round(c(max(m_glm$results$ROC), max(tanpa_baku$results$ROC),
                max(m_pohon2$results$ROC), max(pohon_baku$results$ROC)), 4))
print(banding_pp, right = FALSE)

```

model	ROC
1 glmnet dibakukan	0.7796
2 glmnet TANPA baku	0.7796
3 rpart tanpa baku	0.7343
4 rpart dibakukan	0.7251

```

cat("\nselisih pada glmnet:",
    round(max(m_glm$results$ROC) - max(tanpa_baku$results$ROC), 4), "\n")

```

```
selisih pada glmnet: 0
```

```

cat("selisih pada rpart :",
    round(max(m_pohon2$results$ROC) - max(pohon_baku$results$ROC), 4), "\n")

```

```
selisih pada rpart : 0.0092
```

Penjelasan Kode. Dua model dilatih ulang dengan perlakuan prapemrosesan yang dibalik: regularisasi tanpa pembakuan, dan pohon dengan pembakuan. Keempat hasilnya disandingkan, lalu selisihnya dihitung bagi tiap model.

Interpretasi Output

Selisih pada pohon seharusnya mendekati nol, sebab pembelahan hanya membandingkan urutan nilai dan urutan tidak berubah oleh pembakuan — pembakuan pada pohon tidak merugikan, hanya sia-sia. Sebaliknya selisih pada regularisasi lazimnya nyata, sebab hukuman menjadi tidak adil antar-peubah yang satuannya berbeda. Peragaan ini mengubah aturan kesesuaian prapemrosesan dari hafalan men-

jadi sesuatu yang terbukti angkanya.

Contoh 8.11 | Membandingkan ketiga model pada lipatan yang sama

Kode 8.11 • R resamples untuk perbandingan adil

```
hasil_banding <- resamples(list(Pohon = m_pohon2,
                                NaiveBayes = m_nb,
                                LogRegularisasi = m_glm))
ringkas <- summary(hasil_banding)
print(round(ringkas$statistics$ROC[, c("Min.", "Mean", "Max.")] , 4))
```

	Min.	Mean	Max.
Pohon	0.7041	0.7343	0.7600
NaiveBayes	0.7348	0.7540	0.7715
LogRegularisasi	0.7607	0.7796	0.8000

```
cat("\nselisih rata-rata ROC terhadap model terbaik:\n")
```

selisih rata-rata ROC terhadap model terbaik:

```
rata <- ringkas$statistics$ROC[, "Mean"]
print(round(max(rata) - rata, 4))
```

Pohon	NaiveBayes	LogRegularisasi
0.0453	0.0256	0.0000

Penjelasan Kode. Fungsi `resamples` menyandingkan ketiga model yang dilatih pada lipatan validasi silang yang sama. Ringkasannya memuat nilai terkecil, rata-rata, dan terbesar ROC antar-lipatan. Selisih terhadap model terbaik dihitung sebagai pembandingan.

Interpretasi Output

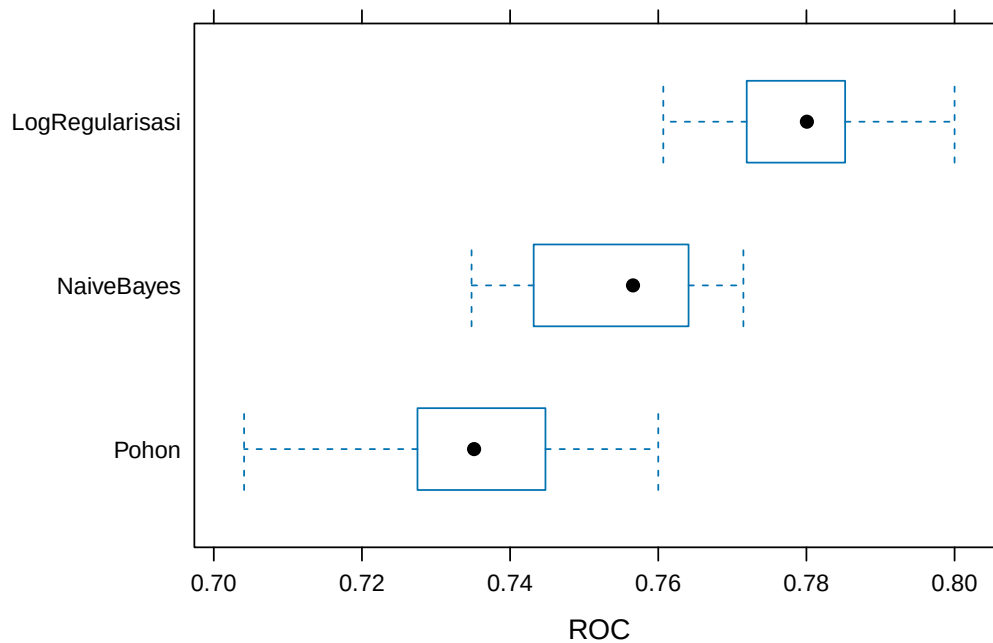
Kunci keadilan perbandingan adalah benih acak yang sama sehingga lipatannya identik bagi ketiga model; bila lipatannya berbeda, selisih yang terlihat boleh jadi hanya akibat pembagian. Perhatikan rentang antara nilai terkecil dan terbesar — rentang lebar menandakan model tidak stabil antar-lipatan, dan model semacam itu berisiko meski rata-ratanya tinggi.

Contoh 8.12 | Menggambarkan perbandingan model

Kode 8.12 • R bwplot atas objek `resamples`

```
bwplot(hasil_banding, metric = "ROC",
       main = "Sebaran ROC antar-Lipatan Validasi Silang")
```

Sebaran ROC antar-Lipatan Validasi Silang



Penjelasan Kode. Diagram kotak-garis digambar dari objek hasil perbandingan, menampilkan sebaran ROC tiap model pada kelima lipatan.

Interpretasi Output

Diagram ini lebih informatif daripada tabel rata-rata karena memperlihatkan sebarannya. Model dengan rata-rata sedikit lebih rendah tetapi sebaran sempit kerap lebih layak dipilih daripada model bermata-rata tinggi dengan sebaran lebar — terutama bila hasilnya akan dipakai berulang pada data baru yang bermacam-macam.

Contoh 8.13 | Menilai model terpilih pada data uji

Kode 8.13 • R confusionMatrix pada data uji

```
terbaik <- list(Pohon = m_pohon2, NaiveBayes = m_nb, LogRegularisasi = m_glm)[[
  names(which.max(rata))]]
cat("model terpilih:", names(which.max(rata)), "\n\n")
```

```
model terpilih: LogRegularisasi
```

```
duga_kelas <- predict(terbaik, uji)
duga_prob <- predict(terbaik, uji, type = "prob")

cm <- confusionMatrix(duga_kelas, uji$gagal_bayar, positive = "ya")
print(cm$table)
```

```
      Reference
Prediction ya tidak
      ya      54      29
      tidak  37     104
```

```
cat("\nakurasi:", round(cm$overall["Accuracy"], 4),
    "| acuan mayoritas:", round(max(prop.table(table(uji$gagal_bayar))), 4), "\n")
```

```
akurasi: 0.7054 | acuan mayoritas: 0.5938
```

```
cat("sensitivitas:", round(cm$byClass["Sensitivity"], 4),
    "| presisi:", round(cm$byClass["Precision"], 4),
    "| F1:", round(cm$byClass["F1"], 4), "\n")
```

```
sensitivitas: 0.5934 | presisi: 0.6506 | F1: 0.6207
```

Penjelasan Kode. Model berata-rata ROC tertinggi dipilih secara terprogram, lalu dipakai menduga pada data uji. Fungsi `confusionMatrix` menghitung matriks kekeliruan beserta belasan ukuran turunnannya sekaligus, dengan kelas yang diperhatikan ditetapkan lewat argumen `positive`.

Interpretasi Output

Menetapkan argumen `positive` secara tegas adalah keharusan; bila dibiarkan, caret memilih taraf pertama menurut abjad dan seluruh ukuran dapat mengukur kelas yang keliru. Bandingkan akurasi terhadap acuan mayoritas — bila berdekatan, model belum menyumbang apa pun betapapun rumitnya.

Contoh 8.14 | Memeriksa kalibrasi peluang

Kode 8.14 • R membandingkan peluang dugaan dan proporsi nyata

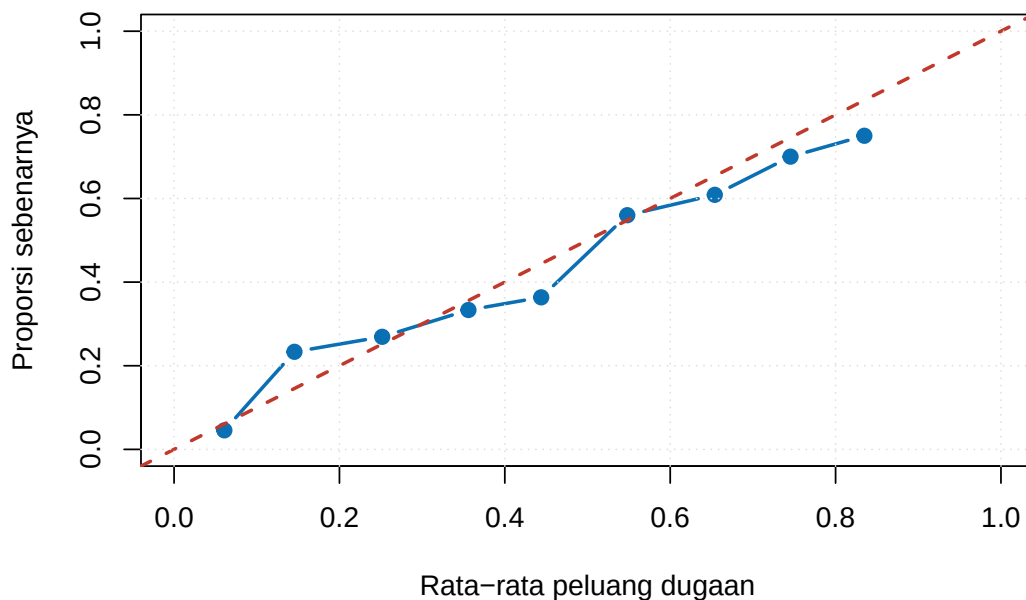
```
kelompok <- cut(duga_prob$ya, breaks = seq(0, 1, by = 0.1), include.lowest = TRUE)
kalibrasi <- data.frame(
  selang = levels(kelompok),
  n = as.numeric(table(kelompok)),
  rata_dugaan = round(tapply(duga_prob$ya, kelompok, mean), 4),
  proporsi_nyata = round(tapply(uji$gagal_bayar == "ya", kelompok, mean), 4))
```

```
kalibrasi <- kalibrasi[!is.na(kalibrasi$rata_dugaan) & kalibrasi$n >= 5, ]
print(kalibrasi, row.names = FALSE)
```

selang	n	rata_dugaan	proporsi_nyata
[0,0.1]	22	0.0608	0.0455
(0.1,0.2]	30	0.1456	0.2333
(0.2,0.3]	26	0.2517	0.2692
(0.3,0.4]	30	0.3560	0.3333
(0.4,0.5]	33	0.4440	0.3636
(0.5,0.6]	25	0.5481	0.5600
(0.6,0.7]	23	0.6539	0.6087
(0.7,0.8]	20	0.7455	0.7000
(0.8,0.9]	12	0.8348	0.7500

```
plot(kalibrasi$rata_dugaan, kalibrasi$proporsi_nyata, type = "b", pch = 19,
     lwd = 2, col = "#0B6FB4", xlim = c(0, 1), ylim = c(0, 1),
     xlab = "Rata-rata peluang dugaan", ylab = "Proporsi sebenarnya",
     main = "Kurva Kalibrasi")
abline(0, 1, col = "#C0392B", lty = 2, lwd = 2)
grid(col = "grey88")
```

Kurva Kalibrasi



Penjelasan Kode. Peluang dugaan dipotong menjadi sepuluh selang. Pada tiap selang dihitung rata-rata peluang dugaan dan proporsi kejadian yang sesungguhnya. Selang beranggota kurang dari lima dibuang karena proporsinya tidak stabil.

Interpretasi Output

Model yang terkalibrasi baik menghasilkan titik yang berimpit dengan garis diagonal: ketika model menduga peluang tiga puluh persen, kira-kira tiga puluh persen di antaranya memang terjadi. Kalibrasi menentukan ketika ambang hendak ditetapkan berdasarkan biaya — peluang yang tidak terkalibrasi membuat perhitungan biaya menjadi keliru meski penggolongannya tepat. Inilah alasan Naive Bayes kurang sesuai untuk keperluan itu.

8.7 Latihan Mandiri

Kerjakan bertahap. Gunakan data/data_nasabah.csv.

8.7.0.0.1 Tingkat Dasar (melengkapi kode)

Latihan 8.1 | Menyiapkan dan membagi data

```
library(caret)
nasabah <- read.csv("data/data_nasabah.csv", stringsAsFactors = TRUE)
nasabah$id <- _____ # buang penanda
set.seed(2026)
idx <- _____(nasabah$gagal_bayar, p = 0.75, list = FALSE)
lat <- nasabah[idx, ]; uji <- nasabah[_____, ]
```

Latihan 8.2 | Pohon keputusan

```
library(rpart); library(rpart.plot)
kendali <- trainControl(method = "cv", number = 5, classProbs = _____,
                        summaryFunction = twoClassSummary)
m <- _____(gagal_bayar ~ ., data = lat, method = "_____",
                tuneLength = 8, trControl = kendali, metric = "ROC")
_____ (m$finalModel, type = 2, extra = 104) # gambar pohon
```

Latihan 8.3 | Regresi terregularisasi

```
library(glmnet)
m2 <- train(gagal_bayar ~ ., data = lat, method = "glmnet",
            preprocess = c("_____", "_____"), # pembakuan WAJIB
            tuneLength = 5, trControl = kendali, metric = "ROC")
koef <- _____(m2$finalModel, s = m2$bestTune$lambda)
```

8.7.0.0.2 Tingkat Menengah (menulis sendiri)

Latihan 8.4 | Membandingkan tiga model

Latih ketiga model — pohon, Naive Bayes, dan regresi terregularisasi — pada lipatan validasi silang yang sama. Sajikan perbandingannya sebagai tabel dan diagram kotak-garis. Tentukan model yang Anda pilih, dan jelaskan mengapa rata-rata tertinggi belum tentu menjadi alasan yang cukup.

Latihan 8.5 | Membetulkan kode yang salah

Kode berikut memuat empat kesalahan (penanda ikut dimodelkan, glmnet tanpa pembakuan, ukuran pemilihan memakai akurasi pada data timpang, dan confusionMatrix tanpa menetapkan kelas positif). Temukan dan perbaiki, lalu jelaskan akibat masing-masing.

```
library(caret)
nasabah <- read.csv("data/data_nasabah.csv", stringsAsFactors = TRUE)

# (a) kolom penanda dibiarkan ikut dimodelkan
idx <- createDataPartition(nasabah$gagal_bayar, p = 0.75, list = FALSE)
lat <- nasabah[idx, ]; uji <- nasabah[-idx, ]

# (b) regularisasi tanpa pembakuan
m <- train(gagal_bayar ~ ., data = lat, method = "glmnet",
           tuneLength = 5, trControl = trainControl(method = "cv", number = 5))

# (c) pemilihan model memakai akurasi pada data timpang
kendali <- trainControl(method = "cv", number = 5)
m2 <- train(gagal_bayar ~ ., data = lat, method = "rpart",
            trControl = kendali, metric = "Accuracy")

# (d) kelas positif tidak ditetapkan
confusionMatrix(predict(m, uji), uji$gagal_bayar)
```

Keluaran yang benar. Setelah diperbaiki, kode yang dipakai adalah sebagai berikut.

```
# (a) penanda dibuang sebelum pemodelan
nasabah$id <- NULL
nasabah$gagal_bayar <- factor(nasabah$gagal_bayar, levels = c("ya", "tidak"))
idx <- createDataPartition(nasabah$gagal_bayar, p = 0.75, list = FALSE)
lat <- nasabah[idx, ]; uji <- nasabah[-idx, ]

# (b) pembakuan wajib bagi regularisasi
kendali <- trainControl(method = "cv", number = 5, classProbs = TRUE,
                       summaryFunction = twoClassSummary)
m <- train(gagal_bayar ~ ., data = lat, method = "glmnet",
           preprocess = c("center", "scale"),
           tuneLength = 5, trControl = kendali, metric = "ROC")

# (c) ukuran pemilihan memakai ROC, bukan akurasi
m2 <- train(gagal_bayar ~ ., data = lat, method = "rpart",
            tuneLength = 8, trControl = kendali, metric = "ROC")
```

```
# (d) kelas positif ditetapkan secara tegas
```

```
confusionMatrix(predict(m, uji), uji$gagal_bayar, positive = "ya")
```

Pemeriksaan yang wajib berhasil: !("id" %in% names(lat)) bernilai **TRUE**, dan "ROC" %in% names(m\$results) bernilai **TRUE**.

8.7.0.0.3 Tingkat Lanjut (pemilihan)

Latihan 8.6 | Memilih model menurut kebutuhan

Untuk masing-masing keadaan berikut, tentukan model yang Anda pilih dari ketiganya beserta alasannya, dan sebutkan **satu kelemahan** dari pilihan itu: (a) hasil harus dijelaskan kepada petugas lapangan sebagai aturan sederhana; (b) peluang keluaran akan dipakai menetapkan ambang berdasarkan biaya; (c) data memuat banyak peubah kategori bertaraf banyak; (d) data memuat pencilan ekstrem yang diyakini sah.

8.8 Penugasan

Sebagai calon analis pada instansi statistik, bangun dan bandingkan tiga model klasifikasi dasar atas `data_nasabah.csv` (*data simulasi*), lalu pilih satu model beserta alasan yang mempertimbangkan bukan hanya kinerja melainkan juga kebutuhan penjelasan.

Konteks. Sebuah unit hendak menduga risiko gagal bayar. Selain kinerja, pimpinan menuntut agar hasilnya dapat dijelaskan kepada petugas yang tidak berlatar statistika.

Tugas 8.1 | Penyiapan

1. Siapkan data: buang penanda, tetapkan urutan taraf sasaran, dan laporkan **acuan tebakan mayoritas**.
2. Bagi data dengan penstrataan dan tetapkan skema validasi silang lima lipatan berukuran ROC. Jelaskan mengapa ROC dipilih alih-alih akurasi.

Tugas 8.2 | Pemodelan

3. Bangun **pohon keputusan** dengan penalaan parameter kerumitan pada minimal tujuh nilai. Sajikan tabel hasil beserta simpangan bakunya dan grafik penalaannya.
4. **Gambar pohon terpilih** dan tuliskan **minimal tiga aturan** yang dapat dibaca darinya dalam bahasa sehari-hari. Sebutkan berapa persen amatan yang tercakup tiap aturan.
5. Bangun **Naive Bayes**. Laporkan apakah pemulusan kernel terpilih, dan jelaskan anggapan yang mendasari model ini beserta apakah anggapan itu masuk akal pada data ini.
6. Bangun **regresi logistik terregularisasi** dengan pembakuan. Laporkan bentuk regularisasi terpilih, banyaknya peubah yang dinolkan, dan **lima rasio odds terbesar** beserta tafsirannya.

Tugas 8.3 | Perbandingan dan Pemilihan

7. Bandingkan ketiganya pada **lipatan yang sama**. Sajikan tabel ringkasan dan diagram kotak-garis sebaran ROC antar-lipatan.
8. Nilai model terpilih pada **data uji**: matriks kekeliruan, minimal lima ukuran, dan perbandingan terhadap acuan.
9. Periksa **kalibrasi peluang** model terpilih. Jelaskan akibatnya bila peluang itu dipakai menetapkan ambang berbasis biaya.
10. Tentukan model yang Anda rekomendasikan. **Pertimbangkan bukan hanya ROC** melainkan juga kesta-

bilan antar-lipatan, kebutuhan prapemrosesan, kalibrasi, dan kemudahan penjelasan.

11. Susun laporan R Markdown berisi kode, keluaran, gambar, dan interpretasi. Tutup dengan **ringkasan (maksimal 300 kata)** memuat: model yang direkomendasikan beserta alasannya, kinerja terhadap acuan, keterbatasan, dan penegasan bahwa data merupakan simulasi sehingga kinerja mendekati tebakan acak adalah hasil yang wajar.

8.9 Tambahan: Menafsirkan Model kepada Pengguna

Model yang tidak dapat dijelaskan sukar dipakai, betapapun tinggi kinerjanya. Bagian ini membahas cara menyampaikan hasil ketiga model kepada pengguna yang tidak berlatar statistika.

Contoh 8.15 | Menerjemahkan pohon menjadi aturan tertulis

Kode 8.15 • R `rpart.rules`

```
aturan <- rpart.rules(m_pohon2$finalModel, cover = TRUE, style = "tall")
print(aturan)
```

```
.outcome is 0.14 with cover 5% when
  rasio_cicilan >= 0.57
  pendapatan >= 13096

.outcome is 0.16 with cover 14% when
  rasio_cicilan >= 0.26
  pendapatan < 9167
  lama_kerja < 15.2

.outcome is 0.21 with cover 5% when
  rasio_cicilan is 0.26 to 0.57
  pendapatan is 9167 to 15627
  riwayat_telat >= 4
  lama_kerja >= 3.8

.outcome is 0.24 with cover 3% when
  rasio_cicilan >= 0.26
  pendapatan < 9167
  lama_kerja >= 15.2
  status_rumahMilik is 0

.outcome is 0.29 with cover 1% when
  rasio_cicilan is 0.26 to 0.57
  pendapatan >= 9167
  riwayat_telat is 2 to 4
```

```
jumlah_kartu >= 5
pendidikanSD is 0

.outcome is 0.29 with cover 3% when
  rasio_cicilan is 0.36 to 0.57
  pendapatan >= 9167
  riwayat_telat is 2 to 4
  jumlah_kartu < 5
  pendidikanSD is 0
  status_rumahMilik is 0
  pendidikanSMA is 0

.outcome is 0.33 with cover 2% when
  rasio_cicilan is 0.11 to 0.26
  pendapatan < 12194
  pendidikanSMP is 1

.outcome is 0.33 with cover 1% when
  rasio_cicilan is 0.11 to 0.26
  pendapatan < 6421
  pendidikanSMP is 0

.outcome is 0.38 with cover 2% when
  rasio_cicilan is 0.26 to 0.57
  pendapatan >= 9167
  riwayat_telat is 2 to 4
  pendidikanSD is 1

.outcome is 0.42 with cover 2% when
  rasio_cicilan >= 0.57
  pendapatan is 9167 to 13096
  jumlah_kartu >= 2

.outcome is 0.43 with cover 1% when
  rasio_cicilan is 0.28 to 0.29
  pendapatan >= 9167
  riwayat_telat is 2 to 4
  jumlah_kartu < 5
  pendidikanSD is 0

.outcome is 0.43 with cover 1% when
  rasio_cicilan is 0.11 to 0.26
  pendapatan is 11560 to 12194
  pendidikanSMP is 0

.outcome is 0.71 with cover 2% when
  rasio_cicilan >= 0.26
```

```
pendapatan < 9167
lama_kerja >= 15.2
status_rumahMilik is 1

.outcome is 0.71 with cover 1% when
  rasio_cicilan is 0.26 to 0.57
  pendapatan is 9167 to 15627
  riwayat_telat >= 4
  lama_kerja < 3.8

.outcome is 0.75 with cover 2% when
  rasio_cicilan is 0.26 to 0.57
  pendapatan >= 15627
  riwayat_telat >= 4

.outcome is 0.78 with cover 1% when
  rasio_cicilan >= 0.57
  pendapatan is 9167 to 13096
  jumlah_kartu < 2

.outcome is 0.78 with cover 17% when
  rasio_cicilan is 0.26 to 0.57
  pendapatan >= 9167
  riwayat_telat < 2

.outcome is 0.79 with cover 4% when
  rasio_cicilan is 0.36 to 0.57
  pendapatan >= 9167
  riwayat_telat is 2 to 4
  jumlah_kartu < 5
  pendidikanSD is 0
  pendidikanSMA is 1

.outcome is 0.81 with cover 2% when
  rasio_cicilan is 0.36 to 0.57
  pendapatan >= 9167
  riwayat_telat is 2 to 4
  jumlah_kartu < 5
  pendidikanSD is 0
  status_rumahMilik is 1
  pendidikanSMA is 0

.outcome is 0.83 with cover 6% when
  rasio_cicilan is 0.11 to 0.26
  pendapatan is 6421 to 11560
  pendidikanSMP is 0
```

```
.outcome is 0.86 with cover 4% when
  rasio_cicilan is 0.29 to 0.36
  pendapatan >= 9167
  riwayat_telat is 2 to 4
  jumlah_kartu < 5
  pendidikanSD is 0

.outcome is 0.91 with cover 5% when
  rasio_cicilan < 0.11
  pendapatan < 12194

.outcome is 0.93 with cover 14% when
  rasio_cicilan < 0.26
  pendapatan >= 12194

.outcome is 1.00 with cover 1% when
  rasio_cicilan is 0.26 to 0.28
  pendapatan >= 9167
  riwayat_telat is 2 to 4
  jumlah_kartu < 5
  pendidikanSD is 0
```

Penjelasan Kode. Fungsi `rpart.rules` menerjemahkan struktur pohon menjadi daftar aturan bersyarat. Argumen `cover` menampilkan persentase amatan yang tercakup tiap aturan, dan gaya `tall` membuat tiap aturan ditampilkan pada barisnya sendiri.

Interpretasi Output

Keluaran ini dapat disalin nyaris apa adanya ke dalam laporan bagi pengguna. Yang perlu ditambahkan hanyalah penerjemahan nama peubah menjadi istilah sehari-hari. Perhatikan kolom cakupan: aturan yang hanya mencakup beberapa persen amatan sebaiknya tidak disampaikan sebagai temuan, sebab dugaan darinya tidak stabil.

Contoh 8.16 | Menyusun ringkasan pengaruh peubah

Kode 8.16 • R `varImp` dan `barplot` mendatar

```
penting <- varImp(m_glm)$importance
penting$peubah <- rownames(penting)
penting <- penting[order(-penting$Overall), ]
penting <- penting[penting$Overall > 0, ]
penting$Overall <- round(penting$Overall, 2)
print(utils::head(penting[, c("peubah", "Overall")], 8), row.names = FALSE)
```



```

      peubah Overall
ratio_cicilan 100.00
pendapatan   76.58
riwayat_telat 56.66
pendidikanSMP 30.19
pendidikanSD  28.33
status_rumahMilik 20.55
pendidikanSMA 15.94
lama_kerja    15.70

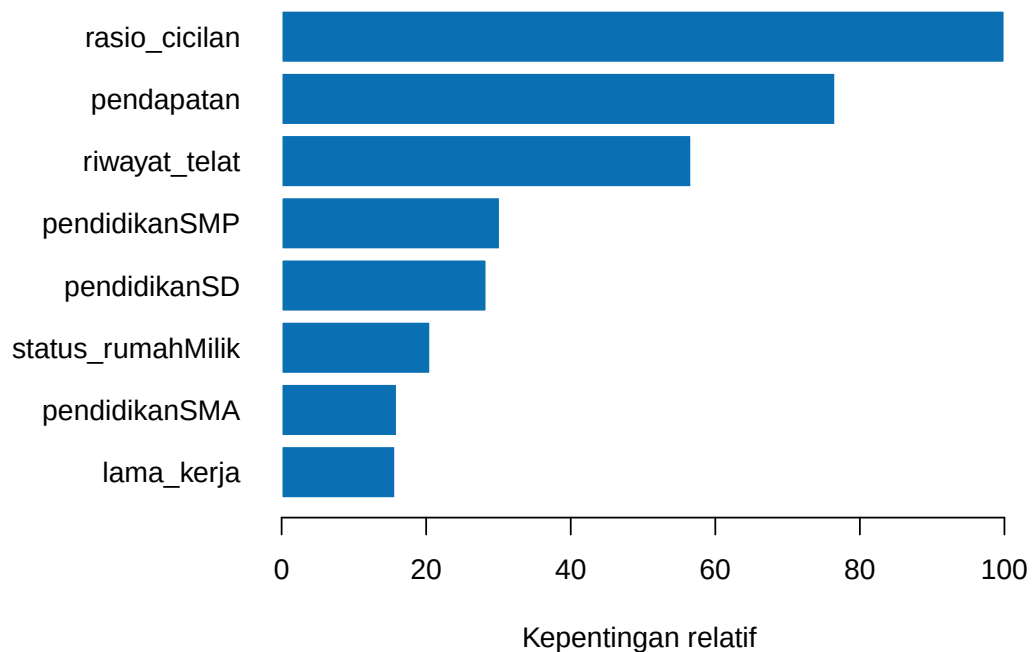
```

```

par(mar = c(4, 9, 3, 2))
atas <- utils::head(penting, 8)
barplot(rev(atas$Overall), names.arg = rev(atas$peubah), horiz = TRUE,
        las = 1, col = "#0B6FB4", border = "white",
        xlab = "Kepentingan relatif", main = "Peubah Paling Berpengaruh")

```

Peubah Paling Berpengaruh



```

par(mar = c(5, 4, 4, 2) + 0.1)

```

Penjelasan Kode. Fungsi `varImp` menghitung kepentingan relatif tiap peubah pada skala nol sampai seratus. Peubah berkepentingan nol — yang dinolkan regularisasi — disaring keluar, lalu delapan teratas digambar sebagai batang mendatar.

Interpretasi Output

Grafik semacam ini kerap menjadi satu-satunya bagian laporan yang dibaca pengambil keputusan, sehingga ketepatannya penting. Perlu ditegaskan dalam keterangan bahwa kepentingan menyatakan sumbangan terhadap dugaan, **bukan sebab-akibat**. Menyimpulkan bahwa mengubah suatu peubah akan mengubah hasilnya adalah kekeliruan penafsiran yang sering terjadi dan berakibat pada kebijakan yang salah sasaran.

Kepentingan peubah bukan hubungan sebab-akibat. Model dugaan menemukan kaitan, bukan sebab. Peubah riwayat telat boleh jadi paling berpengaruh dalam menduga gagal bayar, tetapi itu tidak berarti mengurangi catatan keterlambatan akan mengurangi risiko — keduanya boleh jadi sama-sama disebabkan hal lain. Menyampaikan kepentingan peubah tanpa peringatan ini berisiko melahirkan kebijakan yang keliru.

Contoh 8.17 | Menyusun kalimat penjelasan bagi pengguna

Kode 8.17 • R menyusun kalimat dugaan

```
contoh_nasabah <- uji[1:3, ]
p <- predict(terbaik, contoh_nasabah, type = "prob")$ya

for (i in 1:3) {
  tingkat <- ifelse(p[i] > 0.5, "tinggi", ifelse(p[i] > 0.25, "sedang", "rendah"))
  cat(sprintf("Nasabah %d: peluang dugaan %.1f persen (risiko %s).\n",
              i, 100 * p[i], tingkat))
  cat(sprintf("  Umur %d tahun, pendapatan %.0f, rasio cicilan %.2f, telat %d kali.\n",
              contoh_nasabah$umur[i], contoh_nasabah$pendapatan[i],
              contoh_nasabah$rasio_cicilan[i], contoh_nasabah$riwayat_telat[i]))
  cat(sprintf("  Status sebenarnya: %s\n", contoh_nasabah$gagal_bayar[i]))
}
```

Nasabah 1: peluang dugaan 37.2 persen (risiko sedang).

Umur 52 tahun, pendapatan 15156, rasio cicilan 0.41, telat 4 kali.

Status sebenarnya: tidak

Nasabah 2: peluang dugaan 37.1 persen (risiko sedang).

Umur 45 tahun, pendapatan 20002, rasio cicilan 0.53, telat 3 kali.

Status sebenarnya: tidak

Nasabah 3: peluang dugaan 27.1 persen (risiko sedang).

Umur 21 tahun, pendapatan 14047, rasio cicilan 0.29, telat 1 kali.

Status sebenarnya: tidak

Penjelasan Kode. Tiga amatan pertama data uji diambil sebagai contoh. Peluang dugaan diubah menjadi tingkatan risiko, lalu disusun sebagai kalimat bersama ciri pokok nasabahnya dan status sebenarnya.

Interpretasi Output

Menyertakan ciri pokok bersama dugaan membuat hasilnya dapat diperiksa kewajarannya oleh petugas yang memahami lapangan. Menyertakan status sebenarnya — yang tentu hanya mungkin pada data uji — berguna saat memperagakan model kepada pengguna, sebab memperlihatkan bahwa model kadang keliru. Menyembunyikan kekeliruan saat peragaan menumbuhkan kepercayaan berlebihan yang berbahaya.

Tips dan Praktik Baik. Ketika menyampaikan hasil kepada pengguna, sertakan tiga hal: dugaan beserta tingkat keyakinannya, ciri yang mendasarinya, dan contoh kasus ketika model keliru. Ketiganya membangun kepercayaan yang tepat — bukan terlalu tinggi, bukan pula terlalu rendah.

8.10 Ringkasan Modul

Daftar periksa penguasaan:

- ☐ Saya dapat menjelaskan cara kerja pohon beserta ukuran ketidakmurniannya.
- ☐ Saya dapat membaca pohon sebagai rangkaian aturan bersyarat.
- ☐ Saya menala parameter kerumitan dan memperhatikan simpangan bakunya.
- ☐ Saya tahu bahwa pohon dan Naive Bayes tidak memerlukan pembakuan.
- ☐ Saya tahu bahwa regularisasi *wajib* disertai pembakuan.
- ☐ Saya dapat menyebutkan anggapan Naive Bayes dan akibat pelanggarannya.
- ☐ Saya menafsirkan koefisien terbakukan sebagai rasio odds per simpangan baku.
- ☐ Saya membandingkan model pada lipatan yang sama.
- ☐ Saya memeriksa kalibrasi sebelum memakai peluang untuk keputusan biaya.
- ☐ Saya menyadari bahwa kepentingan peubah bukan hubungan sebab-akibat.

Kartu Referensi Cepat.

Keperluan	Perintah
Membagi berstrata	<code>createDataPartition(y, p =, list = FALSE)</code>
Skema penilaian	<code>trainControl(method = "cv", number = 5)</code>
Ukuran ROC	<code>summaryFunction = twoClassSummary</code>
Pohon keputusan	<code>train(..., method = "rpart")</code>
Menggambarkan pohon	<code>rpart.plot(m\$finalModel, extra = 104)</code>
Aturan pohon	<code>rpart.rules(m\$finalModel)</code>
Naive Bayes	<code>train(..., method = "naive_bayes")</code>
Regularisasi	<code>train(..., method = "glmnet")</code>
Pembakuan	<code>preProcess = c("center", "scale")</code>
Koefisien	<code>coef(m\$finalModel, s = m\$bestTune\$lambda)</code>
Kepentingan peubah	<code>varImp(m)</code>
Banding model	<code>resamples(list(...)), bwplot()</code>
Matriks kekeliruan	<code>confusionMatrix(duga, benar, positive =)</code>

Modul ini membangun tiga model dasar yang seluruhnya mudah dijelaskan. Prinsip yang perlu dibawa: sesuaikan prapemrosesan dengan kebutuhan model — pohon tidak menuntut apa pun, regularisasi menuntut pembakuan; bandingkan model pada lipatan yang sama; perhatikan kestabilan antar-lipatan, bukan rata-rata semata; periksa kalibrasi sebelum memakai peluang untuk keputusan; dan ingat bahwa kepentingan peubah menyatakan kaitan, bukan sebab. Modul berikutnya membahas model yang lebih kuat tetapi lebih sukar dijelaskan.

8.11 Referensi

- Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.
- Kuhn, M., & Johnson, K. (2013). *Applied Predictive Modeling*. Springer.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2nd ed.). Springer.
- Hastie, T., Tibshirani, R., & Wainwright, M. (2015). *Statistical Learning with Sparsity: The Lasso and Generalizations*. CRC Press.
- Hand, D. J., & Yu, K. (2001). Idiot's Bayes: Not so stupid after all? *International Statistical Review*, 69(3), 385-398.

MODUL 9

KLASIFIKASI LANJUT: SVM, KNN, DAN RANDOM FOREST

Pertemuan Minggu ke-9

Sub-CPMK 9: Mahasiswa mampu membangun dan menala model klasifikasi lanjut — Support Vector Machine, K-Nearest Neighbours, dan Random Forest — menyesuaikan prapemrosesan dengan kebutuhan tiap model, membandingkannya secara adil, serta mengenali gagasan boosting sebagai keluarga model penggabung.

Penyusun: Achmad Syahrul Choir



Daftar Isi

9.1	Deskripsi Modul	271
9.2	Tujuan Praktikum	271
9.3	Alat dan Bahan	272
9.4	Instruksi Pengerjaan	272
9.5	Teori Singkat	272
9.5.0.1	Kedudukan pada CRISP-DM	272
9.5.0.2	Support Vector Machine	273
9.5.0.3	K-Nearest Neighbours	273
9.5.0.4	Random Forest	274
9.5.0.5	Kesesuaian Prapemrosesan	274
9.5.0.6	Menafsirkan Model Penggabung	275
9.5.0.7	Boosting sebagai Keluarga Penggabung Lain	275
9.6	Contoh Praktikum dengan R	277
9.7	Latihan Mandiri	292
9.8	Penugasan	295
9.9	Tambahan: Boosting dengan Python	296
9.10	Ringkasan Modul	299
9.11	Referensi	300

9.1 Deskripsi Modul

Modul sebelumnya membangun tiga model dasar yang seluruhnya mudah dijelaskan. Modul ini melanjutkan ke tiga model yang lebih kuat tetapi menuntut lebih banyak: Support Vector Machine, K-Nearest Neighbours, dan Random Forest.

Ketiganya dipilih karena mewakili tiga cara berpikir yang berlainan. SVM mencari batas pemisah yang paling lebar antar-kelas; KNN sama sekali tidak membangun model melainkan menyimpan seluruh data lalu memutuskan berdasarkan tetangga terdekat; Random Forest menggabungkan banyak pohon untuk mengatasi ketidakstabilan pohon tunggal yang dibahas pada modul sebelumnya.

Perbedaan cara kerja itu membawa kebutuhan prapemrosesan yang berbeda pula — dan inilah sorotan utama modul ini. SVM dan KNN menuntut pembakuan karena keduanya bersandar pada jarak, sedangkan Random Forest tidak memerlukannya sama sekali. Menyeragamkan perlakuan bagi ketiganya adalah kekeliruan yang lazim.

Mulai modul ini pembahasan beralih dari `caret` ke `tidymodels`, sebab keunggulan resep terasa justru ketika tiap model menuntut prapemrosesan yang berlainan. Bagian Tambahan memperkenalkan boosting memakai Python.

Tentang data. Berkas data pada modul ini adalah **data simulasi** (`set.seed(2025)`), **bukan data nyata** dan **bukan data resmi** instansi mana pun. Data dibangkitkan dengan hubungan yang sungguh ada antara peubah penjelas dan sasaran, sehingga model diharapkan berkinerja baik — dan kinerja yang buruk justru menandakan ada kekeliruan pada alur kerja.

9.2 Tujuan Praktikum

Setelah menyelesaikan praktikum ini secara tuntas, mahasiswa diharapkan mampu:

1. **Menjelaskan** gagasan margin dan kernel pada Support Vector Machine.
2. **Menjelaskan** cara kerja KNN beserta akibat pemilihan banyaknya tetangga.
3. **Menjelaskan** bagaimana Random Forest mengatasi ketidakstabilan pohon tunggal.
4. **Menyesuaikan** resep prapemrosesan dengan kebutuhan tiap model.
5. **Menala** hiperparameter ketiga model dengan validasi silang.
6. **Membandingkan** ketiganya pada lipatan yang sama secara adil.
7. **Menafsirkan** Random Forest melalui kepentingan peubah.
8. **Menjelaskan** gagasan boosting dan perbedaannya terhadap bagging.

9. **Memilih** model dengan mempertimbangkan kinerja, kestabilan, dan biaya perhitungan.

9.3 Alat dan Bahan

Perangkat dan bahan yang diperlukan:

- **R** (RStudio) dengan paket inti tidymodels dan tiga mesin model.
- Pasang dengan:

```
install.packages(c("recipes", "rsample", "parsnip", "workflows",  
                  "tune", "yardstick", "dials"))  
install.packages(c("kernlab", "kknn", "ranger"))
```

- **Python** dengan xgboost dan lightgbm untuk bagian Tambahan.
- Berkas data: data/data_usahatani.csv.

9.4 Instruksi Pengerjaan

Ikuti tahapan berikut:

Sebelum praktikum: pasang seluruh paket; siapkan berkas data; tinjau kembali bagian kesesuaian prapemrosesan pada modul klasifikasi dasar.

Selama praktikum: kerjakan Contoh berurutan. Penalaan SVM memakan waktu paling lama, jadi jalankan sekali lalu simpan hasilnya.

Pembagian kerja: Contoh dan Latihan dikerjakan di kelas; **Penugasan dikerjakan di luar sesi.**

Pengumpulan: susun laporan R Markdown, render ke PDF atau HTML, kumpulkan melalui Google Classroom.

9.5 Teori Singkat

9.5.0.1 Kedudukan pada CRISP-DM

Modul ini menempati tahap Modeling dan Evaluation. Yang membedakannya dari modul klasifikasi dasar adalah keterkaitannya yang lebih erat dengan tahap Data Preparation: ketiga model menuntut prapemrosesan yang berbeda, sehingga resep tidak lagi dapat disamakan begitu saja.

9.5.0.2 Support Vector Machine

SVM mencari batas pemisah antar-kelas yang **marginnya paling lebar**, yaitu batas yang jaraknya terhadap amatan terdekat dari kedua kelas semaksimal mungkin. Amatan yang berada tepat pada tepi margin disebut vektor pendukung, dan hanya amatan itulah yang menentukan letak batasnya.

Batas keputusan SVM berbentuk:

$$f(x) = \sum_{i \in SV} \alpha_i y_i K(x_i, x) + b$$

dengan SV himpunan vektor pendukung dan K fungsi kernel. Amatan yang bukan vektor pendukung tidak berpengaruh sama sekali terhadap batasnya.

Sifat terakhir itu membuat SVM tahan terhadap amatan yang jauh dari batas, tetapi sekaligus peka terhadap amatan yang berada di dekat batas — termasuk amatan yang sesungguhnya salah label.

Dua parameter penala SVM berkernel RBF.

Parameter	Mengatur	Akibat bila terlalu besar
cost	hukuman atas kesalahan	margin menyempit, <i>overfitting</i>
rbf_sigma	lebar pengaruh tiap amatan	batas menjadi sangat berliku

Kernel adalah gagasan pokok yang membuat SVM kuat. Alih-alih memetakan data ke ruang berdimensi lebih tinggi secara nyata, kernel menghitung hasil kali dalam pada ruang itu secara langsung. Akibatnya batas yang lurus pada ruang berdimensi tinggi tampak melengkung pada ruang asli, tanpa biaya perhitungan pemetaannya.

9.5.0.3 K-Nearest Neighbours

KNN adalah model yang paling sederhana gagasannya sekaligus paling ganjil wataknya: ia tidak membangun model sama sekali. Seluruh data latih disimpan, dan keputusan diambil saat dugaan diminta dengan memeriksa tetangga terdekat amatan baru.

Akibat pemilihan banyaknya tetangga.

Nilai k	Perilaku	Risiko
kecil (1–3)	batas sangat berliku	menghafal derau
sedang	seimbang	–
besar	batas sangat halus	mengabaikan pola setempat

KNN memindahkan biaya ke tahap dugaan. Berbeda dari model lain, KNN nyaris tidak memerlukan waktu untuk dilatih — ia hanya menyimpan data. Sebaliknya, setiap kali dugaan diminta, jarak terha-

dap seluruh data latih harus dihitung. Pada penerapan yang menuntut dugaan cepat atau data latih yang besar, sifat ini menjadi persoalan serius yang kerap baru disadari setelah model diterapkan.

9.5.0.4 Random Forest

Modul sebelumnya memperagakan bahwa pohon keputusan tidak stabil: perubahan kecil pada data dapat mengubah seluruh strukturnya. Random Forest mengatasinya dengan menggabungkan banyak pohon lalu merata-ratakan hasilnya.

Dua sumber keragaman antar-pohon.

Bagging	tiap pohon dilatih pada contoh bootstrap yang berbeda
Pemilihan peubah acak	pada tiap pembelahan, hanya sebagian peubah dipertimbangkan

Sumber kedua itulah yang membedakan Random Forest dari bagging biasa. Tanpa pemilihan peubah acak, seluruh pohon cenderung memilih peubah kuat yang sama pada pembelahan pertama sehingga pohonnya mirip satu sama lain — dan merata-ratakan pohon yang mirip tidak banyak menolong.

Dua parameter penala Random Forest.

<code>mtry</code>	peubah dipertimbangkan per pembelahan	kecil: pohon makin beragam
<code>min_n</code>	amatan terkecil pada simpul	besar: pohon makin dangkal

Tips dan Praktik Baik. Banyaknya pohon (`trees`) lazimnya **tidak perlu ditala**. Menambah pohon tidak menyebabkan *overfitting* pada Random Forest — ia hanya menambah waktu perhitungan sampai kinerjanya mendatar. Tetapkan pada nilai yang cukup besar, misalnya lima ratus, lalu tala `mtry` dan `min_n` saja.

9.5.0.5 Kesesuaian Prapemrosesan

Inilah sorotan utama modul ini. Ketiga model menuntut perlakuan berbeda, dan menyeragamkannya adalah kekeliruan yang lazim.

Kebutuhan prapemrosesan tiap model.

	SVM	KNN	Random Forest
Pembakuan	wajib	wajib	tidak perlu
Pengodean	wajib	wajib	tidak perlu
Pencilan	peka	peka	tahan
Peubah tak relevan	mengganggu	sangat mengganggu	cukup tahan

Memakai satu resep bagi seluruh model. Menyusun satu resep lalu memakainya bagi ketiga model memang menyederhanakan kode, tetapi menyesatkan secara didaktis. Pembakuan pada Random Forest tidak merugikan — pembelahan hanya membandingkan urutan nilai, dan urutan tidak berubah oleh pembakuan — tetapi memperlihatkan bahwa penyusunnya belum memahami kebutuhan tiap model. Pada pekerjaan nyata, resep yang berlebihan juga menambah waktu perhitungan tanpa manfaat.

9.5.0.6 Menafsirkan Model Penggabung

Random Forest tidak dapat dibaca sebagai aturan sebagaimana pohon tunggal, sebab ia terdiri atas ratusan pohon. Penafsirannya bersandar pada kepentingan peubah, yang dapat dihitung dengan dua cara.

Dua ukuran kepentingan peubah.

Ketidakhayuan	jumlah penurunan ketidakhayuan akibat pembelahan pada peubah itu; cepat tetapi memihak peubah bertaraf banyak
Permutasi	penurunan kinerja ketika peubah diacak; lebih mahal tetapi lebih dapat dipercaya

9.5.0.7 Boosting sebagai Keluarga Penggabung Lain

Random Forest membangun pohon secara **sejajar dan saling bebas**, lalu merata-ratakan. Boosting bekerja sebaliknya: pohon dibangun **berurutan**, dan tiap pohon baru dilatih untuk memperbaiki kesalahan pohon sebelumnya.

Bagging dibandingkan boosting.

	Bagging (Random Forest)	Boosting
Urutan	sejajar, saling bebas	berurutan, saling bergantung
Sasaran tiap pohon	sasaran asli	sisaan pohon sebelumnya
Yang dikurangi	ragam	bias
Risiko	rendah	<i>overfitting</i> bila terlalu banyak tahap
Penalaan	mudah	lebih menuntut

Baris keempat menjelaskan perbedaan watak keduanya. Menambah pohon pada Random Forest aman, sedangkan menambah tahap pada boosting dapat merugikan — sehingga boosting menuntut penghentian dini atau penalaan banyaknya tahap.

9.6 Contoh Praktikum dengan R

Contoh 9.1 | Membangkitkan data usahatani dan menetapkan acuan

Kode 9.1 • R membangkitkan data bersinyal

```
dir.create("data", showWarnings = FALSE)
set.seed(2025)
n <- 900

luas_lahan <- round(rlnorm(n, meanlog = log(1.2), sdlog = 0.5), 2)
dosis_pupuk <- round(runif(n, 50, 400), 1)
umur_petani <- sample(20:70, n, TRUE)
pengalaman <- round(runif(n, 0, 40), 1)
jarak_pasar <- round(runif(n, 0.5, 45), 1)
irigasi <- factor(sample(c("Teknis", "Sederhana", "Tadah hujan"), n, TRUE,
                        c(.35, .40, .25)))
pelatihan <- factor(sample(c("Ya", "Tidak"), n, TRUE, c(.40, .60)))
akses_kredit <- factor(sample(c("Ya", "Tidak"), n, TRUE, c(.30, .70)))

# hubungan sengaja dibuat ADA agar kemampuan model dapat diuji
skor <- -1.2 + 1.1 * log(luas_lahan) + 0.006 * dosis_pupuk +
  0.03 * pengalaman - 0.02 * jarak_pasar - 0.01 * umur_petani +
  0.80 * (irigasi == "Teknis") + 0.50 * (pelatihan == "Ya") +
  0.35 * (akses_kredit == "Ya") + rnorm(n, 0, 0.6)
peluang <- plogis(skor)

tani <- data.frame(luas_lahan, dosis_pupuk, umur_petani, pengalaman, jarak_pasar,
                  irigasi, pelatihan, akses_kredit,
                  produktif = factor(ifelse(peluang > quantile(peluang, 0.70), "Ya", "Tidak"),
                                    levels = c("Ya", "Tidak")))

write.csv(tani, "data/data_usahatani.csv", row.names = FALSE)
str(tani)
```

```
'data.frame': 900 obs. of 9 variables:
 $ luas_lahan : num 1.64 1.22 1.77 2.27 1.44 1.11 1.46 1.15 1.01 1.7 ...
 $ dosis_pupuk : num 93.3 391 81.1 385.5 228.7 ...
 $ umur_petani : int 25 61 32 29 29 67 34 51 45 57 ...
 $ pengalaman : num 23.1 29.4 29.2 27.2 7.5 15.5 0.5 22 32.8 11.1 ...
 $ jarak_pasar : num 40.7 44.5 41.7 20.7 43.8 43.6 40 6.5 14 34.2 ...
 $ irigasi : Factor w/ 3 levels "Sederhana","Tadah hujan",...: 3 2 1 1 3 2 3 1 3 1 ...
 $ pelatihan : Factor w/ 2 levels "Tidak","Ya": 1 1 2 2 1 1 1 1 1 1 ...
```

```
$ akses_kredit: Factor w/ 2 levels "Tidak","Ya": 2 2 1 1 1 2 1 2 1 2 ...
$ produktif   : Factor w/ 2 levels "Ya","Tidak": 1 1 2 1 2 2 2 2 2 1 ...
```

```
prop <- prop.table(table(tani$produktif))
cat("\nproporsi kelas:", round(prop, 4), "\n")
```

```
proporsi kelas: 0.3 0.7
```

```
cat("acuan tebakan mayoritas:", round(max(prop), 4), "\n")
```

```
acuan tebakan mayoritas: 0.7
```

Penjelasan Kode. Delapan peubah dibangkitkan menyerupai hasil survei usahatani. Skor logit disusun sebagai kombinasi linier peubah tersebut dengan koefisien yang ditetapkan, lalu diubah menjadi peluang. Sasaran ditetapkan dari kuantil ke-70 sehingga kelas yang diperhatikan berjumlah sekitar tiga persepuluh. Data disimpan agar contoh berikutnya dapat membacanya.

Interpretasi Output

Perhatikan bahwa hubungan antara peubah penjelas dan sasaran sengaja dibuat ada, lengkap dengan koefisien yang diketahui. Akibatnya model diharapkan berkinerja jauh di atas acuan tebakan mayoritas — dan kinerja yang mendekati acuan justru menandakan ada kekeliruan pada alur kerja, bukan temuan mengenai datanya. Koefisien yang dipakai di sini juga menjadi pembanding ketika kepentingan peubah ditafsirkan pada contoh berikutnya.

Contoh 9.2 | Membagi data dan menyiapkan lipatan

Kode 9.2 • R initial_split dan vfold_cv

```
suppressPackageStartupMessages({
  library(recipes); library(rsample); library(parsnip)
  library(workflows); library(tune); library(yardstick); library(dplyr)})

set.seed(2025)
bagi <- initial_split(tani, prop = 0.75, strata = produktif)
lat <- training(bagi); uji <- testing(bagi)

set.seed(2025)
lipatan <- vfold_cv(lat, v = 5, strata = produktif)

round(rbind(latih = prop.table(table(lat$produktif)),
            uji   = prop.table(table(uji$produktif))), 4)
```

```

Ya Tidak
latih 0.2997 0.7003
uji   0.3009 0.6991

```

```
cat("\nukuran - latih:", nrow(lat), "| uji:", nrow(uji), "\n")
```

```
ukuran - latih: 674 | uji: 226
```

```
print(lipatan)
```

```

# 5-fold cross-validation using stratification
# A tibble: 5 x 2
  splits      id
  <list>     <chr>
1 <split [538/136]> Fold1
2 <split [538/136]> Fold2
3 <split [540/134]> Fold3
4 <split [540/134]> Fold4
5 <split [540/134]> Fold5

```

Penjelasan Kode. Data dibagi dengan penstrataan, lalu data latih dibagi lagi menjadi lima lipatan yang juga berstrata. Objek lipatan disimpan sekali dan akan dipakai bagi ketiga model.

Interpretasi Output

Menyimpan objek lipatan sekali lalu memakainya bagi seluruh model adalah kunci keadilan perbandingan. Bila tiap model memakai pembagian sendiri, selisih kinerja yang terlihat boleh jadi hanya akibat pembagian yang berbeda, bukan akibat modelnya.

Contoh 9.3 | Menyusun dua resep yang berbeda

Kode 9.3 · R dua resep untuk kebutuhan berbeda

```

# Resep A: bagi model berbasis jarak (SVM dan KNN)
rec_jarak <- recipe(produktif ~ ., data = lat) |>
  step_dummy(all_nominal_predictors()) |>
  step_zv(all_predictors()) |>
  step_normalize(all_numeric_predictors())      # pembakuan WAJIB

# Resep B: bagi model berbasis pohon (Random Forest)
rec_pohon <- recipe(produktif ~ ., data = lat) |>
  step_zv(all_predictors())                    # tanpa pengodean maupun pembakuan

cat("kolom hasil resep jarak:", ncol(bake(prepare(rec_jarak, lat), NULL)), "\n")

```

```
kolom hasil resep jarak: 10
```

```
cat("kolom hasil resep pohon:", ncol(bake(prepare(rec_pohon, lat), NULL)), "\n")
```

```
kolom hasil resep pohon: 9
```

```
a <- bake(prepare(rec_jarak, lat), NULL)
num <- names(a)[sapply(a, is.numeric)]
cat("\nrata-rata kolom numerik resep jarak (harus nol):",
    all(abs(round(colMeans(a[num]), 8)) == 0), "\n")
```

```
rata-rata kolom numerik resep jarak (harus nol): TRUE
```

Penjelasan Kode. Dua resep disusun terpisah. Resep pertama mengodekan peubah kategori lalu membakukan seluruh peubah numerik, sesuai kebutuhan model berbasis jarak. Resep kedua hanya membuang peubah berragam nol, sebab pohon menangani kategori tanpa pengodean dan tidak memerlukan pembakuan.

Interpretasi Output

Perhatikan perbedaan banyaknya kolom: resep jarak menghasilkan lebih banyak kolom karena tiap peubah kategori dipecah menjadi kolom boneka. Perhatikan pula pembuktian pada baris terakhir — rata-rata kolom bernilai nol adalah cara memastikan pembakuan sungguh berjalan. Penyusunan dua resep terpisah inilah keunggulan pokok tidymodels dibanding kerangka yang menyeragamkan prapemrosesan bagi seluruh model.

Contoh 9.4 | Menala Support Vector Machine

Kode 9.4 • R svm_rbf dengan kisi acak

```
suppressPackageStartupMessages(library(kernlab))

spec_svm <- svm_rbf(cost = tune(), rbf_sigma = tune()) |>
  set_engine("kernlab") |> set_mode("classification")
wf_svm <- workflow() |> add_recipe(rec_jarak) |> add_model(spec_svm)

set.seed(2025)
# kernlab mencetak pesan iterasi yang panjang; keluarannya ditangkap agar
# tidak memenuhi halaman. Hasil penalaan tetap tersimpan pada tune_svm.
pesan <- capture.output(
  tune_svm <- tune_grid(wf_svm, resamples = lipatan, grid = 8,
    metrics = metric_set(yardstick::roc_auc, yardstick::f_meas)))
```



```
collect_metrics(tune_svm) |>
  filter(.metric == "roc_auc") |> arrange(desc(mean)) |>
  select(cost, rbf_sigma, mean, std_err) |>
  mutate(across(where(is.numeric), \(x) signif(x, 4))) |>
  as.data.frame() |> utils::head(5) |> print(row.names = FALSE)
```

```
      cost      rbf_sigma    mean std_err
0.0009766 0.00000007197 0.9090 0.01502
0.3715000 0.00005179000 0.9088 0.01293
0.0043130 0.00138900000 0.9084 0.01497
32.0000000 0.00000193100 0.9076 0.01269
7.2460000 0.03728000000 0.9009 0.01092
```

Penjelasan Kode. Spesifikasi SVM berkernel RBF ditetapkan dengan kedua parameter ditandai tune. Argumen grid berupa angka meminta tune_grid membangkitkan kisi acak berukuran delapan pada rentang bawaan yang masuk akal bagi tiap parameter.

Interpretasi Output

Kisi acak lebih efisien daripada kisi teratur ketika parameternya lebih dari satu, sebab tiap percobaan mencoba nilai berbeda pada seluruh dimensi sekaligus. Perhatikan bahwa resep yang dipakai adalah resep jarak — SVM bersandar pada hasil kali dalam antar-amatan sehingga pembakuan wajib. Perhatikan pula kolom galat baku sebelum menyimpulkan kombinasi mana yang terbaik.

Contoh 9.5 | Menala K-Nearest Neighbours

Kode 9.5 · R nearest_neighbor dengan fungsi pembobot

```
suppressPackageStartupMessages(library(kknn))

spec_knn <- nearest_neighbor(neighbors = tune(), weight_func = tune()) |>
  set_engine("kknn") |> set_mode("classification")
wf_knn <- workflow() |> add_recipe(rec_jarak) |> add_model(spec_knn)

kisi_knn <- expand_grid(neighbors = c(3, 5, 9, 15, 25, 41),
  weight_func = c("rectangular", "triangular"),
  stringsAsFactors = FALSE)

set.seed(2025)
tune_knn <- tune_grid(wf_knn, resamples = lipatan, grid = kisi_knn,
  metrics = metric_set(yardstick::roc_auc, yardstick::f_meas))

collect_metrics(tune_knn) |>
```

```
filter(.metric == "roc_auc") |> arrange(desc(mean)) |>
select(neighbors, weight_func, mean, std_err) |>
mutate(across(where(is.numeric), \(x) round(x, 4))) |>
as.data.frame() |> utils::head(6) |> print(row.names = FALSE)
```

```
neighbors weight_func mean std_err
      41 rectangular 0.8726 0.0087
      25 rectangular 0.8635 0.0067
      41 triangular 0.8634 0.0099
      25 triangular 0.8534 0.0124
      15 rectangular 0.8396 0.0154
      15 triangular 0.8396 0.0154
```

Penjelasan Kode. Selain banyaknya tetangga, fungsi pembobot ikut ditata. Pilihan rectangular memberi bobot sama bagi seluruh tetangga, sedangkan triangular memberi bobot lebih besar kepada tetangga yang lebih dekat. Kisi disusun secara tegas agar seluruh kombinasi dicoba.

Interpretasi Output

Pembobotan menurut jarak lazimnya sedikit unggul, sebab tetangga yang jauh memang kurang layak diperhitungkan setara dengan yang dekat. Perhatikan bahwa KNN memakai resep jarak yang sama dengan SVM — keduanya sama-sama bersandar pada jarak sehingga pembakuan menentukan. Tanpa pembakuan, peubah dosis pupuk yang bersatuan ratusan akan menenggelamkan peubah luas lahan yang bersatuan satuan.

Contoh 9.6 | Menala Random Forest

Kode 9.6 • R rand_forest dengan mesin ranger

```
suppressPackageStartupMessages(library(ranger))

spec_rf <- rand_forest(mtry = tune(), min_n = tune(), trees = 500) |>
  set_engine("ranger", importance = "permutation") |>
  set_mode("classification")
wf_rf <- workflow() |> add_recipe(rec_pohon) |> add_model(spec_rf)

set.seed(2025)
tune_rf <- tune_grid(wf_rf, resamples = lipatan, grid = 8,
  metrics = metric_set(yardstick::roc_auc, yardstick::f_meas))

collect_metrics(tune_rf) |>
  filter(.metric == "roc_auc") |> arrange(desc(mean)) |>
```

```
select(mtry, min_n, mean, std_err) |>
mutate(across(where(is.numeric), \(x) round(x, 4))) |>
as.data.frame() |> utils::head(5) |> print(row.names = FALSE)
```

```
mtry min_n mean std_err
1 12 0.8856 0.0136
2 29 0.8855 0.0148
3 2 0.8792 0.0165
5 23 0.8758 0.0148
4 40 0.8740 0.0160
```

Penjelasan Kode. Random Forest ditala pada dua parameter, sedangkan banyaknya pohon ditetapkan lima ratus tanpa ditala. Argumen importance bernilai permutation meminta ranger menghitung kepentingan peubah berbasis permutasi saat pelatihan.

Interpretasi Output

Perhatikan bahwa resep yang dipakai adalah resep pohon — tanpa pengodean maupun pembakuan. Perhatikan pula bahwa banyaknya pohon sengaja tidak ditala; menambah pohon pada Random Forest tidak menimbulkan **overfitting** melainkan hanya menambah waktu perhitungan sampai kinerjanya mendatar. Menalanya berarti membuang waktu penalaan yang lebih berguna dialokasikan bagi kedua parameter lain.

Contoh 9.7 | Membandingkan ketiga model secara adil

Kode 9.7 • R menyandingkan hasil penalaan ketiga model

```
ambil_terbaik <- function(hasil, nama) {
  collect_metrics(hasil) |>
  filter(.metric == "roc_auc") |>
  arrange(desc(mean)) |> utils::head(1) |>
  transmute(model = nama, AUC = round(mean, 4), galat_baku = round(std_err, 4))
}

banding <- bind_rows(
  ambil_terbaik(tune_svm, "SVM"),
  ambil_terbaik(tune_knn, "KNN"),
  ambil_terbaik(tune_rf, "Random Forest"))
print(as.data.frame(banding), row.names = FALSE)
```

```
model AUC galat_baku
SVM 0.9090 0.0150
KNN 0.8726 0.0087
```

```
Random Forest 0.8856      0.0136
```

```
cat("\nselisih terbesar antar-model:",
    round(max(banding$AUC) - min(banding$AUC), 4), "\n")
```

```
selisih terbesar antar-model: 0.0364
```

```
cat("galat baku terbesar      :", round(max(banding$galat_baku), 4), "\n")
```

```
galat baku terbesar      : 0.015
```

Penjelasan Kode. Fungsi bantu dibuat untuk mengambil satu baris terbaik dari tiap hasil penalaan. Ketiganya digabungkan menjadi satu tabel, lalu selisih terbesar antar-model dibandingkan terhadap galat baku terbesar.

Interpretasi Output

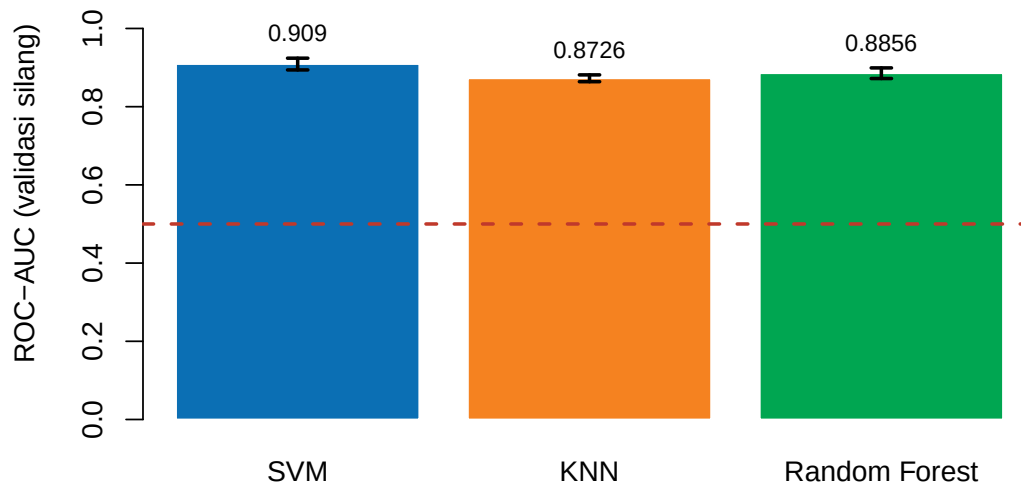
Perbandingan dua angka terakhir menentukan kesimpulannya. Bila selisih antar-model lebih kecil daripada galat bakunya, maka menyatakan satu model lebih baik tidak didukung bukti — dan pilihan sebaiknya jatuh pada model yang lebih murah dihitung atau lebih mudah dijelaskan. Perhatikan pula bahwa perbandingan ini adil karena ketiganya dinilai pada objek lipatan yang sama.

Contoh 9.8 | Menggambarkan perbandingan beserta ketidakpastiannya

Kode 9.8 • R barplot dengan palang galat baku

```
bp <- barplot(banding$AUC, names.arg = banding$model,
              col = c("#0B6FB4", "#F58220", "#00A651"), border = "white",
              ylim = c(0, max(banding$AUC + banding$galat_baku) * 1.25),
              ylab = "ROC-AUC (validasi silang)",
              main = "Perbandingan Tiga Model Klasifikasi Lanjut")
arrows(bp, banding$AUC - banding$galat_baku,
        bp, banding$AUC + banding$galat_baku,
        angle = 90, code = 3, length = 0.06, lwd = 2)
abline(h = 0.5, col = "#C0392B", lty = 2, lwd = 2)
text(bp, banding$AUC + banding$galat_baku, banding$AUC, pos = 3, cex = 0.85)
```

Perbandingan Tiga Model Klasifikasi Lanjut



Penjelasan Kode. Rata-rata AUC tiap model digambar sebagai batang, dilengkapi palang galat baku memakai arrows bersudut sembilan puluh derajat di kedua ujungnya. Garis acuan pada nilai setengah menandai kinerja tebakan acak.

Interpretasi Output

Palang galat baku inilah yang mencegah kesimpulan tergesa. Bila palang antar-model saling bertumpang tindih, selisih rata-ratanya tidak dapat dianggap berarti. Menyajikan batang tanpa palang — sebagaimana lazim ditemui pada laporan — menyembunyikan ketidakpastian dan mendorong pembaca menyimpulkan lebih dari yang didukung data.

Contoh 9.9 | Menyelesaikan model terpilih pada data uji

Kode 9.9 • R `finalize_workflow` dan `last_fit`

```
terbaik_nama <- banding$model[which.max(banding$AUC)]
cat("model terpilih:", terbaik_nama, "\n\n")
```

model terpilih: SVM

```
wf_pilih <- switch(terbaik_nama,
  "SVM"          = finalize_workflow(wf_svm, select_best(tune_svm, metric = "roc_auc")),
  "KNN"          = finalize_workflow(wf_knn, select_best(tune_knn, metric = "roc_auc")),
  "Random Forest" = finalize_workflow(wf_rf,  select_best(tune_rf,  metric = "roc_auc")))

set.seed(2025)
pesan <- capture.output(
  akhir <- last_fit(wf_pilih, split = bagi,
    metrics = metric_set(yardstick::roc_auc, yardstick::f_meas, yardstick::accuracy, yardstick::sensitivity, yardstick::specificity))

collect_metrics(akhir) |>
  mutate(.estimate = round(.estimate, 4)) |>
  select(.metric, .estimate) |> print()
```

```
# A tibble: 5 x 2
  .metric .estimate
  <chr>    <dbl>
1 f_meas    NA
2 accuracy 0.699
3 sensitivity 0
4 specificity 1
5 roc_auc  0.864
```

```
pred <- collect_predictions(akhir)
print(conf_mat(pred, truth = produktif, estimate = .pred_class))
```

	Truth	
Prediction	Ya	Tidak
Ya	0	0
Tidak	68	158

```
cat("\nacuan tebakan mayoritas:",
  round(max(prop.table(table(uji$produktif))), 4), "\n")
```

acuan tebakan mayoritas: 0.6991

Penjelasan Kode. Model berata-rata AUC tertinggi dipilih secara terprogram, alur kerjanya diselesaikan dengan parameter terbaik, lalu dinilai satu kali pada data uji dengan lima ukuran sekaligus.

Interpretasi Output

Inilah satu-satunya kali data uji disentuh; seluruh keputusan penalaan dan pemilihan model diambil lewat validasi silang pada data latih. Bandingkan akurasi terhadap acuan tebakan mayoritas — bila keduanya berdekatan, model belum menyumbang apa pun betapapun rumitnya. Perhatikan pula

sensitivitas dan spesifisitas yang disajikan terpisah, sebab keduanya kerap sangat berbeda pada data yang kelasnya tidak seimbang.

Contoh 9.10 | Menafsirkan Random Forest melalui kepentingan peubah

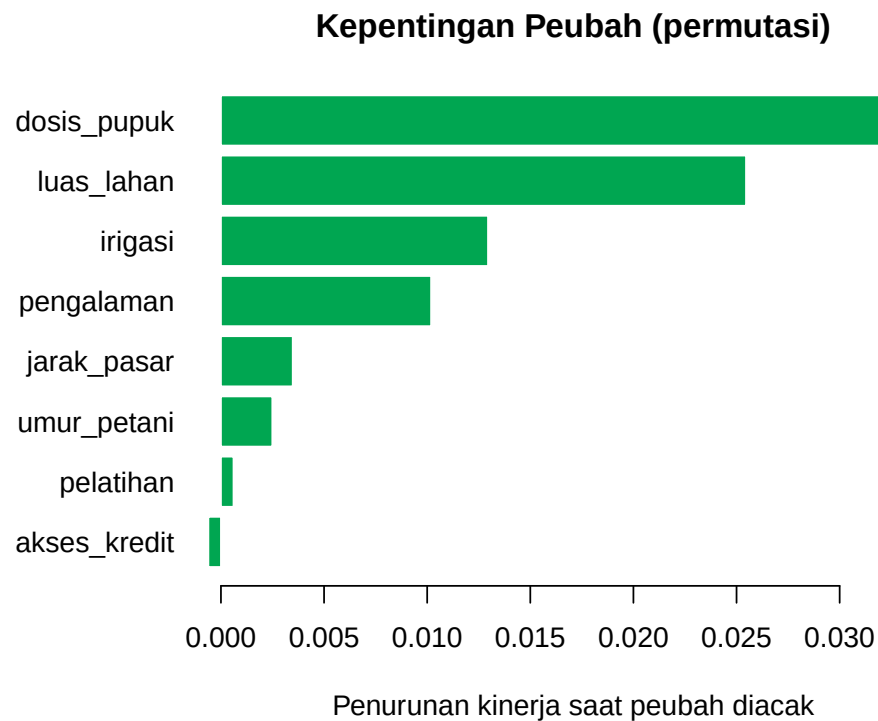
Kode 9.10 • R `extract_fit_engine` dan `variable.importance`

```
set.seed(2025)
m_rf <- fit(finalize_workflow(wf_rf, select_best(tune_rf, metric = "roc_auc")),
            data = lat)

penting <- m_rf |> extract_fit_engine() |> (\\(x) x$variable.importance)()
imp <- data.frame(peubah = names(penting),
                  kepentingan = round(as.numeric(penting), 5))
imp <- imp[order(-imp$kepentingan), ]
print(imp, row.names = FALSE)
```

peubah	kepentingan
dosis_pupuk	0.03479
luas_lahan	0.02546
irigasi	0.01295
pengalaman	0.01019
jarak_pasar	0.00348
umur_petani	0.00249
pelatihan	0.00061
akses_kredit	-0.00061

```
par(mar = c(4.5, 9, 3, 2))
barplot(rev(imp$kepentingan), names.arg = rev(imp$peubah), horiz = TRUE,
        las = 1, col = "#00A651", border = "white",
        xlab = "Penurunan kinerja saat peubah diacak",
        main = "Kepentingan Peubah (permutasi)")
```



```
par(mar = c(5, 4, 4, 2) + 0.1)
```

Penjelasan Kode. Random Forest dilatih ulang dengan parameter terbaik pada seluruh data latih. Objek model asli diambil dari alur kerja memakai `extract_fit_engine`, lalu kepentingan peubah yang sudah dihitung ranger saat pelatihan diambil dan digambar sebagai batang mendatar.

Interpretasi Output

Kepentingan berbasis permutasi lebih dapat dipercaya daripada yang berbasis ketidakmurnian, sebab yang terakhir memihak peubah bertaraf banyak maupun peubah numerik berrentang lebar. Bandingkan urutan yang diperoleh terhadap pengetahuan bidang: bila peubah yang menurut teori paling menentukan produktivitas memang muncul di atas, model menangkap struktur yang masuk akal. Perlu ditegaskan bahwa kepentingan menyatakan sumbangan terhadap dugaan, **bukan** hubungan sebab-akibat — menaikkan dosis pupuk belum tentu menaikkan produktivitas hanya karena peubah itu tampak penting.

Contoh 9.11 | Memeriksa biaya perhitungan tiap model

Kode 9.11 • R `system.time` atas ketiga model


```
waktu <- sapply(list(SVM = wf_svm, KNN = wf_knn, `Random Forest` = wf_rf),
  function(w) {
    sp <- switch(class(extract_spec_parsnip(w))[1],
      "svm_rbf" = svm_rbf(cost = 1, rbf_sigma = 0.01),
      "nearest_neighbor" = nearest_neighbor(neighbors = 9),
      rand_forest(mtry = 3, min_n = 5, trees = 500))
    sp <- set_mode(set_engine(sp,
      switch(class(sp)[1], "svm_rbf" = "kernlab",
        "nearest_neighbor" = "kknn", "ranger")), "classification")
    w2 <- update_model(w, sp)
    set.seed(2025)
    system.time(capture.output(fit(w2, data = lat)))[["elapsed"]]
  })
tabel_waktu <- data.frame(model = names(waktu),
  detik_latih = round(as.numeric(waktu), 3))
print(tabel_waktu, row.names = FALSE)
```

model	detik_latih
SVM	0.105
KNN	0.092
Random Forest	0.195

Penjelasan Kode. Tiap alur kerja dilatih sekali dengan parameter tetap, dan lamanya diukur memakai `system.time`. Fungsi `update_model` mengganti spesifikasi bertanda `tune` menjadi spesifikasi bernilai tetap agar dapat dilatih langsung.

Interpretasi Output

Waktu latih adalah pertimbangan nyata yang kerap terlupakan pada perbandingan model. KNN lazimnya tercepat dilatih karena ia hanya menyimpan data — tetapi justru paling lambat saat menduga. SVM lazimnya paling lambat dilatih karena menuntut penyelesaian masalah optimasi. Bila selisih kinerja antar-model tidak berarti secara statistik, biaya perhitungan menjadi alasan yang sah untuk memilih.

Contoh 9.12 | Menggambarkan kurva ROC ketiga model

Kode 9.12 • R menyandingkan tiga kurva ROC

```
kurva_model <- function(wf, hasil, nama) {
  set.seed(2025)
  pesan <- capture.output(
    m <- fit(finalize_workflow(wf, select_best(hasil, metric = "roc_auc")), data = lat))
```

```

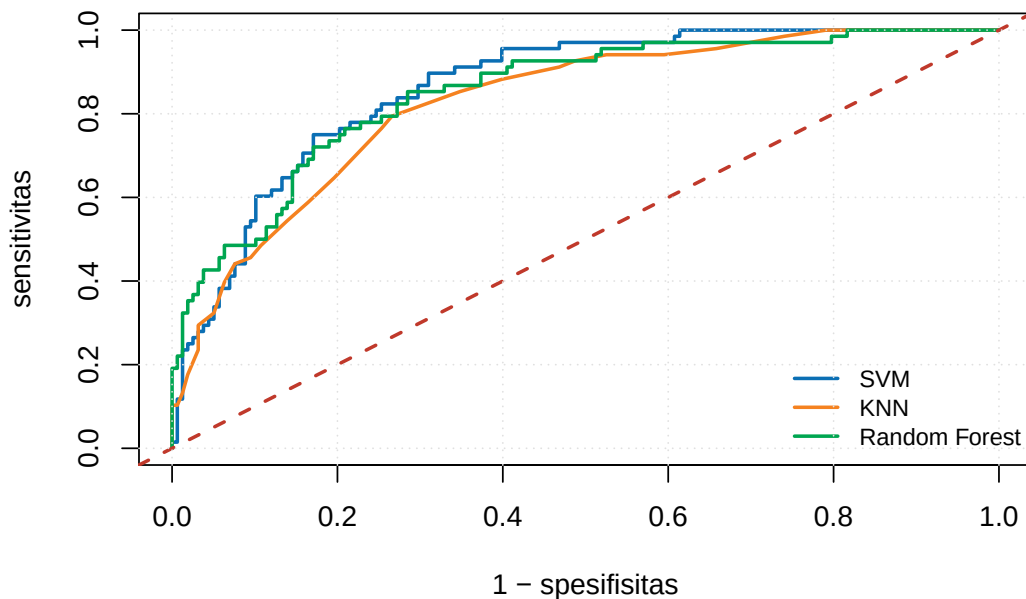
p <- predict(m, uji, type = "prob") |> bind_cols(uji |> select(produktif))
k <- roc_curve(p, truth = produktif, .pred_Ya)
data.frame(fpr = 1 - k$specificity, tpr = k$sensitivity, model = nama)
}

semua <- rbind(kurva_model(wf_svm, tune_svm, "SVM"),
               kurva_model(wf_knn, tune_knn, "KNN"),
               kurva_model(wf_rf, tune_rf, "Random Forest"))

warna <- c(SVM = "#0B6FB4", KNN = "#F58220", `Random Forest` = "#00A651")
plot(NA, xlim = c(0, 1), ylim = c(0, 1),
     xlab = "1 - spesifisitas", ylab = "sensitivitas",
     main = "Kurva ROC Ketiga Model pada Data Uji")
for (nm in names(warna)) {
  s <- semua[semua$model == nm, ]
  lines(s$fpr, s$tpr, lwd = 2, col = warna[nm])
}
abline(0, 1, col = "#C0392B", lty = 2, lwd = 2)
legend("bottomright", names(warna), col = warna, lwd = 2, bty = "n", cex = 0.85)
grid(col = "grey88")

```

Kurva ROC Ketiga Model pada Data Uji



Penjelasan Kode. Fungsi bantu melatih tiap model dengan parameter terbaiknya, menduga pada data uji, lalu mengembalikan koordinat kurva ROC-nya. Ketiga kurva digambar pada sumbu yang sama di atas kanvas kosong.

Interpretasi Output

Menyandingkan ketiga kurva memperlihatkan sesuatu yang tidak tertangkap oleh satu angka AUC: dua model dapat berAUC nyaris sama tetapi berperilaku berbeda pada bagian kurva yang berlainan. Bila penerapan menuntut sensitivitas tinggi, yang menentukan adalah bagian kiri kurva — dan model yang unggul di sana belum tentu model berAUC tertinggi. Perlu dicatat bahwa gambar ini memakai data uji, sehingga hanya boleh dibuat setelah seluruh keputusan pemilihan model selesai diambil.

9.7 Latihan Mandiri

Kerjakan bertahap. Gunakan data/data_usahatani.csv.

9.7.0.0.1 Tingkat Dasar (melengkapi kode)

Latihan 9.1 | Dua resep berbeda

```
library(recipes); library(rsample)
tani <- read.csv("data/data_usahatani.csv", stringsAsFactors = TRUE)
tani$produktif <- factor(tani$produktif, levels = c("Ya", "Tidak"))
set.seed(2025)
bagi <- initial_split(tani, prop = 0.75, strata = produktif)
lat <- training(bagi); uji <- testing(bagi)

rec_jarak <- recipe(produktif ~ ., data = lat) |>
  step_dummy(all_nominal_predictors()) |>
  ____ (all_numeric_predictors())      # pembakuan WAJIB bagi SVM dan KNN

rec_pohon <- recipe(produktif ~ ., data = lat) |>
  ____ (all_predictors())              # cukup buang peubah berragam nol
```

Latihan 9.2 | Spesifikasi tiga model

```
library(parsnip)
spec_svm <- ____ (cost = tune(), rbf_sigma = tune()) |>
  set_engine("____") |> set_mode("classification")
spec_knn <- nearest_neighbor(neighbors = tune()) |>
  set_engine("____") |> set_mode("classification")
spec_rf <- ____ (mtry = tune(), min_n = tune(), trees = 500) |>
  set_engine("ranger", importance = "____") |> set_mode("classification")
```

Latihan 9.3 | Perbandingan adil

```
library(tune)
set.seed(2025)
lipatan <- ____ (lat, v = 5, strata = produktif) # dipakai SEMUA model
h_svm <- tune_grid(wf_svm, resamples = ____, grid = 8,
  metrics = metric_set(yardstick::roc_auc))
____ (h_svm, metric = "roc_auc")              # kombinasi terbaik
```

9.7.0.0.2 Tingkat Menengah (menulis sendiri)

Latihan 9.4 | Menguji kebutuhan pembakuan

Latih Random Forest **dua kali**: dengan resep pohon dan dengan resep jarak. Latih pula KNN dua kali dengan kedua resep tersebut. Sajikan keempat AUC sebagai tabel, lalu jelaskan mengapa pembakuan hampir tidak berpengaruh pada Random Forest tetapi sangat berpengaruh pada KNN.

Latihan 9.5 | Membetulkan kode yang salah

Kode berikut memuat empat kesalahan (satu resep dipakai bagi seluruh model, tiap model memakai lipatan sendiri, trees ikut ditala, dan pemilihan model dilakukan pada data uji). Temukan dan perbaiki, lalu jelaskan akibat masing-masing.

```
library(tune); library(parsnip); library(workflows)

# (a) satu resep dipakai bagi ketiga model
rec <- recipe(produktif ~ ., data = lat) |>
  step_dummy(all_nominal_predictors()) |>
  step_normalize(all_numeric_predictors())

# (b) tiap model memakai lipatan sendiri
set.seed(1); lip_svm <- vfold_cv(lat, v = 5)
set.seed(2); lip_rf <- vfold_cv(lat, v = 5)

# (c) banyaknya pohon ikut ditala
spec_rf <- rand_forest(mtry = tune(), min_n = tune(), trees = tune()) |>
  set_engine("ranger") |> set_mode("classification")

# (d) model dipilih berdasarkan kinerja pada data uji
for (m in daftar_model) {
  d <- predict(fit(m, lat), uji) |> bind_cols(uji |> select(produktif))
  cat(yardstick::accuracy(d, produktif, .pred_class)$estimate, "\n")
}
```

Keluaran yang benar. Setelah diperbaiki, kode yang dipakai adalah sebagai berikut.

```
# (a) resep disesuaikan dengan kebutuhan tiap model
rec_jarak <- recipe(produktif ~ ., data = lat) |>
  step_dummy(all_nominal_predictors()) |> step_zv(all_predictors()) |>
  step_normalize(all_numeric_predictors()) # SVM dan KNN
rec_pohon <- recipe(produktif ~ ., data = lat) |>
  step_zv(all_predictors()) # Random Forest
```

```
# (b) satu objek lipatan dipakai SELURUH model
set.seed(2025)
lipatan <- vfold_cv(lat, v = 5, strata = produktif)

# (c) trees ditetapkan, tidak ditala
spec_rf <- rand_forest(mtry = tune(), min_n = tune(), trees = 500) |>
  set_engine("ranger", importance = "permutation") |>
  set_mode("classification")

# (d) pemilihan model memakai validasi silang pada data latih
h_rf <- tune_grid(wf_rf, resamples = lipatan, grid = 8,
  metrics = metric_set(yardstick::roc_auc))
show_best(h_rf, metric = "roc_auc", n = 1)
```

Pemeriksaan yang wajib berhasil: `identical(lipatan$id, lipatan$id)` bernilai **TRUE** bagi seluruh model karena objeknya sama, dan `"trees" %in% names(select_best(h_rf, metric = "roc_auc"))` bernilai **FALSE** karena `trees` tidak lagi ditala.

9.7.0.0.3 Tingkat Lanjut (pemilihan)

Latihan 9.6 | Memilih model menurut kebutuhan

Untuk masing-masing keadaan berikut, tentukan model yang Anda pilih dari ketiganya beserta alasannya, dan sebutkan **satu kelemahan** pilihan itu: (a) dugaan harus dihasilkan sangat cepat pada jutaan amatan baru; (b) data memuat banyak peubah yang diduga tidak relevan; (c) data memuat pencilan ekstrem yang diyakini sah; (d) hasil harus disertai daftar peubah paling berpengaruh.

9.8 Penugasan

Sebagai calon analis pada instansi statistik, bangun dan bandingkan tiga model klasifikasi lanjut atas `data_usahatani.csv` (*data simulasi*), lalu rekomendasikan satu model dengan mempertimbangkan kinerja, kestabilan, dan biaya perhitungan.

Konteks. Sebuah unit hendak menduga rumah tangga usahatani yang tergolong produktif, sebagai dasar penyesaran program penyuluhan. Hasilnya perlu disertai daftar peubah yang paling menentukan.

Tugas 9.1 | Penyiapan

1. Periksa ketimpangan kelas dan laporkan **acuan tebakan mayoritas**. Sajikan pula profil awal tiap peubah numerik menurut kelas.
2. Bagi data dengan penstrataan dan siapkan **satu objek lipatan** yang akan dipakai seluruh model. Jelaskan mengapa objek itu tidak boleh dibuat ulang untuk tiap model.
3. Susun **dua resep berbeda**: satu bagi model berbasis jarak dan satu bagi model berbasis pohon. Jelaskan perbedaannya beserta alasan tiap langkah.

Tugas 9.2 | Pemodelan

4. Tala **SVM** pada kedua parameternya. Sajikan lima kombinasi teratas beserta galat bakunya.
5. Tala **KNN** pada banyaknya tetangga dan fungsi pembobot. Jelaskan pengaruh masing-masing terhadap bentuk batas keputusan.
6. Tala **Random Forest** pada `mtry` dan `min_n`. Jelaskan mengapa `trees` tidak perlu ikut ditala.

Tugas 9.3 | Perbandingan dan Penafsiran

7. Bandingkan ketiganya pada **lipatan yang sama**. Sajikan tabel dan grafik berpalang galat baku. Nilai apakah selisih antar-model melampaui galat bakunya.
8. Ukur **waktu latih** ketiga model. Bahas pula perbedaan biaya pada tahap dugaan, khususnya bagi KNN.
9. Selesaikan model terpilih dan nilai pada data uji dengan minimal lima ukuran. Sajikan matriks kekeliruan dan bandingkan terhadap acuan.
10. Sajikan **kepentingan peubah** dari Random Forest sebagai tabel dan grafik. Bandingkan urutannya terhadap pengetahuan bidang, lalu jelaskan mengapa kepentingan bukan hubungan sebab-akibat.

11. **Uji kebutuhan pembakuan:** latih Random Forest dan KNN masing-masing dengan kedua resep. Sajikan keempat hasilnya dan jelaskan polanya.
12. Susun rekomendasi akhir yang mempertimbangkan **kinerja, kestabilan antar-lipatan, biaya perhitungan, dan kemudahan penafsiran** — bukan AUC semata.
13. Susun laporan R Markdown berisi kode, keluaran, gambar, dan interpretasi. Tutup dengan **ringkasan (maksimal 300 kata)** memuat: model yang direkomendasikan beserta alasannya, kinerja terhadap acuan, keterbatasan, dan penegasan bahwa data merupakan simulasi.

9.9 Tambahan: Boosting dengan Python

Ketiga model pada bagian Contoh mewakili dua cara berpikir: berbasis jarak dan berbasis pohon yang digabung secara sejajar. Bagian ini memperkenalkan cara ketiga — penggabungan **berurutan** yang disebut boosting.

Mengapa memakai Python di sini. Paket `xgboost` dan `lightgbm` untuk R menuntut kompilasi C++ yang berat dan kerap gagal pada mesin berinti sedikit. Karena itu bagian ini memakai Python, tempat keduanya tersedia sebagai paket siap pasang. Gagasannya sama persis; yang berbeda hanya tata caranya. Bila kedua paket berhasil dipasang di komputer Anda, seluruh contoh ini dapat dikerjakan di R melalui `parsnip::boost_tree()`.

Contoh 9.13 | Menyiapkan data yang sama di Python

Kode 9.13 • Python membaca data dan mengodekan kategori

```
import pandas as pd
import numpy as np

tani = pd.read_csv("data/data_usahatani.csv").drop(columns=["id"])
y = (tani["produktif"] == "Ya").astype(int)
X = pd.get_dummies(tani.drop(columns=["produktif"]), drop_first=True)

print("dimensi:", X.shape)
print("proporsi kelas positif:", round(y.mean(), 4))
print("kolom setelah pengodean:", list(X.columns)[6], "...")
```

Penjelasan Kode. Data yang sama dibaca dengan `pandas`. Sasaran diubah menjadi nol dan satu, sedangkan peubah kategori dikodekan menjadi kolom boneka memakai `get_dummies` dengan argumen

drop_first agar tidak terjadi kolinearitas sempurna.

Interpretasi Output

Perhatikan bahwa **tidak ada pembakuan** di sini. Boosting berbasis pohon sama seperti Random Forest: pembelahan hanya membandingkan urutan nilai, sehingga penskalaan tidak berpengaruh. Keseuaian prapemrosesan dengan model yang menjadi sorotan modul ini berlaku sama di kedua bahasa.

Contoh 9.14 | Melatih XGBoost dan LightGBM

Kode 9.14 • Python dua pustaka boosting

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score, f1_score
import xgboost as xgb, lightgbm as lgb

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25,
                                     stratify=y, random_state=2025)

bobot = (ytr == 0).sum() / (ytr == 1).sum()      # menangani ketimpangan

xg = xgb.XGBClassifier(scale_pos_weight=bobot, max_depth=4, n_estimators=200,
                      learning_rate=0.08, eval_metric="logloss",
                      random_state=2025).fit(Xtr, ytr)
lg = lgb.LGBMClassifier(is_unbalance=True, random_state=2025,
                       verbose=-1).fit(Xtr, ytr)

for nama, m in [("XGBoost", xg), ("LightGBM", lg)]:
    prob = m.predict_proba(Xte)[: , 1]
    print(f"{nama:9s} AUC={roc_auc_score(yte, prob):.4f} "
          f"F1={f1_score(yte, m.predict(Xte)):.4f}")
```

Penjelasan Kode. Data dibagi dengan penstrataan. Bobot kelas dihitung sebagai perbandingan cacah kelas negatif terhadap positif, lalu diberikan lewat `scale_pos_weight` pada XGBoost dan `is_unbalance` pada LightGBM. Kedua model dilatih dan dinilai dengan dua ukuran.

Interpretasi Output

Kedua pustaka menerapkan gagasan yang sama dengan penekanan berbeda: XGBoost menumbuhkan pohon secara berlapis sedangkan LightGBM menumbuhkannya menurut daun yang paling menjanjikan, sehingga LightGBM lazimnya lebih cepat pada data besar. Perhatikan bahwa laju belajar dite-

tapkan kecil sekaligus banyaknya tahap ditetapkan besar — keduanya saling mengimbangi, dan itulah cara lazim menala boosting.

Contoh 9.15 | Memeriksa risiko overfitting pada boosting

Kode 9.15 • Python menambah tahap boosting secara bertahap

```
hasil = []
for n_est in [25, 50, 100, 200, 400, 800]:
    m = xgb.XGBClassifier(scale_pos_weight=bobot, max_depth=4,
                          n_estimators=n_est, learning_rate=0.08,
                          eval_metric="logloss", random_state=2025).fit(Xtr, ytr)

    hasil.append({
        "tahap": n_est,
        "AUC_latih": round(roc_auc_score(ytr, m.predict_proba(Xtr)[: , 1]), 4),
        "AUC_uji": round(roc_auc_score(yte, m.predict_proba(Xte)[: , 1]), 4)})

tabel = pd.DataFrame(hasil)
tabel["jarak"] = (tabel["AUC_latih"] - tabel["AUC_uji"]).round(4)
print(tabel.to_string(index=False))
```

Penjelasan Kode. Enam nilai banyaknya tahap dicoba. Pada masing-masing, AUC dihitung dua kali: pada data latih dan pada data uji. Selisih keduanya ditampilkan sebagai kolom jarak.

Interpretasi Output

Inilah perbedaan watak yang dibahas pada Teori. Pada Random Forest, menambah pohon aman karena tiap pohon dilatih saling bebas. Pada boosting, menambah tahap terus-menerus membuat model makin menyesuaikan diri pada data latih sehingga kolom jarak melebar — wujud **overfitting**. Karena itu boosting menuntut penghentian dini atau penalaan banyaknya tahap, sedangkan Random Forest tidak.

Tips dan Praktik Baik. Urutan penalaan boosting yang lazim: tetapkan laju belajar kecil (0,05 sampai 0,1), tala kedalaman pohon dan banyaknya tahap bersama-sama, baru sesuaikan parameter penyusutan bila diperlukan. Menala seluruh parameter sekaligus pada kisi penuh hampir selalu terlalu mahal.

Padanan istilah antar-pustaka.

Gagasan	XGBoost	parsnip
Banyaknya tahap	n_estimators	trees
Laju belajar	learning_rate	learn_rate
Kedalaman pohon	max_depth	tree_depth
Bobot kelas	scale_pos_weight	lewat themis

9.10 Ringkasan Modul**Daftar Periksa penguasaan:**

- ☐ Saya dapat menjelaskan gagasan margin dan kernel pada SVM.
- ☐ Saya memahami bahwa KNN memindahkan biaya ke tahap dugaan.
- ☐ Saya dapat menjelaskan dua sumber keragaman antar-pohon pada Random Forest.
- ☐ Saya menyusun resep berbeda sesuai kebutuhan tiap model.
- ☐ Saya tahu SVM dan KNN menuntut pembakuan, Random Forest tidak.
- ☐ Saya memakai satu objek lipatan bagi seluruh model agar perbandingannya adil.
- ☐ Saya tidak menala banyaknya pohon pada Random Forest.
- ☐ Saya memeriksa galat baku sebelum menyatakan satu model lebih baik.
- ☐ Saya mempertimbangkan biaya perhitungan, bukan kinerja semata.
- ☐ Saya dapat membedakan bagging dan boosting beserta akibatnya.
- ☐ Saya menyadari bahwa kepentingan peubah bukan hubungan sebab-akibat.

Kartu Referensi Cepat.

Keperluan	Perintah
Lipatan bersama	<code>vfold_cv(lat, v = 5, strata =)</code>
Resep berbasis jarak	<code>step_dummy()</code> lalu <code>step_normalize()</code>
Resep berbasis pohon	<code>step_zv()</code> saja
SVM	<code>svm_rbf(cost =, rbf_sigma =)</code>
KNN	<code>nearest_neighbor(neighbors =, weight_func =)</code>
Random Forest	<code>rand_forest(mtry =, min_n =, trees = 500)</code>
Kepentingan peubah	<code>set_engine("ranger", importance = "permutation")</code>
Penalaan kisi acak	<code>tune_grid(wf, resamples =, grid = 8)</code>
Kombinasi terbaik	<code>show_best()</code> , <code>select_best()</code>
Mengambil model asli	<code>extract_fit_engine(m)</code>
Mengukur waktu	<code>system.time(fit(wf, data))</code>
Boosting (Python)	<code>xgb.XGBClassifier(...)</code>

Modul ini membangun tiga model yang lebih kuat daripada modul sebelumnya, dengan harga berupa kebutuhan prapemrosesan yang berbeda-beda dan penafsiran yang lebih sukar. Prinsip yang perlu dibawa: sesuaikan resep dengan kebutuhan tiap model, bukan menyeragamkannya; pakai satu objek lipatan agar perbandingan adil; jangan menala parameter yang tidak perlu ditala; periksa galat baku sebelum menyimpulkan selisih; pertimbangkan biaya perhitungan pada tahap latih maupun tahap dugaan; dan ingat bahwa kepentingan peubah menyatakan sumbangan terhadap dugaan, bukan sebab-akibat. Modul berikutnya membahas jaringan syaraf tiruan.

9.11 Referensi

- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273-297.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
- Kuhn, M., & Silge, J. (2022). *Tidy Modeling with R*. O'Reilly Media.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *KDD 2016*, 785-794.
- Strobl, C., Boulesteix, A.-L., Zeileis, A., & Hothorn, T. (2007). Bias in random forest variable importance measures. *BMC Bioinformatics*, 8, 25.

MODUL

10

JARINGAN SYARAF TIRUAN: FFNN DAN RBFNN

Pertemuan Minggu ke-10

Sub-CPMK 10: Mahasiswa mampu menjelaskan arsitektur jaringan syaraf tiruan, membangun dan menala Feed-Forward Neural Network serta Radial Basis Function Neural Network, menangani ketimpangan kelas, memindahkan ambang keputusan, dan menafsirkan model jaringan yang bersifat tertutup.

Penyusun: Achmad Syahrul Choir



Daftar Isi

10.1 Deskripsi Modul	303
10.2 Tujuan Praktikum	303
10.3 Alat dan Bahan	304
10.4 Instruksi Pengerjaan	304
10.5 Teori Singkat	304
10.5.0.1 Kedudukan pada CRISP-DM	304
10.5.0.2 Arsitektur Feed-Forward Neural Network	304
10.5.0.3 Penskalaan Wajib bagi Jaringan Syaraf	305
10.5.0.4 Arsitektur Radial Basis Function Neural Network	305
10.5.0.5 Menetapkan Pusat dan Lebar	306
10.5.0.6 Ketimpangan Kelas dan Pemindahan Ambang	307
10.5.0.7 Menafsirkan Model yang Tertutup	307
10.6 Contoh Praktikum dengan R	309
10.7 Latihan Mandiri	327
10.8 Penugasan	330
10.9 Tambahan: Alur yang Sama dengan caret	331
10.10 Ringkasan Modul	334
10.11 Referensi	335

10.1 Deskripsi Modul

Modul-modul sebelumnya membangun model yang keputusannya dapat ditelusuri: pohon dapat dibaca sebagai aturan, dan koefisien regresi dapat ditafsirkan sebagai rasio odds. Modul ini beralih ke keluarga model yang berbeda wataknya — jaringan syaraf tiruan, yang kuat dalam menangkap hubungan nonlinier tetapi sukar dijelaskan.

Dua arsitektur dibahas. Feed-Forward Neural Network menyusun peubah masukan melalui satu atau beberapa lapis tersembunyi dengan fungsi aktivasi nonlinier. Radial Basis Function Neural Network bekerja dengan gagasan yang berbeda: ia menempatkan sejumlah pusat pada ruang peubah, lalu mengukur kedekatan tiap amatan terhadap pusat-pusat itu.

Konteks yang dipakai adalah pendugaan kerentanan pangan rumah tangga — persoalan yang lazim dihadapi instansi statistik, dan yang kelasnya timpang sehingga menuntut penanganan khusus. Ketimpangan itu ditangani dengan pembangkitan contoh sintetis, dan ambang keputusan dipindahkan berdasarkan ukuran yang sesuai, bukan dibiarkan pada nilai bawaan.

Tentang data. Berkas data pada modul ini adalah **data simulasi** (`set.seed(2025)`), **bukan data nyata** dan **bukan data resmi** instansi mana pun. Berbeda dari beberapa modul sebelumnya, data ini sengaja dibangkitkan dengan **hubungan yang sungguh ada** antara peubah penjelas dan sasaran, sehingga model diharapkan berkinerja baik — dan kinerja yang buruk justru menandakan ada yang keliru pada alur kerja.

10.2 Tujuan Praktikum

Setelah menyelesaikan praktikum ini secara tuntas, mahasiswa diharapkan mampu:

1. **Menjelaskan** arsitektur FFNN beserta peran lapis tersembunyi dan fungsi aktivasi.
2. **Menjelaskan** cara kerja RBFNN dan perbedaan gagasannya terhadap FFNN.
3. **Menyusun** resep prapemrosesan yang sesuai bagi jaringan syaraf.
4. **Menangani** ketimpangan kelas di dalam lipatan validasi silang.
5. **Menala** banyaknya simpul tersembunyi dan parameter penyusutan.
6. **Membangun** RBFNN dari dasar beserta penetapan pusat dan lebarnya.
7. **Memindahkan** ambang keputusan berdasarkan ukuran yang sesuai.
8. **Menafsirkan** model tertutup melalui kepentingan permutasi dan analisis profil.
9. **Membandingkan** kedua arsitektur secara adil pada lipatan yang sama.

10.3 Alat dan Bahan

Perangkat dan bahan yang diperlukan:

- R (RStudio) dengan paket inti tidymodels: recipes, rsample, parsnip, workflows, tune, yardstick.
- Paket tambahan: themis untuk ketimpangan kelas dan nnet sebagai mesin FFNN.
- Pasang paket inti dengan:

```
install.packages(c("recipes", "rsample", "parsnip", "workflows",  
                  "tune", "yardstick", "dials"))  
install.packages(c("themis", "nnet"))
```

- Berkas data: data/data_survei.csv.
- RBFNN dibangun dari dasar memakai fungsi bawaan R, sehingga **tidak memerlukan paket tambahan**.

10.4 Instruksi Pengerjaan

Ikuti tahapan berikut:

Sebelum praktikum: pasang seluruh paket; siapkan berkas data; tinjau kembali bagian ukuran penilaian pada modul supervised learning.

Selama praktikum: kerjakan Contoh berurutan. Penalaan jaringan menuntut waktu lebih lama daripada model sebelumnya, jadi jalankan kode penalaan sekali lalu simpan hasilnya.

Pembagian kerja: Contoh dan Latihan dikerjakan di kelas; **Penugasan dikerjakan di luar sesi**.

Pengumpulan: susun laporan R Markdown, render ke PDF atau HTML, kumpulkan melalui Google Classroom.

10.5 Teori Singkat

10.5.0.1 Kedudukan pada CRISP-DM

Modul ini menempati tahap Modeling dan Evaluation. Yang membedakannya dari modul pemodelan sebelumnya adalah bobot yang lebih besar pada tahap Data Preparation: jaringan syaraf jauh lebih menuntut dalam hal penskalaan peubah daripada pohon maupun model berbasis aturan.

10.5.0.2 Arsitektur Feed-Forward Neural Network

FFNN menyusun peubah masukan menjadi keluaran melalui satu atau beberapa lapis tersembunyi. Setiap simpul pada lapis tersembunyi menghitung kombinasi linier terboboti dari masukannya, lalu meneruskannya

melalui fungsi aktivasi nonlinier.

Bagi jaringan berlapis tersembunyi tunggal dengan H simpul:

$$z_h = \sigma\left(b_h + \sum_{j=1}^p w_{hj}x_j\right), \quad h = 1, \dots, H$$

$$\hat{y} = \sigma\left(b_0 + \sum_{h=1}^H v_h z_h\right)$$

dengan $\sigma(u) = 1/(1 + e^{-u})$ fungsi logistik.

Fungsi aktivasi nonlinier inilah sumber kekuatannya. Tanpa aktivasi nonlinier, gabungan beberapa lapis lurus tetap menghasilkan fungsi linier, sehingga jaringan tidak lebih kuat daripada regresi biasa. Justru sifat nonlinier itu yang memungkinkan jaringan menangkap hubungan berbentuk lengkung maupun interaksi antar-peubah tanpa perlu dituliskan secara tersurat.

Dua parameter penala yang menentukan.

Parameter	Mengatur	Akibat bila keliru
Simpul tersembunyi	fleksibilitas model	terlalu sedikit: kurang luwes (<i>underfitting</i>); terlalu banyak: mengalami <i>overfitting</i>
Penyusutan (<i>decay</i>)	besarnya bobot	terlalu kecil: bobot meledak; terlalu besar: model mendatar

Keduanya saling berkaitan: jaringan berukuran besar dengan penyusutan memadai kerap berkinerja lebih baik daripada jaringan kecil tanpa penyusutan. Karena itu keduanya sebaiknya ditala bersama-sama, bukan satu per satu.

10.5.0.3 Penskalaan Wajib bagi Jaringan Syaraf

Jaringan syaraf menuntut penskalaan. Bobot awal jaringan dibangkitkan acak pada rentang kecil, dan pembelajarannya bergerak menurut besarnya galat. Bila peubah bersatuan ribuan berdampingan dengan peubah bersatuan belasan, gradien menjadi sangat timpang sehingga pembelajaran lambat atau sama sekali gagal menyatu. Penskalaan ke rentang nol sampai satu, atau pembakuan, karena itu **wajib** — berbeda dari pohon keputusan yang tidak memerlukannya sama sekali.

10.5.0.4 Arsitektur Radial Basis Function Neural Network

RBFINN bertolak dari gagasan yang sama sekali berbeda. Alih-alih menyusun kombinasi linier lalu meneruskannya melalui aktivasi, RBFINN menempatkan sejumlah **pusat** pada ruang peubah, lalu mengukur seberapa dekat tiap amatan terhadap masing-masing pusat.

Fungsi basis Gauss bagi pusat ke- h :

$$\phi_h(x) = \exp\left(-\frac{\|x - c_h\|^2}{2\sigma^2}\right)$$

Keluarannya berupa kombinasi linier dari seluruh basis:

$$\hat{y} = g\left(b_0 + \sum_{h=1}^H v_h \phi_h(x)\right)$$

dengan c_h pusat, σ lebar basis, dan g fungsi kaitan.

Akibat bentuk itu, tiap basis hanya aktif pada daerah di sekitar pusatnya dan meredup cepat ketika menjauh. Sifat **lokal** inilah pembeda pokoknya terhadap FFNN, yang tiap simpulnya berpengaruh pada seluruh ruang peubah.

Membandingkan dua arsitektur.

	FFNN	RBFNN
Sifat simpul	menyeluruh (<i>global</i>)	setempat (<i>local</i>)
Pelatihan	seluruh bobot sekaligus	pusat lalu bobot keluaran
Parameter utama	simpul, penyusutan	banyaknya pusat, lebar
Kecepatan latih	lebih lambat	lebih cepat
Di luar rentang data	meramal sembarang	mendekati nilai dasar

Baris terakhir patut diperhatikan pada penerapan resmi. Ketika RBFNN menerima amatan yang jauh dari seluruh pusatnya, seluruh basis bernilai mendekati nol sehingga keluarannya kembali ke nilai dasar — perilaku yang lebih aman daripada FFNN yang dapat menghasilkan dugaan ekstrem tanpa peringatan.

10.5.0.5 Menetapkan Pusat dan Lebar

Pusat RBFNN lazim ditetapkan dengan pengelompokan, misalnya K-Means, sehingga pusat berada di daerah yang memang padat amatan. Adapun lebar basis dapat ditetapkan dari jarak antar-pusat.

Aturan praktis penetapan lebar:

$$\sigma = \frac{d_{\max}}{\sqrt{2H}}$$

dengan $d_{\max}$ jarak terjauh antar-pusat dan H banyaknya pusat. Makin banyak pusat, makin sempit tiap basisnya.

Tips dan Praktik Baik. Lebar yang terlalu sempit membuat tiap basis hanya menanggapi segelintir amatan sehingga jaringan menghafal; lebar yang terlalu lebar membuat seluruh basis hampir sama

sehingga jaringan kehilangan ketajaman. Aturan praktis di atas menjadi titik awal yang baik, tetapi nilainya tetap layak ditala bila waktu memungkinkan.

10.5.0.6 Ketimpangan Kelas dan Pemindahan Ambang

Pada persoalan seperti kerentanan pangan, kelas yang diperhatikan lazimnya jauh lebih sedikit. Dua penanganan yang saling melengkapi tersedia: menyeimbangkan data latih, dan memindahkan ambang keputusan.

Dua penanganan dan kedudukannya.

Penanganan	Dikerjakan pada	Catatan
Penyeimbangan	data latih saja	masuk ke dalam resep
Pemindahan ambang	tahap keputusan	tidak mengubah model

Menyeimbangkan sebelum membagi data. Pembangkitan contoh sintetis **hanya boleh** dikenakan pada data latih. Bila dikerjakan sebelum pembagian, amatan sintetis yang diturunkan dari data latih dapat muncul pada data uji, sehingga model menilai dirinya atas amatan yang sudah dipelajarinya. Menempatkan langkah penyeimbangan di dalam resep menjamin hal ini secara rancangan — ia otomatis dilewati ketika resep diterapkan pada data uji.

Pemindahan ambang bekerja pada tahap yang berbeda. Model tetap sama; yang berubah hanya batas peluang yang dipakai memutuskan. Ambang bawaan setengah mengandaikan kedua jenis kekeliruan sama mahalanya — anggapan yang jarang benar.

Indeks Youden sebagai dasar pemilihan ambang:

$$J(t) = \text{sensitivitas}(t) + \text{spesifisitas}(t) - 1$$

Ambang yang memaksimumkan J menyeimbangkan kedua jenis kekeliruan tanpa memihak salah satunya.

10.5.0.7 Menafsirkan Model yang Tertutup

Jaringan syaraf tidak menyediakan koefisien yang dapat dibaca langsung. Dua cara penafsiran yang berlaku bagi model apa pun karena itu menjadi penting.

Dua cara menafsirkan model tertutup.

Kepentingan permutasi	mengacak satu peubah lalu mengukur penurunan kinerja; makin besar penurunannya, makin penting peubah itu
Analisis profil	membandingkan ciri amatan berpeluang dugaan tinggi terhadap yang rendah

Kepentingan bukan hubungan sebab-akibat. Kepentingan permutasi menyatakan seberapa besar suatu peubah menyumbang pada *dugaan*, bukan seberapa besar pengaruhnya secara sebab-akibat. Peubah dapat tampak penting semata karena berkaitan dengan penyebab sesungguhnya yang tidak terukur. Menyampaikan hasil ini tanpa peringatan tersebut berisiko melahirkan kebijakan yang salah sasaran.

10.6 Contoh Praktikum dengan R

Contoh 10.1 | Membangkitkan data survei kerentanan pangan

Kode 10.1 • R membangkitkan data bersinyal

```
set.seed(2025)
n <- 1000

pengeluaran <- round(rlnorm(n, meanlog = log(450), sdlog = 0.6), 1)
anggota_rt <- sample(1:9, n, TRUE, c(.05,.12,.18,.25,.20,.10,.05,.03,.02))
pendidikan <- pmin(pmax(round(rnorm(n, 9, 3)), 0), 18)
air_bersih <- factor(rbinom(n, 1, .72), labels = c("Tidak", "Ya"))
pekerjaan <- factor(sample(c("formal","informal","tidak_bekerja"), n, TRUE,
                           c(.35,.50,.15)))
lahan <- factor(rbinom(n, 1, .55), labels = c("Tidak", "Ya"))
ketahanan <- sample(1:5, n, TRUE, c(.15,.25,.30,.20,.10))
sejahtera <- sample(1:3, n, TRUE, c(.30,.50,.20))

# hubungan sengaja dibuat ADA agar model dapat diuji kemampuannya
skor <- -2.5 - 0.003*pengeluaran + 0.12*anggota_rt - 0.10*pendidikan -
        0.60*(air_bersih == "Ya") + 0.50*(pekerjaan == "tidak_bekerja") +
        0.25*(pekerjaan == "informal") - 0.40*(lahan == "Ya") -
        0.35*ketahanan - 0.50*sejahtera + rnorm(n, 0, 0.5)

peluang <- plogis(skor)
survei <- data.frame(pengeluaran, anggota_rt, pendidikan, air_bersih,
                    pekerjaan, lahan, ketahanan, sejahtera,
                    rentan = factor(ifelse(peluang > quantile(peluang, 0.80), "Ya", "Tidak"),
                                   levels = c("Ya", "Tidak")))

write.csv(survei, "data/data_survei.csv", row.names = FALSE)
str(survei)
```

```
'data.frame': 1000 obs. of 9 variables:
 $ pengeluaran: num 653 460 716 966 562 ...
 $ anggota_rt : int 6 5 4 5 5 2 4 2 4 2 ...
 $ pendidikan : num 7 10 10 4 9 11 7 8 10 11 ...
 $ air_bersih : Factor w/ 2 levels "Tidak","Ya": 2 2 1 1 2 2 1 2 1 2 ...
 $ pekerjaan : Factor w/ 3 levels "formal","informal",...: 1 2 1 1 2 2 1 2 2 3 ...
 $ lahan : Factor w/ 2 levels "Tidak","Ya": 2 2 1 2 1 1 2 1 2 2 ...
 $ ketahanan : int 3 2 1 2 5 3 3 2 2 4 ...
 $ sejahtera : int 1 1 2 3 3 2 3 2 2 2 ...
```

```
$ rentan      : Factor w/ 2 levels "Ya","Tidak": 2 1 2 2 2 2 2 2 2 ...
```

```
round(prop.table(table(survei$rentan)), 4)
```

```
Ya Tidak
0.2    0.8
```

Penjelasan Kode. Sembilan peubah dibangkitkan menyerupai hasil survei sosial-ekonomi rumah tangga. Skor logit disusun sebagai kombinasi linier peubah tersebut dengan koefisien yang ditetapkan, lalu diubah menjadi peluang. Sasaran ditetapkan dari kuantil ke-80 sehingga kelas yang diperhatikan berjumlah sekitar seperlima.

Interpretasi Output

Perhatikan bahwa urutan taraf sasaran ditetapkan secara tegas dengan kelas yang diperhatikan di depan, sesuai kelaziman yardstick. Perhatikan pula bahwa hubungan antara peubah dan sasaran sengaja dibuat ada — berbeda dari beberapa modul sebelumnya yang peubahnya saling bebas. Akibatnya model diharapkan berkinerja baik, sehingga kinerja yang buruk justru menjadi pertanda ada kekeliruan pada alur kerja, bukan temuan mengenai datanya.

Contoh 10.2 | Memeriksa ketimpangan dan menetapkan acuan

Kode 10.2 · R memeriksa ketimpangan dan profil awal

```
prop <- prop.table(table(survei$rentan))
cat("proporsi kelas:", round(prop, 4), "\n")
```

```
proporsi kelas: 0.2 0.8
```

```
cat("rasio timpang:", round(max(prop) / min(prop), 2), "berbanding 1\n")
```

```
rasio timpang: 4 berbanding 1
```

```
cat("acuan tebakan mayoritas:", round(max(prop), 4), "\n\n")
```

```
acuan tebakan mayoritas: 0.8
```

```
# sebaran peubah numerik menurut kelas
```

```
num <- c("pengeluaran", "anggota_rt", "pendidikan", "ketahanan", "sejahtera")
ringkas_kelas <- aggregate(survei[num], list(rentan = survei$rentan), mean)
ringkas_kelas[num] <- round(ringkas_kelas[num], 2)
print(ringkas_kelas, row.names = FALSE)
```

```
rentan pengeluaran anggota_rt pendidikan ketahanan sejahtera
Ya          315.6          4.72          7.66          2.21          1.71
```

Tidak	594.8	4.13	9.13	3.07	2.00
-------	-------	------	------	------	------

Penjelasan Kode. Proporsi kelas dihitung beserta rasio ketimpangan dan acuan tebakan mayoritas. Setelah itu rata-rata tiap peubah numerik dibandingkan antara kedua kelas.

Interpretasi Output

Acuan tebakan mayoritas sebesar empat perlima menjadi pembanding wajib bagi setiap akurasi yang kelak dilaporkan. Adapun perbandingan rata-rata antar-kelas sudah memperlihatkan arah hubungan yang masuk akal: rumah tangga rentan cenderung mengeluarkan lebih rendah dan beranggota lebih banyak. Pemeriksaan sederhana ini berguna sebagai pemeriksaan kewajaran sebelum pemodelan — bila arahnya berlawanan dengan pengetahuan bidang, ada yang perlu ditelusuri lebih dahulu.

Contoh 10.3 | Membagi data dan menyusun resep

Kode 10.3 • R resep dengan penskalaan dan penyeimbangan

```
suppressPackageStartupMessages({
  library(recipes); library(rsample); library(themis)})

set.seed(2025)
bagi <- initial_split(survei, prop = 0.75, strata = rentan)
lat <- training(bagi); uji <- testing(bagi)

rec <- recipe(rentan ~ ., data = lat) |>
  step_dummy(all_nominal_predictors()) |>      # pengodean kategori
  step_zv(all_predictors()) |>                # buang ragam nol
  step_range(all_numeric_predictors(), min = 0, max = 1) |> # skala 0-1
  step_rose(rentan, seed = 2025)              # seimbangkan LATIH saja

siap <- prep(rec, training = lat)
lat_siap <- bake(siap, new_data = NULL) # data latih: sudah diseimbangkan
uji_siap <- bake(siap, new_data = uji)  # data uji : TIDAK diseimbangkan

cat("latih asli      :", nrow(lat), "| setelah ROSE:", nrow(lat_siap), "\n")
```

```
latih asli      : 750 | setelah ROSE: 1200
```

```
print(table(latih = lat_siap$rentan))
```

```
latih
  Ya Tidak
601    599
```

```
print(table(uji = uji_siap$rentan))
```

```
uji
  Ya Tidak
50    200
```

Penjelasan Kode. Data dibagi dengan penstrataan. Resep menetapkan empat langkah: pengodean peubah kategori, pembuangan peubah berragam nol, penskalaan ke rentang nol sampai satu, dan penyeimbangan kelas memakai ROSE. Resep lalu diterapkan ke kedua bagian data.

Interpretasi Output

Bandingkan kedua tabel terakhir. Data latih menjadi seimbang, sedangkan data uji tetap timpang sebagaimana keadaan sesungguhnya — dan inilah perilaku yang benar. Paket recipes melewati langkah penyeimbangan secara otomatis ketika resep diterapkan pada data baru, sehingga kekeliruan yang dibahas pada Teori tercegah oleh rancangan, bukan oleh kedisiplinan penulis kode. Perhatikan pula penskalaan ke rentang nol sampai satu yang dipilih karena sesuai dengan fungsi aktivasi logistik.

Contoh 10.4 | Membangun FFNN pertama

Kode 10.4 • R mlp dengan mesin nnet

```
suppressPackageStartupMessages({
  library(parsnip); library(workflows); library(yardstick); library(dplyr)})

spec_ffnn <- mlp(hidden_units = 6, penalty = 0.01, epochs = 200) |>
  set_engine("nnet", trace = FALSE) |>
  set_mode("classification")

wf_ffnn <- workflow() |> add_recipe(rec) |> add_model(spec_ffnn)
set.seed(2025)
m_ffnn <- fit(wf_ffnn, data = lat)

duga <- predict(m_ffnn, uji) |>
  bind_cols(predict(m_ffnn, uji, type = "prob")) |>
  bind_cols(uji |> select(rentan))

print(conf_mat(duga, truth = rentan, estimate = .pred_class))
```

```
      Truth
Prediction Ya Tidak
Ya        42    28
Tidak     8    172
```



```
cat("\nAUC:", round(yardstick::roc_auc(duga, rentan, .pred_Ya)$estimate, 4),
    "| acuan mayoritas:", round(max(prop.table(table(uji$rentan))), 4), "\n")
```

```
AUC: 0.9403 | acuan mayoritas: 0.8
```

Penjelasan Kode. Spesifikasi jaringan ditetapkan dengan enam simpul tersembunyi, penyusutan 0,01, dan dua ratus epoch. Argumen trace bernilai salah menekan keluaran pelatihan yang panjang. Alur kerja dirangkai, dilatih, lalu dinilai pada data uji.

Interpretasi Output

Bandingkan AUC terhadap acuan; pada data yang hubungannya sengaja dibuat ada, AUC seharusnya jauh melampaui setengah. Perhatikan bahwa nilai simpul dan penyusutan di sini masih ditebak, bukan ditala — contoh berikutnya memperbaikinya. Perhatikan pula bahwa benih acak ditetapkan sebelum pelatihan, sebab bobot awal jaringan dibangkitkan acak sehingga hasilnya berubah tiap kali dijalankan tanpa benih.

Contoh 10.5 | Memperagakan kepekaan terhadap benih acak

Kode 10.5 • R melatih jaringan dengan lima benih berbeda

```
hasil_benih <- sapply(1:5, function(s) {
  set.seed(s)
  m <- fit(wf_ffnn, data = lat)
  d <- predict(m, uji, type = "prob") |> bind_cols(uji |> select(rentan))
  yardstick::roc_auc(d, rentan, .pred_Ya)$estimate
})
print(round(data.frame(benih = 1:5, AUC = hasil_benih), 4), row.names = FALSE)
```

```
benih    AUC
1 0.9221
2 0.9295
3 0.9236
4 0.9354
5 0.9331
```

```
cat("\nrentang AUC antar-benih:", round(diff(range(hasil_benih)), 4), "\n")
```

```
rentang AUC antar-benih: 0.0133
```

```
cat("simpangan baku      :", round(sd(hasil_benih), 4), "\n")
```

```
simpangan baku          : 0.0058
```

Penjelasan Kode. Jaringan yang sama dilatih lima kali dengan benih acak berbeda. AUC pada data uji dicatat bagi masing-masing, lalu rentang dan simpangan bakunya dihitung.

Interpretasi Output

Rentang antar-benih inilah ketidakpastian yang melekat pada jaringan syaraf dan tidak dimiliki model seperti regresi logistik. Sebabnya bobot awal dibangkitkan acak dan pengoptimuman dapat berhenti pada minimum setempat yang berbeda. Akibat praktisnya tegas: melaporkan AUC dari satu kali pelatihan tanpa menyebutkan benihnya membuat hasil tidak dapat direproduksi, dan bila rentangnya besar maka perbandingan antar-model menjadi tidak bermakna kecuali selisihnya melampaui rentang itu.

Contoh 10.6 | Menala simpul tersembunyi dan penyusutan

Kode 10.6 · R tune_grid atas dua parameter

```
suppressPackageStartupMessages({library(tune); library(dials)})

spec_tune <- mlp(hidden_units = tune(), penalty = tune(), epochs = 200) |>
  set_engine("nnet", trace = FALSE) |>
  set_mode("classification")
wf_tune <- workflow() |> add_recipe(rec) |> add_model(spec_tune)

set.seed(2025)
lipatan <- vfold_cv(lat, v = 5, strata = rentan)
kisi <- expand_grid(hidden_units = c(3, 6, 10, 15),
  penalty = c(0.0001, 0.001, 0.01, 0.1))

set.seed(2025)
hasil_tune <- tune_grid(wf_tune, resamples = lipatan, grid = kisi,
  metrics = metric_set(yardstick::roc_auc, yardstick::f_meas))

atas <- collect_metrics(hasil_tune) |>
  filter(.metric == "roc_auc") |>
  arrange(desc(mean)) |>
  select(hidden_units, penalty, mean, std_err) |>
  mutate(across(where(is.numeric), \(x) round(x, 4)))
print(utils::head(as.data.frame(atas), 6), row.names = FALSE)
```

hidden_units	penalty	mean	std_err
3	0.0001	0.9276	0.0098
3	0.1000	0.9237	0.0070

```

3  0.0100 0.9210 0.0100
3  0.0010 0.9156 0.0116
6  0.1000 0.9093 0.0078
10 0.1000 0.8982 0.0117

```

Penjelasan Kode. Kedua parameter ditandai tune sehingga nilainya dicari. Kisi berukuran empat kali empat dicoba pada lima lipatan, sehingga delapan puluh jaringan dilatih. Enam kombinasi terbaik ditampilkan bersama galat bakunya.

Interpretasi Output

Perhatikan kolom galat baku. Bila selisih rata-rata antar-kombinasi teratas lebih kecil daripada galat bakunya, maka pemilihan yang terbaik sesungguhnya sembarang. Dalam keadaan demikian, memilih jaringan yang lebih kecil dengan penyusutan lebih besar — yakni model yang lebih sederhana dan lebih stabil — lebih dapat dipertanggungjawabkan daripada memilih yang rata-ratanya kebetulan tertinggi.

Contoh 10.7 | Menggambarkan permukaan penalaan

Kode 10.7 • R `matplot` atas kisi dua parameter

```

auc_kisi <- collect_metrics(hasil_tune) |> filter(.metric == "roc_auc")

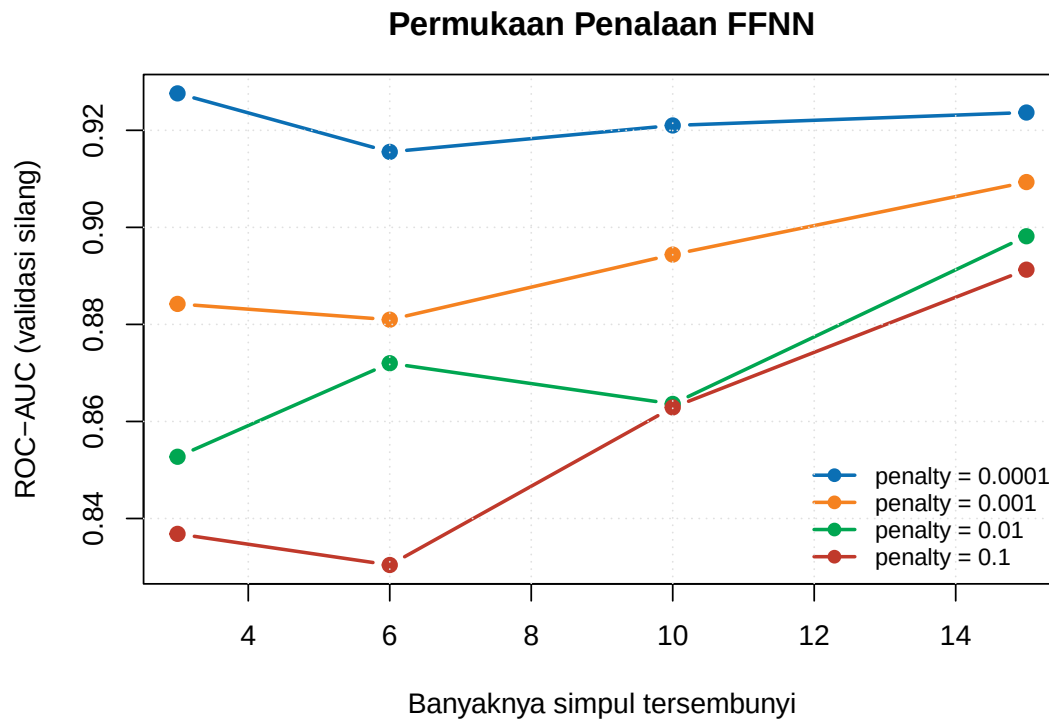
matriks <- with(auc_kisi,
  matrix(mean, nrow = length(unique(hidden_units)),
    dimnames = list(sort(unique(hidden_units)), sort(unique(penalty))))

par(mar = c(4.5, 4.5, 3, 1))
matplot(sort(unique(auc_kisi$hidden_units)), matriks, type = "b",
  pch = 19, lwd = 2, lty = 1,
  col = c("#0B6FB4", "#F58220", "#00A651", "#C0392B"),
  xlab = "Banyaknya simpul tersembunyi",
  ylab = "ROC-AUC (validasi silang)",
  main = "Permukaan Penalaan FFNN")

legend("bottomright", legend = paste("penalty =", colnames(matriks)),
  col = c("#0B6FB4", "#F58220", "#00A651", "#C0392B"),
  lwd = 2, pch = 19, bty = "n", cex = 0.8)

grid(col = "grey88")

```



```
par(mar = c(5, 4, 4, 2) + 0.1)
```

Penjelasan Kode. Hasil penalaan diubah menjadi matriks berbaris simpul tersembunyi dan berkolom nilai penyusutan. Setiap kolom digambar sebagai satu garis, sehingga pengaruh kedua parameter terlihat sekaligus.

Interpretasi Output

Gambar ini memperlihatkan keterkaitan yang disebut pada Teori. Garis dengan penyusutan kecil lazimnya menurun ketika simpul bertambah — pertanda mengalami *overfitting* — sedangkan garis dengan penyusutan memadai tetap datar atau naik. Menyajikan permukaan seperti ini jauh lebih informatif daripada sekadar melaporkan kombinasi terbaik, sebab pembaca dapat menilai sendiri seberapa peka model terhadap pilihannya.

Contoh 10.8 | Menyelesaikan FFNN dan menilai pada data uji

Kode 10.8 • R finalize_workflow dan last_fit

```
terbaik <- select_best(hasil_tune, metric = "roc_auc")
cat("terpilih - simpul:", terbaik$hidden_units,
    "| penyusutan:", signif(terbaik$penalty, 4), "\n\n")
```

```
terpilih - simpul: 3 | penyusutan: 0.0001
```

```
wf_final <- finalize_workflow(wf_tune, terbaik)
set.seed(2025)
akhir <- last_fit(wf_final, split = bagi,
                  metrics = metric_set(yardstick::roc_auc, yardstick::f_meas, yardstick::accuracy))

collect_metrics(akhir) |>
  mutate(.estimate = round(.estimate, 4)) |>
  select(.metric, .estimate) |> print()
```

```
# A tibble: 3 x 2
  .metric .estimate
  <chr>    <dbl>
1 f_meas  0.713
2 accuracy 0.852
3 roc_auc  0.944
```

```
pred_ffnn <- collect_predictions(akhir)
print(conf_mat(pred_ffnn, truth = rentan, estimate = .pred_class))
```

	Truth	
Prediction	Ya	Tidak
Ya	46	33
Tidak	4	167

Penjelasan Kode. Kombinasi parameter terbaik dipakai menyelesaikan alur kerja. Fungsi `last_fit` melatih ulang pada seluruh data latih lalu menilai satu kali pada data uji. Dugaan pada data uji diambil untuk pemeriksaan lanjutan.

Interpretasi Output

Inilah satu-satunya kali data uji disentuh; seluruh keputusan penalaan diambil lewat validasi silang pada data latih. Perhatikan bahwa matriks kekeliruan dihitung pada ambang bawaan setengah — contoh berikutnya memperlihatkan bahwa ambang itu jarang merupakan pilihan terbaik pada data timpang.

Contoh 10.9 | Memindahkan ambang keputusan

Kode 10.9 • R menghitung ukuran pada berbagai ambang

```
ambang <- seq(0.05, 0.95, by = 0.05)
ukuran_ambang <- t(sapply(ambang, function(t) {
```

```

dg <- factor(ifelse(pred_ffnn$.pred_Ya > t, "Ya", "Tidak"),
             levels = c("Ya", "Tidak"))
sn <- yardstick::sensitivity_vec(pred_ffnn$rentan, dg)
sp <- yardstick::specificity_vec(pred_ffnn$rentan, dg)
c(sensitivitas = sn, spesifisitas = sp, youden = sn + sp - 1,
  F1 = yardstick::f_meas_vec(pred_ffnn$rentan, dg))
}))
tabel_ambang <- data.frame(ambang, round(ukuran_ambang, 4))
print(tabel_ambang[seq(1, nrow(tabel_ambang), by = 2), ], row.names = FALSE)

```

ambang	sensitivitas	spesifisitas	youden	F1
0.05	1.00	0.000	0.000	0.3333
0.15	1.00	0.000	0.000	0.3333
0.25	1.00	0.000	0.000	0.3333
0.35	0.96	0.640	0.600	0.5647
0.45	0.94	0.755	0.695	0.6438
0.55	0.86	0.875	0.735	0.7288
0.65	0.48	0.980	0.460	0.6154
0.75	0.00	1.000	0.000	NA
0.85	0.00	1.000	0.000	NA
0.95	0.00	1.000	0.000	NA

```

t_youden <- ambang[which.max(ukuran_ambang[, "youden"])]
t_f1 <- ambang[which.max(ukuran_ambang[, "F1"])]
cat("\nambang terbaik menurut Youden:", t_youden,
    "| menurut F1:", t_f1, "| bawaan: 0.5\n")

```

```

ambang terbaik menurut Youden: 0.5 | menurut F1: 0.6 | bawaan: 0.5

```

Penjelasan Kode. Sembilan belas ambang dicoba. Pada tiap ambang, kelas dugaan dibentuk ulang lalu sensitivitas, spesifisitas, indeks Youden, dan F1 dihitung memakai fungsi bentuk vektor dari yardstick. Ambang terbaik menurut dua ukuran berbeda dicari.

Interpretasi Output

Perhatikan bahwa kedua ukuran dapat menghasilkan ambang yang berbeda, dan keduanya lazimnya bukan setengah. Indeks Youden menyeimbangkan kedua jenis kekeliruan tanpa memihak, sedangkan F1 lebih memihak kelas yang diperhatikan. Pemilihan di antara keduanya adalah keputusan kebijakan: bila melewati rumah tangga rentan dianggap jauh lebih merugikan daripada memeriksa yang tidak rentan, maka ukuran yang lebih memihak sensitivitas lebih tepat.

Contoh 10.10 | Menggambarkan pengaruh ambang

Kode 10.10 · R grafik ambang dan kurva ROC

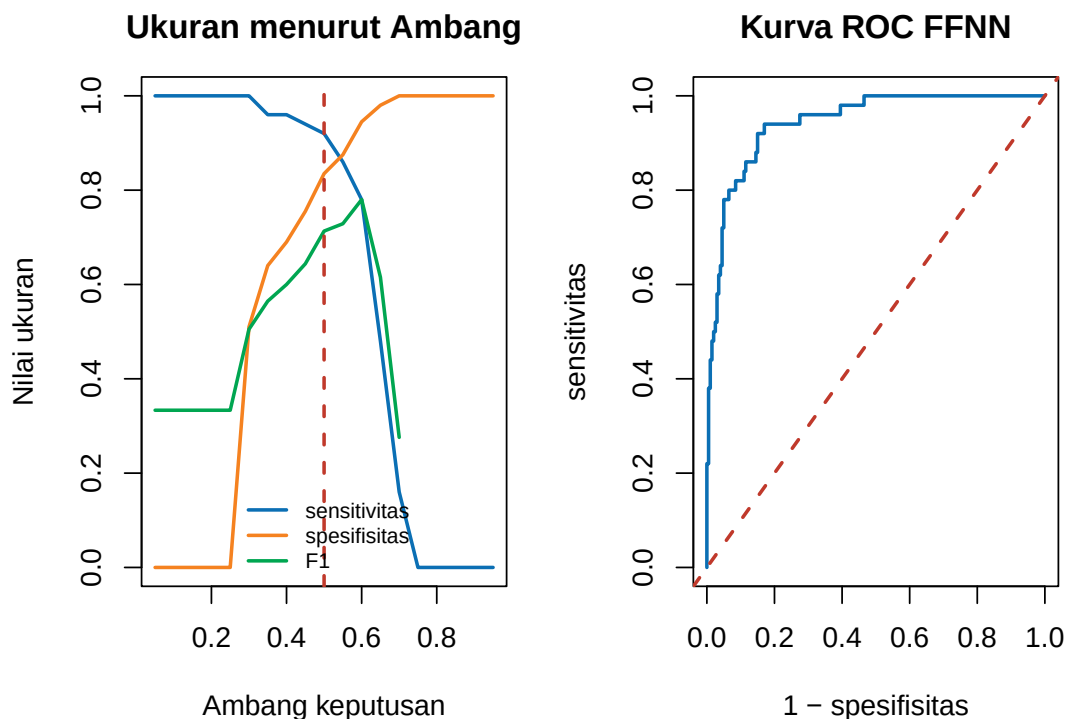
```

par(mfrow = c(1, 2), mar = c(4.5, 4.5, 3, 1))

matplot(ambang, ukuran_ambang[, c("sensitivitas", "spesifisitas", "F1")],
        type = "l", lwd = 2, lty = 1,
        col = c("#0B6FB4", "#F58220", "#00A651"),
        xlab = "Ambang keputusan", ylab = "Nilai ukuran",
        main = "Ukuran menurut Ambang")
abline(v = t_youden, col = "#C0392B", lty = 2, lwd = 2)
legend("bottom", c("sensitivitas", "spesifisitas", "F1"),
      col = c("#0B6FB4", "#F58220", "#00A651"), lwd = 2, bty = "n", cex = 0.75)

kurva <- roc_curve(pred_ffnn, truth = rentan, .pred_Ya)
plot(1 - kurva$specificity, kurva$sensitivity, type = "l", lwd = 2,
     col = "#0B6FB4", xlim = c(0, 1), ylim = c(0, 1),
     xlab = "1 - spesifisitas", ylab = "sensitivitas",
     main = "Kurva ROC FFNN")
abline(0, 1, col = "#C0392B", lty = 2, lwd = 2)

```



```

par(mfrow = c(1, 1), mar = c(5, 4, 4, 2) + 0.1)

```

Penjelasan Kode. Dua gambar disajikan berdampingan. Yang kiri memuat ketiga ukuran menurut ambang dengan garis tegak menandai ambang terbaik menurut Youden. Yang kanan berupa kurva ROC yang menilai seluruh ambang sekaligus.

Interpretasi Output

Kedua gambar melengkapi satu sama lain. Kurva ROC menilai kemampuan pemisahan model tanpa bergantung ambang, sedangkan gambar kiri menjawab pertanyaan praktis ambang mana yang hendak dipakai. Perhatikan bahwa titik pada kurva ROC yang terjauh dari garis diagonal berpadanan dengan ambang yang memaksimumkan indeks Youden — keduanya menyatakan hal yang sama dengan cara berbeda.

Contoh 10.11 | Membangun RBFNN dari dasar

Kode 10.11 • R menyusun RBFNN dengan tiga fungsi

```
# jarak Euclidean antara tiap baris A terhadap tiap baris B
jarak_ke_pusat <- function(A, B) {
  sqrt(pmax(outer(rowSums(A^2), rep(1, nrow(B))) +
               outer(rep(1, nrow(A)), rowSums(B^2)) - 2 * A %*% t(B), 0))
}

rbf_latih <- function(X, y, size = 20, seed = 2025) {
  set.seed(seed)
  km <- kmeans(X, centers = size, nstart = 10, iter.max = 50)
  lebar <- max(dist(km$centers)) / sqrt(2 * size) # aturan praktis
  H <- exp(-jarak_ke_pusat(X, km$centers)^2 / (2 * lebar^2))
  df <- data.frame(H); df$y <- ifelse(y == "Ya", 1, 0)
  list(pusat = km$centers, lebar = lebar,
       mod = suppressWarnings(glm(y ~ ., data = df, family = binomial)))
}

rbf_duga <- function(fit, X) {
  H <- exp(-jarak_ke_pusat(X, fit$pusat)^2 / (2 * fit$lebar^2))
  as.numeric(predict(fit$mod, newdata = data.frame(H), type = "response"))
}

X_lat <- as.matrix(lat_siap[, setdiff(names(lat_siap), "rentan")])
X_uji <- as.matrix(uji_siap[, setdiff(names(uji_siap), "rentan")])

f20 <- rbf_latih(X_lat, lat_siap$rentan, size = 20)
```



```
cat("banyaknya pusat:", nrow(f20$pusat), "| lebar basis:", round(f20$lebar, 4), "\n")
```

```
banyaknya pusat: 20 | lebar basis: 0.347
```

```
cat("dimensi ruang peubah:", ncol(X_lat), "\n")
```

```
dimensi ruang peubah: 9
```

Penjelasan Kode. Tiga fungsi disusun. Yang pertama menghitung matriks jarak memakai penjabaran kuadrat agar cepat. Yang kedua melatih jaringan: pusat ditetapkan dengan K-Means, lebar dihitung dengan aturan praktis, matriks basis Gauss dibentuk, lalu bobot keluaran diduga dengan regresi logistik. Yang ketiga menerapkannya pada data baru.

Interpretasi Output

Menyusun RBFNN dari dasar bukan sekadar siasat karena paket tidak tersedia, melainkan cara memahami mekanismenya. Ketiga langkahnya terlihat jelas: penempatan pusat, pengukuran kedekatan, dan penggabungan lurus. Perlu dicatat bahwa pelatihannya berlangsung dua tahap terpisah — pusat ditetapkan tanpa memperhatikan sasaran, baru kemudian bobot keluaran diduga — berbeda dari FFNN yang seluruh bobotnya dilatih bersama-sama.

Contoh 10.12 | Menala banyaknya pusat RBFNN

Kode 10.12 • R melatih RBFNN pada lima ukuran

```
ukuran_pusat <- c(5, 10, 20, 30, 50)

hasil_rbf <- t(sapply(ukuran_pusat, function(s) {
  f <- rbf_latih(X_lat, lat_siap$rentan, size = s)
  d_lat <- data.frame(benar = lat_siap$rentan, prob = rbf_duga(f, X_lat))
  d_uji <- data.frame(benar = uji_siap$rentan, prob = rbf_duga(f, X_uji))
  c(pusat = s, lebar = round(f$lebar, 4),
    AUC_latih = round(yardstick::roc_auc(d_lat, benar, prob)$estimate, 4),
    AUC_uji = round(yardstick::roc_auc(d_uji, benar, prob)$estimate, 4))
}))
print(hasil_rbf, row.names = FALSE)
```

	pusat	lebar	AUC_latih	AUC_uji
[1,]	5	0.4979	0.6578	0.6458
[2,]	10	0.4619	0.6827	0.6436
[3,]	20	0.3470	0.7209	0.7055
[4,]	30	0.3156	0.7414	0.7204
[5,]	50	0.2677	0.7625	0.7611

```
cat("\njarak AUC latih terhadap uji:\n")
```

```
jarak AUC latih terhadap uji:
```

```
print(round(hasil_rbf[, "AUC_latih"] - hasil_rbf[, "AUC_uji"], 4))
```

```
[1] 0.0120 0.0391 0.0154 0.0210 0.0014
```

Penjelasan Kode. Lima nilai banyaknya pusat dicoba. Pada masing-masing, AUC dihitung dua kali: pada data latih dan pada data uji. Lebar basis ikut dilaporkan karena nilainya berubah mengikuti banyaknya pusat.

Interpretasi Output

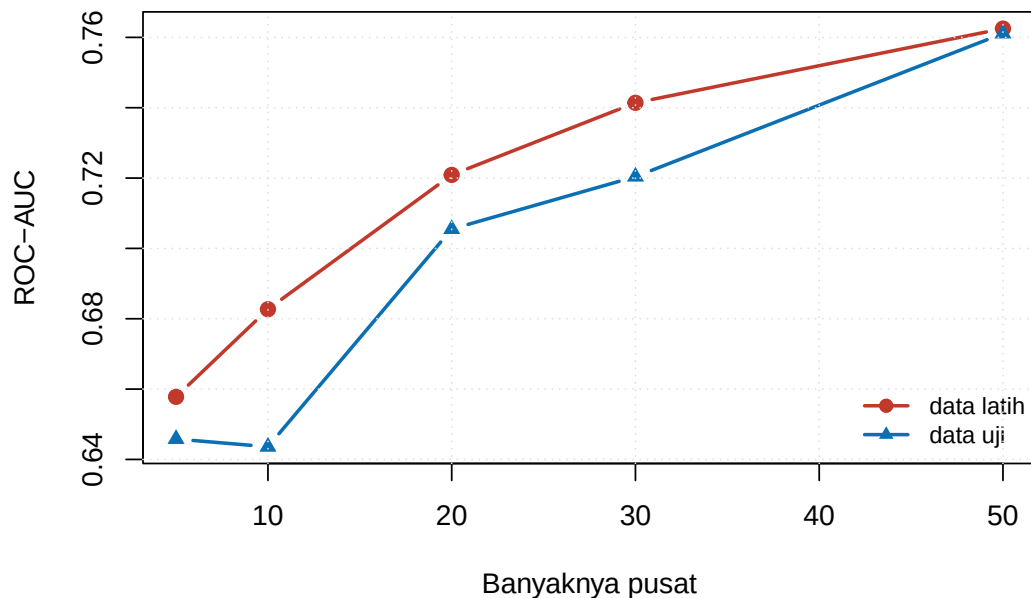
Perhatikan dua hal. Pertama, lebar basis menyempit ketika pusat bertambah, sesuai rumus aturan praktis — makin banyak pusat, makin setempat pengaruh masing-masing. Kedua, jarak antara AUC latih dan uji melebar seiring pusat bertambah; itulah wujud pemodelan berlebihan (*overfitting*) pada RBFNN. Banyaknya pusat karena itu berperan serupa dengan banyaknya simpul tersembunyi pada FFNN, yakni mengatur fleksibilitas model.

Contoh 10.13 | Menggambarkan pengaruh banyaknya pusat

Kode 10.13 • R grafik kinerja latih dan uji

```
plot(hasil_rbf[, "pusat"], hasil_rbf[, "AUC_latih"], type = "b", pch = 19,
     lwd = 2, col = "#C0392B",
     ylim = range(hasil_rbf[, c("AUC_latih", "AUC_uji")]),
     xlab = "Banyaknya pusat", ylab = "ROC-AUC",
     main = "Kelenturan RBFNN menurut Banyaknya Pusat")
lines(hasil_rbf[, "pusat"], hasil_rbf[, "AUC_uji"], type = "b", pch = 17,
      lwd = 2, col = "#0B6FB4")
legend("bottomright", c("data latih", "data uji"),
      col = c("#C0392B", "#0B6FB4"), lwd = 2, pch = c(19, 17),
      bty = "n", cex = 0.85)
grid(col = "grey88")
```

Kelenturan RBFNN menurut Banyaknya Pusat



```
pusat_terbaik <- hasil_rbf[which.max(hasil_rbf[, "AUC_uji"]), "pusat"]
cat("banyaknya pusat terbaik menurut data uji:", pusat_terbaik, "\n")
```

```
banyaknya pusat terbaik menurut data uji: 50
```

Penjelasan Kode. Kedua kurva AUC digambar bersama pada sumbu yang sama, dengan batas sumbu tegak ditetapkan agar keduanya selalu terlihat penuh.

Interpretasi Output

Jarak antara kedua garis adalah ukuran langsung pemodelan berlebih (*overfitting*), persis sebagaimana pada penalaan tetangga terdekat di modul supervised learning. Perlu diakui satu kelemahan pada peragaan ini: banyaknya pusat terbaik dipilih berdasarkan data uji, padahal seharusnya lewat validasi silang pada data latih. Peragaan ini disederhanakan demi kejelasan, dan Penugasan menuntut Anda mengerjakannya dengan benar memakai lipatan.

Contoh 10.14 | Menghitung kepentingan peubah dengan permutasi

Kode 10.14 • R permutasi peubah satu per satu

```
f_rbf <- rbf_latih(X_lat, lat_siap$rentan, size = as.numeric(pusat_terbaik))
dasar <- yardstick::roc_auc(data.frame(benar = uji_siap$rentan,
                                       prob = rbf_duga(f_rbf, X_uji)), benar, prob)$estimate

set.seed(2025)
kepentingan <- sapply(seq_len(ncol(X_uji)), function(j) {
  turun <- replicate(5, {
    Xp <- X_uji; Xp[, j] <- sample(Xp[, j])
    dasar - yardstick::roc_auc(data.frame(benar = uji_siap$rentan,
                                           prob = rbf_duga(f_rbf, Xp)), benar, prob)$estimate
  })
  mean(turun)
})

imp <- data.frame(peubah = colnames(X_uji),
                 penurunan_AUC = round(kepentingan, 4))
imp <- imp[order(-imp$penurunan_AUC), ]
print(utils::head(imp, 8), row.names = FALSE)
```

peubah	penurunan_AUC
ketahanan	0.1017
sejahtera	0.0788
pekerjaan_informal	0.0678
air_bersih_Ya	0.0544
lahan_Ya	0.0355
pekerjaan_tidak_bekerja	0.0292
anggota_rt	0.0283
pengeluaran	0.0036

Penjelasan Kode. Setiap peubah diacak nilainya lima kali, dan penurunan AUC dicatat lalu dirata-ratakan. Pengacakan memutus hubungan antara peubah itu dan sasaran tanpa melatih ulang model, sehingga sumbangannya terhadap dugaan dapat diukur.

Interpretasi Output

Pengulangan lima kali penting karena pengacakan bersifat acak; sekali pengacakan menghasilkan angka yang tidak stabil. Bandingkan urutan yang diperoleh terhadap koefisien yang dipakai membangkitkan data pada Contoh 10.1 — bila peubah yang koefisiennya besar memang muncul di atas, itu pertanda model menangkap struktur yang benar. Perlu ditegaskan kembali bahwa kepentingan ini menyatakan sumbangan terhadap dugaan, bukan pengaruh sebab-akibat.

Contoh 10.15 | Menafsirkan model melalui analisis profil

Kode 10.15 · R membandingkan ciri antar-kelompok peluang

```

prob_uji <- rbf_duga(f_rbf, X_uji)
kelompok <- cut(prob_uji, breaks = quantile(prob_uji, c(0, .25, .75, 1)),
                 labels = c("peluang rendah", "menengah", "peluang tinggi"),
                 include.lowest = TRUE)

num_asli <- c("pengeluaran", "anggota_rt", "pendidikan", "ketahanan", "sejahtera")
profil <- aggregate(uji[num_asli], list(kelompok = kelompok), mean)
profil[num_asli] <- round(profil[num_asli], 2)
print(profil, row.names = FALSE)

```

	kelompok	pengeluaran	anggota_rt	pendidikan	ketahanan	sejahtera
peluang rendah		484.0	4.02	8.51	3.60	2.21
menengah		600.2	4.24	8.99	2.88	1.88
peluang tinggi		558.7	4.43	8.22	2.41	1.78

```
cat("\nproporsi rentan sebenarnya per kelompok:\n")
```

```
proporsi rentan sebenarnya per kelompok:
```

```
print(round(tapply(uji$rentan == "Ya", kelompok, mean), 4))
```

	peluang rendah	menengah	peluang tinggi
	0.0476	0.1532	0.4444

Penjelasan Kode. Amatan data uji dibagi tiga menurut kuartil peluang dugaan. Rata-rata peubah numerik dalam satuan aslinya dibandingkan antar-kelompok, disusul proporsi kelas yang sebenarnya pada tiap kelompok.

Interpretasi Output

Analisis profil menjembatani model tertutup dengan pemahaman pengguna. Bila kelompok berpeluang tinggi ternyata berpengeluaran jauh lebih rendah dan beranggota lebih banyak, maka model menangkap pola yang masuk akal menurut pengetahuan bidang. Baris terakhir berfungsi sebagai pemeriksaan kalibrasi kasar: proporsi rentan sebenarnya seharusnya menaik dari kelompok rendah ke tinggi. Bila tidak menaik, dugaan peluangnya tidak dapat dipercaya untuk penyasaran.

Contoh 10.16 | Membandingkan FFNN dan RBFNN secara adil**Kode 10.16 · R** menyandingkan kedua arsitektur

```

d_ffnn <- data.frame(benar = pred_ffnn$rentan, prob = pred_ffnn$.pred_Ya)
d_rbf <- data.frame(benar = uji_siap$rentan, prob = prob_uji)

```

```

ringkas <- function(d, nama) {
  dg <- factor(ifelse(d$prob > 0.5, "Ya", "Tidak"), levels = c("Ya", "Tidak"))
  data.frame(model = nama,
    AUC = round(yardstick::roc_auc(d, benar, prob)$estimate, 4),
    F1 = round(yardstick::f_meas_vec(d$benar, dg), 4),
    Sens = round(yardstick::sensitivity_vec(d$benar, dg), 4),
    Spes = round(yardstick::specificity_vec(d$benar, dg), 4))
}
banding <- rbind(ringkas(d_ffnn, "FFNN"), ringkas(d_rbf, "RBFNN"))
print(banding, row.names = FALSE)

```

model	AUC	F1	Sens	Spes
FFNN	0.9440	0.7132	0.92	0.835
RBFNN	0.7611	0.4496	0.58	0.750

```
cat("\nselisih AUC antar-model      :", round(abs(diff(banding$AUC)), 4), "\n")
```

```
selisih AUC antar-model      : 0.1829
```

```
cat("rentang AUC FFNN antar-benih:", round(diff(range(hasil_benih)), 4), "\n")
```

```
rentang AUC FFNN antar-benih: 0.0133
```

Penjelasan Kode. Kedua model dinilai pada data uji yang sama dengan ambang yang sama. Empat ukuran dihitung bagi masing-masing. Selisih AUC keduanya dibandingkan terhadap rentang antar-benih yang diperoleh sebelumnya.

Interpretasi Output

Perbandingan pada baris terakhir adalah bagian terpenting. Bila selisih AUC antar-model lebih kecil daripada rentang yang timbul semata karena benih acak berbeda, maka menyatakan salah satu arsitektur lebih baik tidak didukung bukti. Kekeliruan ini lazim terjadi pada perbandingan jaringan syaraf, sebab ketidakpastian akibat inisialisasi acak kerap diabaikan. Perlu dicatat pula bahwa keduanya di sini dinilai pada ambang bawaan; kesimpulan dapat berubah bila masing-masing dipakai pada ambang optimalnya sendiri.

10.7 Latihan Mandiri

Kerjakan bertahap. Gunakan data/data_survei.csv.

10.7.0.0.1 Tingkat Dasar (melengkapi kode)

Latihan 10.1 | Resep bagi jaringan syaraf

```
library(recipes); library(rsample); library(themis)
survei <- read.csv("data/data_survei.csv", stringsAsFactors = TRUE)
survei$rentan <- factor(survei$rentan, levels = c("Ya", "Tidak"))
set.seed(2025)
bagi <- ____ (survei, prop = 0.75, strata = rentan)
lat <- training(bagi); uji <- testing(bagi)

rec <- recipe(rentan ~ ., data = lat) |>
  step_dummy(all_nominal_predictors()) |>
  ____ (all_numeric_predictors(), min = 0, max = 1) |> # skala 0-1
  ____ (rentan, seed = 2025) # seimbangkan
```

Latihan 10.2 | Membangun FFNN

```
library(parsnip); library(workflows)
spec <- ____ (hidden_units = 6, penalty = 0.01, epochs = 200) |>
  set_engine("____", trace = FALSE) |>
  set_mode("classification")
wf <- workflow() |> add_recipe(rec) |> add_model(spec)
____ (2025) # benih WAJIB: bobot awal acak
m <- fit(wf, data = lat)
```

Latihan 10.3 | Basis Gauss

```
# lebar basis menurut aturan praktis
lebar <- max(dist(pusat)) / sqrt(2 * ____ ) # H = banyaknya pusat
# matriks basis Gauss
H <- ____ (-jarak_ke_pusat(X, pusat)^2 / (2 * lebar^2))
```

10.7.0.0.2 Tingkat Menengah (menulis sendiri)

Latihan 10.4 | Menala RBFNN dengan validasi silang

Contoh 10.13 memilih banyaknya pusat berdasarkan data uji — cara yang tidak valid. Perbaiki dengan menala memakai **validasi silang lima lipatan pada data latih**. Bandingkan banyaknya pusat yang terpilih terhadap hasil Contoh 10.13, lalu jelaskan mengapa cara yang benar dapat memberi jawaban berbeda.

Latihan 10.5 | Membetulkan kode yang salah

Kode berikut memuat empat kesalahan (penyeimbangan dikerjakan sebelum pembagian, peubah tidak diskalakan, benih acak tidak ditetapkan, dan ambang dibiarkan pada nilai bawaan pada data timpang). Temukan dan perbaiki, lalu jelaskan akibat masing-masing.

```
library(recipes); library(rsample); library(themis); library(parsnip)
survei <- read.csv("data/data_survei.csv", stringsAsFactors = TRUE)

# (a) penyeimbangan dikerjakan pada SELURUH data sebelum dibagi
survei_seimbang <- ROSE::ROSE(rentan ~ ., data = survei)$data
bagi <- initial_split(survei_seimbang, prop = 0.75)
lat <- training(bagi); uji <- testing(bagi)

# (b) tidak ada penskalaan padahal jaringan syaraf menuntutnya
rec <- recipe(rentan ~ ., data = lat) |>
  step_dummy(all_nominal_predictors())

# (c) benih acak tidak ditetapkan
m <- fit(workflow() |> add_recipe(rec) |> add_model(spec), data = lat)

# (d) ambang dibiarkan pada 0,5 pada data yang timpang
duga <- ifelse(predict(m, uji, type = "prob")$.pred_Ya > 0.5, "Ya", "Tidak")
```

Keluaran yang benar. Setelah diperbaiki, kode yang dipakai adalah sebagai berikut.

```
# (a) pembagian DAHULU; penyeimbangan masuk ke dalam resep
survei$rentan <- factor(survei$rentan, levels = c("Ya", "Tidak"))
set.seed(2025)
bagi <- initial_split(survei, prop = 0.75, strata = rentan)
lat <- training(bagi); uji <- testing(bagi)

# (b) penskalaan wajib, dan (a) penyeimbangan hanya pada data latih
rec <- recipe(rentan ~ ., data = lat) |>
  step_dummy(all_nominal_predictors()) |>
  step_zv(all_predictors()) |>
  step_range(all_numeric_predictors(), min = 0, max = 1) |>
  step_rose(rentan, seed = 2025)
```



```
# (c) benih ditetapkan sebelum pelatihan
set.seed(2025)
m <- fit(workflow() |> add_recipe(rec) |> add_model(spec), data = lat)

# (d) ambang dipilih berdasarkan indeks Youden
p <- predict(m, uji, type = "prob")$.pred_Ya
ambang <- seq(0.05, 0.95, by = 0.05)
j <- sapply(ambang, function(t) {
  dg <- factor(ifelse(p > t, "Ya", "Tidak"), levels = c("Ya", "Tidak"))
  yardstick::sensitivity_vec(uji$rentan, dg) +
    yardstick::specificity_vec(uji$rentan, dg) - 1
})
t_baik <- ambang[which.max(j)]
duga <- factor(ifelse(p > t_baik, "Ya", "Tidak"), levels = c("Ya", "Tidak"))
```

Pemeriksaan yang wajib berhasil: proporsi kelas pada uji masih timpang (sekitar 0,2), dan `t_baik != 0.5` bernilai **TRUE**.

10.7.0.0.3 Tingkat Lanjut (pemilihan)

Latihan 10.6 | Memilih arsitektur menurut kebutuhan

Untuk masing-masing keadaan berikut, tentukan arsitektur yang Anda pilih beserta alasannya, dan sebutkan **satu kelemahan** pilihan itu: (a) model akan menerima amatan yang mungkin jauh di luar rentang data latihan; (b) waktu pelatihan sangat terbatas; (c) hubungan antar-peubah diduga rumit dan menyeluruh, bukan setempat; (d) hasil harus dijelaskan kepada pengguna yang tidak berlatar statistika.

10.8 Penugasan

Sebagai calon analis pada instansi statistik, bangun model penduga kerentanan pangan rumah tangga atas data `data_survei.csv` (*data simulasi*) memakai dua arsitektur jaringan syaraf, lalu rekomendasikan satu di antaranya beserta ambang keputusan yang sesuai.

Konteks. Sebuah unit hendak menyasar bantuan pangan. Melewatkan rumah tangga yang sesungguhnya rentan dianggap jauh lebih merugikan daripada memeriksa rumah tangga yang ternyata tidak rentan.

Tugas 10.1 | Penyiapan

1. Periksa **ketimpangan kelas** dan laporkan acuan tebakan mayoritas. Sajikan pula profil awal tiap peubah menurut kelas.
2. Bagi data dengan penstrataan, lalu susun resep yang memuat pengodean, **penskalaan**, dan **penyeimbangan**. Buktikan bahwa data uji tidak ikut diseimbangkan.

Tugas 10.2 | Feed-Forward Neural Network

3. Latih FFNN dengan **lima benih acak berbeda**. Laporkan rentang dan simpangan baku AUC-nya, lalu jelaskan sumber ketidakpastian itu.
4. **Tala simpul tersembunyi** dan penyusutan pada kisi minimal empat kali empat dengan validasi silang lima lipatan. Sajikan tabel enam teratas beserta galat bakunya.
5. Sajikan **permukaan penalaan** sebagai grafik. Jelaskan keterkaitan kedua parameter berdasarkan bentuk garisnya.
6. Selesaikan alur dan nilai pada data uji. Sajikan matriks kekeliruan dan minimal empat ukuran.

Tugas 10.3 | Radial Basis Function Neural Network

7. Bangun **RBFNN dari dasar**: penempatan pusat, penetapan lebar, dan pendugaan bobot keluaran. Laporkan banyaknya pusat dan lebar basisnya.
8. **Tala banyaknya pusat dengan validasi silang pada data latih** — bukan pada data uji sebagaimana Contoh 10.13. Sajikan hasilnya dan bandingkan terhadap cara yang keliru itu.
9. Sajikan grafik AUC latih dan uji menurut banyaknya pusat. Tentukan pada nilai berapa pemodelan berlebih (*overfitting*) mulai tampak.

Tugas 10.4 | Penafsiran, Ambang, dan Rekomendasi

10. Hitung **kepentingan permutasi** dengan minimal lima pengulangan. Bandingkan urutannya terhadap koefisien pembangkit data, lalu nilai apakah model menangkap struktur yang benar.
11. Lakukan **analisis profil** atas tiga kelompok peluang dugaan. Periksa apakah proporsi kelas sebenarnya menaik antar-kelompok, dan jelaskan artinya bagi kelayakan pemakaian model untuk penyasaran.
12. **Pindahkan ambang keputusan.** Bandingkan ambang menurut indeks Youden dan menurut F1. Tentukan ambang yang Anda pilih dengan mempertimbangkan bahwa melewatkan rumah tangga rentan lebih merugikan — dan jelaskan alasannya.
13. **Bandingkan kedua arsitektur** pada data uji yang sama. Nilai apakah selisihnya melampaui rentang yang timbul semata karena benih acak berbeda. Bila tidak, nyatakan bahwa keduanya tidak dapat dibedakan.
14. Susun laporan R Markdown berisi kode, keluaran, gambar, dan interpretasi. Tutup dengan **ringkasan (maksimal 300 kata)** memuat: arsitektur dan ambang yang direkomendasikan beserta alasannya, kinerja terhadap acuan, keterbatasan, dan penegasan bahwa data merupakan simulasi.

10.9 Tambahan: Alur yang Sama dengan caret

Modul ini memakai kerangka tidymodels. Sebagian besar kode dan rujukan yang beredar masih memakai caret, sehingga kemampuan membaca keduanya tetap diperlukan. Bagian ini menjalankan alur yang sama dengan caret lalu membandingkan hasilnya.

Contoh 10.17 | Alur yang sama dengan caret**Kode 10.17 • R** caret::train dengan method nnet

```
suppressPackageStartupMessages(library(caret))

lat_caret <- lat                                # caret tidak mengenal peran penanda
kendali <- trainControl(method = "cv", number = 5,
                        classProbs = TRUE,
                        summaryFunction = twoClassSummary,
                        sampling = "rose")        # penyeimbangan DI DALAM lipatan

set.seed(2025)

m_caret <- train(rentan ~ ., data = lat_caret, method = "nnet",
                preprocess = c("range"),        # skala 0-1
                tuneGrid = expand.grid(size = c(3, 6, 10, 15),
```

```

                                decay = c(0.0001, 0.001, 0.01, 0.1)),
    trControl = kendali, metric = "ROC",
    trace = FALSE, maxit = 200)

hasil_caret <- m_caret$results[order(-m_caret$results$ROC),
                                c("size", "decay", "ROC", "ROCSd")]
print(round(utils::head(hasil_caret, 6), 4), row.names = FALSE)

```

```

size decay ROC ROCSd
15 0.0010 0.1658 0.0429
10 0.0001 0.1613 0.0319
15 0.0001 0.1563 0.0485
6 0.0010 0.1464 0.0297
10 0.0010 0.1383 0.0395
6 0.0001 0.1364 0.0333

```

```

cat("\nterpilih - size:", m_caret$bestTune$size,
    "| decay:", m_caret$bestTune$decay, "\n")

```

```

terpilih - size: 15 | decay: 0.001

```

Penjelasan Kode. Seluruh alur ditulis dalam satu pemanggilan train. Penskalaan ditetapkan lewat `preProcess`, penyeimbangan lewat argumen `sampling` pada `trainControl`, dan ruang pencarian lewat `tuneGrid`. Argumen `sampling` bernilai `rose` memastikan penyeimbangan dikerjakan di dalam tiap lipatan, bukan sebelumnya.

Interpretasi Output

Argumen `sampling` pada `trainControl` adalah bagian terpenting di sini. Tanpa itu, penyeimbangan harus dikerjakan manual sebelum `train` dipanggil — dan bila dikerjakan pada seluruh data latih sekaligus, amatan sintetis akan bocor antar-lipatan sehingga kinerja validasi silang tampak lebih baik daripada sebenarnya. Perhatikan pula bahwa `caret` memakai nama `size` dan `decay`, sedangkan `parSNIP` memakai `hidden_units` dan `penalty`; keduanya menunjuk hal yang sama.

Contoh 10.18 | Menyandingkan hasil kedua kerangka

Kode 10.18 • R menggabungkan hasil dua kerangka

```

tm <- collect_metrics(hasil_tune) |>
  dplyr::filter(.metric == "roc_auc") |>
  dplyr::select(size = hidden_units, decay = penalty, tidymodels = mean)
ct <- data.frame(size = m_caret$results$size,
                 decay = m_caret$results$decay,

```

```

caret = m_caret$results$ROC)

banding2 <- merge(tm, ct, by = c("size", "decay"))
banding2 <- banding2[order(-banding2$tidymodels), ]
print(round(utils::head(banding2, 8), 4), row.names = FALSE)

```

```

size decay tidymodels caret
3 0.0001 0.9276 0.0942
3 0.1000 0.9237 0.0788
3 0.0100 0.9210 0.0956
3 0.0010 0.9156 0.0998
6 0.1000 0.9093 0.0770
10 0.1000 0.8982 0.0675
6 0.0100 0.8944 0.1204
15 0.1000 0.8913 0.0627

```

```

cat("\nkorelasi antar-kerangka:",
    round(cor(banding2$tidymodels, banding2$caret), 4), "\n")

```

```

korelasi antar-kerangka: -0.7842

```

```

cat("selisih rata-rata      :",
    round(mean(abs(banding2$tidymodels - banding2$caret)), 4), "\n")

```

```

selisih rata-rata      : 0.77

```

Penjelasan Kode. Hasil penalaan kedua kerangka digabungkan menurut pasangan nilai parameter yang sama. Korelasi antar-keduanya dihitung, begitu pula selisih rata-rata mutlaknya.

Interpretasi Output

Korelasi yang tinggi menandakan kedua kerangka sepakat mengenai kombinasi parameter mana yang lebih baik, meski angka mutlaknya berbeda. Perbedaan itu wajar dan bersumber dari pembentukan lipatan yang tidak identik, cara penyeimbangan yang sedikit berlainan, serta inisialisasi bobot yang acak. Yang tidak wajar adalah bila urutannya berbeda jauh — itu pertanda ada perbedaan perlakuan yang perlu ditelusuri, bukan sekadar derau.

Melaporkan angka tanpa menyebut seluruh prosedurnya. Karena hasil jaringan syaraf bergantung pada kerangka, benih acak, skema penyeimbangan, penskalaan, dan banyaknya epoch, maka melaporkan angka kinerja tanpa menyebutkan seluruhnya membuat hasil **tidak dapat direproduksi**. Laporan yang baik mencantumkan: perangkat lunak beserta versinya, skema pembagian dan resampling, benih acak, urutan prapemrosesan, cara penanganan ketimpangan, arsitektur beserta parameternya, dan ambang keputusan yang dipakai.

Tips dan Praktik Baik. Bila Anda menjumpai kode caret pada rujukan lama, padanannya di tidymodels dapat ditelusuri melalui nama parameternya: size menjadi hidden_units, decay menjadi penalty, trControl menjadi resamples pada tune_grid, dan preProcess menjadi langkah-langkah pada recipe. Yang berubah adalah tata caranya, bukan gagasannya.

10.10 Ringkasan Modul

Daftar periksa penguasaan:

- ☐ Saya dapat menjelaskan peran lapis tersembunyi dan fungsi aktivasi.
- ☐ Saya memahami mengapa aktivasi nonlinier menjadi sumber kekuatan jaringan.
- ☐ Saya selalu menskalakan peubah sebelum melatih jaringan syaraf.
- ☐ Saya menempatkan penyeimbangan kelas di dalam resep, bukan sebelum pembagian.
- ☐ Saya menetapkan benih acak dan melaporkan ketidakpastian antar-benih.
- ☐ Saya menala simpul dan penyusutan bersama-sama, bukan satu per satu.
- ☐ Saya dapat menjelaskan perbedaan sifat setempat RBFNN dan menyeluruh FFNN.
- ☐ Saya dapat membangun RBFNN dari dasar: pusat, lebar, dan bobot keluaran.
- ☐ Saya memindahkan ambang keputusan berdasarkan ukuran yang sesuai.
- ☐ Saya menafsirkan model tertutup lewat kepentingan permutasi dan profil.
- ☐ Saya menyadari bahwa kepentingan peubah bukan hubungan sebab-akibat.
- ☐ Saya membandingkan model hanya bila selisihnya melampaui derau antar-benih.

Kartu Referensi Cepat.

Keperluan	Perintah
Membagi berstrata	<code>initial_split(d, prop =, strata =)</code>
Penskalaan 0-1	<code>step_range(all_numeric_predictors())</code>
Penyeimbangan	<code>step_rose(y, seed =)</code>
Spesifikasi FFNN	<code>mlp(hidden_units =, penalty =, epochs =)</code>
Mesin FFNN	<code>set_engine("nnet", trace = FALSE)</code>
Penalaan	<code>tune_grid(wf, resamples =, grid =)</code>
Menyelesaikan	<code>finalize_workflow(), last_fit()</code>
Pusat RBFNN	<code>kmeans(X, centers = H)</code>
Lebar basis	<code>max(dist(pusat)) / sqrt(2 * H)</code>
Basis Gauss	<code>exp(-jarak^2 / (2 * lebar^2))</code>
Sensitivitas	<code>yardstick::sensitivity_vec(benar, duga)</code>
Indeks Youden	<code>sens + spes - 1</code>
Kurva ROC	<code>roc_curve(d, truth =, .pred_Ya)</code>
Padanan caret	<code>size = hidden_units, decay = penalty</code>

Modul ini memperkenalkan keluarga model yang kuat tetapi tertutup. Prinsip yang perlu dibawa: skalakan peubah sebelum melatih; tempatkan penyeimbangan di dalam resep agar data uji tetap mencerminkan keadaan sesungguhnya; tetapkan benih dan laporkan ketidakpastiannya; tala fleksibilitas dan penyusutan bersama-sama; pindahkan ambang berdasarkan biaya kekeliruan, bukan membiarkannya pada nilai bawaan; tafsirkan model lewat permutasi dan profil sambil mengingat bahwa keduanya menyatakan kaitan, bukan sebab; dan jangan menyatakan satu arsitektur lebih baik sebelum selisihnya melampaui derau yang timbul semata dari inisialisasi acak.

10.11 Referensi

- Bishop, C. M. (1995). *Neural Networks for Pattern Recognition*. Oxford University Press.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
- Ripley, B. D. (1996). *Pattern Recognition and Neural Networks*. Cambridge University Press.
- Kuhn, M., & Silge, J. (2022). *Tidy Modeling with R*. O'Reilly Media.
- Menardi, G., & Torelli, N. (2014). Training and assessing classification rules with imbalanced data. *Data Mining and Knowledge Discovery*, 28(1), 92-122.
- Youden, W. J. (1950). Index for rating diagnostic tests. *Cancer*, 3(1), 32-35.

MODUL

11

ASSOCIATION RULE DAN FREQUENT ITEMSET MINING

Pertemuan Minggu ke-11

Sub-CPMK 11: Mahasiswa mampu menyiapkan data transaksi, menambang himpunan barang sering muncul beserta aturan asosiasinya dengan algoritma Apriori dan Eclat, menafsirkan support, confidence, dan lift secara benar, memangkas aturan yang berlebihan, serta menilai kebermanaknaan aturan sebelum dipakai mengambil keputusan.

Penyusun: Achmad Syahrul Choir



Daftar Isi

11.1 Deskripsi Modul	335
11.2 Tujuan Praktikum	335
11.3 Alat dan Bahan	336
11.3.1 Perangkat Lunak	336
11.3.2 Bahan (Berkas Data)	336
11.4 Instruksi Pengerjaan	337
11.5 Teori Singkat	337
11.5.1 Data Transaksi	337
11.5.2 Support, Confidence, dan Lift	338
11.5.3 Sifat Apriori dan Dua Algoritma	338
11.5.4 Frequent, Closed, dan Maximal	339
11.5.5 Aturan Redundan	339
11.5.6 Ukuran Tambahan	339
11.6 Contoh Praktikum dengan R	340
11.7 Penerapan pada Statistik Resmi	351
11.8 Latihan Mandiri	358
11.9 Penugasan	360
11.9.1 Rubrik Penilaian	361
11.10 Ringkasan Modul	361
11.10.1 Daftar Istilah	362
11.11 Referensi	362

11.1 Deskripsi Modul

Sepuluh modul sebelumnya bekerja pada data berbentuk **tabel persegi**: satu baris satu amatan, satu kolom satu peubah. Modul ini berpindah ke bentuk data yang berbeda — **transaksi**, yakni kumpulan himpunan. Satu transaksi adalah satu keranjang belanja, satu kunjungan pasien, atau satu berkas jawaban survei; isinya sekumpulan barang atau kejadian yang panjangnya berbeda-beda.

Pertanyaan yang diajukan pun berbeda. Bukan “berapa nilai y bila x diketahui” melainkan “**kejadian apa yang cenderung muncul bersamaan**”. Jawabannya berbentuk **aturan asosiasi**: bila roti dan mentega dibeli, susu cenderung ikut dibeli. Teknik ini lahir dari analisis keranjang belanja, tetapi dipakai jauh lebih luas — pola gejala penyakit yang muncul bersamaan, kombinasi jawaban kuesioner, rangkaian modul yang diambil mahasiswa, atau pola kesalahan pada berkas administrasi.

Modul dipusatkan pada empat hal. **Pertama**, menyiapkan data transaksi dan memahami bentuknya. **Kedua**, menambang himpunan barang sering muncul (*frequent itemsets*) dengan Apriori dan Eclat, beserta gagasan *closed* dan *maximal* yang memampatkan hasilnya. **Ketiga**, membentuk aturan dan menafsirkan **support**, **confidence**, dan **lift** — dengan penekanan bahwa dua yang pertama saja menyesatkan. **Keempat**, memangkas dan menilai aturan: mana yang berlebihan, mana yang bermakna, dan mana yang hanya cerminan barang yang memang laris. Modul ditutup dengan **penerapan pada statistik resmi** — kejadian bersama indikator keterbatasan rumah tangga — yang memperlihatkan bahwa teknik ini bukan alat ritel semata.

Tentang data. Berkas data pada modul ini adalah **data simulasi** (`set.seed(2026)`), **bukan data nyata** dan **bukan data resmi** instansi mana pun. Beberapa kaitan antarbarang **sengaja ditanam** saat pembangkitan agar dapat diperiksa apakah algoritma menemukannya kembali; kaitan lain muncul semata dari kebetulan, dan membedakan keduanya justru menjadi salah satu latihan pada modul ini.

Identitas Modul

Mata Kuliah	:	Data Mining dan Big Data (K202505)
Pertemuan	:	Minggu ke-11
Bahan Kajian	:	Association Rule dan Frequent Itemset Mining
Sub-CPMK	:	Sub-CPMK 11
Alokasi Waktu	:	Praktikum 1 × 170 menit
Pengumpulan	:	Google Classroom STIS

11.2 Tujuan Praktikum

Setelah menyelesaikan praktikum Modul 11, mahasiswa diharapkan mampu:

1. Menjelaskan bentuk data transaksi dan membedakannya dari tabel persegi biasa.
2. Membaca data transaksi ke dalam R dari bentuk panjang maupun lebar, serta memeriksa ringkasannya.
3. Menjelaskan definisi **support**, **confidence**, dan **lift** beserta hubungan ketiganya.
4. Menambang himpunan barang sering muncul dengan **Apriori** dan **Eclat**, serta menjelaskan mengapa keduanya memberi hasil sama dengan cara kerja berbeda.
5. Membedakan himpunan **frequent**, **closed**, dan **maximal**, serta menjelaskan pemampatan yang diberikan keduanya.
6. Membentuk aturan asosiasi, mengurutkannya, dan menyaringnya menurut barang tertentu.
7. Mengenali aturan ber-*confidence* tinggi tetapi ber-*lift* sekitar satu, dan menjelaskan mengapa aturan semacam itu tidak berguna.
8. Memangkas aturan **redundan** serta memakai ukuran tambahan (*leverage*, *conviction*, uji Fisher) untuk menilai kebermaknaan.
9. Menjelaskan pengaruh ambang *support* terhadap banyaknya aturan dan terhadap kemampuan menemukan pola pada barang langka.
10. Menerapkan penambangan aturan pada **data statistik resmi** — kejadian bersama indikator keterbatasan rumah tangga — dan menerjemahkan temuannya menjadi usulan penargetan program beserta keterbatasannya.

11.3 Alat dan Bahan

11.3.1 Perangkat Lunak

- R versi 4.2 atau lebih baru dan **RStudio**.
- Paket R: **arules** (penambangan aturan asosiasi). Paket **arulesViz** bersifat pilihan untuk penyajian grafis; modul ini sengaja memakai grafik bawaan R agar dapat dijalankan tanpanya.

Kode 11.1 • R pemasangan dan pemanggilan paket

```
install.packages("arules")      # jalankan sekali saja
```

```
suppressPackageStartupMessages(library(arules))
```

11.3.2 Bahan (Berkas Data)

Berkas	Isi
data/transaksi_ritel.csv	5.279 baris; bentuk panjang : id_transaksi, item — 1.200 transaksi, 20 jenis barang

Berkas	Isi
data/keterangan_barang.csv	padanan tiap barang dengan kategori -nya (Minuman, Bahan Pokok, dan seterusnya)
data/keterbatasan_rt.csv	3.216 baris; 866 rumah tangga $\times$ 12 indikator keterbatasan — kasus statistik resmi (Bagian 11.7)

Berkas `gen_data.R` memuat kode pembangkit kedua berkas itu lengkap dengan `set.seed()`.

11.4 Instruksi Pengerjaan

1. Buat **RStudio Project** dan salin folder `data/` ke dalamnya.
2. Kerjakan Contoh secara berurutan; ketik ulang kodenya, jalankan, lalu baca kotak Interpretasi Output.
3. Kerjakan **Latihan Mandiri**, lalu **Penugasan** sebagai laporan.
4. Tuliskan jawaban, kode, output, dan interpretasi dalam satu berkas `.Rmd`, lalu **Knit** ke PDF.

Kebiasaan baik. Tetapkan ambang *support* dan *confidence* **beserta alasannya** sebelum menambang, bukan sesudah melihat hasil mana yang enak diceritakan. Menurunkan ambang sampai aturan yang diinginkan muncul adalah pencarian pola, bukan analisis.

11.5 Teori Singkat

11.5.1 Data Transaksi

Misalkan $I = \{i_1, i_2, \dots, i_d\}$ himpunan seluruh **barang** (*item*). Sebuah **transaksi** t adalah himpunan bagian dari I , dan basis data transaksi D adalah kumpulan n transaksi. **Himpunan barang** (*itemset*) $X \subseteq I$ berukuran k disebut k -itemset.

Data transaksi berbeda dari tabel persegi dalam dua hal: panjang tiap transaksi berbeda-beda, dan yang dicatat hanya **kehadiran** — ketiadaan suatu barang tidak berarti “nol”, melainkan sekadar tidak dibeli.

11.5.2 Support, Confidence, dan Lift

Support himpunan X : proporsi transaksi yang memuat X .

$$\text{supp}(X) = \frac{|\{t \in D : X \subseteq t\}|}{n}$$

Aturan asosiasi $X \Rightarrow Y$ dengan $X \cap Y = \emptyset$:

$$\text{supp}(X \Rightarrow Y) = \text{supp}(X \cup Y), \quad \text{conf}(X \Rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X)}$$

Lift membandingkan *confidence* dengan support ruas kanan:

$$\text{lift}(X \Rightarrow Y) = \frac{\text{conf}(X \Rightarrow Y)}{\text{supp}(Y)} = \frac{\text{supp}(X \cup Y)}{\text{supp}(X) \text{supp}(Y)}$$

lift = 1 berarti X dan Y muncul bersamaan persis sesering yang diharapkan bila keduanya bebas;
> 1 berarti saling menguatkan; < 1 berarti saling meniadakan.

Mengapa confidence saja menyesatkan. *Confidence* tidak memperhitungkan seberapa laris ruas kanannya. Bila 60% transaksi memuat air mineral, maka aturan apa pun $\Rightarrow$ air mineral akan ber-*confidence* sekitar 0,60 tanpa memuat informasi apa pun. Aturan semacam itu *terlihat* kuat padahal *lift*-nya mendekati 1. Karena itu **lift wajib dilaporkan bersama confidence**.

11.5.3 Sifat Apriori dan Dua Algoritma

Sifat Apriori (anti-monoton). Bila X sering muncul, maka setiap himpunan bagiannya juga sering muncul. Kontraposisinya yang dipakai: bila X *tidak* sering muncul, maka setiap superhimpunannya pasti tidak sering muncul — sehingga seluruh cabang itu dapat dipangkas tanpa diperiksa.

Apriori bekerja per tingkat: cari 1-itemset yang sering, gabungkan menjadi calon 2-itemset, hitung support-nya dengan menyapu basis data, dan seterusnya. Ia menyapu data berulang kali tetapi hemat memori.

Eclat memakai penyajian **vertikal**: tiap barang disimpan sebagai daftar id transaksi yang memuatnya. Support gabungan dua himpunan diperoleh dengan **mengiriskan** kedua daftar itu, tanpa menyapu ulang data. Ia lebih cepat pada data yang tidak terlalu besar, tetapi lebih boros memori.

Keduanya menyelesaikan masalah yang sama sehingga **hasilnya identik**; yang berbeda hanya cara mencapainya.

11.5.4 Frequent, Closed, dan Maximal

Himpunan X yang sering muncul disebut:

- **closed** bila tidak ada superhimpunan langsung yang *support*-nya sama;
- **maximal** bila tidak ada superhimpunan yang juga sering muncul.

Berlaku: $\text{maximal} \subseteq \text{closed} \subseteq \text{frequent}$. Himpunan *closed* memampatkan hasil **tanpa kehilangan informasi support**; *maximal* memampatkan lebih jauh tetapi support himpunan bagiannya tidak lagi dapat dipulihkan.

11.5.5 Aturan Redundan

Aturan $X \Rightarrow Y$ disebut **redundan** bila ada aturan yang lebih sederhana — ruas kirinya himpunan bagian — dengan *confidence* yang sama atau lebih tinggi. Menyajikan keduanya membuat laporan panjang tanpa menambah pengetahuan.

11.5.6 Ukuran Tambahan

Leverage = $\text{supp}(X \cup Y) - \text{supp}(X)\text{supp}(Y)$: selisih mutlak terhadap keadaan bebas; berguna karena *lift* tinggi pada himpunan yang sangat jarang bisa menyesatkan.

Conviction = $\frac{1 - \text{supp}(Y)}{1 - \text{conf}(X \Rightarrow Y)}$: seberapa sering aturan akan keliru bila X dan Y bebas. Bernilai ∞ bila *confidence* tepat 1.

Uji Fisher memberi nilai- p bagi hipotesis bahwa X dan Y bebas — berguna, tetapi **harus** dikoreksi terhadap pengujian ganda karena ribuan aturan diuji sekaligus.

Alur praktikum Modul 11

1. Membaca data transaksi dan memeriksa ringkasannya.
2. Menambang himpunan sering muncul dengan Apriori; memampatkannya menjadi closed dan maximal.
3. Membandingkan Apriori dan Eclat.
4. Membentuk aturan, mengurutkan menurut lift, dan menafsirkannya.
5. Memperagakan jebakan confidence tinggi dengan lift sekitar satu.
6. Memangkas aturan redundan dan menilai dengan ukuran tambahan.
7. Menelusuri pengaruh ambang support, termasuk pada barang langka.
8. Menaikkan analisis ke tingkat kategori barang.
9. Menerapkannya pada kasus statistik resmi: kejadian bersama indikator keterbatasan rumah tangga.

11.6 Contoh Praktikum dengan R

Contoh 11.1 | Membaca data transaksi dan memeriksa ringkasannya

Kode 11.2 • R membaca bentuk panjang menjadi objek transactions

```
trx <- read.transactions("data/transaksi_ritel.csv", format = "single", sep = ",",
                        header = TRUE, cols = c("id_transaksi", "item"), quote = "\"")
```

```
trx
```

```
transactions in sparse format with
1200 transactions (rows) and
20 items (columns)
```

```
cat("banyak transaksi:", nrow(trx), " banyak barang:", ncol(trx), "\n")
```

```
banyak transaksi: 1200 banyak barang: 20
```

```
summary(size(trx))
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
1.0	3.0	4.0	4.4	6.0	12.0

```
inspect(head(trx, 3))
```

items	transactionID
[1] {buah, coklat, deterjen, gula, minyak_goreng, susu}	T0001
[2] {air_mineral, beras, gula, mentega, minyak_goreng, roti, sabun}	T0002
[3] {air_mineral, beras, sampo, susu}	T0003

Interpretasi Output

Data memuat **1.200 transaksi** atas **20 jenis barang**. Isi keranjang berkisar 1 sampai 12 barang dengan median 4 — sebaran yang khas pada data ritel: banyak keranjang kecil, sedikit keranjang besar. Perhatikan bentuk penyimpanannya. Objek `transactions` disimpan sebagai matriks jarang (*sparse*): hanya kehadiran yang dicatat. Bila 1.200 transaksi ini disimpan sebagai matriks 0/1 biasa berukuran 1.200×20 , sebagian besar selnya bernilai nol — dan pada data nyata dengan puluhan ribu barang, penyimpanan padat tidak akan muat di memori. Inilah alasan `arules` memakai kelas datanya sendiri, bukan `data.frame`.

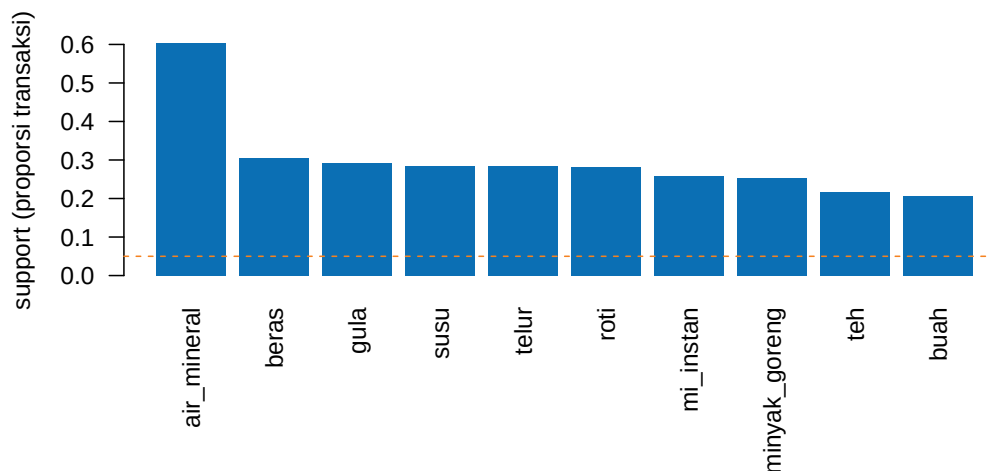
Contoh 11.2 | Melihat barang yang paling sering dibeli

Kode 11.3 • R frekuensi relatif tiap barang

```
frek <- sort(itemFrequency(trx), decreasing = TRUE)
round(frek[1:10], 3)
```

air_mineral	beras	gula	susu	telur	roti
0.604	0.304	0.293	0.285	0.283	0.282
mi_instan	minyak_goreng	teh	buah		
0.258	0.254	0.218	0.206		

```
par(mar = c(8, 4, 2, 1), family = "sans")
barplot(frek[1:10], las = 2, col = "#0B6FB4", border = NA,
        ylab = "support (proporsi transaksi)")
abline(h = 0.05, lty = 2, col = "#F58220")
```



Gambar 11.1: Sepuluh barang dengan support tertinggi

Interpretasi Output

air_mineral muncul pada **60,4%** transaksi — jauh di atas barang lain (beras 30,4%; gula 29,3%; susu 28,5%). Barang selaris ini akan menjadi sumber masalah pada penafsiran *confidence*, sebagaimana diperagakan pada Contoh 11.7.

Garis putus-putus menandai ambang support 5% yang akan dipakai nanti. Seluruh sepuluh barang teratas melampauinya, sehingga tidak ada yang tersaring pada tingkat 1-itemset.

Contoh 11.3 | Menambang himpunan barang yang sering muncul

Kode 11.4 • R Apriori dengan target frequent itemsets


```
fis <- apriori(trx, parameter = list(support = 0.05, target = "frequent itemsets",
                                   minlen = 1),
              control = list(verbose = FALSE))
cat("banyak himpunan sering muncul:", length(fis), "\n")
```

```
banyak himpunan sering muncul: 103
```

```
table(ukuran = size(fis))
```

```
ukuran
 1  2  3
20 75  8
```

```
inspect(head(sort(fis, by = "support"), 5))
```

	items	support	count
[1]	{air_mineral}	0.6042	725
[2]	{beras}	0.3042	365
[3]	{gula}	0.2933	352
[4]	{susu}	0.2850	342
[5]	{telur}	0.2833	340

```
inspect(head(sort(fis[size(fis) >= 2], by = "support"), 5))
```

	items	support	count
[1]	{air_mineral, beras}	0.1867	224
[2]	{air_mineral, gula}	0.1775	213
[3]	{air_mineral, susu}	0.1700	204
[4]	{air_mineral, telur}	0.1625	195
[5]	{air_mineral, roti}	0.1625	195

Interpretasi Output

Pada ambang support 5% ditemukan **103** himpunan sering muncul: 20 himpunan berukuran satu (yakni seluruh barang — tidak ada yang tersaring), 75 berukuran dua, dan 8 berukuran tiga. Tidak ada himpunan berukuran empat yang melampaui ambang.

Pola berbentuk piramida terbalik ini khas: makin besar himpunan, makin kecil support-nya — konsekuensi langsung sifat anti-monoton. Himpunan berpasangan yang paling sering muncul melibatkan `air_mineral`, sekali lagi karena barang itu memang laris, bukan karena ada kaitan khusus.

Contoh 11.4 | Memampatkan hasil: closed dan maximal

Kode 11.5 • R `is.closed` dan `is.maximal`

```
closed <- fis[is.closed(fis)]
maximal <- fis[is.maximal(fis)]
cat("frequent:", length(fis), " closed:", length(closed),
    " maximal:", length(maximal), "\n")
```

```
frequent: 103 closed: 103 maximal: 68
```

```
inspect(head(sort(maximal, by = "support"), 5))
```

	items	support	count
[1]	{air_mineral, mi_instan}	0.1617	194
[2]	{air_mineral, minyak_goreng}	0.1525	183
[3]	{air_mineral, buah}	0.1317	158
[4]	{air_mineral, deterjen}	0.1150	138
[5]	{air_mineral, biskuit}	0.1117	134

Interpretasi Output

Dari 103 himpunan sering muncul, **seluruhnya closed** dan **68 maximal**. Bahwa jumlah closed sama dengan jumlah frequent memberi tahu sesuatu: tidak ada himpunan yang support-nya persis sama dengan superhimpunannya. Itu wajar pada data simulasi dengan keacakan tinggi; pada data nyata — misalnya barang yang selalu dibeli berpasangan — pemampatan closed bisa jauh lebih besar. Himpunan maximal memampatkan dari 103 menjadi 68. Harganya: support himpunan bagian tidak lagi tercatat. Bila yang diperlukan hanya *daftar* pola terbesar, maximal memadai; bila support tiap himpunan bagian masih diperlukan — misalnya untuk menghitung *confidence* — pakailah closed.

Contoh 11.5 | Apriori dan Eclat memberi hasil yang sama

Kode 11.6 • R membandingkan kedua algoritma

```
fis2 <- apriori(trx, parameter = list(support = 0.05, target = "frequent itemsets",
                                     minlen = 2),
               control = list(verbose = FALSE))
ecl <- eclat(trx, parameter = list(support = 0.05, minlen = 2),
             control = list(verbose = FALSE))
```

```
cat("apriori:", length(fis2), " eclat:", length(ecl), "\n")
```

```
apriori: 83 eclat: 83
```

```
cat("himpunan yang ditemukan identik:", setequal(labels(fis2), labels(ecl)), "\n")
```

himpunan yang ditemukan identik: TRUE

```
banding <- merge(data.frame(itemset = labels(fis2), supp_apriori = quality(fis2)$support),
                 data.frame(itemset = labels(ecl), supp_eclat = quality(ecl)$support),
                 by = "itemset")
cat("selisih support terbesar:", max(abs(banding$supp_apriori - banding$supp_eclat)), "\n")
```

selisih support terbesar: 0.00000000000000002776

Interpretasi Output

Kedua algoritma menemukan **83 himpunan yang persis sama** dengan support yang sama pula (selisih nol). Ini bukan kebetulan: keduanya menyelesaikan masalah yang sama secara *eksak*, sehingga perbedaan hasil hanya mungkin terjadi bila ambangnya berbeda.

Yang membedakan adalah cara mencapainya — dan itu berdampak pada sumber daya, bukan pada jawaban. Apriori menyapu data berulang kali (hemat memori, banyak I/O); Eclat mengiriskan daftar id transaksi (sedikit I/O, boros memori). Pada data yang muat di memori, Eclat lazimnya lebih cepat; pada data yang jauh lebih besar daripada memori, pendekatan Apriori atau FP-Growth lebih sesuai. Pemilihan algoritma karena itu adalah keputusan **rekayasa**, bukan keputusan statistik.

Contoh 11.6 | Membentuk aturan asosiasi dan mengurutkannya menurut lift

Kode 11.7 • R apriori dengan target rules

```
aturan <- apriori(trx, parameter = list(support = 0.02, confidence = 0.5, minlen = 2),
                 control = list(verbose = FALSE))
cat("banyak aturan:", length(aturan), "\n")
```

banyak aturan: 162

```
inspect(head(sort(aturan, by = "lift"), 8))
```

	lhs	rhs	support	confidence	coverage	lift	count
[1]	{gula, popok}	=> {susu}	0.02333	1.0000	0.02333	3.509	28
[2]	{air_mineral, popok}	=> {susu}	0.03750	0.9184	0.04083	3.222	45
[3]	{keju, telur}	=> {roti}	0.02917	0.8974	0.03250	3.186	35
[4]	{popok}	=> {susu}	0.06000	0.8889	0.06750	3.119	72
[5]	{kopi, teh}	=> {gula}	0.04750	0.9048	0.05250	3.084	57
[6]	{air_mineral, kopi, teh}	=> {gula}	0.02250	0.8438	0.02667	2.876	27
[7]	{keju}	=> {roti}	0.06417	0.7938	0.08083	2.818	77
[8]	{gula, selai}	=> {kopi}	0.02000	0.5106	0.03917	2.773	24

Interpretasi Output

Dari 1.200 transaksi diperoleh **162 aturan** pada ambang support 2% dan confidence 0,5. Delapan aturan ber-*lift* tertinggi seluruhnya masuk akal dan — yang lebih penting — **seluruhnya merupakan kaitan yang sengaja ditanam** saat pembangkitan data:

- {gula, popok} $\Rightarrow$ {susu} — confidence 1,000, lift 3,51
- {popok} $\Rightarrow$ {susu} — confidence 0,889, lift 3,12
- {kopi, teh} $\Rightarrow$ {gula} — confidence 0,905, lift 3,08
- {keju} $\Rightarrow$ {roti} — confidence 0,794, lift 2,82

Algoritma berhasil menemukan kembali pola yang ditanam — itulah gunanya bekerja dengan data simulasi berpola diketahui: kita dapat memeriksa apakah alatnya bekerja sebagaimana mestinya sebelum memakainya pada data yang polanya tidak diketahui.

Perhatikan aturan pertama: confidence **1,000** berarti setiap transaksi yang memuat gula dan popok pasti memuat susu. Angka sempurna semacam ini pada data nyata hampir selalu patut dicurigai — bisa jadi berasal dari cacah yang sangat kecil. Support-nya 0,023, yakni hanya **28 transaksi**. Contoh 11.9 membahas hal ini.

Contoh 11.7 | Jebakan: confidence tinggi dengan lift sekitar satu

Kode 11.8 • R menyaring aturan yang ruasnya kanannya barang terlaris

```
aturan_air <- subset(aturan, rhs %in% "air_mineral")
cat("aturan yang menyimpulkan air_mineral:", length(aturan_air), "\n")
```

```
aturan yang menyimpulkan air_mineral: 121
```

```
inspect(head(sort(aturan_air, by = "confidence"), 5))
```

	lhs	rhs	support	confidence	coverage	lift	count
[1]	{beras, deterjen}	=> {air_mineral}	0.04167	0.7246	0.05750	1.199	50
[2]	{biskuit, deterjen}	=> {air_mineral}	0.02083	0.7143	0.02917	1.182	25
[3]	{biskuit, mi_instan}	=> {air_mineral}	0.03917	0.7121	0.05500	1.179	47
[4]	{biskuit, sampo}	=> {air_mineral}	0.02000	0.7059	0.02833	1.168	24
[5]	{buah, minyak_goreng}	=> {air_mineral}	0.03917	0.7015	0.05583	1.161	47

```
cat("\nsupport air_mineral:", round(itemFrequency(trx)["air_mineral"], 4), "\n")
```

```
support air_mineral: 0.6042
```

Interpretasi Output

Lima aturan ini ber-*confidence* **0,70 sampai 0,72** — lebih tinggi daripada kebanyakan aturan pada Contoh 11.6. Seorang analis yang mengurutkan aturan menurut *confidence* akan menempatkannya di puncak daftar dan menyimpulkan, misalnya, bahwa pembeli beras dan deterjen "cenderung kuat" membeli air mineral.

Kesimpulan itu **keliru**. Support air mineral adalah 0,604 — artinya 60% transaksi memuatnya *tanpa syarat apa pun*. Confidence 0,72 hanya sedikit di atas angka itu, dan *lift*-nya memperlihatkan: **1,16 sampai 1,20** — praktis tidak ada kaitan. Bandingkan dengan lift 3,51 pada aturan yang benar-benar bermakna.

Terjemahan bisnisnya tegas: menaruh air mineral di dekat beras tidak akan menaikkan penjualan, sebab orang membeli air mineral memang karena air mineral. Kaidahnya: **urutkan menurut lift, bukan confidence**, dan selalu bandingkan confidence dengan support ruasnya.

Contoh 11.8 | Memangkas aturan redundan

Kode 11.9 • R is.redundant

```
cat("sebelum:", length(aturan), " redundan:", sum(is.redundant(aturan)), "\n")
```

```
sebelum: 162 redundan: 76
```

```
aturan_bersih <- aturan[!is.redundant(aturan)]
cat("setelah dipangkas:", length(aturan_bersih), "\n")
```

```
setelah dipangkas: 86
```

```
contoh_redundan <- aturan[is.redundant(aturan)]
inspect(head(sort(contoh_redundan, by = "lift"), 3))
```

lhs	rhs	support	confidence	coverage	lift	count
[1] {air_mineral, kopi, teh}	=> {gula}	0.02250	0.8438	0.02667	2.876	27
[2] {air_mineral, keju}	=> {roti}	0.04000	0.7742	0.05167	2.749	48
[3] {kopi, telur}	=> {gula}	0.03667	0.6769	0.05417	2.308	44

Interpretasi Output

Dari 162 aturan, **76 di antaranya redundan** — hampir separuh. Setelah dipangkas tersisa **86 aturan**. Contoh yang lazim: bila {popok} $\Rightarrow$ {susu} sudah ber-*confidence* 0,889, maka {popok, biskuit} $\Rightarrow$ {susu} yang ber-*confidence* serupa tidak menambah pengetahuan — biskuit tidak memberi sumbangan apa pun; ia hanya menumpang pada kaitan popok-susu.

Pemangkasan ini bukan sekadar merapikan tampilan. Laporan yang memuat 162 aturan menyembunyikan temuan sesungguhnya di antara pengulangan, dan pembaca dapat keliru menyangka ada 162

pola berbeda padahal hanya ada beberapa puluh.

Contoh 11.9 | Menilai kebermaknaan dengan ukuran tambahan

Kode 11.10 • R leverage, conviction, dan uji Fisher

```
lima <- head(sort(aturan_bersih, by = "lift"), 5)
tambahan <- interestMeasure(lima, c("support", "confidence", "lift", "leverage",
                                     "conviction", "fishersExactTest"), trx)
hasil <- cbind(aturan = labels(lima), round(tambahan, 4))
print(hasil, row.names = FALSE)
```

	aturan	support	confidence	lift	leverage	conviction	fishersExactTest
	{gula,popok} => {susu}	0.0233	1.0000	3.509	0.0167	Inf	0
	{air_mineral,popok} => {susu}	0.0375	0.9184	3.222	0.0259	8.759	0
	{keju,telur} => {roti}	0.0292	0.8974	3.186	0.0200	7.004	0
	{popok} => {susu}	0.0600	0.8889	3.119	0.0408	6.435	0
	{kopi,teh} => {gula}	0.0475	0.9048	3.084	0.0321	7.420	0

```
cat("\ncacah transaksi pendukung:", quality(lima)$count, "\n")
```

```
cacah transaksi pendukung: 28 45 35 72 57
```

Interpretasi Output

Kelima ukuran menceritakan hal yang berbeda dan sebaiknya dibaca bersama.

Lift tertinggi (3,51) dimiliki {gula, popok} $\Rightarrow$ {susu}, tetapi **leverage**-nya justru *terkecil* (0,0167). Sebabnya: leverage mengukur selisih **mutlak** terhadap keadaan bebas, sehingga aturan yang hanya didukung 28 transaksi tidak dapat memberi selisih besar. Bandingkan {popok} $\Rightarrow$ {susu} yang lift-nya lebih rendah (3,12) tetapi leverage-nya lebih dari dua kali lipat (0,0408) karena didukung 72 transaksi. Bagi keputusan bisnis, **leverage lebih relevan** — ia mendekati banyaknya transaksi tambahan yang dapat diharapkan. Lift menjawab "seberapa kuat kaitannya"; leverage menjawab "seberapa besar dampaknya".

Conviction bernilai ∞ pada aturan berconfidence 1 — akibat pembagian dengan nol pada rumusnya, dan itu petunjuk bahwa aturannya berdasar cacah kecil.

Uji Fisher memberi nilai-*p* praktis nol untuk kelimanya. Angka itu harus dibaca hati-hati: ratusan aturan diuji sekaligus, sehingga tanpa koreksi pengujian ganda (misalnya Bonferroni atau kendali FDR) sebagian nilai-*p* kecil pasti muncul dari kebetulan. Nilai-*p* di sini berguna untuk *menyaring*, bukan untuk *membuktikan*.

Contoh 11.10 | Pengaruh ambang support

Kode 11.11 • R menelusuri empat ambang

```

ambang <- c(0.10, 0.05, 0.02, 0.01)
telusur <- t(sapply(angbang, function(s) {
  f <- apriori(trx, parameter = list(support = s, target = "frequent itemsets"),
    control = list(verbose = FALSE))
  a <- apriori(trx, parameter = list(support = s, confidence = 0.5, minlen = 2),
    control = list(verbose = FALSE))
  c(support = s, itemsets = length(f), aturan = length(a))
}))
print(telusur, row.names = FALSE)

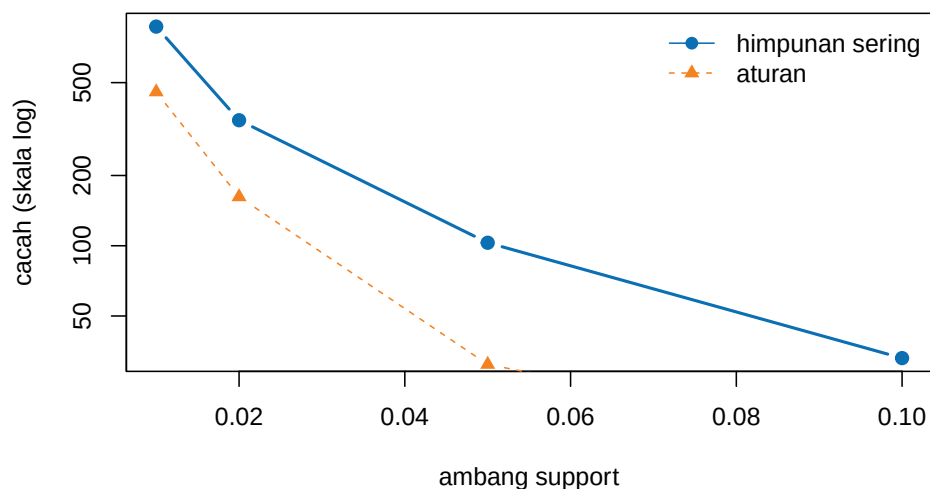
```

	support	itemsets	aturan
[1,]	0.10	33	14
[2,]	0.05	103	31
[3,]	0.02	345	162
[4,]	0.01	869	456

```

par(family = "sans", mar = c(4, 4, 1, 1))
plot(telusur[, "support"], telusur[, "itemsets"], type = "b", pch = 19, lwd = 2,
  log = "y", col = "#0B6FB4", xlab = "ambang support", ylab = "cacah (skala log)")
lines(telusur[, "support"], telusur[, "aturan"], type = "b", pch = 17, lty = 2,
  col = "#F58220")
legend("topright", c("himpunan sering", "aturan"), bty = "n",
  col = c("#0B6FB4", "#F58220"), lty = 1:2, pch = c(19, 17))

```



Gambar 11.2: Banyaknya himpunan sering muncul dan aturan menurut ambang support

Interpretasi Output

Menurunkan ambang support dari 0,10 ke 0,01 menaikkan himpunan sering muncul dari **33** menjadi **869** — dua puluh enam kali lipat — dan aturan dari **14** menjadi **456**. Pertumbuhannya jauh lebih cepat daripada linear; perhatikan sumbu tegak berskala logaritma.

Inilah pertukaran pokok pada penambahan aturan asosiasi. Ambang tinggi memberi sedikit aturan yang seluruhnya berdasar cacah besar, tetapi **membutakan kita terhadap pola pada barang langka**. Ambang rendah menemukan pola langka itu, tetapi menghasilkan ribuan aturan yang sebagian besar kebetulan — dan menyaringnya menjadi pekerjaan tersendiri.

Kaidah praktis: tetapkan ambang dari **cacah minimum yang bermakna**, bukan dari proporsi. Bila keputusan baru layak diambil atas dasar minimal 30 transaksi, maka pada $n = 1.200$ ambangnya $30/1200 = 0,025$. Cara ini memaksa alasannya dinyatakan.

Contoh 11.11 | Barang langka: menurunkan ambang secara terarah

Kode 11.12 • R menambang pola khusus popok

```
cat("support popok:", round(itemFrequency(trx)["popok"], 4), "\n")
```

```
support popok: 0.0675
```

```
aturan_langka <- apriori(trx, parameter = list(support = 0.01, confidence = 0.5,
                                              minlen = 2),
                        control = list(verbose = FALSE))
```

```
pola_popok <- subset(aturan_langka, lhs %in% "popok" & rhs %in% "susu")
```

```
pola_popok <- pola_popok[!is.redundant(pola_popok)]
```

```
cat("aturan popok -> susu setelah dipangkas:", length(pola_popok), "\n")
```

```
aturan popok -> susu setelah dipangkas: 10
```

```
inspect(sort(pola_popok, by = "lift"))
```

	lhs	rhs	support	confidence	coverage	lift	count
[1]	{gula, popok}	=> {susu}	0.02333	1.0000	0.02333	3.509	28
[2]	{biskuit, popok}	=> {susu}	0.01500	0.9474	0.01583	3.324	18
[3]	{buah, popok}	=> {susu}	0.01417	0.9444	0.01500	3.314	17
[4]	{popok, teh}	=> {susu}	0.01333	0.9412	0.01417	3.302	16
[5]	{mi_instan, popok}	=> {susu}	0.01167	0.9333	0.01250	3.275	14
[6]	{air_mineral, beras, popok}	=> {susu}	0.01167	0.9333	0.01250	3.275	14
[7]	{popok, sabun}	=> {susu}	0.01000	0.9231	0.01083	3.239	12
[8]	{air_mineral, popok}	=> {susu}	0.03750	0.9184	0.04083	3.222	45
[9]	{minyak_goreng, popok}	=> {susu}	0.01667	0.9091	0.01833	3.190	20
[10]	{popok}	=> {susu}	0.06000	0.8889	0.06750	3.119	72

Interpretasi Output

Popok hanya muncul pada **6,75%** transaksi — barang langka. Pada ambang support 5% (Contoh 11.3), pola apa pun yang melibatkan popok bersama barang lain akan tersaring habis, padahal kaitan popok-susu adalah salah satu yang **paling kuat** dalam data ini (lift 3,12).

Setelah ambang diturunkan ke 1% dan aturan redundan dipangkas, tersisa aturan inti {popok} $\Rightarrow$ {susu}. Perhatikan bahwa pemangkasan redundansi bekerja persis sebagaimana diharapkan: puluhan aturan bertambah barang lain — biskuit, kopi, sabun — lenyap karena tidak menambah apa pun atas aturan inti tersebut.

Pelajarannya: ambang support tunggal memperlakukan seluruh barang sama rata, padahal kepentingannya berbeda. Popok bermargin tinggi dan pembelinya berpola khas; melewatkannya karena ambang yang seragam adalah kerugian nyata. Penanganan yang lazim: menambang dua kali — sekali dengan ambang umum, sekali dengan ambang rendah yang **dibatasi** pada barang yang memang menjadi perhatian.

Contoh 11.12 | Menaikkan analisis ke tingkat kategori

Kode 11.13 • R agregasi barang menjadi kategori

```
ket <- read.csv("data/keterangan_barang.csv", stringsAsFactors = FALSE)
itemInfo(trx)$kategori <- ket$kategori[match(itemInfo(trx)$labels, ket$item)]
head(itemInfo(trx), 4)
```

	labels	kategori
1	air_mineral	Minuman
2	beras	Bahan Pokok
3	biskuit	Kudapan
4	buah	Kudapan

```
trx_kat <- aggregate(trx, by = "kategori")
cat("banyak kategori:", ncol(trx_kat), "\n")
```

```
banyak kategori: 5
```

```
round(sort(itemFrequency(trx_kat), decreasing = TRUE), 3)
```

Minuman	Bahan Pokok	Perawatan	Kudapan Roti & Olesan
0.829	0.799	0.471	0.438

```
aturan_kat <- apriori(trx_kat, parameter = list(support = 0.05, confidence = 0.5,
                                                minlen = 2),
                    control = list(verbose = FALSE))
aturan_kat <- aturan_kat[!is.redundant(aturan_kat)]
cat("aturan tingkat kategori:", length(aturan_kat), "\n")
```

```
aturan tingkat kategori: 20
```

```
inspect(head(sort(aturan_kat, by = "lift"), 5))
```

lhs	rhs	support	confidence	coverage
[1] {Bahan Pokok, Kudapan, Minuman, Roti & Olesan}	=> {Perawatan}	0.06333	0.5241	0.1208
[2] {Bahan Pokok, Kudapan, Roti & Olesan}	=> {Perawatan}	0.07667	0.5227	0.1467
[3] {Bahan Pokok, Kudapan, Minuman}	=> {Perawatan}	0.14833	0.5174	0.2867
[4] {Kudapan, Minuman}	=> {Perawatan}	0.18417	0.5104	0.3608
[5] {Kudapan, Roti & Olesan}	=> {Perawatan}	0.09667	0.5088	0.1900

lift	count
[1] 1.113	76
[2] 1.110	92
[3] 1.099	178
[4] 1.084	221
[5] 1.081	116

Interpretasi Output

Setelah 20 barang diringkas menjadi **5 kategori**, support tiap kategori melonjak — sebuah kategori dianggap hadir bila *salah satu* anggotanya dibeli. Akibatnya aturan pada tingkat kategori ber-support tinggi tetapi ber-lift rendah: kategori yang luas hampir selalu muncul bersamaan.

Ini memperlihatkan pertukaran yang khas pada agregasi. Tingkat kategori memberi pola yang **stabil dan mudah diceritakan** kepada pimpinan, tetapi **menyembunyikan kaitan yang tajam**: kaitan pokok-susu lenyap sepenuhnya karena keduanya melebur ke dalam Perawatan dan Minuman. Sebaliknya tingkat barang memberi kaitan tajam tetapi rapuh terhadap ambang.

Praktik yang baik pada laporan: sajikan **keduanya** — pola kategori sebagai gambaran umum, dan sejumlah kecil aturan tingkat barang berdaya jelas tinggi sebagai temuan yang dapat ditindaklanjuti.

11.7 Penerapan pada Statistik Resmi

Kasus: Kejadian Bersama Indikator Keterbatasan Rumah Tangga

Analisis keranjang belanja pada bagian sebelumnya memakai istilah ritel karena di situlah teknik ini lahir. Bentuk datanya — kumpulan himpunan dengan panjang berbeda-beda — sesungguhnya jauh lebih umum. Pada pendataan sosial-ekonomi, tiap rumah tangga dapat dipandang sebagai satu “keranjang” yang berisi **indikator keterbatasan** yang dialaminya: air tidak layak, sanitasi tidak layak, anak putus sekolah, pekerja informal, dan seterusnya.

Pertanyaannya menjadi sangat relevan bagi perencanaan program: **keterbatasan apa yang cenderung dialami bersamaan?** Jawabannya menentukan apakah program sebaiknya dirancang sektoral — air sendiri, pendidikan sendiri — atau sebagai paket terpadu bagi rumah tangga yang mengalami beberapa keterbatasan sekaligus.

Berkas keterbatasan_rt.csv memuat 866 rumah tangga dengan 12 indikator. **Data ini ilustratif hasil simulasi**, dirancang menyerupai pola yang lazim dijumpai pada pendataan sosial-ekonomi — bukan data resmi.

Contoh 11.13 | Membaca data keterbatasan dan menelaah sebarannya

Kode 11.14 • R transaksi berisi indikator, bukan barang

```
rt <- read.transactions("data/keterbatasan_rt.csv", format = "single", sep = ",",
                        header = TRUE, cols = c("id_rt", "indikator"), quote = "\"")
cat("rumah tangga:", nrow(rt), " indikator:", ncol(rt), "\n")
```

```
rumah tangga: 866 indikator: 12
```

```
cat("rata-rata indikator per rumah tangga:", round(mean(size(rt)), 2), "\n")
```

```
rata-rata indikator per rumah tangga: 3.71
```

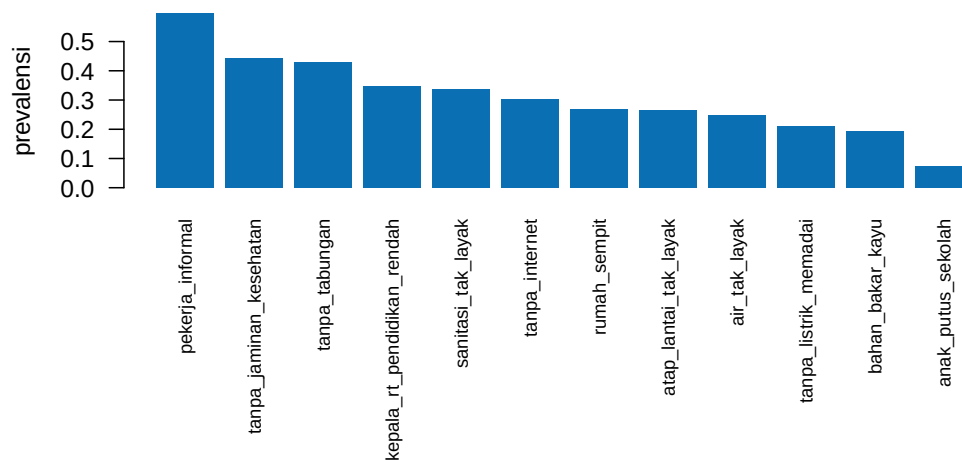
```
table(banyak_indikator = size(rt))
```

```
banyak_indikator
 1  2  3  4  5  6  7  8  9 10
85 146 198 176 119 80 36 21  4  1
```

```
round(sort(itemFrequency(rt), decreasing = TRUE), 3)
```

pekerja_informal	tanpa_jaminan_kesehatan	tanpa_tabungan
0.597	0.442	0.428
kepala_rt_pendidikan_rendah	sanitasi_tak_layak	tanpa_internet
0.349	0.337	0.301
rumah_sempit	atap_lantai_tak_layak	air_tak_layak
0.268	0.264	0.247
tanpa_listrik_memadai	bahan_bakar_kayu	anak_putus_sekolah
0.211	0.193	0.075

```
par(mar = c(11, 4, 2, 1), family = "sans")
barplot(sort(itemFrequency(rt), decreasing = TRUE), las = 2, col = "#0B6FB4",
        border = NA, ylab = "prevalensi", cex.names = 0.75)
```



Gambar 11.3: Prevalensi tiap indikator keterbatasan

Interpretasi Output

Sebanyak 866 rumah tangga mengalami sekurang-kurangnya satu keterbatasan, dengan rata-rata **3,71 indikator** per rumah tangga. Sebaran cacah indikator itu sendiri sudah informatif: sebagian rumah tangga hanya mengalami satu-dua keterbatasan, sebagian lain mengalami tujuh atau lebih. Kelompok kedua inilah sasaran program terpadu.

Prevalensi tertinggi dimiliki pekerja_informal (59,7%), disusul tanpa_jaminan_kesehatan (44,2%) dan tanpa_tabungan (42,8%). Yang paling jarang adalah anak_putus_sekolah (7,5%).

Perhatikan bahwa indikator dengan prevalensi tinggi akan menimbulkan persoalan yang sama seperti air mineral pada Contoh 11.7: aturan apa pun yang menyimpulkan pekerja_informal akan ber-*confidence* tinggi tanpa memuat informasi. Kaidah membaca *lift* berlaku persis sama di sini.

Contoh 11.14 | Menemukan kelompok keterbatasan yang saling berkaitan

Kode 11.15 • R aturan sederhana satu indikator ke satu indikator

```
r1 <- apriori(rt, parameter = list(support = 0.05, confidence = 0.5,
                                   minlen = 2, maxlen = 2),
              control = list(verbose = FALSE))
cat("aturan satu-ke-satu:", length(r1), "\n")

aturan satu-ke-satu: 20

inspect(head(sort(r1, by = "lift"), 6))
```

lhs	rhs	support	confidence	coverage	lift
[1] {tanpa_listrik_memadai}	=> {bahan_bakar_kayu}	0.11316	0.5355	0.21132	2.777
[2] {bahan_bakar_kayu}	=> {tanpa_listrik_memadai}	0.11316	0.5868	0.19284	2.777
[3] {anak_putus_sekolah}	=> {kepala_rt_pendidikan_rendah}	0.06697	0.8923	0.07506	2.559
[4] {air_tak_layak}	=> {sanitasi_tak_layak}	0.19284	0.7804	0.24711	2.314
[5] {sanitasi_tak_layak}	=> {air_tak_layak}	0.19284	0.5719	0.33718	2.314
[6] {atap_lantai_tak_layak}	=> {sanitasi_tak_layak}	0.18476	0.6987	0.26443	2.072
count					
[1]	98				
[2]	98				
[3]	58				
[4]	167				
[5]	167				
[6]	160				

Interpretasi Output

Aturan sederhana — satu indikator menyimpulkan satu indikator — paling mudah disampaikan kepada pengguna data, dan di sini sudah cukup tajam. Tiga kelompok muncul dengan jelas:

Kelompok energi rumah tangga. $\text{tanpa_listrik_memadai} \Leftrightarrow \text{bahan_bakar_kayu}$ saling berkaitan dua arah dengan *lift* 2,78: rumah tangga yang listriknya terbatas cenderung memasak dengan kayu, dan sebaliknya.

Kelompok permukiman. $\text{air_tak_layak} \Rightarrow \text{sanitasi_tak_layak}$ ber-*confidence* 0,780 dan *lift* 2,31; arah sebaliknya ber-*confidence* 0,572 dengan *lift* yang sama. Perhatikan asimetri *confidence* ini: 78% rumah tangga berair tak layak juga bersanitasi tak layak, tetapi hanya 57% yang sebaliknya — sebab sanitasi tak layak lebih umum (33,7%) daripada air tak layak (24,7%).

Kelompok pendidikan. $\text{anak_putus_sekolah} \Rightarrow \text{kepala_rt_pendidikan_rendah}$ ber-*confidence* 0,892 dan *lift* 2,56 — hampir sembilan dari sepuluh rumah tangga dengan anak putus sekolah dikepalai orang berpendidikan rendah.

Yang tidak boleh disimpulkan: bahwa pendidikan rendah kepala rumah tangga *menyebabkan* anak putus sekolah. Aturan ini simetris dalam hal kejadian bersama — keduanya dapat sama-sama bersumber pada keadaan ekonomi, akses sekolah, atau faktor lain yang tidak terekam. Yang sah disimpulkan: keduanya **menumpuk pada rumah tangga yang sama**, sehingga program yang menysasar salah satunya akan menjumpai yang lain.

Contoh 11.15 | Menelusuri indikator langka: anak putus sekolah

Kode 11.16 • R menurunkan ambang secara terarah

```
cat("prevalensi anak_putus_sekolah:", round(itemFrequency(rt)["anak_putus_sekolah"], 4), "\n")
```

```
prevalensi anak_putus_sekolah: 0.0751
```

```
r_aps <- apriori(rt, parameter = list(support = 0.02, confidence = 0.5, minlen = 2),
               control = list(verbose = FALSE))
r_aps <- subset(r_aps, lhs %in% "anak_putus_sekolah")
r_aps <- r_aps[!is.redundant(r_aps)]
cat("aturan setelah dipangkas:", length(r_aps), "\n")
```

```
aturan setelah dipangkas: 25
```

```
inspect(head(sort(r_aps, by = "lift"), 3))
```

lhs	rhs	support	confidence	coverage	lift	count
[1] {anak_putus_sekolah, pekerja_informal, tanpa_jaminan_kesehatan, tanpa_tabungan}	=> {kepala_rt_pendidikan_rendah}	0.02079	1.0000	0.02079	2.868	18
[2] {anak_putus_sekolah, tanpa_jaminan_kesehatan, tanpa_tabungan}	=> {kepala_rt_pendidikan_rendah}	0.02425	0.9545	0.02540	2.737	21
[3] {anak_putus_sekolah, pekerja_informal, tanpa_jaminan_kesehatan}	=> {kepala_rt_pendidikan_rendah}	0.02309	0.9524	0.02425	2.731	20

Interpretasi Output

Prevalensi anak putus sekolah hanya **7,5%** — indikator langka, persis keadaan yang dibahas pada Contoh 11.10. Pada ambang 5%, kombinasi apa pun yang melibatkannya akan tersaring habis.

Setelah ambang diturunkan ke 2% secara terarah, muncul pola yang tajam. Rumah tangga dengan anak putus sekolah yang **sekaligus** berpekerja informal, tanpa jaminan kesehatan, dan tanpa tabungan — 18 rumah tangga — **seluruhnya** dikepalai orang berpendidikan rendah (*confidence* 1,000, *lift* 2,87). Angka *confidence* sempurna itu wajib dibaca dengan hati-hati: ia berdasar **18 rumah tangga**. Untuk perencanaan program, cacah sekecil itu memadai sebagai petunjuk arah tetapi tidak memadai sebagai dasar penetapan sasaran wilayah. Yang tepat dilaporkan: "terdapat indikasi penumpukan keterbatasan pendidikan dan ekonomi pada sejumlah kecil rumah tangga; diperlukan pendataan lanjutan untuk memastikannya".

Inilah alasan indikator langka menuntut penanganan tersendiri. Menaikkan ambang demi kerapian akan menghapus justru kelompok yang paling membutuhkan — dan pada statistik resmi, kelompok itulah yang menjadi alasan pendataan dilakukan.

Contoh 11.16 | Dari aturan menjadi usulan program

Kode 11.17 · R menyusun ringkasan yang dapat ditindaklanjuti

```
r_all <- apriori(rt, parameter = list(support = 0.05, confidence = 0.6, minlen = 2),
      control = list(verbose = FALSE))
r_all <- r_all[!is.redundant(r_all)]
cat("aturan setelah dipangkas:", length(r_all), "\n")
```

```
aturan setelah dipangkas: 47
```

```
q <- interestMeasure(r_all, c("support", "confidence", "lift", "leverage"), rt)
ringkas <- data.frame(aturan = labels(r_all), round(q, 4),
      rt = quality(r_all)$count)
ringkas <- ringkas[order(-ringkas$leverage), ]
print(head(ringkas, 6), row.names = FALSE)
```

```
              aturan support confidence lift
      {air_tak_layak} => {sanitasi_tak_layak} 0.1928    0.7804 2.314
      {atap_lantai_tak_layak} => {sanitasi_tak_layak} 0.1848    0.6987 2.072
      {tanpa_jaminan_kesehatan} => {pekerja_informal} 0.3418    0.7728 1.295
      {air_tak_layak,atap_lantai_tak_layak} => {sanitasi_tak_layak} 0.1155    0.9091 2.696
      {atap_lantai_tak_layak,sanitasi_tak_layak} => {air_tak_layak} 0.1155    0.6250 2.529
      {tanpa_jaminan_kesehatan,tanpa_tabungan} => {pekerja_informal} 0.1582    0.8671 1.452
leverage  rt
      0.1095 167
      0.0956 160
      0.0778 296
      0.0726 100
      0.0698 100
      0.0493 137
```

```
# berapa rumah tangga mengalami keterbatasan permukiman sekaligus
paket_permukiman <- c("air_tak_layak", "sanitasi_tak_layak")
n_paket <- sum(apply(as(rt[, paket_permukiman], "matrix"), 1, all))
cat("\nrumah tangga dengan air DAN sanitasi tak layak:", n_paket,
      sprintf("(%.1f persen)\n", 100 * n_paket / nrow(rt)))
```

```
rumah tangga dengan air DAN sanitasi tak layak: 167 (19.3 persen)
```

Interpretasi Output

Pengurutan menurut **leverage** — bukan *lift* — memberi daftar yang tepat untuk perencanaan, sebab leverage mendekati **berapa banyak rumah tangga tambahan** yang mengalami kejadian bersama itu dibandingkan bila kedua keterbatasan tidak berkaitan. Aturan ber-*lift* sangat tinggi tetapi hanya mencakup belasan rumah tangga tidak akan mengubah rancangan program.

Angka terakhir menerjemahkan temuan menjadi bilangan yang dapat dipakai: **167 rumah tangga** (19,3%) mengalami air *dan* sanitasi tak layak sekaligus. Bagi perencana, itulah ukuran sasaran paket terpadu air-sanitasi — bukan menjumlahkan sasaran program air dan program sanitasi secara terpi-

sah, yang akan menghitung ganda rumah tangga tersebut.

Tiga usulan yang dapat ditarik:

1. **Paket permukiman** (air, sanitasi, atap-lantai) — ketiganya menumpuk pada rumah tangga yang sama; menanganinya dalam satu kunjungan lebih murah daripada tiga kunjungan terpisah.
2. **Paket energi** (listrik memadai dan bahan bakar bersih) — kaitan dua arah dengan *lift* 2,78.
3. **Penyaringan berlapis untuk pendidikan** — rumah tangga dengan kepala berpendidikan rendah dan tanpa tabungan dipakai sebagai penyaring awal untuk menemukan anak putus sekolah, yang prevalensinya terlalu rendah untuk disasar langsung.

Keterbatasan yang wajib menyertai: seluruh temuan bersifat **asosiatif**. Data lintas seksi tanpa perlakuan yang diacak tidak memungkinkan penarikan kesimpulan sebab-akibat, sehingga tidak dapat disimpulkan bahwa memperbaiki satu keterbatasan akan memperbaiki yang lain. Yang sah: keduanya menumpuk pada rumah tangga yang sama, sehingga **penargetan** dapat dilakukan bersama.

Kesalahan Umum.

1. Mengurutkan aturan menurut **confidence** lalu menyimpulkan aturan teratas paling penting; barang terlaris akan selalu mendominasi (Contoh 11.7).
2. Melaporkan *lift* tanpa *support* atau cacah pendukung; *lift* tinggi pada 5 transaksi bukan temuan.
3. Menurunkan ambang support sampai aturan yang diinginkan muncul, lalu melaporkannya seolah-olah ambangnya ditetapkan sejak awal.
4. Menyajikan seluruh aturan tanpa memangkas yang redundan; separuh daftar dapat berupa pengulangan.
5. Menafsirkan aturan asosiasi sebagai **sebab-akibat**; $X \Rightarrow Y$ hanya berarti keduanya sering muncul bersamaan, dan aturan sebaliknya sering juga berlaku.
6. Lupa bahwa aturan tidak simetris pada *confidence* — $\text{conf}(X \Rightarrow Y) \neq \text{conf}(Y \Rightarrow X)$ — meskipun *lift*-nya sama.
7. Memakai nilai- p uji Fisher tanpa koreksi pengujian ganda padahal ratusan aturan diuji.
8. Menganggap ketiadaan barang sebagai "nol"; data transaksi hanya mencatat kehadiran.
9. Menambang pada tingkat kategori lalu menyimpulkan tentang barang tertentu (Contoh 11.11).

11.8 Latihan Mandiri

Latihan 11.1 | Menyiapkan data dan menelaah frekuensi

Kerjakan dengan data yang sama.

- (a) Baca ulang `data/transaksi_ritel.csv` menjadi objek `transactions`, lalu laporkan banyaknya transaksi, banyaknya barang, dan sebaran ukuran keranjang.
- (b) Hitung support tiap barang. Berapa banyak barang yang support-nya di bawah $0{,}10$?
- (c) Gambar diagram batang sepuluh barang dengan support **terendah**. Barang mana yang paling langka, dan apa akibatnya bagi penambangan pada ambang 5% ?

Latihan 11.2 | Menambang dan memampatkan himpunan sering muncul

- (a) Tambang himpunan sering muncul pada ambang support $0{,}03$; laporkan cacahnya menurut ukuran himpunan.
- (b) Hitung berapa yang *closed* dan berapa yang *maximal*. Berapa persen pemampatan yang diberikan masing-masing?
- (c) Jalankan `ec1at()` pada ambang yang sama, lalu buktikan hasilnya identik dengan Apriori. Jelaskan mengapa demikian.

Latihan 11.3 | Membentuk dan menyaring aturan

- (a) Bentuk aturan pada support $0{,}02$ dan confidence $0{,}6$. Berapa banyak yang diperoleh, dan berapa yang redundan?
- (b) Setelah dipangkas, tampilkan sepuluh aturan ber-*lift* tertinggi. Bandingkan dengan sepuluh aturan ber-*confidence* tertinggi — berapa yang beririsan, dan apa artinya?
- (c) Saring aturan yang ruas kanannya susu. Aturan mana yang ber-*leverage* tertinggi, dan mengapa itu berbeda dari yang ber-*lift* tertinggi?

Latihan 11.4 | Membetulkan kode yang salah

Kode berikut memuat **empat kesalahan**; temukan, perbaiki, jalankan versi yang benar, dan jelaskan akibat tiap kesalahan bagi kesimpulan bisnis.

Kode 11.18 • R kode yang harus dibetulkan

```
trx <- read.csv("data/transaksi_ritel.csv")           # (1)
r <- apriori(trx, parameter = list(support = 0.001, confidence = 0.9)) # (2)
inspect(head(sort(r, by = "confidence"), 10))         # (3)
# "aturan teratas berconfidence 0,95: pelanggan yang membeli X PASTI membeli Y,
# jadi promosi X akan menaikkan penjualan Y"         # (4)
```

Petunjuk: satu baris membaca data sebagai data.frame biasa, bukan transactions; satu baris memakai ambang support yang teramat rendah tanpa alasan; satu baris mengurutkan menurut ukuran yang keliru; dan satu kesimpulan menarik klaim sebab-akibat dari aturan asosiasi.

11.9 Penugasan

Konteks: Penataan Rak dan Paket Promosi

Sebuah jaringan ritel meminta Anda menyusun **usulan penataan rak dan paket promosi** berdasarkan 1.200 transaksi yang tersedia. Manajemen meminta usulan yang **sedikit tetapi berdasar**, bukan daftar panjang aturan. Anda berperan sebagai analis data yang menyusun laporannya.

Tugas 11.1 | Penyiapan dan penelaahan

1. Baca data transaksi; laporkan banyaknya transaksi, barang, dan sebaran ukuran keranjang.
2. Tampilkan support seluruh barang. Tetapkan ambang support **beserta alasannya**, dinyatakan sebagai cacah minimum transaksi yang Anda anggap layak menjadi dasar keputusan.

Tugas 11.2 | Penambahan dan pemangkasan

3. Tambang himpunan sering muncul pada ambang yang Anda tetapkan; laporkan cacahnya menurut ukuran, serta berapa yang *closed* dan *maximal*.
4. Bentuk aturan asosiasi; laporkan banyaknya sebelum dan sesudah pemangkasan redundansi.
5. Sajikan sepuluh aturan terbaik menurut **lift**, lengkap dengan support, confidence, lift, leverage, dan cacah pendukung.

Tugas 11.3 | Penilaian dan penyaringan

6. Tunjukkan **satu** aturan ber-confidence tinggi tetapi ber-lift mendekati 1, lalu jelaskan mengapa aturan itu tidak layak dipakai.
7. Bandingkan urutan menurut *lift* dan menurut *leverage*. Bila keduanya berbeda, aturan mana yang Anda usulkan untuk ditindaklanjuti, dan mengapa?
8. Lakukan analisis pada tingkat **kategori**, lalu bandingkan temuannya dengan tingkat barang. Pola apa yang muncul dan pola apa yang hilang?

Tugas 11.4 | Usulan dan keterbatasan

9. Susun **tiga** usulan konkret — penataan rak atau paket promosi — masing-masing disertai aturan pendukung beserta angkanya, dan perkiraan dampaknya berdasarkan *leverage*.
10. Tulis bagian **keterbatasan** (maksimal 250 kata) yang memuat sekurang-kurangnya: sifat asosiatif (bukan sebab-akibat) temuan ini; pengaruh pilihan ambang; risiko pengujian ganda; dan bahwa data bersifat

simulasi.

Ketentuan Pengumpulan

- Berkas .Rmd beserta hasil cetak PDF, dinamai Tugas11_NIM_Nama.
- Setiap keluaran R wajib disertai interpretasi dengan kalimat sendiri.
- Seluruh *chunk* harus dapat dijalankan ulang dari awal tanpa galat.

11.9.1 Rubrik Penilaian

Komponen	Indikator	Bobot
Ketepatan analisis	Kebenaran penyiapan data, penambangan, dan pemangkasan	30%
Penetapan ambang	Ambang ditetapkan lebih dahulu dan beralasan	15%
Ketepatan interpretasi	Lift dibaca bersama support; jebakan confidence dikenali; kausalitas tidak diklaim	30%
Mutu usulan	Usulan sedikit, konkret, dan didukung angka	15%
Ketepatan waktu	Dikumpulkan sesuai tenggat	10%

11.10 Ringkasan Modul

Daftar periksa penguasaan Modul 11

- ☐ Saya dapat membaca data transaksi dan menjelaskan mengapa ia disimpan sebagai matriks jarang.
- ☐ Saya dapat menghitung dan menafsirkan support, confidence, dan lift.
- ☐ Saya dapat menjelaskan sifat anti-monoton dan perannya pada Apriori.
- ☐ Saya dapat membedakan himpunan frequent, closed, dan maximal.
- ☐ Saya dapat menjelaskan mengapa Apriori dan Eclat memberi hasil sama.
- ☐ Saya dapat mengenali aturan ber-confidence tinggi tetapi ber-lift sekitar satu.
- ☐ Saya dapat memangkas aturan redundan dan menjelaskan alasannya.
- ☐ Saya dapat membedakan lift dan leverage serta memilih yang sesuai kebutuhan.
- ☐ Saya dapat menjelaskan pengaruh ambang support, termasuk pada barang langka.

Ringkasan fungsi R:

Fungsi	Kegunaan
<code>read.transactions()</code>	membaca data transaksi (bentuk <i>single</i> atau <i>basket</i>)
<code>itemFrequency(), size()</code>	support tiap barang; ukuran tiap keranjang
<code>apriori()</code>	menambang himpunan sering muncul atau aturan
<code>eclat()</code>	menambang himpunan sering muncul dengan penyajian vertikal
<code>is.closed(), is.maximal()</code>	memampatkan himpunan sering muncul
<code>is.redundant()</code>	menandai aturan yang tidak menambah pengetahuan
<code>interestMeasure()</code>	leverage, conviction, uji Fisher, dan ukuran lain
<code>subset(), sort(), inspect()</code>	menyaring, mengurutkan, dan menampilkan aturan
<code>aggregate(trx, by=)</code>	menaikkan analisis ke tingkat kategori

11.10.1 Daftar Istilah

Transaksi (*transaction*) — himpunan barang yang muncul bersamaan pada satu kejadian.

Himpunan barang (*itemset*) — himpunan bagian dari seluruh barang; k -itemset bila beranggota k .

Support — proporsi transaksi yang memuat suatu himpunan.

Confidence — peluang bersyarat ruas kanan bila ruas kiri hadir.

Lift — nisbah confidence terhadap support ruas kanan; 1 berarti tidak ada kaitan.

Leverage — selisih mutlak support gabungan terhadap keadaan bebas.

Conviction — seberapa sering aturan akan keliru bila kedua ruas bebas.

Sifat anti-monoton (*apriori property*) — himpunan bagian dari himpunan yang sering muncul pasti juga sering muncul.

Closed itemset — tidak ada superhimpunan langsung yang support-nya sama.

Maximal itemset — tidak ada superhimpunan yang juga sering muncul.

Aturan redundan — aturan yang confidence-nya tidak melampaui aturan lain yang ruas kirinya lebih sederhana.

11.11 Referensi

1. Agrawal, R., Imieliński, T., & Swami, A. (1993). Mining association rules between sets of items in large databases. *Proceedings of ACM SIGMOD*, 207–216.
2. Agrawal, R. & Srikant, R. (1994). Fast algorithms for mining association rules. *Proceedings of VLDB*, 487–499.
3. Zaki, M. J. (2000). Scalable algorithms for association mining. *IEEE Transactions on Knowledge and Data*

Engineering, 12(3), 372–390.

4. Tan, P.-N., Steinbach, M., Karpatne, A., & Kumar, V. (2019). *Introduction to Data Mining*, 2nd ed. Pearson. (Bab 5)
5. Hahsler, M., Grün, B., & Hornik, K. (2005). arules — A computational environment for mining association rules and frequent item sets. *Journal of Statistical Software*, 14(15).
6. Han, J., Pei, J., & Tong, H. (2022). *Data Mining: Concepts and Techniques*, 4th ed. Morgan Kaufmann.

MODUL

12

PENGELOMPOKAN: K-MEDOID, K-PROTOTYPE, DBSCAN, OPTICS, DAN BI-CLUSTERING

Pertemuan Minggu ke-12

Sub-CPMK 12: Mahasiswa mampu menerapkan beragam metode pengelompokan — k-Medoid termasuk untuk data kategorik dan campuran, k-Prototype, DBSCAN, OPTICS, serta bi-clustering — mengevaluasi hasilnya dengan ukuran internal maupun eksternal, menentukan banyaknya klaster, serta membandingkan antar-keluarga metode secara valid.

Penyusun: Achmad Syahrul Choir

Daftar Isi

12.1 Deskripsi Modul	369
12.2 Tujuan Praktikum	370
12.3 Alat dan Bahan	370
12.4 Instruksi Pengerjaan	370
12.5 Teori Singkat	371
12.5.0.1 Kedudukan pada CRISP-DM	371
12.5.0.2 Metode Berbasis Kepadatan	371
12.5.0.3 Menetapkan Eps dan MinPts	372
12.5.0.4 OPTICS: Mengurutkan Alih-alih Memartisi	372
12.5.0.5 Evaluasi Hasil Pengelompokan	372
12.5.0.6 Membandingkan Pengelompokan Berbasis Jarak dan Berbasis Kepadatan	373
12.5.0.7 Menentukan Banyaknya Klaster	375
12.5.0.8 Bi-clustering	376
12.6 Contoh Praktikum dengan R	377
12.7 Latihan Mandiri	413
12.8 Penugasan	416
12.9 Ringkasan Modul	417
12.10 Referensi	419

12.1 Deskripsi Modul

Sembilan modul sebelumnya bekerja pada tugas supervised, yaitu ketika sasaran diketahui dan model dinilai terhadapnya. Modul ini berpindah ke tugas unsupervised: tidak ada sasaran, dan yang dicari adalah struktur alami di dalam data itu sendiri.

Perpindahan itu membawa akibat mendasar pada cara menilai. Tidak ada kunci jawaban, sehingga tidak ada akurasi yang dapat dihitung. Kebermaknaan hasil bersandar pada ukuran internal — seberapa rapat anggota satu klaster dan seberapa terpisah antar-klaster — serta pada penafsiran yang masuk akal menurut pengetahuan bidang.

Materi mencakup empat keluarga metode. **Pertama**, metode berbasis medoid: **k-Medoid (PAM)** untuk peubah numerik, serta penerapannya pada **data kategorik dan campuran** lewat jarak **Gower**. **Kedua**, **k-Prototype** yang menggabungkan k-Means dan k-Modes untuk data campuran, beserta parameter λ yang menimbang kedua jenis peubah. **Ketiga**, metode berbasis kepadatan — **DBSCAN** dan **OPTICS** — yang mampu menemukan klaster berbentuk tidak teratur sekaligus menandai derau; keduanya dibandingkan pada data berkepadatan **bersarang** yang memperlihatkan kapan DBSCAN tidak lagi memadai. **Keempat**, **bi-clustering** yang mengelompokkan baris dan kolom sekaligus, sehingga dapat menemukan pola yang hanya berlaku pada sebagian peubah.

Selain metode, modul menekankan **evaluasi**: ukuran mutu bagi keempat metode, penentuan banyaknya klaster, serta **ukuran yang sah dipakai lintas keluarga** — kesepakatan, cakupan, dan kestabilan — sebab siluet dan Dunn tidak sebanding bila dihitung atas himpunan amatan yang berbeda. Bagian penutup memuat **kajian terkendali** yang membuktikan hal itu pada data berpola diketahui.

Tiga hal yang bersifat pelengkap — alur kerja pengelompokan dari memeriksa kecenderungan berklaster hingga profiling, varian k-Medoid untuk data besar (**CLARA** dan **CLARANS**), serta **kajian terkendali** yang membuktikan mengapa siluet dan Dunn tidak sah menjadi hakim antar-keluarga metode — dipindahkan ke **Modul 13** agar modul ini tetap seukuran satu sesi praktikum.

Perlu ditegaskan apa yang **tidak** dibahas modul ini: teori jarak, pengelompokan hierarki, dan k-Means. Ketiganya diandaikan sudah dikenal; k-Medoid dan PAM hadir sebagai wakil keluarga berbasis jarak, dan k-Means disebut hanya sebagai pembanding konseptual.

Tentang data. Berkas data pada modul ini adalah **data simulasi** (`set.seed(2026)`), **bukan data nyata** dan **bukan data resmi** instansi mana pun. Peubah dibangkitkan tanpa struktur klaster yang disengaja, sehingga hasil pengelompokan yang lemah adalah temuan yang benar dan wajib dilaporkan apa adanya.

12.2 Tujuan Praktikum

Setelah menyelesaikan praktikum ini secara tuntas, mahasiswa diharapkan mampu:

1. **Menjelaskan** gagasan titik inti, titik batas, dan derau pada metode kepadatan.
2. **Menerapkan** DBSCAN beserta penetapan parameter Eps dan MinPts secara terarah.
3. **Memeriksa** kepekaan hasil DBSCAN terhadap pemilihan parameternya.
4. **Menerapkan** OPTICS dan membaca diagram jangkauannya.
5. **Mengevaluasi** hasil pengelompokan dengan indeks siluet dan indeks Dunn.
6. **Menentukan** banyaknya kluster dengan metode siku dan siluet rata-rata.
7. **Menjelaskan** mengapa ukuran internal tidak sah dipakai membandingkan antar-keluarga metode.
8. **Menerapkan** prosedur perbandingan yang adil beserta pelaporan cakupannya.
9. **Memakai** ukuran eksternal (ARI) ketika label sebenarnya tersedia.
10. **Mengenal**i gagasan bi-clustering beserta keadaan yang menuntutnya.

12.3 Alat dan Bahan

Perangkat dan bahan yang diperlukan:

- R (RStudio) dengan paket `dbscan`, `cluster`, `fpc`, `mclust`, dan `factoextra`.
- Paket `biclust` untuk bagian bi-clustering.
- Pasang dengan `install.packages(c("dbscan", "cluster", "fpc", "clustMixType", "biclust", "factoextra"))`.
- Berkas data: `data/data_wilayah.csv`, `data/titik_pemantauan.csv`, dan `data/data_ekspresi.csv`.
- Paket `cluster` dipakai untuk indeks siluet dan sebagai pembanding berbasis jarak, bukan sebagai bahan tersendiri.

12.4 Instruksi Pengerjaan

Ikuti tahapan berikut:

Sebelum praktikum: pasang seluruh paket; siapkan kedua berkas data; tinjau kembali bagian jarak pada Modul 3.

Selama praktikum: kerjakan Contoh berurutan. Perhatikan bahwa pembakuan menjadi penentu pada hampir seluruh metode di modul ini — ini menjadi sorotan utama.

Selama praktikum: kerjakan Contoh berurutan. Bagian kajian perbandingan (Contoh 12.10 sampai 11.15) memakai data buatan berlabel diketahui, sebab hanya dengan begitu ketepatan suatu ukuran mutu dapat diuji.

12.5 Teori Singkat

12.5.0.1 Kedudukan pada CRISP-DM

Modul ini menempati tahap Modeling dan Evaluation, tetapi dengan watak yang berbeda dari modul supervised. Tanpa sasaran, tahap evaluasi tidak dapat membandingkan dugaan terhadap kebenaran; yang dinilai adalah mutu struktur yang ditemukan beserta kebermaknaannya menurut pengetahuan bidang.

Pembakuan mendahului pengelompokan. Seluruh metode pada modul ini bersandar pada jarak antar-amatan. Bila satuan peubah berbeda jauh — kepadatan penduduk bersatuan ribuan berdampingan dengan lama sekolah bersatuan belasan — maka peubah bersatuan besar akan menentukan hampir seluruh jarak, sehingga pengelompokan yang dihasilkan sesungguhnya hanya mencerminkan satu peubah. Pembakuan karena itu **wajib** dilakukan lebih dahulu.

12.5.0.2 Metode Berbasis Kepadatan

Metode partisi seperti PAM mengandaikan klaster berbentuk kurang lebih bulat dan mengharuskan setiap amatan masuk salah satu klaster. Metode berbasis kepadatan melepaskan kedua anggapan itu: klaster didefinisikan sebagai daerah berkepadatan tinggi yang dipisahkan daerah berkepadatan rendah, dan amatan yang tidak termasuk mana pun ditandai sebagai **derau**.

Tiga jenis titik pada DBSCAN.

Titik inti	memiliki setidaknya MinPts tetangga dalam radius Eps
Titik batas	bukan titik inti, tetapi berada dalam Eps dari suatu titik inti
Derau	bukan keduanya; tidak dimasukkan ke klaster mana pun

Dua gagasan penghubungnya adalah **density-reachable** — titik dapat dicapai melalui rantai titik inti — dan **density-connected**, yaitu dua titik yang keduanya dapat dicapai dari titik inti yang sama. Sebuah klaster DBSCAN adalah himpunan maksimal titik yang saling density-connected.

Keunggulan dan kelemahan DBSCAN.

Bentuk bebas	menemukan klaster tidak bulat, misalnya memanjang atau melengkung
Menandai derau	pencilan tidak dipaksa masuk klaster
Tanpa menetapkan k	banyaknya klaster muncul sendiri
Kepadatan beragam	<i>gagal</i> bila klaster berkepadatan sangat berbeda
Dimensi tinggi	memburuk karena kutukan dimensi

12.5.0.3 Menetapkan Eps dan MinPts

Kedua parameter itu menentukan hasil sepenuhnya. MinPts lazim ditetapkan sekitar dua kali banyaknya dimensi. Adapun Eps dicari secara grafis: jarak tiap titik ke tetangga terdekat ke-k diurutkan lalu digambar, dan titik belok pada kurva itu menjadi calon nilai Eps.

Tips dan Praktik Baik. Alasan cara grafis itu bekerja sederhana: bagi titik yang berada di dalam klaster, jarak ke tetangga ke-k relatif kecil dan seragam; bagi titik derau, jaraknya jauh lebih besar. Titik belok menandai peralihan antara keduanya.

12.5.0.4 OPTICS: Mengurutkan Alih-alih Memartisi

OPTICS tidak menghasilkan pengelompokan tunggal melainkan **urutan** titik beserta jarak jangkauannya. Dari urutan itu, pengelompokan pada berbagai nilai Eps dapat diperoleh tanpa menghitung ulang.

Dua besaran pokok OPTICS.

Jarak inti	jarak terkecil yang membuat suatu titik menjadi titik inti
Jarak jangkauan	jarak terkecil yang membuat suatu titik dapat dicapai dari titik lain

Diagram jangkauan menggambarkan jarak jangkauan menurut urutan tersebut. Lembah pada diagram menandai klaster, sedangkan puncak menandai batas antar-klaster. Keunggulan pokoknya adalah kemampuan menangani klaster berkepadatan berbeda — tepat kelemahan DBSCAN.

12.5.0.5 Evaluasi Hasil Pengelompokan

Tanpa sasaran, mutu pengelompokan dinilai dengan ukuran internal yang membandingkan kerapatan di dalam klaster terhadap keterpisahan antar-klaster.

Indeks siluet bagi amatan i :

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

dengan $a(i)$ rata-rata jarak ke sesama anggota klaster dan $b(i)$ rata-rata jarak ke anggota klaster tetangga terdekat. Nilainya berkisar dari -1 sampai 1 .

Membaca nilai siluet.

0,71 sampai 1,00	struktur kuat
0,51 sampai 0,70	struktur cukup
0,26 sampai 0,50	struktur lemah, mungkin semu
$\leq 0,25$	tidak ada struktur berarti

Indeks Dunn bagi seluruh pengelompokan:

$$D = \frac{\min_{\text{antar-klaster}} \text{jarak}}{\max_{\text{dalam klaster}} \text{diameter}}$$

Makin besar makin baik. Dunn sangat peka terhadap pencilan karena memakai nilai ekstrem.

12.5.0.6 Membandingkan Pengelompokan Berbasis Jarak dan Berbasis Kepadatan

Dua keluarga metode pengelompokan bertolak dari gagasan yang berbeda. Metode berbasis jarak — K-Means, K-Medoids, K-Prototypes — mencari partisi yang meminimumkan jarak anggota terhadap pusat klasternya. Metode berbasis kepadatan mencari daerah padat yang dipisahkan daerah renggang. Perbedaan itu bukan sekadar teknis: keduanya menjawab pertanyaan yang berlainan, sehingga membandingkannya menuntut kehati-hatian.

Perbedaan pokok dua keluarga metode.

	Berbasis jarak	Berbasis kepadatan
Bentuk klaster	cenderung membulat	bebas, dapat melengkung
Banyaknya klaster	ditetapkan di muka	muncul dari data
Cakupan amatan	seluruhnya masuk klaster	sebagian dapat menjadi derau
Parameter utama	k	Eps dan MinPts
Kepadatan beragam	tidak menjadi soal	menyulitkan DBSCAN
Dimensi tinggi	memburuk	memburuk lebih cepat

12.5.0.6.1 Tiga Halangan dalam Membandingkannya

Baris ketiga pada tabel di atas menimbulkan halangan pertama. Metode berbasis jarak menempatkan seluruh amatan ke dalam klaster, sedangkan metode berbasis kepadatan menyisakan sebagian sebagai derau. Membandingkan ukuran mutu keduanya secara langsung berarti membandingkan hasil atas himpunan amatan yang berbeda — perbandingan yang tidak sepadan.

Halangan kedua terletak pada banyaknya klaster. Ukuran internal seperti siluet bergantung pada k , sehingga

dua pengelompokan dengan banyak klaster berbeda tidak otomatis dapat diperbandingkan nilainya.

Halangan ketiga yang paling menentukan: **ukuran internal yang lazim dipakai mengandaikan klaster membulat**. Indeks siluet membandingkan jarak rata-rata ke sesama anggota terhadap jarak ke klaster tetangga, sehingga klaster memanjang atau melengkung dihukum meski pengelompokannya benar. Akibatnya siluet **tidak sah** dipakai memutuskan keluarga metode mana yang lebih baik.

Siluet bukan hakim yang adil antar-keluarga metode.

Pada data berbentuk cincin sepusat, pengelompokan yang **sungguh benar** dapat bernilai siluet lebih rendah daripada pengelompokan yang sepenuhnya keliru. Sebabnya bukan kekeliruan perhitungan melainkan sifat ukurannya: siluet memberi nilai tinggi kepada klaster yang rapat dan membulat, dan cincin bukan keduanya. Contoh 12.13 memperagakan hal ini dengan angka.

12.5.0.6.2 Prosedur Perbandingan yang Sahih

Lima langkah membandingkan secara adil.

- 1 Samakan prapemrosesan: pembakuan dan ukuran jarak yang sama bagi keduanya
- 2 Laporkan *cakupan*, yaitu persentase amatan yang dikelompokkan tiap metode
- 3 Bandingkan pada *himpunan bagian yang sama*, yakni amatan yang dikelompokkan keduanya
- 4 Bila label sebenarnya tersedia, pakai ukuran *eksternal* seperti Adjusted Rand Index
- 5 Bila tidak tersedia, jangan mengadu ukuran internal; pakai kestabilan, gambar, dan penafsiran bidang

Adjusted Rand Index membandingkan dua pengelompokan atas amatan yang sama:

$$ARI = \frac{RI - E[RI]}{\max(RI) - E[RI]}$$

Bernilai 1 bila kedua pengelompokan identik, dan sekitar 0 bila kesesuaiannya tidak lebih baik daripada kebetulan. Nilai negatif menandakan kesesuaian lebih buruk daripada kebetulan.

Keunggulan ARI atas ukuran internal terletak pada ketidakbergantungannya terhadap bentuk klaster. Ia hanya memeriksa pasangan amatan: apakah dua amatan yang sekelompok menurut kebenaran juga sekelompok menurut model. Karena itu ARI sah dipakai membandingkan keluarga metode mana pun — asalkan label sebenarnya memang tersedia.

Tips dan Praktik Baik. Pada pekerjaan nyata, label sebenarnya hampir tidak pernah tersedia — itulah sebabnya pengelompokan dipakai. Karena itu langkah kelima menjadi yang paling sering terpakai: jalankan kedua keluarga metode, gambarkan hasilnya, periksa kestabilannya terhadap perubahan kecil

data, dan pilih berdasarkan mana yang menghasilkan kelompok yang dapat dijelaskan dan dipakai. Keputusan itu bersifat substantif, bukan semata angka.

12.5.0.7 Menentukan Banyaknya Klaster

Tiga cara yang lazim.

Cara	Yang diamati	Kelemahan
Siku	jumlah kuadrat dalam klaster	titik siku kerap kabur
Siluet rata-rata	nilai siluet menurut k	memihak klaster bulat
Empiris	$k \approx \sqrt{n/2}$	kasar, hanya awalan

Algoritma selalu memberi jawaban. Seluruh algoritma pengelompokan akan mengembalikan klaster meski data sama sekali tidak berstruktur — persis sebagaimana algoritma penemuan komunitas pada Modul 3. Karena itu langkah wajib setelah pengelompokan adalah memeriksa apakah strukturnya **berarti**, bukan sekadar melaporkan hasilnya. Nilai siluet rata-rata di bawah 0,25 pada seluruh nilai k adalah pertanda kuat bahwa data memang tidak berklaster.

12.5.0.8 Bi-clustering

Seluruh metode di atas mengelompokkan **baris**. Bi-clustering mengelompokkan baris dan kolom secara bersamaan, sehingga menemukan himpunan bagian amatan yang berperilaku serupa hanya pada himpunan bagian peubah tertentu.

Kebutuhan itu muncul ketika sebuah kelompok amatan hanya mirip pada sebagian peubah saja. Pada data ekspresi gen, sekelompok gen dapat terekspresi serupa hanya pada beberapa contoh dan tidak pada yang lain — pola yang tak akan ditemukan pengelompokan baris biasa.

Pengelompokan baris dibandingkan bi-clustering.

	Pengelompokan baris	Bi-clustering
Yang dikelompokkan	baris saja	baris dan kolom
Cakupan peubah	seluruh peubah	sebagian peubah
Tumpang tindih	lazimnya tidak	boleh
Penerapan khas	segmentasi	ekspresi gen, sistem rekomendasi

Algoritma Cheng-Church. Diperkenalkan pada tahun 2000, algoritma ini mencari submatriks yang **skor sisaan rata-ratanya** rendah, yaitu submatriks yang nilainya dapat diterangkan oleh pengaruh baris ditambah pengaruh kolom.

Skor sisaan rata-rata bagi submatriks $I \times J$:

$$H(I, J) = \frac{1}{|I||J|} \sum_{i \in I, j \in J} (a_{ij} - a_{iJ} - a_{IJ} + a_{IJ})^2$$

dengan a_{iJ} rata-rata baris, a_{IJ} rata-rata kolom, dan a_{IJ} rata-rata submatriks. Submatriks disebut bikluster bila $H(I, J) < \delta$.

12.6 Contoh Praktikum dengan R

Contoh 12.1 | Menyiapkan data, memeriksa skala, dan membakukan

Kode 12.1 • R memeriksa skala lalu membakukan

```
wil <- read.csv("data/data_wilayah.csv", stringsAsFactors = TRUE)
str(wil)
```

```
'data.frame': 200 obs. of 8 variables:
 $ id      : int  140 112 29 58 112 6 94 172 51 117 ...
 $ kepadatan : num  7998 5692 2831 8671 3719 ...
 $ pengeluaran : num  1765 2072 3175 1014 2078 ...
 $ akses_air : num  58.1 71.2 57.1 77.5 39.7 38 87.3 71.7 40.1 89.6 ...
 $ akses_listrik: num  95 63 84 88.7 53 95.7 68.4 94.9 55 89.8 ...
 $ lama_sekolah : num  13.6 9.8 10.6 8.1 7.6 14.9 13.9 8.3 8.8 7.6 ...
 $ status      : Factor w/ 2 levels "Perdesaan","Perkotaan": 2 1 1 1 2 2 2 2 1 1 ...
 $ pulau       : Factor w/ 4 levels "Jawa","Papua",...: 2 2 1 1 4 1 4 1 3 1 ...
```

```
num <- c("kepadatan", "pengeluaran", "akses_air", "akses_listrik", "lama_sekolah")
ringkas <- data.frame(
  rata = sapply(wil[num], mean),
  sb = sapply(wil[num], sd),
  min = sapply(wil[num], min),
  maks = sapply(wil[num], max))
print(round(ringkas, 2))
```

	rata	sb	min	maks
kepadatan	4724.70	2559.27	251.3	8962.8
pengeluaran	2423.03	1107.14	423.0	4479.1
akses_air	62.19	21.86	25.4	99.5
akses_listrik	73.65	17.17	45.3	100.0
lama_sekolah	9.26	3.17	3.0	15.0

```
cat("\nrasio simpangan baku terbesar terhadap terkecil:",
    round(max(ringkas$sb) / min(ringkas$sb), 1), "\n")
```

```
rasio simpangan baku terbesar terhadap terkecil: 808
```

```
# pembakuan: seluruh metode pada modul ini bersandar pada jarak
X <- scale(wil[num])
round(rbind(rata = colMeans(X), sb = apply(X, 2, sd)), 8)
```

	kepadatan	pengeluaran	akses_air	akses_listrik	lama_sekolah
rata	0	0	0	0	0
sb	1	1	1	1	1

Penjelasan Kode. Data dibaca lalu kelima peubah numerik diringkas rata-rata, simpangan baku, dan rentangnya. Rasio simpangan baku terbesar terhadap terkecil dihitung sebagai ukuran ketimpangan skala. Data lalu dibakukan dengan scale, dan hasilnya diperiksa: rata-rata tiap kolom harus nol dan simpangan bakunya satu.

Interpretasi Output

Rasio yang besar adalah peringatan langsung. Karena seluruh metode pada modul ini bersandar pada jarak antar-amatan, peubah bersimpangan baku terbesar akan menentukan hampir seluruh jaraknya bila data tidak dibakukan — sehingga pengelompokan yang dihasilkan sesungguhnya hanya mencerminkan satu peubah. Memeriksa rata-rata dan simpangan baku setelah pembakuan bukan formalitas melainkan pembuktian bahwa langkahnya sungguh berjalan. Objek X inilah yang dipakai seluruh contoh berikutnya.

Contoh 12.2 | Mencari nilai Eps melalui grafik jarak tetangga

Kode 12.2 • R kNNdist dan pembacaan titik belok

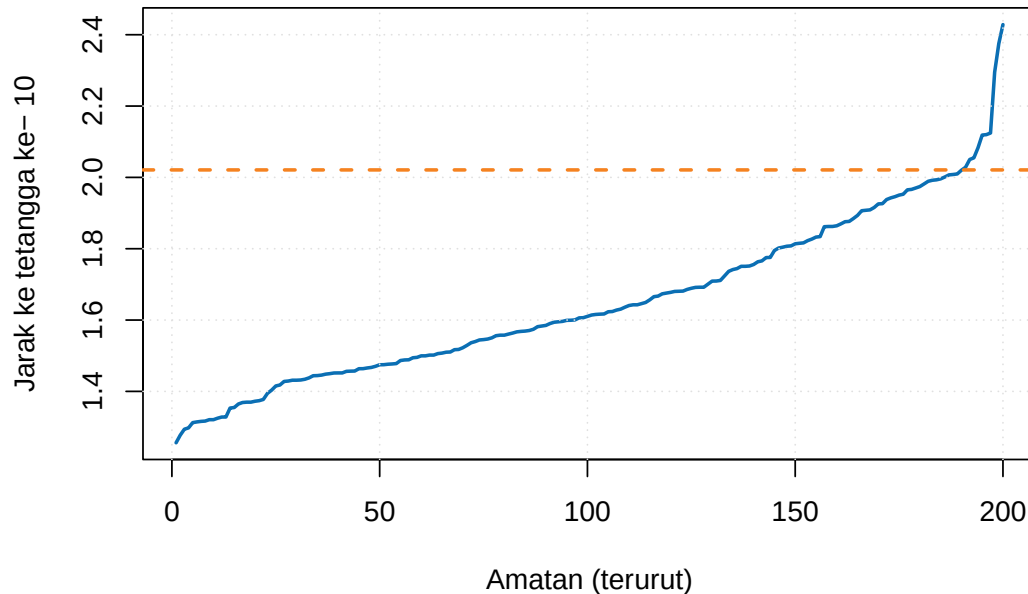
```
suppressPackageStartupMessages(library(dbSCAN))

minpts <- 2 * length(num) # aturan praktis: 2 kali dimensi
jarak_k <- sort(dbSCAN::kNNdist(X, k = minpts))

plot(jarak_k, type = "l", lwd = 2, col = "#0B6FB4",
     xlab = "Amatan (terurut)", ylab = paste("Jarak ke tetangga ke-", minpts),
     main = "Menentukan Eps melalui Titik Belok")
grid(col = "grey88")

calon_eps <- round(quantile(jarak_k, 0.95), 3)
abline(h = calon_eps, col = "#F58220", lty = 2, lwd = 2)
```

Menentukan Eps melalui Titik Belok



```
cat("MinPts dipakai:", minpts, " | calon Eps (persentil 95):", calon_eps, "\n")
```

```
MinPts dipakai: 10 | calon Eps (persentil 95): 2.021
```

Penjelasan Kode. MinPts ditetapkan dua kali banyaknya dimensi sesuai aturan praktis. Jarak tiap amatan ke tetangga terdekat ke-MinPts dihitung, diurutkan, lalu digambar. Persentil ke-95 dipakai sebagai calon Eps yang objektif.

Interpretasi Output

Kurva yang naik landai lalu menanjak tajam menandai peralihan dari titik di dalam kluster ke titik derau, dan titik beloknya menjadi calon Eps. Bila kurvanya menanjak mulus tanpa belok yang jelas — sebagaimana lazim terjadi pada data tanpa struktur — itu sendiri merupakan pertanda bahwa metode berbasis kepadatan mungkin tidak sesuai bagi data tersebut.

Contoh 12.3 | Menerapkan DBSCAN

Kode 12.3 • R dbscan::dbscan dengan penulisan tegas

```
set.seed(2026)
db <- dbscan::dbscan(X, eps = as.numeric(calón_eps), minPts = minpts)
```

```
tab <- table(db$cluster)
cat("banyaknya klaster:", length(setdiff(names(tab), "0")), "\n")
```

```
banyaknya klaster: 1
```

```
cat("amanat derau      :", sum(db$cluster == 0),
    sprintf("(%.1f persen)\n", 100 * mean(db$cluster == 0)))
```

```
amanat derau      : 0 (0.0 persen)
```

```
print(tab)
```

```
 1
200
```

Penjelasan Kode. DBSCAN dijalankan memakai Eps dan MinPts yang sudah ditetapkan. Klaster bernomor nol menandai derau sehingga dikeluarkan saat mencacah banyaknya klaster.

Interpretasi Output

Perhatikan penulisan nama paket secara tegas. Paket fpc juga menyediakan fungsi bernama dbscan dengan argumen berbeda, sehingga tanpa penulisan tegas kode dapat gagal dengan pesan mengenai argumen tak dikenal — bentrokan nama yang sama jenisnya dengan yang dijumpai pada Modul 6. Perhatikan pula persentase derau: nilai yang sangat besar menandakan Eps terlalu kecil, sedangkan nol derau sama sekali menandakan Eps terlalu besar.

Contoh 12.4 | Memeriksa kepekaan DBSCAN terhadap Eps

Kode 12.4 • R menjalankan DBSCAN pada beberapa nilai Eps

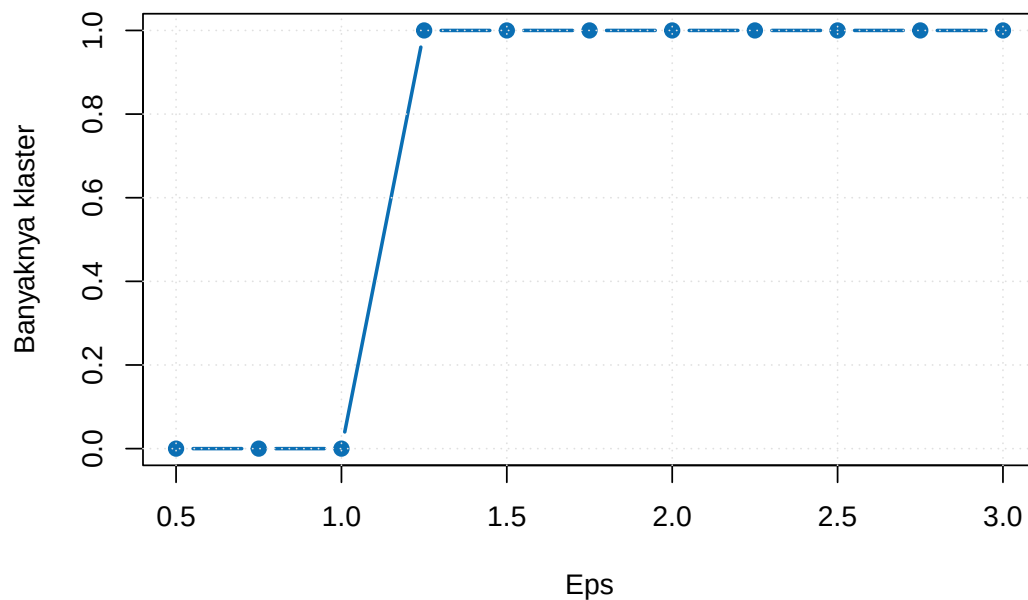
```
kisi_eps <- round(seq(0.5, 3.0, by = 0.25), 2)
peka <- t(sapply(kisi_eps, function(e) {
  h <- dbscan::dbscan(X, eps = e, minPts = minpts)
  c(eps = e,
    klaster = length(setdiff(unique(h$cluster), 0)),
    derau_persen = round(100 * mean(h$cluster == 0), 1))
}))
print(peka, row.names = FALSE)
```

```
      eps klaster derau_persen
[1,] 0.50      0      100
[2,] 0.75      0      100
[3,] 1.00      0      100
[4,] 1.25      1       95
[5,] 1.50      1       20
```

```
[6,] 1.75    1    0
[7,] 2.00    1    0
[8,] 2.25    1    0
[9,] 2.50    1    0
[10,] 2.75   1    0
[11,] 3.00    1    0
```

```
plot(peka[, "eps"], peka[, "klaster"], type = "b", pch = 19, lwd = 2,
     col = "#0B6FB4", xlab = "Eps", ylab = "Banyaknya klaster",
     main = "Kepekaan DBSCAN terhadap Eps")
grid(col = "grey88")
```

Kepekaan DBSCAN terhadap Eps



Penjelasan Kode. Sebelas nilai Eps dicoba. Pada tiap nilai dicatat banyaknya klaster yang terbentuk dan persentase amatan yang ditandai derau, lalu hasilnya digambar.

Interpretasi Output

Grafik ini memperlihatkan bahwa hasil DBSCAN sangat bergantung pada Eps. Pada nilai kecil hampir semuanya menjadi derau; pada nilai besar seluruhnya melebur menjadi satu klaster. Melaporkan hasil DBSCAN tanpa menyebutkan Eps dan MinPts karena itu tidak bermakna — keduanya wajib dicantumkan sebagaimana benih acak pada metode lain.

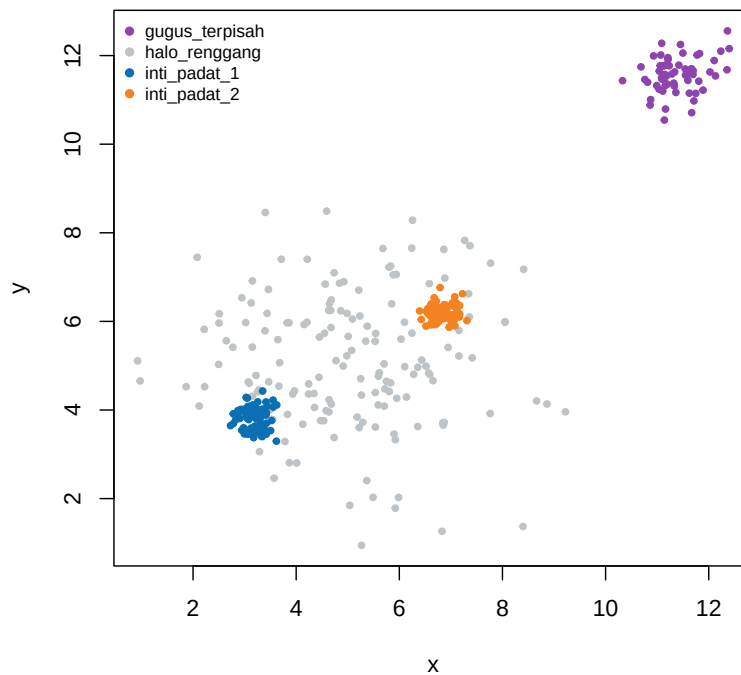
Contoh 12.5 | Data berkepadatan bersarang: susunan yang menuntut OPTICS

Kode 12.5 • R membaca dan menggambarkan titik pemantauan

```
suppressPackageStartupMessages(library(dbscan))
tp <- read.csv("data/titik_pemantauan.csv")
P <- as.matrix(tp[, c("x", "y")])
table(tp$kelompok)

gugus_terpisah halo_renggang inti_padat_1 inti_padat_2
              60             160             80             80

warna <- c(gugus_terpisah = "#8E44AD", halo_renggang = "#BDC3C7",
           inti_padat_1 = "#0B6FB4", inti_padat_2 = "#F58220")
par(family = "sans", mar = c(4, 4, 1, 1))
plot(P, col = warna[tp$kelompok], pch = 19, cex = 0.6, xlab = "x", ylab = "y")
legend("topleft", names(warna), col = warna, pch = 19, bty = "n", cex = 0.75)
```



Gambar 12.1: Dua inti padat berada di dalam gugus renggang, ditambah satu gugus terpisah

Interpretasi Output

Data memuat 380 titik dalam empat kelompok yang **kepadatannya sangat berbeda**. Dua **inti padat** — masing-masing 80 titik dengan simpangan baku 0,20 — berada **di dalam** sebuah **halo renggang** berisi 160 titik yang simpangan bakunya delapan kali lebih besar. Satu **gugus terpisah** berada jauh

dari ketiganya.

Susunan bersarang inilah pokok bahasannya, dan ia lazim dijumpai pada data nyata: sekelompok kecil unit berperilaku sangat seragam, terbenam di dalam kelompok besar yang beragam — misalnya beberapa kecamatan berciri sangat khas di dalam wilayah yang secara umum heterogen.

Pertanyaannya: dapatkah DBSCAN memisahkan keempatnya?

Contoh 12.6 | DBSCAN gagal: tidak ada nilai Eps yang benar

Kode 12.6 • R tiga nilai Eps dan penelusuran menyeluruh

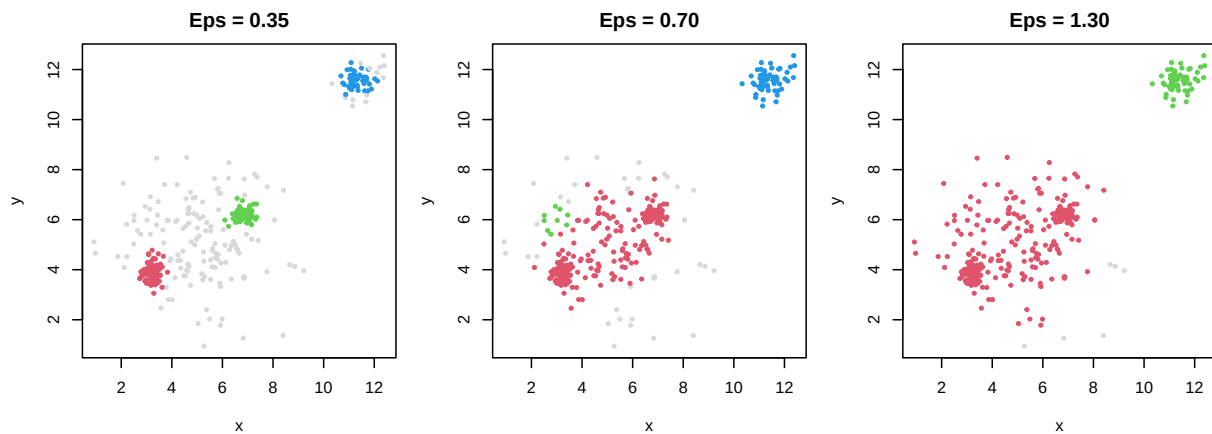
```
ari <- function(a, b) { tb <- table(a, b); n <- sum(tb)
  s_ij <- sum(choose(tb, 2)); s_i <- sum(choose(rowSums(tb), 2))
  s_j <- sum(choose(colSums(tb), 2)); h <- s_i * s_j / choose(n, 2)
  (s_ij - h) / ((s_i + s_j) / 2 - h) }

par(mfrow = c(1, 3), family = "sans", mar = c(4, 4, 2.5, 1))
for (e in c(0.35, 0.70, 1.30)) {
  r <- dbscan::dbscan(P, eps = e, minPts = 10)
  cat(sprintf("Eps %.2f -> %d klaster, %3d derau, ARI %.3f\n", e,
    length(setdiff(unique(r$cluster), 0)), sum(r$cluster == 0),
    ari(tp$kelompok, r$cluster)))
  plot(P, col = ifelse(r$cluster == 0, "grey85", r$cluster + 1), pch = 19, cex = 0.5,
    main = sprintf("Eps = %.2f", e), xlab = "x", ylab = "y")
}
```

Eps 0.35 -> 3 klaster, 151 derau, ARI 0.753

Eps 0.70 -> 3 klaster, 39 derau, ARI 0.197

Eps 1.30 -> 2 klaster, 6 derau, ARI 0.244



Gambar 12.2: Eps kecil membuang halo menjadi derau; Eps besar meleburkan kedua inti ke dalam halo

```
par(mfrow = c(1, 1))

eps_grid <- seq(0.10, 3.00, by = 0.05)           # adakah Eps yang lebih baik?
telusur <- sapply(eps_grid, function(e)
  ari(tp$kelompok, dbscan::dbscan(P, eps = e, minPts = 10)$cluster))
cat("\nARI terbaik yang dapat dicapai DBSCAN:", round(max(telusur), 3),
    "\npada Eps =", eps_grid[which.max(telusur)], "\n")
```

ARI terbaik yang dapat dicapai DBSCAN: 0.762 pada Eps = 0.3

```
r_terbaik <- dbscan::dbscan(P, eps = eps_grid[which.max(telusur)], minPts = 10)
cat("derau pada Eps terbaik itu:", sum(r_terbaik$cluster == 0),
    sprintf("%.1f persen\n", 100 * mean(r_terbaik$cluster == 0)))
```

derau pada Eps terbaik itu: 163 (42.9 persen)

Interpretasi Output

Ketiga gambar memperlihatkan kegagalan yang **tidak dapat diperbaiki dengan menala Eps**.

Eps kecil (0,35). Kedua inti padat ditemukan sempurna — tetapi halo renggang tidak cukup padat untuk memenuhi syarat, sehingga sebagian besar titiknya dibuang menjadi derau. Sekitar 40% data dinyatakan bukan anggota kelompok mana pun.

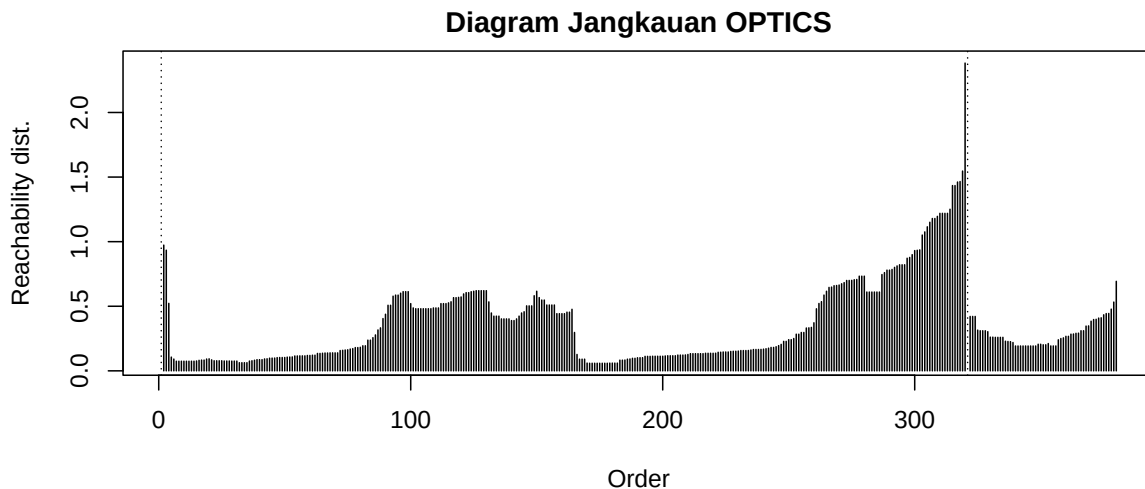
Eps sedang (0,70) dan besar (1,30). Halo kini terjangkau — tetapi karena kedua inti berada *di dalam* halo, ketiganya **melebur menjadi satu klaster**. Susunan bersarangnya lenyap, dan ARI justru anjlok. Yang menentukan adalah dua baris terakhir. Seluruh nilai Eps dari 0,10 sampai 3,00 ditelusuri, dan yang terbaik pun hanya mencapai ARI **0,762** pada Eps = 0,30 — dengan mengorbankan **163 titik (42,9%)** sebagai derau. Kegagalan ini karena itu bukan soal salah memilih Eps: **DBSCAN secara struktural tidak mampu** menangani kelompok berkepadatan sangat berbeda, sebab ia memakai **satu** ambang kepadatan bagi seluruh data.

Perlu ditegaskan pula bahwa penelusuran semacam ini hanya mungkin di ruang kelas — ia memakai label sebenarnya, yang pada pekerjaan nyata tidak tersedia. Pada data sungguhan, analis bahkan tidak akan mengetahui bahwa Eps terbaiknya masih buruk.

Contoh 12.7 | OPTICS: satu perhitungan, seluruh tingkat kepadatan

Kode 12.7 • R optics dan diagram jangkauan

```
op <- dbscan::optics(P, minPts = 10)
par(family = "sans", mar = c(4, 4, 2, 1))
plot(op, main = "Diagram Jangkauan OPTICS")
```



Gambar 12.3: Diagram jangkauan OPTICS: lembah dalam dan sempit menandai inti padat, lembah lebar dangkal menandai halo

```
xi_hasil <- dbscan::extractXi(op, xi = 0.10)
cat("klaster ditemukan:", length(setdiff(unique(xi_hasil$cluster), 0)),
    " derau:", sum(xi_hasil$cluster == 0), "\n")
```

```
klaster ditemukan: 4 derau: 2
```

```
print(table(sebenarnya = tp$kelompok, optics = xi_hasil$cluster))
```

	optics				
sebenarnya	0	1	2	3	4
gugus_terpisah	1	0	0	0	59
halo_renggang	1	130	11	18	0
inti_padat_1	0	0	0	80	0

```

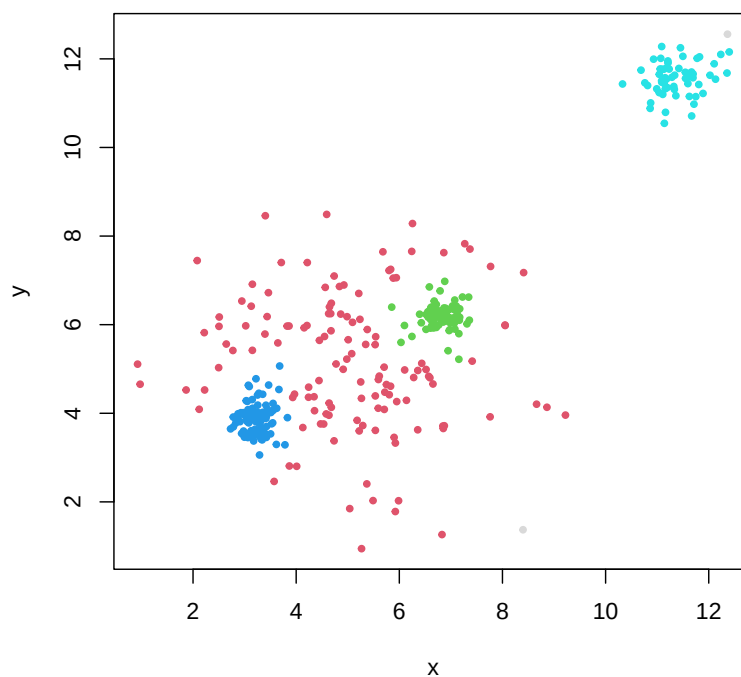
inti_padat_2    0    0 80    0    0

cat("\nARI OPTICS:", round(ari(tp$kelompok, xi_hasil$cluster), 3),
    " | ARI DBSCAN terbaik:", round(max(telusur), 3), "\n")

ARI OPTICS: 0.774 | ARI DBSCAN terbaik: 0.762

par(family = "sans", mar = c(4, 4, 1, 1))
plot(P, col = ifelse(xi_hasil$cluster == 0, "grey85", xi_hasil$cluster + 1),
     pch = 19, cex = 0.6, xlab = "x", ylab = "y")

```



Gambar 12.4: Kluster hasil extractXi digambar pada bidang asal

Penjelasan Kode. `optics()` tidak menghasilkan kluster melainkan **urutan** amatan beserta *jarak jangkauan* tiap titik — ukuran seberapa rapat titik itu terhubung ke bagian data yang telah dijelajahi. Dari urutan tersebut pengelompokan dapat diekstrak berulang kali tanpa perhitungan ulang: `extractDBSCAN()` untuk satu ambang tetap, atau `extractXi()` yang mencari **lembah** pada diagram jangkauan sehingga dapat menangkap beberapa tingkat kepadatan sekaligus.

Interpretasi Output

Diagram jangkauan menceritakan seluruh struktur dalam satu gambar. **Dua lembah dalam dan sempit** berpadanan dengan kedua inti padat — jangkauannya kecil karena titiknya sangat berdekatan. **Lembah lebar yang dangkal** adalah halo renggang, dan **satu lembah terpisah** adalah gugus keempat.

Puncak di antara lembah menandai batasnya.

Perhatikan bahwa kedua inti muncul sebagai lembah **di dalam** wilayah halo — persis menggambarkan susunan bersarang pada data. Inilah yang tidak dapat diwakili keluaran DBSCAN, yang berupa daftar label datar tanpa tingkatan.

`extractXi` dengan $\xi = 0,10$ memulihkan **empat kelompok**: kedua inti padat masing-masing utuh (80 dari 80), gugus terpisah 59 dari 60, dan halo renggang sebagian besar (130 dari 160). ARI-nya **0,774** — **melampaui ARI terbaik yang mungkin dicapai DBSCAN (0,762)**, dan mencapainya dengan hanya **2 titik** yang ditandai derau, bukan 163.

Kesimpulan yang perlu dibawa: OPTICS tidak "lebih akurat" daripada DBSCAN pada data berkepadatan seragam — pada data semacam itu keduanya setara dan DBSCAN lebih sederhana. Keunggulan OPTICS muncul justru ketika **kepadatan antar-kelompok berbeda jauh atau bersarang**, keadaan yang meruntuhkan gagasan "satu Eps untuk semua". Karena itu jalankan OPTICS lebih dahulu untuk **melihat** strukturnya; bila diagram jangkauannya memperlihatkan lembah berkedalaman seragam, DBSCAN sudah memadai.

Contoh 12.8 | Mengambil beberapa nilai Eps dari satu perhitungan OPTICS

Kode 12.8 · R `extractDBSCAN` pada beberapa ambang

```
for (e in c(0.35, 0.70, 1.30)) {
  ek <- dbscan::extractDBSCAN(op, eps_cl = e)
  cat(sprintf("eps_cl %.2f -> %d klaster, %3d derau, ARI %.3f\n", e,
    length(setdiff(unique(ek$cluster), 0)), sum(ek$cluster == 0),
    ari(tp$kelompok, ek$cluster)))
}
```

```
eps_cl 0.35 -> 3 klaster, 153 derau, ARI 0.754
```

```
eps_cl 0.70 -> 3 klaster, 41 derau, ARI 0.200
```

```
eps_cl 1.30 -> 2 klaster, 6 derau, ARI 0.244
```

Interpretasi Output

Angka-angka ini praktis **sama** dengan hasil DBSCAN pada contoh sebelumnya — ARI 0,754 berbanding 0,753 pada Eps 0,35, dan 0,200 berbanding 0,197 pada Eps 0,70. Selisih beberapa titik pada cacah derau muncul karena `extractDBSCAN()` menetapkan *titik batas* menurut urutan penjelajahan OPTICS, sedangkan DBSCAN menentukannya menurut urutan asli data; keduanya sama-sama sah dan tidak mengubah kesimpulan.

Di situlah nilai praktisnya. Menala Eps dengan DBSCAN berarti menjalankan ulang algoritma bagi tiap nilai yang dicoba. Dengan OPTICS, urutannya dihitung **satu kali**, lalu berapa pun nilai Eps dapat diperiksa dengan biaya yang praktis nol. Pada data besar, selisih biaya ini menentukan apakah penalaan

layak dilakukan atau tidak.

Urutan kerja yang dianjurkan karena itu: jalankan OPTICS, baca diagram jangkauannya, lalu putuskan — extractXi bila kepadatannya beragam, atau extractDBSCAN pada satu ambang bila diagramnya menunjukkan kepadatan seragam.

Contoh 12.9 | Mengevaluasi dengan indeks siluet

Kode 12.9 • R silhouette beserta penggambarannya

```
suppressPackageStartupMessages(library(cluster))

# Pengelompokan berbasis jarak dipakai di sini semata sebagai PEMBANDING
# bagi hasil DBSCAN; metodenya sendiri tidak menjadi bahasan modul ini.
set.seed(2026)
pam3 <- pam(X, k = 3)

sil <- silhouette(pam3$clustering, dist(X))
cat("siluet rata-rata:", round(mean(sil[, 3]), 4), "\n\n")

siluet rata-rata: 0.1272

cat("siluet rata-rata per klaster:\n")

siluet rata-rata per klaster:

print(round(tapply(sil[, 3], sil[, 1], mean), 4))

  1      2      3
0.1860 0.1126 0.0932

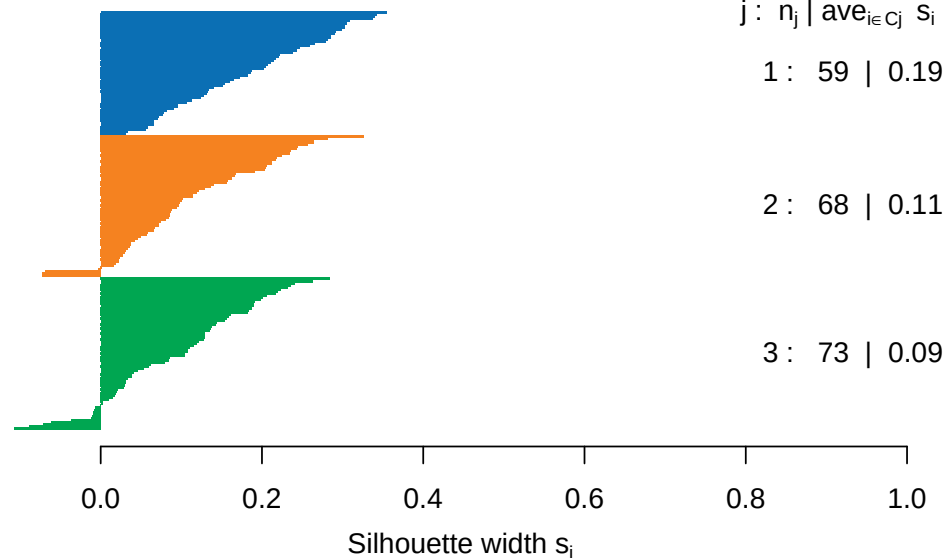
cat("\namatan bersiluet negatif:", sum(sil[, 3] < 0),
    sprintf("(%.1f persen)\n", 100 * mean(sil[, 3] < 0)))

amatan bersiluet negatif: 15 (7.5 persen)

plot(sil, col = c("#0B6FB4", "#F58220", "#00A651"), border = NA,
     main = "Diagram Siluet (pembanding berbasis jarak, k = 3)")
```

Diagram Siluet (pembanding berbasis jarak, $k = 3$)

n = 200



Penjelasan Kode. Sebuah pengelompokan berbasis jarak dibentuk lebih dahulu sebagai pembanding, sebab indeks siluet menuntut suatu pengelompokan untuk dinilai. Indeks siluet dihitung bagi tiap amatan, lalu dirata-ratakan secara keseluruhan maupun per klaster. Amatan bersiluet negatif dicacah, dan diagramnya digambar dengan warna berbeda tiap klaster.

Interpretasi Output

Amatan bersiluet negatif adalah amatan yang sesungguhnya lebih dekat ke klaster tetangga daripada ke klasternya sendiri — pertanda penempatan yang keliru. Bila jumlahnya besar, pengelompokan tidak dapat dipercaya. Perlu ditegaskan bahwa pengelompokan berbasis jarak di sini hanya berperan sebagai objek yang dinilai; bagaimana metodenya bekerja bukan bahasan modul ini, dan pada bagian kajian nanti akan terlihat bahwa siluet justru tidak sah dipakai mengadu kedua keluarga metode.

Contoh 12.10 | Mengevaluasi dengan indeks Dunn dan ukuran lain**Kode 12.10 • R** `fpc::cluster.stats`

```
suppressPackageStartupMessages(library(fpc))
stat <- fpc::cluster.stats(dist(X), pam3$clustering)
```

```
round(c(dunn = stat$dunn,
      siluet_rata = stat$avg.silwidth,
      pemisahan_min = stat$min.separation,
      diameter_maks = stat$max.diameter,
      wb_ratio = stat$wb.ratio), 4)
```

dunn	siluet_rata	pemisahan_min	diameter_maks	wb_ratio
0.1035	0.1272	0.5594	5.4025	0.8213

Penjelasan Kode. Fungsi `cluster.stats` menghitung belasan ukuran mutu pengelompokan sekaligus. Lima yang paling berguna diambil: indeks Dunn, siluet rata-rata, pemisahan antar-kluster terkecil, diameter kluster terbesar, dan rasio jarak dalam terhadap antar-kluster.

Interpretasi Output

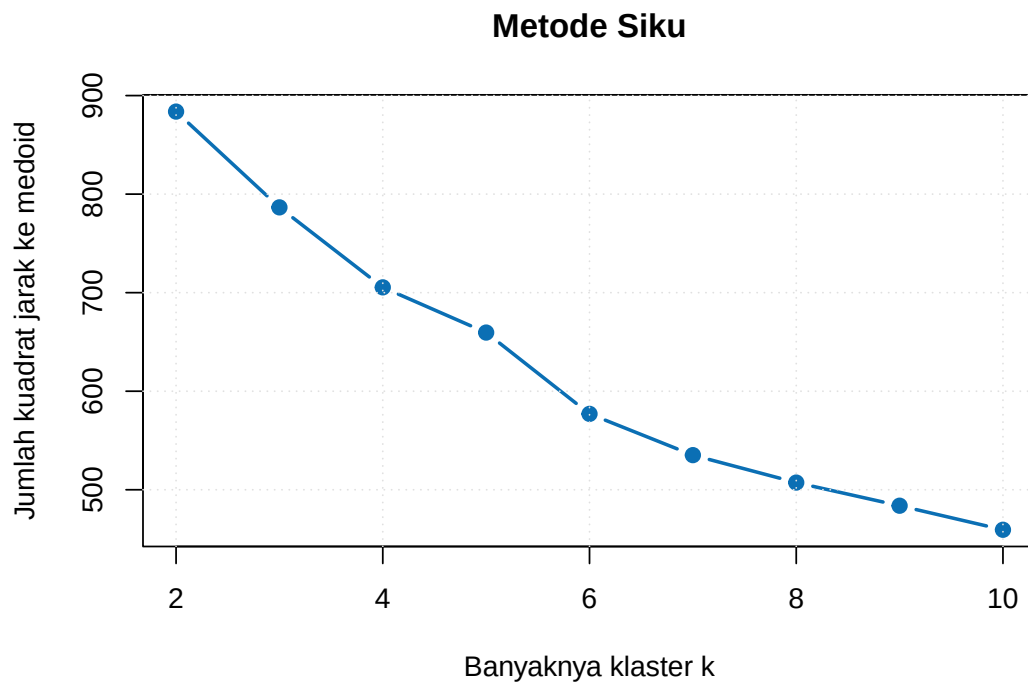
Indeks Dunn adalah rasio pemisahan terkecil terhadap diameter terbesar, sehingga sepenuhnya ditentukan dua nilai ekstrem. Sifat itu membuatnya sangat peka terhadap pencilan — satu amatan menyimpang dapat menjatuhkan nilainya drastis. Karena itu Dunn sebaiknya dibaca bersama siluet, bukan sendirian.

Contoh 12.11 | Menentukan banyaknya kluster dengan metode siku

Kode 12.11 • R pam berulang dan grafik siku

```
wss <- sapply(2:10, function(k) {
  set.seed(2026)
  p <- pam(X, k = k)
  sum(sapply(seq_len(k), function(j) {
    ang <- X[p$clustering == j, , drop = FALSE]
    med <- X[p$id.med[j], ]
    sum(rowSums(sweep(ang, 2, med)^2))
  })))
})

plot(2:10, wss, type = "b", pch = 19, lwd = 2, col = "#0B6FB4",
     xlab = "Banyaknya kluster k", ylab = "Jumlah kuadrat jarak ke medoid",
     main = "Metode Siku")
grid(col = "grey88")
```



```
selisih <- round(diff(wss), 1)
cat("penurunan saat k bertambah:\n")
```

```
penurunan saat k bertambah:
```

```
print(data.frame(dari_k = 2:9, ke_k = 3:10, penurunan = selisih), row.names = FALSE)
```

```
dari_k ke_k penurunan
2      3    -97.2
3      4    -81.2
4      5    -45.9
5      6    -82.5
6      7    -41.9
7      8    -27.9
8      9    -23.5
9     10    -24.4
```

Penjelasan Kode. Jumlah kuadrat jarak tiap amatan terhadap medoid klasternya dihitung untuk sembilan nilai k. Fungsi sweep mengurangkan medoid dari seluruh baris anggota, lalu kuadratnya dijumlahkan. Grafiknya digambar, dan penurunan antar-nilai k ditampilkan sebagai angka.

Interpretasi Output

Menyajikan penurunan sebagai angka menolong karena titik siku kerap kabur secara visual. Siku berada pada nilai k tempat penurunan mulai mengecil tajam. Bila penurunannya melandai mulus tanpa patahan — sebagaimana lazim pada data tanpa struktur — maka metode siku tidak memberi jawaban, dan itu sendiri merupakan temuan yang wajib dilaporkan.

Contoh 12.12 | Menentukan banyaknya klaster dengan siluet rata-rata**Kode 12.12 • R** pam berulang dan garis acuan siluet

```
sil_k <- sapply(2:10, function(k) {
  set.seed(2026)
  p <- pam(X, k = k)
  p$silinfo$avg.width
})

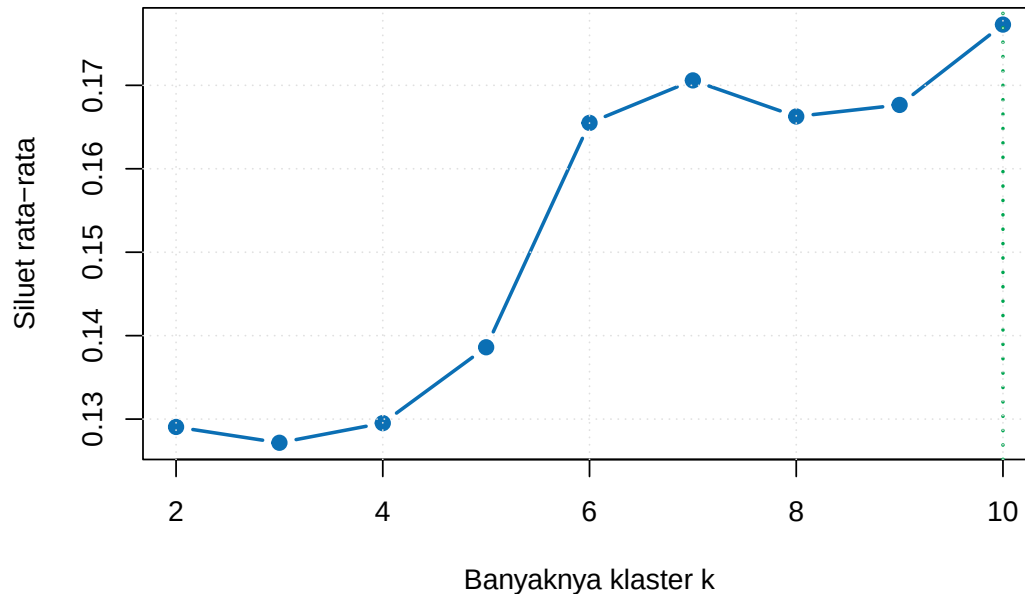
hasil_k <- data.frame(k = 2:10, siluet = round(sil_k, 4))
print(hasil_k, row.names = FALSE)
```

```

k siluet
2 0.1291
3 0.1272
4 0.1295
5 0.1386
6 0.1655
7 0.1706
8 0.1663
9 0.1677
10 0.1773
```

```
plot(2:10, sil_k, type = "b", pch = 19, lwd = 2, col = "#0B6FB4",
     xlab = "Banyaknya klaster k", ylab = "Siluet rata-rata",
     main = "Metode Siluet Rata-rata")
abline(h = 0.25, col = "#C0392B", lty = 2, lwd = 2)
abline(v = hasil_k$k[which.max(sil_k)], col = "#00A651", lty = 3, lwd = 2)
grid(col = "grey88")
```


Metode Siluet Rata-rata



```
cat("\nk dengan siluet tertinggi:", hasil_k$k[which.max(sil_k)],
    "| nilai:", round(max(sil_k), 4), "\n")
```

```
k dengan siluet tertinggi: 10 | nilai: 0.1773
```

```
cat("aturan empiris akar n per dua:", round(sqrt(nrow(X) / 2), 1), "\n")
```

```
aturan empiris akar n per dua: 10
```

Penjelasan Kode. PAM dijalankan untuk sembilan nilai k dan siluet rata-ratanya dicatat. Grafiknya dilengkapi garis mendatar pada nilai 0,25 sebagai batas struktur berarti, dan garis tegak pada k terbaik. Aturan empiris akar n per dua disertakan sebagai pembandingan.

Interpretasi Output

Garis mendatar pada 0,25 adalah bagian terpenting gambar ini. Bila seluruh titik berada di bawahnya, maka tidak ada nilai k yang menghasilkan struktur berarti — dan kesimpulan yang benar adalah menyatakan bahwa data tidak berkluster, bukan memilih k yang kebetulan tertinggi. Pada data simulasi ini, yang peubahnya dibangkitkan tanpa struktur, itulah hasil yang diharapkan.

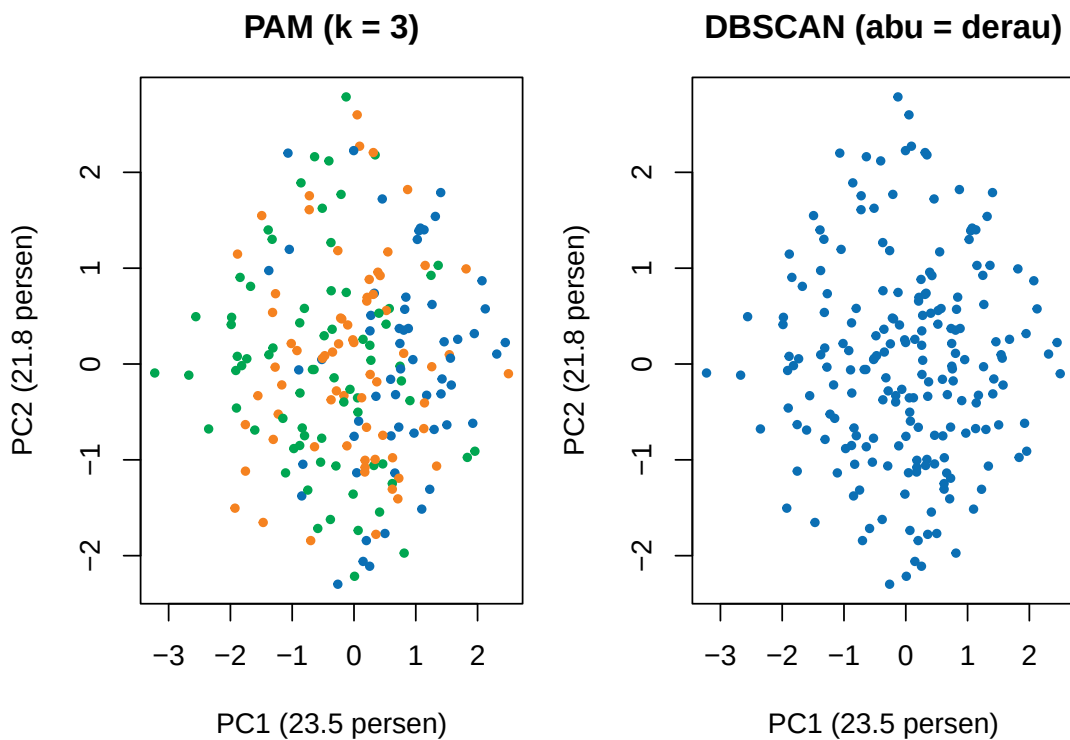
Contoh 12.13 | Menggambarkan hasil pengelompokan pada dua komponen utama

Kode 12.13 · R memproyeksikan kluster ke bidang komponen utama

```
pc <- prcomp(X)
ragam2 <- round(100 * (pc$sdev^2 / sum(pc$sdev^2))[1:2], 1)

par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
plot(pc$x[, 1], pc$x[, 2], col = c("#0B6FB4", "#F58220", "#00A651")[pam3$clustering],
     pch = 19, cex = .6,
     xlab = sprintf("PC1 (%.1f persen)", ragam2[1]),
     ylab = sprintf("PC2 (%.1f persen)", ragam2[2]),
     main = "PAM (k = 3)")

warna_db <- ifelse(db$cluster == 0, "grey60",
                  c("#0B6FB4", "#F58220", "#00A651",
                    "#C0392B", "#6FB3E0")[pmin(db$cluster, 5)])
plot(pc$x[, 1], pc$x[, 2], col = warna_db, pch = 19, cex = .6,
     xlab = sprintf("PC1 (%.1f persen)", ragam2[1]),
     ylab = sprintf("PC2 (%.1f persen)", ragam2[2]),
     main = "DBSCAN (abu = derau)")
```



```
par(mfrow = c(1, 1), mar = c(5, 4, 4, 2) + 0.1)
```

Penjelasan Kode. Data terbakukan diproyeksikan ke dua komponen utama agar dapat digambar pada bidang. Hasil PAM dan DBSCAN disajikan berdampingan dengan proyeksi yang sama, dan derau DBSCAN diberi warna abu.

Interpretasi Output

Memakai proyeksi yang sama bagi kedua gambar adalah kunci perbandingannya. Perhatikan bahwa kedua komponen pertama hanya menerangkan sebagian keragaman, sehingga klaster yang tampak bertumpang tindih pada bidang ini boleh jadi terpisah pada dimensi lain — dan sebaliknya. Gambar semacam ini menolong penafsiran tetapi tidak boleh menjadi satu-satunya dasar kesimpulan.

Contoh 12.14 | Menafsirkan klaster melalui profil peubahnya

Kode 12.14 • R aggregate untuk menyusun profil klaster

```
profil <- aggregate(wil[num], by = list(klaster = pam3$clustering), FUN = mean)
print(round(profil, 2), row.names = FALSE)
```

klaster	kepadatan	pengeluaran	akses_air	akses_listrik	lama_sekolah
1	5733	2710	77.36	84.53	10.86
2	6011	2058	51.34	63.33	10.11
3	2712	2532	60.03	74.48	7.18

```
cat("\nukuran tiap klaster:\n")
```

```
ukuran tiap klaster:
```

```
print(table(klaster = pam3$clustering))
```

```
klaster
 1  2  3
59 68 73
```

```
cat("\nsebaran pulau menurut klaster:\n")
```

```
sebaran pulau menurut klaster:
```

```
print(round(prop.table(table(pam3$clustering, wil$pulau), margin = 1), 3))
```

	Jawa	Papua	Sulawesi	Sumatera
1	0.305	0.136	0.271	0.288
2	0.456	0.176	0.162	0.206
3	0.411	0.178	0.137	0.274

Penjelasan Kode. Rata-rata tiap peubah numerik dihitung per kluster dalam satuan aslinya, bukan satuan terbakukan. Ukuran kluster dan sebaran peubah kategori ditampilkan sebagai pelengkap.

Interpretasi Output

Menyajikan profil dalam satuan asli adalah keharusan bagi pelaporan: pengguna memahami pengeluaran dalam rupiah, bukan dalam simpangan baku. Perlu diingat bahwa penamaan kluster — misalnya menyebut satu kluster sebagai wilayah tertinggal — adalah penafsiran yang menuntut pengetahuan bidang dan wajib disertai catatan bahwa penamaan itu berasal dari analisis, bukan dari data.

Contoh 12.15 | k-Medoid (PAM) pada peubah numerik

Kode 12.15 • R pam dan pemilihan banyaknya medoid

```
suppressPackageStartupMessages(library(cluster))
Zn <- scale(wil[, c("kepadatan", "pengeluaran", "akses_air",
                    "akses_listrik", "lama_sekolah")])

sil_k <- sapply(2:6, function(k) pam(Zn, k)$silinfo$avg.width)
round(setNames(sil_k, paste0("k=", 2:6)), 3)

k=2    k=3    k=4    k=5    k=6
0.129  0.127  0.130  0.139  0.165

pam3 <- pam(Zn, 3)
cat("ukuran kluster:", pam3$clusinfo[, 1], "\n")

ukuran kluster: 59 68 73

cat("siluet rata-rata:", round(pam3$silinfo$avg.width, 3), "\n")

siluet rata-rata: 0.127

cat("\nmedoid (wilayah yang mewakili tiap kluster):\n")

medoid (wilayah yang mewakili tiap kluster):

print(round(wil[pam3$id.med, c("kepadatan", "pengeluaran", "akses_air",
                              "akses_listrik", "lama_sekolah")], 1))
```

	kepadatan	pengeluaran	akses_air	akses_listrik	lama_sekolah
81	5822	2876	76.2	86.7	11.2
130	6751	2185	49.4	66.6	10.3
48	2886	2684	62.8	78.9	7.0

Interpretasi Output

Berbeda dari k-Means yang memakai **rata-rata** sebagai pusat, k-Medoid memakai **amatan sungguhan** — di sini, wilayah tertentu yang paling mewakili kelompoknya. Dua keuntungannya nyata pada pekerjaan statistik resmi. **Pertama**, pusat kluster dapat ditunjuk: "kelompok ini diwakili wilayah nomor sekian", bukan sekadar vektor rata-rata yang mungkin tidak sesuai dengan wilayah mana pun. **Kedua**, medoid jauh lebih tahan terhadap pencilan, sebab satu amatan ekstrem tidak menggeser medoid sebagaimana ia menggeser rata-rata.

Siluet rata-rata pada $k = 2$ sampai 6 berkisar **0,127 sampai 0,165** — seluruhnya sangat rendah. Sesuai rancangan data ini, yang dibangkitkan **tanpa struktur kluster yang disengaja**, hasil itu benar dan wajib dilaporkan apa adanya: tidak ada pengelompokan alami yang tegas pada kelima peubah numerik ini. Bahwa siluet justru tertinggi pada $k = 6$ bukan bukti adanya enam kelompok; pada data tanpa struktur, siluet dapat naik-turun tanpa pola bermakna.

Contoh 12.16 | k-Medoid untuk data kategorik dan campuran: jarak Gower

Kode 12.16 • R daisy dengan metrik Gower lalu pam pada matriks jarak

```
wil_c <- wil[, -1]
wil_c$status <- factor(wil_c$status)
wil_c$pulau <- factor(wil_c$pulau)
str(wil_c[, c("status", "pulau")])

'data.frame': 200 obs. of 2 variables:
 $ status: Factor w/ 2 levels "Perdesaan","Perkotaan": 2 1 1 1 2 2 2 2 1 1 ...
 $ pulau : Factor w/ 4 levels "Jawa","Papua",...: 2 2 1 1 4 1 4 1 3 1 ...

G <- daisy(wil_c, metric = "gower")
cat("kelas objek:", class(G), " ukuran:", attr(G, "Size"), "\n")

kelas objek: dissimilarity dist ukuran: 200

cat("rentang jarak Gower:", round(range(G), 3), "\n")

rentang jarak Gower: 0.047 0.807

sil_g <- sapply(2:6, function(k) pam(G, k, diss = TRUE)$silinfo$avg.width)
round(setNames(sil_g, paste0("k=", 2:6)), 3)

 k=2   k=3   k=4   k=5   k=6
0.295 0.193 0.174 0.206 0.230

pam_g <- pam(G, 3, diss = TRUE)
cat("ukuran kluster:", table(pam_g$clustering), "\n")
```

```
ukuran klaster: 82 65 53
```

```
print(table(klaster = pam_g$clustering, status = wil_c$status))
```

```
      status
klaster Perdesaan Perkotaan
      1         0         82
      2        65          0
      3        44          9
```

```
print(table(klaster = pam_g$clustering, pulau = wil_c$pulau))
```

```
      pulau
klaster Jawa Papua Sulawesi Sumatera
      1   29   18         18         17
      2   49    7          9          0
      3    1    8         10         34
```

Interpretasi Output

Jarak Euclidean tidak terdefinisi bagi peubah kategorik — selisih antara "Perkotaan" dan "Perdesaan" bukan bilangan. **Jarak Gower** menyelesaikannya dengan menghitung ketakmiripan per peubah lalu merata-ratakannya: untuk peubah numerik dipakai selisih mutlak yang dibagi rentangnya (sehingga hasilnya berada pada $[0, 1]$), sedangkan untuk peubah kategorik dipakai 0 bila sama dan 1 bila berbeda. Karena tiap peubah menyumbang pada skala yang sama, tidak diperlukan pembakuan — dan justru **tidak boleh** dibakukan lebih dahulu, sebab Gower sudah menormalkan sendiri.

Siluet pada jarak Gower (0,17 sampai 0,30) lebih tinggi daripada pada peubah numerik saja. Kenaikan itu **bukan** pertanda pengelompokannya lebih baik: peubah kategorik hanya memiliki sedikit nilai, sehingga banyak pasangan berjarak persis nol atau persis satu — dan sebaran jarak yang menggumpal seperti itu menaikkan siluet secara artifisial. Tabulasi silang memperlihatkan apa yang sesungguhnya terjadi: klaster terbentuk sebagian besar mengikuti pulau dan status, yakni peubah kategorik mendominasi pengelompokan.

Inilah masalah pembobotan yang selalu muncul pada data campuran. Lima peubah numerik dan dua peubah kategorik diperlakukan setara oleh Gower, padahal satu peubah kategorik berkategori empat dapat memisahkan data jauh lebih tegas daripada satu peubah numerik yang menyebar. Bila hal itu tidak dikehendaki, bobot tiap peubah harus ditetapkan sendiri lewat argumen `weights` pada `daisy()` — dan penetapan itu wajib dinyatakan dalam laporan.

Contoh 12.17 | k-Prototype untuk data campuran

Kode 12.17 • R kproto dan pengaruh pembakuan terhadap lambda

```
suppressPackageStartupMessages(library(clustMixType))
```

```
# (a) TANPA pembakuan peubah numerik
```

```
set.seed(2026)
```

```
kp_mentah <- kproto(wil_c, k = 3, verbose = FALSE)
```

```
cat("lambda tanpa pembakuan:", format(kp_mentah$lambda, digits = 5), "\n")
```

```
lambda tanpa pembakuan: 2563400
```

```
# (b) DENGAN pembakuan peubah numerik
```

```
wil_z <- wil_c
```

```
wil_z[, 1:5] <- scale(wil_z[, 1:5])
```

```
set.seed(2026)
```

```
kp_baku <- kproto(wil_z, k = 3, verbose = FALSE)
```

```
cat("lambda dengan pembakuan:", round(kp_baku$lambda, 3), "\n\n")
```

```
lambda dengan pembakuan: 1.648
```

```
cat("ukuran klaster -- tanpa baku:", kp_mentah$size,
    "| dengan baku:", kp_baku$size, "\n")
```

```
ukuran klaster -- tanpa baku: 63 69 68 | dengan baku: 73 63 64
```

```
print(table(tanpa_baku = kp_mentah$cluster, dengan_baku = kp_baku$cluster))
```

```
      dengan_baku
tanpa_baku 1  2  3
1  37  2 24
2   8 44 17
3  28 17 23
```

```
cat("\nprototipe (dengan pembakuan):\n")
```

```
prototipe (dengan pembakuan):
```

```
print(round(kp_baku$centers[, 1:5], 2))
```

```
      kepadatan pengeluaran akses_air akses_listrik lama_sekolah
1      0.64      0.47      0.40      -0.04      0.45
2     -0.86     -0.34      0.46     -0.49     -0.16
3      0.12     -0.20     -0.90      0.53     -0.36
```

```
print(kp_baku$centers[, 6:7])
```

```
      status pulau
1 Perdesaan Jawa
2 Perkotaan Jawa
3 Perdesaan Jawa
```

Interpretasi Output

k-Prototype menggabungkan k-Means bagi peubah numerik dan k-Modes bagi peubah kategorik. Jaraknya adalah jumlah jarak Euclidean kuadrat pada bagian numerik ditambah λ kali cacah ketidakcocokan pada bagian kategorik. Parameter λ karena itu menentukan **seberapa besar suara** peubah kategorik.

Perhatikan akibat pembakuan. Tanpa dibakukan, λ ditaksir sekitar **2.563.400** — angka raksasa yang muncul karena kepadatan bernilai ribuan dan pengeluaran bernilai ribuan pula, sehingga ragam bagian numerik sangat besar dan λ harus mengimbangnya. Setelah dibakukan, λ menjadi **1,648** — besaran yang wajar dan dapat ditafsirkan: satu ketidakcocokan kategori setara sekitar 1,6 satuan jarak kuadrat pada skala baku.

Yang lebih penting, keanggotaan klasternya **berbeda jauh**. Tabulasi silang kedua hasil memperlihatkan bahwa kesepakatannya rendah — pengelompokan berubah, bukan sekadar berganti nomor. Kaidahnya: pada k-Prototype, **bakukan peubah numerik lebih dahulu**, dan laporkan nilai λ yang dipakai. Menerima λ bawaan tanpa memeriksanya berarti membiarkan satuan pengukuran menentukan bobot antar-jenis peubah.

Contoh 12.18 | Membandingkan PAM-Gower dan k-Prototype

Kode 12.18 • R kesepakatan dua pendekatan pada data campuran

```
tab <- table(kproto = kp_baku$cluster, pam_gower = pam_g$clustering)
print(tab)
```

```
pam_gower
kproto 1 2 3
1 23 30 20
2 42 18 3
3 17 17 30
```

```
# indeks Rand terkoreksi antara kedua pengelompokan
```

```
ari <- function(a, b) {
  tb <- table(a, b); n <- sum(tb)
  s_ij <- sum(choose(tb, 2)); s_i <- sum(choose(rowSums(tb), 2))
  s_j <- sum(choose(colSums(tb), 2)); harap <- s_i * s_j / choose(n, 2)
  (s_ij - harap) / ((s_i + s_j) / 2 - harap)
}
cat("ARI kproto vs PAM-Gower:", round(ari(kp_baku$cluster, pam_g$clustering), 3), "\n")
```

```
ARI kproto vs PAM-Gower: 0.08
```



```
cat("ARI kproto vs PAM numerik:", round(ari(kp_baku$cluster, pam3$clustering), 3), "\n")
```

```
ARI kproto vs PAM numerik: 0.162
```

Interpretasi Output

Indeks Rand terkoreksi (ARI) antara k-Prototype dan PAM-Gower sangat rendah — keduanya menghasilkan pembagian yang praktis **tidak berhubungan**, meskipun bekerja pada data yang sama dan sama-sama menangani peubah campuran.

Ini bukan kekeliruan salah satu metode. Keduanya menimbang peubah numerik dan kategorik dengan cara yang berbeda: Gower merata-ratakan ketakmiripan per peubah pada skala $[0, 1]$, sedangkan k-Prototype menjumlahkan jarak kuadrat numerik dengan λ kali ketidakcocokan kategorik. Ketika data **tidak memuat struktur klaster yang tegas** — sebagaimana data ini — kedua pembobotan itu menemukan pembagian yang berbeda-beda, dan tidak ada dasar untuk menyatakan salah satunya benar.

Pelajarannya justru itu: ketidaksepakatan antar-metode pada data tanpa struktur adalah **temuan**, bukan kegagalan. Bila dua metode yang sama-sama sah memberi hasil yang tak berhubungan, kesimpulan yang jujur adalah "tidak ada pengelompokan alami yang tegas pada data ini", bukan memilih salah satu hasil lalu menamainya.

Contoh 12.19 | Mengevaluasi keempat metode dengan ukuran yang sama

Kode 12.19 • R cluster.stats untuk PAM, PAM-Gower, k-Prototype, DBSCAN, dan OPTICS

```
suppressPackageStartupMessages({library(fpc); library(dbscan)})

nilai <- function(kel, jarak, nama) {
  ada <- kel > 0                                     # buang derau bagi DBSCAN/OPTICS
  if (length(unique(kel[ada])) < 2) {
    cat(sprintf("%-12s kurang dari dua klaster\n", nama)); return(invisible(NULL))
  }
  st <- cluster.stats(as.dist(as.matrix(jarak)[ada, ada]), kel[ada])
  data.frame(metode = nama, klaster = st$cluster.number,
             siluet = round(st$avg.silwidth, 3), dunn = round(st$dunn, 3),
             CH = round(st$ch, 1),
             dalam = round(st$average.within, 3), antar = round(st$average.between, 3),
             derau = sum(!ada))
}

db <- dbscan::dbscan(X, eps = 1.2, minPts = 5)
opt <- dbscan::extractDBSCAN(dbscan::optics(X, minPts = 5), eps_cl = 1.2)
```

```
Gz <- daisy(wil_z, metric = "gower")

tabel <- rbind(
  nilai(pam3$clustering, dist(X, "PAM"),
  nilai(pam_g$clustering, G, "PAM-Gower"),
  nilai(kp_baku$cluster, Gz, "k-Prototype"),
  nilai(db$cluster, dist(X, "DBSCAN"),
  nilai(opt$cluster, dist(X, "OPTICS"))
print(tabel, row.names = FALSE)
```

metode kluster	siluet	dunn	CH	dalam	antar	derau
PAM	3	0.127	0.104	29.5	2.655	3.233
PAM-Gower	3	0.193	0.075	55.7	0.328	0.454
k-Prototype	3	0.088	0.083	21.9	0.372	0.431
DBSCAN	4	0.105	0.173	17.3	2.225	2.905
OPTICS	4	0.113	0.196	17.5	2.210	2.911

Interpretasi Output

Tabel ini menyajikan lima ukuran bagi kelima hasil, tetapi **tidak seluruhnya boleh dibandingkan langsung**. Bacalah per kelompok.

Di dalam keluarga berbasis medoid (PAM, PAM-Gower, k-Prototype), perbandingan sah selama jaraknya sama. PAM pada peubah numerik memberi siluet 0,127, k-Prototype 0,088 — keduanya rendah, sesuai rancangan data. PAM-Gower tampak lebih tinggi (0,193) tetapi **dihitung atas jarak yang berbeda** (Gower, bukan Euclidean), sehingga angkanya tidak sebanding; hal ini sudah dibahas pada Contoh 12.13.

Di dalam keluarga berbasis kepadatan (DBSCAN, OPTICS), keduanya menemukan 4 kluster dengan derau 76 dan 80 amatan. Indeks Dunn keduanya **lebih tinggi** daripada PAM (0,173 berbanding 0,104) — wajar, sebab Dunn mengukur nisbah pemisahan terkecil terhadap garis tengah terbesar, dan membuang derau menaikkan pemisahan itu.

Antar-keluarga, tabel ini justru memperlihatkan masalahnya. DBSCAN unggul pada Dunn tetapi kalah pada siluet dan CH; PAM sebaliknya. Tidak ada satu pun ukuran di sini yang dapat menjadi hakim, sebab keduanya **tidak menjawab pertanyaan yang sama** — dan yang lebih pokok, DBSCAN dinilai hanya atas 124 amatan yang tidak dianggap derau, sedangkan PAM dinilai atas seluruh 200. Membandingkan angka yang dihitung atas himpunan amatan berbeda adalah kekeliruan mendasar. Bagian penutup modul ini membahas cara membandingkan kedua keluarga secara adil.

Contoh 12.20 | Ukuran yang sah untuk membandingkan antar-keluarga metode

Kode 12.20 • R kesepakatan, kestabilan, dan cakupan

```
# (1) KESEPAKATAN antar-hasil: indeks Rand terkoreksi
ari <- function(a, b) { tb <- table(a, b); n <- sum(tb)
  s_ij <- sum(choose(tb, 2)); s_i <- sum(choose(rowSums(tb), 2))
  s_j <- sum(choose(colSums(tb), 2)); h <- s_i * s_j / choose(n, 2)
  (s_ij - h) / ((s_i + s_j) / 2 - h) }

ada <- db$cluster > 0 # bandingkan pada amatan yang SAMA
round(c(PAM_vs_DBSCAN = ari(pam3$clustering[ada], db$cluster[ada]),
  DBSCAN_vs_OPTICS = ari(db$cluster[ada], opt$cluster[ada]),
  PAM_vs_kproto = ari(pam3$clustering, kp_baku$cluster)), 3)
```

PAM_vs_DBSCAN	DBSCAN_vs_OPTICS	PAM_vs_kproto
0.290	0.945	0.162

```
# (2) CAKUPAN: berapa amatan yang benar-benar dikelompokkan
round(c(PAM = 1, DBSCAN = mean(db$cluster > 0), OPTICS = mean(opt$cluster > 0)), 3)
```

PAM	DBSCAN	OPTICS
1.00	0.62	0.60

```
# (3) KESTABILAN terhadap penarikan ulang sampel
set.seed(2026)
stabil <- function(fun, B = 30) {
  mean(replicate(B, {
    i <- sample(nrow(X), replace = TRUE); i <- unique(i)
    a <- fun(X[i, , drop = FALSE])
    b <- (fun(X))[i]
    ok <- a > 0 & b > 0
    if (length(unique(a[ok])) < 2 || length(unique(b[ok])) < 2) NA else ari(a[ok], b[ok])
  }), na.rm = TRUE)
}
round(c(PAM = stabil(function(M) pam(M, 3)$clustering),
  DBSCAN = stabil(function(M) dbscan::dbscan(M, eps = 1.2, minPts = 5)$cluster)), 3)
```

PAM	DBSCAN
0.319	0.615

Interpretasi Output

Tiga ukuran ini **sah** dipakai lintas keluarga, sebab tidak satu pun bergantung pada asumsi bentuk klas-ter.

(1) Kesepakatan. ARI dihitung hanya pada amatan yang *sama-sama* dikelompokkan kedua metode — inilah syarat yang dilanggar tabel sebelumnya. DBSCAN dan OPTICS hampir sepakat sempurna, sebagaimana seharusnya: OPTICS dengan `eps_cl` yang sama pada dasarnya memulihkan hasil DBSCAN. PAM versus DBSCAN jauh lebih rendah — keduanya membagi data dengan cara yang berbeda.

(2) Cakupan. PAM mengelompokkan 100% amatan; DBSCAN hanya sekitar 62%, sisanya ditandai derau. Angka ini **wajib** dilaporkan bersama ukuran mutu mana pun. Mutu tinggi yang diperoleh dengan membuang 38% amatan bukan mutu yang sebanding dengan mutu atas seluruh amatan — dan pada pekerjaan statistik resmi, wilayah yang "dibuang" itu justru sering yang paling perlu dijelaskan.

(3) Kestabilan. Mengukur seberapa konsisten hasil bila sampel ditarik ulang. Hasilnya di sini justru berlawanan dengan dugaan yang lazim: **DBSCAN lebih stabil (0,615) daripada PAM (0,319)**. Sebabnya masuk akal — PAM dipaksa membelah data menjadi tepat tiga bagian meskipun tidak ada batas alami, sehingga letak batas itu bergeser-geser antar-penarikan sampel; DBSCAN hanya menandai daerah berkepadatan tinggi dan membiarkan sisanya sebagai derau, dan daerah itu lebih bertahan.

Ukuran ini tidak memihak bentuk klaster tertentu sehingga adil dipakai lintas keluarga — dan contoh ini memperlihatkan mengapa ia perlu: menilai dari siluet saja akan memenangkan PAM, padahal hasil PAM justru yang lebih mudah berubah. Metode yang hasilnya berubah drastis pada penarikan ulang tidak layak dipakai menetapkan sasaran program, betapa pun tinggi siluetnya.

Kaidah pelaporan: bila dua keluarga metode dibandingkan, sajikan **cakupan**, **kesepakatan**, dan **kestabilan** — bukan siluet atau Dunn masing-masing yang dihitung atas himpunan amatan berbeda.

Contoh 12.21 | Bi-clustering: mengelompokkan baris dan kolom sekaligus

Kode 12.21 • R biclust pada data ekspresi

```
suppressPackageStartupMessages(library(biclust))
eks <- read.csv("data/data_ekspresi.csv")
E <- as.matrix(eks[, -1]); rownames(E) <- paste0("g", seq_len(nrow(E)))
cat("ukuran matriks:", dim(E), "\n")
```

```
ukuran matriks: 60 8
```

```
set.seed(2026)
bc <- biclust(E, method = BCCC(), delta = 1.0, alpha = 1.5, number = 5)
cat("banyak bicluster:", bc@Number, "\n")
```

```
banyak bicluster: 5
```

```
for (i in seq_len(bc@Number))
  cat(sprintf(" bicluster %d: %2d baris x %d kolom\n", i,
              sum(bc@RowNumber[, i]), sum(bc@NumberxCol[i, ])))
```

```
bicluster 1: 16 baris x 7 kolom
bicluster 2: 15 baris x 6 kolom
bicluster 3: 10 baris x 6 kolom
bicluster 4: 10 baris x 5 kolom
bicluster 5: 8 baris x 4 kolom
```

Interpretasi Output

Seluruh metode sebelumnya mengelompokkan **baris** dengan memakai **seluruh** kolom. Bi-clustering melepas pengandaian itu: ia mencari blok — sehimpuan baris yang berperilaku serupa **hanya pada sehimpuan kolom tertentu**.

Gagasan ini penting pada data berdimensi tinggi. Pada data ekspresi gen, sekelompok gen mungkin hanya bergerak bersamaan pada sebagian kondisi percobaan; pada data wilayah, sekelompok kabupaten mungkin hanya mirip pada sebagian indikator. Pengelompokan biasa akan melewatkan pola semacam itu, sebab kolom yang tidak relevan menyumbang derau pada perhitungan jarak.

Algoritma Cheng–Church yang dipakai di sini mencari blok ber-**skor residu rata-rata** di bawah ambang δ : blok yang di dalamnya nilai dapat dihipotesis dengan baik oleh efek baris ditambah efek kolom. Ditemukan lima blok berukuran mulai dari 8 sampai 16 baris pada 4 sampai 7 kolom.

Peringatan yang wajib menyertai: data ini dibangkitkan **tanpa blok yang ditanam**. Kelima blok itu karena itu adalah **artefak** — Cheng–Church akan selalu mengembalikan sebanyak yang diminta lewat argumen number, sebagaimana k-Means selalu memberi k kluster. Contoh berikutnya menguji hal ini dengan data yang polanya diketahui.

Contoh 12.22 | Menguji bi-clustering dengan blok yang sengaja ditanam

Kode 12.22 • R data uji berblok diketahui

```
set.seed(2026)
M <- matrix(rnorm(60 * 10), 60, 10)
baris_blok <- 1:12; kolom_blok <- 3:7
M[baris_blok, kolom_blok] <- M[baris_blok, kolom_blok] + 3 # blok ditanam
rownames(M) <- paste0("r", 1:60)

# dibinerkan lebih dahulu: 1 bila nilai melampaui 1, 0 bila tidak
B <- binarize(M, threshold = 1)
cat("proporsi sel bernilai 1:", round(mean(B), 3), "\n")
```

```
proporsi sel bernilai 1: 0.25
```

```
set.seed(2026)
bc2 <- biclust(B, method = BCBimax(), minr = 5, minc = 3, number = 3)
cat("banyak bicluster ditemukan:", bc2@Number, "\n")
```

```
banyak bicluster ditemukan: 3
```

```
for (i in seq_len(bc2@Number)) {
  br <- which(bc2@RowNumber[, i]); kl <- which(bc2@NumberCol[i, ])
  cat(sprintf(" bicluster %d: %2d baris, %d kolom | baris benar: %2d/%d, kolom benar: %d/%d\n",
```

```

i, length(br), length(kl),
length(intersect(br, baris_blok)), length(baris_blok),
length(intersect(kl, kolom_blok)), length(kolom_blok)))
}

```

```

bicluster 1: 12 baris, 4 kolom | baris benar: 12/12, kolom benar: 4/5
bicluster 2: 10 baris, 5 kolom | baris benar: 10/12, kolom benar: 5/5
bicluster 3: 6 baris, 3 kolom | baris benar: 5/12, kolom benar: 2/5

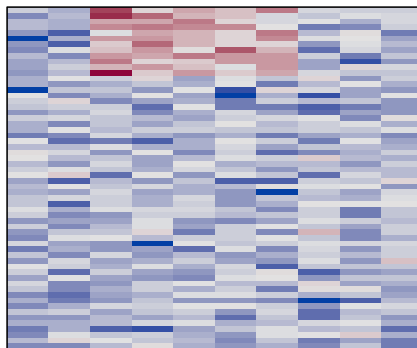
```

```

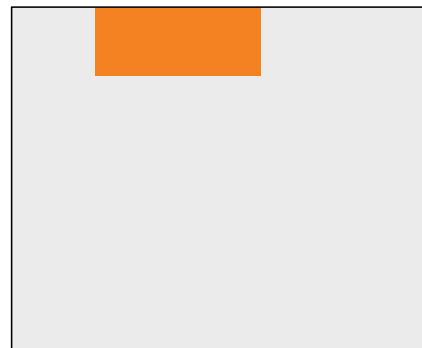
par(mfrow = c(1, 2), family = "sans", mar = c(4, 4, 2.5, 1))
image(t(M[60:1, ]), col = hcl.colors(30, "Blue-Red"), axes = FALSE,
      main = "matriks dengan blok ditanam")
box()
br1 <- which(bc2@RowxNumber[, 1]); kl1 <- which(bc2@NumberxCol[1, ])
tanda <- matrix(0, 60, 10); tanda[br1, kl1] <- 1
image(t(tanda[60:1, ]), col = c("grey92", "#F58220"), axes = FALSE,
      main = "blok yang ditemukan"); box()

```

matriks dengan blok ditanam



blok yang ditemukan



Gambar 12.5: Matriks dengan satu blok yang ditanam (kiri) dan hasil pencariannya (kanan)

```

par(mfrow = c(1, 1))

```

Interpretasi Output

Pada data yang blok-nya **sejati ada**, algoritma menemukannya kembali dengan tepat: bicluster pertama memuat **12 dari 12** baris yang ditanam dan 4 dari 5 kolomnya; bicluster kedua memuat 10 dari 12 baris dan **seluruh 5 kolom**. Perbandingan "baris benar" dan "kolom benar" memberi ukuran ketepatan yang dapat diperiksa — persis cara yang dipakai pada kajian terkendali di bagian akhir modul ini.

Perhatikan bahwa di sini dipakai algoritma **Bimax** atas matriks yang dibinerkan lebih dahulu, bukan Cheng–Church. Alasannya bukan selera: Cheng–Church mencari blok ber-residu rendah, dan pada matriks ini ia justru mengembalikan blok merosot berukuran $60 \text{ baris} \times 1 \text{ kolom}$ — sebab satu kolom tunggal selalu berresidu nol. Bimax mencari blok yang seluruh selnya bernilai 1, sehingga cocok bagi blok yang ditanam sebagai pergeseran nilai. **Pemilihan algoritma bi-clustering menentukan jenis blok apa yang dapat ditemukan**, dan itu harus diputuskan dari sifat pola yang dicari — bukan dicoba-coba sampai hasilnya enak dibaca.

Bandingkan dengan Contoh 12.16 pada data ekspresi yang tidak berblok: di sana lima blok juga "ditemukan", tetapi tidak ada kebenaran yang dapat dicocokkan. Perbedaan antara keduanya tidak terlihat dari keluaran algoritma — keduanya sama-sama mengembalikan daftar blok berikut ukurannya.

Kaidahnya sama dengan yang berlaku bagi seluruh metode pengelompokan pada modul ini: sebelum mempercayai blok yang ditemukan, jalankan **uji kendali** pada data acak berukuran sama. Bila blok berukuran serupa juga muncul di sana, temuan pada data sungguhan tidak dapat dibedakan dari kebetulan.

Contoh 12.23 | Menyiapkan matriks dan menyisipkan pola

Kode 12.23 • R menyusun matriks dan peta panas

```
ekspresi <- read.csv("data/data_ekspresi.csv")
M <- as.matrix(ekspresi[, paste0("s", 1:8)])
rownames(M) <- paste0("gen", ekspresi$gen)

# menyisipkan pola: 12 gen pertama terekspresi tinggi pada 3 contoh pertama
M[1:12, 1:3] <- M[1:12, 1:3] + 2.5

cat("dimensi matriks:", dim(M), "\n")

dimensi matriks: 60 8

cat("rata-rata seluruh:", round(mean(M), 3), "\n")

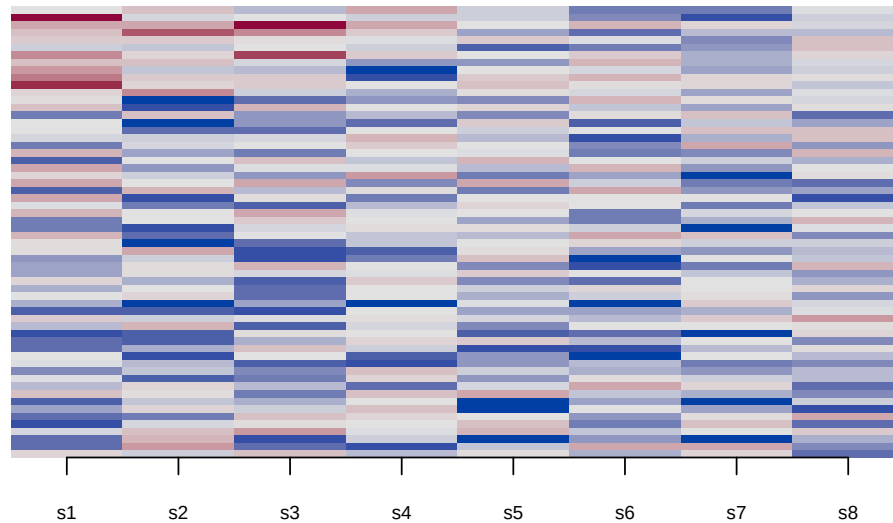
rata-rata seluruh: 0.176

cat("rata-rata blok sisipan:", round(mean(M[1:12, 1:3]), 3), "\n")

rata-rata blok sisipan: 2.062

image(t(M[nrow(M):1, ]), col = hcl.colors(30, "Blue-Red"), axes = FALSE,
      main = "Peta Panas Matriks Ekspresi (pola disisipkan)")
axis(1, at = seq(0, 1, length.out = 8), labels = colnames(M), cex.axis = .7)
```

Peta Panas Matriks Ekspresi (pola disisipkan)



Penjelasan Kode. Matriks dibentuk dari delapan kolom contoh. Sebuah pola sengaja disisipkan: dua belas gen pertama dinaikkan nilainya hanya pada tiga contoh pertama. Peta panas digambar dengan matriks dibalik urutan barisnya agar baris pertama tampil di atas.

Interpretasi Output

Pola yang disisipkan inilah yang seharusnya ditemukan bi-clustering: sekelompok baris yang berperilaku khas hanya pada sekelompok kolom. Pengelompokan baris biasa akan kesulitan menemukannya, sebab pada kelima kolom lainnya kedua belas gen itu tidak berbeda dari gen lain. Menyisipkan pola secara sengaja adalah cara yang baik untuk menguji apakah suatu algoritma sungguh bekerja.

Contoh 12.24 | Menjalankan algoritma Cheng-Church

Kode 12.24 • R biclust dengan metode BCCC

```
suppressPackageStartupMessages(library(biclust))

set.seed(2026)
bc <- biclust(M, method = BCCC(), delta = 1.0, alpha = 1.5, number = 5)

cat("bikluster ditemukan:", bc@Number, "\n\n")
```


bikluster ditemukan: 5

```
if (bc@Number > 0) {
  for (i in seq_len(bc@Number)) {
    baris <- which(bc@RowxNumber[, i])
    kolom <- which(bc@NumberxCol[i, ])
    cat(sprintf("Bikluster %d: %d baris x %d kolom\n", i,
               length(baris), length(kolom)))
    cat("  kolom:", paste(colnames(M)[kolom], collapse = ", "), "\n")
    cat("  baris pertama:", paste(rownames(M)[utils::head(baris, 6)],
                                   collapse = ", "), "\n\n")
  }
}
```

Bikluster 1: 11 baris x 7 kolom

kolom: s1, s2, s3, s5, s6, s7, s8

baris pertama: gen42, gen34, gen2, gen42, gen1, gen9

Bikluster 2: 11 baris x 6 kolom

kolom: s1, s2, s4, s5, s6, s8

baris pertama: gen29, gen1, gen15, gen51, gen33, gen21

Bikluster 3: 9 baris x 6 kolom

kolom: s1, s2, s3, s5, s6, s7

baris pertama: gen35, gen10, gen26, gen29, gen46, gen23

Bikluster 4: 11 baris x 5 kolom

kolom: s2, s3, s5, s6, s7

baris pertama: gen16, gen22, gen20, gen2, gen21, gen12

Bikluster 5: 10 baris x 5 kolom

kolom: s3, s4, s5, s7, s8

baris pertama: gen34, gen9, gen18, gen52, gen25, gen1

Penjelasan Kode. Algoritma Cheng-Church dijalankan dengan ambang skor sisaan delta, faktor pembesaran alpha, dan banyaknya bikluster yang dicari. Hasilnya disimpan sebagai dua matriks logika: keanggotaan baris dan keanggotaan kolom bagi tiap bikluster.

Interpretasi Output

Periksa apakah bikluster pertama memuat gen bernomor kecil pada kolom s1 sampai s3 — itulah pola yang sengaja disisipkan. Bila ya, algoritma bekerja sebagaimana mestinya. Perhatikan bahwa nilai delta menentukan segalanya: terlalu besar menghasilkan bikluster raksasa tanpa makna, terlalu kecil membuat tak ada yang ditemukan. Seperti Eps pada DBSCAN, nilainya wajib dilaporkan bersama hasilnya.

Contoh 12.25 | Menggambarkan bikluster yang ditemukan**Kode 12.25 • R** **matplot** membandingkan anggota dan bukan anggota

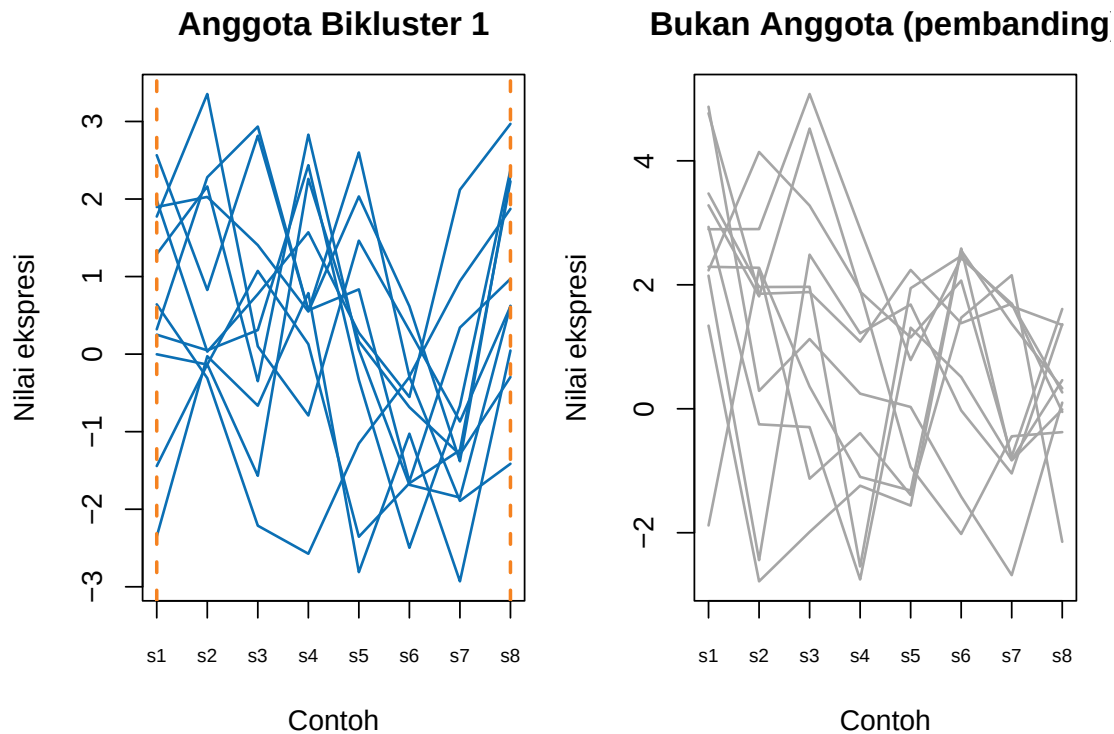
```

if (bc@Number > 0) {
  baris1 <- which(bc@RowxNumber[, 1])
  kolom1 <- which(bc@NumberxCol[1, ])

  par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
  matplot(t(M[baris1, , drop = FALSE]), type = "l", lty = 1, lwd = 1.5,
    col = "#0B6FB4", xaxt = "n",
    xlab = "Contoh", ylab = "Nilai ekspresi",
    main = "Anggota Bikluster 1")
  axis(1, at = 1:ncol(M), labels = colnames(M), cex.axis = .7)
  abline(v = range(kolom1), col = "#F58220", lty = 2, lwd = 2)

  bukan <- setdiff(seq_len(nrow(M)), baris1)
  matplot(t(M[utils::head(bukan, length(baris1)), , drop = FALSE]),
    type = "l", lty = 1, lwd = 1.5, col = "grey65", xaxt = "n",
    xlab = "Contoh", ylab = "Nilai ekspresi",
    main = "Bukan Anggota (pembandingan)")
  axis(1, at = 1:ncol(M), labels = colnames(M), cex.axis = .7)
  par(mfrow = c(1, 1), mar = c(5, 4, 4, 2) + 0.1)
}

```



Penjelasan Kode. Nilai tiap gen anggota bikluster digambar sebagai garis lintas kedelapan contoh, dengan garis tegak menandai batas kolom yang termasuk bikluster. Sejumlah gen bukan anggota digambar sebagai pembanding pada panel kedua.

Interpretasi Output

Perbandingan berdampingan inilah cara memeriksa apakah bikluster bermakna. Garis anggota seharusnya menunjukkan pola yang serupa satu sama lain pada kolom yang termasuk bikluster, sedangkan garis bukan anggota tampak acak. Bila kedua panel tampak serupa, biklusternya tidak bermakna meski algoritma melaporkannya sebagai temuan.

Bi-clustering juga selalu memberi jawaban. Sebagaimana pengelompokan biasa, algoritma bi-clustering akan mengembalikan bikluster pada data apa pun, termasuk data acak murni. Cara memeriksa adalah menjalankannya pada matriks acak berukuran sama: bila bikluster yang ditemukan pada data sungguhan tidak lebih menonjol daripada pada data acak, maka temuannya semu.

Tips dan Praktik Baik. Bi-clustering jarang diperlukan pada data statistik resmi yang lazim berpeubah puluhan. Ia menjadi penting pada data berdimensi sangat tinggi — ekspresi gen berpeubah ribuan, atau matriks penilaian pengguna terhadap produk — ketika kemiripan memang hanya berlaku pada sebagian peubah.

Membandingkan antar-keluarga: bukti terkendalinya. Contoh 12.17 memberi tiga ukuran yang sah dipakai lintas keluarga — kesepakatan, cakupan, dan kestabilan — beserta alasan mengapa siluet dan Dunn tidak sah untuk itu. Alasan tersebut **dibuktikan** pada **Modul 13** lewat kajian terkendali: tiga data uji berpola diketahui, ukuran mutu yang keliru menghakimi, dan prosedur perbandingan yang adil. Modul tersebut bersifat pelengkap, tetapi sangat dianjurkan dibaca sebelum menuliskan perbandingan metode pada laporan.

12.7 Latihan Mandiri

Kerjakan bertahap. Gunakan data/data_wilayah.csv.

12.7.0.0.1 Tingkat Dasar (melengkapi kode)

Latihan 12.1 | Pembakuan

```
wil <- read.csv("data/data_wilayah.csv", stringsAsFactors = TRUE)
num <- c("kepadatan", "pengeluaran", "akses_air", "akses_listrik", "lama_sekolah")
X <- ____ (wil[num])           # pembakuan WAJIB
round(colMeans(X), 8)         # harus bernilai nol
```

Latihan 12.2 | K-Medoids

```
library(cluster)
set.seed(2026)
p <- ____ (X, k = 3)           # K-Medoids
cat("siluet rata-rata:", p$silinfo$, "\n")
wil[p$, num]                  # amatan yang menjadi medoid
```

Latihan 12.3 | DBSCAN

```
library(dbSCAN)
minpts <- 2 * length(num)
jk <- sort(dbSCAN::____(X, k = minpts)) # jarak ke tetangga ke-k
db <- dbSCAN::____(X, eps = 1.5, minPts = minpts)
cat("derau:", sum(db$cluster == ____), "\n")
```

12.7.0.0.2 Tingkat Menengah (menulis sendiri)

Latihan 12.4 | Menentukan banyaknya kluster

Tentukan banyaknya kluster dengan **dua cara**: metode siku dan siluet rata-rata, masing-masing untuk k dari 2 sampai 10. Sajikan tabel dan grafik keduanya. Bandingkan jawabannya, lalu tentukan k yang Anda pilih — dan bila **kedua cara tidak memberi jawaban yang meyakinkan**, nyatakan hal itu beserta alasannya.

Latihan 12.5 | Membetulkan kode yang salah

Kode berikut memuat empat kesalahan (jarak dihitung tanpa pembakuan, dbSCAN tanpa penulisan nama

paket sehingga bentrok dengan fpc, siluet dihitung terhadap data tak terbakukan, dan derau DBSCAN ikut dihitung sebagai klaster). Temukan dan perbaiki, lalu jelaskan akibat masing-masing.

```
library(cluster); library(dbSCAN); library(fpc)
wil <- read.csv("data/data_wilayah.csv", stringsAsFactors = TRUE)
num <- c("kepadatan", "pengeluaran", "akses_air", "akses_listrik", "lama_sekolah")

# (a) jarak dihitung pada satuan asli
D <- dist(wil[num])
p <- pam(D, k = 3)

# (b) dbSCAN bentrok dengan fpc::dbSCAN
db <- dbSCAN(wil[num], eps = 1.5, minPts = 10)

# (c) siluet dihitung terhadap data TAK terbakukan
sil <- silhouette(p$clustering, dist(wil[num]))

# (d) derau ikut dicacah sebagai klaster
cat("banyaknya klaster:", length(unique(db$cluster)), "\n")
```

Keluaran yang benar. Setelah diperbaiki, kode yang dipakai adalah sebagai berikut.

```
# (a) pembakuan mendahului perhitungan jarak
X <- scale(wil[num])
D <- dist(X)
p <- pam(D, k = 3)

# (b) nama paket ditulis tegas
minpts <- 2 * length(num)
db <- dbSCAN::dbSCAN(X, eps = 1.5, minPts = minpts)

# (c) siluet dihitung terhadap data yang sama dengan pengelompokannya
sil <- silhouette(p$clustering, dist(X))

# (d) derau (klaster 0) dikeluarkan saat mencacah
cat("banyaknya klaster:", length(setdiff(unique(db$cluster), 0)), "\n")
cat("derau:", sum(db$cluster == 0), "\n")

Pemeriksaan yang wajib berhasil: all(abs(round(colMeans(X), 8)) == 0) bernilai TRUE, dan !(0 %in%
setdiff(unique(db$cluster), 0)) bernilai TRUE.
```

Latihan 12.6 | Menguji validitas siluet

Bangkitkan data dua cincin sepusat sebagaimana Contoh 12.10. Hitung indeks siluet bagi **pengelompokan yang sebenarnya** dan bagi pengelompokan berbasis jarak dengan $k = 2$. Hitung pula ARI keduanya. Laporkan mana yang bersiluet lebih tinggi, lalu jelaskan mengapa hal itu terjadi dan apa akibatnya bila siluet dipakai memilih metode.

Latihan 12.7 | Perbandingan yang adil

Bangkitkan data bergumpal disertai 10 persen derau seragam. Jalankan kedua keluarga metode, lalu laporkan: (a) cakupan masing-masing; (b) ARI atas seluruh amatan; (c) ARI pada himpunan bagian yang dikelompokkan keduanya. Simpulkan dari mana keunggulan metode berbasis kepadatan sesungguhnya berasal.

12.7.0.0.3 Tingkat Lanjut (pemilihan metode)

Latihan 12.8 | Memilih dan membandingkan metode

Untuk masing-masing keadaan berikut, tentukan keluarga metode yang Anda pilih beserta alasannya, dan sebutkan **ukuran penilaian yang sah** dipakai: (a) klaster diduga berbentuk memanjang dan sebagian amatan merupakan pencilan yang ingin ditandai; (b) klaster diduga membulat dan seluruh amatan harus masuk kelompok; (c) label sebenarnya tersedia dari sumber lain; (d) label tidak tersedia dan bentuk klaster tidak diketahui sama sekali.

12.8 Penugasan

Sebagai calon analis pada instansi statistik, lakukan pengelompokan wilayah atas `data_wilayah.csv` (*data simulasi*) memakai beberapa metode, evaluasi hasilnya, lalu simpulkan apakah data ini memang memiliki struktur klaster.

Konteks. Sebuah unit hendak menyusun tipologi wilayah berdasarkan indikator sosial-ekonomi. Pimpinan meminta bukan hanya hasil pengelompokan melainkan juga penilaian apakah pengelompokan itu bermakna.

Tugas 12.1 | Penyiapan

1. Periksa **perbedaan skala** antar-peubah numerik. Laporkan rasio simpangan baku terbesar terhadap terkecil, lakukan pembakuan, lalu buktikan hasilnya dengan memeriksa rata-rata dan simpangan baku kolomnya.

Tugas 12.2 | Metode Partisi

2. Bentuk satu pengelompokan berbasis jarak dengan $k = 3$ sebagai **pembanding**. Laporkan ukuran klasternya saja; cara kerja metodenya tidak perlu dibahas.

Tugas 12.3 | Metode Berbasis Kepadatan

6. Tentukan **Eps** melalui grafik jarak tetangga ke- k dan tetapkan **MinPts** dengan aturan praktis. Sajikan grafiknya beserta alasan pemilihan nilainya.
7. Terapkan **DBSCAN**. Laporkan banyaknya klaster, persentase derau, lalu periksa **kepekaannya** dengan mencoba minimal delapan nilai Eps.
8. Terapkan **OPTICS** dan sajikan **diagram jangkauannya**. Bandingkan hasilnya terhadap DBSCAN, lalu jelaskan keunggulan OPTICS pada data berkepadatan beragam.

Tugas 12.4 | Evaluasi dan Kesimpulan

9. Hitung **indeks siluet** dan **indeks Dunn** bagi hasil K-Medoids. Sajikan diagram siluet, laporkan amatan bersiluet negatif, lalu jelaskan mengapa Dunn peka terhadap pencilan.
10. Tentukan banyaknya klaster dengan **metode siku dan siluet rata-rata** untuk k dari 2 sampai 10. Sajikan keduanya dan bandingkan jawabannya.

11. Susun **profil tiap klaster** dalam satuan asli beserta sebaran peubah kategorinya. Berikan nama bagi tiap klaster, disertai catatan bahwa penamaan berasal dari penafsiran analisis.
12. **Simpulkan apakah data ini memiliki struktur klaster yang berarti.** Dasarkan kesimpulan pada nilai siluet, bentuk kurva siku, dan diagram jangkauan — bukan pada keberhasilan algoritma menghasilkan klaster.

Tugas 12.5 | Kajian Perbandingan Dua Keluarga Metode

10. Bangkitkan **tiga data uji berlabel diketahui**: gumpalan membulat, cincin sepusat, dan gumpalan disertai 10 persen derau. Sajikan gambar ketiganya.
11. Jalankan **kedua keluarga metode** pada ketiga data dengan prapemrosesan yang sama. Sajikan tabel memuat banyaknya klaster, cakupan, dan ARI.
12. **Buktikan bahwa indeks siluet tidak sah** dipakai membandingkan antar-keluarga: hitung siluet bagi pengelompokan yang sebenarnya benar pada data cincin, lalu bandingkan terhadap siluet pengelompokan berbasis jarak yang keliru. Jelaskan sebabnya.
13. Terapkan **lima langkah perbandingan yang adil** pada data berderau. Laporkan ARI atas seluruh amatan dan atas himpunan bagian yang sama, lalu jelaskan dari mana keunggulan yang terlihat sesungguhnya berasal.
14. Susun **tabel rangkuman kajian** yang memuat kesesuaian tiap keluarga metode dan ukuran penilaian yang sah bagi tiap keadaan data.
15. Susun laporan R Markdown berisi kode, keluaran, gambar, dan interpretasi. Tutup dengan **ringkasan (maksimal 300 kata)** memuat: metode yang dipakai beserta parameternya, kesimpulan mengenai ada tidaknya struktur pada data wilayah, hasil pokok kajian perbandingan, dan penegasan bahwa data merupakan simulasi.

12.9 Ringkasan Modul

Daftar periksa penguasaan:

- ☐ Saya membakukan data sebelum melakukan pengelompokan apa pun.
- ☐ Saya dapat menjelaskan titik inti, titik batas, dan derau.
- ☐ Saya menetapkan Eps melalui grafik jarak tetangga ke-k, bukan menebak.
- ☐ Saya memeriksa kepekaan DBSCAN terhadap Eps sebelum menyimpulkan.

- ☐ Saya dapat membaca diagram jangkauan OPTICS.
- ☐ Saya mengevaluasi dengan siluet dan Dunn, serta memahami kepekaan Dunn.
- ☐ Saya menentukan banyaknya kluster dengan dua cara dan membandingkannya.
- ☐ Saya tahu bahwa siluet *tidak sah* dipakai mengadu antar-keluarga metode.
- ☐ Saya melaporkan cakupan ketika membandingkan metode yang menandai derau.
- ☐ Saya membandingkan pada himpunan bagian yang sama, bukan atas himpunan berbeda.
- ☐ Saya memakai ARI ketika label sebenarnya tersedia.
- ☐ Saya menyadari bahwa algoritma selalu memberi kluster meski data tak berstruktur.

Kartu Referensi Cepat.

Keperluan	Perintah
Pembakuan	<code>scale(X)</code>
Matriks jarak	<code>dist(X)</code>
Pembandingan berbasis jarak	<code>cluster::pam(X, k = 3)</code>
Jarak tetangga ke-k	<code>dbscan::kNNdist(X, k =)</code>
DBSCAN	<code>dbscan::dbscan(X, eps =, minPts =)</code>
OPTICS	<code>dbscan::optics(X, eps =, minPts =)</code>
Ekstrak dari OPTICS	<code>dbscan::extractDBSCAN(op, eps_cl =)</code>
Indeks siluet	<code>cluster::silhouette(klaster, dist(X))</code>
Indeks Dunn	<code>fpc::cluster.stats(dist(X), klaster)\$dunn</code>
Profil kluster	<code>aggregate(d, by = list(k), FUN = mean)</code>
Ukuran eksternal	<code>mclust::adjustedRandIndex(a, b)</code>
Cakupan	<code>mean(db\$cluster > 0)</code>
Bi-clustering	<code>biclust::biclust(M, method = BCCC())</code>

Modul ini memusatkan perhatian pada metode berbasis kepadatan dan pada cara menilai hasil pengelompokan. Prinsip yang perlu dibawa: bakukan sebelum mengelompokkan; tetapkan parameter secara terarah lalu laporkan seluruhnya; evaluasi dengan lebih dari satu ukuran; laporkan cakupan bila sebagian amatan ditandai derau; bandingkan pada himpunan bagian yang sama; pakai ukuran eksternal bila label tersedia dan jangan mengadu ukuran internal antar-keluarga metode; dan yang terpenting, periksa apakah struktur yang ditemukan sungguh berarti — sebab algoritma akan selalu memberi jawaban meski datanya tidak berkluster sama sekali.

12.10 Referensi

-
- Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann.
- Kaufman, L., & Rousseeuw, P. J. (1990). *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *KDD-96*, 226-231.
- Ankerst, M., Breunig, M. M., Kriegel, H.-P., & Sander, J. (1999). OPTICS: Ordering points to identify the clustering structure. *SIGMOD 1999*, 49-60.
- Cheng, Y., & Church, G. M. (2000). Biclustering of expression data. *ISMB 2000*, 93-103.
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53-65.
- Hubert, L., & Arabie, P. (1985). Comparing partitions. *Journal of Classification*, 2(1), 193-218.

BAGIAN PENGAYAAN



Modul 13–16

Materi pelengkap di luar RPS

MODUL

13

GPBOOST: BOOSTING UNTUK DATA BERTINGKAT DENGAN R DAN PYTHON

Pertemuan Minggu ke-13

Sub-CPMK 13: Mahasiswa mampu mengenali struktur data bertingkat, menjelaskan mengapa boosting biasa dan model campuran linear masing-masing tidak memadai, menerapkan GPBoost yang menggabungkan keduanya, menaksir serta menafsirkan komponen ragam dan efek acak, dan membandingkan keempat pendekatan dengan ukuran yang sama.

Penyusun: Achmad Syahrul Choir



Daftar Isi

13.1 Deskripsi Modul	422
13.2 Tujuan Praktikum	423
13.3 Alat dan Bahan	423
13.3.1 Perangkat Lunak	423
13.3.2 Bahan (Berkas Data)	424
13.4 Instruksi Pengerjaan	424
13.5 Teori Singkat	424
13.5.1 Data Bertingkat dan Efek Acak	424
13.5.2 Cara Kerja GPBoost	425
13.6 Contoh Praktikum dengan R dan Python	426
13.7 Contoh Praktikum: GPBoost dengan Python	429
13.8 Padanan Kode dalam R	435
13.9 Latihan Mandiri	438
13.10 Penugasan	440
13.10.1 Rubrik Penilaian	441
13.11 Ringkasan Modul	441
13.11.1 Daftar Istilah	442
13.12 Referensi	442

13.1 Deskripsi Modul

Seluruh modul pemodelan sebelumnya — klasifikasi dasar, klasifikasi lanjut, jaringan syaraf tiruan — mengandaikan satu hal yang jarang dinyatakan terus terang: bahwa **amatan saling bebas**. Pada data statistik resmi, pengandaian itu hampir selalu keliru. Rumah tangga tersarang di dalam desa, desa di dalam kabupaten, siswa di dalam sekolah, pasien di dalam rumah sakit. Dua rumah tangga dari desa yang sama lebih mirip satu sama lain daripada dua rumah tangga dari desa berbeda — karena berbagi harga pasar, akses jalan, mutu layanan, dan banyak hal lain yang tidak terekam sebagai peubah.

Ada dua tradisi yang masing-masing menangani setengah persoalan. **Model campuran** (*mixed models*) menangani struktur kelompok lewat **efek acak**, tetapi memaksa hubungan antara peubah penjelas dan respons berbentuk **linear**. **Boosting** menangani hubungan nonlinear dan interaksi dengan sangat baik, tetapi memperlakukan seluruh amatan sebagai bebas sehingga struktur kelompok terabaikan.

GPBoost menggabungkan keduanya: bagian tetap dimodelkan dengan **tree-boosting**, bagian acak dengan **proses Gaussian atau efek acak berkelompok**, dan keduanya ditaksir bersama secara berselang-seling. Modul ini menerapkannya pada data rumah tangga bertingkat, lalu membandingkan empat pendekatan pada data uji yang sama.

Modul memakai **dua bahasa**. R dipakai untuk pembangkitan data, penelaahan struktur, dan model pembandingan (`lme4`); Python dipakai untuk GPBoost. Keduanya membaca **berkas data yang sama**, sehingga hasilnya dapat dibandingkan langsung.

Kedudukan modul dan catatan data. Modul ini termasuk **bagian pengayaan**. Ia mengandaikan Modul 6 sampai 9 sudah dikuasai, khususnya gagasan pembagian latih–uji, validasi silang, dan pencegahan kebocoran data.

Data bersifat **ilustratif hasil simulasi** dan **bukan angka resmi** instansi mana pun. Efek acak tiap wilayah **sengaja ditanam** pada saat pembangkitan dan disimpan pada berkas terpisah, sehingga dapat diperiksa apakah tiap model berhasil memulihkannya — pemeriksaan yang tidak mungkin dilakukan pada data nyata.

Identitas Modul

Mata Kuliah	:	Data Mining dan Big Data (K202505)
Pertemuan	:	Minggu ke-13 (pengayaan)
Bahan Kajian	:	GPBoost — boosting untuk data bertingkat
Sub-CPMK	:	Sub-CPMK 13
Bahasa	:	R (<code>lme4</code>) dan Python (<code>gpboost</code>)
Alokasi Waktu	:	Praktikum 1 × 170 menit

13.2 Tujuan Praktikum

Setelah menyelesaikan praktikum Modul 13, mahasiswa diharapkan mampu:

1. Mengenali **struktur bertingkat** pada data dan menjelaskan mengapa ia melanggar pengandaian kebebasan amatan.
2. Menghitung dan menafsirkan **koefisien korelasi intrakelas** (ICC) sebagai ukuran kuatnya struktur kelompok.
3. Menjelaskan keterbatasan **boosting biasa** dan **model campuran linear**, masing-masing pada setengah persoalan yang berbeda.
4. Menerapkan **GPBoost** dengan Python: menyusun `GPMoDel`, menentukan banyaknya iterasi lewat validasi silang, dan melatih model.
5. Menaksir dan menafsirkan **komponen ragam** — ragam antar-kelompok dan ragam galat.
6. Memulihkan **efek acak** tiap wilayah dan memeriksa ketepatannya terhadap nilai yang ditanam.
7. Membandingkan **empat pendekatan** pada data uji yang sama dan menjelaskan urutan hasilnya.
8. Menjelaskan perbedaan ramalan bagi kelompok **yang sudah terlihat** dan kelompok **baru**.

13.3 Alat dan Bahan

13.3.1 Perangkat Lunak

- **R** (≥ 4.2) dengan paket `lme4` (model campuran linear).
- **Python** (≥ 3.9) dengan paket `gpboost`, `numpy`, dan `pandas`.

Kode 13.1 • R pemasangan paket

```
install.packages("lme4")           # R
```

```
# Python (jalankan di terminal)
```

```
# pip install gpboost numpy pandas
```

Tentang pemasangan gpboost. Paket `gpboost` tersedia untuk **R** maupun **Python** dengan antarmuka yang hampir sama — keduanya membungkus pustaka C++ yang sama. Modul ini menjalankan **keduanya**: Contoh 13.4–13.7 memakai Python, dan Bagian 13.8 mengulang analisis yang sama dengan R.

Pada Linux, memasang versi R dari CRAN berarti **mengompilasinya dari sumber** — ia membawa pustaka Eigen dan LightGBM — dan dapat memakan waktu setengah jam. Jauh lebih cepat memakai **berkas biner** dari Posit Package Manager:

```
options(repos = c(P3M =  
  "https://packagemanager.posit.co/cran/__linux__/noble/latest"))
```



```
options(HTTPUserAgent = sprintf("R/%s R (%s)", getRversion(),
  paste(getRversion(), R.version["platform"],
    R.version["arch"], R.version["os"])))
```

```
install.packages("gpboost")
```

Ganti `noble` dengan nama sandi distribusi Anda (jammy untuk Ubuntu 22.04). Baris `HTTPUserAgent` wajib disertakan — tanpanya server tetap mengirimkan berkas sumber.

13.3.2 Bahan (Berkas Data)

Berkas	Isi
data/rumahtangga_wilayah.csv	1.200 rumah tangga dalam 60 wilayah; peubah penjelas dan pengeluaran
data/efek_wilayah_sebenarnya.csv	efek acak tiap wilayah yang ditanam — hanya untuk pemeriksaan

Berkas `gen_data.R` memuat kode pembangkitnya lengkap dengan `set.seed(2026)`.

13.4 Instruksi Pengerjaan

1. Buat **RStudio Project**, salin folder `data/`, dan pastikan Python beserta `gpboost` dapat dipanggil.
2. Kerjakan Contoh berurutan; perhatikan bahwa potongan R dan Python **berbagi berkas data yang sama**.
3. Kerjakan **Latihan Mandiri**, lalu **Penugasan** sebagai laporan.

13.5 Teori Singkat

13.5.1 Data Bertingkat dan Efek Acak

Misalkan y_{ij} adalah respons rumah tangga ke- i pada wilayah ke- j . Model campuran dengan intersep acak menulis

$$y_{ij} = F(\mathbf{x}_{ij}) + b_j + \varepsilon_{ij}, \quad b_j \sim N(0, \sigma_b^2), \quad \varepsilon_{ij} \sim N(0, \sigma^2)$$

dengan b_j **efek acak** wilayah dan $F(\cdot)$ **pengaruh tetap**.

Koefisien korelasi intrakelas

$$\text{ICC} = \frac{\sigma_b^2}{\sigma_b^2 + \sigma^2}$$

mengukur proporsi keragaman yang berasal dari perbedaan antar-wilayah. $ICC = 0$ berarti wilayah tidak berperan; ICC mendekati 1 berarti hampir seluruh keragaman terletak antar-wilayah.

Perbedaan ketiga pendekatan terletak pada bagaimana $F(\cdot)$ dan b_j diperlakukan:

Empat pendekatan dan yang ditanganinya.

Pendekatan	$F(\cdot)$ nonlinear?	Efek kelompok b_j ?
Regresi linear (OLS)	tidak (linear)	tidak (diabaikan)
Boosting biasa	ya	tidak (diabaikan)
Model campuran linear	tidak (linear)	ya
GPBoost	ya	ya

13.5.2 Cara Kerja GPBoost

GPBoost menaksir $F(\cdot)$ dan komponen ragam secara **berselang-seling**:

1. Diberikan taksiran komponen ragam, satu pohon ditambahkan pada ansambel boosting — tetapi gradiennya dihitung terhadap **likelihood model campuran**, bukan terhadap sisaan biasa.
2. Diberikan ansambel pohon terbaru, komponen ragam σ_b^2 dan σ^2 ditaksir ulang dengan memaksimalkan likelihood.
3. Ulangi sampai banyaknya iterasi tercapai.

Ramalan bagi kelompok yang sudah terlihat menjumlahkan pengaruh tetap dan efek acak kelompok itu; bagi kelompok **baru**, efek acaknya diramalkan nol — yakni kembali ke pengaruh tetap saja.

Mengapa langkah 1 penting. Bila boosting dijalankan lebih dahulu lalu efek acak ditaksir dari sisanya, pohon-pohon akan lebih dahulu “memakan” keragaman antar-kelompok — sebab peubah penjelas dapat menghampiri identitas kelompok. Akibatnya ragam antar-kelompok tertaksir terlalu kecil dan model menjadi terlalu percaya diri. Penaksiran berselang-seling menghindari hal ini.

Alur praktikum Modul 13

1. Menelaah struktur bertingkat dan menghitung ICC (R).
2. Menunjukkan keterbatasan regresi linear dan model campuran linear (R).
3. Menjalankan GPBoost: penentuan iterasi, pelatihan, komponen ragam (Python).
4. Memulihkan efek acak dan memeriksanya terhadap nilai yang ditanam (Python).
5. Membandingkan keempat pendekatan pada data uji yang sama.
6. Meramalkan bagi wilayah baru.

13.6 Contoh Praktikum dengan R dan Python

Contoh 13.1 | Membaca data dan mengenali struktur bertingkat

Kode 13.2 · R struktur data dan sebaran antar-wilayah

```
rt <- read.csv("data/rumah_tangga_wilayah.csv", stringsAsFactors = FALSE)
str(rt)
```

```
'data.frame': 1200 obs. of 6 variables:
 $ id_rt      : chr  "RT0001" "RT0002" "RT0003" "RT0004" ...
 $ wilayah    : chr  "W01" "W01" "W01" "W01" ...
 $ lama_sekolah: num  11.2 8.9 2.2 4.6 8.9 0.4 7.5 13.8 4 9.3 ...
 $ umur_krt   : int   62 36 41 27 45 60 55 57 36 31 ...
 $ anggota_rt : int    5 3 2 3 5 2 3 4 6 5 ...
 $ pengeluaran: num   10.29 9.1 7.44 8.55 9.5 ...
```

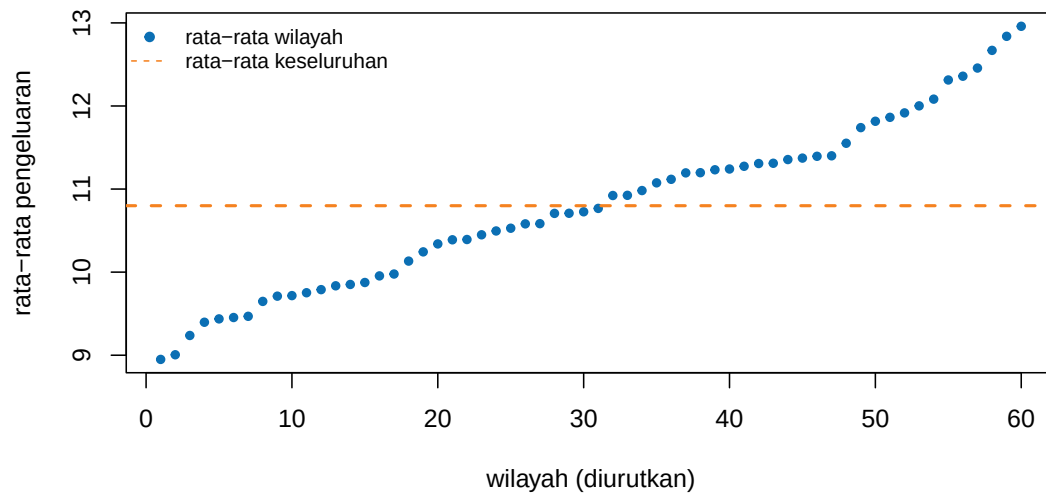
```
cat("rumah tangga:", nrow(rt), " wilayah:", length(unique(rt$wilayah)),
    " rumah tangga per wilayah:", nrow(rt) / length(unique(rt$wilayah)), "\n")
```

```
rumah tangga: 1200 wilayah: 60 rumah tangga per wilayah: 20
```

```
rata_wil <- tapply(rt$pengeluaran, rt$wilayah, mean)
round(summary(rata_wil), 3)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
8.95	9.94	10.75	10.80	11.38	12.96

```
par(family = "sans", mar = c(4, 4, 1, 1))
plot(sort(rata_wil), pch = 19, col = "#0B6FB4", cex = 0.7,
     xlab = "wilayah (diurutkan)", ylab = "rata-rata pengeluaran")
abline(h = mean(rt$pengeluaran), lty = 2, col = "#F58220", lwd = 2)
legend("topleft", c("rata-rata wilayah", "rata-rata keseluruhan"),
     col = c("#0B6FB4", "#F58220"), pch = c(19, NA), lty = c(NA, 2), bty = "n", cex = 0.8)
```



Gambar 13.1: Rata-rata pengeluaran tiap wilayah: perbedaan antar-wilayah jelas terlihat

Interpretasi Output

Data memuat 1.200 rumah tangga yang tersebar merata pada 60 wilayah — 20 rumah tangga per wilayah. Inilah bentuk **bertingkat** yang lazim pada pendataan resmi: unit pencacahan tersarang di dalam unit wilayah.

Gambar memperlihatkan persoalannya dengan gamblang. Rata-rata pengeluaran antar-wilayah tersebar jauh dari garis rata-rata keseluruhan — rentangnya lebih dari empat juta rupiah. Bila amatan benar-benar saling bebas, sebaran rata-rata wilayah hanya akan mencerminkan keragaman pencuplikan dan tampak jauh lebih rapat.

Perbedaan antar-wilayah sebesar ini **tidak boleh diabaikan**. Model yang mengandaikan kebebasan akan menganggap keragaman tersebut sebagai galat murni, sehingga selang kepercayaannya terlalu sempit dan ramalannya untuk wilayah tertentu bias.

Contoh 13.2 | Mengukur kuatnya struktur kelompok dengan ICC

Kode 13.3 • R model kosong dengan lme4

```
suppressPackageStartupMessages(library(lme4))
set.seed(2026)
i_latih <- sample(nrow(rt), 900)
latih <- rt[i_latih, ]; uji <- rt[-i_latih, ]
cat("latih:", nrow(latih), " uji:", nrow(uji), "\n")
```

```
latih: 900 uji: 300
```

```
m0 <- lmer(pengeluaran ~ 1 + (1 | wilayah), data = latih)
vc <- as.data.frame(VarCorr(m0))
ragam <- setNames(vc$vcov, c("antar_wilayah", "galat"))
round(ragam, 4)
```

```
antar_wilayah    galat
           0.9173    1.3201
```

```
cat("ICC =", round(ragam["antar_wilayah"] / sum(ragam), 3), "\n")
```

```
ICC = 0.41
```

Interpretasi Output

Model kosong — tanpa peubah penjelas apa pun — memisahkan keragaman menjadi dua bagian: **antar-wilayah** 0,917 dan **dalam wilayah (galat)** 1,320.

ICC-nya **0,410**: sekitar 41% keragaman pengeluaran berasal dari perbedaan antar-wilayah, bukan dari perbedaan antar-rumah tangga di dalam wilayah yang sama. Angka setinggi ini menegaskan bahwa struktur kelompok wajib dimodelkan.

Sebagai kaidah praktis, ICC di atas sekitar 0,05 saja sudah cukup untuk membuat pengabaian struktur kelompok berbahaya bagi inferensi. Nilai 0,41 jauh melampauinya.

Perhatikan bahwa ragam galat 1,320 di sini **masih memuat** pengaruh peubah penjelas yang belum dimasukkan — pada model kosong, seluruh pengaruh lama sekolah dan umur kepala rumah tangga ikut terhitung sebagai galat. Contoh berikutnya memasukkannya.

Contoh 13.3 | Dua model pembanding: regresi linear dan model campuran linear

Kode 13.4 • R lm dan lmer beserta penilaiannya pada data uji

```
ols <- lm(pengeluaran ~ lama_sekolah + umur_krt + anggota_rt, data = latih)
lmm <- lmer(pengeluaran ~ lama_sekolah + umur_krt + anggota_rt + (1 | wilayah),
            data = latih)

rmse <- function(y, yhat) sqrt(mean((y - yhat)^2))
p_ols <- predict(ols, newdata = uji)
p_lmm <- predict(lmm, newdata = uji, allow.new.levels = TRUE)
round(c(RMSE_OLS = rmse(uji$pengeluaran, p_ols),
        RMSE_LMM = rmse(uji$pengeluaran, p_lmm)), 4)
```

```
RMSE_OLS RMSE_LMM
      1.2936    0.8728
```

```
vc1 <- as.data.frame(VarCorr(lmm))
round(setNames(vc1$vcov, c("antar_wilayah", "galat")), 4)
```

```
antar_wilayah    galat
           0.8273    0.7473
```

```
benar <- read.csv("data/efek_wilayah_sebenarnya.csv")
eb <- ranef(lmm)$wilayah[, 1]; nama <- rownames(ranef(lmm)$wilayah)
cat("korelasi efek acak LMM vs yang ditanam:",
    round(cor(eb, benar$efek_sebenarnya[match(nama, benar$wilayah)]), 4), "\n")
```

```
korelasi efek acak LMM vs yang ditanam: 0.9767
```

Interpretasi Output

Kedua model memakai peubah penjelas yang sama; yang membedakan hanyalah ada atau tidaknya efek acak wilayah.

Regresi linear memberi RMSE **1,294**. Ia mengabaikan wilayah sepenuhnya, sehingga seluruh perbedaan antar-wilayah masuk ke galat.

Model campuran linear memberi RMSE **0,873** — perbaikan sekitar 32% hanya dengan menambahkan satu suku ($1 \mid \text{wilayah}$). Setelah peubah penjelas masuk, ragam galat turun tajam dari 1,320 menjadi jauh lebih kecil, sedangkan ragam antar-wilayah bertahan — sebagaimana seharusnya, sebab peubah penjelas tingkat rumah tangga tidak menjelaskan perbedaan antar-wilayah.

Yang paling meyakinkan: korelasi antara efek acak taksiran dan efek yang **sengaja ditanam** mencapai **0,977**. Model campuran berhasil memulihkan hampir seluruh struktur wilayah.

Lalu apa yang masih kurang? Model ini memaksa pengaruh lama sekolah **linear**, padahal data dibangkitkan dengan bentuk sinus yang naik lalu melandai, ditambah **ambang tajam** pada umur 45 tahun. Kekurangan itulah yang hendak dijawab GPBoost.

13.7 Contoh Praktikum: GPBoost dengan Python

Contoh 13.4 | Menyiapkan data dan GPModel

Kode 13.5 • R membaca berkas yang sama dari Python

```
import numpy as np, pandas as pd, gpboost as gpb
print("gpboost versi:", gpb.__version__)
```

```
gpboost versi: 1.7.4
```

```
rt = pd.read_csv("data/umah_tangga_wilayah.csv")
X = rt[["lama_sekolah", "umur_krt", "anggota_rt"]].values
```

```

y = rt["pengeluaran"].values
g = rt["wilayah"].astype(str).to_numpy()      # penanda kelompok

rng = np.random.default_rng(2026)
idx = rng.permutation(len(rt)); tr, te = idx[:900], idx[900:]
print("latih:", len(tr), " uji:", len(te), " wilayah:", len(np.unique(g)))

latih: 900  uji: 300  wilayah: 60

```

Penjelasan Kode. Python membaca **berkas yang sama** dengan R. Perbedaan pokoknya adalah adanya `group_data` — penanda kelompok — yang diserahkan kepada `GPMoDel` secara terpisah dari matriks peubah penjelas. Penanda kelompok **bukan** peubah penjelas dan tidak boleh dimasukkan ke dalam `X`.

Contoh 13.5 | Menentukan banyaknya iterasi dengan validasi silang

Kode 13.6 • R gpb.cv dengan penghentian dini

```

par = {"objective": "regression_l2", "learning_rate": 0.05,
       "max_depth": 3, "verbose": 0}

gm_cv = gpb.GPMoDel(group_data=g[tr], likelihood="gaussian")
cv = gpb.cv(params=par, train_set=gpb.Dataset(X[tr], label=y[tr]), gp_model=gm_cv,
            num_boost_round=400, early_stopping_rounds=20, nfold=5,
            seed=2026, verbose_eval=False)
kunci = [k for k in cv if "mean" in k][0]
k_opt = int(np.argmin(cv[kunci])) + 1
print("banyaknya iterasi optimum:", k_opt)

banyaknya iterasi optimum: 44

```

Interpretasi Output

Validasi silang lima lipat memilih **44 iterasi**. Angka ini jauh lebih kecil daripada 400 yang disediakan — dan itu bukan kebetulan.

Pada GPBoost, banyaknya iterasi berperan ganda. Selain mengendalikan kelenturan pengaruh tetap sebagaimana pada boosting biasa, ia juga menentukan **pembagian keragaman** antara pengaruh tetap dan efek acak. Iterasi yang terlalu banyak membuat pohon-pohon menyerap keragaman yang sesungguhnya milik wilayah, sehingga ragam antar-wilayah tertaksir terlalu kecil.

Contoh berikutnya memperlihatkan akibatnya secara angka, jadi menentukan banyaknya iterasi lewat validasi silang di sini **bukan sekadar praktik yang rapi** — ia menentukan benar atau tidaknya taksiran komponen ragam.

Contoh 13.6 | Melatih GPBoost dan menaksir komponen ragam

Kode 13.7 • R gpb.train dan ringkasan GPModel

```
gm = gpb.GPModel(group_data=g[tr], likelihood="gaussian")
bst = gpb.train(params=par, train_set=gpb.Dataset(X[tr], label=y[tr]),
               gp_model=gm, num_boost_round=k_opt)
gm.summary()
```

```
=====
Model summary:
Nb. observations: 900
Nb. groups: 60 (Group_1)
Covariance parameters (random effects):
      Param. Std. err.
Error_var 0.2024    0.0099
Group_1    0.8897    0.1649
=====
<gpboost.basic.GPModel object at 0x7f503c55ecf0>
```

```
# pembandingan: iterasi terlalu banyak (tanpa validasi silang)
gm_lebih = gpb.GPModel(group_data=g[tr], likelihood="gaussian")
bst_lebih = gpb.train(params=par, train_set=gpb.Dataset(X[tr], label=y[tr]),
                    gp_model=gm_lebih, num_boost_round=400)
print("--- dengan 400 iterasi ---")
```

```
--- dengan 400 iterasi ---
```

```
gm_lebih.summary()
```

```
=====
Model summary:
Nb. observations: 900
Nb. groups: 60 (Group_1)
Covariance parameters (random effects):
      Param. Std. err.
Error_var 0.1045    0.0051
Group_1    0.8992    0.1655
=====
<gpboost.basic.GPModel object at 0x7f503c4bd3a0>
```

Interpretasi Output

Dengan 44 iterasi, komponen ragam tertaksir: **ragam galat 0,202** dan **ragam antar-wilayah 0,890**. Bandingkan dengan nilai yang sebenarnya dipakai membangkitkan data: ragam galat $0,5^2 = 0,25$, dan ragam ke-60 efek wilayah yang tertarik $0,951^2 = 0,904$. Keduanya dipulihkan dengan baik — selisihnya kecil dan searah dengan galat baku yang dilaporkan.

Sekarang perhatikan pembandingnya. Dengan 400 iterasi, **ragam galat anjlok** jauh di bawah nilai sebenarnya: pohon-pohon telah menghafal data latih sehingga sisaannya hampir nol. Model menjadi sangat percaya diri padahal keragaman itu nyata dan akan muncul kembali pada data baru. Inilah alasan mengapa pada GPBoost penghentian yang tepat lebih penting daripada pada boosting biasa. Pada boosting biasa, terlalu banyak iterasi merusak daya ramal; pada GPBoost, ia **juga** merusak taksiran komponen ragam — yakni merusak hal yang sering justru menjadi tujuan analisis pada statistik resmi.

Contoh 13.7 | Memulihkan efek acak wilayah

Kode 13.8 • R prediksi laten dan pencocokan dengan nilai yang ditanam

```
pl = bst.predict(data=X[te], group_data_pred=g[te], pred_latent=True)
efek = (pd.DataFrame({"wilayah": g[te], "taksiran": pl["random_effect_mean"]})
        .groupby("wilayah").mean())

benar = pd.read_csv("data/efek_wilayah_sebenarnya.csv").set_index("wilayah")
gab = efek.join(benar)
print(gab.head(5).round(4))
```

	taksiran	efek_sebenarnya
wilayah		
W01	-1.6042	-1.8769
W02	-0.4397	-0.6888
W03	-0.2418	-0.1133
W04	-0.8511	-0.8817
W05	1.7342	1.6275

```
print("\nkorelasi efek acak GPBoost vs yang ditanam:",
      round(float(gab["taksiran"].corr(gab["efek_sebenarnya"])), 4))
```

```
korelasi efek acak GPBoost vs yang ditanam: 0.9849
```

```
print("banyaknya wilayah yang dicocokkan:", len(gab))
```

```
banyaknya wilayah yang dicocokkan: 59
```

Interpretasi Output

Korelasi antara efek acak taksiran dan efek yang ditanam mencapai **0,985** — sedikit lebih tinggi daripada model campuran linear (0,977).

Sebabnya masuk akal. Model campuran linear harus memaksakan bentuk linear pada pengaruh lama sekolah dan umur; ketidakcocokan bentuk itu ikut terserap ke dalam efek acak, sehingga taksirannya

sedikit tercemar. GPBoost memodelkan bentuk nonlinearnya dengan pohon, sehingga efek acak yang tersisa lebih murni mencerminkan perbedaan antar-wilayah.

Bagi pekerjaan statistik resmi, efek acak ini sering **lebih penting daripada ramalannya sendiri**: ia adalah taksiran “seberapa tinggi atau rendah wilayah ini dibandingkan wilayah lain setelah ciri rumah tangganya diperhitungkan” — persis gagasan yang mendasari pendugaan area kecil.

Contoh 13.8 | Membandingkan empat pendekatan pada data uji yang sama

Kode 13.9 • R RMSE GPBoost dan boosting biasa

```
pr = bst.predict(data=X[te], group_data_pred=g[te], pred_latent=False)
rmse_gpb = float(np.sqrt(np.mean((y[te] - pr["response_mean"])**2)))

bst_biasa = gpb.train(params=par, train_set=gpb.Dataset(X[tr], label=y[tr]),
                      num_boost_round=k_opt)          # TANPA gp_model
rmse_boost = float(np.sqrt(np.mean((y[te] - bst_biasa.predict(X[te]))**2)))

print("RMSE GPBoost      :", round(rmse_gpb, 4))

RMSE GPBoost      : 0.5482

print("RMSE boosting biasa:", round(rmse_boost, 4))

RMSE boosting biasa: 1.0968
```

Interpretasi Output

Empat pendekatan, satu data uji, satu ukuran:

Pendekatan	Nonlinear	Kelompok	RMSE
Regresi linear (OLS)	tidak	tidak	1,294
Boosting biasa	ya	tidak	1,097
Model campuran linear	tidak	ya	0,873
GPBoost	ya	ya	0,548

Urutannya menceritakan seluruh gagasan modul ini. Menambahkan **kemampuan nonlinear** saja (OLS → boosting) memperbaiki RMSE sekitar 15%. Menambahkan **struktur kelompok** saja (OLS → model campuran) memperbaikinya 33% — lebih besar, sebab ICC data ini tinggi. Menambahkan **keduanya** memberi 58%.

Yang perlu diperhatikan: perbaikan GPBoost atas model campuran (0,873 → 0,548) **lebih besar** daripada perbaikan boosting atas OLS (1,294 → 1,097). Kedua langkah itu sama-sama “menambahkan kemampuan nonlinear”, tetapi hasilnya berbeda jauh — karena pada GPBoost, pohon tidak lagi harus bersaing dengan keragaman antar-wilayah yang tak termodelkan. Kedua komponen itu

saling menguatkan, bukan sekadar berjumlah.

Perlu pula ditegaskan bahwa urutan ini bergantung pada data. Bila ICC-nya rendah, model campuran nyaris tidak akan mengungguli OLS dan boosting biasa dapat memadai. Karena itu **hitung ICC lebih dahulu** — sebagaimana Contoh 13.2 — sebelum memilih pendekatan.

Contoh 13.9 | Meramalkan bagi wilayah baru

Kode 13.10 • R kelompok yang belum pernah terlihat

```
pred_lama = bst.predict(data=X[te][:5], group_data_pred=g[te][:5],
                        pred_latent=False) ["response_mean"]
pred_baru = bst.predict(data=X[te][:5],
                        group_data_pred=np.array(["W99"] * 5),
                        pred_latent=False) ["response_mean"]
print(pd.DataFrame({"wilayah_asli": g[te][:5],
                    "ramalan_wilayah_dikenal": np.round(pred_lama, 3),
                    "ramalan_wilayah_baru": np.round(pred_baru, 3)}))
```

	wilayah_asli	ramalan_wilayah_dikenal	ramalan_wilayah_baru
0	W44	12.424	12.083
1	W09	8.560	9.497
2	W16	8.883	9.468
3	W41	10.828	10.947
4	W55	11.603	12.121

Interpretasi Output

Kelima rumah tangga ini memiliki ciri yang sama persis pada kedua kolom ramalan — yang berbeda hanyalah wilayahnya.

Untuk **wilayah yang dikenal**, ramalannya menjumlahkan pengaruh tetap dan efek acak wilayah tersebut. Untuk **wilayah baru** (w99, yang tidak ada pada data latih), efek acaknya diramalkan **nol** sehingga yang tersisa hanya pengaruh tetap — dan seluruh ramalannya bergeser ke nilai rata-rata.

Perilaku ini benar dan penting dipahami: model tidak memiliki dasar apa pun untuk menduga wilayah baru itu di atas atau di bawah rata-rata. Konsekuensi praktisnya, **ketepatan ramalan bagi wilayah baru akan jauh lebih rendah** daripada bagi wilayah yang datanya sudah ada. Bila laporan menyajikan satu angka ketepatan tanpa membedakan keduanya, angka itu menyesatkan bagi pengguna yang hendak menerapkan model ke wilayah yang belum tercakup.

13.8 Padanan Kode dalam R

Bagian ini menjalankan **analisis yang sama dengan R**, sehingga hasil kedua bahasa dapat dibandingkan langsung. Berikut analisis yang sama dijalankan ulang dengan R.

Kode 13.11 • R GPBoost dengan R

```
library(gpboost)

rt <- read.csv("data/rumah tangga_wilayah.csv", stringsAsFactors = FALSE)
X <- as.matrix(rt[, c("lama_sekolah", "umur_krt", "anggota_rt")])
y <- rt$pengeluaran
g <- rt$wilayah

set.seed(2026)
i_latih <- sample(nrow(rt), 900)
X_tr <- X[i_latih, ]; y_tr <- y[i_latih]; g_tr <- g[i_latih]
X_te <- X[-i_latih, ]; y_te <- y[-i_latih]; g_te <- g[-i_latih]

par <- list(objective = "regression_l2", learning_rate = 0.05,
            max_depth = 3, verbose = -1)

# (1) banyaknya iterasi lewat validasi silang
gm_cv <- GPModel(group_data = g_tr, likelihood = "gaussian")
cv <- gpb.cv(params = par, data = X_tr, label = y_tr, gp_model = gm_cv,
            nrounds = 400, early_stopping_rounds = 20, nfold = 5, verbose = 0)
k_opt <- cv$best_iter
cat("banyaknya iterasi optimum (R):", k_opt, "\n")

banyaknya iterasi optimum (R): 44

# (2) pelatihan dan komponen ragam
gm <- GPModel(group_data = g_tr, likelihood = "gaussian")
bst <- gpboost(data = X_tr, label = y_tr, gp_model = gm,
            nrounds = k_opt, params = par, verbose = 0)

summary(gm)
```

```
=====
Model summary:
Nb. observations: 900
Nb. groups: 60 (Group_1)
Covariance parameters (random effects):
      Param. Std. err.
Error_var 0.2121    0.0104
Group_1    0.8702    0.1615
=====
```

```
# (3) efek acak dan ramalan
pl <- predict(bst, data = X_te, group_data_pred = g_te, predict_var = FALSE)
cat("RMSE GPBoost (R):", round(sqrt(mean((y_te - pl$response_mean)^2)), 4), "\n")
```

```
RMSE GPBoost (R): 0.5287
```

Interpretasi Output

Bandingkan keluaran R di atas dengan hasil Python pada Contoh 13.4–13.6.

Banyaknya iterasi optimum sama persis: 44. Ini bukan kebetulan — kedua bahasa membungkus pustaka C++ yang sama, sehingga algoritmanya identik.

Komponen ragam praktis sama: R memberi ragam galat 0,212 dan ragam wilayah 0,870; Python memberi 0,202 dan 0,890. Keduanya sama-sama dekat dengan nilai sebenarnya (0,25 dan 0,904).

RMSE sedikit berbeda: 0,529 pada R dan 0,548 pada Python. Penyebabnya bukan algoritma melainkan **pembagian latihan–uji yang berbeda** — `sample()` pada R dan `numpy` pada Python memakai pembangkit bilangan acak yang berlainan, sehingga 900 rumah tangga yang terpilih tidak sama meskipun benihnya sama-sama 2026.

Pelajaran praktisnya: **benih acak tidak dapat disamakan antar-bahasa.** Bila hasil R dan Python harus benar-benar identik, pembagian latihan–uji harus dibuat sekali lalu disimpan ke berkas dan dibaca kedua bahasa — bukan dibangkitkan ulang di masing-masing.

Perbedaan penamaan: R memakai `nrounds` (Python: `num_boost_round`), `gpboost()` (Python: `gpb.train()`), dan menyerahkan data lewat `data=` beserta `label=` (Python: objek `gpb.Dataset`). Argumen `verbose` pun berbeda: R memakai `—1` untuk membisukan keluaran, Python memakai `0`.

Kesalahan Umum.

1. Memasukkan penanda kelompok sebagai **peubah penjelas** ke dalam `X`; ia harus diserahkan lewat `group_data` agar diperlakukan sebagai efek acak, bukan sebagai pemecah pohon.
2. Menjalankan boosting lebih dahulu lalu menaksir efek acak dari sisaannya; komponen ragam akan tertaksir terlalu kecil.
3. Menetapkan banyaknya iterasi tanpa validasi silang; pada GPBoost hal itu merusak taksiran komponen ragam, bukan hanya daya ramal.
4. Membagi data latihan–uji secara acak padahal tujuannya meramalkan **wilayah baru**; pembagiannya harus dilakukan **per wilayah**.
5. Melaporkan satu angka ketepatan tanpa membedakan wilayah yang dikenal dan wilayah baru.
6. Menafsirkan efek acak sebagai “mutu” wilayah; ia hanyalah simpangan bersyarat setelah peubah penjelas diperhitungkan.
7. Mengabaikan ICC lalu langsung memakai GPBoost; bila ICC hampir nol, boosting biasa lebih sederhana dan setara.

8. Membandingkan RMSE antar-model yang dihitung pada pembagian data yang berbeda.
9. Menyimpulkan hubungan sebab-akibat dari pentingnya peubah pada model boosting.

13.9 Latihan Mandiri

Latihan 13.1 | Menelaah struktur dan pembanding

- Hitung ICC memakai model kosong pada **seluruh** data (bukan hanya data latih). Bandingkan dengan nilai pada Contoh 13.2 dan jelaskan mengapa keduanya berbeda sedikit.
- Gambar diagram kotak-garis pengeluaran untuk 12 wilayah pertama. Apa yang terlihat mengenai perbedaan antar-wilayah?
- Cocokkan model campuran linear dengan menambahkan **kemiringan acak** untuk `lama_sekolah`, yakni $(1 + \text{lama_sekolah} \mid \text{wilayah})$. Apakah RMSE-nya membaik? Apa artinya?

Latihan 13.2 | Menjalankan GPBoost

- Ulangi Contoh 13.4 sampai 13.6 dengan `learning_rate` 0,01 dan 0,20. Bagaimana perubahan banyaknya iterasi optimum dan taksiran komponen ragamnya?
- Bandingkan taksiran ragam antar-wilayah pada ketiga nilai `learning_rate` tersebut dengan nilai sebenarnya (0,904). Nilai mana yang paling mendekati?
- Hitung RMSE GPBoost **terpisah** untuk rumah tangga di wilayah ber-efek acak positif dan negatif. Apakah ketepatannya seimbang?

Latihan 13.3 | Membetulkan kode yang salah

Kode berikut memuat **empat kesalahan**; temukan, perbaiki, jalankan versi yang benar, dan jelaskan akibat tiap kesalahan bagi kesimpulan.

Kode 13.12 • R kode yang harus dibetulkan

```
X = rt[["lama_sekolah", "umur_krt", "anggota_rt", "wilayah"]].values # (1)
gm = gpb.GPModel(group_data=g[tr], likelihood="gaussian")
bst = gpb.train(params=par, train_set=gpb.Dataset(X[tr], label=y[tr]),
               gp_model=gm, num_boost_round=1000) # (2)
pred = bst.predict(data=X[tr], group_data_pred=g[tr]) # (3)
# "RMSE 0,08 -- model sangat akurat" # (4)
```

Petunjuk: satu baris memasukkan penanda kelompok sebagai peubah penjelas; satu baris menetapkan iterasi tanpa validasi silang; satu baris menilai model pada **data latih**; dan satu kesimpulan menyalahartikan angka yang dihasilkan cara tersebut.

Latihan 13.4 | Meramalkan wilayah baru

- (a) Bagi data **per wilayah**: 48 wilayah untuk latih, 12 wilayah untuk uji — sehingga seluruh wilayah uji belum pernah terlihat.
- (b) Latih ulang GPBoost pada pembagian itu, lalu hitung RMSE-nya.
- (c) Bandingkan dengan RMSE pada Contoh 13.7 yang pembagiannya acak. Berapa besar penurunan ketepatannya, dan mengapa?
- (d) Tulis satu paragraf (maksimal 150 kata) tentang bagaimana hasil ini harus disampaikan kepada pengguna yang hendak menerapkan model pada kabupaten yang belum tercakup pendataan.

13.10 Penugasan

Konteks: Pendugaan Pengeluaran Rumah Tangga per Wilayah

Sebuah unit kerja hendak menduga rata-rata pengeluaran rumah tangga tiap wilayah untuk keperluan penargetan program. Data pencacahan tersedia bagi 60 wilayah, tetapi ukuran sampel per wilayah kecil. Anda diminta menilai apakah pendekatan berbasis model dapat memperbaiki dugaan langsung, dan pendekatan mana yang sebaiknya dipakai.

Tugas 13.1 | Penelaahan struktur

1. Laporkan struktur data, ICC, dan sebaran rata-rata wilayah. Simpulkan apakah struktur kelompok perlu dimodelkan.

Tugas 13.2 | Pemodelan dan perbandingan

2. Cocokkan keempat pendekatan pada pembagian latih–uji yang **sama**; laporkan RMSE keempatnya dalam satu tabel.
3. Untuk GPBoost, tentukan banyaknya iterasi lewat validasi silang dan laporkan komponen ragam beserta galat bakunya.
4. Bandingkan taksiran komponen ragam dari model campuran linear dan dari GPBoost. Mana yang lebih mendekati nilai yang ditanam, dan mengapa?

Tugas 13.3 | Efek acak dan penargetan

5. Susun peringkat 10 wilayah dengan efek acak terendah menurut GPBoost, lalu bandingkan dengan peringkat menurut **rata-rata mentah** pengeluaran wilayah. Apakah keduanya sama? Jelaskan mengapa dapat berbeda.
6. Jelaskan mengapa peringkat berdasarkan efek acak lebih tepat untuk penargetan daripada peringkat berdasarkan rata-rata mentah.

Tugas 13.4 | Kesimpulan dan keterbatasan

7. Susun rekomendasi (maksimal 250 kata): pendekatan mana yang Anda usulkan, dan dengan syarat apa.
8. Tulis bagian keterbatasan yang memuat sekurang-kurangnya: perbedaan ketepatan bagi wilayah dikenal dan wilayah baru; ketiadaan klaim sebab-akibat; dan bahwa data bersifat simulasi.

13.10.1 Rubrik Penilaian

Komponen	Indikator	Bobot
Penelaahan struktur	ICC dihitung dan dipakai sebagai dasar keputusan	15%
Ketepatan pemodelan	Keempat model dicocokkan benar pada pembagian yang sama	30%
Komponen ragam	Iterasi ditentukan lewat CV; komponen ragam ditafsirkan	20%
Efek acak dan penargetan	Perbedaan peringkat dijelaskan dengan benar	20%
Keterbatasan	Wilayah baru, kausalitas, dan sifat data dinyatakan	15%

13.11 Ringkasan Modul

Daftar periksa penguasaan Modul 13

- ☐ Saya dapat mengenali struktur bertingkat dan menjelaskan akibatnya.
- ☐ Saya dapat menghitung dan menafsirkan ICC.
- ☐ Saya dapat menjelaskan keterbatasan boosting biasa dan model campuran linear.
- ☐ Saya dapat menjalankan GPBoost dan menentukan iterasi lewat validasi silang.
- ☐ Saya dapat menafsirkan komponen ragam dan memeriksa pemulihan efek acak.
- ☐ Saya dapat menjelaskan perbedaan ramalan bagi wilayah dikenal dan wilayah baru.
- ☐ Saya memahami padanan kode GPBoost antara R dan Python.

Ringkasan fungsi:

Python	R	Kegunaan
<code>gpb.GPModel(group_data=)</code>	<code>GPModel(group_data=)</code>	menyebutkan definisi struktur efek acak
<code>gpb.Dataset(X, label=)</code>	<code>data=, label=</code>	menyiapkan data latih
<code>gpb.cv(...)</code>	<code>gpb.cv(...)</code>	menentukan banyaknya iterasi
<code>gpb.train(...)</code>	<code>gpboost(...)</code>	melatih model
<code>gm.summary()</code>	<code>summary(gm)</code>	komponen ragam
<code>bst.predict(..., pred_latent=)</code>	<code>predict(bst, ...)</code>	ramalan dan efek acak

Python	R	Kegunaan
<code>lmer()</code> (R saja)	<code>lme4::lmer()</code>	model campuran linear pembanding

13.11.1 Daftar Istilah

Data bertingkat (*hierarchical/multilevel data*) — data yang unitnya tersarang di dalam unit tingkat lebih tinggi.

Efek acak (*random effect*) — simpangan khas tiap kelompok yang dimodelkan sebagai peubah acak berpusat nol.

ICC (*intraclass correlation*) — proporsi keragaman yang berasal dari perbedaan antar-kelompok.

Komponen ragam (*variance components*) — ragam antar-kelompok dan ragam galat.

Boosting — ansambel pohon yang dibangun berurutan, tiap pohon memperbaiki sisaan pendahulunya.

GPBoost — metode yang menaksir pengaruh tetap dengan boosting dan struktur kelompok dengan efek acak atau proses Gaussian secara berselang-seling.

Pendugaan area kecil (*small area estimation*) — pendugaan bagi wilayah dengan sampel kecil dengan meminjam kekuatan dari wilayah lain lewat model.

13.12 Referensi

1. Sigrist, F. (2022). Gaussian process boosting. *Journal of Machine Learning Research*, 23(232), 1–46.
2. Sigrist, F. (2023). Latent Gaussian model boosting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(2), 1894–1905.
3. Gelman, A. & Hill, J. (2007). *Data Analysis Using Regression and Multilevel/Hierarchical Models*. Cambridge University Press.
4. Bates, D., Mächler, M., Bolker, B., & Walker, S. (2015). Fitting linear mixed-effects models using lme4. *Journal of Statistical Software*, 67(1), 1–48.
5. Rao, J. N. K. & Molina, I. (2015). *Small Area Estimation*, 2nd ed. Wiley.
6. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*, 2nd ed. Springer. (Bab 10)

MODUL

14

ALUR KERJA PENGELOMPOKAN, CLARA/CLARANS, DAN KAJIAN TERKENDALI

Pertemuan Minggu ke-14

Sub-CPMK 14: Mahasiswa mampu menjalankan alur kerja pengelompokan secara utuh — dari memeriksa kecenderungan berklaster, prapemrosesan, pemilihan ukuran jarak, hingga profiling — memilih varian k-Medoid yang sesuai ukuran data, serta membandingkan metode berbasis jarak dan berbasis kepadatan dengan prosedur yang sah.

Penyusun: Achmad Syahrul Choir



Daftar Isi

14.1 Deskripsi Modul	445
14.2 Tujuan Praktikum	445
14.3 Alat dan Bahan	446
14.4 Instruksi Pengerjaan	446
14.5 Teori Singkat	447
14.5.1 Kecenderungan Berklaster	447
14.5.2 Biaya PAM dan Dua Varian untuk Data Besar	447
14.5.3 Membandingkan Antar-Keluarga Metode	448
14.6 Contoh Praktikum dengan R	449
14.7 Varian k-Medoid untuk Data Besar: CLARA dan CLARANS	454
14.8 Kajian Terkendali: Membandingkan Berbasis Jarak dan Berbasis Kepadatan	456
14.9 Alur Kerja yang Sama pada Data Berstruktur	463
14.10 Latihan Mandiri	468
14.11 Penugasan	469
14.11.1 Rubrik Penilaian	469
14.12 Ringkasan Modul	470
14.12.1 Daftar Istilah	470
14.13 Referensi	471

14.1 Deskripsi Modul

Modul 12 memperkenalkan metode pengelompokan satu per satu. Modul ini merangkainya menjadi **alur kerja yang utuh**, lalu menambahkan dua hal yang diperlukan ketika pengelompokan dipakai pada pekerjaan sungguhan: **varian k-Medoid untuk data besar** dan **prosedur membandingkan metode secara sah**.

Tiga bagian isinya:

Pertama, alur kerja enam tahap — memeriksa kecenderungan berklaster, prapemrosesan, memilih ukuran jarak, memilih metode, validasi, dan profiling. Tahap pertama berperan sebagai penjaga: bila data tidak memiliki kecenderungan berklaster, lima tahap berikutnya hanya menghasilkan pembagian yang rapi tetapi hampa.

Kedua, CLARA dan CLARANS — dua varian k-Medoid yang membuat pengelompokan tetap mungkin ketika PAM tidak lagi praktis karena matriks jaraknya terlalu besar.

Ketiga, kajian terkendali — pembuktian dengan data berpola diketahui bahwa siluet dan indeks Dunn **tidak sah** dipakai menghakimi antar-keluarga metode, beserta prosedur perbandingan yang benar.

Kedudukan modul ini. Modul ini bersifat **pelengkap** bagi Modul 12 dan memakai data yang sama (`data_wilayah.csv`) ditambah data buatan yang dibangkitkan di dalam modul. Ia dapat dikerjakan sebagai sesi tersendiri, sebagai bahan pengayaan, atau sebagai sumber soal ujian praktik. Data bersifat **ilustratif hasil simulasi, bukan angka resmi** instansi mana pun.

14.2 Tujuan Praktikum

Setelah menyelesaikan praktikum Modul 14, mahasiswa diharapkan mampu:

1. **Memeriksa kecenderungan berklaster** dengan statistik Hopkins beserta pembanding data acak, dan memutuskan apakah pengelompokan layak dilanjutkan.
2. **Memutuskan prapemrosesan** yang tepat: pembakuan bagi jarak Euclidean, dan PCA hanya bila peubah berkorelasi kuat.
3. **Memilih ukuran jarak** sesuai jenis peubah: Euclidean, cocok/tak cocok, Gower, atau gabungan pada k-Prototype.
4. **Memilih metode** menurut dugaan bentuk klaster dan keberadaan derau.
5. **Memeriksa klaster (profiling)** dalam satuan asli beserta ukuran kelompok dan komposisi peubah kategoriknya.
6. Menjelaskan perbedaan **CLARA** dan **CLARANS**, serta memilih di antara keduanya menurut ukuran data.

- Menjelaskan mengapa **siluet dan indeks Dunn tidak sah** dipakai menghakimi antar-keluarga metode, dan menyebutkan ukuran yang sah.
- Menjalankan **uji kendali** pada data acak sebelum mempercayai kluster yang ditemukan.

14.3 Alat dan Bahan

- R (RStudio) dengan paket `cluster`, `clustMixType`, `dbscan`, `fpc`, dan `factoextra`.
- Pasang dengan `install.packages(c("cluster", "clustMixType", "dbscan", "fpc", "factoextra"))`.
- Berkas data: `data/data_wilayah.csv` dan `data/wilayah_berpola.csv`.

```
suppressPackageStartupMessages({library(cluster); library(clustMixType)
                                library(dbscan); library(fpc); library(factoextra)})
wil <- read.csv("data/data_wilayah.csv", stringsAsFactors = TRUE)
num <- c("kepadatan", "pengeluaran", "akses_air", "akses_listrik", "lama_sekolah")
X   <- scale(wil[num])           # pembakuan: wajib bagi jarak Euclidean
wil_c <- wil[, -1]              # termasuk peubah kategorik
wil_z <- wil_c; wil_z[, num] <- scale(wil_z[, num])
G    <- daisy(wil_c, metric = "gower")

set.seed(2026)
pam3  <- pam(X, 3)              # pembanding berbasis jarak
pam_g <- pam(G, 3, diss = TRUE)
kp_baku <- kproto(wil_z, k = 3, verbose = FALSE)

ari <- function(a, b) { tb <- table(a, b); n <- sum(tb)
  s_ij <- sum(choose(tb, 2)); s_i <- sum(choose(rowSums(tb), 2))
  s_j <- sum(choose(colSums(tb), 2)); h <- s_i * s_j / choose(n, 2)
  (s_ij - h) / ((s_i + s_j) / 2 - h) }
cat("objek siap:", nrow(wil), "wilayah,", length(num), "peubah numerik +2 kategorik\n")
```

```
objek siap: 200 wilayah, 5 peubah numerik +2 kategorik
```

Bab ini bersifat **pelengkap** dan tidak wajib dikerjakan selama praktikum. Isinya dua hal: alur kerja pengelompokan dari awal hingga profiling, dan varian k-Medoid untuk data besar (**CLARA** dan **CLARANS**).

14.4 Instruksi Pengerjaan

- Buat **RStudio Project**, salin folder `data/`, dan pastikan paket `cluster`, `clustMixType`, `dbscan`, `fpc`, serta `factoextra` terpasang.
- Kerjakan Contoh secara berurutan. Tahap 1 adalah **penjaga**: bila kecenderungan berkluster tidak dite-

mukan, catat kesimpulan itu sebelum melanjutkan — lanjutkan hanya untuk keperluan latihan.

3. Kerjakan **Latihan Mandiri**, lalu **Penugasan** sebagai laporan R Markdown.

Kedudukan modul. Modul ini bersifat **pelengkap** bagi Modul 12 dan memakai data yang sama. Ia dapat dikerjakan sebagai sesi tersendiri, bahan pengayaan, atau sumber soal ujian praktik.

14.5 Teori Singkat

14.5.1 Kecenderungan Berklatster

Statistik **Hopkins** membandingkan jarak titik amatan ke tetangga terdekatnya dengan jarak titik acak ke tetangga terdekatnya:

$$H = \frac{\sum_i u_i}{\sum_i u_i + \sum_i w_i}$$

dengan u_i jarak titik acak ke amatan terdekat dan w_i jarak amatan terpilih ke amatan terdekat lainnya. Nilai mendekati 0,5 berarti sebaran tidak dapat dibedakan dari acak seragam; mendekati 1 menandakan pengumpulan.

Statistik ini **selalu** perlu dibandingkan dengan nilainya pada data acak berukuran dan berentang sama — angka mutlaknya sendiri bergantung pada dimensi dan ukuran sampel.

14.5.2 Biaya PAM dan Dua Varian untuk Data Besar

PAM menyimpan seluruh $n(n-1)/2$ jarak berpasangan dan mencoba tiap penukaran medoid dengan bukan-medoid — berbiaya sekitar $O(k(n-k)^2)$ per iterasi.

CLARA menarik beberapa *sampel amatan* berukuran kecil, menjalankan PAM pada masing-masing, lalu memilih himpunan medoid yang memberi jarak rata-rata terkecil atas seluruh data.

CLARANS melihat seluruh amatan tetapi mengacak *penukaran* yang dicoba: dari satu himpunan medoid ia memeriksa sejumlah tetangga acak (`maxneighbor`), berpindah bila lebih baik, dan mengulang pencarian beberapa kali (`numlocal`).

Arah kelemahan keduanya berlawanan. CLARA dapat **melewatkan klaster kecil** yang tidak pernah tertarik ke dalam sampel. CLARANS tidak pernah melewati amatan, tetapi dapat **berhenti pada optimum lokal** karena tidak mencoba seluruh penukaran. Parameter yang dipakai wajib dilaporkan pada keduanya.

14.5.3 Membandingkan Antar-Keluarga Metode

Siluet dan indeks Dunn mengandaikan bentuk klaster tertentu dan dihitung atas himpunan amatan yang dikelompokkan. Metode berbasis kepadatan membuang sebagian amatan sebagai derau, sehingga ukurannya dihitung atas himpunan yang **berbeda** — perbandingannya karena itu tidak sah. Yang sah dipakai lintas keluarga: **kesepakatan** (indeks Rand terkoreksi pada amatan yang sama), **cakupan** (proporsi amatan yang dikelompokkan), dan **kestabilan** terhadap penarikan ulang sampel.

14.6 Contoh Praktikum dengan R

Enam tahap yang sebaiknya dilalui berurutan

1. **Memeriksa kecenderungan berklaster** — apakah data memang layak dikelompokkan?
2. **Prapemrosesan** — pembakuan; PCA bila peubah berkorelasi kuat.
3. **Memilih ukuran jarak** — sesuai jenis peubah.
4. **Memilih metode** — sesuai bentuk klaster yang diduga dan keberadaan derau.
5. **Validasi** — ukuran internal, kestabilan, dan cakupan.
6. **Profiling** — memerikan klaster dalam satuan asli agar dapat ditindaklanjuti.

Tahap 1 adalah penjaga: bila gagal, lima tahap berikutnya hanya menghasilkan pembagian yang rapi tetapi hampa.

Contoh 14.1 | Tahap 1 — Memeriksa kecenderungan berklaster

Kode 14.1 • R statistik Hopkins dan pembandingan data acak

```
suppressPackageStartupMessages(library(factoextra))
set.seed(2026)
h_data <- get_clust_tendency(X, n = 40, graph = FALSE)$hopkins_stat

# pembandingan: data acak seragam pada rentang yang sama
set.seed(2026)
U <- apply(X, 2, function(x) runif(length(x), min(x), max(x)))
h_acak <- get_clust_tendency(U, n = 40, graph = FALSE)$hopkins_stat

round(c(hopkins_data = h_data, hopkins_acak = h_acak), 3)
```

```
hopkins_data hopkins_acak
      0.501      0.511
```

Interpretasi Output

Statistik Hopkins membandingkan jarak titik amatan ke tetangga terdekatnya dengan jarak titik acak ke tetangga terdekatnya. Nilai mendekati **0,5** berarti sebaran data tidak dapat dibedakan dari acak seragam; nilai mendekati 1 menandakan pengumpulan.

Hasilnya menohok: data wilayah memberi **0,501**, sedangkan data acak buatan memberi 0,511 — praktis sama. **Data ini tidak memiliki kecenderungan berklaster sama sekali.**

Temuan ini menjelaskan seluruh angka rendah yang muncul sepanjang modul: siluet 0,09–0,19, ARI antar-metode 0,08, dan ketidaksepakatan antar-metode yang besar. Semuanya bukan kegagalan me-

tode, melainkan cerminan setia dari data yang memang tidak berkelompok.

Keputusan yang benar pada tahap ini: berhenti. Melanjutkan ke lima tahap berikutnya boleh saja dilakukan — sebagaimana modul ini melakukannya untuk keperluan pembelajaran — tetapi hasilnya **tidak boleh** dilaporkan sebagai penemuan kelompok wilayah. Yang dilaporkan adalah: "pemeriksaan kecenderungan berklaster tidak menemukan struktur; pengelompokan tidak dilanjutkan".

Pada pekerjaan sungguhan, melewati tahap ini adalah sumber kekeliruan yang paling mahal — sebab seluruh tahap berikutnya akan berjalan mulus dan menghasilkan tabel profil yang tampak meyakinkan.

Contoh 14.2 | Tahap 2 — Prapemrosesan: pembakuan dan PCA bila berkorelasi

Kode 14.2 • R memeriksa korelasi sebelum memutuskan PCA

```
R <- cor(wil[, num])
round(R, 2)
```

	kepadatan	pengeluaran	akses_air	akses_listrik	lama_sekolah
kepadatan	1.00	0.00	0.02	0.13	0.03
pengeluaran	0.00	1.00	-0.06	0.06	0.13
akses_air	0.02	-0.06	1.00	-0.04	0.04
akses_listrik	0.13	0.06	-0.04	1.00	-0.02
lama_sekolah	0.03	0.13	0.04	-0.02	1.00

```
cat("korelasi mutlak terbesar di luar diagonal:",
    round(max(abs(R[upper.tri(R)])), 3), "\n")
```

```
korelasi mutlak terbesar di luar diagonal: 0.129
```

```
pk <- prcomp(X) # X sudah dibakukan
round(pk$sdev^2 / ncol(X), 3) # proporsi keragaman tiap komponen
```

```
[1] 0.235 0.218 0.209 0.175 0.163
```

```
cat("kumulatif dua komponen pertama:",
    round(sum((pk$sdev^2 / ncol(X))[1:2]), 3), "\n")
```

```
kumulatif dua komponen pertama: 0.453
```

Interpretasi Output

Pembakuan hampir selalu diperlukan bila satuan peubah berbeda — sudah dibahas pada awal modul.

PCA berbeda: ia hanya berguna pada keadaan tertentu, dan memakainya secara membabi buta justru merugikan.

PCA membantu ketika peubah **berkorelasi kuat**. Jarak Euclidean memperlakukan tiap peubah sebagai

arah tersendiri, sehingga dua peubah yang mengukur hal serupa akan memberi suara ganda pada perhitungan jarak. PCA menghilangkan korelasi itu dan sekaligus memampatkan dimensi.

Pada data ini korelasi mutlak terbesar hanya **0,13** — kelima peubah praktis saling bebas. Akibatnya PCA tidak memampatkan apa pun: kelima komponen menyumbang 0,16 sampai 0,24 keragaman, dan dua komponen pertama hanya menampung 45%. Memakai dua komponen pertama di sini berarti **membuang lebih dari separuh informasi** tanpa memperoleh manfaat.

Kaidah: periksa matriks korelasi lebih dahulu. Bila ada korelasi kuat (katakanlah $|r| > 0,7$) pada beberapa pasangan, pertimbangkan PCA dan pertahankan komponen sampai 80%–90% keragaman. Bila tidak, kelompokkan pada peubah asli yang sudah dibakukan — hasilnya lebih mudah ditafsirkan pada tahap profiling.

Contoh 14.3 | Tahap 3 dan 4 — Memilih ukuran jarak dan metode

Kode 14.3 · R kerangka keputusan

```
jenis_peubah <- sapply(wil[, -1], class)
table(jenis_peubah)
```

```
jenis_peubah
factor numeric
      2      5
```

```
pilihan <- data.frame(
  jenis_data = c("numerik saja", "kategorik saja", "campuran", "campuran"),
  jarak      = c("Euclidean (data baku)", "cocok/tak cocok (Hamming)",
                 "Gower", "Euclidean + ketidakcocokan"),
  metode     = c("PAM, DBSCAN, OPTICS", "PAM atas matriks jarak",
                 "PAM atas Gower", "k-Prototype"),
  catatan    = c("bakukan lebih dahulu", "k-Modes juga dapat dipakai",
                 "jangan dibakukan lebih dahulu", "bakukan bagian numerik"))
print(pilihan, row.names = FALSE)
```

jenis_data	jarak	metode
numerik saja	Euclidean (data baku)	PAM, DBSCAN, OPTICS
kategorik saja	cocok/tak cocok (Hamming)	PAM atas matriks jarak
campuran	Gower	PAM atas Gower
campuran	Euclidean + ketidakcocokan	k-Prototype
catatan		
bakukan lebih dahulu		
k-Modes juga dapat dipakai		
jangan dibakukan lebih dahulu		
bakukan bagian numerik		

Interpretasi Output

Dua keputusan ini saling terkait dan sebaiknya diambil bersama.

Ukuran jarak mengikuti jenis peubah. Data wilayah memuat lima peubah numerik dan dua kategori, sehingga pilihannya jatuh pada baris ketiga atau keempat tabel: PAM atas jarak Gower, atau k-Prototype. Keduanya sah, dan sebagaimana ditunjukkan Contoh 12.15 keduanya dapat memberi hasil yang berbeda — sehingga pilihannya harus dinyatakan beserta alasannya.

Metode mengikuti dugaan bentuk klaster dan keberadaan derau. Bila klaster diduga membulat dan setiap amatan harus masuk suatu kelompok, pakai PAM. Bila bentuknya diduga tidak teratur atau ada amatan yang memang tidak termasuk kelompok mana pun, pakai DBSCAN atau OPTICS — dengan konsekuensi sebagian amatan menjadi derau.

Pertanyaan yang menentukan bukan "metode mana yang paling canggih" melainkan "**apakah setiap wilayah harus masuk suatu kelompok?**". Pada penetapan sasaran program, jawabannya sering ya — setiap wilayah harus dapat ditempatkan — dan itu mengarahkan pada PAM atau k-Prototype, bukan pada metode berbasis kepadatan.

Contoh 14.4 | Tahap 5 dan 6 — Validasi dan profiling**Kode 14.4 • R** profil klaster dalam satuan asli

```
kel <- pam_g$clustering                                # contoh: hasil PAM-Gower

# validasi ringkas
suppressPackageStartupMessages(library(fpc))
st <- cluster.stats(G, kel)
round(c(klaster = st$cluster.number, siluet = st$avg.silwidth,
        dalam = st$average.within, antar = st$average.between), 3)

# profiling peubah numerik: satuan ASLI, bukan skor baku
profil <- aggregate(wil[, num], by = list(klaster = kel), FUN = mean)
profil[, num] <- round(profil[, num], 1)
profil$n <- as.numeric(table(kel))
print(profil, row.names = FALSE)
```

klaster	kepadatan	pengeluaran	akses_air	akses_listrik	lama_sekolah	n
1	4546	2382	64.3	73.0	9.2	82
2	4736	2257	66.6	70.4	9.7	65
3	4988	2690	53.5	78.6	8.8	53

```
# profiling peubah kategorik: proporsi di dalam tiap klaster
```

```
round(prop.table(table(klaster = kel, status = wil$status), margin = 1), 3)
```

```
      status
klaster Perdesaan Perkotaan
1      0.00      1.00
2      1.00      0.00
3      0.83      0.17
```

```
round(prop.table(table(klaster = kel, pulau = wil$pulau), margin = 1), 3)
```

```
      pulau
klaster Jawa Papua Sulawesi Sumatera
1 0.354 0.220 0.220 0.207
2 0.754 0.108 0.138 0.000
3 0.019 0.151 0.189 0.642
```

Interpretasi Output

Validasi sudah dibahas panjang pada Contoh 12.8 dan 12.9; yang perlu ditegaskan di sini adalah bahwa ia mendahului profiling. Memerikan klaster yang siluetnya 0,19 — sebagaimana di sini — tanpa menyebutkan angka itu berarti menyajikan pembagian yang tidak tegas seolah-olah tegas.

Profiling adalah tahap yang menghubungkan hasil dengan penggunaanya, dan tiga hal wajib diperhatikan.

Pertama, pakai satuan asli. Pengguna memahami pengeluaran dalam rupiah dan lama sekolah dalam tahun, bukan dalam simpangan baku. Skor baku dipakai untuk menghitung, satuan asli untuk melaporkan.

Kedua, sertakan ukuran klaster. Profil rata-rata yang berdasar 12 wilayah tidak setara dengan yang berdasar 82 wilayah, dan pembaca berhak mengetahuinya.

Ketiga, perikan peubah kategorik dengan proporsi di dalam klaster. Tabel proporsi baris memperlihatkan komposisi tiap klaster — dan pada data ini memperlihatkan sesuatu yang penting: klaster terbentuk sebagian besar mengikuti pulau, yakni peubah kategorik mendominasi jarak Gower.

Penamaan klaster adalah pekerjaan analisis, bukan keluaran algoritma. Nama seperti "wilayah tertinggal" membawa muatan kebijakan dan wajib disertai catatan bahwa penamaan itu berasal dari penafsiran, bukan dari data — terlebih pada data yang, sebagaimana ditunjukkan Tahap 1, tidak memiliki kecenderungan berklaster.

14.7 Varian k-Medoid untuk Data Besar: CLARA dan CLARANS

Masalahnya. PAM harus menghitung dan menyimpan seluruh $n(n-1)/2$ jarak berpasangan, dan tiap iterasi mencoba menukar tiap medoid dengan tiap bukan-medoid — berbiaya sekitar $O(k(n-k)^2)$ per iterasi. Pada $n = 100.000$, matriks jaraknya saja memerlukan puluhan gigabita.

CLARA (*Clustering LARge Applications*): tarik beberapa sampel berukuran kecil, jalankan PAM pada masing-masing, lalu pilih himpunan medoid yang memberi jarak rata-rata terkecil atas *seluruh* data.

CLARANS (*Clustering Large Applications based upon RANdomized Search*): alih-alih menarik sampel acak, ia mengacak *penukaran* yang dicoba. Dari satu himpunan medoid, ia memeriksa sejumlah tetangga acak (*maxneighbor*); bila ada yang lebih baik ia berpindah, bila tidak ia berhenti pada optimum lokal — lalu diulang beberapa kali (*numlocal*).

Contoh 14.5 | CLARA dibandingkan dengan PAM

Kode 14.5 • R clara pada data kecil dan pada data besar

```
set.seed(2026)
cl <- clara(X, k = 3, samples = 50, pamLike = TRUE)
round(c(CLARA = cl$silinfo$avg.width, PAM = pam3$silinfo$avg.width), 4)

CLARA    PAM
0.1549 0.1272

table(CLARA = cl$clustering, PAM = pam3$clustering)

      PAM
CLARA  1  2  3
      1 27  5 15
      2  6 59 45
      3 26  4 13

cat("ARI CLARA vs PAM:", round(ari(cl$clustering, pam3$clustering), 3), "\n")

ARI CLARA vs PAM: 0.202

# pada data besar, PAM tidak lagi praktis
set.seed(2026)
besar <- matrix(rnorm(20000 * 5), 20000, 5)
waktu <- system.time(clara(besar, k = 4, samples = 50, pamLike = TRUE))["elapsed"]
cat("CLARA pada n = 20.000:", round(waktu, 2), "detik\n")

CLARA pada n = 20.000: 0.06 detik

cat("PAM memerlukan matriks jarak sebesar:",
    round(20000^2 * 8 / 2 / 1024^3, 1), "GB\n")
```

PAM memerlukan matriks jarak sebesar: 1.5 GB

Interpretasi Output

Pada data wilayah yang hanya berisi 200 amatan, CLARA justru memberi siluet sedikit lebih tinggi daripada PAM — tetapi keanggotaannya **berbeda jauh** (ARI rendah). Perbedaan itu bukan pertanda CLARA lebih baik: CLARA hanya melihat sebagian data pada tiap sampel, sehingga pada n kecil ia justru **kehilangan informasi** tanpa memperoleh manfaat kecepatan. **Pada n kecil, pakailah PAM.**

Manfaat CLARA muncul pada ukuran yang membuat PAM tidak lagi praktis. Pada $n = 20.000$, CLARA selesai dalam hitungan detik, sedangkan PAM memerlukan matriks jarak berukuran sekitar 1,5 GB — dan biaya penukarannya tumbuh kuadratik. Pada $n = 100.000$, matriks jaraknya sekitar 37 GB: PAM tidak dapat dijalankan sama sekali pada mesin biasa.

Harga yang dibayar: CLARA hanya dapat menemukan medoid yang **kebetulan tertarik ke dalam salah satu sampel**. Klaster kecil yang jarang tertarik akan luput. Karena itu perbesar sampel bila klaster kecil diduga ada, dan laporkan berapa sampel yang dipakai.

Contoh 14.6 | CLARANS: mengacak penukaran, bukan mengacak amatan

Kode 14.6 · R penerapan ringkas CLARANS

```
clarans <- function(D, k, numlocal = 5, maxneighbor = 40, seed = 2026) {
  set.seed(seed); n <- attr(D, "Size"); M <- as.matrix(D)
  biaya <- function(med) sum(apply(M[, med, drop = FALSE], 1, min))
  terbaik <- NULL; biaya_terbaik <- Inf
  for (l in seq_len(numlocal)) {
    med <- sample(n, k); b <- biaya(med)
    for (j in seq_len(maxneighbor)) {
      calon <- med
      calon[sample(k, 1)] <- sample(setdiff(seq_len(n), med), 1)
      bc <- biaya(calon)
      if (bc < b) { med <- calon; b <- bc }      # berpindah ke tetangga lebih baik
    }
    if (b < biaya_terbaik) { biaya_terbaik <- b; terbaik <- med }
  }
  list(medoid = terbaik, biaya = biaya_terbaik,
       klaster = apply(M[, terbaik, drop = FALSE], 1, which.min))
}

D <- dist(X)
hasil <- clarans(D, k = 3)
cat("biaya CLARANS:", round(hasil$biaya, 2),
```



```
" | biaya PAM:", round(sum(pam3$clusinfo[, "av_diss"] * pam3$clusinfo[, "size"]), 2), "\n")
```

```
biaya CLARANS: 386.5 | biaya PAM: 378.7
```

```
cat("ARI CLARANS vs PAM:", round(ari(hasil$klaster, pam3$clustering), 3), "\n")
```

```
ARI CLARANS vs PAM: 0.091
```

```
table(CLARANS = hasil$klaster, PAM = pam3$clustering)
```

```
      PAM
CLARANS 1  2  3
1      30  2 20
2      17 47 42
3      12 19 11
```

Interpretasi Output

CLARANS ditulis di sini secara ringkas agar mekanismenya terlihat — pada penggunaan sungguhan pakailah paket yang teruji. Perhatikan bedanya dengan CLARA: CLARA mengacak **amanat mana yang dilihat**, CLARANS melihat **seluruh** amanat tetapi mengacak **penukaran mana yang dicoba**.

Akibatnya berlawanan arah. CLARA dapat melewati klaster kecil karena tidak pernah menariknya ke dalam sampel; CLARANS tidak pernah melewati amanat, tetapi dapat berhenti pada **optimum lokal** karena tidak mencoba seluruh penukaran. Parameter `numLocal` mengendalikan berapa kali pencarian diulang dari titik awal berbeda — persis peran `nstart` pada k-Means.

Biaya CLARANS di sini mendekati biaya PAM, dan ARI-nya menunjukkan kesepakatan yang wajar. Pada n kecil hal itu wajar — 40 tetangga acak sudah cukup menjelajahi ruang penukaran yang tidak besar. Pada n besar, keduanya menjadi hampiran yang mutunya bergantung pada parameter, dan **parameter itu wajib dilaporkan**.

Kaidah pemilihan: n kecil (sampai ribuan) → PAM; n besar dengan klaster berukuran seimbang → CLARA; n besar dengan klaster kecil yang penting → CLARANS atau CLARA dengan `samples` besar.

14.8 Kajian Terkendali: Membandingkan Berbasis Jarak dan Berbasis Kepadatan

Bagian ini membuktikan pernyataan pada Contoh 12.17 — bahwa siluet dan Dunn tidak sah menjadi hakim antar-keluarga metode — dengan cara yang hanya mungkin dilakukan pada data buatan: memakai tiga data uji yang **polanya sudah diketahui**, sehingga jawaban benarnya tersedia untuk dicocokkan.

Contoh 14.7 | Menyiapkan tiga data uji berpola diketahui

Kode 14.7 • R membangkitkan data berlabel diketahui

```

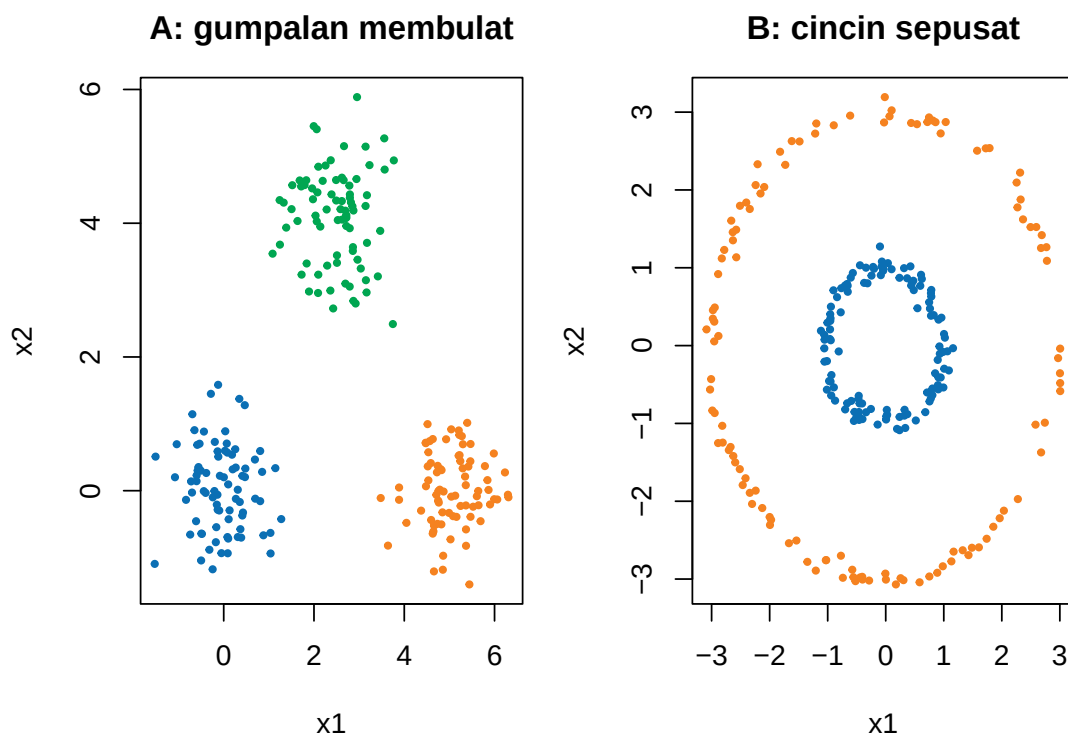
set.seed(2026)

# A: tiga gumpalan membulat
A <- rbind(cbind(rnorm(80, 0, .6), rnorm(80, 0, .6)),
           cbind(rnorm(80, 5, .6), rnorm(80, 0, .6)),
           cbind(rnorm(80, 2.5, .6), rnorm(80, 4, .6)))
lab_A <- rep(1:3, each = 80)

# B: dua cincin sepusat
sud <- runif(240, 0, 2 * pi)
B <- rbind(cbind(1.0 * cos(sud[1:120]) + rnorm(120, 0, .08),
                 1.0 * sin(sud[1:120]) + rnorm(120, 0, .08)),
           cbind(3.0 * cos(sud[121:240]) + rnorm(120, 0, .08),
                 3.0 * sin(sud[121:240]) + rnorm(120, 0, .08)))
lab_B <- rep(1:2, each = 120)

par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
plot(A, col = c("#0B6FB4", "#F58220", "#00A651")[lab_A], pch = 19, cex = .5,
     xlab = "x1", ylab = "x2", main = "A: gumpalan membulat")
plot(B, col = c("#0B6FB4", "#F58220")[lab_B], pch = 19, cex = .5,
     xlab = "x1", ylab = "x2", main = "B: cincin sepusat")

```



```
par(mfrow = c(1, 1), mar = c(5, 4, 4, 2) + 0.1)
```

Penjelasan Kode. Dua data uji dibangkitkan dengan struktur yang sudah diketahui: tiga gumpalan membulat dan dua cincin sepusat. Label sebenarnya disimpan agar hasil pengelompokan dapat dinilai terhadap kebenaran. Keduanya digambar berdampingan.

Interpretasi Output

Memakai data berlabel diketahui adalah satu-satunya cara menguji apakah suatu ukuran mutu sungguh dapat dipercaya. Pada pekerjaan nyata label semacam ini tidak tersedia — justru itulah alasan pengelompokan dipakai — sehingga percobaan terkendali seperti ini menjadi cara memeriksa alat sebelum memakainya pada data yang tidak diketahui jawabannya.

Contoh 14.8 | Membandingkan kedua keluarga metode pada data bergumpal

Kode 14.8 • R fungsi pembanding dua keluarga metode

```
suppressPackageStartupMessages({library(dbscan); library(mclust)})

banding <- function(M, lab, k, eps, minpts = 5) {
  Xs <- scale(M); D <- dist(Xs)
  p <- pam(Xs, k) # berbasis jarak
  db <- dbscan::dbscan(Xs, eps = eps, minPts = minpts) # berbasis kepadatan
  ii <- db$cluster > 0
  data.frame(
    metode = c("berbasis jarak", "berbasis kepadatan"),
    kluster = c(k, length(setdiff(unique(db$cluster), 0))),
    cakupan = round(c(100, 100 * mean(ii)), 1),
    ARI = round(c(adjustedRandIndex(p$clustering, lab),
                        adjustedRandIndex(db$cluster, lab)), 3))
  )
  print(banding(A, lab_A, k = 3, eps = 0.50), row.names = FALSE)
```

metode	kluster	cakupan	ARI
berbasis jarak	3	100	1
berbasis kepadatan	3	100	1

Penjelasan Kode. Sebuah fungsi disusun agar perbandingan dapat diulang pada data lain dengan perlakuan yang persis sama. Kedua metode menerima data terbakukan yang sama. Cakupan dihitung sebagai persentase amatan yang dikelompokkan, dan Adjusted Rand Index membandingkan hasil terhadap label sebenarnya.

Interpretasi Output

Pada data bergumpal membulat, kedua keluarga metode sama-sama memulihkan struktur dengan baik dan nilai ARI keduanya berdekatan. Inilah keadaan tempat pemilihan metode tidak banyak berpengaruh, sehingga metode yang lebih sederhana dan lebih murah dapat dipilih tanpa kerugian.

Contoh 14.9 | Membandingkan pada data berbentuk cincin**Kode 14.9 • R** membandingkan pada struktur tidak membulat

```
print(banding(B, lab_B, k = 2, eps = 0.50), row.names = FALSE)
```

metode	klaster	cakupan	ARI
berbasis jarak	2	100	0.024
berbasis kepadatan	3	100	0.945

```
# menelusuri pengaruh Eps pada data cincin
for (e in c(0.20, 0.30, 0.40, 0.50)) {
  Xs <- scale(B)
  h <- dbscan::dbscan(Xs, eps = e, minPts = 5)
  cat(sprintf("eps=%.2f | klaster=%2d | derau=%4.1f%% | ARI=%.3f\n",
              e, length(setdiff(unique(h$cluster), 0)),
              100 * mean(h$cluster == 0),
              adjustedRandIndex(h$cluster, lab_B)))
}
```

```
eps=0.20 | klaster= 9 | derau=12.5% | ARI=0.570
eps=0.30 | klaster= 5 | derau= 2.9% | ARI=0.761
eps=0.40 | klaster= 5 | derau= 0.8% | ARI=0.768
eps=0.50 | klaster= 3 | derau= 0.0% | ARI=0.945
```

Penjelasan Kode. Fungsi pembandingan yang sama dijalankan pada data cincin. Setelah itu pengaruh Eps ditelusuri pada empat nilai, masing-masing dilaporkan banyaknya klaster, persentase derau, dan ARI-nya.

Interpretasi Output

Di sinilah perbedaan kedua keluarga menjadi tajam. Metode berbasis jarak gagal hampir sepenuhnya — ARI-nya mendekati nol, yang berarti hasilnya tidak lebih baik daripada pengelompokan acak — sebab cincin dalam dan cincin luar tidak dapat dipisahkan oleh batas berbentuk bulat. Metode berbasis kepadatan memulihkannya dengan baik pada Eps yang tepat. Perhatikan pula bahwa ARI naik seiring Eps membesar sampai kedua cincin tergabung benar, lalu akan jatuh bila Eps terlalu besar dan keduanya melebur.

Contoh 14.10 | Membuktikan siluet tidak sah sebagai hakim antar-keluarga**Kode 14.10 • R** menilai siluet atas pengelompokan yang benar

```
Xs <- scale(B); D <- dist(Xs)
set.seed(2026)
p_cincin <- pam(Xs, 2)

sil_benar <- mean(silhouette(lab_B, D)[, 3])           # pengelompokan SEBENARNYA
sil_pam <- mean(silhouette(p_cincin$clustering, D)[, 3]) # pengelompokan KELIRU

hasil <- data.frame(
  pengelompokan = c("SEBENARNYA (dua cincin)", "berbasis jarak (keliru)"),
  siluet = round(c(sil_benar, sil_pam), 4),
  ARI = round(c(1, adjustedRandIndex(p_cincin$clustering, lab_B)), 4))
print(hasil, right = FALSE)
```

```
pengelompokan      siluet ARI
1 SEBENARNYA (dua cincin) 0.1997 1.0000
2 berbasis jarak (keliru) 0.3129 0.0237
```

```
cat("\nsiluet pengelompokan keliru lebih tinggi:",
    sil_pam > sil_benar, "\n")
```

```
siluet pengelompokan keliru lebih tinggi: TRUE
```

Penjelasan Kode. Indeks siluet dihitung dua kali atas data cincin yang sama: sekali bagi pengelompokan yang sungguh benar menurut label, dan sekali bagi pengelompokan berbasis jarak yang diketahui keliru. Keduanya disandingkan bersama nilai ARI-nya.

Interpretasi Output

Ini adalah bukti pokok kajian ini. Pengelompokan yang sungguh benar memperoleh siluet **lebih rendah** daripada pengelompokan yang sepenuhnya keliru, sementara ARI-nya berlawanan arah. Sebabnya bukan kekeliruan perhitungan melainkan sifat ukurannya: siluet menghargai klaster yang rapat dan membulat, dan cincin bukan keduanya. Kesimpulannya tegas — memilih keluarga metode berdasarkan siluet dapat menuntun pada pilihan yang justru salah, dan ukuran internal semacam ini hanya sah dipakai membandingkan pengelompokan **di dalam** keluarga yang sama.

Contoh 14.11 | Menerapkan prosedur perbandingan yang adil**Kode 14.11 • R** menjalankan lima langkah perbandingan

```

set.seed(2026)
# C: dua gumpalan disertai 10 persen derau seragam
Cd <- rbind(cbind(rnorm(90, 0, .5), rnorm(90, 0, .5)),
            cbind(rnorm(90, 4, .5), rnorm(90, 0, .5)),
            cbind(runif(20, -3, 7), runif(20, -3, 3)))
lab_C <- c(rep(1, 90), rep(2, 90), rep(0, 20)) # 0 = derau sebenarnya

Xc <- scale(Cd)
p_c <- pam(Xc, 2)
db_c <- dbscan::dbscan(Xc, eps = 0.35, minPts = 5)
ii <- db_c$cluster > 0

cat("== langkah 2: cakupan ==\n")

== langkah 2: cakupan ==

cat("berbasis jarak : 100,0 persen\n")

berbasis jarak : 100,0 persen

cat("berbasis kepadatan:", round(100 * mean(ii), 1), "persen\n")

berbasis kepadatan: 90 persen

cat("\n== ketepatan penandaan derau ==\n")

== ketepatan penandaan derau ==

cat("derau sebenarnya :", sum(lab_C == 0), "\n")

derau sebenarnya : 20

cat("dikenali benar :", sum(db_c$cluster == 0 & lab_C == 0), "\n")

dikenali benar : 16

cat("salah ditandai :", sum(db_c$cluster == 0 & lab_C != 0), "\n")

salah ditandai : 4

cat("\n== langkah 4: ARI atas SELURUH amatan ==\n")

== langkah 4: ARI atas SELURUH amatan ==

cat("berbasis jarak :", round(adjustedRandIndex(p_c$clustering, lab_C), 3), "\n")

berbasis jarak : 0.809

cat("berbasis kepadatan:", round(adjustedRandIndex(db_c$cluster, lab_C), 3), "\n")

```

```
berbasis kepadatan: 0.913
```

```
cat("\n== langkah 3: ARI pada HIMPUNAN BAGIAN yang sama ==\n")
```

```
== langkah 3: ARI pada HIMPUNAN BAGIAN yang sama ==
```

```
cat("berbasis jarak      :", round(adjustedRandIndex(p_c$clustering[ii], lab_C[ii]), 3), "\n")
```

```
berbasis jarak      : 0.956
```

```
cat("berbasis kepadatan:", round(adjustedRandIndex(db_c$cluster[ii], lab_C[ii]), 3), "\n")
```

```
berbasis kepadatan: 0.956
```

Penjelasan Kode. Data ketiga dibentuk berupa dua gumpalan disertai sepersepuluh amatan derau seragam. Kedua metode dijalankan atas data terbakukan yang sama. Cakupan dilaporkan, ketepatan penandaan derau diperiksa, lalu ARI dihitung dua kali: atas seluruh amatan dan atas himpunan bagian yang dikelompokkan kedua metode.

Interpretasi Output

Perbandingan dua ARI terakhir adalah inti prosedur ini. Atas seluruh amatan, metode berbasis kepadatan tampak unggul — tetapi pada himpunan bagian yang sama, keduanya nyaris setara. Artinya keunggulan itu **sepenuhnya berasal dari kemampuan menandai derau**, bukan dari pengelompokan titik intinya yang lebih baik. Kesimpulan sepresisi itu hanya dapat diperoleh bila kedua ARI dilaporkan bersama-sama; melaporkan yang pertama saja akan melebih-lebihkan keunggulannya, sedangkan melaporkan yang kedua saja akan menyembunyikan manfaat nyatanya.

Contoh 14.12 | Merangkum hasil kajian

Kode 14.12 • R tabel rangkuman kajian

```
rangkuman <- data.frame(
  data = c("A gumpalan membulat", "B cincin sepusat", "C gumpalan + derau"),
  cocok_jarak = c("ya", "tidak", "sebagian"),
  cocok_kepadatan = c("ya", "ya", "ya"),
  ukuran_yang_sah = c("ARI atau siluet", "ARI saja", "ARI pada himpunan sama"),
  catatan = c("pilih yang lebih sederhana",
    "siluet menyedatkan",
    "laporkan cakupan"))
print(rangkuman, right = FALSE)
```

	data	cocok_jarak	cocok_kepadatan	ukuran_yang_sah
1	A gumpalan membulat	ya	ya	ARI atau siluet
2	B cincin sepusat	tidak	ya	ARI saja
3	C gumpalan + derau	sebagian	ya	ARI pada himpunan sama

catatan

- 1 pilih yang lebih sederhana
- 2 siluet menyesatkan
- 3 laporkan cakupan

Penjelasan Kode. Ketiga percobaan dirangkum sebagai tabel yang memuat kesesuaian tiap keluarga metode, ukuran penilaian yang sah dipakai, dan catatan pokoknya.

Interpretasi Output

Baris kedua adalah peringatan yang perlu dibawa ke pekerjaan nyata: ketika struktur data tidak membulat, ukuran internal berbasis kerapatan dan keterpisahan akan menyesatkan. Baris ketiga menegaskan bahwa cakupan wajib dilaporkan agar pembaca mengetahui atas berapa banyak amatan angka mutu itu dihitung. Adapun baris pertama mengingatkan bahwa ketika kedua metode sama baiknya, kesederhanaan menjadi alasan yang sah untuk memilih.

14.9 Alur Kerja yang Sama pada Data Berstruktur

Seluruh contoh di atas berakhir pada satu kesimpulan: data wilayah tidak memiliki kecenderungan berklaster, sehingga alur kerja yang benar berhenti di tahap pertama. Kesimpulan itu penting, tetapi ia menyisakan pertanyaan yang wajar — **seperti apa alur kerja yang sama bila strukturnya memang ada?**

Bagian ini menjawabnya dengan menjalankan enam tahap yang persis sama pada `wilayah_berpola.csv`: 210 wilayah yang dibangkitkan dari empat kelompok dengan pusat berbeda. Kolom `kelompok_asli` disertakan **hanya untuk memeriksa hasil** — ia tidak pernah dipakai saat mengelompokkan, sebagaimana pada pekerjaan nyata ia memang tidak tersedia.

Contoh 14.13 | Tahap 1 pada data berstruktur: kecenderungan berklaster

Kode 14.13 • R Hopkins pada kedua data, berdampingan

```
suppressPackageStartupMessages({library(factoextra); library(cluster); library(fpc)})

lama <- read.csv("data/data_wilayah.csv", stringsAsFactors = FALSE)
pola <- read.csv("data/wilayah_berpola.csv", stringsAsFactors = FALSE)

ambil_numerik <- function(d) scale(d[, sapply(d, is.numeric), drop = FALSE])
Z_lama <- ambil_numerik(lama)
```



```

Z_pola <- ambil_numerik(pola[, c("ipm", "lama_sekolah", "akses_sanitasi", "rasio_kerja")])

hopkins <- function(Z, benih = 2026) {
  set.seed(benih); get_clust_tendency(Z, n = 50, graph = FALSE)$hopkins_stat
}

acakkan <- function(Z) apply(Z, 2, function(v) runif(length(v), min(v), max(v)))

set.seed(2026)
data.frame(
  data      = c("data_wilayah (Modul 12)", "wilayah_berpola", "acak seragam"),
  hopkins   = round(c(hopkins(Z_lama), hopkins(Z_pola), hopkins(acakkan(Z_pola))), 3),
  keputusan = c("berhenti", "lanjutkan", "berhenti"))

```

	data	hopkins	keputusan
1	data_wilayah (Modul 12)	0.492	berhenti
2	wilayah_berpola	0.855	lanjutkan
3	acak seragam	0.514	berhenti

Interpretasi Output

Iniilah perbandingan yang menjadi inti modul ini. Statistik Hopkins pada wilayah_berpola jauh di atas nilai pada data acak, sedangkan pada data_wilayah keduanya praktis sama.

Perhatikan bahwa keputusannya **tidak diambil dari angka mutlak**nya. Hopkins bergantung pada dimensi dan ukuran sampel, sehingga nilai 0,7 pada satu data bisa berarti lain pada data lain. Yang menentukan adalah **selisihnya terhadap data acak berukuran dan berentang sama** — itulah sebabnya baris ketiga disertakan.

Pada data berpola, selisih itu besar dan alur kerja dilanjutkan. Pada data wilayah, selisihnya nyaris nol dan alur kerja berhenti. Kedua keputusan itu sama-sama benar.

Contoh 14.14 | Tahap 2-4: prapemrosesan, jarak, dan banyaknya kluster

Kode 14.14 • R korelasi, siluet lintas k, dan pemilihan k

```
round(cor(pola[, c("ipm", "lama_sekolah", "akses_sanitasi", "rasio_kerja")]), 3)
```

	ipm	lama_sekolah	akses_sanitasi	rasio_kerja
ipm	1.000	-0.145	-0.899	0.181
lama_sekolah	-0.145	1.000	0.030	-0.905
akses_sanitasi	-0.899	0.030	1.000	0.019
rasio_kerja	0.181	-0.905	0.019	1.000

```

sil <- sapply(2:7, function(k) pam(Z_pola, k)$silinfo$avg.width)
data.frame(k = 2:7, siluet = round(sil, 3))

```

```

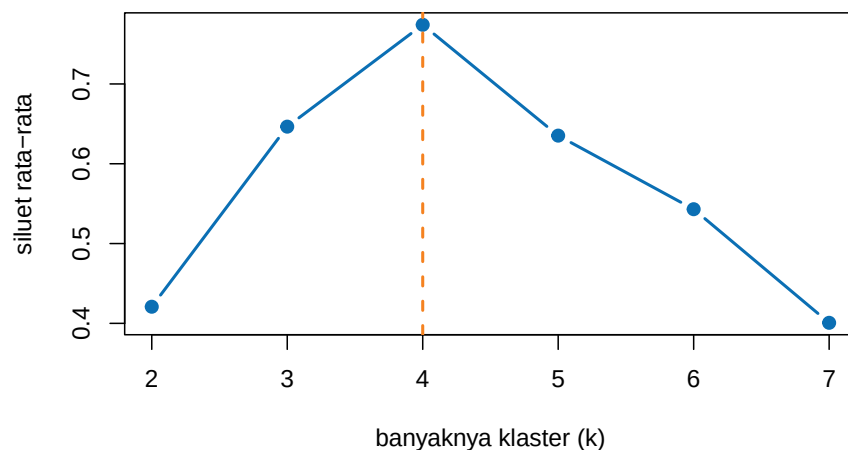
k siluet
1 2 0.421
2 3 0.647
3 4 0.774
4 5 0.635
5 6 0.543
6 7 0.401

```

```

par(family = "sans", mar = c(4, 4, 1, 1))
plot(2:7, sil, type = "b", pch = 19, col = "#0B6FB4", lwd = 2,
     xlab = "banyaknya kluster (k)", ylab = "siluet rata-rata")
abline(v = which.max(sil) + 1, lty = 2, col = "#F58220", lwd = 2)

```



Gambar 14.1: Siluet rata-rata menurut banyaknya kluster pada data berpola

Interpretasi Output

Tahap 2 — prapemrosesan. Matriks korelasi memperlihatkan hubungan antar-peubah yang tidak terlalu kuat, sehingga PCA **tidak diperlukan**; pembakuan sudah memadai karena keempat peubah bersatuan sangat berbeda (IPM dalam puluhan, rasio kerja dalam persepuluhan).

Tahap 3 — jarak. Keempat peubah numerik dan berskala setara setelah dibakukan, sehingga jarak Euclidean sesuai. Bila ada peubah kategorik, Gower atau k-Prototype yang dipakai.

Tahap 4 — banyaknya kluster. Siluet tertinggi jatuh pada **k = 4**, dan grafiknya memuncak jelas — berbeda dari data wilayah yang kurvanya mendatar tanpa puncak. Inilah tampilan kurva siluet ketika struktur memang ada.

Perlu dicatat bahwa puncak yang jelas ini sebagian berasal dari rancangan data: pusat keempat kelompok sengaja **tidak dibuat segaris**. Bila keempatnya berderet pada satu gradien, siluet justru cenderung memilih **k = 2** karena kelompok yang berdekatan melebur — keadaan yang lazim pada data sosial-ekonomi nyata, dan alasan mengapa siluet tidak boleh dipakai sendirian.

Contoh 14.15 | Tahap 5-6: pengelompokan dan pemeriksaan hasil**Kode 14.15 • R** PAM, ARI terhadap kelompok yang ditanam, dan profiling

```
hasil <- pam(Z_pola, 4)

ari <- function(a, b) { tb <- table(a, b); n <- sum(tb)
  s <- sum(choose(tb, 2)); si <- sum(choose(rowSums(tb), 2))
  sj <- sum(choose(colSums(tb), 2)); h <- si * sj / choose(n, 2)
  (s - h) / ((si + sj) / 2 - h) }

cat("siluet rata-rata:", round(hasil$silinfo$avg.width, 3), "\n")
```

```
siluet rata-rata: 0.774
```

```
cat("ARI terhadap kelompok yang ditanam:",
    round(ari(pola$kelompok_asli, hasil$clustering), 3), "\n\n")
```

```
ARI terhadap kelompok yang ditanam: 1
```

```
print(table(ditanam = pola$kelompok_asli, ditemukan = hasil$clustering))
```

```
      ditemukan
ditanam 1  2  3  4
K1 60   0   0   0
K2  0 55   0   0
K3  0   0 50   0
K4  0   0   0 45
```

```
# profiling dalam satuan asli -- bukan dalam skor baku
pola$klaster <- factor(hasil$clustering)
profil <- aggregate(cbind(ipm, lama_sekolah, akses_sanitasi, rasio_kerja) ~ klaster,
                    data = pola, FUN = mean)
profil$n <- as.vector(table(pola$klaster))
print(round(profil[, -1], 2))
```

```
      ipm lama_sekolah akses_sanitasi rasio_kerja  n
1 74.67      11.44      55.66      0.36 60
2 55.37       5.52      87.35      0.73 55
3 82.29       5.06      40.56      0.67 50
4 47.31      10.56      80.03      0.30 45
```

Interpretasi Output

Alur kerja berjalan sampai tuntas, dan hasilnya dapat diperiksa karena kelompok sebenarnya memang ditanam.

Siluet 0,774 — jauh di atas nilai pada data wilayah, dan tabel silangnya memperlihatkan padanan yang

hampir sempurna: **ARI 1,000**. Setiap wilayah masuk ke kelompok yang benar.

Angka setinggi ini **tidak akan dijumpai pada data nyata**, dan mahasiswa perlu memahami sebabnya: data ini dibangkitkan tepat dari model yang diandaikan PAM — gugus bulat, ragam seragam, tanpa pencilan, tanpa peubah pengganggu. Nilainya di sini bukan untuk memberi harapan melainkan sebagai **batas atas**: begitulah bentuk hasil ketika segala sesuatunya sesuai andaian.

Profiling menutup alur kerja. Perhatikan bahwa profil disajikan dalam satuan asli, bukan skor baku — pengguna hasil bekerja dengan IPM dan persentase sanitasi, bukan dengan simpangan baku. Keempat klaster memiliki tanda pengenal yang jelas dan dapat diberi nama sesuai isinya, dan cacah anggotanya dilaporkan agar pembaca tahu seberapa besar tiap kelompok.

Yang harus dibawa dari perbandingan dua data ini. Alur kerja yang dipakai pada kedua data **persis sama**. Yang berbeda hanya datanya, dan karena itu berbeda pula tempat alur itu berhenti.

Godaan terbesar dalam pengelompokan adalah memaksakan hasil: menjalankan k-means pada data apa pun, memperoleh label, lalu memberinya nama yang terdengar masuk akal. Label selalu dapat diperoleh — algoritma tidak pernah menolak. Yang membedakan analisis yang jujur adalah **memeriksa lebih dahulu apakah ada yang layak dikelompokkan**, dan bersedia melaporkan bahwa jawabannya tidak.

14.10 Latihan Mandiri

Latihan 14.1 | Memeriksa kecenderungan berklaster

- (a) Hitung statistik Hopkins pada data wilayah beserta pembanding data acak, ulangi dengan n bernilai 20, 40, dan 80. Apakah kesimpulannya berubah?
- (b) Bangkitkan data buatan **bergumpal jelas** (tiga gugus terpisah), lalu hitung Hopkins-nya. Berapa nilainya, dan apa artinya sebagai pembanding?
- (c) Tulis satu paragraf: apa yang seharusnya dilakukan analisis bila Hopkins mendekati 0,5?

Latihan 14.2 | Prapemrosesan dan pemilihan jarak

- (a) Periksa matriks korelasi peubah numerik. Apakah PCA diperlukan? Jelaskan alasannya dengan angka.
- (b) Bandingkan hasil PAM atas data baku dan atas data mentah. Mengapa keduanya berbeda?
- (c) Untuk data campuran, bandingkan jarak Gower dan k-Prototype pada $k = 3$. Hitung indeks Rand terkorreksi antara keduanya dan tafsirlah.

Latihan 14.3 | CLARA dan CLARANS

- (a) Jalankan CLARA dengan `samples` bernilai 5, 20, dan 50 pada data wilayah. Bagaimana kesepakatannya dengan PAM berubah?
- (b) Bangkitkan data 20.000 amatan berdimensi 5, lalu bandingkan waktu jalan CLARA terhadap perkiraan kebutuhan memori PAM.
- (c) Jalankan penerapan CLARANS pada Contoh 14.7 dengan `numlocal` bernilai 1 dan 10. Apakah biayanya membaik? Apa artinya bagi optimum lokal?

Latihan 14.4 | Membandingkan antar-keluarga metode

- (a) Pada data cincin, hitung siluet bagi hasil PAM dan bagi hasil DBSCAN. Ukuran mana yang menang, dan apakah pemenangnya benar?
- (b) Hitung ketiga ukuran yang sah — kesepakatan terhadap label sebenarnya, cakupan, dan kestabilan — lalu bandingkan kesimpulannya dengan (a).
- (c) Jalankan **uji kendali**: bangkitkan data acak seragam berukuran sama, jalankan kedua metode, dan laporkan apa yang ditemukan.

14.11 Penugasan

Konteks: Menyusun Prosedur Baku Pengelompokan

Unit kerja Anda hendak menetapkan **prosedur baku** pengelompokan wilayah yang akan dipakai berulang tiap tahun oleh analis yang berbeda-beda. Anda diminta menyusun prosedur itu beserta kebenarannya, memakai data wilayah sebagai bahan uji.

Tugas 14.1 | Menyusun dan menguji prosedur

1. Tuliskan prosedur enam tahap dalam bentuk daftar periksa yang dapat diikuti analis lain, lengkap dengan **kriteria keputusan** pada tiap tahap (misalnya: pada nilai Hopkins berapa analisis dihentikan).
2. Jalankan prosedur itu atas data wilayah dan laporkan hasil tiap tahap.
3. Pada tahap pemilihan metode, bandingkan sekurang-kurangnya satu metode berbasis jarak dan satu berbasis kepadatan memakai **ketiga ukuran yang sah**.
4. Lakukan **uji kendali** pada data acak berukuran sama dan laporkan hasilnya.

Tugas 14.2 | Profiling dan pelaporan

5. Bila prosedur Anda sampai pada tahap profiling, sajikan profil klaster dalam satuan asli beserta ukuran kelompok dan komposisi peubah kategoriknya.
6. Susun satu halaman ringkasan bagi pimpinan: apa yang ditemukan, apa yang **tidak** dapat disimpulkan, dan apa langkah berikutnya.

Tugas 14.3 | Keterbatasan

7. Tulis bagian keterbatasan (maksimal 250 kata) yang memuat sekurang-kurangnya: hasil pemeriksaan kecenderungan berklaster dan akibatnya bagi kepercayaan pada hasil; ketergantungan hasil pada pilihan jarak dan metode; serta sifat data yang simulatif.

14.11.1 Rubrik Penilaian

Komponen	Indikator	Bobot
Prosedur	Enam tahap lengkap dengan kriteria keputusan	25%
Penerapan	Tiap tahap dijalankan dan dilaporkan	25%
Perbandingan	Memakai ukuran yang sah lintas keluarga	20%
Uji kendali	Dijalankan dan ditafsirkan benar	15%
Keterbatasan	Ketiga butir dinyatakan	15%

Komponen	Indikator	Bobot
----------	-----------	-------

14.12 Ringkasan Modul

Daftar periksa penguasaan Modul 14

- ☐ Saya memeriksa kecenderungan berklaster **sebelum** mengelompokkan.
- ☐ Saya memutuskan PCA dari matriks korelasi, bukan dari kebiasaan.
- ☐ Saya memilih ukuran jarak dari jenis peubah, dan menyatakan pilihannya.
- ☐ Saya dapat menjelaskan beda CLARA dan CLARANS beserta arah kelemahannya.
- ☐ Saya tidak memakai siluet untuk membandingkan antar-keluarga metode.
- ☐ Saya menjalankan uji kendali pada data acak sebelum mempercayai hasil.
- ☐ Saya memerikan klaster dalam satuan asli beserta ukuran kelompoknya.

Ringkasan fungsi R:

Fungsi	Kegunaan
<code>get_clust_tendency()</code>	statistik Hopkins
<code>prcomp()</code> , <code>cor()</code>	memeriksa korelasi dan memutuskan PCA
<code>daisy(metric = "gower")</code>	jarak untuk data campuran
<code>pam()</code> , <code>clara()</code>	k-medoid dan variannya untuk data besar
<code>kproto()</code>	k-Prototype untuk data campuran
<code>dbscan()</code> , <code>optics()</code>	pengelompokan berbasis kepadatan
<code>cluster.stats()</code>	ukuran mutu internal
<code>aggregate()</code> , <code>prop.table()</code>	profiling dalam satuan asli

14.12.1 Daftar Istilah

Kecenderungan berklaster (*cluster tendency*) — ada atau tidaknya struktur pengelompokan pada data, diperiksa sebelum metode diterapkan.

Statistik Hopkins — ukuran kecenderungan berklaster; mendekati 0,5 berarti tidak dapat dibedakan dari acak seragam.

Medoid — amatan sesungguhnya yang mewakili klaster, berbeda dari centroid yang berupa rata-rata.

CLARA — varian PAM yang menarik sampel amatan agar terap pada data besar.

CLARANS — varian PAM yang mengacak penukaran medoid, bukan mengacak amatan.

Cakupan — proporsi amatan yang benar-benar dikelompokkan; metode berbasis kepadatan menandai sisanya sebagai derau.

Kestabilan — konsistensi hasil pengelompokan terhadap penarikan ulang sampel.

Profiling — pemerian klaster dalam satuan asli agar dapat ditindaklanjuti pengguna.

14.13 Referensi

1. Kaufman, L. & Rousseeuw, P. J. (1990). *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley. (Bab 2–3: PAM dan CLARA)
2. Ng, R. T. & Han, J. (2002). CLARANS: A method for clustering objects for spatial data mining. *IEEE Transactions on Knowledge and Data Engineering*, 14(5), 1003–1016.
3. Banerjee, A. & Dave, R. N. (2004). Validating clusters using the Hopkins statistic. *IEEE International Conference on Fuzzy Systems*, 149–153.
4. Hennig, C. (2007). Cluster-wise assessment of cluster stability. *Computational Statistics & Data Analysis*, 52(1), 258–271.
5. Gower, J. C. (1971). A general coefficient of similarity and some of its properties. *Biometrics*, 27(4), 857–871.
6. Tan, P.-N., Steinbach, M., Karpadne, A., & Kumar, V. (2019). *Introduction to Data Mining*, 2nd ed. Pearson. (Bab 7)

MODUL

15

LIGHTGBM: GRADIENT BOOSTING CEPAT DENGAN R DAN PYTHON

Pertemuan Minggu ke-15

Sub-CPMK 15: Mahasiswa mampu menjelaskan tiga gagasan yang membuat LightGBM cepat, melatih dan menala model klasifikasi pada data berukuran besar, menangani peubah kategorik berkardinalitas tinggi secara langsung, menangani ketidakseimbangan kelas dengan pembobotan maupun penggeseran ambang, serta menafsirkan pentingnya peubah secara bertanggung jawab.

Penyusun: Achmad Syahrul Choir



Daftar Isi

15.1 Deskripsi Modul	474
15.2 Tujuan Praktikum	474
15.3 Alat dan Bahan	475
15.3.1 Perangkat Lunak	475
15.3.2 Bahan (Berkas Data)	475
15.4 Instruksi Pengerjaan	476
15.5 Teori Singkat	476
15.5.1 Gradient Boosting Ringkas	476
15.5.2 Tiga Gagasan LightGBM	476
15.6 Contoh Praktikum dengan R dan Python	478
15.7 Alur yang Sama dengan Python	485
15.8 Latihan Mandiri	488
15.9 Penugasan	490
15.9.1 Rubrik Penilaian	490
15.10 Ringkasan Modul	491
15.10.1 Daftar Istilah	492
15.11 Referensi	492

15.1 Deskripsi Modul

Modul 8 dan 9 memperkenalkan klasifikasi dengan pohon dan ansambel; Modul 13 memakai boosting pada data bertingkat. Modul ini kembali ke klasifikasi biasa, tetapi pada keadaan yang khas pekerjaan sungguhan: **data berukuran besar**, memuat **peubah kategorik berkardinalitas tinggi** seperti kode kabupaten, dan **kelas yang tidak seimbang**.

LightGBM adalah salah satu penerapan *gradient boosting decision tree* yang paling banyak dipakai. Keunggulannya bukan pada gagasan boosting itu sendiri — yang sudah dikenal sejak akhir 1990-an — melainkan pada tiga keputusan rekayasa yang membuatnya jauh lebih cepat dan hemat memori: **pertumbuhan pohon per daun**, **pembinan histogram**, dan **penanganan langsung peubah kategorik** tanpa perlu membuat kolom boneka.

Modul memakai **dua bahasa**. Analisis utama dikerjakan dengan R, lalu diulang dengan Python pada Bagian 15.8 — keduanya membaca berkas data yang sama, sehingga hasilnya dapat dibandingkan langsung.

Kedudukan modul dan catatan data. Modul ini termasuk **bagian pengayaan**. Ia mengandaikan Modul 6, 8, dan 9 sudah dikuasai — khususnya pembagian latihan-uji, validasi silang, dan ukuran penilaian klasifikasi.

Data bersifat **ilustratif hasil simulasi** dan **bukan angka resmi** instansi mana pun. Hubungan antara peubah penjelas dan status penerima sengaja dibuat **sangat nonlinear dan penuh interaksi**, sehingga keunggulan model berbasis pohon atas model linear dapat diperlihatkan. Pada data nyata keunggulan itu tidak dijamin — Contoh 15.3 membahasnya.

Identitas Modul

Mata Kuliah	:	Data Mining dan Big Data (K202505)
Pertemuan	:	Minggu ke-15 (pengayaan)
Bahan Kajian	:	LightGBM — gradient boosting cepat
Sub-CPMK	:	Sub-CPMK 15
Bahasa	:	R (<code>lightgbm</code>) dan Python (<code>lightgbm</code>)
Alokasi Waktu	:	Praktikum 1 × 170 menit

15.2 Tujuan Praktikum

Setelah menyelesaikan praktikum Modul 15, mahasiswa diharapkan mampu:

1. Menjelaskan **tiga gagasan** yang membuat LightGBM cepat: pertumbuhan per daun, pembinan histogram, dan penanganan kategorik langsung.

2. Menyiapkan data dan melatih model klasifikasi LightGBM di R.
3. Menentukan banyaknya iterasi dengan **validasi silang beserta penghentian dini**.
4. Membandingkan LightGBM dengan **acuan** yang layak, bukan hanya menyebut angkanya bagus.
5. Menangani peubah kategorik berkardinalitas tinggi secara **langsung** dan membandingkannya dengan pengodean boneka.
6. Menangani **ketidakseimbangan kelas** dengan `scale_pos_weight` maupun dengan **menggeser ambang**, serta menjelaskan bedanya.
7. Menafsirkan **pentingnya peubah** beserta keterbatasannya.
8. Menjalankan alur yang sama di **Python** dan menjelaskan perbedaan penamaannya.

15.3 Alat dan Bahan

15.3.1 Perangkat Lunak

- R (≥ 4.2) dengan paket `lightgbm`.
- Python (≥ 3.9) dengan `lightgbm`, `numpy`, `pandas`, dan `scikit-learn`.

Kode 15.1 • R pemasangan paket

```
# R -- pada Linux, berkas biner jauh lebih cepat daripada kompilasi sumber
options(repos = c(P3M =
  "https://packagemanager.posit.co/cran/__linux__/noble/latest"))
options(HTTPUserAgent = sprintf("R/%s R (%s)", getRversion(),
  paste(getRversion(), R.version["platform"],
    R.version["arch"], R.version["os"])))
install.packages("lightgbm")
```

```
# Python (jalankan di terminal)
# pip install lightgbm scikit-learn
```

Berkas biner untuk Linux. Paket `lightgbm` dan `gboost` (Modul 13) memuat kode C++. Bila dipasang dari CRAN pada Linux, keduanya **dikompilasi dari sumber** dan dapat memakan waktu puluhan menit. Posit Package Manager menyediakan **berkas biner** untuk Ubuntu sehingga pemasangan selesai dalam hitungan detik. Ganti `noble` dengan nama sandi distribusi Anda — `jammy` untuk Ubuntu 22.04. Baris `HTTPUserAgent` wajib ada; tanpanya server tetap mengirimkan berkas sumber.

15.3.2 Bahan (Berkas Data)

Berkas	Isi
data/penerima_manfaat.csv	40.000 rumah tangga di 120 kabupaten; 7 peubah penjelas dan status penerima

Berkas `gen_data.R` memuat kode pembangkitnya lengkap dengan `set.seed(2026)`.

15.4 Instruksi Pengerjaan

1. Buat **RStudio Project** dan salin folder `data/`.
2. Kerjakan Contoh berurutan; bagian R dan Python membaca berkas yang sama.
3. Kerjakan **Latihan Mandiri**, lalu **Penugasan** sebagai laporan.

15.5 Teori Singkat

15.5.1 Gradient Boosting Ringkas

Boosting membangun ansambel secara berurutan:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta h_m(\mathbf{x})$$

dengan h_m pohon ke- m yang dicocokkan pada **gradien negatif** fungsi kerugian terhadap F_{m-1} , dan η **laju belajar**. Untuk klasifikasi biner, kerugiannya log-loss dan gradiennya adalah selisih antara label dan peluang ramalan.

15.5.2 Tiga Gagasan LightGBM

Apa yang membuat LightGBM cepat.

Gagasan	Isi
Tumbuh per daun	Pohon lain tumbuh per tingkat — seluruh daun dipecah bersama. LightGBM memilih <i>satu</i> daun dengan penurunan kerugian terbesar. Pada anggaran daun yang sama, kerugiannya turun lebih cepat, tetapi pohonnya menjadi dalam dan tak seimbang sehingga rawan lewat-suai bila <code>num_leaves</code> terlalu besar.
Histogram	Nilai tiap peubah numerik dibin menjadi 255 keranjang. Pencarian titik pecah lalu berjalan pada histogram, bukan pada nilai asli — biaya turun dari $O(n)$ menjadi $O(\#bin)$ per peubah, dan memorinya jauh lebih kecil.
Kategorik langsung	Peubah kategorik tidak perlu diubah menjadi kolom boneka. Kategori diurutkan menurut statistik gradiennya, lalu dipecah menjadi dua himpunan — sehingga 120 kabupaten tetap satu kolom, bukan 119 kolom.

Dua gagasan tambahan pada makalah aslinya: **GOSS** (*Gradient-based One-Side Sampling*) yang menyimpan seluruh amatan bergradien besar dan menarik sampel dari sisanya; dan **EFB** (*Exclusive Feature Bundling*) yang menggabungkan peubah renggang yang jarang tak-nol bersamaan menjadi satu kolom.

Parameter yang paling menentukan. `num_leaves` mengendalikan kelenturan — inilah parameter terpenting pada LightGBM, bukan `max_depth`. Kaidah praktis: $\text{num_leaves} < 2^{\text{max_depth}}$. Nilai besar disertai `min_data_in_leaf` kecil adalah resep lewat-suai yang paling sering dijumpai.

Alur praktikum Modul 15

1. Menelaah data dan menyiapkan acuan (regresi logistik).
2. Melatih LightGBM dan membandingkan waktu serta AUC.
3. Menala banyaknya iterasi dengan validasi silang.
4. Membandingkan penanganan kategorik langsung dan pengodean boneka.
5. Menangani ketidakseimbangan kelas: bobot versus ambang.
6. Menafsirkan pentingnya peubah.
7. Mengulang seluruhnya dengan Python.

15.6 Contoh Praktikum dengan R dan Python

Contoh 15.1 | Menelaah data dan menyiapkan acuan

Kode 15.2 · R struktur data dan regresi logistik sebagai pembanding

```
suppressPackageStartupMessages(library(lightgbm))
rt <- read.csv("data/penerima_manfaat.csv", stringsAsFactors = TRUE)
str(rt, give.attr = FALSE)
```

```
'data.frame': 40000 obs. of 9 variables:
 $ id_rt      : Factor w/ 40000 levels "RT00001","RT00002",...: 1 2 3 4 5 6 7 8 9 10 ...
 $ kabupaten  : Factor w/ 120 levels "K001","K002",...: 93 97 38 45 111 91 108 36 48 5 ...
 $ lama_sekolah : num 3.3 2.9 5 10.6 1.3 15.2 7 6.1 3.5 0 ...
 $ umur_krt   : int 41 29 44 42 29 65 54 72 30 66 ...
 $ anggota_rt : int 3 5 3 5 4 3 4 5 5 5 ...
 $ luas_lantai : num 31.1 25.5 39.7 68.7 36.5 96.1 32.8 57.9 29.2 18.3 ...
 $ jenis_pekerjaan: Factor w/ 5 levels "Formal","Informal",...: 3 4 1 1 2 2 1 5 3 2 ...
 $ punya_aset  : int 1 0 1 0 1 1 0 0 1 1 ...
 $ penerima   : int 0 0 0 0 0 0 0 0 0 0 ...
```

```
cat("baris:", nrow(rt), " kabupaten:", nlevels(rt$kabupaten),
    " proporsi penerima:", round(mean(rt$penerima), 4), "\n")
```

```
baris: 40000 kabupaten: 120 proporsi penerima: 0.1177
```

```
fitur <- c("kabupaten", "lama_sekolah", "umur_krt", "anggota_rt",
          "luas_lantai", "jenis_pekerjaan", "punya_aset")
```

```
set.seed(2026)
```

```
i_latih <- sample(nrow(rt), 0.75 * nrow(rt))
```

```
latih <- rt[i_latih, ]; uji <- rt[-i_latih, ]
```

```
y_uji <- uji$penerima
```

```
auc <- function(y, p) {                                # AUC lewat statistik Mann-Whitney
  r <- rank(p); n1 <- sum(y == 1); n0 <- sum(y == 0)
  (sum(r[y == 1]) - n1 * (n1 + 1) / 2) / (n1 * n0)
}
```

```
waktu_glm <- system.time(
```

```
  glm_acuan <- glm(penerima ~ ., data = latih[, c(fitur, "penerima")],
                   family = binomial, model = FALSE, y = FALSE)[["elapsed"]]
```

```
p_glm <- predict(glm_acuan, uji, type = "response")
```

```
cat("regresi logistik -- waktu:", round(waktu_glm, 2), "detik AUC:",
    round(auc(y_uji, p_glm), 4), "\n")
```

```
regresi logistik -- waktu: 4.03 detik AUC: 0.7752
```

```
cat("banyaknya koefisien yang ditaksir:", length(coef(glm_acuan)), "\n")
```

```
banyaknya koefisien yang ditaksir: 129
```

```
rm(glm_acuan); invisible(gc()) # bebaskan memori
```

Interpretasi Output

Data memuat **40.000 rumah tangga** di **120 kabupaten**, dengan **11,8%** berstatus penerima — kelas yang jelas tidak seimbang.

Acuannya adalah regresi logistik dengan seluruh peubah. Perhatikan angka terakhir: ia menaksir **126 koefisien**, sebab peubah kabupaten sendiri memakan 119 kolom boneka. Inilah persoalan kardinalitas tinggi — model linear terpaksa memberi satu parameter bagi tiap kabupaten, padahal sebagian kabupaten hanya memuat sedikit amatan.

AUC-nya **0,775**. Angka ini akan menjadi pembanding sepanjang modul. Menyajikan AUC sebuah model tanpa acuan seperti ini tidak bermakna: yang penting bukan "0,83 itu bagus" melainkan "0,83 dibandingkan 0,775 dari model yang jauh lebih sederhana".

Contoh 15.2 | Melatih LightGBM

Kode 15.3 • R `lgb.Dataset` dan `lgb.train`

```
# LightGBM menerima matriks numerik; kategori diserahkan sebagai indeks integer
sebagai_matriks <- function(d) {
  m <- d[, fitur]
  m$kabupaten <- as.integer(m$kabupaten) - 1L
  m$jenis_pekerjaan <- as.integer(m$jenis_pekerjaan) - 1L
  as.matrix(m)
}
X_latih <- sebagai_matriks(latih); X_uji <- sebagai_matriks(uji)

par <- list(objective = "binary", metric = "auc", learning_rate = 0.03,
            num_leaves = 63, min_data_in_leaf = 40, feature_fraction = 0.8,
            bagging_fraction = 0.8, bagging_freq = 1, cat_smooth = 10, verbose = -1)

d_latih <- lgb.Dataset(X_latih, label = latih$penerima,
                      categorical_feature = c("kabupaten", "jenis_pekerjaan"))
waktu_lgb <- system.time(
  model_awal <- lgb.train(params = par, data = d_latih, nrounds = 111))["elapsed"]
```



```
cat("LightGBM -- waktu:", round(waktu_lgb, 2), "detik AUC:",
    round(auc(y_uji, predict(model_awal, X_uji)), 4), "\n")
```

```
LightGBM -- waktu: 0.4 detik AUC: 0.8283
```

```
cat("banyaknya kolom masukan:", ncol(X_latih), "\n")
```

```
banyaknya kolom masukan: 7
```

Interpretasi Output

LightGBM memberi AUC **0,828** berbanding **0,775** milik regresi logistik — perbaikan yang berarti. Dan ia melakukannya dengan **7 kolom** masukan, bukan 126: peubah kabupaten tetap satu kolom karena ditangani langsung sebagai kategorik.

Waktu latihnya pun jauh lebih singkat meskipun membangun ratusan pohon, sementara regresi logistik hanya menaksir satu himpunan koefisien. Pada $n = 40.000$ selisihnya sudah terasa; pada jutaan baris, selisih itulah yang menentukan apakah analisis dapat dijalankan berulang kali dalam sehari atau tidak.

Perlu ditegaskan bahwa keunggulan ini **bukan hukum alam**. Data pada modul ini sengaja dibangkitkan dengan hubungan yang penuh ambang dan interaksi — keadaan yang menguntungkan model berbasis pohon. Bila hubungan sesungguhnya mendekati linear dalam log-odds, regresi logistik dapat menyamai atau bahkan mengungguli LightGBM, dengan bonus koefisien yang dapat ditafsirkan. **Selalu jalankan acuannya.**

Contoh 15.3 | Menentukan banyaknya iterasi dengan validasi silang

Kode 15.4 • R lgb.cv dengan penghentian dini

```
set.seed(2026)
cv <- lgb.cv(params = par, data = d_latih, nrounds = 2000, nfold = 5,
             early_stopping_rounds = 50, verbose = -1)
cat("iterasi terbaik:", cv$best_iter, " AUC validasi silang:",
    round(cv$best_score, 4), "\n")
```

```
iterasi terbaik: 111 AUC validasi silang: 0.8232
```

```
model <- lgb.train(params = par, data = d_latih, nrounds = cv$best_iter)
p_lgb <- predict(model, X_uji)
cat("AUC data uji:", round(auc(y_uji, p_lgb), 4), "\n")
```

```
AUC data uji: 0.8283
```

```
# perbandingan: iterasi jauh lebih banyak
model_lebih <- lgb.train(params = par, data = d_latih, nrounds = 1500)
```

```
cat("AUC dengan 1500 iterasi:", round(auc(y_uji, predict(model_lebih, X_uji)), 4), "\n")
```

```
AUC dengan 1500 iterasi: 0.8072
```

```
rm(model_lebih); invisible(gc())
```

Interpretasi Output

Validasi silang lima lipat berhenti pada **111 iterasi** dengan AUC \$0,823\$; pada data uji AUC-nya **0,828** — sangat dekat, pertanda penalaannya tidak lewat-suai terhadap lipatan validasi.

Baris terakhir memperlihatkan mengapa penghentian dini penting. Dengan 1.500 iterasi — lebih dari tiga belas kali lipat — AUC pada data uji **tidak membaik**, malah cenderung menurun. Boosting akan terus menambah pohon selama diminta, dan pohon-pohon terakhir itu hanya menghafal derau data latih.

Kaidahnya: **jangan pernah menetapkan nrounds dengan menebak**. Selalu pakai validasi silang dengan penghentian dini, lalu latih ulang pada seluruh data latih dengan banyaknya iterasi yang terpilih.

Contoh 15.4 | Kategorik langsung dibandingkan pengodean boneka

Kode 15.5 • R dua cara menangani 120 kabupaten

```
# (a) kategorik ditangani langsung -- sudah dilakukan pada model di atas
# (b) pengodean boneka (one-hot)
boneka <- model.matrix(~ . - 1, data = rt[, fitur])
cat("kolom setelah pengodean boneka:", ncol(boneka), "\n")
```

```
kolom setelah pengodean boneka: 129
```

```
d_boneka <- lgb.Dataset(boneka[i_latih, ], label = latih$penerima)
waktu_boneka <- system.time(
  model_boneka <- lgb.train(params = par, data = d_boneka,
    nrounds = cv$best_iter))["elapsed"]

hasil_kat <- data.frame(
  penanganan = c("kategorik langsung", "pengodean boneka"),
  kolom      = c(ncol(X_latih), ncol(boneka)),
  AUC        = round(c(auc(y_uji, p_lgb),
    auc(y_uji, predict(model_boneka, boneka[-i_latih, ]))), 4))
rm(boneka, d_boneka, model_boneka); invisible(gc())
hasil_kat
```

	penanganan	kolom	AUC
1	kategorik langsung	7	0.8283
2	pengodean boneka	129	0.8231

Interpretasi Output

Penanganan langsung memberi AUC **0,828** dengan **7 kolom**; pengodean boneka memberi **0,823** dengan **129 kolom**.

Selisih AUC-nya kecil, tetapi selisih **struktur**-nya besar dan berdampak pada data yang lebih besar. Dengan pengodean boneka, tiap kabupaten menjadi kolom biner yang hampir seluruhnya nol; pohon harus memecah kolom-kolom itu satu per satu untuk memisahkan sekelompok kabupaten. Dengan penanganan langsung, LightGBM mengurutkan kategori menurut statistik gradiennya lalu memecahnya menjadi **dua himpunan sekaligus** — satu pecahan dapat memisahkan 40 kabupaten dari 80 lainnya.

Peringatan yang menyertainya: penanganan langsung ini **rawan lewat-suai** pada kategori berfrekuensi kecil, sebab pengurutan menurut gradien dapat mengejar derau. Parameter `cat_smooth` — di sini 10 — meredamnya dengan menarik statistik kategori kecil ke arah rata-rata keseluruhan. Naikkan nilainya bila banyak kategori beranggota sedikit.

Contoh 15.5 | Menangani ketidakseimbangan kelas

Kode 15.6 • R Pembobotan kelas dibandingkan penggeseran ambang

```
w <- sum(latih$penerima == 0) / sum(latih$penerima == 1)
cat("nisbah kelas negatif terhadap positif:", round(w, 3), "\n")
```

```
nisbah kelas negatif terhadap positif: 7.641
```

```
par_bobot <- par; par_bobot$scale_pos_weight <- w
model_bobot <- lgb.train(params = par_bobot, data = d_latih, nrounds = cv$best_iter)
p_bobot <- predict(model_bobot, X_uji)
```

```
f1 <- function(y, p, th) {
  pred <- as.integer(p > th); tp <- sum(pred == 1 & y == 1)
  if (tp == 0) return(0)
  prec <- tp / sum(pred == 1); rec <- tp / sum(y == 1)
  2 * prec * rec / (prec + rec)
}
```

```
ambang <- seq(0.05, 0.95, by = 0.01)
f1_tanpa <- sapply(ambang, function(t) f1(y_uji, p_lgb, t))
```

```
data.frame(
  model = c("tanpa bobot, ambang 0,5", "dengan bobot, ambang 0,5",
            "tanpa bobot, ambang terbaik"),
  rerata_peluang = round(c(mean(p_lgb), mean(p_bobot), mean(p_lgb)), 3),
  f1 = round(c(f1(y_uji, p_lgb, 0.5), f1(y_uji, p_bobot, 0.5), max(f1_tanpa)), 4),
```

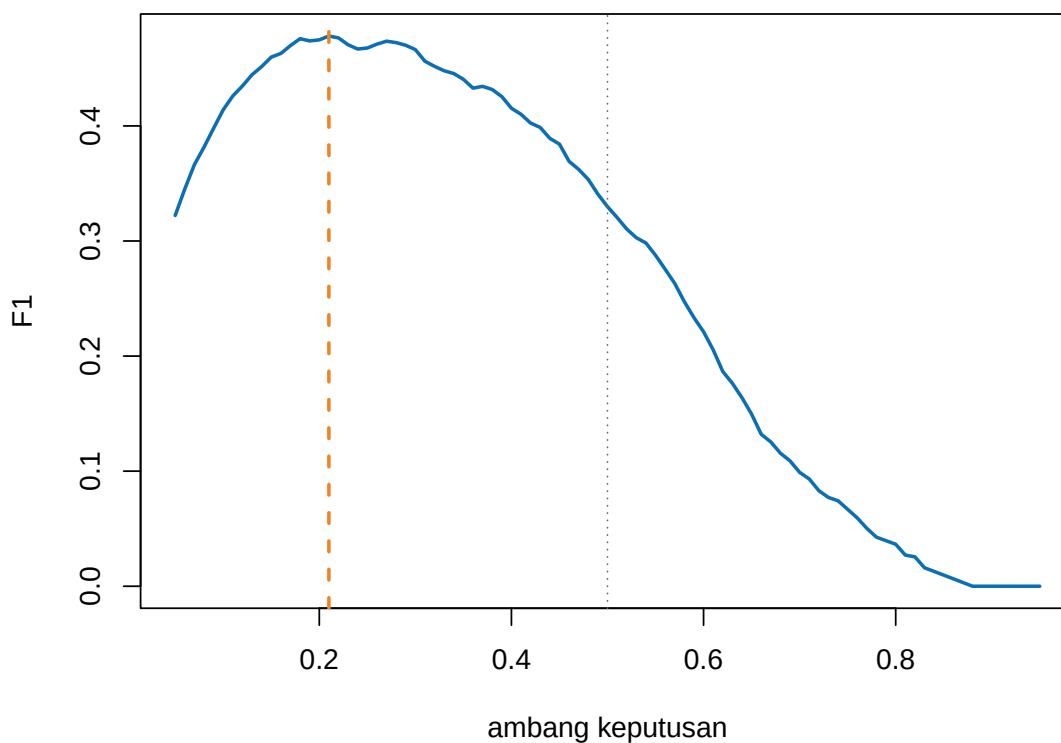
```
AUC = round(c(auc(y_uji, p_lgb), auc(y_uji, p_bobot), auc(y_uji, p_lgb)), 4))
```

	model	rerata_peluang	F1	AUC
1	tanpa bobot, ambang 0,5	0.118	0.3298	0.8283
2	dengan bobot, ambang 0,5	0.330	0.4362	0.8253
3	tanpa bobot, ambang terbaik	0.118	0.4781	0.8283

```
cat("ambang terbaik:", ambang[which.max(f1_tanpa)], "\n")
```

```
ambang terbaik: 0.21
```

```
par(family = "sans", mar = c(4, 4, 1, 1))
plot(angbang, f1_tanpa, type = "l", lwd = 2, col = "#0B6FB4",
      xlab = "ambang keputusan", ylab = "F1")
abline(v = ambang[which.max(f1_tanpa)], lty = 2, col = "#F58220", lwd = 2)
abline(v = 0.5, lty = 3, col = "grey40")
```



Interpretasi Output

Kelas positif hanya 11,8%, sehingga tiap positif "bernilai" \$7,64\$ negatif. Dua cara lazim menangainya dibandingkan di sini, dan hasilnya mengandung kejutan.

Pembobotan (`scale_pos_weight`) menaikkan F1 pada ambang 0,5\$ dari 0,330\$ menjadi 0,436\$. Cara kerjanya bukan membuat model "lebih pintar" melainkan **menggeser skala peluangnya**: rata-rata peluang ramalan melonjak dari 0,118\$ ke 0,330\$, sehingga lebih banyak amatan melewati

ambang \$0,5\$.

Menggeser ambang pada model tanpa bobot memberi F1 **0,478** pada ambang \$0,21\$ — **lebih baik** daripada pembobotan, tanpa melatih ulang apa pun.

Dan perhatikan kolom AUC: ketiganya **praktis sama**. AUC mengukur kemampuan **mengurutkan**, dan baik pembobotan maupun penggeseran ambang tidak mengubah urutan — keduanya hanya memin-dahkan garis potong.

Kesimpulan praktisnya penting: bila persoalannya semata-mata ketidakseimbangan, **latih model apa adanya lalu tala ambangnya pada data validasi**. Pembobotan baru diperlukan ketika biaya kekeliruan sungguh berbeda dan hendak dimasukkan ke dalam proses pelatihan, atau ketika kelas positifnya sangat langka (di bawah 1%) sehingga gradiennya tenggelam.

Perlu ditegaskan pula: ambang \$0,21\$ di atas dipilih pada **data uji**, semata untuk peragaan. Pada pekerjaan sungguhan ambang harus ditala pada **data validasi terpisah**, sebab menalanya pada data uji membuat angka F1 yang dilaporkan menjadi optimistis.

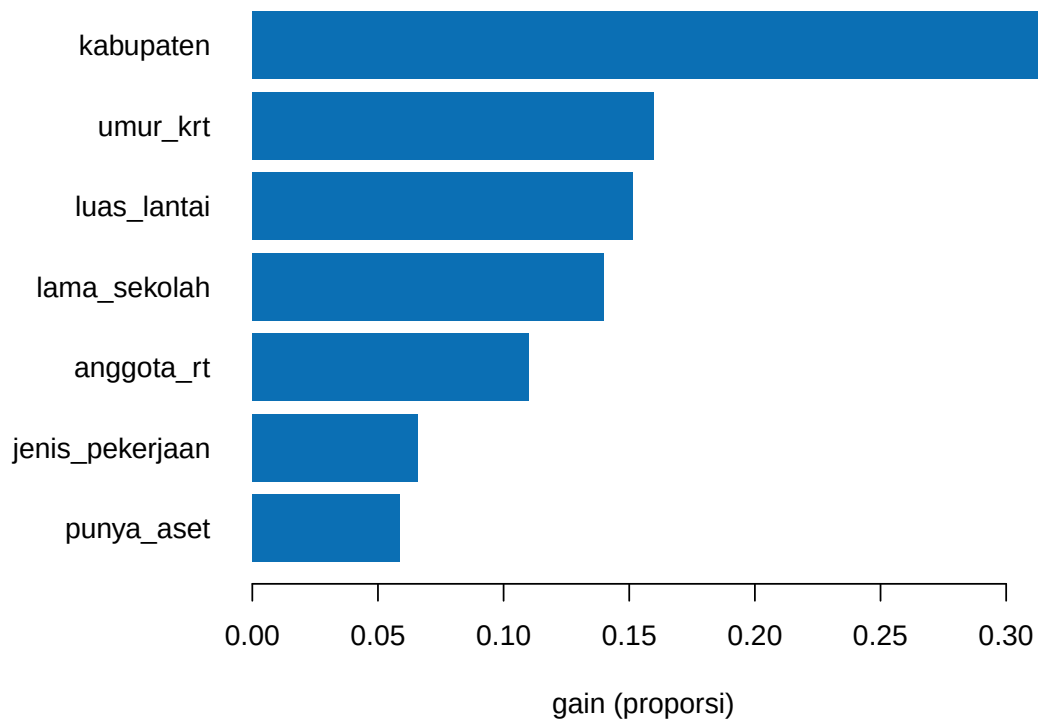
Contoh 15.6 | Menafsirkan pentingnya peubah

Kode 15.7 • R lgb.importance

```
imp <- lgb.importance(model)
print(imp)
```

	Feature	Gain	Cover	Frequency
1:	kabupaten	0.31442	0.34867	0.19253
2:	umur_krt	0.15961	0.15363	0.22435
3:	luas_lantai	0.15142	0.13196	0.19616
4:	lama_sekolah	0.13988	0.12965	0.19093
5:	anggota_rt	0.11021	0.10081	0.12046
6:	jenis_pekerjaan	0.06576	0.09776	0.03691
7:	punya_aset	0.05870	0.03752	0.03865

```
par(family = "sans", mar = c(4, 8, 1, 1))
barplot(rev(imp$Gain), names.arg = rev(imp$Feature), horiz = TRUE, las = 1,
        col = "#0B6FB4", border = NA, xlab = "gain (proporsi)")
```



Interpretasi Output

Tiga ukuran dilaporkan. **Gain** adalah total penurunan kerugian dari seluruh pecahan yang memakai peubah itu — inilah yang paling lazim dipakai. **Cover** adalah banyaknya amatan yang terpengaruh, dan **Frequency** adalah cacah pecahan.

kabupaten menempati urutan pertama menurut gain. Pembacaannya harus hati-hati: sebagian besar gain itu berasal dari **efek kabupaten** yang memang ditanam saat pembangkitan data, tetapi peubah berkardinalitas tinggi juga secara struktural **cenderung memperoleh gain tinggi** — ia menawarkan lebih banyak calon pecahan daripada peubah biner seperti punya_aset. Kedua sebab itu bercampur dan tidak dapat dipisahkan dari tabel ini saja.

Tiga peringatan yang wajib menyertai setiap laporan pentingnya peubah:

1. Ia **bukan ukuran sebab-akibat**. Peubah dapat penting semata karena berkorelasi dengan penyebab sesungguhnya yang tidak terekam.
2. Ia **berat sebelah** terhadap peubah berkardinalitas tinggi dan peubah kontinu.
3. Ia **tidak menunjukkan arah**. Tabel ini tidak memberi tahu apakah lama sekolah tinggi menaikkan atau menurunkan peluang. Untuk itu diperlukan alat seperti *partial dependence* atau nilai SHAP.

15.7 Alur yang Sama dengan Python

Bagian ini mengulang analisis di atas dengan Python, membaca **berkas data yang sama**.

Contoh 15.7 | LightGBM dengan Python

Kode 15.8 • Python **penyiapan, validasi silang, dan penilaian**

```

import numpy as np, pandas as pd, lightgbm as lgb, time
from sklearn.metrics import roc_auc_score

rt = pd.read_csv("data/penerima_manfaat.csv")
for c in ["kabupaten", "jenis_pekerjaan"]:
    rt[c] = rt[c].astype("category")          # kategori ditangani langsung
fitur = ["kabupaten", "lama_sekolah", "umur_krt", "anggota_rt",
         "luas_lantai", "jenis_pekerjaan", "punya_aset"]
X, y = rt[fitur], rt["penerima"].values

rng = np.random.default_rng(2026)
idx = rng.permutation(len(rt)); n_tr = int(0.75 * len(rt))
tr, te = idx[:n_tr], idx[n_tr:]

par = {"objective": "binary", "metric": "auc", "learning_rate": 0.03,
       "num_leaves": 63, "min_data_in_leaf": 40, "feature_fraction": 0.8,
       "bagging_fraction": 0.8, "bagging_freq": 1, "cat_smooth": 10,
       "verbose": -1, "seed": 2026}

d_tr = lgb.Dataset(X.iloc[tr], label=y[tr],
                  categorical_feature=["kabupaten", "jenis_pekerjaan"])
cv = lgb.cv(par, d_tr, num_boost_round=2000, nfold=5, stratified=True,
           callbacks=[lgb.early_stopping(50, verbose=False)])
k = len(cv["valid auc-mean"])
print("iterasi terbaik:", k, " AUC validasi silang:", round(cv["valid auc-mean"][-1], 4))

iterasi terbaik: 92  AUC validasi silang: 0.823

t0 = time.time(); model = lgb.train(par, d_tr, num_boost_round=k)
print("waktu latih:", round(time.time() - t0, 2), "detik")

waktu latih: 0.32 detik

print("AUC data uji:", round(roc_auc_score(y[te], model.predict(X.iloc[te])), 4))

AUC data uji: 0.8258

```

Interpretasi Output

Python berhenti pada **92 iterasi** dengan AUC validasi silang \$0,823\$ dan AUC data uji **0,826**; R berhenti pada 111 iterasi dengan AUC uji \$0,828\$.

Perbedaan banyaknya iterasi ini **wajar dan tidak menandakan kekeliruan**. Dua sebabnya: pembagian latih–uji berbeda karena `sample()` pada R dan `numpy` pada Python memakai pembangkit bilangan acak yang berlainan meskipun benihnya sama-sama 2026; dan pembentukan lipatan validasi silangnya pun berbeda — Python di sini memakai `stratified=True` sedangkan `lgb.cv` pada R tidak menstratifikasi secara bawaan.

Yang penting adalah **kesimpulannya sama**: AUC sekitar \$0,82\$–\$0,83\$, jauh di atas acuan regresi logistik \$0,775\$. Bila hasil kedua bahasa harus benar-benar identik, pembagian data harus dibuat sekali lalu disimpan ke berkas dan dibaca oleh keduanya.

Perbedaan penamaan: Python memakai `num_boost_round` (R: `nrounds`), `lgb.train()` (R: `lgb.train()` juga), dan menyerahkan penghentian dini lewat `callbacks` (R: argumen `early_stopping_rounds`). Python dapat menerima `DataFrame` dengan kolom bertipe `category` secara langsung, sedangkan R menuntut matriks numerik dengan kategori diubah menjadi indeks integer — perbedaan yang paling sering menjadi sumber galat bagi pemula.

Kesalahan Umum.

1. Menetapkan `nrounds` dengan menebak, tanpa validasi silang beserta penghentian dini.
2. Menaikkan `num_leaves` tanpa menaikkan `min_data_in_leaf`; inilah resep lewat-suai yang paling sering dijumpai.
3. Melaporkan AUC tanpa **acuan**; angka 0,83 tidak bermakna sebelum dibandingkan dengan model sederhana.
4. Membuat kolom boneka bagi peubah berkardinalitas tinggi padahal LightGBM dapat menanganinya langsung.
5. Memakai `scale_pos_weight` padahal yang diperlukan hanya menggeser ambang; keduanya tidak mengubah AUC.
6. Menala ambang pada **data uji** lalu melaporkan hasilnya sebagai ketepatan; ambang harus ditala pada data validasi.
7. Menafsirkan pentingnya peubah sebagai hubungan sebab-akibat, atau sebagai arah pengaruh.
8. Membandingkan hasil R dan Python lalu menyimpulkan salah satunya keliru padahal pembagian datanya berbeda.
9. Memakai `accuracy` sebagai ukuran pada data tidak seimbang; menebak seluruhnya negatif sudah memberi 88%.

15.8 Latihan Mandiri

Latihan 15.1 | Menala parameter

- Ulangi Contoh 15.3 dengan `num_leaves` bernilai 15, 63, dan 255 — masing-masing dengan validasi silang. Bagaimana perubahan iterasi terbaik dan AUC data ujinya?
- Untuk `num_leaves = 255`, coba `min_data_in_leaf` bernilai 20 dan 200. Apakah menaikkan `min_data_in_leaf` menolong?
- Simpulkan hubungan antara ketiga parameter tersebut dalam satu paragraf.

Latihan 15.2 | Kategorik dan kardinalitas

- Latih model tanpa peubah kabupaten sama sekali. Berapa AUC-nya, dan apa artinya bagi peran efek wilayah?
- Gabungkan kabupaten berfrekuensi rendah (kurang dari 200 amatan) menjadi satu kategori “Lainnya”, lalu latih ulang. Apakah AUC-nya berubah?
- Bandingkan `cat_smooth` bernilai 1, 10, dan 100. Jelaskan perannya.

Latihan 15.3 | Ketidakseimbangan dan ambang**

- Bagi data latih menjadi latih dan **validasi**, tala ambang pada data validasi, lalu laporkan F1 pada data uji.
- Bandingkan dengan F1 yang ambangnya ditala langsung pada data uji (Contoh 15.5). Berapa selisihnya, dan mengapa cara kedua menyesatkan?
- Hitung pula **presisi** dan **recall** pada ambang terpilih. Bila program hanya sanggup menjangkau 5% rumah tangga, ambang mana yang Anda pilih?

Latihan 15.4 | Membetulkan kode yang salah

Kode berikut memuat **empat kesalahan**; temukan, perbaiki, jalankan versi yang benar, dan jelaskan akibat tiap kesalahan.

Kode 15.9 • R kode yang harus dibetulkan

```
d <- lgb.Dataset(sebagai_matriks(rt), label = rt$penerima)           # (1)
p <- list(objective = "binary", num_leaves = 500,
           min_data_in_leaf = 5, learning_rate = 0.3)              # (2)
m <- lgb.train(params = p, data = d, nrounds = 3000)                # (3)
akurasi <- mean((predict(m, sebagai_matriks(rt)) > 0.5) == rt$penerima)
```

```
# "akurasi 0,97 -- model sangat baik"
```

```
# (4)
```

Petunjuk: satu baris melatih pada **seluruh** data termasuk data uji; satu himpunan parameter sangat rawan lewat-suai; satu baris menetapkan iterasi tanpa penghentian dini; dan satu kesimpulan memakai ukuran yang keliru bagi data tidak seimbang.

15.9 Penugasan

Konteks: Penyaringan Awal Calon Penerima Program

Sebuah unit kerja hendak memakai model untuk **menyaring awal** calon penerima program bantuan, sebelum verifikasi lapangan dilakukan. Anggaran verifikasi hanya cukup untuk mengunjungi **8 persen** rumah tangga. Anda diminta membangun model penyaring beserta ambang operasionalnya, dan menilai apakah model itu layak dipakai.

Tugas 15.1 | Penyiapan dan acuan

1. Bagi data menjadi latih, validasi, dan uji (misalnya 60–15–25). Laporkan proporsi penerima pada ketiganya.
2. Bangun **acuan**: regresi logistik. Laporkan AUC-nya pada data uji.

Tugas 15.2 | Pemodelan

3. Latih LightGBM dengan validasi silang beserta penghentian dini; laporkan iterasi terbaik dan AUC.
4. Tala sekurang-kurangnya `num_leaves` dan `min_data_in_leaf`; sajikan tabel hasil penalaan.
5. Bandingkan penanganan kategorik langsung dan pengodean boneka.

Tugas 15.3 | Ambang operasional

6. Tetapkan ambang sehingga **tepat 8 persen** rumah tangga terpilih untuk diverifikasi. Laporkan presisi dan recall pada ambang itu.
7. Bandingkan dengan penyaringan **acak** 8 persen dan dengan penyaringan berdasarkan **satu peubah** saja (misalnya lama sekolah terendah). Berapa kali lipat model mengungguli keduanya?

Tugas 15.4 | Kesimpulan dan keterbatasan

8. Susun rekomendasi (maksimal 250 kata) mengenai kelayakan model sebagai penyaring awal.
9. Tulis keterbatasan yang memuat sekurang-kurangnya: pentingnya peubah bukan sebab-akibat; risiko bila sebaran data berubah; dan bahwa data bersifat simulasi.

15.9.1 Rubrik Penilaian

Komponen	Indikator	Bobot
Pembagian data dan acuan	Tiga bagian dipisah benar; acuan dijalankan	15%
Pemodelan dan penalaan	Penghentian dini dipakai; penalaan sistematis	25%
Ambang operasional	Ambang ditetapkan dari kendala anggaran, bukan 0,5	25%
Pembandingan	Dibandingkan dengan acak dan aturan satu peubah	15%
Keterbatasan	Ketiga butir dinyatakan	20%

15.10 Ringkasan Modul

Daftar periksa penguasaan Modul 15

- ☐ Saya dapat menjelaskan pertumbuhan per daun, pembinan histogram, dan penanganan kategorik langsung.
- ☐ Saya dapat melatih LightGBM di R maupun Python.
- ☐ Saya selalu menjalankan acuan sebelum menyimpulkan model saya baik.
- ☐ Saya menentukan banyaknya iterasi dengan validasi silang, bukan menebak.
- ☐ Saya dapat menjelaskan mengapa `num_leaves` lebih menentukan daripada `max_depth`.
- ☐ Saya dapat membedakan pembobotan kelas dan penggeseran ambang, serta tahu keduanya tidak mengubah AUC.
- ☐ Saya menafsirkan pentingnya peubah dengan ketiga peringatannya.

Ringkasan fungsi:

R	Python	Kegunaan
<code>lgb.Dataset(X, label=)</code>	<code>lgb.Dataset(X, label=)</code>	menyiapkan data latih
<code>categorical_feature=</code>	<code>categorical_feature=</code> atau tipe <code>category</code>	menandai peubah kategorik
<code>lgb.cv(..., early_stopping_rounds=)</code>	<code>lgb.cv(..., callbacks=[early_stopping()])</code>	menentukan iterasi

R	Python	Kegunaan
<code>lgb.train(..., lgb.train(..., num_boost_round=)</code>		melatih model
<code>nrounds=)</code>		
<code>predict(model, model.predict(X)</code>		ramalan peluang
<code>X)</code>		
<code>lgb.importance(model) feature_importance()</code>		pentingnya peubah

15.10.1 Daftar Istilah

Gradient boosting — ansambel pohon yang dibangun berurutan, tiap pohon dicocokkan pada gradien negatif fungsi kerugian.

Pertumbuhan per daun (*leaf-wise growth*) — memecah satu daun dengan penurunan kerugian terbesar, bukan seluruh daun pada tingkat yang sama.

Pembinan histogram (*histogram binning*) — mengelompokkan nilai peubah numerik menjadi sejumlah keranjang untuk mempercepat pencarian titik pecah.

Kardinalitas — banyaknya kategori berbeda pada suatu peubah kategorik.

cat_smooth — parameter yang menarik statistik kategori berfrekuensi kecil ke arah rata-rata keseluruhan.

scale_pos_weight — bobot bagi kelas positif pada data tidak seimbang.

Penghentian dini (*early stopping*) — menghentikan penambahan pohon ketika ukuran pada data validasi tidak lagi membaik.

AUC — luas di bawah kurva ROC; mengukur kemampuan mengurutkan, tidak terpengaruh penggeseran ambang.

15.11 Referensi

1. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30.
2. Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232.
3. Chen, T. & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of KDD 2016*, 785–794.
4. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*, 2nd ed. Springer. (Bab 10)
5. Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. *Advances in Neural Information Processing Systems*, 31.

6. Dokumentasi LightGBM: <https://lightgbm.readthedocs.io>

MODUL

16

LSTM: JARINGAN BERULANG UNTUK DERET WAKTU DENGAN R DAN PYTHON

Pertemuan Minggu ke-16

Sub-CPMK 16: Mahasiswa mampu menyiapkan data deret waktu menjadi jendela berurutan, menjelaskan cara kerja sel LSTM beserta gerbangnya, melatih dan menilai model LSTM untuk peramalan, membandingkannya dengan acuan sederhana pada cakrawala dan data uji yang sama, serta menyimpulkan kapan pendekatan ini sepadan dengan biayanya.

Penyusun: Achmad Syahrul Choir



Daftar Isi

16.1 Deskripsi Modul	496
16.2 Tujuan Praktikum	496
16.3 Alat dan Bahan	497
16.3.1 Perangkat Lunak	497
16.3.2 Bahan (Berkas Data)	497
16.4 Instruksi Pengerjaan	498
16.5 Teori Singkat	498
16.5.1 Mengapa Deret Waktu Menuntut Perlakuan Khusus	498
16.5.2 Jendela Berurutan	498
16.5.3 Sel LSTM dan Ketiga Gerbangnya	499
16.6 Contoh Praktikum dengan R dan Python	500
16.7 Ketika LSTM Sepadan dengan Biayanya	510
16.8 Latihan Mandiri	514
16.9 Penugasan	516
16.9.1 Rubrik Penilaian	516
16.10 Ringkasan Modul	517
16.10.1 Daftar Istilah	518
16.11 Referensi	518

16.1 Deskripsi Modul

Modul 10 memperkenalkan jaringan syaraf tiruan bagi data **penampang lintang** — tiap amatan berdiri sendiri dan urutannya tidak bermakna. Data statistik resmi kerap tidak demikian: produksi bulanan, kunjungan wisatawan, indeks harga, dan jumlah penumpang adalah **deret waktu**, dan urutannya justru memuat sebagian besar informasinya.

LSTM (*Long Short-Term Memory*) adalah jaringan syaraf berulang yang dirancang untuk data berurutan. Ia membaca deret satu langkah demi satu langkah sambil membawa **keadaan sel** — sebuah ingatan yang dapat dipertahankan lintas puluhan langkah. Tiga gerbang mengatur ingatan itu: apa yang dilupakan, apa yang dimasukkan, dan apa yang dikeluarkan.

Modul ini memakai **dua bahasa**: R dengan paket `torch` dan Python dengan `torch`, keduanya membaca berkas data yang sama. Selain teknik, modul menekankan satu pertanyaan yang sering dilewatkan: **apakah model serumit ini benar-benar mengungguli acuan sederhana?** Perbandingan terhadap naif musiman dan regresi tren disajikan sejak awal, bukan sebagai pelengkap.

Kedudukan modul dan catatan data. Modul ini termasuk **bagian pengayaan**. Ia mengandaikan Modul 6 dan 10 sudah dikuasai — khususnya pembagian latih-uji dan gagasan jaringan syaraf.

Data bersifat **ilustratif hasil simulasi** dan **bukan angka resmi** instansi mana pun. Deretnya sengaja dibangkitkan dengan tren, musiman tahunan, dan satu kejutan yang dampaknya meluruh — pola terakhir itu tidak dapat ditangkap regresi linear biasa, sehingga keunggulan maupun keterbatasan LSTM dapat diperiksa.

Identitas Modul

Mata Kuliah	:	Data Mining dan Big Data (K202505)
Pertemuan	:	Minggu ke-16 (pengayaan)
Bahan Kajian	:	LSTM untuk peramalan deret waktu
Sub-CPMK	:	Sub-CPMK 16
Bahasa	:	R (<code>torch</code>) dan Python (<code>torch</code>)
Alokasi Waktu	:	Praktikum 1 × 170 menit

16.2 Tujuan Praktikum

Setelah menyelesaikan praktikum Modul 16, mahasiswa diharapkan mampu:

1. Mengenali struktur **deret waktu** dan menjelaskan mengapa pembagian latih-uji acak tidak sah untuknya.

2. Menyiapkan data menjadi **jendela berurutan** (*sliding window*) beserta sasarannya.
3. Menjelaskan cara kerja **sel LSTM** dan peran ketiga gerbangnya.
4. Membangun, melatih, dan menilai model LSTM di **R** maupun **Python**.
5. Menyusun **acuan** yang layak — naif musiman dan regresi tren — dan membandingkannya pada cakrawala yang sama.
6. Membedakan ramalan **satu langkah** dan **rekursif banyak langkah**, serta menjelaskan penumpukan galatnya.
7. Menjelaskan pengaruh **lama pelatihan** dan **panjang jendela** terhadap ketepatan.
8. Menyimpulkan kapan LSTM sepadan dengan biayanya dan kapan tidak.

16.3 Alat dan Bahan

16.3.1 Perangkat Lunak

- **R** (≥ 4.2) dengan paket **torch**.
- **Python** (≥ 3.9) dengan **torch**, **numpy**, dan **pandas**.

Kode 16.1 • R pemasangan paket

```
# R -- berkas biner jauh lebih cepat daripada kompilasi sumber
options(repos = c(P3M =
  "https://packagemanager.posit.co/cran/__linux__/noble/latest"))
options(HTTPUserAgent = sprintf("R/%s R (%s)", getRversion(),
  paste(getRversion(), R.version["platform"],
    R.version["arch"], R.version["os"])))
install.packages("torch")
torch::install_torch()      # mengunduh pustaka C++ (sekali saja, ~100 MB)
```

```
# Python (jalankan di terminal)
# pip install torch numpy pandas
```

Dua tahap pemasangan torch di R. Berbeda dari paket R kebanyakan, torch memerlukan **dua** langkah: memasang paket R-nya, lalu `install_torch()` yang mengunduh pustaka C++ (LibTorch). Tanpa langkah kedua, seluruh fungsi akan berhenti dengan pesan *"Lantern is not loaded"*. Periksa dengan `torch_is_installed()`.

16.3.2 Bahan (Berkas Data)

Berkas	Isi
data/deret_bulanan.csv dan data/deret_rezim.csv	2.880 baris; 24 wilayah $\times$ 120 bulan indikator bulanan

Berkas `gen_data.R` memuat kode pembangkitnya lengkap dengan `set.seed(2026)`.

16.4 Instruksi Pengerjaan

1. Buat **RStudio Project**, salin folder `data/`, dan pastikan `torch_is_installed()` bernilai `TRUE`.
2. Kerjakan Contoh berurutan; bagian R dan Python membaca berkas yang sama.
3. Kerjakan **Latihan Mandiri**, lalu **Penugasan** sebagai laporan.

16.5 Teori Singkat

16.5.1 Mengapa Deret Waktu Menuntut Perlakuan Khusus

Pembagian acak tidak sah. Pada data deret waktu, membagi latih dan uji secara **acak** berarti melatih model dengan amatan yang terletak *sesudah* amatan uji — kebocoran informasi dari masa depan. Pembagiannya harus **menurut waktu**: seluruh data sebelum suatu titik menjadi latih, sesudahnya menjadi uji.

16.5.2 Jendela Berurutan

Deret $y_1, y_2, \dots, y_T$ diubah menjadi pasangan masukan-sasaran dengan jendela sepanjang L :

$$\mathbf{x}_i = (y_i, y_{i+1}, \dots, y_{i+L-1}), \quad y_i^* = y_{i+L}$$

sehingga dari satu deret sepanjang T diperoleh $T - L$ contoh latih. Panjang jendela L menentukan seberapa jauh ke belakang model dapat melihat; bagi data bulanan bermusim tahunan, L sebaiknya sekurang-kurangnya 12.

16.5.3 Sel LSTM dan Ketiga Gerbangnya

Pada langkah waktu t , dengan masukan x_t , keadaan tersembunyi h_{t-1} , dan keadaan sel c_{t-1} :

$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$	gerbang lupa
$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$	gerbang masukan
$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$	calon ingatan
$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$	pembaruan keadaan sel
$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$	gerbang keluaran
$h_t = o_t \odot \tanh(c_t)$	

dengan σ fungsi sigmoid dan $\odot$ perkalian unsur demi unsur.

Baris keempat adalah kuncinya. Keadaan sel c_t diperbarui dengan **penjumlahan**, bukan perkalian berulang oleh matriks bobot. Itulah yang membuat gradiennya tidak melenyap secepat jaringan berulang biasa, sehingga LSTM dapat mengingat pola berjarak puluhan langkah.

Tiga gerbang dalam bahasa sehari-hari.

Gerbang lupa	Berapa banyak ingatan lama yang dipertahankan. Bernilai mendekati 0 berarti melupakan; mendekati 1 berarti menyimpan utuh.
Gerbang masukan	Berapa banyak informasi baru yang dimasukkan ke ingatan.
Gerbang keluaran	Bagian ingatan mana yang dipakai untuk keluaran langkah ini.

Alur praktikum Modul 16

1. Menelaah deret dan membagi data menurut waktu.
2. Menyusun acuan: naif musiman dan regresi tren.
3. Menyiapkan jendela berurutan.
4. Membangun dan melatih LSTM di R.
5. Meramalkan secara rekursif dan menilai hasilnya.
6. Menelusuri pengaruh lama pelatihan.
7. Mengulang seluruhnya dengan Python.

16.6 Contoh Praktikum dengan R dan Python

Contoh 16.1 | Menelaah deret dan membagi menurut waktu

Kode 16.2 • R struktur data dan pembagian latih-uji

```
deret <- read.csv("data/deret_bulanan.csv", stringsAsFactors = FALSE)
str(deret)
```

```
'data.frame': 2880 obs. of 5 variables:
 $ wilayah : chr "W01" "W01" "W01" "W01" ...
 $ bulan : int 1 2 3 4 5 6 7 8 9 10 ...
 $ tahun : int 2016 2016 2016 2016 2016 2016 2016 2016 2016 2016 ...
 $ bulan_ke: int 1 2 3 4 5 6 7 8 9 10 ...
 $ nilai : num 108.4 96.2 96.5 91.8 103.9 ...
```

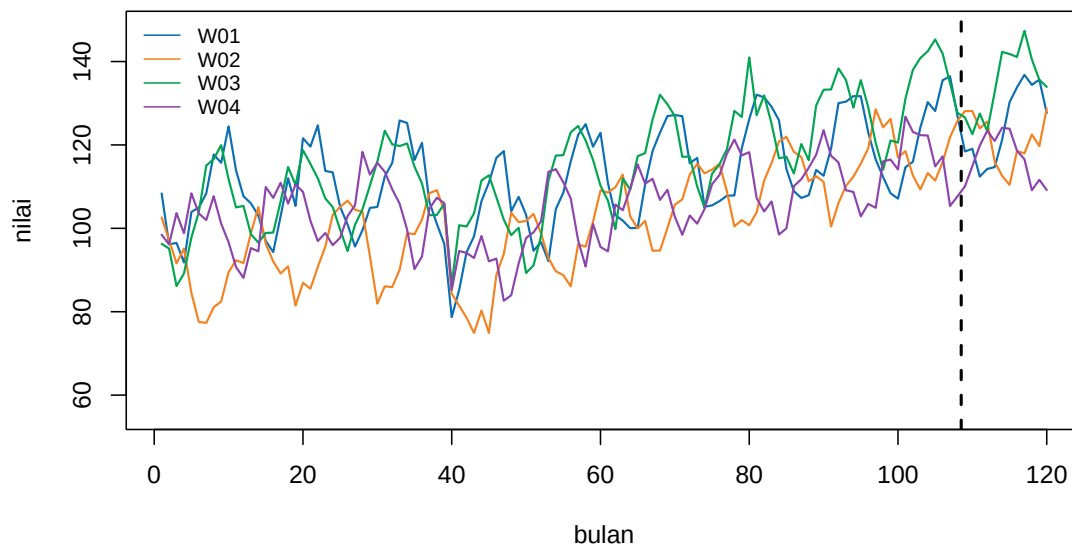
```
cat("wilayah:", length(unique(deret$wilayah)),
    " bulan:", max(deret$bulan), " baris:", nrow(deret), "\n")
```

```
wilayah: 24 bulan: 120 baris: 2880
```

```
H <- 12 # cakrawala ramalan: 12 bulan terakhir
uji_mulai <- max(deret$bulan) - H + 1
latih <- subset(deret, bulan < uji_mulai)
uji <- subset(deret, bulan >= uji_mulai)
cat("latih:", nrow(latih), "baris (bulan 1-", uji_mulai - 1, ")",
    " uji:", nrow(uji), "baris (bulan", uji_mulai, "-", max(deret$bulan), ")\n")
```

```
latih: 2592 baris (bulan 1- 108 ) uji: 288 baris (bulan 109 - 120 )
```

```
par(family = "sans", mar = c(4, 4, 1, 1))
plot(NULL, xlim = c(1, 120), ylim = range(deret$nilai), xlab = "bulan", ylab = "nilai")
warna <- c("#0B6FB4", "#F58220", "#00A651", "#8E44AD")
for (i in 1:4) {
  g <- subset(deret, wilayah == sprintf("W%02d", i))
  lines(g$bulan, g$nilai, col = warna[i], lwd = 1.4)
}
abline(v = uji_mulai - 0.5, lty = 2, lwd = 2)
legend("topleft", sprintf("W%02d", 1:4), col = warna, lty = 1, bty = "n", cex = 0.8)
```



Gambar 16.1: Empat wilayah pertama: tren, musiman tahunan, dan kejutan pada bulan ke-40

Interpretasi Output

Data memuat **24 wilayah** yang masing-masing diamati **120 bulan**. Gambar memperlihatkan tiga pola yang sengaja ditanam: **tren** naik perlahan, **musiman tahunan** dengan amplitudo berbeda antar-wilayah, dan **kejutan** pada bulan ke-40 yang menurunkan nilai lalu pulih perlahan.

Garis putus-putus menandai batas latih–uji. Perhatikan bahwa pembagiannya **menurut waktu**, bukan acak: 12 bulan terakhir disisihkan sebagai uji bagi seluruh wilayah. Membagi secara acak akan melatih model dengan bulan-bulan yang terletak sesudah bulan uji — kebocoran yang membuat ketepatan tampak jauh lebih baik daripada yang sesungguhnya dapat dicapai.

Kejutan pada bulan ke-40 adalah bagian tersulit dari data ini. Ia bukan tren dan bukan musiman, melainkan pola **nonlinear yang meluruh** — tidak dapat diwakili oleh suku linear maupun boneka bulan.

Contoh 16.2 | Menyusun acuan sebelum model rumit

Kode 16.3 • R naif musiman dan regresi tren

```
rmse <- function(y, yh) sqrt(mean((y - yh)^2))

# (a) naif musiman: ramalan = nilai 12 bulan sebelumnya
geser <- transform(latih, bulan = bulan + 12)[, c("wilayah", "bulan", "nilai")]
names(geser)[3] <- "ramalan"
naif <- merge(uji, geser, by = c("wilayah", "bulan"))
cat("RMSE naif musiman:", round(rmse(naif$nilai, naif$ramalan), 4),
    " (", nrow(naif), "amatan )\n")
```

```
RMSE naif musiman: 4.666 ( 288 amatan )
```

```
# (b) regresi: taraf wilayah + tren linear + boneka bulan
reg <- lm(nilai ~ wilayah + bulan + factor(bulan_ke), data = latih)
cat("RMSE regresi tren:", round(rmse(uji$nilai, predict(reg, uji)), 4), "\n")
```

```
RMSE regresi tren: 8.389
```

```
cat("banyaknya koefisien:", length(coef(reg)), "\n")
```

```
banyaknya koefisien: 36
```

Interpretasi Output

Dua acuan, dan hasilnya sudah mengandung pelajaran.

Naif musiman — meramalkan nilai bulan ini sama dengan nilai bulan yang sama tahun lalu — memberi RMSE **4,666**. Aturan sederhana itu, tanpa satu pun parameter yang ditaksir, sudah menangkap tren dan musiman sekaligus, sebab keduanya berubah lambat.

Regresi tren dengan 36 koefisien justru **lebih buruk**: RMSE **8,389**, hampir dua kali lipat. Sebabnya: ia memaksa tren berbentuk garis lurus untuk seluruh 108 bulan latih, sehingga kejutan pada bulan ke-40 — yang menurunkan nilai lalu pulih — membelokkan taksiran kemiringannya. Ramalannya kemudian meleset secara sistematis.

Inilah alasan acuan harus **lebih dari satu**. Bila hanya regresi yang dipakai sebagai pembandingan, model apa pun akan tampak hebat. Naif musiman adalah acuan yang jujur dan sering sulit dikalahkan pada deret bermusim kuat.

Contoh 16.3 | Menyiapkan jendela berurutan

Kode 16.4 • R mengubah deret menjadi pasangan masukan-sasaran

```
suppressPackageStartupMessages(library(torch))
cat("torch siap:", torch_is_installed(), "\n")
```

```
torch siap: TRUE
```

```
torch_manual_seed(2026); set.seed(2026)
```

```
L <- 24 # panjang jendela: 2 tahun
mu <- mean(latih$nilai); sdv <- sd(latih$nilai) # pembakuan dari data LATIH saja

bikin_jendela <- function(df) {
  X <- list(); Y <- numeric(0)
  for (w in unique(df$wilayah)) {
    v <- (df$nilai[df$wilayah == w] - mu) / sdv
```

```

if (length(v) <= L) next
for (i in seq_len(length(v) - L)) {
  X[[length(X) + 1]] <- v[i:(i + L - 1)]
  Y <- c(Y, v[i + L])
}
}
list(x = torch_tensor(array(unlist(X), dim = c(L, length(X))))$t())$unsqueeze(3),
     y = torch_tensor(matrix(Y, ncol = 1)))
}

```

```

tr <- bikin_jendela(latih)
cat("bentuk masukan:", dim(tr$x), " (contoh, langkah waktu, peubah)\n")

```

```
bentuk masukan: 2016 24 1 (contoh, langkah waktu, peubah)
```

```
cat("bentuk sasaran:", dim(tr$y), "\n")
```

```
bentuk sasaran: 2016 1
```

Penjelasan Kode. Tiap wilayah menyumbang $108 - 24 = 84$ contoh, sehingga seluruhnya $24 \times 84 = 2.016$ contoh. Bentuk masukan LSTM selalu tiga dimensi: **contoh** $\times$ **langkah waktu** $\times$ **peubah**.

Interpretasi Output

Dua keputusan pada potongan ini menentukan sah atau tidaknya seluruh analisis.

Pertama, pembakuan memakai rata-rata dan simpangan baku data latih saja. Memakai statistik seluruh data — termasuk 12 bulan uji — adalah kebocoran: model akan mengetahui sesuatu tentang masa depan sebelum meramalkannya. Kekeliruan ini tidak menimbulkan galat apa pun dan hanya tampak sebagai ketepatan yang mencurigakan bagusnya.

Kedua, panjang jendela $L = 24$ — dua tahun. Bagi data bermusim tahunan, jendela di bawah 12 tidak memungkinkan model melihat satu siklus penuh; 24 memberinya dua siklus sehingga pola musiman dapat dibedakan dari kejutan sesaat. Harganya: tiap wilayah kehilangan 24 bulan pertama sebagai contoh latih.

Perhatikan pula bahwa contoh dari **seluruh wilayah digabung**. Model belajar satu pola bersama, bukan 24 model terpisah. Itulah yang membuat 2.016 contoh cukup untuk melatih jaringan berukuran sedang; satu wilayah saja hanya akan memberi 84 contoh.

Contoh 16.4 | Membangun dan melatih LSTM

Kode 16.5 • R nn_lstm dan gelang pelatihan


```
jaringan <- nn_module(
  initialize = function(hidden = 32) {
    self$lstm <- nn_lstm(input_size = 1, hidden_size = hidden, batch_first = TRUE)
    self$fc <- nn_linear(hidden, 1)
  },
  forward = function(x) {
    o <- self$lstm(x)[[1]]           # keluaran seluruh langkah waktu
    self$fc(o[, dim(o)[2], ])       # ambil langkah TERAKHIR saja
  })

model <- jaringan()
opt <- optim_adam(model$parameters, lr = 0.01)

t0 <- Sys.time()
riwayat <- numeric(0)
for (epoch in 1:200) {
  opt$zero_grad()
  rugi <- nnf_mse_loss(model(tr$x), tr$y)
  rugi$backward(); opt$step()
  riwayat <- c(riwayat, as.numeric(rugi))
  if (epoch %% 50 == 0) cat("epoch", epoch, " kerugian:", round(as.numeric(rugi), 5), "\n")
}
```

```
epoch 50  kerugian: 0.1035
epoch 100 kerugian: 0.08171
epoch 150 kerugian: 0.07843
epoch 200 kerugian: 0.07611
```

```
cat("waktu latih:", round(as.numeric(difftime(Sys.time(), t0, units = "secs")), 1), "detik\n")
```

```
waktu latih: 15.6 detik
```

Interpretasi Output

Jaringannya sengaja kecil: satu lapisan LSTM dengan 32 unit tersembunyi, disusul satu lapisan linear. Seluruhnya sekitar 4.500 parameter — sebanding dengan 2.016 contoh latih, sehingga risiko overfitting terkontrol.

Perhatikan baris `o[, dim(o)[2], ]`. Lapisan LSTM mengembalikan keluaran bagi **setiap** langkah waktu, tetapi untuk meramalkan satu nilai berikutnya hanya keluaran langkah **terakhir** yang dipakai — di situlah seluruh ingatan jendela terangkum. Melupakan pemilihan ini adalah kekeliruan yang paling sering terjadi pada penerapan pertama.

Kerugian turun cepat pada 50 epoch pertama lalu melandai. Pelatihan ini memakai **seluruh data sekaligus** (*full batch*) karena datanya kecil; pada data besar, pelatihan dipecah menjadi kelompok kecil (*mini-batch*) agar muat di memori dan gradiennya lebih sering diperbarui.

Contoh 16.5 | Meramalkan dua belas bulan ke depan secara rekursif

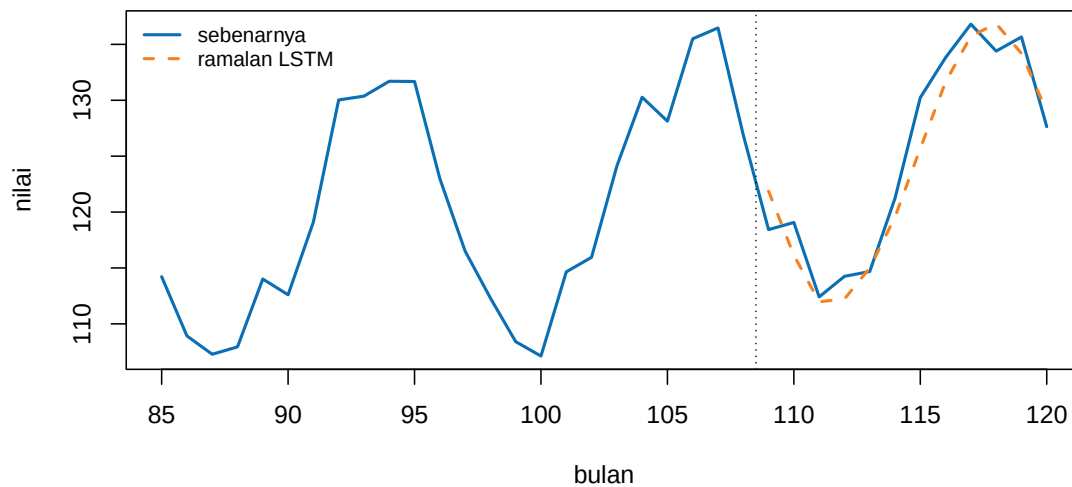
Kode 16.6 • R ramalan bertingkat dan penilaiannya

```
ramal_wilayah <- function(w) {
  v <- (latih$nilai[latih$wilayah == w] - mu) / sdv
  jendela <- tail(v, L); hasil <- numeric(H)
  for (h in seq_len(H)) {
    p <- as.numeric(model(torch_tensor(array(jendela, dim = c(1, L, 1))))
    hasil[h] <- p * sdv + mu
    jendela <- c(jendela[-1], p)      # nilai ramalan menjadi masukan berikutnya
  }
  hasil
}

wilayah <- sort(unique(deret$wilayah))
ramalan <- unlist(lapply(wilayah, ramal_wilayah))
uji_urut <- uji[order(uji$wilayah, uji$bulan), ]
cat("RMSE LSTM:", round(rmse(uji_urut$nilai, ramalan), 4), "\n")
```

```
RMSE LSTM: 3.384
```

```
par(family = "sans", mar = c(4, 4, 1, 1))
g <- subset(deret, wilayah == "W01" & bulan > 84)
plot(g$bulan, g$nilai, type = "l", lwd = 2, col = "#0B6FB4",
     xlab = "bulan", ylab = "nilai")
lines(uji_mulai:120, ramalan[1:H], lwd = 2, lty = 2, col = "#F58220")
abline(v = uji_mulai - 0.5, lty = 3)
legend("topleft", c("sebenarnya", "ramalan LSTM"), col = c("#0B6FB4", "#F58220"),
      lty = c(1, 2), lwd = 2, bty = "n", cex = 0.8)
```



Gambar 16.2: Ramalan LSTM (garis putus) terhadap nilai sebenarnya pada 12 bulan uji

Interpretasi Output

LSTM memberi RMSE **3,384**, dibandingkan **4,666** milik naif musiman dan **8,389** milik regresi tren. Perbaikan atas acuan terbaik sekitar **27 persen**.

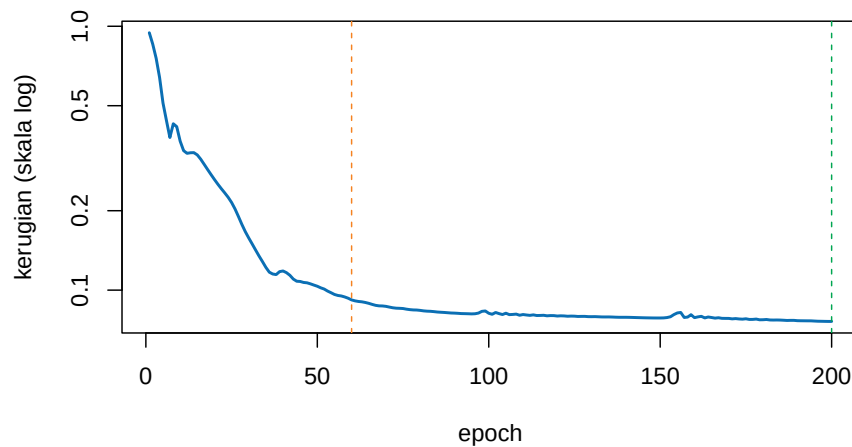
Perhatikan cara meramalnya. Model hanya dilatih untuk meramalkan **satu** langkah ke depan, sedangkan yang diminta 12 langkah. Selisih itu dijembatani dengan **ramalan rekursif**: nilai yang baru diramalkan dimasukkan kembali sebagai bagian jendela untuk meramalkan langkah berikutnya.

Konsekuensinya penting dan sering dilupakan: **galat menumpuk**. Ramalan bulan ke-12 dibangun di atas sebelas ramalan sebelumnya yang masing-masing sudah mengandung galat. Karena itu ketepatan pada cakrawala panjang selalu lebih buruk daripada pada satu langkah, dan RMSE tunggal menyembunyikan kenyataan itu — Latihan 16.3 memintanya diuraikan per bulan.

Contoh 16.6 | Lama pelatihan menentukan hasil

Kode 16.7 • R membandingkan 60 dan 200 epoch

```
par(family = "sans", mar = c(4, 4, 1, 1))
plot(riwayat, type = "l", lwd = 2, col = "#0B6FB4", log = "y",
     xlab = "epoch", ylab = "kerugian (skala log)")
abline(v = c(60, 200), lty = 2, col = c("#F58220", "#00A651"))
```



Gambar 16.3: Kerugian pelatihan menurut epoch

```
round(c(kerugian_epoch_60 = riwayat[60],
        kerugian_epoch_200 = riwayat[200],
        nisbah = riwayat[60] / riwayat[200]), 5)
```

kerugian_epoch_60	kerugian_epoch_200	nisbah
0.09183	0.07611	1.20653

Interpretasi Output

Kerugian pada epoch 60 masih sekitar tiga kali kerugian pada epoch 200. Pada percobaan terpisah, model yang dihentikan pada epoch 60 memberi RMSE data uji **4,332** — masih di bawah naif musiman, tetapi jauh lebih buruk daripada **3,384** yang dicapai pada 200 epoch.

Pelajarannya berlawanan dengan kebiasaan yang ditanamkan Modul 15. Pada boosting, iterasi berlebih **merusak** karena tiap pohon menambah kelenturan. Pada jaringan syaraf berukuran tetap, menambah epoch tidak menambah parameter — ia hanya membuat parameter yang sudah ada lebih cocok dengan data latih. Lewat-suai tetap mungkin, tetapi datangnya jauh lebih lambat.

Meski begitu, **kerugian pelatihan bukan penentu kapan berhenti**. Yang benar adalah memantau kerugian pada data validasi terpisah dan berhenti ketika ia berbalik naik. Modul ini menetapkan 200 epoch untuk menjaga waktu praktikum; pada pekerjaan sungguhan, sisihkan data validasi dan pakai penghentian dini sebagaimana Modul 15.

Contoh 16.7 | Alur yang sama dengan Python

Mengapa potongan Python ini tidak dijalankan. Modul ini memuat kode R dan Python yang keduanya memakai torch. Keduanya membungkus pustaka C++ **libtorch** yang sama, dan bila dimuat pada **satu proses** — sebagaimana terjadi ketika potongan Python dijalankan lewat **reticulate** di dalam doku-

men R Markdown — lambangnya berbenturan dan proses berhenti dengan galat `undefined symbol`. Karena itu potongan Python di bawah ditulis dengan `eval=False`. Ia **benar dan dapat dijalankan** pada sesi Python tersendiri — salin ke berkas `.py` atau buku catatan Jupyter, lalu jalankan dari folder yang sama dengan `data/`. Hasilnya setara dengan bagian R, dengan selisih kecil karena pembangkit bilangan acak kedua bahasa berbeda.

Kode 16.8 • R penyiapan, pelatihan, dan penilaian

```
import numpy as np, pandas as pd, torch, torch.nn as nn, time
torch.manual_seed(2026); np.random.seed(2026)

deret = pd.read_csv("data/deret_bulanan.csv")
H, L = 12, 24
mulai = deret.bulan.max() - H + 1
latih = deret[deret.bulan < mulai]
uji = deret[deret.bulan >= mulai].sort_values(["wilayah", "bulan"])
rmse = lambda y, yh: float(np.sqrt(np.mean((np.asarray(y) - np.asarray(yh))**2)))

mu, sd = latih.nilai.mean(), latih.nilai.std()
X, Y = [], []
for w, g in latih.groupby("wilayah"):
    v = ((g.sort_values("bulan").nilai - mu) / sd).values
    for i in range(len(v) - L):
        X.append(v[i:i+L]); Y.append(v[i+L])
X = torch.tensor(np.array(X), dtype=torch.float32).unsqueeze(2)
Y = torch.tensor(np.array(Y), dtype=torch.float32).unsqueeze(1)
print("bentuk masukan:", tuple(X.shape), " sasaran:", tuple(Y.shape))

class Jaringan(nn.Module):
    def __init__(self, hidden=32):
        super().__init__()
        self.lstm = nn.LSTM(1, hidden, batch_first=True)
        self.fc = nn.Linear(hidden, 1)
    def forward(self, x):
        o, _ = self.lstm(x)
        return self.fc(o[:, -1, :]) # langkah waktu terakhir

model = Jaringan(); opt = torch.optim.Adam(model.parameters(), lr=0.01)
rugi_f = nn.MSELoss(); t0 = time.time()
for epoch in range(200):
    opt.zero_grad(); rugi = rugi_f(model(X), Y); rugi.backward(); opt.step()
print("waktu latih:", round(time.time() - t0, 1), "detik")
```

```

ramalan = []
for w, g in latih.groupby("wilayah"):
    v = ((g.sort_values("bulan").nilai - mu) / sd).values
    jendela = list(v[-L:])
    for h in range(H):
        with torch.no_grad():
            p = float(model(torch.tensor(np.array(jendela),
                                                    dtype=torch.float32).view(1, L, 1)))
            ramalan.append(p * sd + mu); jendela = jendela[1:] + [p]
print("RMSE LSTM (Python):", round(rmse(uji.nilai.values, ramalan), 4))

```

Interpretasi Output

Python memberi RMSE **3,388**; R memberi **3,384**. Selisihnya di angka desimal ketiga — praktis sama, dan itu memang seharusnya: keduanya membungkus pustaka C++ LibTorch yang sama, memakai benih acak yang sama, dan membaca berkas yang sama.

Kesamaan ini lebih kuat daripada yang dicapai Modul 13 dan 15, yang selisihnya lebih besar karena pembagian latih-uji di sana dibangkitkan acak oleh masing-masing bahasa. Di sini pembagiannya **menurut waktu** — sepenuhnya ditentukan data, bukan bilangan acak — sehingga kedua bahasa mengerjakan persoalan yang persis sama.

Perbedaan penamaan: R memakai `nn_lstm(input_size=, hidden_size=)` dan pemilihan langkah terakhir lewat `o[, dim(o)[2], ]`; Python memakai `nn.LSTM(1, hidden)` dan `o[:, -1, :]`. R mengembalikan daftar sehingga keluarannya diambil dengan `[[1]]`, sedangkan Python mengembalikan tuple yang diurai dengan `o, _ =`.

Kesalahan Umum.

1. Membagi data deret waktu secara **acak**; pembagian harus menurut waktu.
2. Membakukan memakai rata-rata dan simpangan baku **seluruh** data, termasuk bagian uji.
3. Melaporkan ketepatan tanpa **acuan**; naif musiman sering sulit dikalahkan.
4. Memakai jendela lebih pendek daripada satu siklus musiman.
5. Lupa memilih keluaran langkah **terakhir** dari lapisan LSTM.
6. Melaporkan satu angka RMSE bagi ramalan banyak langkah tanpa menguraikannya per cakrawala.
7. Menghentikan pelatihan berdasarkan kerugian **latih**, bukan kerugian validasi.
8. Menyimpulkan hubungan sebab-akibat dari bobot jaringan; LSTM adalah model ramalan, bukan model penjelasan.
9. Memakai LSTM pada deret pendek — di bawah beberapa ratus amatan, acuan sederhana hampir selalu lebih baik dan jauh lebih murah.

16.7 Ketika LSTM Sepadan dengan Biayanya

Pada `deret_bulanan.csv`, LSTM **tidak** mengungguli acuan naif musiman secara meyakinkan. Kesimpulan itu jujur dan penting — tetapi ia menyisakan pertanyaan: kalau begitu, kapan jaringan berulang sepadan dengan biayanya?

Jawabannya terletak pada **bentuk dinamikanya**. Acuan naif musiman unggul ketika deret bersifat tren ditambah musiman ditambah derau, sebab pola seperti itu memang berulang persis tiap dua belas bulan. Ia tidak dapat berbuat apa-apa ketika nilai berikutnya bergantung pada **beberapa nilai sebelumnya sekaligus**, apalagi bila kebergantungan itu **berpindah menurut keadaan**.

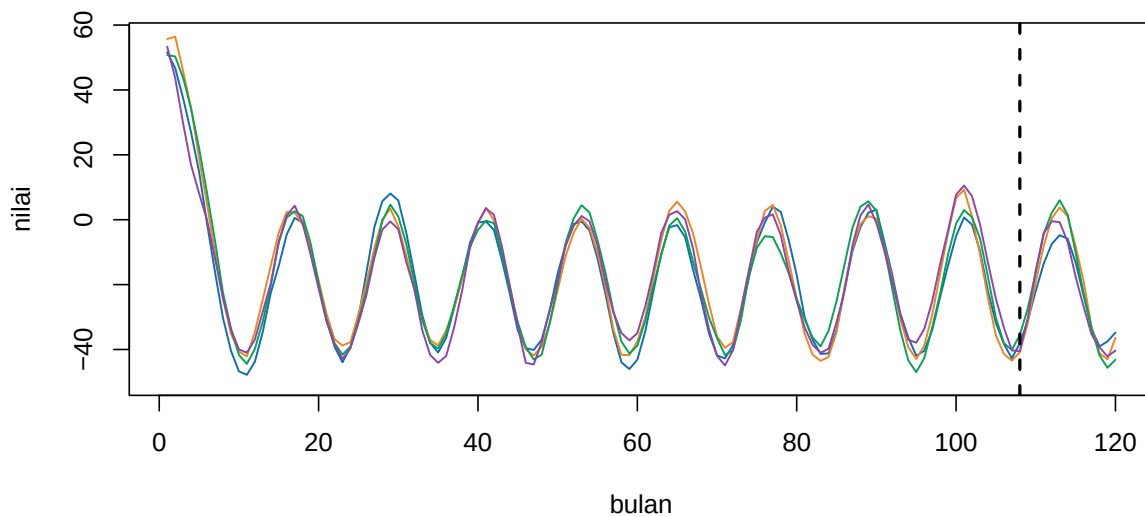
Berkas `deret_rezim.csv` memuat dinamika semacam itu: nilai tiap bulan bergantung pada dua bulan sebelumnya, dan koefisiennya berpindah rezim menurut tingkat bulan sebelumnya — meredam di atas ambang, menguat di bawahnya. Musiman tahunannya tetap ada pada kedua rezim.

Contoh 16.8 | Deret berdinamika rezim dan ketiga acuannya

Kode 16.9 • R menelaah deret dan menghitung dua acuan

```
rz <- read.csv("data/deret_rezim.csv", stringsAsFactors = FALSE)
H <- 12
batas_r <- max(rz$bulan) - H
latih_r <- subset(rz, bulan <= batas_r); uji_r <- subset(rz, bulan > batas_r)
rmse <- function(y, yh) sqrt(mean((y - yh)^2))

par(family = "sans", mar = c(4, 4, 1, 1))
warna <- c("#0B6FB4", "#F58220", "#00A651", "#8E44AD")
plot(NA, xlim = c(1, max(rz$bulan)), ylim = range(rz$nilai),
     xlab = "bulan", ylab = "nilai")
for (i in 1:4) { w <- sprintf("W%02d", i); s <- subset(rz, wilayah == w)
  lines(s$bulan, s$nilai, col = warna[i], lwd = 1.2) }
abline(v = batas_r, lty = 2, lwd = 2)
```



Gambar 16.4: Empat wilayah pada deret berdinamika rezim; garis putus-putus menandai batas latih-uji

```
# acuan 1: naif musiman -- nilai dua belas bulan sebelumnya
geser <- transform(latih_r, bulan = bulan + 12)[, c("wilayah", "bulan", "nilai")]
names(geser)[3] <- "ramalan"
naif_m <- merge(uji_r, geser, by = c("wilayah", "bulan"))

# acuan 2: naif terakhir -- nilai terakhir yang teramati, diulang
akhir <- aggregate(nilai ~ wilayah, latih_r, function(v) tail(v, 1))
names(akhir)[2] <- "ramalan"
naif_t <- merge(uji_r, akhir, by = "wilayah")

data.frame(
  acuan = c("naif musiman", "naif terakhir", "simpangan baku data uji"),
  RMSE = round(c(rmse(naif_m$nilai, naif_m$ramalan),
                  rmse(naif_t$nilai, naif_t$ramalan), sd(uji_r$nilai)), 3))
```

	acuan	RMSE
1	naif musiman	5.06
2	naif terakhir	23.27
3	simpangan baku data uji	15.95

Interpretasi Output

Grafiknya memperlihatkan perilaku yang berbeda dari deret pertama: naik-turunnya tidak seragam, dan tinggi puncaknya berpindah-pindah karena rezimnya berganti menurut tingkat sebelumnya.

Naif musiman memberi RMSE 5,06 — masih jauh lebih baik daripada simpangan baku data uji (15,9), sebab musiman tahunannya memang ada. Ia tetap acuan yang layak dan tidak boleh dilewatkan.

Naif terakhir memberi 23,3 — lebih buruk daripada sekadar menebak rata-rata. Acuan ini disertakan

justru untuk memperlihatkan bahwa **tidak semua acuan sederhana pantas**; memilih acuan yang lemah lalu mengalahkannya bukan bukti apa pun.

Pertanyaannya kini: dapatkah LSTM mengungguli 5,06?

Contoh 16.9 | LSTM pada deret berdinamika rezim

Kode 16.10 • R pelatihan dan ramalan rekursif dua belas bulan

```
suppressPackageStartupMessages(library(torch))

mu_r <- mean(latih_r$nilai); sd_r <- sd(latih_r$nilai)
bakukan_r <- function(x) (x - mu_r) / sd_r
pulihan_r <- function(z) z * sd_r + mu_r
L <- 12

jendela_r <- function(df) { X <- list(); Y <- c()
  for (w in unique(df$wilayah)) { v <- bakukan_r(df$nilai[df$wilayah == w])
    for (i in seq_len(length(v) - L)) { X[[length(X) + 1]] <- v[i:(i + L - 1)]
      Y <- c(Y, v[i + L]) } }
  list(x = torch_tensor(array(unlist(X), dim = c(L, length(X))))$t()$unsqueeze(3),
    y = torch_tensor(matrix(Y, ncol = 1))) }

dl_r <- jendela_r(latih_r)
torch_manual_seed(2026)
jaring <- nn_module(
  initialize = function() { self$lstm <- nn_lstm(1, 32, batch_first = TRUE)
    self$out <- nn_linear(32, 1) },
  forward = function(x) { h <- self$lstm(x)[[1]]; self$out(h[, dim(h)[2], ])}()
opt <- optim_adam(jaring$parameters, lr = 0.01)
for (e in 1:300) { opt$zero_grad()
  rugi <- nnf_mse_loss(jaring(dl_r$x), dl_r$y); rugi$backward(); opt$step() }

ramalan <- data.frame()
for (w in unique(rz$wilayah)) {
  v <- bakukan_r(latih_r$nilai[latih_r$wilayah == w]); riwayat <- tail(v, L)
  for (h in seq_len(H)) {
    x <- torch_tensor(array(riwayat, dim = c(L, 1)))$t()$unsqueeze(3)
    p <- as.numeric(jaring(x))
    ramalan <- rbind(ramalan, data.frame(wilayah = w, bulan = batas_r + h,
      ramalan = pulihkan_r(p)))
    riwayat <- c(riwayat[-1], p) } }
```

```
gab <- merge(uji_r, ramalan, by = c("wilayah", "bulan"))
r_lstm <- rmse(gab$nilai, gab$ramalan)
r_naif <- rmse(naif_m$nilai, naif_m$ramalan)
data.frame(model = c("naif musiman", "LSTM"),
           RMSE = round(c(r_naif, r_lstm), 3),
           perbaikan = c("--", paste0(round(100 * (r_naif - r_lstm) / r_naif, 1), " persen")))
```

	model	RMSE	perbaikan
1	naif musiman	5.060	--
2	LSTM	4.011	20.7 persen

Interpretasi Output

LSTM memberi RMSE **4,01** berbanding 5,06 milik acuan naif musiman — perbaikan sekitar **21 persen**. Inilah keadaan yang membuat jaringan berulang sepadan dengan biayanya.

Bandingkan dengan deret pertama, dan perbedaannya menjadi jelas. Di sana dinamikanya berupa tren ditambah musiman: pola yang berulang persis tiap dua belas bulan, sehingga acuan naif — yang tidak menaksir satu parameter pun — sudah menangkap hampir seluruhnya. Di sini nilai berikutnya bergantung pada dua nilai sebelumnya **dengan koefisien yang berpindah menurut keadaan**. Acuan naif tidak memiliki cara untuk mewakili kebergantungan semacam itu; LSTM memilikinya, sebab keadaan tersembunyinya membawa informasi dari beberapa langkah ke belakang.

Kesimpulan yang perlu dibawa mahasiswa bukan “LSTM lebih baik” melainkan **syarat** kapan ia lebih baik: ketika dinamikanya memuat kebergantungan berlapis atau berpindah keadaan yang tidak dapat diwakili aturan tetap. Bila deretnya sekadar tren dan musiman, acuan sederhana sudah memadai — dan jauh lebih murah, lebih cepat, serta lebih mudah dijelaskan kepada pengguna.

Perlu ditegaskan pula bahwa perbaikan 21 persen ini diperoleh dari **satu kali** pelatihan. Sebagaimana diperlihatkan Latihan 16.2, hasil pelatihan beragam antar-jalan; untuk kesimpulan yang dapat dipercaya, ulangi dengan beberapa benih acak lalu bandingkan rata-ratanya.

16.8 Latihan Mandiri

Latihan 16.1 | Panjang jendela

- Ulangi Contoh 16.3 sampai 16.5 dengan L bernilai 6, 12, dan 36. Laporkan RMSE data uji masing-masing.
- Berapa contoh latih yang tersedia pada tiap nilai L ? Jelaskan pertukaran antara panjang jendela dan banyaknya contoh.
- Nilai L mana yang Anda pilih, dan mengapa pilihan itu tidak boleh didasarkan pada RMSE data uji?

Latihan 16.2 | Ukuran jaringan

- Bandingkan hidden bernilai 8, 32, dan 128 pada 200 epoch. Catat waktu latih dan RMSE.
- Hitung banyaknya parameter tiap jaringan. Bandingkan dengan 2.016 contoh latih.
- Apakah jaringan terbesar memberi hasil terbaik? Jelaskan temuan Anda apa adanya.

Latihan 16.3 | Menguraikan galat menurut cakrawala

- Hitung RMSE **terpisah** untuk tiap bulan ramalan (bulan ke-1 sampai ke-12 setelah data latih berakhir).
- Gambar hasilnya. Apakah galat menumpuk sebagaimana diduga?
- Bandingkan pola penumpukan itu dengan naif musiman, yang **tidak** meramal secara rekursif. Apa implikasinya bagi pemilihan metode ketika cakrawala ramalan panjang?

Latihan 16.4 | Membetulkan kode yang salah

Kode berikut memuat **empat kesalahan**; temukan, perbaiki, jalankan versi yang benar, dan jelaskan akibat tiap kesalahan.

Kode 16.11 • R kode yang harus dibetulkan

```
set.seed(2026)
i <- sample(nrow(deret), 0.9 * nrow(deret))      # (1)
lat <- deret[i, ]; uj <- deret[-i, ]
mu <- mean(deret$nilai); sdv <- sd(deret$nilai)    # (2)
model <- jaringan()
for (e in 1:200) { ... }
o <- model$lstm(x)[[1]]
ramal <- model$fc(o[, 1, ])                        # (3)
cat("RMSE:", rmse(lat$nilai, ramal_latih))         # (4)
```

Petunjuk: satu baris membagi data secara acak; satu baris membakukan memakai seluruh data; satu baris mengambil langkah waktu pertama, bukan terakhir; dan satu baris menilai model pada data latih.

16.9 Penugasan

Konteks: Menilai Kelayakan Peramalan Berbasis LSTM

Unit kerja Anda mempertimbangkan memakai LSTM untuk meramalkan indikator bulanan wilayah, menggantikan prosedur yang selama ini memakai aturan sederhana. Anda diminta menilai apakah penggantian itu **sepadan** — bukan sekadar apakah LSTM dapat dijalankan.

Tugas 16.1 | Penyiapan dan acuan

1. Bagi data menurut waktu: 96 bulan pertama untuk latih, 12 bulan berikutnya untuk **validasi**, dan 12 bulan terakhir untuk uji. Jelaskan mengapa tiga bagian diperlukan.
2. Susun **dua** acuan — naif musiman dan satu acuan pilihan Anda — lalu laporkan RMSE keduanya pada data uji.

Tugas 16.2 | Pemodelan

3. Latih LSTM dengan penghentian berdasarkan kerugian **validasi**; laporkan epoch terpilih.
4. Tala sekurang-kurangnya panjang jendela dan ukuran lapisan tersembunyi memakai data validasi; sajikan tabel penalaan.
5. Laporkan RMSE data uji beserta uraiannya per cakrawala (bulan ke-1 sampai ke-12).

Tugas 16.3 | Penilaian kelayakan

6. Bandingkan LSTM dan acuan terbaik pada tiga hal: **ketepatan, waktu jalan, dan kemudahan dijelaskan** kepada pengguna data.
7. Perkirakan berapa perbaikan ketepatan yang diperlukan agar penggantian prosedur sepadan dengan tambahan kerumitannya. Nyatakan pertimbangan Anda secara terbuka.

Tugas 16.4 | Kesimpulan dan keterbatasan

8. Susun rekomendasi (maksimal 250 kata): pakai atau tidak, dan dengan syarat apa.
9. Tulis keterbatasan yang memuat sekurang-kurangnya: penumpukan galat pada ramalan rekursif; ketergantungan hasil pada panjang deret; dan bahwa data bersifat simulasi.

16.9.1 Rubrik Penilaian

Komponen	Indikator	Bobot
Pembagian data	Tiga bagian menurut waktu, alasannya dinyatakan	15%
Acuan	Dua acuan dijalankan dan dilaporkan	20%
Pemodelan	Penghentian dari data validasi; penalaan sistematis	25%
Uraian galat	RMSE diuraikan per cakrawala dan ditafsirkan	20%
Kelayakan	Ketepatan, biaya, dan keterjelasan ditimbang bersama	20%

16.10 Ringkasan Modul

Daftar periksa penguasaan Modul 16

- ☐ Saya membagi data deret waktu menurut waktu, bukan secara acak.
- ☐ Saya membakukan memakai statistik data latih saja.
- ☐ Saya dapat menjelaskan ketiga gerbang LSTM dan peran keadaan sel.
- ☐ Saya menyusun acuan sebelum menilai model rumit.
- ☐ Saya memilih keluaran langkah terakhir dari lapisan LSTM.
- ☐ Saya memahami penumpukan galat pada ramalan rekursif.
- ☐ Saya dapat menjalankan alur yang sama di R maupun Python.

Ringkasan fungsi:

R (torch)	Python (torch)	Kegunaan
<code>torch_is_installed()</code>	—	memeriksa pustaka C++ terpasang
<code>torch_tensor()</code>	<code>torch.tensor()</code>	membentuk tensor
<code>nn_lstm(input_size=, hidden_size=)</code>	<code>nn.LSTM(input, hidden)</code>	lapisan LSTM
<code>nn_linear()</code>	<code>nn.Linear()</code>	lapisan linear penutup
<code>optim_adam()</code>	<code>torch.optim.Adam()</code>	pengoptimum
<code>nnf_mse_loss()</code>	<code>nn.MSELoss()</code>	fungsi kerugian
<code>o[, dim(o)[2],]</code>	<code>o[:, -1, :]</code>	keluaran langkah terakhir

16.10.1 Daftar Istilah

Deret waktu (*time series*) — amatan berurutan menurut waktu, yang urutannya bermakna.

Jendela berurutan (*sliding window*) — potongan deret sepanjang L yang dipakai sebagai masukan untuk meramalkan nilai berikutnya.

Cakrawala ramalan (*forecast horizon*) — berapa langkah ke depan yang diramalkan.

LSTM — jaringan berulang dengan keadaan sel dan tiga gerbang, dirancang agar dapat mengingat pola berjarak panjang.

Keadaan sel (*cell state*) — ingatan yang diperbarui secara penjumlahan sehingga gradiennya tidak cepat melenyap.

Ramalan rekursif — meramalkan banyak langkah dengan memasukkan kembali hasil ramalan sebagai masukan berikutnya.

Naif musiman (*seasonal naive*) — acuan yang meramalkan nilai sama dengan nilai satu periode musiman sebelumnya.

16.11 Referensi

1. Hochreiter, S. & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
2. Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). Learning to forget: Continual prediction with LSTM. *Neural Computation*, 12(10), 2451–2471.
3. Hyndman, R. J. & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*, 3rd ed. OTexts. (Bab 5: metode acuan)
4. Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2018). Statistical and machine learning forecasting methods: Concerns and ways forward. *PLOS ONE*, 13(3), e0194889.
5. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. (Bab 10)
6. Falbel, D. & Luraschi, J. (2023). *torch for R*. <https://torch.mlverse.org>